序

本序主要分为三个部分：1000 题的内容构成、使用方法、写在后面的话。

1000 题背景

本书为计算机考研 408 专业课的练习册，共 1576 道选择题，其中数据结构 411 题、计算机组成原理 373 题、操作系统 417 题、计网 375 题。

relax1000 题伊始于 2025 年考研。在第一版中，本着后期缺乏质量高的题目做，我们团队耗时几个月搜集筛选题目，题目主要来自各机构的教辅书和习题册、袁春风老师教材、湖科大教书匠（高军老师）的每日一题和模拟卷、24 考研中质量比较好的模拟题，以及部分原创题目。

自 2026 考研开始，我们下定决心重做 1000 题，做一本属于 408 考生的 1000 题。出题的老师是各院校 140+、130+的高分学员，也有具备多年辅导经验的学长，我们清楚学生的痛点，命题的风格。本着覆盖真题考点、拓展真题考点、预测未来考点的方针，对第一版进行重做。第二版除了保留一些质量较高的题目。我们所有新题目都是自己原创，都是反复研究真题、紧扣教材细节出的一些比较严谨，又适合大家学习知识的风格的题目。在 2026 考研中，有小几千余名考生做过我们的题目，很多考生在出成绩后给了我们肯定和高度赞赏，也得知很多真题的创新考法在我们 1000 题里均有高度相似的题目，这也再次印证我们的出题方向是没错的，也让我们团队几个月的付出没有白费。

其次，去年的 1000 题是没有写序的，今年把这部分补上。第一个方面，让大家使用 1000 题有个大概了解，以及正确的使用方法。第二个方面，去年有几个考生可能不太了解我们，然后做了 1000 题，给了比较差的评价，并且说我们抄袭湖科大，我们第一年的序正如前文描述了出处，甚至一些解析我都保留出处的原样，我是非常尊重各位老师的付出的，也绝不是那种蔑视劳动成果，白嫖劳动成果的人。去年也和高军老师建立过联系，后续也对其所有题目进行了替换，今年也对一些机构的模拟题进行了替换，但是经过这么多年，我们也很难分辨是谁家的，希望各位老师多多包涵，如有侵权，请告知我具体题目，我一定全力配合进行删除，我们是站在巨人的肩膀上的，在这里也感谢各位前辈。最后我想说的是，1000 题从一开始到现在都是开源 PDF 的。去年开始是因为大家有需要纸质的，我们才出售，大家要知道，排版、做封面、核对、勘误这些事情，一旦出售，就必须对消费者负责，这种赚的钱其实相比我们付出的精力，是微不足道的，也希望大家轻点喷，理解一下。也希望通过 1000 题和大家认识，一起探讨 408，一起优化好这本书，成为计算机考研人手一本的能真正学到东西的高质量题目集。

使用方法

相信大部分人拿到 1000 题的时候都已经进入强化或结束强化了，这时候大家就会疑惑：又有一些知识点不同程度的遗忘，我是应该重新看一遍辅导书，还是去钻研更深的教材，亦或是开历年模拟卷提升做题能力查漏补缺？我觉得都是可以的，我的分析如下：

1. 对于基础比较薄弱的同学：还在基础阶段或者强化阶段，做题常常没有思路

如果你是这种情况，我建议你每天可以稍微多花点时间在 408 学习上，早点结束强化阶段。那么，结束强化阶段后，你可以考虑花一段时间把每个章节的课后选择错题复习一遍，特别是真题的选择和大题，不要放过任何一个知识细节，想想这个题涉及哪些知识点？这个知识点背后的原理（更深层次，需要查阅资料）？这个知识点所属的板块（建立知识框架）？这个知识点还有什么出题角度（很难自己联系，需要多做题）？这类题目是否有好的解题方法或思路？（总结并记笔记）。如果你当前进度比较慢，那你更应该抓住以上几个核心，到真题中去串联知识点、形成自己的知识框架，提高解题能力。

2. 对于已经结束强化的人来说：所有知识点都了解，但是碰到有深度的题，常常犯错

这部分同学应当是刷过真题并对真题常规题型都比较熟悉，我的建议是直接使用这本 1000 题进行练习。首先，有些书里面的题目有的是比较好的，值得深究，但是由于答案编的晦涩难懂，失去了其原本的价值，部分这种题目我都亲自撰写答案，其中就有一些题目，我把我笔记中总结的做题方法或者知识点总结放在了解析。我觉得大家作为考生，更应该去做这个事情，对于一些自己不会的题，首先把这个题的知识点弄懂，这是第一层；然后这个知识点你会串联起很多其他知识，比如你看到驱动程序的题目，你是否会联想到设备独立性软件，中断程序？我相信任何一个人想变强都会有这种思维，那么你此刻不应该仅仅记个正确答案，然后直接跳过这道题。正确的做法是去翻辅导书、亦或是教材、查 csdn 相关博客（这里推荐一下月暗笔记，可以省去很多查教材的时间），去把这一块的知识点都串起来，要是有一个例子能帮助你贴合知识点记忆，那就更棒了！那么恭喜你，你学会了 408 的复习之道！这是成为高手的第二层。（其次，我也在我的经验贴中多次强调的做笔记，你应该把这些收获自己动手记录下来以便后期反复享用和深层次理解记忆）。最后一层便是把你学会的点表达出来，你可以尝试在考研群去答疑，帮助别人解惑，当你能把一个问题的分析从头到尾在别人的追问下表达出来，那么可以肯定，你已经是这个知识点的主人了，这个环节你会很仔细地考量条件是否严谨？和别人的交流对峙中是否学到新的东西，自己的理解是不是有偏差？等你再做到这个知识点的相关题时，你会迸发出各种想法，甚至可以随时随地出一个好题分享给自己的研友做！

3. 对知识点非常熟练，想冲击 135 分+

这部分同学应该做了大量的工作去攻克难点，但是考场上的冷门知识点层出不穷，每年或多或少都会占 5 分左右。那么如何解决这个问题？首先，第②点提到，对于一个错题，你要弄懂这个知识点本身，你应该去看对应的教材或者博客，那么肯定会连带出来很多知识点，比如文件操作系统调用涉及的参数、具体过程，这是一个抽象的概念，但是你可以尝试去看看源码。又比如中断的整个实现过程，你可以去看看中断机构的硬件实现体会他整个流程，当你把底层的东西弄懂，在考场上出了一个从未考过的概念辨析，你就明显能够通过冷静分析选出正确答案，而不是二选一总是选错的那个。用一句话来说就是：究其根本。其次，对于一些纯记忆性的东西，而各类教材都没有或者不容易发现，这种题目就需要大家相互分享了，一个人不可能把所有的书籍看完，但是当很多高手在讨论的时候，交叉补齐很可能就会超越任何一本书籍，所以我推荐大家到考研群里交流，我这里提供一个群供大家交流：

945751916，后续 1000 题大家有问题也可以在这里探讨，我看到了也会回复大家，希望能帮助到大家。最后要提的一点是，对于高手来说，应该对待数学一样对待 408，多加练习，408 今年风格也越来越重视计算和推演，这就导致很容易在简单的计算上出错，这种分实在不应该丢的，希望大家以我为戒！

写在最后

容我介绍一下这本书的主要贡献者，第一版中其中四位为 24 上岸北大软微的高分学长：初尧、不语、amark、relax，还有月暗学长，各位学长从暑假开始，就花了大量时间搜集题目，识别校对文字，挑选等等工作。我们本可以上岸后用大量的时间学习或者休息娱乐，也许我们都知道在考研这个道路上一路走来太不容易了，所以都很爽快的答应了我的请求并完成了各自的工作，这才有了现在的 1000 题。当然，

第二版的主要贡献者，除了我和月暗学长，还有上岸科软的 thinker 学长、豆芽学长，上岸软微的发呆学长，他们在 25 考研都取得 137、135、128 的高分。也在编写 1000 题时奉献了自己的很多精力。

这本练习册还存在很多不足，比如考点覆盖还存在少许欠缺，小部分题目质量还不够高，答案解析可能会有小错误等等，在今年我们已经尽力完善了，但是肯定还是存在问题的。希望大家以包容的心态看待~ 这个我希望大家在后续群里交流的时候指出错误、提出建议、贡献好题，大家一起来完善这本书。

不知不觉考研已到暑假白热化阶段，不知道你们内心的梦想是否还依旧坚定，功不唐捐，玉汝于成，希望大家抛弃一切杂念全力备考，再吃亿点苦，27 皆上岸！

> 2026.6 relax

目　录

第一部分　408 1000 题 ds 解析册.....1

第 1 章　绪论.....2
第 2 章　线性表.....10
第 3、4 章　栈、队列、数组、字符串.....15
第 5 章　树.....27
第 6 章　图.....40
第 7 章　查找.....51
第 8 章　排序.....65

第二部分　408 1000 题 co 解析册.....77

第 1 章　计算机系统概述.....78
第 2 章　数据的表示和运算.....85
第 3 章　存储器层次结构.....102
第 4 章　指令系统.....118
第 5 章　中央处理器（CPU）.....122
第 6 章　总线和输入/输出系统.....142

第三部分　408 1000 题 os 解析册.....151

第 1 章　计算机系统概述.....152
第 2 章　进程管理.....166
第 3 章　内存管理.....203
第 4 章　文件管理.....224
第 5 章　输入/输出(I/0)管理.....240

第四部分　408 1000 题 cn 解析册.....255

第 1 章　绪论.....256
第 2 章　物理层.....262
第 3 章　数据链路层.....267
第 4 章　网络层.....279
第 5 章　传输层.....293
第 6 章　应用层.....302

第一部分

408 1000 题 ds 解析册

第 1 章

绪论

【答案速对】

ABABC　DDDDC　CBBDB　CBDAB

BAADA　B

1．A。数据的逻辑结构是从面向实际问题的角度出发的，只采用抽象表达方式，独立于存储结构，数据的存储方式有多种不同的选择：而数据的存储结构是逻辑结构在计算机上的映射，它不能独立于逻辑结构而存在。数据结构包括三个要素，缺一不可。

2．B。【解析】用排除法。由于元素间有次序关系，故排除选项 D 集合，而元素可能存在多个前驱或后继结点，故排除选项 C。该数据结构虽是层次关系，但可能不存在树根，故选项 A 的树型结构不合要求。本题的答案是选项 B，图形结构。

3．A。【解析】I 错误，数据元素是数据结构的基本单位，数据项是构成数据结构的最小单位，不可再分。比如 int 类型数据元素，一个数据元素是一个数据项；而结构体包含若干数据项，此时结构体是数据元素，多个成员是数据项。

II 正确，抽象数据结构（ADT）描述了数据的逻辑结构和抽象运算，通常用（数据对象，数据关系，基本操作集）这样的三元组来表示，从而构成一个完整的数据结构定义。

III、IV 正确，循环队列和线索二叉树都属于物理结构。

注意：数据项是数据结构中讨论的最小单位，但是有的数据项是结构体是可以再分的，这种称为组合项。(这里不用过分纠结)

对三者关系的举例：

运动员（数据元素）

姓名	俱乐部名称	出生日期			参加日期	职务	业绩
		年	月	日			

组合项

假设一个运动员包含 6 个数据项:姓名、俱乐部名称、出生日期、参加日期、职务、业绩。

其中，出生日期这个数据项又分为 3 个数据项:年、月、日，那么出生日期这个数据项称为组合项。

4．B。【解析】同一逻辑结构中，数据元素的数据项个数和每个数据项的类型都要一致。

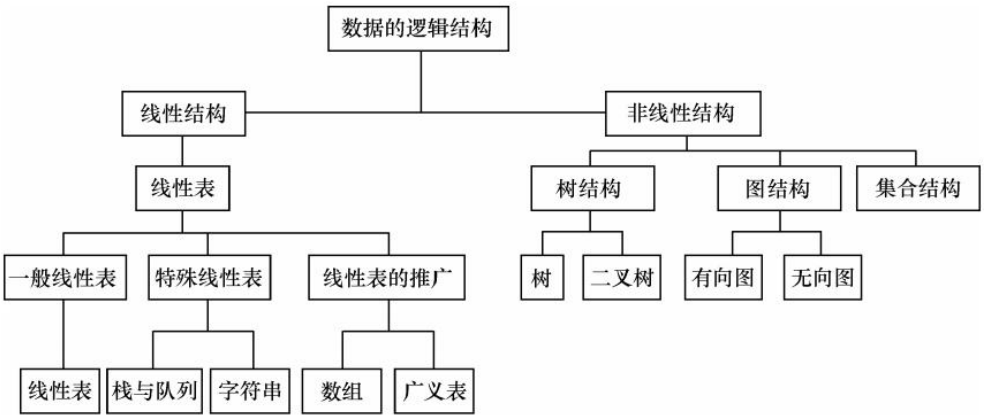
5．C。【解析】问题规模是 n。

$T(n)=O(n^2)$表示$T(n)\le c\cdot n^2$，对所有足够大的 n 成立。即执行时间的上界是n^2，系数和低阶部分可以忽略。

补充：

- $O(f(n))$：上界，即$T(n)\le cf(n)$，最多这么慢
- $\Omega(f(n))$：下界，即$T(n)\ge cf(n)$，至少这么慢
- $\Theta(f(n))$：确界，即$c_1f(n)\le T(n)\le c_2f(n)$，上下界都为 f(n)

6．D。【解析】逻辑结构分为以下几类：



7．D。【解析】四大基本物理结构有：顺序存储结构、链式存储结构、索引存储结构、散列存储结构。有序表是逻辑结构中的线性结构。

8．D。【解析】数据类型的定义：一个值的集合和定义在这个值集上的一组操作的总称。各字母含义：D 是数据对象，表示相同性质的数据元素的集合；R 是数据之间存在的关系；P 是数据基本操作。

抽象数据类型(Abstract Data Type，ADT)一般指由用户定义的、表示应用问题的数学模型，以及定义在这个模型上的一组操作的总称，具体包括 3 个部分：数据对象、数据对象上关系的集合以及对数据对象的基本操作的集合。

ADT 是概念上描述问题，DT 是在软件实现上描述问题。

9．D。【解析】因为 x，y 是固定的常数，所以时间复杂度是常数级别的。

10．C。【解析】外面两个 for 循环，执行循环次数分别是 n, n；每次循环执行一次 while 循环，一次 while 循环的时间开销$T(n)=log_5n=O(log_2n)$，所以整个时间复杂度$T(n)=n*n*log_n=O(n^2log_2n)$。

11．C。【解析】对于 sum

第一次：sum = 0 + 1 = 1

第二次：sum = 1 + 2 = 3

第三次：sum = 3 + 3 = 6

第四次：sum = 6 + 4 = 10

……

直到 sum > n 时停止。这段代码本质上是构造前 k 个自然数的和：$1+2+3+\cdots+k=\frac{k(k+1)}{2}$。

设执行完循环后 sum>nsum > nsum>n，即：$\frac{k(k+1)}{2}>n$

我们粗略估算数量级关系：$\frac{k^2}{2} > n \Rightarrow k^2 > 2n \Rightarrow k > \sqrt{2n}$，因此循环次数$k \in O(\sqrt{n})$，答案选 C。

12．B 【解析】

外层循环：i 从 1 到 n-2（包含），所以执行次数为 n−2 次（近似为 O(n)）。

内层循环：对每个 i，j 从 n 递减到 i，即执行 n - i + 1 次。

我们要求的是语句 S 被执行的总次数：$T(n) = \sum_{i=1}^{n-2} \quad (n+1-i) = \sum_{k=3}^{n} \quad k = \frac{(n+3)(n-2)}{2}$，答案选 B。

13．B。【解析】外层循环：i 从 1 到 n，共执行 n 次；

内层循环：对每个 i，j 从 i 到 n，共执行 n−i+1 次（当 i≤n 时）

我们要计算语句 count ++ 总共执行了多少次：

$T(n) = \sum_{i=1}^{\lfloor n/2 \rfloor}(n-i+1)$，令$m = \lfloor n/2 \rfloor$，有

$T(n) = \sum_{i=1}^{m}(n-i+1) = \frac{m(n+n-m+1)}{2} = \frac{m(2n-m+1)}{2} = O(n^2)$，答案选 B。

14．D。【解析】该算法为矩阵乘法，所有语句频度之和，是矩阵阶数 n 的函数，用 f(n)表示之。换句话说，上例算法的执行时间与 f(n)成正比：$f(n) = 2n^3 + 3n^2 + 2n + 1$，即时间复杂度是$O(n^3)$。

```
for(i=1;i<=n;i++)                          // 频度为 n+1
   for(j=1;j<=n;j++)                       // 频度为 n*(n+1)
   {
      c[i][j]=0;                           // 频度为 n^2
      for(k=1;k<=n;k++)                    // 频度为 n^2 * (n+1)
         c[i][j]=c[i][j]+a[i][k]*b[k][j];  // 频度为 n^3
   }
```

15．B。【解析】

$$\sum_{i=\{1,3,5,\cdots,2p+1\}}^{(2p+1<=n)} \sum_{j=1,2,4,\cdots,2^q}^{(2^q<=n)} \sum_{k=1}^{j} 1$$

$$= \sum_{i=\{1,3,5,\cdots,2p+1\}}^{(2p+1<=n)} \sum_{j=1,2,4,\cdots,2^q}^{(2^q<=n)} j$$

$$= \sum_{i=\{1,3,5,\cdots,2p+1\}}^{(2p+1<=n)} (1+2+4+\cdots+2^q)$$

$$= \sum_{i=\{1,3,5,\cdots,2p+1\}}^{(2p+1<=n)} 2^{q+1}-1 (2^q = n)$$

$$= \sum_{i=\{1,3,5,\cdots,2p+1\}}^{(2p+1<=n)} (2n-1)$$

$$=^{p \approx n/2} (2n-1)(n/2)$$

$$= O(n^2)$$

16．C。【解析】考虑到内部循环每次执行$\frac{n}{i}$次操作。总的执行次数就是$\sum \frac{n}{k} = n\sum \frac{1}{k}$，即 O(nlogn)

调和级数$H_n=\sum_{k=1}^{n}\ \frac{1}{k}$近似 O(nlogn)证明如下：

证明下界：$\sum_{k=1}^{n}\ \frac{1}{k}\sum_{k=2}^{n}\ \frac{1}{k}+1\geq\int_{2}^{n+1}\ \frac{1}{x}dx+1=\ln(n+1)-\ln(2)+1=\ln\left(\frac{n+1}{2}\right)+1=\ln(n+1)\Rightarrow Hn\in\Omega(logn)$

证明上界：$\sum_{k=1}^{n}\ \frac{1}{k}\leq 1+\int_{1}^{n}\ \frac{1}{x}dx=1+\ln n\Rightarrow H_n\in O(\log n)$

综合以上，$\ln(n+1)\leq H_n\leq 1+\ln n\Rightarrow H_n\in\Theta(\log n)$

17．B。【解析】

$$\sum_{i=\{1,2,4,\ \cdots,2^q\}}^{1}\ \sum_{j=1}^{1}\sum_{k=1}^{1}1$$
$$=\sum_{i=\{1,2,4,\cdots,\ 2^q\}}\ \sum_{j=1}^{i}j$$
$$=\sum_{i=\{1,2,4,\cdots,2^q\}}\ \frac{i(i+1)}{2}$$
$$=\frac{1}{2}\sum_{i=\{1,2,4,\cdots,2^q\}}\ i^2+i$$
$$=\frac{1}{2}(1+4+4^2+\cdots+4^q)+\frac{1}{2}(1+2+\cdots+2^q)$$
$$=\frac{1}{2}(\frac{1-4^{q+1}}{1-4})+\frac{1}{2}\frac{1-2^{q+1}}{1-2}$$
$$=\frac{4^{q+1}-1}{6}+\frac{2^{q+1}-1}{2}\rightarrow^{2^q\sim n}O(n^2)$$

18．D。【解析】根据递推关系得 T(n)=T(n-1)+T(n-2)+O(1)，可以用递归树展开的方式，树高为 O（n），时间复杂度是递归树的结点个数，显然这是一个指数级时间复杂度。

19．A。【解析】此题也可考察盘子移动次数，难度会更高。设 n 个盘子的汉诺塔问题移动次数为 S(n)，推导如下：

①n=1，只需要将这个盘子从源塔移动到目标塔，S(1) = 1；

②分析代码：

- hanoi(n-1, from_rod, aux_rod, to_rod); //将上面的 n-1 个盘子从起始柱子通过目标柱子移动到辅助柱子这个递归的移动次数为 S(n-1)
- 将最大的盘子（第 n 个）从起始柱子移动到目标柱子由①可知，移动 1 次即可
- hanoi(n-1, aux_rod, to_rod, from_rod);//将那 n-1 个盘子从辅助柱子通过起始柱子移动到目标柱子这个递归的移动次数也为 S(n-1)

③　$S(n)=2S(n-1)+1\rightarrow S(n)+1=2(S(n-1)+1)=2^2(S(n-2)+1)=\ldots=2^{n-1}(S(1)+1)=2^n\rightarrow S(n)=2^n-1$

综上，盘子的移动次数为2^n-1，即时间复杂度为$O(2^n)$，答案选 A。

20．B。【解析】此题目的算法的意思是计算出二叉树的最大深度，通过简单地遍历一遍二叉树内的所有结点即可求出结果。此算法的时间复杂度是 O(n)。

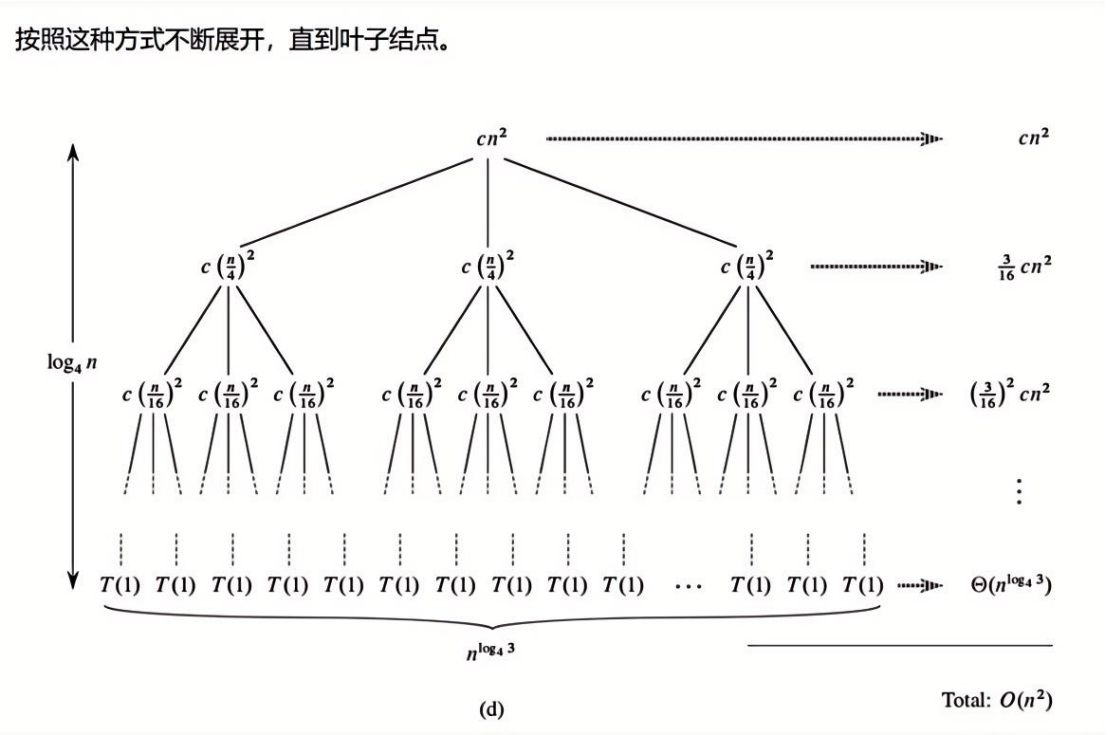
21．B。【解析】递归关系式为 T(n)=T(n−1)+O(1)，为什么这里是 O（1）不是 O（n），因为 * n 是单个乘法操作，时间消耗为常数。展开递归：T(n)=T(n−1)+O(1)=(T(n−2)+O(1))+O(1)=(T(n−3)+O(1))+

O(1)+O(1)=...=T(1)+O(n)。因为 **T(1) = O(1)**，最终得：T(n)=O(n)

22．A。【解析】

【方法一】递归树法。

展开第一层，我们画出根节点下面的三个子节点。其中，cn^2表示根节点这一层分解和合并的代价，也就是递归式中 $\Theta(n^2)$表示的部分。



这样就得到了一棵完整的递归树。每层数据规模缩小为之前的四分之一，直到叶子结点数据规模为1，因此总层数为。上图最右侧计算了每层所有结点的总代价，计算方式很简单，就是单个结点的代价乘以该层的结点数量。其中，最后一层单个结点的代价为常数，结点数量为 $3^{log_4 n} = n^{log_4 3}$，总代价我们记做。

接下来，我们只需要把所有结点的代价加起来，就可以得到递归式的解。

$$
\begin{aligned}
T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{log_4 n-1} cn^2 + \Theta\left(n^{log_4 3}\right) \\
&= \sum_{i=0}^{log_4 n-1} \left(\frac{3}{16}\right)^i cn^2 + \Theta\left(n^{log_4 3}\right) \\
&< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta\left(n^{log_4 3}\right) \\
&= \frac{1}{1-3/16} cn^2 + \Theta\left(n^{log_4 3}\right) \\
&= \frac{16}{13} cn^2 + \Theta\left(n^{log_4 3}\right) \\
&= O(n^2)
\end{aligned}
$$

【方法二】主定理（直接记结论）

递归表达式形如：$T(n) = aT(n/b) + f(n)$

- $n^{log_b a} > f(n)$，则$T(n) = \Theta\left(n^{log_b a}\right) + O\left(n^{log_b a}\right) = \Theta\left(n^{log_b a}\right)$
- $n^{log_b a} < f(n)$，且对于某个常数c<1和所有足够大的n有$af(n/b) \le cf(n)$，则可以推导出$T(n) = \Theta\left(f(n)\right)$.
- $n^{log_b a} = f(n)$，则$T(n) = \Theta\left(n^{log_b a}\right) + \Theta\left(n^{log_b a} log_2 n\right) = \Theta\left(n^{log_b a} log_2 n\right)$

例题①：$T(n) = 9T(n/3) + n$

> 对于这个递归式，我们有$a = 9$、$b = 3$、$f(n) = n$，此时$n^{log_b a} = n^{log_3 9} = n^2$。由于$f(n) = n$ 渐进小于$\Theta(n^2)$，所以可以应用于主定理的第一种情况，从而得到解$T(n) = \Theta(n^2)$.

例题②：$T(n) = T(2n/3) + 1$

> 对于该式，有$a = 1$、$b = 3/2$、$f(n) = 1$，此时$n^{log_b a} = n^{log_{3/2} 1} = n^0 = 1$，恰好与$f(n)$相等，所以可以应用于主定理的第二种情况，从而得到解$T(n) = \Theta(log_2 n)$。

例题③：$T(n) = 3T(n/4) + nlog_2 n$

> 对于该式，有$a = 3$、$b = 4$、$f(n) = nlgn$，此时$n^{log_b a} = n^{log_4 3} = n^{0.793}$。显然$f(n) = nlgn$渐进大于$n^{0.793}$，应考虑主定理的第三种情况。但需要检查是否满足附加条件：对于某个常数 c<1 和所有足够大的 n 有$af(n/b) \le cf(n)$。带入a、b和$f(n)$，得
>
> $$3\frac{n}{4}\lg\frac{n}{4} \le cnlgn\frac{3}{4}(lgn - 2) \le clgn\left(\frac{3}{4} - c\right)lgn \le \frac{3}{2}$$
>
> $c \ge \frac{3}{4}$ 时，该式恒成立，附加条件满足，可以应用主定理的第三种情况，从而得到解
>
> $$T(n) = \Theta(nlgn)$$

例题④：$T(n) = 2T(n/2) + nlgn$

> 对于该式，有$a = 2$、$b = 2$、$f(n) = nlgn$，此时$n^{log_b a} = n^{log_2 2} = n$。显然，$f(n) = nlgn$渐进大于 n，应考虑主定理的第三种情况。将其带入附加条件，得
>
> $$\begin{aligned} 2\frac{n}{2}\lg\frac{n}{2} &\le cnlgn \\ lgn - 1 &\le clgn \\ (1 - c)lgn &\le 1 \end{aligned}$$
>
> - 当$c \ge 1$ 时，该式恒成立。但**附加条件要求 c<1**，因此该式不能使用主方法的第三种情况。也就是说，并非所有形如$T(n) = aT(n/b) + f(n)$的递归式都可以应用主方法求解，**情况二和情况三之间是存在空隙的**。这种情况我们就只能采用递归树的方法求解了，同学们可以试一下，求出来的结果为$T(n) = \Theta(nlg^2 n)$.

23．A。【解析】

【方法一】代入法

$$\begin{aligned} T(n) = 2T(n/2) + O(n) &= 2（2T(n/4) + O(n/2)） + O(n) = 4T(n/4) + O(n) + O(n) \\ &= 4T(n/4) + 2O(n) = 4\left(2T(n/8) + T(n/4)\right) + 2O(n) = 8T(n/8) + 3O(n) \\ &= 2^{log_2 n} T(1) + log_2 n\, O(n) = O(nlog_2 n) \end{aligned}$$

> 适合代入法的例子：12 年真题阶乘

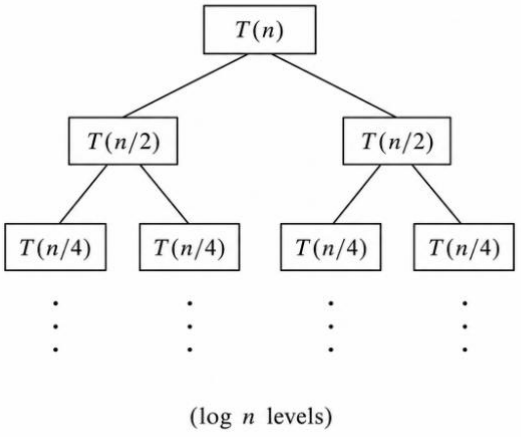
```
C
int fact(int n) {
    if (n <= 1) return 1;
    else return n * fact(n - 1);
}
```

$$T(n)=\begin{cases} O(1) & n\le 1 \\ O(1)+T(n-1) & n>1 \end{cases}$$

$$T(n)=O(1)+T(n-1)=O(1)+O(1)+T(n-1)=2O(1)+T(n-2)=\dots$$
$$=(n-1)O(1)+T(1)=(n-1)O(1)+O(1)=nO(1)=O(n)$$

【方法二】递归树法。根据递归式，其中 $2T（n/2）$是每次递归都会产生 2 个子递归，规模变为原来的 1/2，且每次参数为 n 的递归中，for 循环累计求和的开销是$\Theta(n)$。

那么我们可以画出一棵递归树：



(log n levels)

递归树的高度为$log_2 n$，每层的工作量如下：

- 第 0 层（根层）：执行 $O(n)$
- 第 1 层：执行 $2\times O(n/2)=O(n)$
- 第 2 层：执行 $4\times O(n/4)=O(n)$
- 第 3 层：执行 $8\times O(n/8)=O(n)$
- ...

每一层的工作量都是 $O(n)$，递归树有$\log_2 n$层。

【方法三】主定理（记结论）

递归表达式形如：$T(n)=aT(n/b)+f(n)$

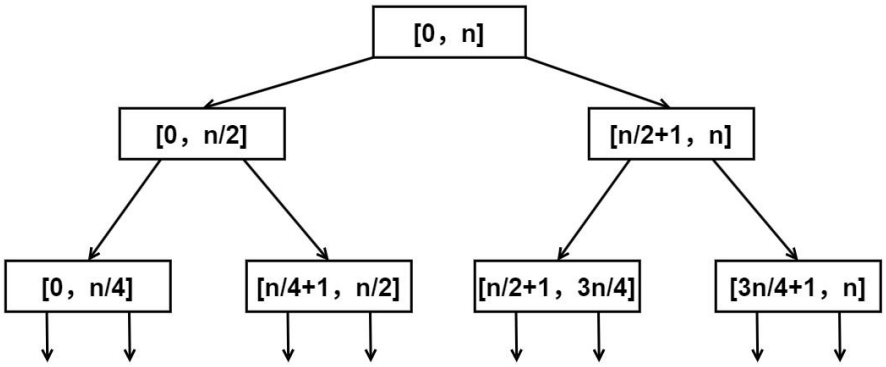
- $n^{log_b a}>f(n)$，则$T(n)=\Theta\left(n^{log_b a}\right)+O\left(n^{log_b a}\right)=\Theta\left(n^{log_b a}\right)$
- $n^{log_b a}<f(n)$，且对于某个常数 c<1 和所有足够大的 n 有$af(n/b)\le cf(n)$，则可以推导出$T(n)=\Theta\left(f(n)\right)$
- $n^{log_b a}=f(n)$，则$T(n)=\Theta\left(n^{log_b a}\right)+\Theta\left(n^{log_b a}log_2 n\right)=\Theta\left(n^{log_b a}log_2 n\right)$

回到这题：

$$T(n)=2T(n/2)+\Theta(n)a=2,b=2,f(n)=O(n),n^{log_b a}=n$$
$$=f(n),\text{满足第三种情况，答案为}O(nlog_2 n)$$

24．D。【解析】观察程序发现，语句“A[j]与 A[j+1]对换;”执行频率较高，且该语句并不影响双重 for 循环判断语句，但会影响 if 的判断语句。当 A 数组的所有元素都逆序时，语句“A[j]与 A[j+1]对换;”会一直执行，此时为最坏情况。该语句在最坏情况下的频度是$O(n^2)$，选 D。

25．A。【解析】可以写出算法的递归表达式是 S(n) = 2×S(n/2) + 1；可以画出该递归式对应的递归树，如下图所示：



在最坏情况下，递归会一直分解到 m 等于 n，每次递归将区间一分为二，因此递归深度为 O(logn)，选择 A。

26．B。【解析】算法的空间复杂度为 O(1)，表示执行该算法所需的辅助空间大小相比输入数据的规模来说是一个常量，而不表示该算法执行时不需要任何空间或辅助空间。

第 2 章

线性表

【答案速对】

ABCBC　ACCDA　D（CD）BAC　DACDB

ADBDB　DCABB　D

1．A。【解析】由数据结构的有穷性可以知道，线性表必须是有限序列。若元素个数为 0，表示线性表空。

2．B。【解析】A 错误，集合不是线性表，而是另一种逻辑结构，线性表有前驱后继的数据关系。

C 错误，不满足有穷性，线性表必须是有限序列。

D 错误，邻接表是表示非线性关系的存储结构。

注意：线性表各数据元素必须是同一类型。

3．C。【解析】A 错误，存储密度=有效存储空间/全部空间。链式存储中，需要引入指针来记录元素的前驱后继关系，所以存储密度更低。

B 错误，链式存储只支持顺序存取，顺序存储两种都支持。

C 正确，这是链式存储结构的特点。

D 错误，两者时间复杂度都是 O(n)，顺序表的时间开销在移动，链表的时间开销在遍历。

4．B。【解析】线性表由有限个相同类型的元素组成的序列，元素之间是一对一的有前驱后继的线性关系。A、C、D 都属于线性表，栈和队列是受限的线性表，有序表是元素之间按序排放的线性表。散列表不属于线性表。

5．C。【解析】I 错误，头尾分别没有前驱和后继；II 正确，单链表的存储密度比顺序表低，存储密度是指一个节点数据本身所占的存储空间和整个结点所占的存储空间之比，单链表存储密度小于 1；III 错误，经常添加或者删除用链表比较方便，修改结点值链表复杂度为 O（n）。IV 正确，只需要修改静态链表中的指针即可。

6．A。【解析】A 错误，数组一旦确定，数据元素的容量和位置关系就是固定的，因此不能进行插入和删除等操作；注意和顺序表进行区分，顺序表是线性表的顺序存储，是物理结构，是可以增删改查的。

B 正确，稀疏矩阵压缩存储后，不能直接根据元素所在数组中的位置在内存中定址，也就意味着失

去了随机存取功能；

C 正确，n 维数组的任一个元素在每一维上都有确定的位置，该维上的所有元素构成一个向量；

D 正确，数组虽然适合采用顺序存储，但在一些特殊场合，如稀疏矩阵为了节约空间，用链式存储（十字链表法）也是一种必要的选择。

7．C。**【解析】**A 错误，顺序表是顺序存储的线性表，表中所有元素的类型必须相同，且必须连续存放。一维数组中的元素可以不连续存放，数组是一种扩展的逻辑结构；一维数组就是线性表，多维数组等价于多个线性表。（注意这个数组是逻辑结构的概念，注意和编程语言中的数组区分）。

B 错误，在顺序表中，逻辑上相邻的元表物理位置上也相邻。

C 正确，顺序表可以进行按下标存取，即支持随机存取。

D 错误，每个元素的类型必须相同，这是数据结构的性质。

8．C。**【解析】**当在双链表的两个结点(分别用第一个、第二个结点表示)之间**插入**一个新结点时，需要修改四个指针域，分别是：新结点的前驱指针域，指向第一个结点； 新结点的后继指针域，指向第二个结点； 第一个结点的后继指针域，指向新结点；第二个结点的前驱指针域，指向新结点。（删除修改 2 个即可）

9．D。**【解析】**在插入和删除操作上，单链表和双链表都不用移动元素，都很方便，但双链表修改指针的操作更为复杂，A 错误。双链表中可以快速访问任何一个结点的前驱和后继结点，D 正确。

10．A。**【解析】**在链表的末尾插入和删除一个结点时，需要修改其相邻结点的指针域。而寻找尾结点及尾结点的前驱结点时，只有带头结点的双循环链表所需要的时间最少。

11．D。**【解析】**在链表中链尾插入新节点需要找到链尾指针，所以提供尾指针最合适，而删除尾结点通常需要找到尾结点的前驱结点，只有 D 选项可以在 O（1）时间复杂度完成两种操作。

12．C、D。**【解析】**因为每次插入都要遍历找到尾结点，没有尾指针的单链表头插法建立的复杂度为 n×O(1)=O(n)，尾插法建立的复杂度为。

13．B。**【解析】**B 删除尾部元素，需要找到尾部元素的前一个元素，与链表长度有关。

14．A。**【解析】**首先用指针变量 q 指向结点 A 的后继结点 B，然后将结点 B 的值复制到结点 A 中，最后删除结点 B。

15．C。**【解析】**B 选项是带头结点的单链表头插相关操作；

C 选项是头插的操作。

16．D。**【解析】**其本质是依次遍历 A、B，每次各取一个元素间隔插入 C。所以间隔插入的时间复杂度是$O(min(m,n))$，多余的只需要一次指针的修改，链接在 C 后面即可，所以时间复杂度为$O(min(m,n))$，答案选 D。

注意：$O(max(m,n)) = O(m+n)$，但是$O(min(m,n))$并不等价

17．A。**【解析】**在循环单链表中，删除首元结点后，要保持链表的循环性，因此需要找到首元结点的前驱。当链表带头结点时，其前驱就是头结点，因此不论是表头指针还是表尾指针，删除首元结点的时间都为 O(1)。当链表不带头结点时，其前驱是尾结点，因此，若有表尾指针，就可在 0(1)的时间找到尾结点；若只有表头指针，则需要遍历整个链表找到尾结点，时间为 0(n)。

18．C。【解析】要把长度为 n 的单链表链接在长度为 m 的单链表后面，那么首先需要找到长度为 m 的单链表尾结点，时间开销为 O(m)。再进行 O(1)的链接操作。所以本题选 C。

19．D。【解析】为了使 p 的后继不丢失，我们要先处理 s->next=p->next。再用 p->next=s；

注意：指针是一种变量类型，这个类型的存储的内容是它指向数据的地址，结构体中，要用“->”表示他指向结点的内部数据项。如果是结构体本身，那么用 node.next 的 “.” 来表示内部的数据项。

20．B。【解析】删除首元结点，只需执行如下操作：p=head->next; head->next=head->next->next; free(p)；所以只需要修改一个指针：head 的 next 指针。

注意：带头结点的循环链表尾结点的 next 指针是指向头结点，而非首元结点，要注意这个细节。

21．A。【解析】对于 A，在单链表和顺序表上实现的时间复杂度都为 O(n)，但后者要移动很多元素，所以在单链表上实现效率更高些。

22．D。【解析】A 错误。分配给单链表的内存单元地址可以是不连续的。B 错误。在顺序表和链表上顺序访问所有节点，时间复杂度均为 O(n)。C 错误。删除最后一个元素所需时间是 O(1)。但从长度为 n 的顺序表中删除任一个元素，平均时间复杂度是 O(n)。

23．B。【解析】I 错误，首元结点（链表中第一个存储有效数据的结点）没有前驱，其删除不需要找到前驱。

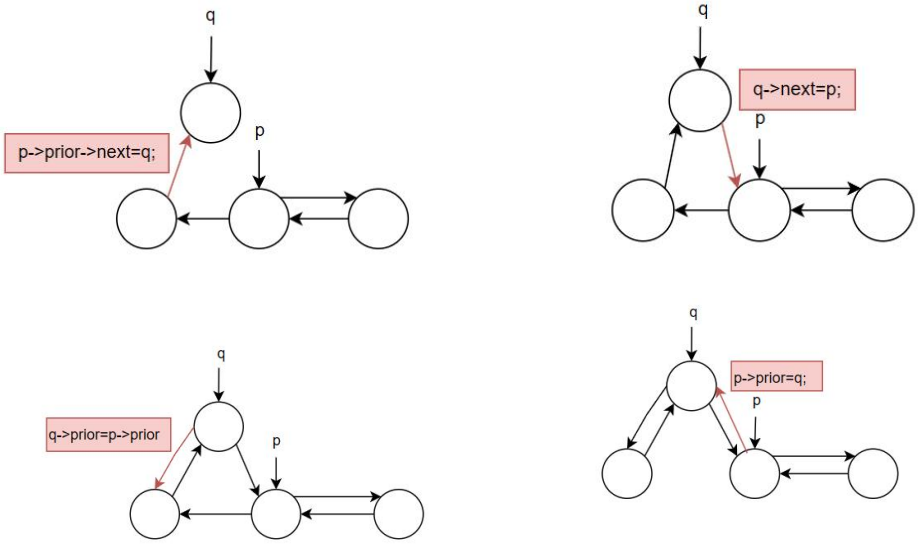
II 正确，删除修改 2 个指针，在 p、q 两非空结点后插入 s 要修改 4 个指针。

III 错误，删除尾结点需要找到尾结点的前驱，因此要 O（n）时间遍历到其前驱结点。

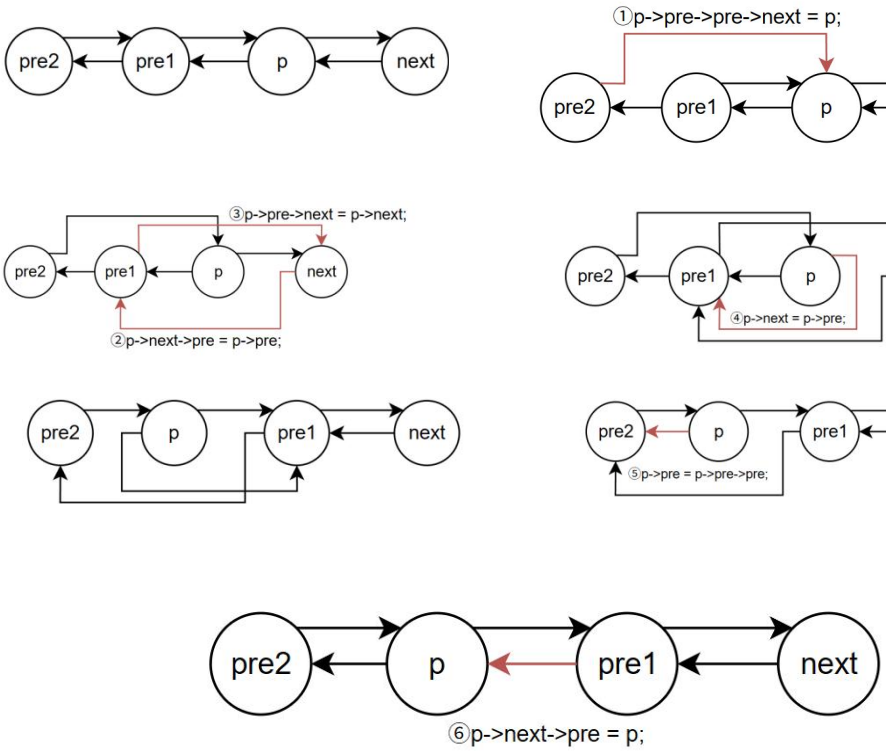
IV 正确，要注意的细节是带头结点的循环单链表中，尾结点的 next 始终指向头结点，无需修改尾结点的指针

V 错误，静态链表是用数组表示各结点的前后关系，非动态申请空间。

24．D。【解析】具体过程如下图所示：



25．B。【解析】假设初始状态如下所示，p 指向的结点为 p 结点（p 指针忽略），p 的前驱为 pre1，p 的前驱的前驱为 pre2，p 的后继节点为 next。

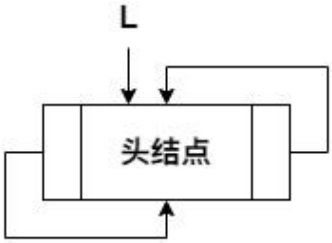


至此六条指令执行完，可知完成的操作是 p 和和其前驱进行调换。答案选 B。

26．D。【解析】选项 A、B、C 的时间复杂度均为是 O(n)。选项 C 中，需要通过遍历链表才可以找到末尾元素，然后才能正确地设置末尾元素的 next 结点。D 的时间复杂度是 O（1），可以通过首元素的 prior 指针找到链表的末尾元素，然后设置相应的指针指向。

27．C。【解析】删除最后一个元素，要找到最后一个元素的前驱结点，所以单链表时无法实现 O(1)的时间复杂度，排除 A、D。非循环双链表，无法通过尾结点找到头结点或者首元结点（第一个携带有效信息的结点），所以 B 不能 O(1)实现插入和删除第一个元素，排除。因此正确答案是 C。

28．A。【解析】双向循环链表 L 为空表时的结构如下图所示，L 的 next 和 prior 都指向本身，只有 A 选项满足条件。



29．B。【解析】静态链表用数组来表示线性表的链式存储结构，需要事先申请一片较大的连续的空间。且插入和删除只需要改编表项的指针编号，无需移动元素，方便插入和删除。

30．B。【解析】静态链表的存储空间虽然是顺序分配的，但元素的存储不是顺序的，查找时仍然需要按链依次进行，而插入、删除都不需要移动元素。静态链表的存储空间是一次性申请的，能容纳的最大元素个数在定义时就已确定。由于并非每个空间都存储了元素，因此会造成存储空间的浪费。

31．D。【解析】原链表为 c->a->e->b->d->NULL，现在删除 a，那么变为 c->e->b->d->NULL，-1 表示指向 NULL，-2 表示空表项（如果空表项也是-1 的话，分配新的结点无法找到空闲的表项，原理类似 OS 里的 FAT 表），所以应该把 a 所在的表项 0 中的地址改为-2，把 c 所在的表项 2 中的地址改为 4（指向 e）。如果最后一个结束的结点和空结点都用-1 表示，如果要插入一个元素，就无法分辨哪个是空闲的结点哪个是已分配的尾部结束结点，所以一般用-2 表示空闲结点，D 正确。

第 3、4 章

栈、队列、数组、字符串

【答案速对】

BDBBB　CBBCA　BBBAB　ADBDC

CDDCA　DCDBA　CADAA　CCDCA

BCCAB　ACACB　CCCCD　BCBDC

CDCB

1．B。【解析】对于链队，进队操作是在队尾插入节点，出队操作是删除队首节点。对于带队首指针 f 和队尾指针 r 的非循环单链表，这两种操作的时间复杂度均为 0(1)，所以本题答案为 B。

2．D。【解析】根据题意，push 后，栈顶指针+1，pop 后指针-1，而 push 操作 6 次，pop 操作 2 次，相差 4 次，所以操作完成后，指针净增+4，由于是 16 进制，结果指针为 100AH，答案选 D。

3．B。【解析】A 选项得到的出栈序列是“2341”，C 选项得到的是“3214”，D 选项得到的是“1342”。

4．B。【解析】sp 指向栈顶元素，栈往高地址增长，所以要入栈需使 sp 先加 1，指向空的位置在进行插入。符合逻辑的是 B。如果 sp 指向栈顶元素的下一个空闲位置，那么答案就是 A。此外，如果给出空栈的栈顶指针是-1，等价于栈顶指针指向栈顶元素。初始为 0，等价指向栈顶元素的下一个空闲位置。

5．B。【解析】第 k 个出栈元素是“1”，那么前面 k 个元素一定是 1~k，也就是 1~k-1 个位置包括 2~k 一共 k-1 个元素，否则元素超过 k 个，栈底的“1”在位置 k 一定出不来。剩余的 k+1~n 一共 n-k 个元素进行入栈和出栈，k+1 个位置的不同元素可能个数也就变成了，k+1~n 依次入栈，第一个出栈的元素可能个数，易知任何一个都可能在第一个出栈，都有其对应的合法操作，答案选 B。

6．C。【解析】容易推得当$P_i = 1$ 时，入栈顺序可以是 i, i-1, ..., 1；然后，依次从栈顶出栈。这样可以推得 1 能够在任意位置。

反过来思考这道题，当$P_i = 1$ 时，$P_1, P_2, \ldots, P_{i-1}$的所有可能，一定是由 2，3，...，i-1，i 共计 i-1 个元素构成的不同的出栈序列状态，i 的位置右边刚好是 i+1, i+2, ..., n 共 n-i-2 个元素构成的不同出栈序列个数，那我们设 n 个元素的出栈序列个数为 H(n)。

当 1 在 i=1 的位置上时，左边 0 个元素，右边 2~n 共 n-1 个元素，出栈序列有 H(0)*H(n-1)

当 1 在 i=2 的位置上时，左边一定是 2，共 1 个元素，右边 3~n 共 n-2 个元素，出栈序列有 H(1)*H(n-2)

…

当 1 在 i=n 的位置上时，左边 2~n 共 n-1 个元素，右边 0 个元素，出栈序列有 H(n-1)*H(0)

所以 n 个元素的不同出栈序列个数总数$H(n)=H(0)*H(n-1)+H(1)*H(n-2)+\ldots+H(n-1)*H(0)=\frac{C_{2n}^{n}}{n+1}$

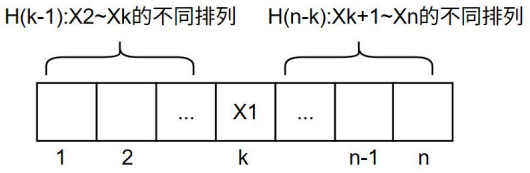
这就是我们的卡特兰数，上述就是他的递推过程。

7．B。【解析】I 正确，P1=1，P2=3，此时栈里还有元素 2，P3 的取值可以是 2,4,5,...,n 共 n-2 种。II 正确，P1=2，P2=3，此时栈里还有元素 1，P3 的取值可以是 1,4,5,...,n 共 n-2 种。III 错误，P1=4，P2=3，此时栈里还有元素 1，2；P3 的取值可以是 2,5,...,n 共 n-3 种。

8．B。**【解析】**此题答案是 n=4 的卡特兰数

证明方法一：利用卡特兰数的递推式：$H(n)=H(0)H(n-1)+H(1)H(n-2)+\ldots+H(n-1)H(0)=\frac{C_{2n}^{n}}{n+1}$

当元素 1 放在出栈序列第 k 个位置时，前面 k-1 个位置一定是 2~k 共计 k-1 个元素的出栈排列个数 H(k-1)，右边位置是 k+1~n 共计 n-k 个元素的出栈序列排列个数 H(n-k)，由此得到上述递推式。

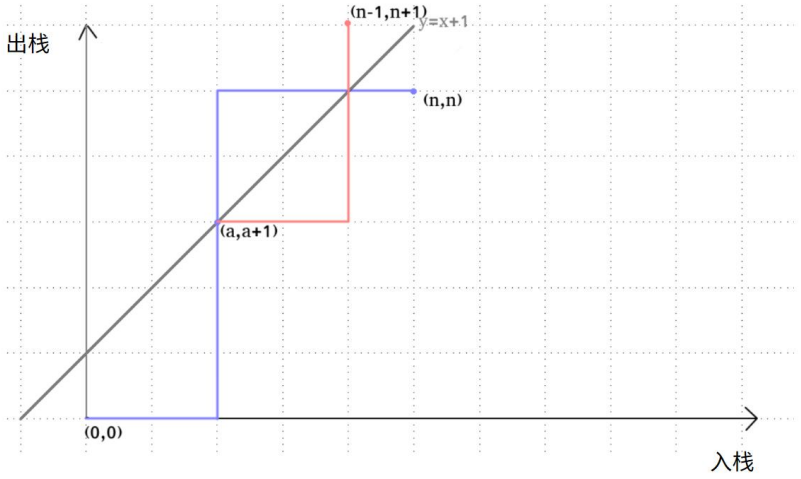


证明方法二：网格法，$\frac{C_{2n}^{n}}{n+1}=C_{2n}^{n}-C_{2n}^{n-1}$

入栈操作为横坐标，出栈操作为纵坐标，n 次入栈+n 次出栈操作即从原点出发到(n, n)的路径个数，即 2n 次操作里面选择 n 次入栈（剩下的是出栈）的可能数，即C_{2n}^{n}，但是每个时刻，出栈次数不能比入栈次数多，否则就是不合法。如下图，我们可以知道，不合法的操作路径，一定会触碰 y=x+1 到达(n, n)，而这样的路径经过 y=x+1 反射都会到达点（n-1, n+1），所以所有不合法的路径即从原点出发到点(n-1, n+1)的所有路径数量。

综合以上，合法操作路径条数=$C_{2n}^{n}-C_{2n}^{n-1}=\frac{C_{2n}^{n}}{n+1}$

（只要触碰到 y=x+1 说明当前状态出栈比入栈次数多，非法序列），



补充：出栈序列为 12...n 的入栈序列个数？

逆向思维，根据上述网格法，其实合法序列就是不同的路径的条数，每一条路径的出栈出栈次序有差异，得到的出栈序列也有差异，那么我们固定出栈序列，通过路径还原成入栈序列，肯定也是不同的。所以结果还是卡特兰数。

9．C。【解析】若栈容量为 4，则不同出栈序列数为卡特兰数$\frac{C_{2n}^{n}}{n+1}$，n=4 时经计算为 14。但题目容量限制为 3，则仅 4, 3, 2, 1 出栈序列需要栈的容量为 4，无法实现，故不同的出栈序列数量为 14-1=13。

10．A。【解析】当 Pn=1 时，进栈序列是 P1, P2, P3, …, 1，由输出序列可知，P1, P2, P3, …, Pn 依次进栈，然后依次出栈，即 P(n-1)=2, P(n-2)=3, …, P1=n，也就是说 Pi=n - i + 1。所以本题答案为 A。

11．B。【解析】可以模拟各个选项，对于 B，必须先将 1.2.3 入栈，然后先后出栈 3.2，栈中剩 1，下一个出栈元素为 5，所以需要将 4.5 先后入栈，此时栈中元素有 1, 4, 5，出栈 5，栈内元素有 1，4，下一个出栈元素为 4，但是选项中是 1，所以无法实现，故选 B。

12．B。【解析】本题要求出栈序列以 d 开头，则 a，b，c，d4 个元素需要依次进行入栈操作，然后 d 出栈，栈中元素还剩 a，b，c，且 a，b，c 三个元素的出栈序列固定为 c，b，a，所以只有 e 元素的出栈序列位置没有固定，即可以在 d 出栈后、e 进栈后直接出栈，出栈序列为 d，e，c，b，a；也可以在 d 出栈后，c 出栈，e 再进栈后直接出栈，出栈序列为 d，c，e，b，a；同理可得 e 元素可以在 b 或 a 出栈之后再进行入栈和出栈操作，所以本题以元素 d 开头的序列个数是 4。

13．B。【解析】上溢是指存储器满，还往里写；下溢是指存储器空，还往外读。为了解决上溢，可给栈分配很大的存储空间，但这样又会造成存储空间的浪费。共享栈的提出就是为了在解决上溢的基础上节省存储空间，将两个栈放在同一段更大的存储空间内，这样，当一个栈的元素增加时，可使用另一个栈的空闲空间，从而降低发生上溢的可能性。

14．A。【解析】可举例子自行模拟。读者可以思考若 top1 的初值为 0，top2 的初值为 n-1 时栈满的条件。

注意：栈顶、队头与队尾的指针的定义是不唯一的，做题时务必仔细审题和思考

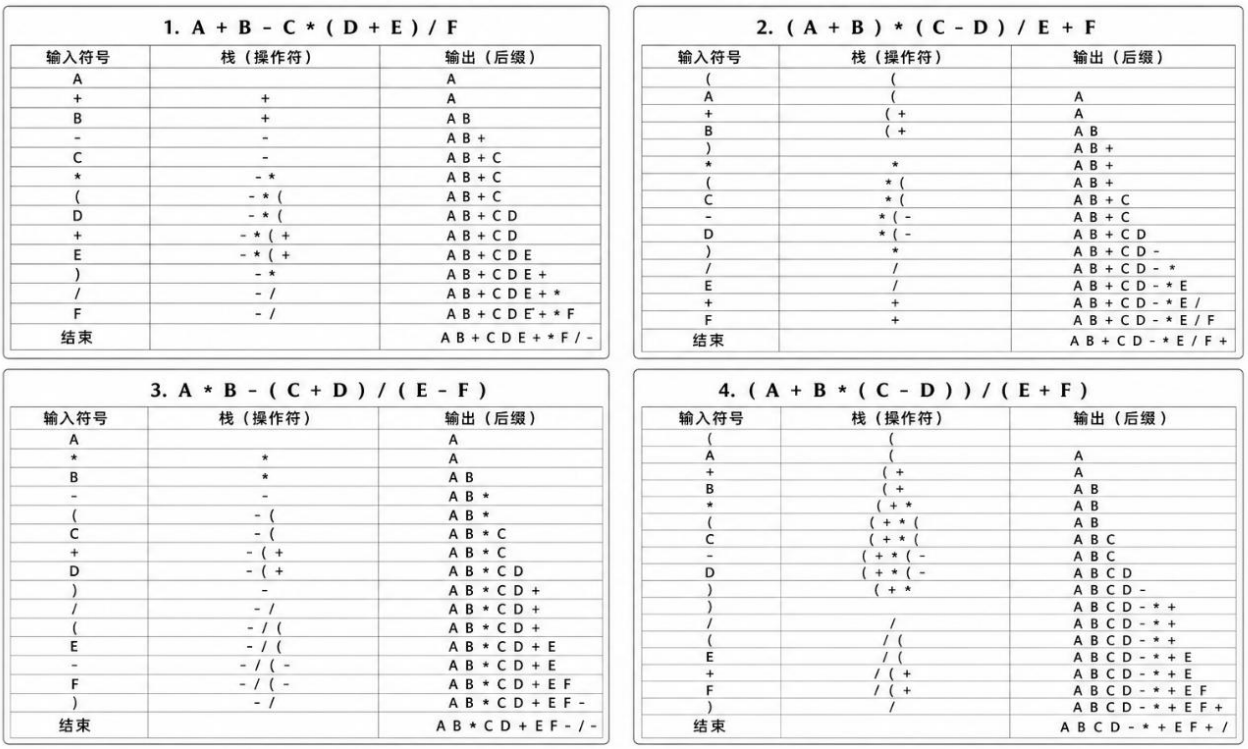
15．B。【解析】通过一个运算符栈将中缀表达式转换为后缀表达式。本题答案为 B。

16．A。【解析】AB*CD+-E+ --->　(AB * CD+-)+E ---> (AB*-(C+D))+E ---> (A*B-(C+D))+E ---> 40-(6+4)+15 = 45。

17．D。【解析】题干的后缀表达式转换成中缀表达式为：((5*8)*((2*7)-4))+8=40*10+8=408，答案选 D。

18．B。【解析】利用栈求表达式的值时，可以分别设立运算符栈和运算数栈，其原理不变。选项 B 中 A 入栈，B 入栈，计算得 R1，C 入栈，计算得 R2，D 入栈，计算得 R3，由此得深为 2。A、C、D 依次计算得栈深为 4、3、3。因此选 B。

19．D。【解析】　四个选项的出入栈如图所示。



1. A + B - C * (D + E) / F

输入符号	栈（操作符）	输出（后缀）
A		A
+	+	A
B	+	A B
-	-	A B +
C	-	A B + C
*	- *	A B + C
(	- * (	A B + C
D	- * (	A B + C D
+	- * (+	A B + C D
E	- * (+	A B + C D E
)	- *	A B + C D E +
/	- /	A B + C D E + *
F	- /	A B + C D E + * F
结束		A B + C D E + * F / -

2. (A + B) * (C - D) / E + F

输入符号	栈（操作符）	输出（后缀）
(	(	
A	(	A
+	(+	A
B	(+	A B
)		A B +
*	*	A B +
(	* (	A B +
C	* (	A B + C
-	* (-	A B + C
D	* (-	A B + C D
)	*	A B + C D -
/	/	A B + C D - *
E	/	A B + C D - * E
+	+	A B + C D - * E /
F	+	A B + C D - * E / F
结束		A B + C D - * E / F +

3. A * B - (C + D) / (E - F)

输入符号	栈（操作符）	输出（后缀）
A		A
*	*	A
B	*	A B
-	-	A B *
(	- (	A B *
C	- (	A B * C
+	- (+	A B * C
D	- (+	A B * C D
)	-	A B * C D +
/	- /	A B * C D +
(	- / (	A B * C D +
E	- / (	A B * C D + E
-	- / (-	A B * C D + E
F	- / (-	A B * C D + E F
)	- /	A B * C D + E F -
结束		A B * C D + E F - / -

4. (A + B * (C - D)) / (E + F)

输入符号	栈（操作符）	输出（后缀）
(	(	
A	(	A
+	(+	A
B	(+	A B
*	(+ *	A B
(	(+ * (	A B
C	(+ * (	A B C
-	(+ * (-	A B C
D	(+ * (-	A B C D
)	(+ *	A B C D -
)		A B C D - * +
/	/	A B C D - * +
(	/ (	A B C D - * +
E	/ (	A B C D - * + E
+	/ (+	A B C D - * + E
F	/ (+	A B C D - * + E F
)	/	A B C D - * + E F +
结束		A B C D - * + E F + /

选项	中缀表达式	后缀表达式	最大操作符栈深度
A	A + B - C * (D + E) / F	A B + C D E + * F / -	4
B	(A + B) * (C - D) / E + F	A B + C D - * E / F +	3
C	A * B - (C + D) / (E - F)	A B * C D + E F - / -	4
D	(A + B * (C - D)) / (E + F)	A B C D - * + E F + /	5

20．C。【**解析**】我们要知道一个原则，在中缀表达式里面，操作符的优先级在同级情况下，左边的比右边的优先级高，而前缀用栈模拟过程中：

- 操作数直接输出（从右往左输出）
- 右括号直接入栈
- 其他运算符入栈前，要比较和栈顶运算符的优先级，只有比栈顶优先级大，才入栈，否则出栈到栈顶运算符优先级比当前小，或者栈空为止，再入栈当前操作符。
- 若是左括号，那就出栈到对应右括号为止。

输入符号	栈	输出
c		c
-	-	c
b	-	bc
+	+ -	bc
a	+ -	abc
结束		-+abc

21．C。【解析】

【方法一】括号法

按运算顺序添加括号：((a * (b + c)) - d)，按计算顺序依次把括号内的运算符提到括号外，再去掉括号

① 括号先算：(b + c) → 前缀是 + b c

② 然后：(a * (b + c)) → 前缀是 * a + b c

③ 最后：* a + b c - d → 前缀是 - * a + b c d

【方法二】栈模拟，参考上题。

22．D。【解析】A 错误，在实现函数调用和递归过程中，通常使用**栈**来存储局部变量和返回地址；B 错误，BFS 中用**队列**作为辅助数据结构；C 错误，括号匹配通常采用**栈**作为辅助数据结构

23．D。【解析】删除操作虽然发生在队头，但是若当前队列只有一个数据元素时，则队头也是队尾。考虑到这种特殊情况，删除操作也会带来队尾的改变。

24．C。【解析】初始状态为 front=rear=NULL 说明是不带头结点的链队列：入队列，rear+1，front 不变；出队列，front+1，rear 不变（第一次入队列除外，rear+1，front+1）。因为不带头结点的队列，第一个元素入队时需要特别处理，所以只有当队列为空时，或者只有一个元素时，rear 才会等于 front。

25．A。【解析】注意题目中这里 Queue MaxSize 等于 M+1。题目里 f 作为队头指针指向队头元素的前一个位置，r 作为队尾指针指向队尾元素，用(r+1)%(M+1)==f 作为队满的标志，说明牺牲一个单元来区分队空和队满，队列中最多可以有 M 个元素，注意王道书 79 页（循环队列那里）的是 f 作为队头指针指向队头元素，r 作为队尾指针指向队尾元素的后一个位置（下一次要插入元素的位置），但判空判满的条件本质还是不变的。

26．D。【解析】front 指向队头，说明 front 的位置就是出队元素的位置，所以应该是先出队，再 front 自增。rear 指向队尾元素的下一个位置，说明这个位置是空的也就是下一个要入队元素的位置，所以是先入队，再 rear 自增。由于是循环队列，所以自增后都要取模，因此答案选 D。

> 这道题改编自真题，真题问的是判空盘满的条件，大家可以不看选项的情况下自己写出来
>
> 队空：front == rear；队满：front == (rear + 1) mod M
>
> 其次，要是改成 rear 指向队尾元素位置，大家也要会自己分析。

27．C。【解析】模 M-1 的范围是[0，m-2]，会导致数组下标为 m-1 的结果缺失，所以必须模 M，直接排除 B。队列为空时，当 end1 指向队头元素，end2 指向队尾元素时，rear 在 front 前面一格，由于

要牺牲一个位置区分循环队列判空判满，所以队满时 end2 和 end1 相差 2，因此答案按选 C。

28．D。【解析】循环队列由固定大小 M 的数组 A 实现，队列必须空出一个单元来区别队空和队满的判断。也可以用其他方法，但是都需要标志信息的额外开销，比如用 count 来记录队列元素个数，使用标记位（入队置 true，出队置 false，若 rear 和 front 无法区分队满还是队空，用标记位 true 表示满，false 表示空）等，但都需要额外的开销，所以必须牺牲一个队列空间，故队满情况下元素个数为 M-1。(rear - front) % M 为当前队列的元素个数，因此答案选 D。

29．B。【解析】删除一个元素时 front 循环增 1，加入两个元素时 rear 循环增 2。本题答案为 B。

30．A。【解析】因为是循环队列，所以 rear 可能会从 m-1 回到 0，导致比 front 小，所以要采取取模来计算元素个数，答案选 A。这类题目可以举例来快速得出答案。

31．C。【解析】由题给出条件可得队首在 rear-(length-1)的位置(因为 rear 占一个位置)，由于减法可能结果为负数，考虑到循环特性，需要加上 m 再对 m 取模。或特殊值法：取 m=3，length=2，rear=0，则此时 front 在 2 的位置，带入计算，C 满足条件。

32．A。【解析】在栈中，栈底指针保持不变，有元素入栈，栈顶指名增加，有元素出栈，栈顶指针减少。在循环队列中，队头指针和队尾指针的动态变化决定队列的长度。在循环链表中，前一个结点指向后一个结点，而最后一个结点指向头结点，只有头结点是固定的。线性链表中，由于前一个结点包含下一个结点的指针，尾结点指针为空，要插入或删除元素，只需要改变相应位置的结点指针即可，头指针和尾指针无法决定链表长度。故本题答案为 A 选项。

33．D。【解析】选项 A 可以进一个出一个，右侧入队，左侧或右侧出队，Ka、Kb、Kc、Kd、Ke 每个进出完再继续下一个入队。

选项 B 可以所有的进去之后再依次出去，Ka，Kb，Kc，Kd，Ke 依次右侧进入，再依次右侧出来，类似于栈。

选项 C 可以先 Ka，Kb 都右侧进，左或右侧出； Kc，Kd，Ke 依次进，Ke 右侧出，Kc 左侧出，Kd 左右均可出

选项 D 无法得出这个序列。

34．A。【解析】如果题目改成所有元素依次入队后再进行入队操作（一次性出队）（10 年真题），则 III、IV 不可能得到。但是本题是可以中间出队的，I、II、III 比较好判断。对于 IV（假设只能右侧出队）：

①a 左入，b 右入得 ab，b 出队（b）

②c 右入得 ac，c 出 a 出（bca）

③d 左入，e 右入，e 出 d 出（bcade）

35．A。【解析】A 选项错误，d 第一个输出，那么此时队列里面还剩 abc，由于输入在两端都可以进行，且三个元素是按 abc 依次输入，那么 a、b 一定相邻，即 acb 一定不可能出现，A 错误。

B 选项正确，假设只能左边出，进出队列如下：a 左边入队得到 **a**；b 右边入队得到 **ab**；c 左边入队得到 **cab**；c 出队，得到 **ab**；a 出队，得到 **bc**；d 左边入队，得到 **db**，再依次出队 d，b，得到答案 B；

C 选项显然正确；

对于 D 选项：a 左入队，得 **a**；b 左边入队得 **ba**；b 出队，得 a；c 右边入队，得 **ac**；d 左边入队，得 **dac**；再依次出 dac，D 选项正确；

36．C。【**解析**】输入受限的双端队列，当$P_1 = n, P_2$的可能取值只可能是 1 和 n-1 两种情况。

37．C。【**解析**】 入队顺序和出队顺序一定一样，所以 A 正确。第一个出栈是 n-2，说明此时 1 到 n-2 都已经入栈，只需要再入栈 n-1 和 n 再出栈 n 即可满足 B 的情况。第一个出栈是 n-3，则 1 到 n-3 都已经出栈，此时 n-3 出栈后，再入栈 n-2 再出栈，随后出栈 n-4 即可满足 D 的情况，但是无论如何都不可能使 n-6 第三个出栈，C 错误。

38．D。【**解析**】A 正确，使用两个队列实现栈时，一个用于存储栈内的元素，一个用于出栈操作的辅助队列 push（）时，新元素在非空队列中入队；pop（）时，将元素逐个出队再入队到另一个队列，至原队列只剩最后一个元素时，将该元素出队，完成 pop（）操作。

B 正确，使用一个队列模拟栈，需要满足队列 前端元素是最后入栈的元素。push（）时，首先将新元素入队，再将队列其他元素逐个出队再入队，这样可以使得新元素处于队首，模拟栈的功能。

C 正确，将栈 stack1 当作输入栈，存储 push（）传入的数据；栈 stack2 当作输出栈，用于 pop（）和 push（）操作。每次 pop（）或 push（）时，若输出栈为空则将输入栈的全部数据依次弹出并压入输出栈，这样输出栈从栈顶往栈底的顺序就是队列从队首往队尾的顺序。

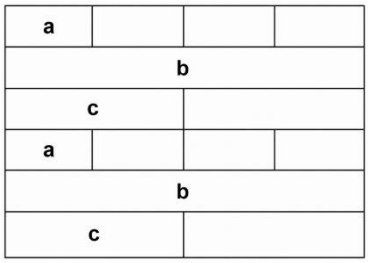
D 错误，不能够实现该要求。

39．C。【**解析**】操作过程如下：e1 进栈，e2 进栈，e2 出栈后进队，e3 进栈，e4 进栈，e4 出栈后进队，e3 出栈后进队，e5 进栈，e6 进栈，e6 出栈后进队，e5 出栈后进队，e1 出栈后进队，栈中最多元素时为 3 个。本题答案为 C。

40．A。【**解析**】a[5] [5] 的地址＝ 1000+ (5 * 6+5) * 5=1175。

41．B【**解析**】M 为 5 行 6 列，在按行存储时，元素 M[3, 5] 的起始地址为((3-0)*6+(5-0))*4=92。在按列存储时，元素 M[2, 4] 的起始地址为((4-0) * 5+(2-0)) * 4=88，元素 M[3, 4] 的起始地址为((4-0)* 5+(3-0)) * 4=92，元素 M[3, 5]的起始地址为((5-0) * 5+(3-0)) * 4=112，元素 M[4, 4]的起始地址为((4-0) * 5+(4-0)) * 4=96。

42．C。【**解析**】存的的下三角，元素$m_{6,10}$是上三角部分的元素，因此要按$m_{10,6}$来求地址。列优先，则$m_{10,6}$是第 10+9+8+7+6+5=45 个元素。结构体需要考虑对齐，结构体整体按内部最大空间的类型对齐，int 类型 4B 为最大，所以一个结构体占 12B，44*12+4=532，首地址是 1000，因此答案是 C。



43．C。【**解析**】第一行到第 7 行分别存放 1 到 7 个元素，A8 是第 8 行第 5 个元素，是所有元素中的第 1+2+...+7+5=33 个元素，答案选 C。

44．A。【解析】只考虑下三角，第 i 行有 i+1 个元素，设 B[i]的索引为 S[i]，则 S[i] = S[i-1] + i + 1，初始条件 S[0] = 0，S[1] = 2，S[2] = 5，S[i] = S[i-1] + i + 1 = S[0] + 2 + 3 + 4 + ... + i + i+1 = (i+1)(i+2)/2 - 1 = i(i+3)/2。即 B[i]的索引为 i(i+3)/2。

45．B。【解析】按列优先顺序，数组到第 j 列之前存放了 n+(n-1)+..+(n-j+1)=(2n-j+1) * j/2 个元素，然后再加上 i-j 可得答案。

46．A。【解析】指定的元素 aij，满足 i>=j，则该元素位于主对角线的下方，是未被直接存储的元素。由于是对称矩阵，它的存储位置和 aji 一致。对在对角线上方的 aji 而言，它之前共有（i-1）列元素，在第 1 列上有 1 个元素，在第 2 列上有 2 个元素……在第（i-1）列上共有（i-1）个元素。这若干列上的元素个数总和为 1+2+…+（i-1）=i（i-1）/ 2，而在第 i 列上，aji 是第 j 个元素，所以在整个存储排列中 a 是第 i（i-1） / 2+j 个元素，B 的下标从 1 起始，也就是说它在 B 中的位置为 i（i-1） / 2+j。

47．C。【解析】A 错误，三元组的内容为（行，列，值），其次还要存储稀疏矩阵的行列总数（否则无法分辨矩阵规格）和非零元素个数（三元组表的大小）。交换每个三元组的行列之外，还要交换总行和总列才可以实现转置。B 错误，稀疏矩阵存储方式有三元组法和十字链表法，三元组存储是顺序存储方式，十字链表法是链式存储方式。D 错误，十字链表适合频繁插入删除和快速定位行列元素，三元组存储效率一般低于十字链表，同理 C 正确。

48．A。【解析】用三元组存放矩阵中的元素，形式为(i，j，e)　其中 i 表示行号，j 表示列号，e 表示元素值

该题矩阵形式为：

$$\begin{bmatrix} 0 & 0 & 5 & 0 & -3 \\ 3 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 5 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

49．C。【解析】稀疏矩阵采用二维数组存储时，它具有随机存取特性，而采用压缩存储后不再具有随机存取特性，本题答案为 C。

50．B。【解析】广义表是线性表的推广，408 还未考过，但是教材有相关介绍，这题作为防出题老头 gank。

广义表是**元素的有序集合**，其中每个元素可以是：

- 原子（Atom）：即普通的数据元素，如整数、字符等；
- 也可以是另一个广义表（即子表）。

相关术语：

表头（Head）：广义表的第一个元素；

表尾（Tail）：去掉表头后剩下的子广义表；

深度（Depth）：广义表中最长嵌套层数；

长度（Length）：广义表中顶层元素个数。

回到本题：(a, (b, c), ((d, e), f))，表头是 a，A 正确。

表尾是（(b, c), ((d, e), f)），B 不正确。

顶层一共有三项，所以长度为 3，C 正确。

深度为广义表的嵌套层数，第三个子表中还嵌套了一个子表，所以深度为 3 正确，D 正确。

51．C。【解析】这道题采用手工求 next 数组的方法。先求串 s='ababaaababaa'的部分匹配值：

'a'的前后缀都为空，最长相等前后缀长度为 0。

'ab'的前缀{a}∩后缀{b}=Ø，最长相等前后缀长度为 0。

'aba'的前缀{a, ab}∩后缀(a, ba}={a}，最长相等前后缀长度为 1。

'abab'的前缀{a, ab, aba}∩后缀{b, ab, bab}={ab}，最长相等前后缀长度为 2。

依次求出的部分匹配值见下表第 3 行，将其整体右移一位，低位用-1 填充，见下表第 4 行。

编号	1	2	3	4	5	6	7	8	9	10	11	12
S	a	b	a	b	a	a	a	b	a	b	a	a
PM	0	0	1	2	3	1	1	2	3	4	5	6
next	-1	0	0	1	2	3	1	1	2	3	4	5

52．C。【解析】见上题，在实际 KMP 算法中，为了使公式更简洁、计算简单，若串的位序是从 1 开始的，则 next 数组才需要整体加 1；若串的位序是从 0 开始的，则 next 数组不需要整体加 1。

53．C。【解析】nextval 从 0 开始，可知串的位序从 1 开始。第一步，令 nextval[1]=next[1]=0。

编号	1	2	3	4	5	6	7	8	9	10	11	12
S	a	b	a	b	a	a	a	b	a	b	a	a
next	0	1	1	2	3	4	2	2	3	4	5	6
nextval	0	1	0	1	0	4	2	1	0	1	0	4

从 j=2 开始，依次判断 s[j]是否等于 s[next[j]]?若是则将 next[j]修正为 next[next[j]]，直至两者不相等为止。答案选 C。

54．C。【解析】先求 next 数组为-1, 0, 1, 0, 1, 0, 1, 2, 3, 4。再求 nextval 数组，假设模式串为 S，规则如下：

- nextval[0]直接写-1
- 从第二个开始，判断第 i 个字符 S[i]是否等于 S[next[i]]
  - 如果不等，直接 nextval[i] = next[i];
  - 相等，nextval[i] = nextval[next[i]];

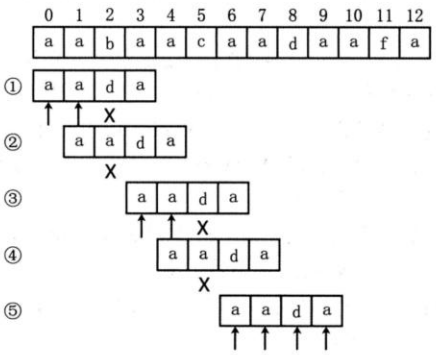
根据以上规则答案选 C。

55．D。【解析】next 数组本质是反映模式串自身的特性，在求它的值时，跟此后会与哪个主串进行匹配操作完全无关。

56．B。【解析】首先要计算模式串(P = "aada")的(nextval)数组：

- j = 0，默认(next[0] = -1)，(nextval[1] = -1)；
- j = 1，前面字符串为"a"，(next[1] = 0)，(nextval[1] = -1)；
- j = 2，前面字符串为"aa"，(next[2] = 1)，(nextval[1] = 1)；
- j = 3，前面字符串为"aad"，(next[3] = 0)，(nextval[3] = -1)。

下标 j	0	1	2	3
模式串	a	a	d	a
nextval	−1	−1	1	−1



匹配过程：

- 第一次匹配，前两位字符匹配上，失配发生在(S[2])和(P[2])，此时已经比较了三次；
- 查(nextval)数组知，下次对比(S[2])和(P[1])，比较一次
- 发现不同($S[2] \neq P[1]$），(nextval[1])是(-1)，下次比较时主串向后移动一位，模式串回到首位，即对比(S[3])和(P[0])，这次依次向后比较三次，在(S[5])和(P[2])失配
- 查(nextval)数组知，下次对比(S[5])和(P[1])，比较一次
- 同样发现不同，主串后移一位，模式串(P)回到首位，依次比较四次发现匹配上。共匹配(3 + 1 + 3 + 1 + 4 = 12 次。

57．C。【解析】I 正确，普通匹配算法的时间复杂度为 O（mn），kmp 算法时间复杂度为 O（m+n），如果每次匹配部分匹配都接近模式串长度 n，那么普通匹配算法达到最坏时间复杂度 O（mn），相反普通匹配算法和 kmp 接近。

II 错误，主串指针在 j=0 失配时，i 最后会+1。

III 错误，正确 next 数组为'-1，0，1，2'，部分前缀和后缀长度应小于总长度。

IV 错误，和 II 一样，在 j=0 失配时，j 置为-1，还需要 i, j 都+1

58．B。【解析】A 错误，在 j 位置失配，说明主串 T[i]≠S[j]，那么当 S[j]==S[next[j]]的时候，下一次匹配仍然会失配，这也是为什么要引入 nextval 数组的原因，可以消除这种冗余的比较。B 正确，在计算 nextval 数组时，当 S[j]≠S[next[j]]时，令 nextval[j]和 next[j]相等；否则，当 S[j]=S[next[j]]时，nextval[j]=nextval[next[j]]，而不是 nextval[j]=next[next[j]]，D 错误。C 同样错误，当下标从 0 开始时，nextval[0]=-1，但 nextval[1]不一定为 0，也有可能是-1，比如模式串“aa”，nextval={-1,-1}。

59．D。【解析】假设 T、S 的下标都从 0 开始，首先计算出 S 的 next 数组为‘-1, 0, 0, 1, 1, 2’，失配情况如下：

- `i=5, j=5`，失配修改后，`i=5, j=2`　(右移 5-2=**3**)
- `i=8, j=5`，失配修改后，`i=8, j=2`　(右移 5-2=**3**)
- `i=9, j=3`，失配修改后，`i=9, j=1`　(右移 3-1=**2**)
- `i=9, j=1`，失配修改后，`i=9, j=0`　(右移 1-0=**1**)

- i=9, j=0，失配后，修改为 i=9, j=-1，然后两者同时+1，变为 i=10, j=0 (首个失配右移 1)
- i=11, j=1，失配修改后，i=11, j=0 (右移 1-0=1)
- i=11, j=0，失配后，修改为 i=11, j=-1，然后两者同时+1，变为 i=12, j=0 (首个失配右移 1)
- i=13, j=1，失配修改后，i=13, j=0 (右移 1-0=1)
- 匹配成功

总计右移距离 3+3+2+1+1+1+1+1=13。

这题也可以用更快速的思路：设模式串长度为 m（本题是 6），正常我们一次性匹配成功，比较的次数为 m 次即可。但是由于我们中间会失配，失配后模式串右滑，我们要从某个位置和主串接着比较，主串 i 是不回退的，所以右滑的距离就是我们要多比较的次数。设匹配的位置是 x（本题是 19），那么原本 m 次比较现在变成了 x 次比较，多比较的次数就是模式串的右滑总距离，即 x-m=19-6=13。

60．C。【解析】A 错误，n 至少是 4，利用贪心的思想，每次入队如果有某个队列的队尾元素比自己小，那就选择一个和自己最相近的元素所在队列插入，没有的话就重新开辟一个队列。4 个队列分别入队[8,9] [4,5,6,7][2,3][1]时，所用队列数最少。B 错误，k 尽可能小，队列大小 k 最小为 1，至少需要 9 个队列，C 正确，D 错误；n 尽可能小(n=4)，k 最小为 3，4 个队列分别入队[8,9] [4,5,6][2,3,7][1]。

61．C。【解析】本题是栈的应用考察。左括号视为入栈，右括号视为出栈，本题就变成了问 3 个元素入栈有多少种不同的出栈序列个数。答案是卡特兰数，n=3 的卡特兰数是 5。也可自己枚举：((()))、(()())、(())()、()(())、()()()。

62．D。【解析】A 正确，所有元素依次入队，再从左端依次出队。B 正确，所有元素依次入队，再从右端依次出队。C 正确，1、2 入队，右边出 2，左边出 1，3、4、5 依次入队，右侧依次出队 5、4、3，正确。D 错误，5 第一个出队，说明所有元素都依次入队了，此时队列还剩 1、2、3、4。无论左右都无法出 3。

63．C。【解析】下标从 0 开始编号，先求 next 数组=[-1,0,1,0,1,2]，再求 nextval 数组=[-1,-1,1,-1,-1,1]。用下标 i 表示主串 T 位置，j 表示模式串 S 位置，每次失配时，根据 nextval 来修改 j 的回溯位置。

当 i=j=1 时，T[1]=’b’≠S[1]=’a’，一共比较 2 次，第一次失配后，i=2,j=0;

当 i=4，j=2 时，T[4]=’a’≠S[2]=’b’，一共 比较 3 次，第二次失配后，i=4,j=1;

当 i=6，j=3 时，T[6]=’c’≠S[3]=’a’，一共比较 3 次，第三次失配后，i=7,j=0;

当 i=12，j=5 时，匹配成功，一共比较 6 次。总计比较 2+3+3+6=14 次，答案选 C。

64．B。【解析】I 正确，普通顺序队列（数组实现、非循环）中，元素不断出队导致队头 front 后移，数组前部出现大量空闲空间；但队尾指针 rear 已到达数组最大下标，无法继续入队。数组明明未满，却判定为“溢出”，这就是假溢出。循环队列将一维数组在逻辑上首尾相连，利用取模运算 (rear+1) % MaxSize 让队尾指针走到数组末尾后回到数组头部，复用前部空闲空间，彻底解决假溢出。II 正确，教材上队列的两种存储结构分别为循环队列和链式队列，所以说循环队列本身也是一种

存储结构。III 错误，链式队列，队头一般设置在链表头部，出队涉及到删除操作，如果设置在链表尾部，删除单链表尾部元素的时间复杂度为 O(n)；IV 错误，逻辑上连续的两个结点的指针在物理上不一定连续，链式存储是动态申请结点空间，在内存是不连续的，所以无法通过指针做差计算队列长度。答案选 B。

第 5 章

树

【答案速对】

BDABB　DADAB　CCCDA　CCCCC

DBBCC　DBACD　CABDB　ADDCC

BBCCD　DBCCA　BDBDB　BDDCD

BDCAD　CDACD　ABACC　DDDDB

BD

1．B。【解析】度数之和 + 1 = 结点总数。可以得到 $81 + N_2 = N0$，$N_1 + N_2 = 10$，根据题目意思，N_1和N_2都至少存在一个，所以N_2最多取到 9。N_0最大是 81 + 9 = 90 个。

2．D 度的总和+1=结点总数，设n_m为度为 m 的节点个数，由题：$n_3 = 2, n_2 = 1, n_1 = 7$ 所以 $4n_4 + 3n_3 + 2n_2 + n_1 + 1 = n_4 + n_3 + n_2 + n_1 + n_0 \rightarrow 3n_4 - n_0 = -6$ ①，结点总数为 20，所以$n_4 + n_3 + n_2 + n_1 + n_0 = 20 \rightarrow n_4 + n_0 = 10$ ②，联立①②，解得$n_0 = 9$。

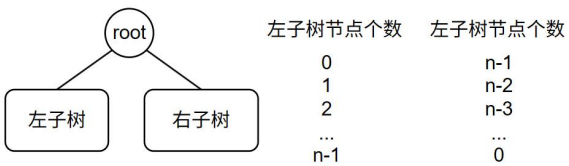
3．A。【解析】这样的树中至少有一个节点的度为 4，也就是说，至少有一层中有 4 个或 4 个以上的节点，因此，树的高度至多是 n-3。本题答案为 A。算最小高度应该用满 4 叉树来推。

4．B。【解析】n 个结点 m 叉树，一共有 mn 个指针域，除了根节点没有父亲结点，其他 n-1 个结点都有唯一一条链指向它（一个分支对应一个子结点），所以使用的指针域有 n-1 个，空的指针域有 $mn - (n-1) = (m-1)n + 1$。

5．B。【解析】本题答案为 n=4 的卡特兰数。

证明：利用卡特兰数的递推式：$H(n) = H(0)H(n-1) + H(1)H(n-2) + \ldots + H(n-1)H(0)$

思想：确定根节点后，剩余 n-1 个结点各自分配在左右子树，形成以上卡特兰数的递推数学模型。



6．D。【解析】高度=结点数，说明每层只有一个结点，高度为 8 的二叉树，第 8 层最多容纳 $2^{8-1} = 128$ 个结点，即第 8 层每个结点到根节点的路径都是符合本题的二叉树的树型。答案选 D。

7．A。【解析】二叉树中结点个数可以为 0，称为空树，所以 B 错。满二叉树中，任何一个结点的度，或者为 0，或者为 2。完全二叉树中，任何一个结点的度，或者为 0，或者为 1，或者为 2，所以 C 错。二叉树的度可以是 0，1，2，所以 D 错

8．D。【解析】设度为 0 和 2 的结点数分别为。在二叉树中，存在结论：分支数 +1= 结点数。在正则二叉树中不存在度为 1 的结点。则存在：$2 \times n_2 + 1 = n_2 + n_0$，可得出：$n_2 = n_0 - 1$，$n_2 + n_0 = n$，则。

9．A。【解析】二叉链表表示的二叉树中空指针的数量为 n+1，三叉链表表示的二叉树多了一个根结点指向双亲的空指针，所以树中空指针的数量为 n+2, I正确。若根结点的度为 2，则只有左、右两个孩子指向它，II错误。若整棵树只有一个根结点，则没有指针指向它，III错误。

10．B。【解析】采用排除法，由二叉树的性质知，若二叉树只有度为 0 和度为 2 的结点，则总结点是一个奇数，可以排除 A 和 C。当 h = 2 时，一个二叉树最多有结点数 2^2-1 = 3 个，而 D 项等于 5，可以排除 D。

11．C。【解析】A：不满足 n0 =n2 +1 的性质，所以不存在这样的二叉树。

B：在完全二叉树中，n1 =0 或 1，而 n0=n2+1，所以分支节点个数 n1 +n2 =n0 或 n1 +n2 =n0-1。

C：通过只有两个节点（其中 a 为根节点，b 为 a 的右孩子）的二叉树来验证本叙述是不正确的。

D：在完全二叉树中，已知节点总数就可以确定其形状。已知其先序序列，便可知其节点总数，再根据先序序列确定每个节点的位置。实际上，只要已知完全二叉树的任一种遍历序列都可以唯一确定该二叉树。

12．C。【解析】当根结点下标为 1 时，下标为 i 的结点的父结点下标为$\lfloor i/2 \rfloor$，那么下标为 17 的祖先的下标有 8, 4, 2, 1，下标为 19 的祖先的下标有 9, 4, 2, 1，因此两者最近的公共祖先的下标是 4。

13．C。【解析】$511 = 2^9 - 1 < 513 < 2^{10} - 1 = 1023$，所以层高最小应该是完全二叉树的形态，高度为 10，最高的树形是一条单链，层高为结点数 513，所以 10-513 都可以取到，答案为 513-10+1=504 种。

思考：m 叉树呢？度为 m 的树呢？层高可能是多少种。

14．D。【解析】本题考查几种特殊二叉树的性质。对于 A 选项，满二叉树，设层数为 h，则 $2^h - 1 = n$，求出 h，叶结点都在最后一层上，即叶结点数为2^{h-1}。

对于 B 选项，在完全二叉树中，度为 1 的结点数为 0 或 1，$N = 2N_0 + N_1 + 1$，则$\mathrm{N}_0 = \lfloor (n+1)/2 \rfloor$。

对于 C 选项，哈夫曼树只有度数为 2 和 0 的结点，$N_0 = N_2 + 1$，$N_0 + N_2 = n$，即$N_0 = (n+1)/2$ 可得叶结点个数。

对于 D 选项，则无法求出叶结点个数。

15．A。【解析】度为 0 的比度为 2 的结点多一个，度为 0 的结点 30 个，所以度为 2 的结点 29 个，度为 1 的结点 20 个，所以一共有 30+29+20=79 个。

16．C。【解析】完全二叉树这种题目，因为叶子结点只可能在最后一层和倒数第二层，所以既可以问最多的情况，也可以问最少的情况。

最多的情况：8 个叶子是倒数第二层上的叶子数量。这时候倒数第二层是第 6 层，一共有$2^{6-1}=32$个结点，分支结点有 32-8=24 个结点，最多的情况就是这 26 个分支结点都有 2 个孩子，也就是前 6 层是满二叉树，最后一层有 24*2=48 个叶子结点的情况，最多有$2^6-1+48=111$个结点。

最少的情况：8 个叶子是最后一层上的叶子数量，由完全二叉树的性质，前 5 层节点总数由满二叉树公式$2^5-1=31$，加上第 6 层，最少 31+8=39 个结点。

17．C。【解析】完全二叉树一共有 8 个叶结点，而普通二叉树最少可以有 1 个叶结点(单链)，相差最大 8-1=7。

18．C。【解析】

I 正确，对于正常二叉树，空指针域数量是 n+1 个，再讨论本题三叉链表引入的父亲指针，只有根结点没有父亲结点，所以多出来一个空指针域，因此总的空指针域是 n+2 个。

II 错误，比如一棵高度为 h 的二叉树，根节点左子树高度为 h-1，那么根结点的右孩子的前驱结点是根节点左子树里的先序序列最后一个结点，这个结点一定在最底层，所以最坏结果，时间开销是 O（h）。

III 正确，叶子结点没有孩子结点，所以指向他的结点只有他的父亲节点，但要注意根节点是叶子结点的例外，这里题目说了 n>1，所以不存在根节点是叶子结点的情况，因此该选项正确。

IV 错误，度为 2 的分支结点可能是根节点，根节点的双亲指针为空。

19．C。【解析】先序序列是…a…b...，则 b 在 a 的左子树中或者 b 在 a 的某个祖先 x 的右子树中，中序序列是…b…a…，则 b 不可在 a 的某个祖先 x 的右子树中，即 b 只能在 a 的左子树中。

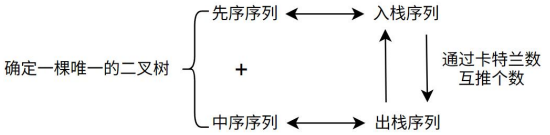
20．C。【解析】二叉树中序遍历的最后一个结点一定是从根开始沿右子女指针链走到底的结点，设用 p 指示。若结点 p 不是叶结点(其左子树非空)，则前序遍历的最后一个结点在它的左子树中，A、B 错误：若结点 p 是叶结点，则前序与中序遍历的最后一个结点就是它，C 正确。若中序遍历的最后一个结点 p 不是叶结点，它还有一个左子女 q，结点 q 是叶结点，那么结点 q 是前序遍历的最后一个结点，但不是中序遍历的最后一个结点，D 错误。

21．D。【解析】后序遍历的顺序是 LRN，若 n 在 N 的左子树，m 在 N 的右子树，则在后序遍历的过程中 n 在 m 之前访问；若 n 是 m 的子孙，设 m 在 N 的位置，则 n 无论是在 m 的左子树还是在右子树，在后序遍历的过程中 n 都在 m 之前访问。其他都不可以。选项 C 要成立，就需加上两个结点位于同一层这个条件。

22．B。【解析】首先观察先序和中序的非递归代码，可以发现，先序序列是在入栈的时候输出元素，中序序列是在出栈的时候输出元素，其他代码都一致。即得出：先序等价于入栈序列，中序等价于出栈序列。入栈序列可以通过卡特兰数推出栈序列个数。先序+中序可以确定一棵唯一的二叉树，先序确定，中序序列的不同个数是卡特兰数，所以二叉树的树型即卡特兰数种。n=4 的卡特兰数是$Catalan(4)=\frac{C_{2*4}^4}{4+1}=14$。

```c
//先序非递归算法
void PreOrder2(BiTree T){
    InitStack(S);   //初始化栈S
    BiTree p=T      //p是遍历指针
    while(p||!IsEmpty(S)){//栈不空或p不空时循环
        if(p){   //一路向左
            visit(p);//访问当前结点
            Push(S,p);//当前节点入栈
            p=p->lchild;//左孩子不空，一直向左走
        }
        else{  //出栈，并转向出栈结点的右子树
            Pop(S,p);//栈顶元素出栈
            p=p->rchild;//向右子树走，p赋值为当前结点的右孩子
        }//返回while循环继续进入if-else语句
    }
}
```

```c
//中序非递归算法
void InOrder2(BiTree T){
    InitStack(S);BiTree p=T;//初始化栈S；p是遍历指针
    while(p||!IsEmpty(S)){//栈不空或p不空时循环
        if(p){   //一路向左
            Push(S,p);//当前节点入栈
            p=p->lchild;//左孩子不空，一直向左走
        }
        else{  //出栈，并转向出栈结点的右子树
            Pop(S,p);//栈顶元素出栈
            visit(p);//访问栈顶元素
            p=p->rchild;//向右子树走，p赋值为当前结点的右孩子
        }//返回while循环继续进入if-else语句
    }
}
```



23．B。【解析】原题同上题，由于二叉排序树（BST）中序是升序序列，n 个不同的元素组成的二叉排序树，中序序列唯一。确定中序序列（出栈序列），可以推不同的先序序列个数（入栈序列）是卡特兰数种，再由先序+中序确定一棵唯一的二叉树可以得到，不同的二叉树树型是 n=4 的卡特兰数 14，答案为 B。

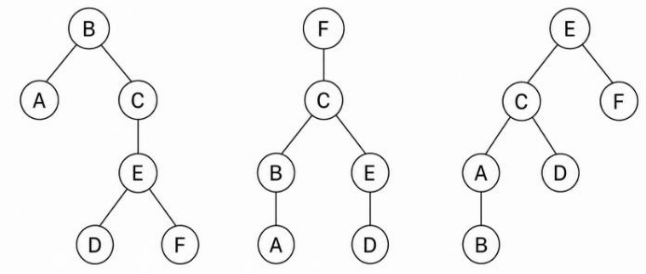
24．C。【解析】通过中序遍历即可得到正确的算术表达式，注意，每读出一个操作符，都需要在对应位置加上括号：(((A * B)+C)) / ((D * E)+(F - G)))。再根据无意义的括号合理删减，找出选项中与之匹配的正确答案。答案选 C。

25．C。【解析】在后序遍历退回时访问根结点，就可以从下向上把从 n 到 m 的路径上的结点输出，若采用非递归的算法，则当后序遍历访问到 n 时，栈中把从根到 n 的父指针的路径上的结点都记忆下来，也可以找到从 m 到 n 的路径。

26．D。【解析】I错误，树的层序遍历一般使用队列。II正确，图的深度优先遍历必须使用栈。III错误，图的广度优先遍历一般使用队列。IV正确，见上一题解析。

27．B。【解析】只要按照前序序列的顺序入栈，无论怎么出栈肯定是符合中序序列的，所以这个问题就变成了，如果给定一个栈，入栈顺序是 ABCDEFG，那么下面哪种出栈顺序是可能的。只有 B 选项可以满足。

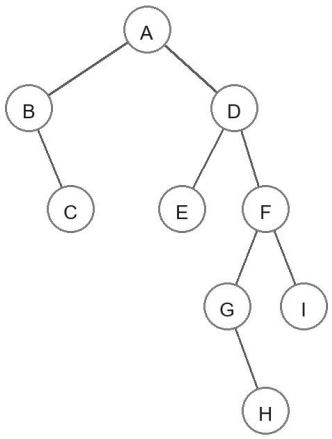
28．A。【解析】中序遍历和后序遍历能唯一确定一棵树，能画出一棵树即符合，只有 A 不符合，其他三个选项树的结构如下：



29．C。【解析】二叉树的前序序列为 ABCDEFG，A 为根节点。中序序列为 DCBAEFG，可知 DCB 为左子树节点，EFG 为右子树节点。同理 B 为 C 父节点，C 为 D 父节点。同理 E 为 F 根节点，F

为 G 根节点。故二叉树深度为 4 层。C 选项正确。

30．D。【解析】由中序序列和层序序列可以唯一确定一棵二叉树，对于层序序列，可以确定子树的根节点，在中序序列中的左右位置可以确定这个节点是左子树还是右子树的根结点，由此直到遍历完整个序列。二叉树如下图所示，可知正确答案是 D。



31．C。【解析】二叉排序树的中序遍历为有序序列 10.15.20.23.25.30.35.39.42。而题目给出了后序遍历序列 15.10, 23.25.20.35.42.39.30。因此可以构建唯一一棵二叉树，前序遍历为 30.20.10.15.25.23.39.35.42。

32．A。【解析】I错误，任一二叉树的叶结点在先序、中序、后序遍历序列中相对位置一定是相同的。II错误，要交换二叉树的所有分支结点的左右子树的位置，利用前序或后序遍历框架解决最合适，只要保证先处理好左右子树，再处理根结点的思路就行。III正确，含有 n 个结点的二叉树最多有$\frac{C_{2n}^{n}}{n+1}$种形态。IV正确，NLR 和 LNR 顺序相反，说明对于每一层的 L，N，R，中 L 为空或 R 为空，得到的树的结果就是该二叉树仅有一个叶子结点。

33．B。【解析】三种遍历分别是“根左右”，“左根右”，“左右根”，所以只有分支结点之间顺序有差异，叶子节点都是从左到右按顺序排放的。

34．D。【解析】由题可知，该二叉树只可能是左单链或右单链，故答案选 D。

35．B。【解析】中序是“左根右”，后序是“左右根”，因为根（分支结点）是不可或缺的。所以考虑减去左子树或者右子树保证他们是一致的。如果没有左子树，那么中序是“根右”，后序是“右根”，两个相反；没有右子树，中序是“左根”，后序也是“左根”

36．A。【解析】元素个数确定，完全二叉树的树型也是确定，可以知道是一棵高度为 3 的满二叉树，再按中序遍历将元素值依次填到各个结点，然后写出先序序列，可以知道答案为 A。

37．D。【解析】首先分析根结点孩子的情况，因为先序序列是 a，e，后序结尾是 e，a，那么有几种情况：①**若根结点有左孩子**。那么 e 一定是左孩子，且一定没有右孩子（否则右孩子结点后序一定夹在 e，a 之间，不符合题意）②**若根节点没有左孩子**。那么 e 一定是右孩子。因此根节点只有一个孩子，A 选项正确。

既然根只有一个孩子，那么可以不考虑根 a 了，只考虑以 e 为根的子树。先序为 e, b, d, c，后序是 b, c, d, e，可以观察在先序中，b 在 e 后面，那么只可能 b 为 e 的左孩子（反证法：如果 b 是右孩子，那

么由前序 e，b 相邻可推一定没有左子树，但是后序结尾一定是 b，e，但题目是 d，e，可知此情况不符合题意）。由于后序中 b 在最前面，所以 b 一定没有孩子（那么 e 的左子树只有 b 一个结点）。最后只剩一下一个结点 c 了，可知 c 在 d 的左右皆可。

综上，B 正确，该度为 2 的结点为 e。C 正确。D 选择，c 在 d 左右皆可。

由上面分析，此题有 4 种树型，e 居 a 左右皆可（2 种），c 在 d 左右皆可（2 种），两者独立，所以总树型 2*2=4。大家看到这里应该也能很轻松画出来。

这题是真题改编，做到这里也算把这道题目研究到位了，希望大家体会这种逐帧分析的思维过程。

38．D。【解析】A 正确：前序与中序一致 → 无左子树，每次直接访问根，再遍历右子树。

B 正确：后序与中序一致 → 无右子树，遍历完左子树直接访问根。

C 正确：单节点二叉树（仅有根，无左右孩子），三种遍历序列完全相同。

D 错误：任意一棵二叉树的所有叶子节点，在三种遍历中都遵循先左后右，相对顺序永远不变，这是重要推论。

39．C。【解析】先序遍历(根、左、右)：首先访问根结点，然后遍历左子树，最后遍历右子树。后序遍历(左、右、根)：首先遍历左子树，然后遍历右子树，最后访问根结点。假设存在一个结点 z 把 x、y 分居 z 两侧，由先序遍历可知，x 在 z 的左子树，y 在 z 的右子树，那么在后序遍历中，x 一定也在 y 前面，与题意不符。因此不存在这样的 z，即 x、y 一定同时是根节点到他们各自的路径上的中间结点，即 x 是 y 的祖先。

40．C。【解析】当 5 出队的时候，此时队列里面有 6、7、8、9、10、11 一共 6 个元素，所以队列空间的最小值是 6。

41．B。【解析】A 错误，树的遍历中，没有中序遍历的概念；对于 B，树转换为二叉树时，结点的左孩子是它在树中的第一个孩子，右孩子是它在树中的兄弟，而根结点不存在兄弟结点的概念，必然没有右子树；

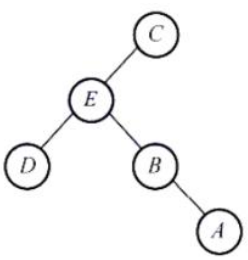
C 错误，树的先序遍历就是相应二叉树的先序遍历；

D 错误，当树中的叶结点有兄弟结点时，转换后它就有右孩子了，而不再是叶子结点。

42．B。【解析】完全二叉树中度为 1 的结点数为 0 或 1。当 n1=0 时，有 n2=n, no=n+1，共 2(n+1) 个空指针作线索。当 n1=1 时，有 n2=n-1, no=n，共 2n+1 个空指针作线索。

43．C。【解析】根据线索二叉树的定义，rtag=0 表示有右孩子，rtag=1 表示没有有孩子，右指针表示的是后继的线索。

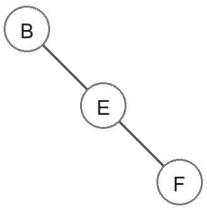
44．C。【解析】中序+后序可以确定一棵唯一的二叉树，如下图所示，右孩子为空的结点在线索二叉树中，右孩子指针指向线索。由图可知有 CDA 三个。



45．D。【解析】D 选项错误，若该非根结点是其父亲结点的左孩子，且存在右孩子，那么右指针一定指向右孩子所在子树后序遍历的第一个结点。

B 正确，先序线索二叉树可能无法直接找到前驱，后序线索二叉树可能无法直接找到后继，需要通过从头遍历或者三叉链表（多一个指针指向父亲结点）找到。

46．D。【解析】后序线索二叉树不能有效解决求后序后继的问题。如下图所示，结点 E 的右指针指向右孩子，而在后序序列中 E 的后继结点为 B，在查找 E 的后继时仍然只能按常规方法来查找。



注意：先序线索二叉树中求前驱也是不能有效求解

47．B。【解析】后序遍历序列为 f、d、e、b、c、a，左右线索分别指向 c 的后序前驱 b 和后继 a，因此答案选 B。

48．C。【解析】先序线索二叉树的缺点：无法找到先序序列中某结点的前驱；后序线索二叉树的缺点：无法找到后序序列中某结点的后继。

	中序线索二叉树	先序线索二叉树	后序线索二叉树
找前驱	√	x	√
找后继	√	√	x

49．C。【解析】A 正确，该结点的中序后继结点是它的右子树的最左边的结点，所以它没有左孩子。它的中序前驱结点是它的左子树的最右边的结点，所以它没有右孩子。B 正确，可以通过具体的例子来验证该结论。C 错误，中序线索二叉树当右孩子为空可以通过右线索直接找到后继，如果右孩子不为空，也可以找到中序后继（通过右孩子找到右子树中最左的结点），因此可以完成中序遍历。D 正确，因为 x 是一个根结点，x 要么是 y 的右孩子（y 在 x 的左边，y 的数值比 x 的小），要么是 y 的右孩子（y 在 x 的右边，y 的数值比 x 的大），又因为 x 没有左子树和右子树，所以 y 即是 x 的前驱或后继的结点。即 y 的数值要么是树中大于 x 的数值的最小关键字，要么是树中小于 x，数值的最大关键字。

50．A。【解析】B：如果仅有右孩子，则中序形式为：根、右，此时后继节点存在。

C：先序与后序的形式分别是根、左、右和左、右、根，根据这两种遍历序列的信息无法区分左、右子树。D：通常有左子树而无右子树。

51．B。【解析】在树的孩子兄弟表示法中，若一个结点没有孩子(即叶结点)，则表现为该结点的左指针域为空，因此本题答案为“6”。至于“5 个结点的左、右指针域都为空”，表示树中有 5 个结点既没有孩子又没有兄弟，约束条件比题中的“求叶结点的个数”要求更严格。

52．D。【解析】

A 正确，原森林中每一个分支结点产生唯一一个对应二叉树中没有右孩子的结点，且这个结点是原

森林中这个分支节点最右边的孩子（左孩子右兄弟，最右边的孩子没有右兄弟，所以对应二叉树中右指针为空），森林中最右边子树的根节点在二叉树中也没有右孩子，这 1 个是特殊的。由此可以知道原森林中分支节点个数+1=二叉树中没有右孩子的节点数量。

B 正确，森林左孩子右兄弟转换中，在二叉树中只有每棵树的根节点相连接的分支是比原森林多出来，所以当树为 1 棵时，没有多余的分支多出。

C 正确，“左孩子右兄弟”，二叉树中没有左孩子的结点即原森林中的叶子节点。

D 错误，若二叉树中该结点是根节点，那么它的左右孩子不存在亲缘关系。

53．B。【**解析**】F 的先根遍历对应 T 的先序遍历，所以画出 T 的图即可得到答案为 B。

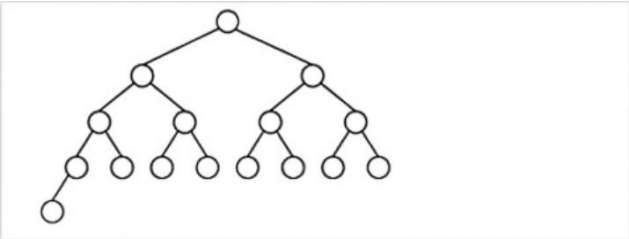
54．D。【**解析**】二叉排序树的中序遍历为有序序列 10, 15, 20, 23, 25, 30, 35, 39, 42。二叉树转换的森林的后根遍历对应二叉树的中序遍历，故答案为 D。也可尝试层序+中序构造一棵二叉树可知 B 为先序，C 为后序，不过稍显复杂。

55．B。【**解析**】树和森林的遍历与二叉树遍历的对应关系

树	森林	二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

56．B。【**解析**】该树至少有两个节点，对于有两个或两个以上节点的树，至少有两个节点没有右兄弟(根节点和其最右孩子节点)，而右指针域指向兄弟节点。所以本题答案为 B。

57．D。【**解析**】画出图像如右发现根结点一直向右走直到叶结点的路径上，共有 3 个结点，所以森林中一共有 4 棵树，其中最大的那棵树是森林的第 1 棵树，共有 9 个结点。



58．D。【**解析**】将森林转换成二叉树方法是“左孩子右兄弟”。森林中每棵树都相互独立，可将每棵树的根结点全部视作兄弟。题目内的森林转换后，树 2 作为树 1 的根结点的右子树，树 3 作为树 2 的根结点的右子树。森林 F 对应二叉树根结点的右子树上的结点个数是 M_{2} + M_{3}。

59．C。【**解析**】每个指向双亲的指针都对应树中的一个分支，含有 n 个结点的树中最少有 n - 1 个分支。

60．D。【**解析**】A 正确，森林中有多少棵树，要看根节点开始往右遍历一共有多少个结点，高 h 的满二叉树有 h 个，所以有 h 棵树。

B 正确，原森林中每一个分支结点产生唯一一个对应二叉树中没有右孩子的结点，且这个结点是原森林中这个分支节点最右边的孩子（左孩子右兄弟，最右边的孩子没有兄弟，所以对应二叉树中右指针为空），森林中最右边子树的根节点本身也没有右孩子，这个是特殊的，由此可以知道原森林中分支节

点个数+1=二叉树中没有右孩子的节点数量。满二叉树没有右孩子的节点数量即最后一排的叶子节点2^{h-1}个，减去 1 即为原森林中的分支结点数。

C 正确，第 i 棵树对应的二叉树中的部分是根（根为第 1 个）的第 i 个右分支作为根节点，加上一个高度为 h-i 的满二叉树，所以总结点个数是$2^{h-i}-1+1=2^{h-i}$个

D 错误，第 i 棵树的二叉树部分见 C 选项解释，易知度最大的结点是二叉树中根节点的孩子结点。该结点在二叉树中往右一直遍历有 h-i 个结点（包括自己），即有 h-i-1 个右兄弟，所以度最大的结点为根结点，最大为 h-i。

61．B。**【解析】**树可以通过孩子兄弟表示法转换成唯一一棵二叉树，所以本题转换成求对应的二叉树的树型有多少种。

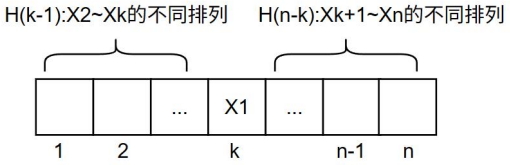
我们首先分析转化得到的二叉树树型特点，由于树的根节点没有兄弟结点，所以在转换的二叉树中，根节点没有右兄弟，即除根节点之外的 3 个结点构成一棵左子树。这个左子树的树型数目就是我们要求的答案。3 个结点的二叉树不同树型可用卡特兰数来求，$\frac{C_{2n}^{n}}{n+1}$代入 n=3 求得答案为 5。

卡特兰数的题型总结：

- 入栈序列为 12...n 的出栈序列个数？

证明方法一：利用卡特兰数的递推式：$H(n)=H(0)H(n-1)+H(1)H(n-2)+\ldots+H(n-1)H(0)=\frac{C_{2n}^{n}}{n+1}$

当元素 1 放在出栈序列第 k 个位置时，前面 k-1 个位置一定是 2~k 共计 k-1 个元素的出栈排列个数 H(k-1)，右边位置是 k+1~n 共计 n-k 个元素的出栈序列排列个数 H(n-k)，由此得到上述递推式。

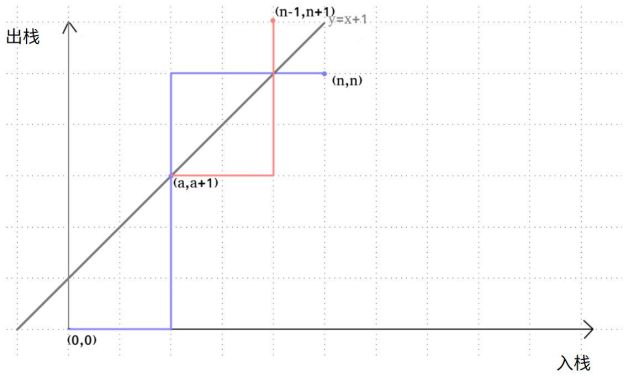


证明方法二：网格法，$\frac{C_{2n}^{n}}{n+1}=C_{2n}^{n}-C_{2n}^{n-1}$

入栈操作为横坐标，出栈操作为纵坐标，n 次入栈+n 次出栈操作即从原点出发到(n, n)的路径个数，即 2n 次操作里面选择 n 次入栈（剩下的是出栈）的可能数，即C_{2n}^{n}，但是每个时刻，出栈次数不能比入栈次数多，否则就是不合法。如下图，我们可以知道，不合法的操作路径，一定会触碰 y=x+1 到达(n, n)，而这样的路径经过 y=x+1 反射都会到达点（n-1, n+1），所以所有不合法的路径即从原点出发到点(n-1, n+1)的所有路径数量。

综合以上，合法操作路径条数=$C_{2n}^{n}-C_{2n}^{n-1}=\frac{C_{2n}^{n}}{n+1}$

（只要触碰到 y=x+1 说明当前状态出栈比入栈次数多，非法序列），



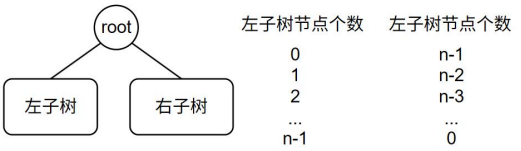
- 出栈序列为 12...n 的入栈序列个数？

逆向思维，根据上述网格法，其实合法序列就是不同的路径的条数，每一条路径的出栈出栈次序有差异，得到的出栈序列也有差异，那么我们固定出栈序列，通过路径还原成入栈序列，肯定也是不同的。所以结果还是卡特兰数。

- n 个结点二叉树的树型有多少种？

证明：利用卡特兰数的递推式：$H(n)=H(0)H(n-1)+H(1)H(n-2)+\ldots+H(n-1)H(0)$

思想：确定根节点后，剩余 n-1 个结点各自分配在左右子树，形成以上卡特兰数的递推数学模型。

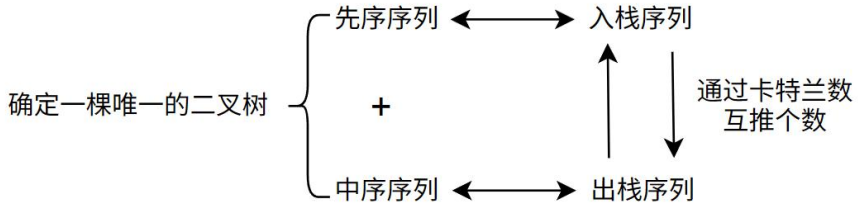


- 进一步，结合树和二叉树的转换（左孩子右兄弟）：含有 n 个结点的树的有多少种形态 （n-1 的卡特兰数）
- 结合森林和二叉树的转换（左孩子右兄弟）：含有 n 个结点的森林的有多少种形态 （n 的卡特兰数）
- 结合二叉树先序中序：
  - 给定先序序列（n 个值不同）的不同二叉树的个数
  - n 个不同权值的结点的二叉排序树的个数（中序确定）

首先观察先序和中序的非递归代码，可以发现，先序序列是在入栈的时候输出元素，中序序列是在出栈的时候输出元素，其他代码都一致。即得出：先序等价于入栈序列，中序等价于出栈序列。入栈序列可以通过卡特兰数推出栈序列个数。先序+中序可以确定一棵唯一的二叉树，先序确定，中序序列的不同个数是卡特兰数，所以二叉树的树型即卡特兰数种。n=4 的卡特兰数是$Catalan(4)=\frac{C_{2*4}^{4}}{4+1}=14$。

```
//先序非递归算法
void PreOrder2(BiTree T){
    InitStack(S);    //初始化栈S
    BiTree p=T       //p是遍历指针
    while(p||!IsEmpty(S)){//栈不空或p不空时循环
        if(p){    //一路向左
            visit(p);//访问当前结点
            Push(S,p);//当前节点入栈
            p=p->lchild;//左孩子不空，一直向左走
        }
        else{  //出栈，并转向出栈结点的右子树
            Pop(S,p);//栈顶元素出栈
            p=p->rchild;//向右子树走，p赋值为当前结点的右孩子
        }//返回while循环继续进入if-else语句
    }
}
```

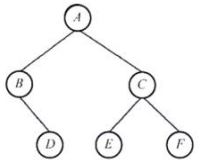
```
//中序非递归算法
void InOrder2(BiTree T){
    InitStack(S);BiTree p=T;//初始化栈S；p是遍历指针
    while(p||!IsEmpty(S)){//栈不空或p不空时循环
        if(p){    //一路向左
            Push(S,p);//当前节点入栈
            p=p->lchild;//左孩子不空，一直向左走
        }
        else{  //出栈，并转向出栈结点的右子树
            Pop(S,p);//栈顶元素出栈
            visit(p);//访问栈顶元素
            p=p->rchild;//向右子树走，p赋值为当前结点的右
        }//返回while循环继续进入if-else语句
    }
}
```



- n 对括号的不同排列个数：证明可以用坐标路径法。

62．D。【解析】原森林的先根遍历是二叉树的先序遍历为 5, 2, 1, 4, 3, 6, 10, 8, 7, 9。

63．C。【解析】中序+后序可以确定一棵唯一的二叉树。该二叉树如右图：



该二叉树对应的森林，树的个数即根结点到最右边叶子节点路径上的结点个数，即 A、C、F 三个。

64．A。【解析】T1，T2，T3 先分别转化为二叉树，除掉根结点，其它结点都转化为根结点的左孩子，第一棵二叉树不动，后面的二叉树分别是前面二叉树的右孩子。故本题中 T1 不动，左孩子结点 N1-1。

65．D。【解析】D：通过只有 3 个节点（其中 a 为根节点，b 为 a 的左孩子，c 为 b 的右孩子）的二叉树来验证本叙述是不正确的。C：当二叉树只有一个根节点时，任何遍历的序列均相同。

66．C。【解析】I 错误，比如 3 个结点构成的链；II、III 正确；IV 错误，3214 和 3421 和 3241 构成的 BST 都是一致的；V 错误，比如只有一个根节点。

67．D。【解析】在一组哈夫曼编码中，哈夫曼树中只有度为 0 或 2 的节点，D 不满足这种条件。

68．A。【解析】首先排除不满足前缀特性的选项，易知 B 选项出现 00 是 000 的前缀，排除。C 选项，10 是 1011 的前缀，排除。此时还剩 A，D。

接着根据频次越高，编码越短的特性。A，D 选项第一个和第二个编码长度刚好是相反的，第一个频次是 6，第二个频次是 3，所以第一个编码必然比第二个短，所以答案选 A。

这道题是真题，大家应该学会这种不画图的解法，多利用某些数据结构的性质。

69．C。【解析】A 错误，对于题目中的场景，使用哈夫曼编码对应的哈夫曼树是一棵满二叉树。

因为从最底层开始，最底层是两两配对的，根据题意对任一频率 f_i 有 $f_i \le f_{max} < 2f_i$，所以没有任何一对的频率超过另外一对的 2 倍，进而得出没有任何一个内部结点的频率是同一层的内部结点频率的两倍，所以该哈夫曼树是一个满二叉树。对于编码相应的前缀树是满二叉树情形，没有更优的编码方式了。B 错误，哈夫曼树中不存在度为 1 的结点。C 正确，哈夫曼树中 $n_1 = 0, n_2 = n_0 - 1$，那么总的结点数目 $n = n_0 + n_2 = 2n_0 - 1$，它是一个奇数。D 错误，对于权值都是正数的结点构成的哈夫曼树，父结点的权值一定是大于它的孩子结点的。

70．D。【解析】初始森林中的 n 棵二叉树，每棵树有一个孤立的结点，它们既是根，又是叶子。n 个叶子的哈夫曼树要经过 n-1 次合并，产生 n-1 个新结点。

71．A。【解析】题目里要求左节点的值小于右节点的值，所以构造下图所示的哈夫曼树：一棵哈夫曼树的带权路径长度等于树中所有的叶结点的权值乘上其到根结点的路径长度。所以长度：（2+3）* 4+（4+6+7）* 3+15 *1=86，B 的编码（也就是 3）为 1011。

72．B。【解析】A 不符合，可以知道，16+10=26，也就是第二层是符合的。但是第三层，16=5+11，10=6+4，4，5 更小，应该优先合并，而不是 4 跟 6 合并，不合符哈夫曼树最优二叉树的性质，因此错误。B 选择正确。

C 选项错误，26≠10+12，第二层出错。

D 选项错误，16≠7+8，第三层出错。

73．A。【解析】A 正确，带全路径长度 WPL 是每个叶节点的带权路径长度之和；

B 错误，哈夫曼树只有度为 2 和 0 的结点，并不一定是完全二叉树。

C 错误，度为 2 的有序树不可以是空树，但是二叉树可以。

D 错误，BST 不同于 AVL，删除分支结点一定要找一个前驱或后继来代替，而重新插入这个结点，一定在叶子结点，导致树形发生改变；同样地，删除叶子结点再插入是一定一样的。而 AVL 中，删除分支结点在插入，树形可能不改变，比如 123 依次插入，删除 2 再插入 2，树不改变。

74．C。【解析】度为 m 的哈夫曼树中，只有度为 m 和叶子结点，由公式：度数之和+1=结点总数可以得到，$mn_m + 1 = n_m + n_0 \to (m-1)n_m = n_0 - 1 \to n_m = (n-1)/(m-1)$，但我们要注意到，度为 m 的哈夫曼树，当$(n-1)\%(m-1) = 0$，不需要补充虚结点；当$(n-1)\%(m-1) \neq 0$，我们要补充虚结点，补充的虚结点个数为$(m-1) - (n-1)\%(m-1)$。$n_m$是整数，实际上加上虚结点的叶结点数是>=n 的，所以要取上整。

75．C。【解析】初始时，0~9 各自成一个集合。查找 1-2 时，合并{1}和{2}；查找 3-4 时，合并{3}和{4}；查找 5-6 时，合并{5}和{6}；查找 7-8 时，合并{7}和{8}；查找 8-9 时，合并{7, 8}和{9}；查找 1-8 时，合并{1, 2)和{7, 8, 9}；查找 0-5 时，合并{0}和{5, 6}；查找 1-9 时，它们属于同一个集合。最终的集合为{0, 5, 6}、{1, 2, 7, 8, 9}和{3, 4}，因此答案选择选项 C。

76．D。【解析】依次探测图的各条边，用并查集检查该边依附的两个顶点是否已属于同一集合(两个顶点的根结点是否相同)。若是，则说明图中存在环路，A 错误。经过路径优化后，并查集在最坏情况下的高度远小于 O(n)，B 错误。Find 操作返回的是集合根结点的下标，元素个数的相反数是 P[根的下标]，C 错误。

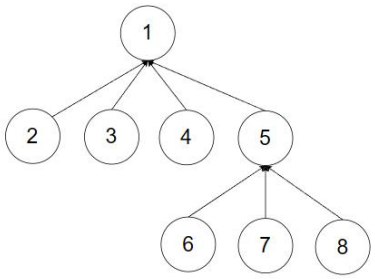
77．D。【解析】在用并查集实现 Kruskal 算法求图的最小生成树时：判断是否加入一条边之前，先查找这条边关联的两个顶点是否属于同一个集合(即判断加入这条边之后是否形成回路)，若形成回路，则继续判断下一条边：若不形成回路，则将该边和边对应的顶点加入最小生成树 T，并继续判断下一条边，直到所有顶点都已加入最小生成树 T。B 正确。用并查集判断无向图连通性的方法：遍历无向图的边，每遍历到一条边，就把这条边连接的两个顶点合并到同一个集合中，处理完所有边后，只要是相互连通的顶点都会被合并到同一个子集合中，相互不连通的顶点一定在不同的子集合中。C 正确。未做路径优化的并查集在最坏情况下的高度为 n，此时查找操作的时间复杂度为 O（n），时间复杂度通常指最坏情况下的时间复杂度。D 错误。

78．D。【解析】III 错误，返回的是根节点 root 的编号(大于 0)，而不是 S[root]（小于 0）。

IV 错误，Union()操作始终都是 O(1)，改进是小树合并到大树，的目是降低整个集合树的高度至 $O(log_2 n)$。

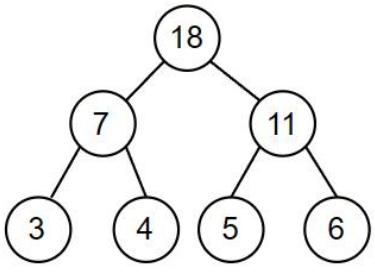
V 错误，Find()经过压缩路径优化后，可使集合树的深度为$O(\alpha(n))$，$\alpha(n)$是一个接近常数的函数。

79．D。【解析】得到的并查集如图所示，要注意最后一个 Union 要路径压缩，parent[8]=5，parent[4]=1，因为属于一个集合，所以 find(任意结点)=1。只有 D 正确。

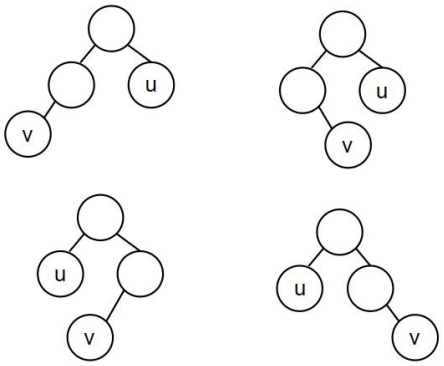


80．B。【解析】I 正确，BT 根结点一直往右遍历可以得到 k 个结点，那么 F 一定有 k 棵树，森林中，边+树棵树=结点个数，所以 F 中边数=n-k。II 正确，F 中任意一个分支结点，根据左孩子右兄弟，最靠右的孩子结点在对应二叉树中一定没有右指针。根节点是特殊的一个，在森林中，最右边那棵树的根节点一定没有右孩子，所以一共有 m+1 个空右指针的结点。III 正确，根据左孩子右兄弟，F 中没有左孩子，在原森林中一定是叶子结点（没有孩子）。所以二者是相等关系。IV 错误，也有可能是 h-1 棵树。只有满二叉树的情况下，一定保证是 h 棵树。

81．B。【解析】哈夫曼编码的加权平均长度是 wpl/总频次。构造的哈夫曼树如下图所示，wpl=(3+4+5+6)*2=36，所以加权平均长度=36/(3+4+5+6)=2。



82．D。【**解析**】分四种情况如下图所示。在树中：第一种情况，u 是 v 的爷爷的兄弟。第二种情况，u 是 v 的叔叔（A 选项），第三种情况，u 和 v 有共同爷爷（B 选项）。第四种情况，v 是 u 的叔叔（C 选项）。



第 6 章

图

【答案速对】

BADDC　CBDDC　ABCAB　ABCCB

ACCDD　BADCC　CAADA　BDCCA

BBAAD　DCCAC　ADDDC　BDAAA

CDCAD

1．B。【解析】边数 * 2 = 度数总和。答案为(16*4+2*3+5*2+4*1)/2=42。

2．A。【解析】选项 B、D 均为生成树的特点，而选项 A 为概念错误：生成树 G'为最小连通子图而非连通分量（最大连通子图），本题答案为 A。

3．D。【解析】A 错误。根据强连通分量的定义，u 与 v 之间互相存在路径，u' 与 v' 之间互相存在路径。若图 G 存在一条从点 v' 到点 v 的路径 $v' \to v$，则 G 也将包含路径 $(u \to u' \to v')$ 和。如此就会有 u 和 v' 相互可以到达，从而与 c 和 c' 是不同的强连通分量的假设矛盾。B 错误，若在图 G 中加入一条新边，若该边的两个顶点都处于同一个连通分量，则不会生成 G 的新的一个连通分量。C 错误。理由见 D。D 正确，若在图 G 中加入一条新边，若该边的两个顶点 A 和 A' 分别处于不同的连通分量，并且 A 和 A' 之间原先有一条路径，假设该路径简化后表示成 A->mids->A'(同理也可假设 A-mids->A)，若添加一条边生成路径 A'->A，形成一个环路，就会生成 A 和 A' 间的新的一个强连通分量，就会使得 G 的强连通分量的个数加 1。

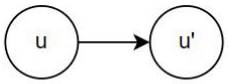
4．D。【解析】若一个无向图有 n 个顶点和 n-1 条边，可以使它连通但没有环(即生成树)，但若再加一条边，在不考虑重边的情形下，则必然会构成环。

5．C。【解析】A 错误。强连通分量内的任意两个元素都存在到彼此的路径。不同强连通分量的顶点可以只有单向的边。如下图。

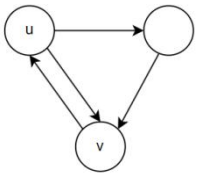
B 错误。u→v 的直接边被删除，但是可能仍然存在 u→v 的路径，比如下图，1→3 被删除后，仍然是一个强连通分量（环）。

C 正确。如果存在 u→u'的路径，那么强连通分量 c 的任意顶点可以通过 u→u'的路径到达强连通分量 c'的任意顶点。如果存在<v', v>，同理 c'任意节点可以通过<v', v>到达 c 中的任意节点，他们就是同一个强连通分量了，与题意不符。

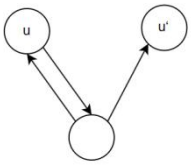
D 错误。添加后，强连通分量可能合并，导致减少。



A选项例子



B选项例子



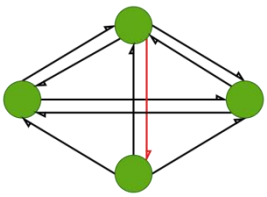
D选项例子

6．C。【解析】考查至少有多少个顶点的情形，我们考虑该非连通图最极端的情况，即它由一个完全图加一个独立的顶点构成，此时若再加一条边，则必然使图变成连通图。在 28=n(n-1)/2= 8x7/2 条边的完全无向图中，总共有 8 个顶点，再加上 1 个不连通的顶点，共 9 个顶点。

7．B。【解析】6 个顶点的完全有向图边数是 $2C_6^2 = 30$，所以 31 条弧的强连通有向图至少有 7 个顶点。

8．D。【解析】这题是根据真题改编，真题是无向图，要满足任意情况下所有结点都是联通的，那么 n 个结点中 n-1 个结点尽可能的多浪费边，即 n-1 个结点构成完全无向图，边数为C_{n-1}^2，最后一个结点再用一条边连接，这样能保证任意情况下都能连通，公式是 $1 + C_{n-1}^2$。

而这道题，要保证任意性，同样 n-1 个结点要保证多浪费尽可能多的边数，也就是完全有向图，这时候边数是 $2C_{n-1}^2$。最后一个顶点向 n-1 个顶点各发出一条出边，这样保证了 n-1 个结点是强连通的，而最后一个顶点不连通，此时任加一条边，都会构成强连通图。经以上分析，可以得到公式：$n + 2C_{n-1}^2$，n=4 时答案为 10，即选 D。



9．D。【解析】k > 1，说明整个图不连通，不连通的情况下最大边数是$(n-1)^2$，解析见上题。A、B 反例：如果 k=n，也就是每个节点都是一个强连通分量，那么边数 m=0，所以 m 最小值是 n-k。注意：如果 k<n，那么 B 是正确的，因为 k 个强连通分量，那么让 k-1 个结点为单结点强连通分量，剩余 n-k+1 个结点为剩余的一个强连通分量，n-k+1 个结点至少有 n-k+1 条边。

10．C。【解析】初始考虑只有 51 个顶点的无向图 G，此时 G 中每个顶点都是连通分量，问题转化为向 G 中添加 21 条边，如何添加这 21 条边使得连通分量数目最多。若向两个不同的连通分量之间添加边，则连通分量数目会减 1，所以应尽可能地将这 21 条边加入同一个连通分量且让其接近完全图，含有 7 个顶点的完全图有 21 条边，所以用 7 个顶点构成一个含有 21 条边的连通分量，剩下 51-7=44 个顶点对应 44 个连通分量，共有 45 个连通分量。

11．A。【解析】A 错误，强连通的有向图的任何顶点到其他顶点都有路径，而非弧。

12．B。【解析】n 个顶点的生成树是具有 n-1 条边的极小连通子图，因为 n 个顶点构成的环共有 n 条边，去掉任意一条边就是一棵生成树，所以共有 n 种情况，所以可以有 n 棵不同的生成树。

13．C。【解析】在树中出了根节点（没有父亲结点），其他每个结点对应一条边，所以多出的来边即根节点的个数，根节点的个数即树的棵树，一共 n-e 棵，答案选 C。

14．A。【解析】选项 A 错误。当且仅当有向图中每个强连通分量顶点数都大于 1 个，才满足 m≥n。反例：只要取一个连通分量顶点只有一个，就不满足结论。

15．B。【解析】强连通分量是极大强连通子图，任意两个顶点之间有方向相反的两条路径。由定义不难得出，若一个顶点只有出边或入边，则该顶点必定单独构成一个连通分量。图中，顶点 B 只有出边，其他所有顶点都不可能有到顶点 B 的路径，所以顶点 B 单独构成一个强连通分量。在顶点 A、C、D、E 中，任意两个顶点之间都有方向相反的两条路径，所以可构成一个强连通分量。

16．A。【解析】I 错误，任意无向图的生成森林中顶点数量=边数+连通分量数

II 正确.在一个连通无向图中，顶点数为 n，边数为 m，则它的生成树的边数为 n-1。

III 错误.对于有向无环图，拓扑排序可以得到一个线性次序，但不能直接判断两个顶点之间是否有路径。

IV 正确，强连通的定义是从任意一个顶点出发，都可以到达其他所有顶点。对于无向图，连通性即意味着强连通性。

17．B。【解析】为了使顶点数最少，我们需要构成一个完全连通图和一个单独点的子图，所以设有 10 条边的完全连通图的顶点数为 n，则 n(n-1)/2=10，可以求出 n=5，即该图的顶点数至少有 5+1=6 个。

18．C。【解析】A 错误，邻接表找寻有向图某指定弧比如<i,j>，只需要遍历顶点 i 的边表中是否存在顶点 j 的边表结点，时间复杂度是 O(e)或 O(k)，k 是顶点 i 出边数量。

B 错误，无向图没有入度和出度的概念，无向图的度就是与之相连的所有边，遍历对应邻接矩阵中的行/列，无需都遍历。

C 正确，无向图的表示中，邻接表每条边在边表中对应 2 个结点，所以结点总数有 2e 个（e 是边数），邻接多重表中边表结点只有 e 个，但每条边对应的边表结点都被其两个端点关联，这种关联在邻接多重表中体现在指针上，要么是某个顶点上一条关联边的指针指向该条关联边，要么是顶点表的指针指向该关联边，总体指针数量是 2e。

D 错误，前半句正确后半句错误，十字链表法统计入度和出度都很方便。

19．C。【解析】本题考查深度优先遍历。深度优先遍历是找到新的访问结点后，就从新结点开始找新的访问结点，如果没有找到，那么回溯到上一个找到的新的访问结点继续查找。从顶点 1 出发，下一个新访问结点 3，从 3 开始，找到 4，从 4 开始，没有新结点，回溯到 3，找到新访问结点 5，从 5 开始，找到 2，从 2 开始没有新结点，回溯到 5，没有新结点，回溯到 3，没有新节结，回溯到 1，没有新结点，访问结束。所以得到的顶点序列为 1, 3, 4, 5, 2。

注：当一个图只给了相应的图形时，那么它采用哪种遍历方式，遍历序列一般都是不唯一的，但是在给定了存储结构(邻接矩阵或邻接表等)时，一般相应的遍历序列都是唯一的。

20．B。【解析】从邻接矩阵中删除一个顶点，需要删除对应的行和列，也就是要重新定义一个邻接矩阵并赋值，时间复杂度为 O(N^2)，其中 N 是顶点的个数，因此 A 选项错误。

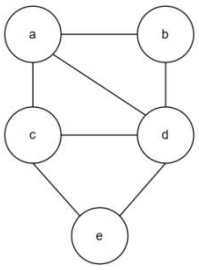
B 正确，有向图中的边有一个起点和一个终点，因此每条边会使起点的出度加 1 和终点的入度加 1，所以入度和出度之和等于边数的两倍。

C 错误，子图的概念是顶点属于原图的顶点子集，边也是原图的子集，且所有边子集中的边对应的两端顶点也属于子图。强连通分量内的子图可能是不连通的。

D 错误，拓扑排序的结果可能并不唯一。具体来说，当有向无环图中存在多个入度为 0 的顶点时，可以从这些顶点中任选一个进行排序，这导致拓扑排序的结果有多种可能。

21．A。【解析】计算顶点 i 的入度，要遍历第 i 列非 0 元素个数。由邻接矩阵可以知道，各顶点入度依次是 1, 2, 1, 2。

22．C。【解析】该邻接多重表表示的无向图如下图所示，可知 C 选项的序列是深度优先遍历。



23．C。【解析】n 个点连通且无环的简单无向图为连通图，连通则至少有 n-1 条边，无环则只有 n-1 条边。

n 个点连通且无环的简单无向图有 n-1 条边，非零个数为 2(n-1)，零元素个数为 n^2-2(n-1)。

得出零元素个数为 n^2-2n+2。

24．D。【解析】带权有向图的邻接矩阵中，元素 0 和∞表示的都不是有向边，而入度是由邻接矩阵的列中元素个数计算出来的。本题答案为 D。（请回顾拓扑排序算法实现过程）

25．D。【解析】D 错误。下三角矩阵说明编号大的顶点是弧尾，编号小的顶点是弧头，不会出现编号小的顶点指向编号大的顶点的情形，此时说明该有向图拥有拓扑排序，但拥有拓扑排序序列的有向图的邻接矩阵不一定是三角矩阵，有向无环图具有拓扑排序序列，其邻接矩阵没有明显的特征。

26．B。【解析】邻接矩阵只跟图的顶点有关，邻接表既和顶点有关也和边有关。

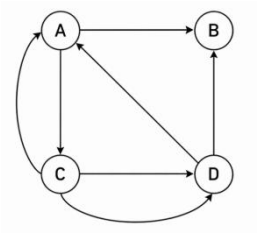
27．A。【解析】对于连通图而言，从它的某个顶点出发，进行一次深度优先或广度优先搜索，即可访问图的每个顶点；对于非连通图而言，从某个顶点出发，进行一次深度优先或广度优先搜索，只能访问连通分量内的每个顶点，若要访问图的所有顶点，还需从余下的连通分量中选择一顶点出发进行遍历。

28．D。【解析】要求生成树中不能有回路。

29．C。【解析】先转换为邻接矩阵 A，$A^2[2][4]$即所求答案。

$$A=\begin{bmatrix}0&1&1&1&0\\1&0&1&1&1\\1&1&0&1&0\\1&1&1&0&1\\0&1&0&1&0\end{bmatrix},A^2=\begin{bmatrix}3&2&2&2&1\\2&4&2&3&1\\2&2&3&2&2\\2&3&2&4&1\\1&1&2&1&2\end{bmatrix},A^2[2][4]=3$$

30．C。【解析】表示的有向图如下图所示，入度为 2 的顶点有 A，B，C 三个。也可不同画出原图。只需要从顶点表每个橙色格子出发，一直沿着橙色格子继续遍历到橙色格子为^的边表结点为止，图中经过的结点数量为 2 即为入度为 2，即 A、B、C 三个顶点。



31．C。【解析】因为是深度优先，找到与顶点 0 直接相连的结点，由邻接矩阵知道是顶点 1（多个相邻节点取第一个找到的未遍历到的结点），然后再在邻接矩阵中找与顶点 1 直接相连的结点，得到顶点 3.相同方法找到后续结点为：顶点 4，顶点 2.因为顶点 2 的相连结点都已被遍历，所以退回到顶点 4 继续遍历，遍历到顶点 5，然后是顶点 6。顺序为 ：0 1 3 4 2 5 6。

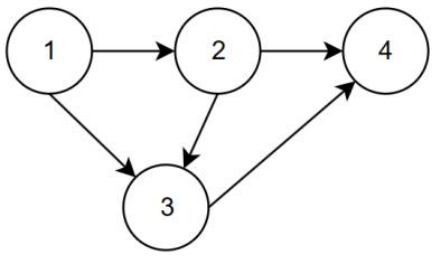
32．A。【解析】广度优先搜索以起始结点为中心，一层一层地向外层扩展遍历图的顶点，因此无法考虑到边权值，只适合求边权值相等的图的单源最短路径。广度优先搜索相当于树的层序遍历，选项 III错误。广度优先搜索需要用到队列，深度优先搜索需要用到栈，选项 IV 正确。

33．A。【解析】I 错误，无向图的深度优先遍历中，DFS 的调用次数是连通分量的个数。在有向图中并不满足，比如“1→2”。从顶点 1 开始，这个图只调用了一次 DFS，但是强连通分量有两个，分别是 1，2。

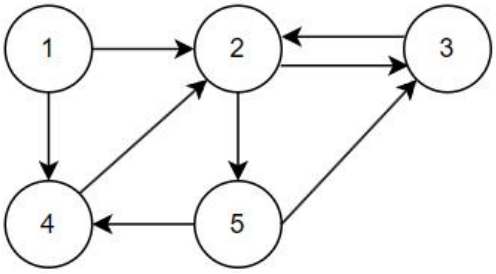
II 正确，回边表示当前调用的 dfs 过程中，正在访问的结点的边 u，有一条边<u, v>，目的顶点 v 仍在当前递归栈中，它保证了两个顶点在一次 dfs()中。也就是说，u 是从 v 走过来的，同时也能回到 v，也就是产生了环。

III 错误，不在一个 DFS()中，被访问两次，可能没有环。比如 1←2，从 1 出发，调用两次 DFS()，1 被访问后，第二次 DFS（2），有边<2, 1>，但是没有环。（注意和 II 的回边进行区分）

IV 正确。如下图，逆拓扑序列是 4321（想想为什么不能是 2431？）



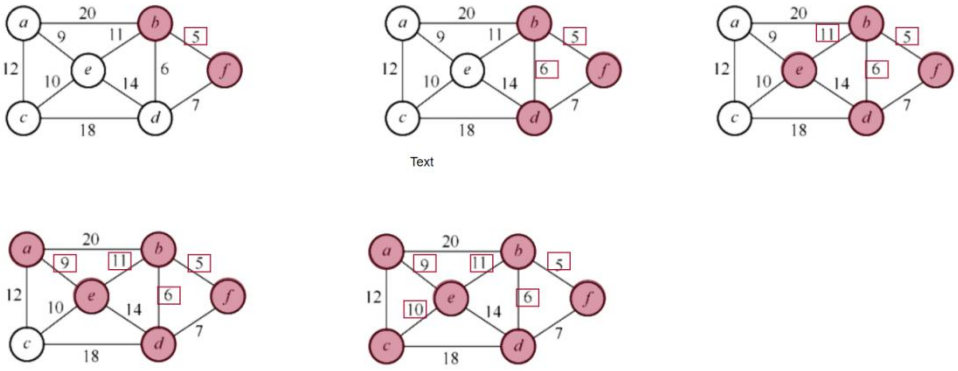
34．D。【解析】 D。【解析】该有向图如下图所示，可能的深度优先遍历序列有 12543 12534 12354 14235 14253



35．A。【解析】U= {1, 2, 3}，V- U={4, 5, …}，候选边只能是这两个顶点集之间的边，只有 A 符

合题意。

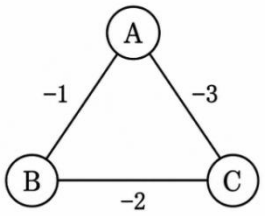
36．B。【解析】如下图所示



注意：答案 A 是 kruskal 算法的结果，可自己验证一下。

37．D。【解析】当前选取的顶点集合为{1,3,4}，Prim 只能从已选取的顶点集合出发，选取其他顶点。Kruskal 算法按照边权从小到大依次考察边，若加入该边不会形成回路，则选取该边。本题 A 选项 1,4 顶点已经在一个集合，所以 Prim 算法和 Kruskal 都不可能选取。B 选项 Kruskal 算法和 Prim 算法都可能选；C 选项因为权值 2<3，(2,5)2 不可能后于(3,4)3 选取，只可能是 Prim 算法选取。只有 D 选项符合。

38．C。【解析】C 错误，若图的所有边的权重都是正数，则任一连接所有结点且总权重最小的一个边集合必然形成一棵树。因为假设边子集 T 中存在环，则某两点间存在多条通路，移除其中一条通路可得子集 T‘，T’仍连通所有点。因边的权重为正，有 w(A')<w(A)，结论与条件矛盾，所以 T 必定为树。若边的权重准许为负，则子集 T 不一定就是树，例如：如下图所示的图中三条边的总权重最小。



39．C。【解析】I 错误，**Prim** 每一轮选取的是离当前生成树距离最近的顶点。

II 正确，不使用堆优化的 **Prim** 算法，无论是邻接矩阵还是邻接表存储，都是$O(V^2)$。使用堆优化更新 dist 数组的开销，算法步骤如下：

- 初始把<起点，0>插入堆中，第二项是 dist 值。
- 出堆直到取到未被访问过的顶点 **i**（visited 数组实现），然后遍历该顶点的邻边V_j，直接向堆中插入$< V_j, G[i][j] >$
- 继续第二步，直到选取出所有顶点(堆空即可)

由于堆中会有顶点重复的边，每次出堆是惰性删除已经被访问过的顶点的数据，所以最多访问 E 次，每次出堆或者入堆的时间开销是$O(log_2V)$，所以整体复杂度是$O(Elog_2V)$

III 错误，稠密图是$E = O(V^2)$，所以临接矩阵存储下，时间复杂度是$max\left(O(v^2),O(Elog_2E)\right)$，因为$E = O(V^2)$，所以$O(Elog_2E) \approx O(v^2logV) > O(V^2)$，所以时间复杂度是$Elog_2E$。

注意：在稀疏图下，$E=O(V)$，邻接矩阵存储下，**Kruskal** 算法时间复杂度为$O(V^2)$。

IV 正确，**Kruskal** 算法需要并查集来判断选取的边是否属于同一个集合，以及合并集合操作，所以要使用并查集。

40．A。【**解析**】A 正确，见严蔚敏撰写的教材《数据结构》。Diikstra 算法适合求解有回路的带权图的最短路径，也可以求任意两个顶点的最短路径，不适合求带负权值的最短路径问题。在用 Floyd 算法求两个顶点的最短路径时，当最短路径发生更改时，$path_{k-1}$就不是$path_k$的子集。

41．B。【**解析**】A 错误，Dijkstra 算法不能求上图的结点 1 单源最短路径，从结点 1 出发，先选择结点 2，最短路径长度确定为 6，然后选择结点 3，最短路径长度为 7。说明了 **Dijkstra 算法不能求解带负权图的单源最短路路径。**

B 正确，Prim 算法①从结点 1 出发，结点 1 的相关边的权值为 6、7，所以优先选 6，即结点 2 加入。②以 1，2 作为一个整体，找出离这个整体的最短路径连接的未处理的结点，-3 是所有的边中最短的，所以结点 3 加入，至此所有结点都已在生成树里面，B 正确。

C 错误，更新的策略并不一致，这也是 Prim 和 Dijkstra 的区别，设当前选定的结点为 cur，图的边权用 w 数组存储，要更新的未处理的结点为 i，

$$Prim\text{算法：}path[i]=min(path[i],w[i])$$

$$Dijkstra\text{算法：}path[i]=min(path[i],w[i]+path[cur])$$

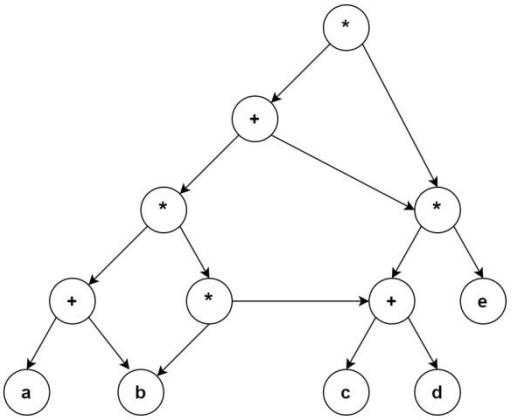
注：Prim 是以整个集合为源，Dijkstra 是以单个结点如本题的结点 0 为源，需要加上 path[cur]来存放 0→cur 的最短路径长度信息。

D 错误，这是最后一轮计算得到矩阵A^2。因为唯一会更新的是 A[1] [2]＝A[1] [3]＋A[3] [2]，以结点 3 为中间结点，那就是在第三轮计算得到的。

通过本题，希望大家去复习和整理最小生成树，最短路径算法的相关考点，对照代码去理解算法的思想，真题考到了不至于整个空着。

42．B。【**解析**】在负权图中，Diikstra 算法既不能保证每次选出的顶点都是真正的最近顶点，又不能保证已确定的最短路径不再被改变，因此 Diikstra 算法不允许边的权为负，I正确。求每对顶点间的最短路径需要调用 Dikstra 算法 n 次，时间复杂度为 $O(n^3)$，II错误。Floyd 算法求每对顶点间的最短路径允许有负边存在，但不允许有负权环，即环路上的边权值之和为负。它是允许有环中某条边的权值为负，III错误。

43．A。【**解析**】：见下图

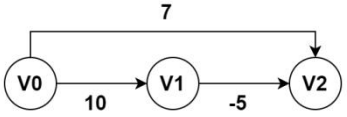


44．A。【解析】符合条件的有：DFS、BFS、拓扑排序。Prim 算法和 Dijsktra 在不使用堆优化下时间复杂度为 $O(v^2)$，优化后是 $O(elog_2v)$。Kruskal 算法主要开销在对边的排序算法中，时间复杂度为 $O(elog_2e)$。

45．D。【解析】A、B、C、D 分别对应A^{-1}、A^2、A^0、A^1。

46．D。【**解析**】D 正确，对于选择题，简单的有向图可以通过手算以快速得到 Floyd 算法的结果，若通过课本上给出的标准解法则比较浪费时间。

47．C。【**解析**】I 错误，Dijkstra 算法不能求带负权图的单源最短路径，反例如下



	V0	V1	V2
final[3]	✓	✓	✓

dist[3]	0	10	7

path[3]	-1	0	0

II、III 正确。

IV 错误，求**每对**顶点的最短路径，应该是$O(V^3)$。

时间复杂度总结如下：

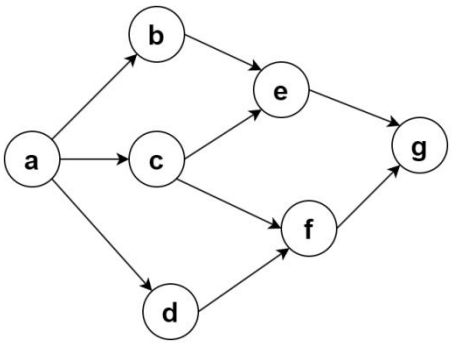
	Dijkstra	Floyd	Prim	Kruskal	DFS	BFS	拓扑排序	关键路径
邻接矩阵	$O(n^2)$	$O(n^3)$	$O(n^2)$	-	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$
邻接表	-	-	-	$O(elog_2e)$	$O(n+e)$	$O(n+e)$	$O(n+e)$	$O(n+e)$

48．C。【**解析**】本图的拓扑排序序列有 ABCFDEG，ABCDFEG，ABCDEFG，ABDCFEG 和 ABDCEFG。

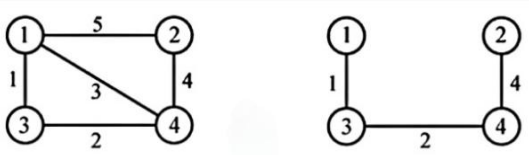
49．A。【**解析**】有向图如图。拓扑排序：①从 AOV 网中选择一个没有前驱(入度为 0)的顶点并输出

② 从网中删除该顶点和所有以它为起点的有向边

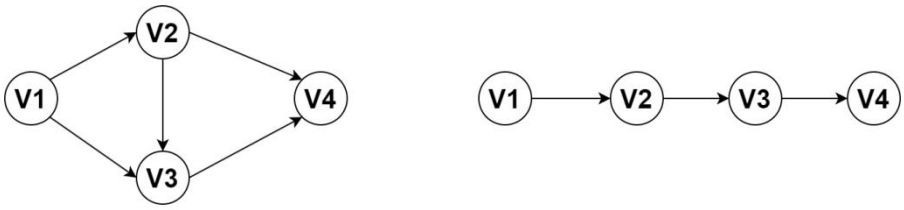
③ 重复①和② 直到当前的 AOV 网为空



50．C。【解析】有向图邻接矩阵的第 V 行中 1 的个数是顶点 V 的出度，而有向图中顶点的度为入度与出度之和，I 错。无向图的邻接矩阵一定是对称矩阵，但当有向图中任意两个顶点之间有边相连，且是两条方向相反的有向边时，有向图的邻接矩阵也是一个对称矩阵，II 错。最小生成树中的 n-1 条边不能保证是图中权值最小的 n-1 条边，因为权值最小的 n-1 条边并不一定能使图连通。在下图中，左图的最小生成树如下图所示，权值为 3 的边不在其最小生成树中。



有向无环图的拓扑序列唯一并不能唯一确定该图。在下图所示的两个有向无环图中，拓扑序列都为 V1, V2, V3, V4，IV 错。



51．A。【解析】比如 1→3→2，邻接矩阵不是上三角也不是下三角矩阵，但是存在拓扑序列{1，3，2}，A 错误。

52．D。【解析】给出有向图 G=（V，E），对于 V 中顶点的线性序列（Vi1，Vi2....，Vin），如果满足如下条件：若在 G 中顶点 Vi 到 Vj 有一条路径，则在序列中顶点 Vi 必在顶点 Vj 之前，则该序列称为 G 的一个拓扑序列。根据定义可知，顶点 Vi 顶点 Vj 之前，并不能说明顶点 Vi 和顶点 Vj 之间邻接关系，但不可能存在一条从 Vj 到 Vi 的路径，因为存在从 Vj 到 Vi 的路径则说明顶点 Vj 必须在顶点 Vi 之前，这与题目产生矛盾。因此答案应选择 D。

53．D。【解析】该图中存在回路，该回路构成一个顶点个数大于 1 的强连通分量。

54．D。【解析】I 正确，DFS 在进行遍历的时候，按照退出递归的顺序输出顶点序号可以得到逆拓扑排序，再逆置就可以得到拓扑排序。II 正确，Dijkstra 是用来求最短路径的，更新的策略和 Prim 不同，不能生成最小生成树。III、IV 正确。IV 中的拓扑序列不唯一，但是一定有且只有一条有序的拓扑序列。相反，能得到有序的拓扑序列的有向图一定是上三角或者下三角矩阵这句话也是对的。

55．C。【解析】对于 A，解决拓扑排序问题用到的是 AOV 网（顶点表示活动的网），而解决关键路径问题用到的是 AOE 网（边表示活动的网）。

对于 B，从源点到汇点的最长路径称作关键路径。

对于 C，在 AOE 网中，如果有环，则可以认为环上的活动都以自身的结束作为自身可能开始的条件，那么将不存在所有活动都完成的状态，也就不存在关键路径。

对于 D，因为关键路径可能是不唯一的，所以某个活动的时间缩短，可能并不能使另一条关键路径的长度也缩短。

56．B。【解析】一个关键活动提前完成，只能让这条关键路径变成非关键路径。当关键路径不止一条的时候，单单提高一条关键路径上关键活动的速度，是不能缩短整个工程工期的。所以，并不是网中任何一个关键活动提前完成，整个工程都能提前完成。但如果网中任何一个关键活动延期完成，整个工程一定延期。

57．D。【解析】可以知道，缩短单条关键路径的长度不能缩短整个工程的关键路径长度，必须同时减小所有关键路径长度才会产生新的关键路径，也就是关键路径长度才会减小。该 AOE 网有三条关键路径，分别是：①$v_1 \to v_2 \to v_5 \to v_7$，对应关键路径上的活动是$a_1a_4a_7$②$v_1 \to v_3 \to v_5 \to v_7$，对应关键路径上的活动是$a_2a_5a_7$③$v_1 \to v_4 \to v_6 \to v_7$，对应关键路径上的活动是$a_3a_6a_8$，可以知道只有一条关键路径①②公共的关键活动$a_7$，要想缩短两个活动能同时缩短三条关键路径，那么必须包含a_7，另外一个从$a_3a_6a_8$中选一个，答案选 D。

58．A。【解析】此题可用关键路径常规的求解方法，一步步计算。$ve(\text{源点}) = 0$，$ve(j) = max_i\{ve(i) + w(i,j)\}$，从前往后计算最早开始事件的值，将其余从源点到$v_j$的所有路径中，路径长度比 ve 小的且终点为$V_j$的边从图中删除和 ve 值相等的路径上的边不用删除），即删除的一定是非关键的边。当$ve(V_j)$求解完毕时，剩余能连通源点和汇点的路径都是关键路径。可得到如下图所示的结果，由此可知，AOE 图的关键路径长度是 16。

59．A。【解析】bdcg, bdeh, bfh 都是关键路径，关键路径长是 27，只有活动 a 不是关键路径上的活动。同学们可手动计算一下 a 的时间余量，最晚开始时间是 9，最早开始时间是 0，所以时间余量是 9。

60．A。【解析】增加活动a_3或者增加活动a_7的任意时间，工程的工期一定延长，说明a_3和a_7一定是关键路径上的活动，通过a_3的路径只有一条，即$v_1 \to v_4 \to v_6 \to v_7$，对应关键路径上的活动是$a_3a_6a_8$，关键路径长度=6+2+1=9。而通过$a_7$的路径有两条一条是$v_1 \to v_3 \to v_5 \to v_7$，$v_1 \to v_2 \to v_5 \to v_7$，但前者路径长度为 1+4+3=8 不是关键路径。所以后者必然为关键路径，而后者对应关键路径上的活动是$a_1a_4a_7$，所以a_1+3+3=9，a_1=3，答案选 A。

61．C。【解析】BFS 适合求无权图或者等权图的最短路径，其核心思想是看结点在 BFS 遍历的第几层最短路径长度就是多少。第一层有结点 1,6，第二层有结点 3,5,7。所以路径长度为 2 的有 3,5,7 共 3 个顶点。

62．D。【解析】第一轮前，dist 数组为{8,8,8}，三个顶点都可以选取：①当第一轮选取 V1，dist 变为{8,8,5}；②当第一轮选取 V2，dist 变为{8,8,8}；③当第一轮选取 V4，dist 变为{5,8,8}；所以只有 D

是正确的。

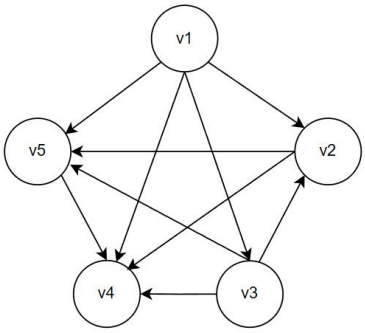
63．C。【解析】Dijkstr 算法中，dist 数组的更新原理是：每次选出当前未确定顶点中 dist 最小的顶点作为新的确定顶点，并用它到各邻接点的距离尝试“松弛”更新这些邻接点的 dist 值。Dijkstra 算法的 dist 更新公式为：

$$dist[j] = \min(dist[j], dist[k] + w(k,j))$$

其中：k：本轮选出的当前 dist 最小的未确定顶点；j：k 的邻接顶点；w(k, j)：边 <k, j> 的权值；若 dist[k] + w(k, j) < dist[j]，则更新 dist[j]。过程如下表所示。

步骤	已确定最短路径的顶点	本轮选出的顶点	更新依据	dist，对应顶点顺序为(2, 3, 4, 5)
初始	{1}	—	从 1 出发初始化	(26, 3, ∞, 6)
第 1 条最短路径	{1, 3}	3	1→3=3，3 用 3→2=22 更新：3+22=25<26	(25, 3, ∞, 6)
第 2 条最短路径	{1, 3, 5}	5	1→5=6，用 5→2=15：6+15=21<25；用 5→4=8：6+8=14	(21, 3, 14, 6)

64．A。【解析】首先题目是反邻接表，即边表都是入边，有向图如下左图所示。逆拓扑排序是从出度为 0 的顶点开始，每轮输出一个出度为 0 的顶点，然后消除与之相关的入边，反复操作直到所有顶点全部输出。（若不能输出所有顶点即表示该有向图存在环）逆拓扑排序过程如下右图所示。



步骤	当前顶点	输出顶点	更新说明
初始	v_4	v_4	$v_1:4\to3$, $v_2:2\to1$, $v_3:3\to2$, $v_5:1\to0$
第 2 步	v_5	v_5	$v_1:3\to2$, $v_2:1\to0$, $v_3:2\to1$
第 3 步	v_2	v_2	$v_1:2\to1$, $v_3:1\to0$
第 4 步	v_3	v_3	$v_1:1\to0$
第 5 步	v_1	v_1	结束

65．D。【解析】A 错误，关键路径有两条：$(v_0, v_1, v_4, v_6, v_8)$或$(v_0, v_1, v_4, v_7, v_8)$，长度均为 18，即关键路径长度位 18。

B 错误，活动a_6的最早开始时间和最晚开始时间是 5、8，若在 t=9 才开始，工程一定会延期。

C 错误，活动a_5的最早开始时间和最晚开始时间是 4、6，时间余量是 6-4=2。

D 正确，事件v_4的最晚发生时间是关键路径上的事件，最晚发生时间是 关键路径长度-11=18-11=7。

第 7 章

查找

【答案速对】

ACDAB　ACCCD　CBBAA　DABBB

CCCDB　DDBCD　ABBDC　DDDCA

BCCCA　(CB)BABC　ACBAC　BACCD

DCCAD

1．A。【解析】采用顺序存储结构，数据元素按其查找概率降序排列可得到最小的查找成功查找长度。

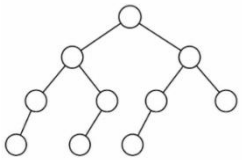
查找成功时的平均查找长度=0.35x1+0.35x2+0.15x3 +0.15x4=2.1

2．C。【解析】当折半查找判定树是一棵满二叉树，那么失败平均查找长度等于树高，C 错误。在线性查找中，每轮只需要 1 次判断操作；而在二分查找中，需要 1 次加法、1 次除法、1～3 次判断操作、1 次加法（减法），共 4～6 个单元操作；因此，当数据量 n 较小时，线性查找反而比二分查找更快。

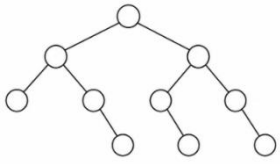
3．D。【解析】在由 n 个元素组成的有序表上进行折半查找时，对任一个元素进行查找的长度都不会大于$log_2 n + 1$。

4．A。【解析】450 比 200 大，说明后续查找走的是 200 的右边区间，该区间内的所有数都大于 200，因此后续所有查找的关键字都应该比 200 大，但是 A 选项后续出现 180<200，不符合折半查找的规则，因此答案选 A。

5．B。【解析】此二分查找判定树两种情况如下图所示：



每个分支结点要么左右子树结点总数相等，要么左子树比右子树结点总数多一个



每个分支结点要么左右子树结点总数相等，要么左子树比右子树结点总数少一个

I 错误，没考虑左右子树结点总数相等的情况。

II 正确，如上图所示。

III 错误，IV 正确，此树是一棵 BST，也是一棵 AVL，但不一定是完全二叉树，因为最后一排叶子不一定连续排列。

6．A。【解析】一共 10 个元素，在选取 mid 元素时，可以取第 5 个（编号 1~10），也可以取第 6 个，一旦确定左右谁多谁少的规则，后续都要满足这种规则。A 选项中，第一次选 20，即左边比右边少一个元素。58>20，应当继续查找右边区间。(30, 50, 70, 88, 100)中，mid 元素是 70，58<70，所以走左边。[30, 50]，mid 元素取 30，58>30，走右边。(50)最后 58>50，到失败结点，查找失败。查找的元素依次是 20、70、30、50，A 正确。

若是 B 选项，确定 30 为第一个 mid 元素，也就确定了划分时，左边比右边多一个的规则。

58>30，走右边区间(50, 70, 88, 100)，继续遵循左边比右边多一个的规则，mid 元素应该是 88。

58<88，走左边区间(50, 70)，mid 元素应该是 70，58<70，最后走(50)，所以查找依次是 30、88、70、50。

7．C【解析】$511 = 2^9 - 1$，所以折半查找判定树是一棵高度为 9 的满二叉树，所有的失败结点都在第 10 层，$ASL_{失败} = \frac{9*512}{512} = 9$。A 选项是查找成功的平均查找长度，$ASL_{成功} = \frac{1*1+2*2^1+3*2^2+\ldots+9*2^8}{511} = \frac{4097}{511}$（差比数列求和）

8．C。【解析】有 n 个元素的有序顺序表，二分查找元素的次数上界为$\lfloor log_2 n \rfloor + 1$ 次，代入 n=100 即可得到答案为 7。

9．C。【解析】查找一次成功的节点数为 1，下标为 15（假设偶数个结点分两个区间时，左边比右边少一个）

查找二次成功的节点数为 2，下标 7，23

查找三次成功的节点数为 4，下标 3，11，19，27

查找四次成功的节点数为 8，下标 1，5，9，13，17，21，25，29

查找五次成功的节点数为 15，下标 2，4，6，8，10，12，14，16，18，20，22，24，26，28，30

10．D。【解析】二叉查找树在最坏情况下是 O（n）。

11．C。【解析】每块长度为 8，则共有 152 / 8=19 块，即索引表项为 19，那么顺序查找索引表时的平均查找长度为（19+1）/ 2=10，而在一个块内的平均查找长度为（8+1）/ 2=4.5，因此综合上面两项为 C 答案。

12．B。【解析】分块查找是将数据分为若干块，不要求索引块内数据个数相等，且块内数据不必有序，块间有序，每块内最大(或最小)的数据组成索引块，故本题选 B。

13．B。【解析】$ASL_{成功} = ASL_{二分查找查找成功} + ASL_{顺序查找查找成功} = \lceil \frac{1*1+2*2^1+3*2^2+4*2^3+5*2^4}{31} + \frac{31+1}{2} \rceil = \lceil \frac{129}{31} + 16 \rceil = 21$

14．A。【解析】设块长为 b，索引表包含 n/b 项，索引表的 ASL=(n/b+1)/2，块内的 ASL=(b+1)/2，$ASL_{总} = ASL_{索引表} + ASL_{块内} = (b + n/b + 2)/2$，其中对于 b+ n/b，由均值不等式知 b=n/b 时有最小值，此时。则最理想块长为 2500=50。

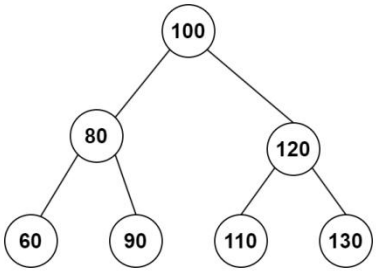
15．A。【解析】对于 A，因为二叉排序树的中序遍历序列一定是关键字的有序序列，所以可以将其先序遍历序列排序从而得到中序序列。而后由先序序列和中序序列来确定该二叉排序树。

对于 B，一般意义上，二叉排序树优于顺序表的平均查找时间，但是若这组记录构造出的二叉排序树恰巧是单枝状时，那么在其上的平均查找时间和顺序表是一样的，都是（n+1）/2，其中 n 是表中的结点数。

对于 C，在二叉排序树上的一个结点被成功查找时，关键字比较次数就是它所在的层数，由于是等概率查找情况，要使所有结点总的比较次数最小，则平均查找长度也必为最佳，显然二叉排序树的树高最小可以满足要求。

对于 D，在二叉排序树中，删除哪一个结点是可以按情况任意指定的，可以是根，可以是非叶结点，也可以是叶子结点；但是二叉排序树在插入结点时一定是发生在分支末端，即新添加一个叶子结点，所以删插结点前后的二叉排序树不能保证是一样的。另外，若给定一组记录，而给出的记录顺序不同，生成的二叉排序树也不一定相同，因为生成的过程是从空二叉排序树开始不断插入记录结点，其中的第一个记录生成根结点。

16．D。【解析】其余三个选项为



17．A。【解析】只需依次判断后序结点是否全比当前结点大或者小，显然只有 A 符合。

18．B。【解析】依次插入将带插入值直接放入二叉搜索树符合条件位置，使得整棵树左子树均比根小，右子树均比根大，查找 62 需要比较 3 次，分别是和 54、73、62 比较。

19．B。【解析】不妨假设非空的高度为 h 的平衡二叉树的结点数至少为 f（h）。它是由 1 个根结点及其左右子树构成的，那么左右子树可以分别是树高为 h-1 和 h-2 的具有最少结点数的平衡二叉树。由递推公式 F(n)=F(n-1) + F(n-2) + 1;F(1) = 1, F(2) = 2，可递推 1, 2, 4, 7, 12, 20, 33, 54。

20．B。【解析】设 N_h 表示高度为 h 的平衡二叉树中含有的最少结点数。由递推式$N_h = N_{h-1} + N_{h-2} + 1$ 可推：$N_0 = 0, N_1 = 1, N_2 = 2, ..., N_3 = 4, N_4 = 7, N_5 = 12, N_6 = 20 > 18$

所以 18 个结点的最大高度为 5，答案选 B。

21．C。【解析】设 N_h 表示高度为 h 的平衡二叉树中含有的最少结点数。由递推式$N_h = N_{h-1} + N_{h-2} + 1$ *可推：*$N_0 = 0, N_1 = 1, N_2 = 2, ..., N_3 = 4, N_4 = 7, N_5 = 12, N_6 = 20 > 15$。（推导：套娃！）也就是说，高度为 6 的平衡二叉树最少有 20 个结点，因此 15 个结点的平衡二叉树的高度最大为 5，即叶子结点的最大高度为 5。

最小叶子结点的层高为 3，因为高度为 5 的平衡二叉树，根据上述递归式$N_h = N_{h-1} + N_{h-2} + 1$，根结点一侧树高为 4 的结点数最少平衡二叉树，一侧为树高为 3 的结点数最少平衡二叉树，高度为 3 的结点数最少平衡二叉树中，叶子结点最低为 2，所以在原来 h=5 的平衡二叉树中，叶子节点最低在第 3 层。

所以叶子结点所在层高 h∈[3，5]，A、D 选项查找高度为 2，6，故错误。B 选项的查找过程不能构

成二叉排序树，故错误。正确答案选 C。

22．C。【解析】插入结点导致不平衡，一定是某棵子树高度增加，导致比另外一边的子树高度相差 2，导致出现不平衡结点（平衡因子绝对值>1，即 2 或-2）。接下来应该判断是 2 还是-2，经过以上分析，插入之后势必所在子树的平衡因子要么是 1，要么是-1（如果插入后平衡因子还是 0，高度一定不会增加）。满足条件的是右孩子，所以插入元素的位置一定是右孩子的左子树。所以应该进行 RL 调整。

23．C。【解析】每个分支结点平衡因子为 1（或全为-1）时，高度最大，此时 h 层所需最少结点数应按递推式 H(n) = H(n - 1) + H(n - 2) + 1 计算，各层的最少结点数分别为 1, 2, 4, 7 , 12, 20…，所以在这种情况下，12 个结点刚好在第 5 层，即 AVL 的最大高度为 5。树中，由度数+1=结点总数可以推出，$2n_2+n_1+1=n_2+n_1+n_0$，即$n_2+1=n_0 \rightarrow 2n_0-1+n_1=12$，完全二叉树满足度为 1 的结点数最少，此时度为 1 的结点数为 1，得叶子结点数为 6，选 C。

24．D。【解析】按高度依次来分析不同平衡二叉树的情况：

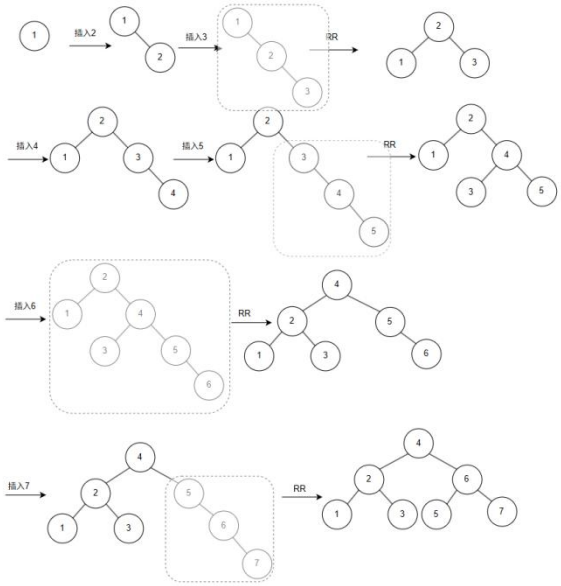
h=1，只有一个根节点，所以只有 1 种情况。

h=2，一共 3 种情况，①根节点+左孩子②根节点+右孩子③根节点+左右孩子

h=3，根节点的左右子树高度之差不超过 1，有三种情况：①左子树 h=2，右子树 h=1，此时左子树 h=2 的平衡二叉树有 3 种，h=1 只有一种，所以是 3*1=3；②左子树 h=1，右子树 h=2，同样也是 1*3=3 种；③左子树 h=2，右子树 h=2，一共是 3*3=9 种；综合以上分析，h=3 的平衡二叉树的形态有 3+3+9=15 种。

25．B。【解析】首先根据高度为 h 的平衡二叉树最少结点数公式：$N_h=N_{h-1}+N_{h-2}+1$，可以得到$N_1=1, N_2=2, N_3=4, N_4=7$；所以 5 个结点的平衡二叉树高度最大为 3。而高度为 2 的满二叉树最多可容纳 3 个结点，所以 5 个结点的平衡二叉树高度一定是 3，且第二层是排满的 2 个结点，否则不平衡，第三层也就剩下 5-3=2 个结点。即从第三层的 4 个结点任意选出 2 个，都满足平衡二叉树，答案为 $C_4^2=6$ 种。在考场，第一时间也可以枚举，枚举的过程应该顺着上面的思路来确定情况。

26．D。【解析】插入过程如下图所示，平衡因子为 0 的分支结点个数为 3 个。



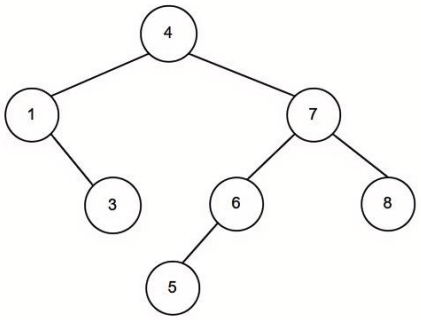
27．D。【解析】看看进行操作前是否符合 AVL。

I、IV 正确，显然删除 2 和插入 5 之前是符合 AVL 树的。

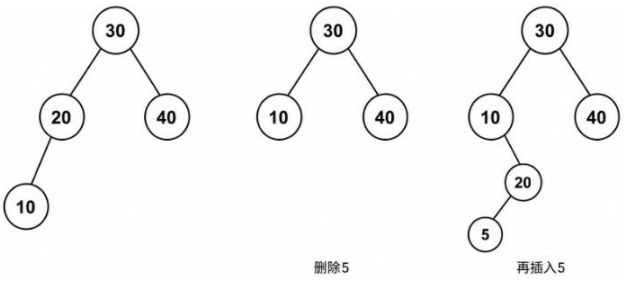
II 错误，BST 的 insert 操作一定是在叶子结点，不可能插入到根结点。

III 正确，删除 4 的原树如右图所示，BST 删除后用前驱 3 作为新的根节点。

V 错误，插入 8 之前不符合 AVL。



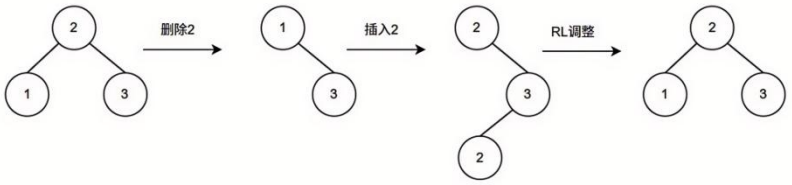
28．B。【解析】A 选项，若删除的是分支结点，则再插入后，二叉树一定改变。例子如下图：



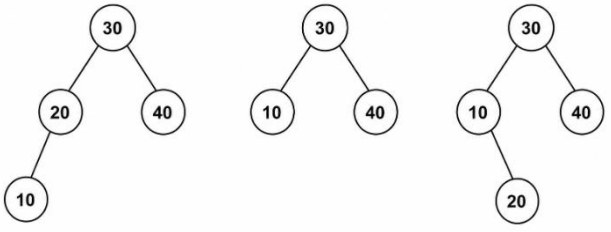
B 选项正确，插入的结点一定在叶子结点，再删除恢复到原状。

C、D 选项错误，调整后可能会导致变化，也可能不变；

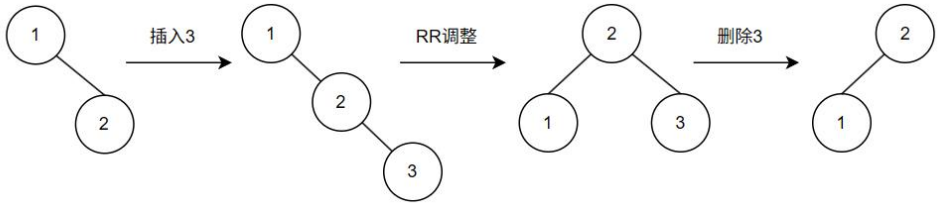
C 选项不变化的图例：（同样，在第二步只有 1，3 结点的情况中，插入 2，再删除 2，D 选项可能无变化）



C 选项变化的图例：删除 20，再插入 20，发生改变。



D 选项发生变化的图例：



29．C。【解析】

I 错误，子树结点若是红色，则不符合红黑树根结点为黑色的要求。

II 正确，最多的情况是黑色和红色交替出现的满二叉树。

III 错误，比如三个结点，根为黑色，两个孩子为红色的树就是反例。

IV 正确，任何一个结点的黑高确定，则两种极端是都是黑色和黑红交替，长的一侧不会超过短的一侧的两倍。

V 正确，对比如下图所示。

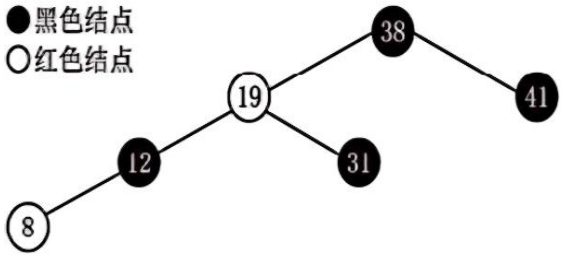
最坏时间复杂度	BST	AVL 树	红黑树
Search(查)	$O(n)$	$O(log_2 n)$	$O(log_2 n)$
Insert（插）	$O(n)$	$O(log_2 n)$	$O(log_2 n)$
Delete（删）	$O(n)$	$O(log_2 n)$	$O(log_2 n)$

30．D。【解析】I 和II都是红黑树的性质。AVL 是高度平衡的二叉查找树，红黑树是适度平衡的二叉查找树，从这一点也可以看出 AVL 的查询效率往往更优，III 错误。AVL 的某结点的左右孩子的平衡因子都是 0，并不能说明左右子树的高度相等，因此该结点的平衡因子不一定为零，IV 错误。

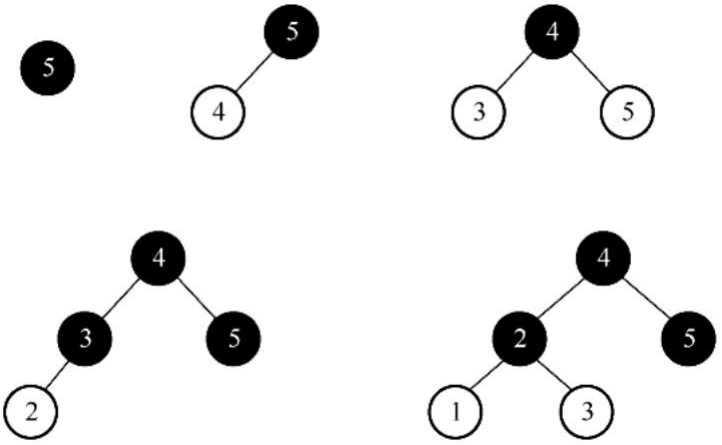
31．A。【解析】红黑树是一种特殊的二叉排序树，选项 B 不满足二叉排序树的性质。选项 C 中，结点 2 的左右黑结点数不同。在选项 D 中，结点 3 的左右黑结点数不同。只有选项 A 满足红黑树的定义。

32．B。【解析】插入结点 2 旦将其染成红色后违反不红红原则，并且叔结点是黑色，应进行 L 旋转，将结点 3 右旋旋转到结点 5 的位置，结点 2 和结点 5 分别成为结点 3 的左、右孩子，然后将结点 3 染成黑色，结点 2 和结点 5 染成红色。因此，下一步应进行右旋操作。

33．B。【解析】最后的红黑树的形态如下图所示，注意：黑高是指除了自身之外到叶子结点（失败结点）的黑色结点数，B 正确。



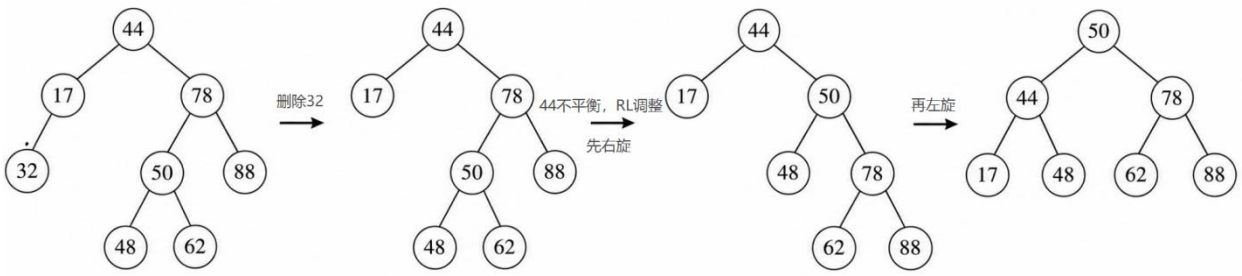
34．D。【解析】关键字 5, 4.3.2, 1 依次插入红黑树后的形态变化如下：



35．C。【解析】AVL 的删除步骤如下：

- 用 BST 的删除结点方法删除
  - 当待删除节点的度为 0 时，表示待删除节点是叶节点，可以直接删除。
  - 当待删除节点的度为 1 即只有左子树或右子树时，将待删除节点替换为其子节点即可。
  - 当待删除节点的度为 2 即左右子树都存在时，我们无法直接删除它，而需要使用一个节点替换该节点。由于要保持二叉搜索树“左 < 根 < 右”的性质，因此这个节点可以是**右子树的最小节点（在右子树的最左下位置）或左子树的最大节点(左子树的最右下位置)。最终转换成对叶子节点的删除。**
- 从删除结点一直往上回溯，寻找第一个失衡结点，找不到说明此时 AVL 还是平衡的，到这里就结束了；否则标记它为 node，其高度最高的儿子结点为 child，高度最高的孙子结点为 grandchild；(只有调整长得高的那部分子树才能重新恢复平衡)
- 对以 node 的最小不平衡子树进行调整，先观察 node、child 哪一种；然后执行相应的旋转操作，将以 node 为根的子树调整平衡；grandchild 的关系，判断是执行(LL/RR/LR/RL)中的
- 如果调整后由于子树高度-1 导致不平衡向上传导，则从第 2 步继续执行调整操作，直至该树平衡；

本题的删除过程如下，进行 RL 调整。

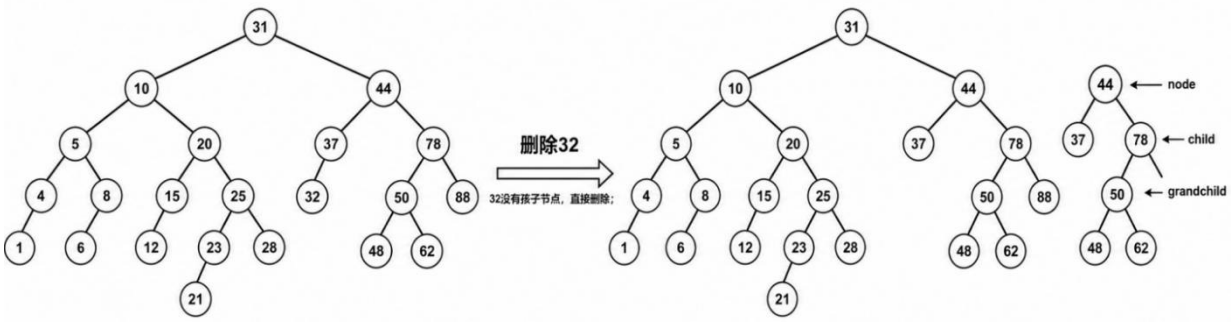


36．D。【解析】删除的规则如下：

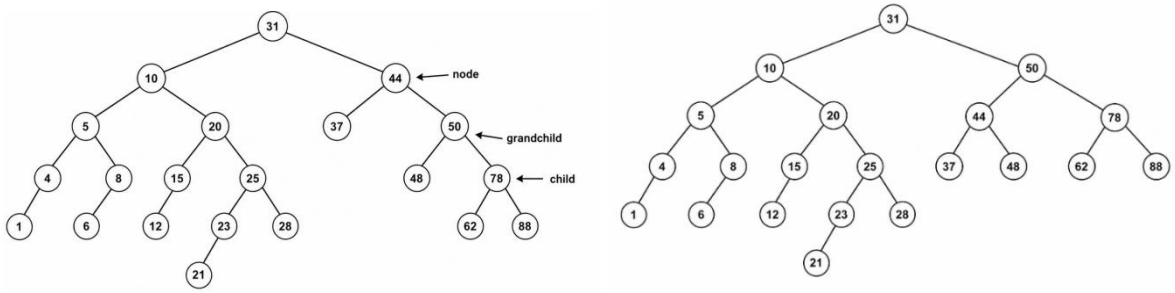
- 若删除的是非终端结点，则找到其前驱/后继关键字补到当前结点中，最终转换成删除终端结点

- 删除终端节点，自底向上找到第一个失衡的结点，接着看这个失衡结点的子树中，到所有叶子节点路径最长的那一条，确定这个路径的前两个结点是 LL、RR、LR、RL 中的哪一种。比如这一题，删除 32 后，第一个失衡的结点时 44。44 子树中到叶子结点的最长路径经过的孩子孙子是 78（R），60（L），所以是 RL 调整，旋转两次。
- 第一次调整完后，继续往上找失衡的结点，进行调整，直到全部平衡。可以发现第二次失衡的是根结点 33，进行 LR 调整，旋转两次。

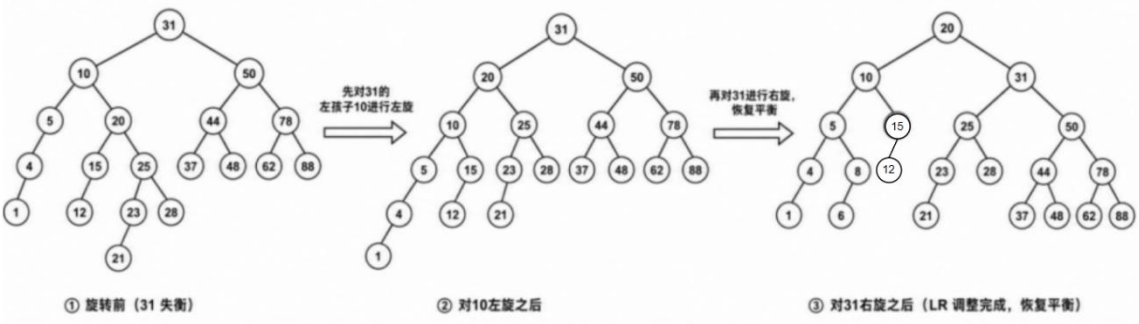
综上所述，两次调整 RL 和 LR，旋转次数是 4 次。



判断出 44 是失衡的结点，找到距离 44 最远的叶子结点(48/62)，可以确定是 RL 调整，先右旋再左旋：



调整完后，继续往上进行观察，31 失去平衡，找到最远的叶子节点是 21，确定是 LR 调整：



37．D。【**解析**】A 正确，当关键字最多的情况时，B 树是满 m 叉树，每个结点都有 m - 1 个关键字。C 正确，除根结点外所有非失败结点至少 $\lceil m/2 \rceil - 1$ 个关键字，根结点至少有 1 个关键字。D 错误，和二叉搜索树不同，二叉搜索树高度的增加发生在底部，而 B 树的高度增加发生在顶部，只有根节点分裂才会导致高度加 1。

38．D。【**解析**】AVL 和二叉查找树很类似，从根结点到叶子结点不一定有序，红黑树是一种二叉查找树，同理该路径不一定有序；堆符合这样的定义，应选 D。

39．C。【**解析**】本题考查 B 树的高度，磁盘存取次数取决于 B 树的高度。对有 n 个关键字的 m

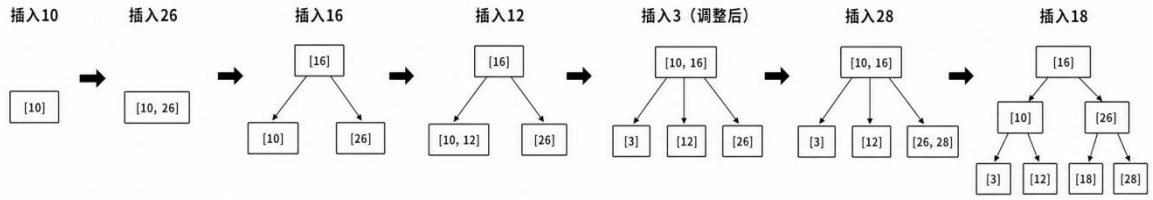
阶 B 树，让每个结点中的关键字个数达到最少，则容纳同样多关键字的 B 树的高度达到最大。也就是说，第一层至少有 1 个结点；第二层至少有 2 个结点；除根结点外的每个非终端结点至少有$\lceil m/2 \rceil$ 棵子树，则第三层至少有 $2\lceil m/2 \rceil$个结点……第 h+1 层至少有$2(\lceil m/2 \rceil)^{h-1}$ 个结点，注意，第 h+1 层是不包含任何信息的叶结点。对关键字个数为 n 的 B 树，叶结点即查找不成功的结点为 n+1，由此有 $n+1 \geqslant 2(\lceil m/2 \rceil)^{h-1}, \leqslant \log_{\lceil m/2 \rceil}((n+1)/2)+1$。

40．A。【**解析**】II 错误，B+ 树支持顺序查找，因为它的叶结点通过指针串联起来，而 B 树没有。III 错误，(B+) 树每次都需查询到叶结点，B 树查找到元素即可，不一定需要查找到叶结点处。

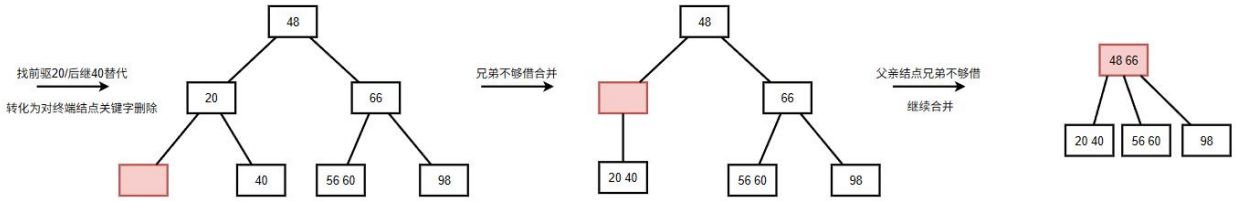
41．B。【**解析**】插入规则：

- 找到当前关键字的底层结点插入位置
- 判断插入后是否超过单个结点的关键字上限（阶数-1）
  - 不超过，插入成功，结束。
  - 超过了则进行分裂，取中间结点放入父亲结点，若父亲结点关键字也溢出，则继续分裂。若到根节点还溢出，则生成新的根节点，B 树高度+1。

本题插入过程如下图所示：



42．C。【**解析**】首先删除非终端结点关键字要通过前驱或者后继替换直到转化为对终端结点关键字的删除，这里选前驱 20 代替（也可以选后继 40 替代，结果相同），接着终端结点删除关键字后为空，要跟兄弟结点借，兄弟只有关键字 40 不够借，所以合并（见中间图）。合并后父亲结点关键字要放下去一个，导致父亲结点关键字低于关键字下限 1 且他自己的兄弟只有 56 也不够借，继续合并，得到最右边的图，即最终结果。

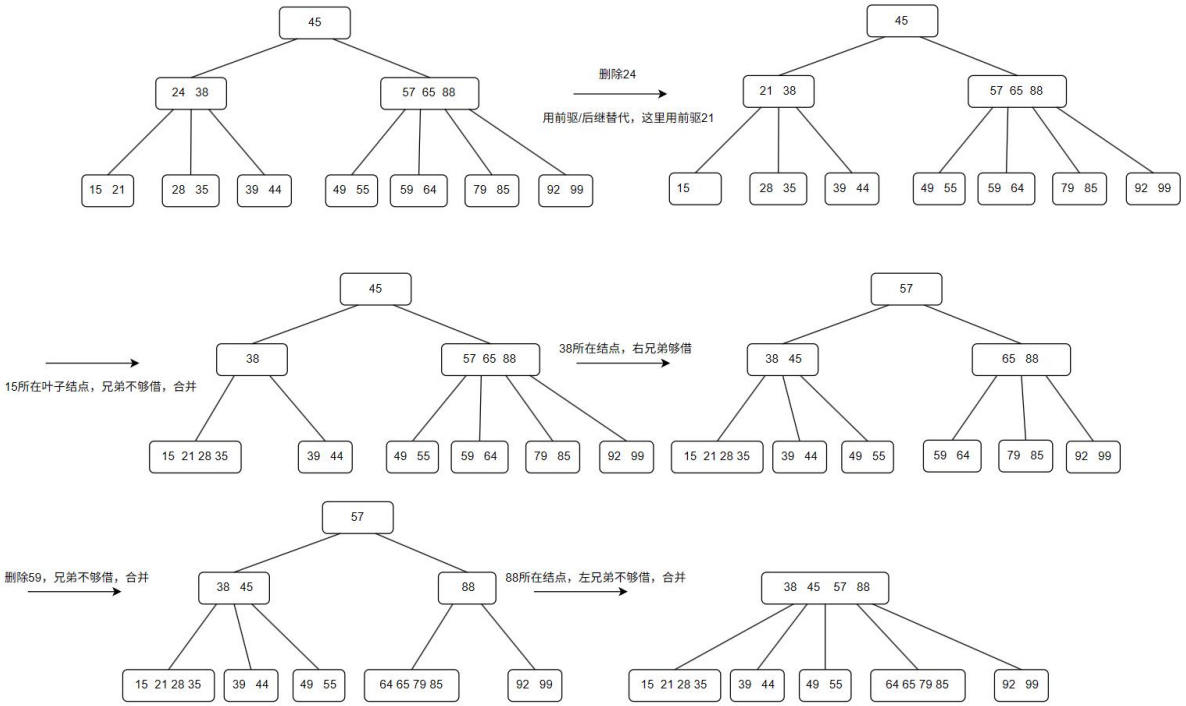


43．C。【**解析**】5 阶 B 树，关键字的范围为[2, 4]，B 树删除的规则：

- 若删除的是分支结点，则找到分支结点关键词的前驱或者后继关键词，转换成对终端节点的删除。
- 删除终端结点的关键字
  - 若删除后结点关键字没有下溢（不少于最小关键词个数），则直接结束。

- 若删除后下溢，则需要向兄弟结点借，左右兄弟皆可。
- 兄弟借了后没有下溢（兄弟够借），则直接借，兄弟借出的关键字放到父结点，父结点中的下放到删除关键字的位置。
- 兄弟借了后下溢（兄弟不够借），则和兄弟合并，合并需要把父节点中的关键字和本身、兄弟的关键字合并到一个结点。若合并后，父亲结点也下溢，则继续找父亲结点的兄弟借，按上述步骤套娃，直到满足条件，或者直到根节点也被合并（B 树高度-1）。

具体的删除过程如下图所示。



44．C。【**解析**】A 错误，若根节点不为叶子结点，则根节点的范围下限是 2。B 错误，B+树中删除仅在叶子结点进行，非叶子结点的关键字不删除，作为后续查找的“分解关键字”，所以非叶子结点关键字个数总和和叶子节点关键字总和不一定相等。

C 正确，B+树叶子结点之间通过指针链接成一个链表，且所有关键字均出现在叶子节点，所以支持顺序查找。而且更适合查找某个范围内的所有关键字。例如，在 B+树上找出值在[a, b]内的所有关键字。处理方法如下：通过一次查找找出关键字 a，不管它是否存在，都可以到达可能出现 a 的叶子节点，然后在叶子节点中查找值等于 a（如果有）或大于 a 的那些关键字，对于所找到的每个关键字都有一个指针指向相应的记录，这些记录的关键字在所需要的范围。如果在当前节点中没有发现大于 b 的关键字，就可以使用当前叶子节点的指向下一个结点的指针找到下一个叶子结点，并继续进行同样的处理，直至在某个叶子节点中找到大于 b 的关键字，才停止查找。

D 错误，B+树一定要查找到最后一层，因为中间结点不保存关键字代表记录的指针，只有叶子结点才有。

45．A。【**解析**】B+树能够有效地支持顺序查找，因为它的叶子结点之间具有指针链接，可以顺序访问所有关键字。然而，B 树不能像 B+树那样高效地支持顺序查找，因为关键字分布在整个树中，没

有指针链接叶子结点，因此 A 选项错误。

在 B 树中进行随机查找时，从根结点开始，根据查找的关键字，按照结点中关键字的大小关系，选择合适的子结点进行查找，直到找到对应的关键字或者查找失败。B+树在随机查找时的过程与 B 树类似，从根结点开始，根据关键字的大小关系选择合适的子结点进行查找，直到找到对应的叶子结点。由于叶子结点形成了有序链表，可以更快地进行范围查询和顺序遍历操作。B 树和 B+树都能有效地支持随机查找。由于它们都是平衡的多叉树，它们的查找性能在最坏情况下为 O(logn)，因此 B 选项正确。

B 树和 B+树都是平衡的多叉树，它们在插入和删除操作时会自动调整树的结构，确保树保持平衡，因此 C 选项正确。

B 树和 B+树都可以用作文件索引结构，它们可以有效地支持随机查找和范围查询，因此在数据库和文件系统中经常被用作索引结构，因此 D 选项正确。

46．C，B。**【解析】**5 阶 B 树中共有 53 个关键字，由最大高度公式 $H\leqslant\log_{\lceil m/2\rceil}((n+1)/2)+1=4$ 得最大高度 4；由最小高度公式 $h\geqslant\log_m(n+1)\approx 2.5$ 得最小高度从而最小高度为 3。

47．B。**【解析】**搜索第 2025 个关键字并不代表只有 2025 个关键字，所以需要每层放尽可能少的关键字，第一层 1，第二层 2 * 3，第三层 2 * 4 * 3，第四层 2 * 4^2 * 3，第五层 2 * 4^3 * 3，...，以此累加，到第 5 层最少需要 511 个关键字，到第 6 层最少需要 2047 个关键字，因此第 2025 个关键字最高在第 6 层，根节点在内存，所以最多启动 5 次，答案选 B。

提问：若问 7 阶 B 树只有 2025 个关键字，根节点已经读入内存，读取任意关键字至多需要多少次 I/O？

A。**【解析】**跟原题不同的是，本题确定了总的结点数，那么 2025 个关键字 B 树最高层高，根据 B 树树高 h 的计算公式：$\log_m(n+1)\leqslant h\leqslant\log_{\lceil m/2\rceil}((n+1)/2)+1$，计算得 h=5(取整数)，因为 B 树的根结点已读入内存，所以最多需再启动 4 次磁盘 I/O。

48．A。**【解析】** n 个关键字的 B 树必有 n+1 个叶子结点（n 个关键字将数域切分为 n+1 个区间），注意最多是迷惑条件。

49．B。**【解析】**在除留余数法中，p 应取小于存储区容量的质数，因此答案为 B。

为什么要选质数作为模长？比如，若 p = 12，键值为 24, 36, 48 都会被映射到相同的桶（0），因为它们都被 12 整除。如果用质数作为模数，例如 11，这些值会均匀地落在不同位置。

为什么模长要小于或等于 m？散列表的长度 m 是桶的数量，最终散列值必须映射到 [0, m-1] 之间：

- 如果你用 `p > m`，那么得到的哈希值还需要再次取模 `m`，多此一举。
- 如果你直接用 `p ≤ m` 的最大质数作为模长，就能直接映射到有效位置，效率更高。

50．C。**【解析】**同义词指的是在关键字不同，通过哈希函数映射到同一个值。而线性探测法处理冲突时，会把冲突的关键字按顺序往后放到空闲位置，可能导致不同同义词交叉排放，所以查找路径上的关键字可能是同义词。

举例，比如哈希函数取 key % 7，依次插入 1，9，8 三个元素，8 放在下标为 3 的位置，查找 8 时，路径上依次是 1，9，8。只有 1，8(% 7 = 1)是同义词，9 不是同义词。因此答案选 C。

51．A。【解析】对于 A，不存在特别好与坏的哈希函数，哈希函数计算不能过于复杂，同时还要兼顾减少冲突，因此选取怎样的哈希函数和哈希表中的记录关键字的特点相关。

对于 B，聚集现象是非同义词发生的地址冲突，在链地址法的哈希表中，只有同义词才会发生地址冲突。

对于 C，理想状态哈希表的查找应该不需要关键字之间的比较，而是由关键字通过哈希函数定址即可，但是由于冲突的不可避免性，在查找中也就需要通过比较来甄别冲突的情况。

对于 D，a<1 只能说明哈希表的长度大于存储的记录个数，但存储的记录完全有可能发生冲突，冲突与否取决于两个关键字经过哈希函数运算后的值（即哈希地址），是否相同。

52．C。【解析】表中记录个数为 n=5 时，装填因子为 a=0.33，则哈希表长为$m = n / a = 5 / 0.33 \approx 15$，在放入关键字为 49 的记录之前哈希表 HT［0..14］，发生冲突，然后按照$d + 1^2, d - 1^2, d + 2^2, d - 2^2$..进行探查。关键字 49 的哈希地址为 49 MOD 11=5，第一次探测与 60 冲突。根据二次探测再散列的方法，第 2 次探测地址为 $5 + 1^2 = 6$，又与 83 冲突；第 3 次探测地址为 $5 - 1^2 = 4$，又与 37 冲突；第 4 次探测地址为 $5 + 2^2 = 9$，没有冲突发生，因此它的存放位置是 9。

53．B。【解析】依次取余后序列为{2, 8, 0, 8, 8, 10, 8, 6, 10, 5, 2, 1}，H（key）=0 ~ 10，查找失败对应的地址有 11 个，每次查找对应链的时候，查到空结点为止，故 0~10 的查找失败次数和为 2+2+3+1+1+2+2+1+5+1+3=23，故答案选 B。

注意：有的书上空指针不算一次比较，这个也是合理的，应当在算/不算都计算一遍结果，如果发现有答案和自己结果对上，就果断选，一般的题目答案中不会同时包含两种方式的结果的。

54．A。【解析】34%7=6，69%7=6，9%7=2，56%7=0，24%7=3，44%7=2。初始如下：

删除 24，再插入 51（%7=2），发生冲突后，插入下标为 3 的位置。$ASL_{失败} = \frac{2+1+4+3+2+1+3}{7} = \frac{16}{7}$

55．C。【解析】再散列在处理冲突时依然会产生“堆积”现象，A 错误。在哈希表中进行查找依然需要进行关键字比较，B 错误。采用线性探测法处理冲突时，所有同义词在散列表中不一定相邻，D 错误。

56．B。【解析】拉链法的节点空间动态申请更适合无法确定表长的情况。

57．A。【解析】A 错误，链地址法处理冲突，需要遍历链表，查找的时间不一定相同。

58．C。【解析】线性探测是解决冲突的策略。第一个关键字最好情况是没有发生冲突，至少探测 0 次，第 2 个关键字至少要探测 1 次，第 3 个关键字至少要探测 2 次，第 4 个关键字至少要探测 3 次，……，第 k 个关键字至少要探测 k-1 次，至少共需进行 k(k - 1) / 2 次探测。

59．C。【解析】顺序查找和折半查找都不适合动态变化。哈希查找插入的时间复杂度取决于冲突处理函数，最坏是 O(n)。

分块查找的优点是：在表中插入和删除数据元素时，只要找到该元素对应的块，就可以在该块内进行插入和删除运算。由于块内是无序的，故插入和删除比较容易，无须进行大量移动。如果线性表既经常动态变化，又需对其进行快速查找，则可采用分块查找。其缺点是：要增加一个索引表的存储空间并对初始索引表进行排序运算。因此此题选 C。

60．D。【解析】散列函数和处理冲突的方法决定了值的映射地址和进行冲突探测后插入的位置，

散列函数的好坏首先影响出现冲突的频繁程度，若是均匀的散列函数（对同一组随机的关键字，产生冲突的可能性相同），则平均查找长度只有处理冲突的方法和装填因子。装填因子$\alpha=\frac{表中填入的记录数}{散列表的长度}$，$\alpha$越小，发生冲突的可能性就越小；反之越大。所以答案选 D。

61．D。【解析】h 层的平衡二叉树，所需最少结点个数满足递推式 $N_h=N_{h-1}+N_{h-2}+1$，$N_1=1,N_2=2$ 计算得 $N_5=12$，I 正确；II 正确，h=5 时，要使得左右子树结点数相差最大，那么左子树高度 h=4，右子树高度 h=3，进一步拉开差距，左子树是 h=4 的满二叉树，共 15 个结点；而右子树是 h=3 的结点数最少的平衡二叉树，用 A 选项的递推式可知，h=3 时，至少需要 4 个结点。所以结点数之差最大为 11。III 正确，所有分支结点平衡因子为 1，即满足 h=5 的结点数最少的平衡二叉树形状，整体左倾，可知左子树是高度为 4 的结点数最少的平衡二叉树形状，根据递推式，结点数为 7。IV 正确，归纳法可知，h=1 或 2，层高差为 0；每增加 2 层，叶子节点层高差距+1，即 $\Delta_{max}=\lfloor\frac{h-1}{2}\rfloor$。h=5 代入得 2。

62．C。【解析】首先从高度为 1 的开始逐层归纳。高度为 1 的 AVL 只有 1 种树形也就一个根结点。高度为 2 的 AVL 有 3 种不同的树形，分别是根结点有左孩子无右孩子、有右孩子无左孩子、左右孩子都有。高度为 3 的 AVL 有 15 种，第一种情况是左子树高度为 2（3 种），右子树高度为 1（1 种），一共有 3*1=3 种，第二种情况左子树高度为 1 右子树高度为 2，一共有 1*3=3 种，第三种情况左右子树高度都为 2，一共有 3*3=9 种，共计 3+3+9=15 种。 高度为 4 时，一样的思路。①左子树高度为 3 右子树高度为 2，共 15*3=45 种②左子树高度为 2 右子树高度为 3，共 3*15=45 种③左、右子树高度均为 3，共 15*15=225 种，一共 315 种。答案选 C。 本题出成 h=4 是让大家不想去用枚举法做题，而是用归纳的思想去总结规律。一般来说考试考 h=3 的概率比较大，一方面可以枚举暴力解题，另一方面也是本题思路，具有区分度。

63．C。【解析】高度为 h 的平衡二叉树所需最少结点满足递推式 $N_h=N_{h-1}+N_{h-2}+1$，其中 $N_1=1,N_2=2$，递推得到 $N_4=7,N_5=12$，当结点数为 10 时，高度最大可达 4。由因为题目说任意一棵 10 个结点的平衡二叉树，那么假设根结点存在顺序表的第 1 个单元，第 4 层上的结点最大可在第 15 的单元处，因此要满足任意性，至少需要 15 个存储单元。

64．A。【解析】4 阶 B 树所有结点关键字范围[1,3]。①根结点关键字为 1 个：第二层有两个结点，要想到达第三层，那么第三层至少 4 个结点，一共 7 个结点，每个结点刚好 1 个关键字，此时第二层只有 2 个关键字，不符合题目要求第二层关键字不小于 3 个。所以最高只有 2 层，且根结点只有 1 个关键字的情况下，第二层每个结点一定都是 3 个关键字。 ②根结点关键字为 2 个：第二层有三个结点，还剩 5 个关键字，那么三个结点的关键字可能是 311 的组合，也可能是 221 的组合，311 组合有三种情况即 311、131、113，同理 221 也有三种情况，一共 3+3=6 种。③根结点关键字为 3 个：第二层有 4 个结点，还剩 4 个关键字，刚好每个结点 1 个关键字，只有 1 种可能。综上所述，一共有 1+6+1=8 种，答案选 A。

65．D。【解析】I 错误，插入一个新关键字，可能导致结点分裂，但不一定会使树高加 1，只有传递到根结点分裂，高度才会加 1，即生成一个新的根结点。II 正确，删除一个关键字导致结点合并不一定使树高减 1，只有合并操作传递到根节点，导致根节点和左右孩子也合并，才会使树高减 1。III 错

误，合并会向上传递，最坏的情况下，每层合并后减少一个孩子，第二层上合并的时候，根节点和左右孩子也合并，导致减少 2 个，所以一共 h-1 次合并，最多减少 h 个结点。IV 正确，每次分裂会把一个结点分裂成两个新结点，每个新结点（关键字发生改变）都要写回磁盘，所以一次分裂写 2 次磁盘。分裂会向上传递，如果一直传递到根结点，根结点分裂，不光自身会一分为二，还会产生一个新的根结点，所以最后一次分裂会发生 3 次写磁盘操作，那么 h 层，最多有 2h+1 次写磁盘操作。综上所述，II、IV 正确，答案选 D。

第 8 章

排序

【答案速对】

DACBB　BDACA　BBCDA　BBDBD

BACBA　DBCCB　ACACC　CDABB

AAADA　DDCBB　DC(DC)C(DA)　CCDAB

BDBAD　DADBB　CBCBB　CCC

1．D。【解析】直接插入排序从第二个元素开始，前面的序列为有序序列，后面的序列为待排序序列。每次都要需要将待排序序列第一个元素，在有序序列找到他插入的位置，进行元素移动和插入。最坏的情况是每轮待插入元素的位置都是有序序列的第一个，这样比较次数为 1+2+3+...+(n-1)=n(n-1)/2。举例：{n, n-1, ...2, 1}排序成升序序列。补充：最少比较次数是(n-1)次，即每次插入的元素都在最后面。如：{1, 2, ..., n-1, n}。

2．A。【解析】A 正确，直接插入排序最优的时间复杂度是 O(n)，最坏和平均时间复杂度是$O(n^2)$。

B 错误，折半插入排序相较于直接插入排序，仅仅减少了每次比较的数量级（$O(log_2 n)$），平均时间复杂度是$O(n^2)$，最好的情况$O(nlog_2 n)$。

C 错误，希尔排序不稳定。考纲内不稳定的排序算法有：简单选择排序、希尔排序、堆排序、快速排序。

D 错误，初始序列不同，折半插入排序比较的次数是有差异的，因为折半查找判定树失败结点不一定在同一层。

3．C。【解析】A 错误，简单选择排序，最后两个元素一定是最大值和次大值。B 错误，同 A。

C 正确。直接插入排序第二趟排序后，前面三个元素应局部有序，20，26，40 符合要求。

D 错误，两趟堆排序后，末尾两个元素一定是最大值（大根堆）或者最小值（小根堆），题目序列不符合要求。

4．B。【解析】越接近正序的序列，直接插入排序的比较次数就越少。

选项 B 和 C 是比较接近正序的，然后分别判断两个序列的比较次数，以选项 B 为例：第一趟，插入 32，比较 1 次；第二趟，插入 46，比较 1 次：第三趟，插入 40，因为 40 比 46 小但比 2 大，所以

比较 2 次；第四趟，插入 80，比较 1 次；第五趟，插入 69，比较 2 次；以此类推，共比较 9 次。

同理求出选项 C 的比较次数为 11 次。所以选择选项 B。

5．B。【解析】增量 d=4，于是序列 50、40、95、20、15、70、60、45 中，50、15 是一组，40、70 是一组，95、60 是一组，20、45 是一组，经过组内排序后变为：15、40、60、20、50、70、95、45。

6．B。【解析】希尔排序将序列根据增量分成若干组，在组内进行简单插入排序。由观察可知，经过一趟后 9 和-1 交换，7 和 4 交换，可知增量为 4。

7．D。【解析】A 选项的排序过程为：[49₁, 38, 65, 49₂, 76, 13]→[49₁, 38, 13, 49₂, 76, 65]→[13, 38, 49₁, 49₂, 65, 76]，整个过程没有破坏稳定性。

B 选项的排序过程为：[49₁, 38, 49₂, 65, 13, 76]→[49₁, 13, 49₂, 65, 38, 76]→[13, 38, 49₁, 49₂, 65, 76]，同样没有破坏稳定性。

C 选项的排序过程为：[49₁, 49₂, 13, 65, 76, 38]→[49₁, 49₂, 13, 65, 76, 38]→[13，49₁, 49₂, , 65, 76, 38]，同样没有破坏稳定性。

D 选项的排序过程为：[49₁, 13, 65, 38, 76, 49₂]→[38, 13, 49₂, 49₁, 76, 65]→[13, 38, 49₂, 49₁, 65, 76]，破坏了稳定性。答案选 D。

8．A。【解析】冒泡排序最少进行 1 趟冒泡，最多进行 n-1 趟冒泡。初始序列为逆序时，需进行 n-1 趟冒泡，并且元素交换的次数最多。初始序列为正序时，进行 1 趟冒泡(无交换)就可结束算法。

9．C 【解析】给出的初始待排序列为完全逆序序列，当记录个数为 n 时，冒泡排序需要进行 n－1 趟，每一趟的比较次数分别为 n－1，n－2，…，1，此题中 n=6，因此总的比较次数是 5+4+3+2+1=15。

10．A。【解析】这题没说明排序的方向是从前往后还是从后往前，如果是从前往后排序，那么至少需要 8 趟，反之最少需要 6 趟，A 正确；由排序过程可知，元素的交换次数是 22 次，选项 B 排除；无论从前往后还是从后往前，序列的逆序数不变，交换的次数不变，都是 22 次，但是选项中说的是移动次数，每交换一次（swap）需要移动 3 次，所以移动次数是 66 次，选项 D 错误；从后往前进行冒泡排序，第三趟的结果是 {1，2，4，5，9，10，8，6，20}，选项 C 错误。

11．B。【解析】第一趟排序结果为 4, 7, 3, 5, 6, 8, 9, 1, 2, [10], [10]表示已经排好的序列，下同；第二趟排序结果为[1], 4, 7, 3, 5, 6, 8, 9, 2，[10]；第三趟排序结果为[1], 4, 3, 5, 6, 7, 8, 2, [9, 10]；第四趟排序结果为[1, 2], 4, 3, 5, 6, 7, 8, [9, 10]；第五趟排序结果为[1, 2], 3, 4, 5, 6, 7, [8, 9, 10]，此时已有序，但仍需要一趟无交换的排序才能判断已经有序。所以总共需要 6 趟排序，答案为 B。

12．B。【解析】I 错误，最坏的时间复杂度是$O(n^2)$。

II 错误，快速排序的空间复杂度为递归深度，最大可达$O(n)$。

III 正确，每趟划分越均分，比较次数就越少，性能越佳。

IV 正确，快速排序是不稳定的排序算法。不稳定的还有简单选择，堆排序，希尔排序。

V 正确，左边区间的元素比基准元素小，右边区间的元素比基准元素大，所以左右区间相对有序。区间内无序。

13．C。【解析】C 选项(20，20，30，10，40)，默认第一个元素为枢轴元素，根据快排原理，第

一趟结束后，20 应该确定在原本 30 的位置，即(10，20，20，30，40)，所以不稳定。

14．D。【解析】每趟划分都会在区间中选择一个基准元素，使其左边的元素都比他小，右边的元素都比他大，应逐个观察每个元素的左右区间的元素，如果左边出现比他大的，或者右边出现比他小的，就不符合基准元素的要求，符合条件的只有 81。

15．A。【解析】每次枢轴元素将区间划分得越均匀，排序速度越快。选择第一个元素为枢轴，那么 A、C 选项的 21 正好是序列的正中，所以排除 B、D。A 经过一次排序后结果为 9、17、5、（21）、25、23、30.C 经过一次排序后结果为 5、9、17、（21）、25、23、30.对于序列 9、17、5 和 5、9、17，后者在有序状态下用快速排序方法的速度没有前者快，故本题选 A。

16．B。【解析】因为 C、D 选项基本有序，所以递归深度一定比较深。我们来比较 A、B 选项。A 选项每趟划分如下：

[4, 2, 6, 1, 5, 3, 8, 7]→[3, 2, 1, 4, 5, 6, 8, 7]→[1, 2, 3, 4, 5, 6, 8, 7]→[1, 2, 3, 4, 5, 6, 8, 7]→[1, 2, 3, 4, 5, 6, 7, 8]→[1, 2, 3, 4, 5, 6, 7, 8]，递归深度为 5。

B 选项每趟划分如下：

[5, 2, 6, 1, 4, 3, 8, 7]→[3, 2, 4, 1, 5, 6, 8, 7]→[1, 2, 3, 4, 5, 6, 8, 7]→[1, 2, 3, 4, 5, 6, 7, 8]→[1, 2, 3, 4, 5, 6, 7, 8]，递归深度为 4

类比，B 选项的递归深度为 4，本题答案选 B。

17．B。【解析】对各序列分别执行一趟快速排序，可做如下分析(以选项 A 为例)：枢轴值为 92，因此 35 移动到第一个位置，96 移动到第六个位置，30 移动到第二个位置，再将枢轴值移动到 30 所在的单元，即第五个位置，所以选项 A 中序列移动的次数为 4。同样，可以分析出选项 B 中序列的移动次数为 8，选项 C 中序列的移动次数为 4，选项 D 中序列的移动次数为 2。

18．D。【解析】对 n 个元素进行第一趟快速排序后，会确定一个基准元素，在这个基准元素的左边的元素（如果有）全部比他小，右边的元素（如果有）都比他大。A 选项中，第一轮可以选取 28 或者 72 作为基准元素，我们只要找一种符合的就可以，所以我们任选一种往后推进，这里我们选 72。第二轮我们对 72 的左区间进行第二轮排序，我们可以找到 28 作为基准元素，因此选项 A 可以是第二趟结果。

对于 B 选项，第一轮可以选取 2，第二轮选取 72 作为基准元素。

对于 C 选项，第一趟，我们可以选取 2，第二轮选择 28 作为基准元素。

对于 D 选项，第一趟可选择 12 或者 32，若选 12，此时有左右两个区间，第二趟左区间没有符合的基准元素。若选 32，右边区间没有符合的，因此答案选 D。

19．B。【解析】A 错误，每趟把未排序的元素中选择最小的元素加入已排序序列的尾部，移动的次数固定为每趟一次，n 趟 n 次。B 正确，每趟确定一个元素的最终位置。C 错误，简单选择排序，每轮都要从未排序序列选择最小的元素，需要比较当前剩余未排序元素个数-1 次。比较次数固定为(n-1) + (n-2) + ... + 1 = n(n-1)/2，所以比较次数是$O(n^2)$，D 错误。

20．D。【解析】A 选项，设 5(A)为原序列在前的，5(B)为原序列在后的。初始：[4, 3, 5(A), 7, 6, 5(B), 2]

找最小值 2 → 与 4 交换 → [2, 3, 5(A), 7, 6, 5(B), 4]

找次小 3 → 与 3 本身交换（不变）→ [2, 3, 5(A), 7, 6, 5(B), 4]

找次小 4 → 与 5(A) 交换 → [2, 3, 4, 7, 6, 5(B), 5(A)]

找次小 5(B) → 与 7 交换→ [2, 3, 4, 5(B), 6, 7, 5(A)]

找次小 5(A) → 与 6 交换→ [2, 3, 4, 5(B), 5(A), 7, 6]

找次小 6 → 与 7 交换→ [2, 3, 4, 5(B), 5(A), 6, 7]

同理 B、C 也是不稳定的，D 选项两个元素 5 的位置先后次序始终不变，并没有说明不稳定，答案选 D。

21．B。【解析】第一趟选择 1，将 1 和 12 交换位置，序列变为 1，15，12，18，2，35，30，11；第二趟选择 2，将 2 和 15 交换位置，序列变为 1，2，12，18，15，35，30，11。故 B 正确，A、C、D 错误。

22．A。【解析】堆数据结构是一种数组对象，它可以被视为一颗完全二叉树结构。它的特点是父节点的值大于（小于）两个子节点的值（分别称为大顶堆和小顶堆）。判断方法：将序列转换成一棵完全二叉树，再看各个子树是否都满足最大堆或者最小堆的要求。

23．C。【解析】题目所给序列是无序的，且要进行降序排列，因此每次换到堆尾的是最小的元素，因此要先构建初始小根堆。构建完后堆为{1, 2, 5, 16, 20, 19, 34}，第一趟调整后为{2, 16, 5, 34, 20, 19, 1}，第二趟调整后为{5, 16, 19, 34, 20, 2, 1}，因此答案选 C。

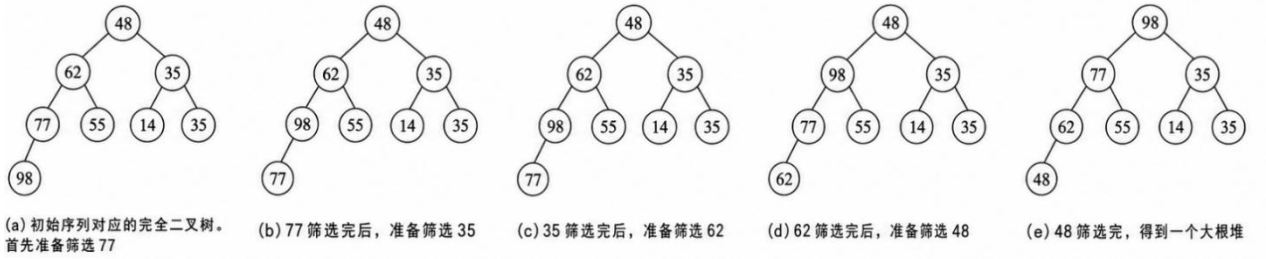
24． B。【解析】使用堆排序时，初始堆需要一个大根堆，建堆时先画出完全二叉树，然后从(heap_size-1)/2 号元素，即 57 开始从后往前调整，每次调整 i 号元素时将 i 号元素、i* 2+1 号元素、i*2+2 号元素中的最大元素和 i 号元素交换位置，模拟此过程最终可以得到初始堆为 89，78，57，25，41，45。

25．A。【解析】注意区分题目的信息，如果题目说的是将关键字进行堆排序或者构建一个小根堆，那么需要进行建堆（调整）的操作。但此题已经是一个小根堆，需要一个个插进去，每次将关键字查到堆尾，向上调整。然后下一次插入，直到所有的关键字都插入到小根堆为止。过程如下：

已插入的关键字	插入调整后的小根堆
7	7
7, 5	5, 7
7, 5, 1	1, 7, 5
7, 5, 1, 4	1, 4, 5, 7
7, 5, 1, 4, 6	1, 4, 5, 7, 6
7, 5, 1, 4, 6, 8	1, 4, 5, 7, 6, 8
7, 5, 1, 4, 6, 8, 9	1, 4, 5, 7, 6, 8, 9

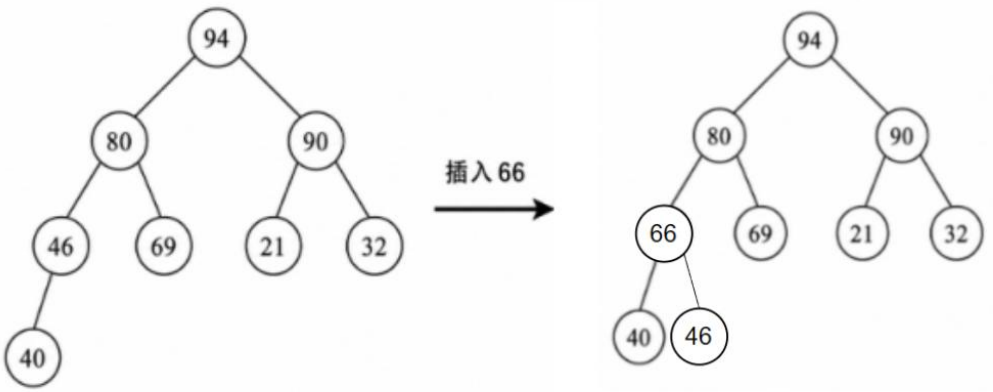
26．D。【解析】本题考查堆排序的执行过程。筛选法初始建堆为{8，17，23，52，25，72，68，71，60}，输出 8 后重建的堆为{17，25，23，52，60，72，68，71}，输出 17 后重建的堆为{23, 25, 68, 52, 60, 72, 71}。

27．B。【解析】首先对以第[n/2]个结点为根的子树筛选，使该子树成为堆，之后向前依次对各结点为根的子树进行选，直到选到根结点。序列(48.62.35.77.55.14.35.98}建立初始堆的过程如下所示。如图所示，(a)调整结点 77，交换 1 次； (b)调整结点 35，不交换；(c)调整结点 62，交换 2 次：(d)调整结点 48，交换 3 次。所以上述序列建初始堆，共交换元素 6 次。



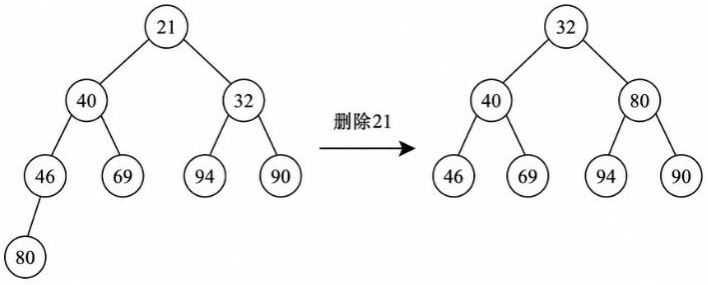
28．C。【解析】建大根堆后，序列为{94, 80, 90, 46, 69, 21, 32, 40}。如下左图；

插入 66，应放在堆尾，然后向上调整，遇到比他小的，就交换。调整结束后的序列为{94, 80, 90, 66, 69, 21, 32, 40, 46}，如下右图。所以 66 在第 3 层。



29．C。【解析】建成小根堆后，序列为{21, 40, 32, 46, 69, 94, 90, 80}。如下左图；

删除 21，将堆尾元素放到删除位置，然后向下调整，调整结束后的序列为{32, 40, 80, 46, 69, 94, 90}，如下图所示。堆顶元素的两个孩子为 40, 80，答案选 C。



30．B。【解析】堆并不是一棵二叉排序树，他满足根节点大于孩子结点即可，不需要保证左<根<右。

31．A。【解析】采用空间 k=10 的小根堆，遍历剩余的 9990 个元素：

①当前元素>堆顶：说明它有资格成为 “前 10 大”，替换堆顶并调整堆（确保新堆顶仍是堆内最

小值）；

②当前元素≤堆顶：说明它不可能是前 10 大，直接跳过；

遍历结束后，堆内 10 个元素就是前 10 大（堆顶是第 10 大，堆内是 “从小到大” 的前 10 大）。时间复杂度：O(k + (n-k)log k)，建堆 O(k)，后续插入调整 O((n-k)logk)。本题答案为 A。

32．C。【解析】归并趟数 S=$\lceil log_k r \rceil = \lceil log_k 64 \rceil = 3$，解得 k=4。

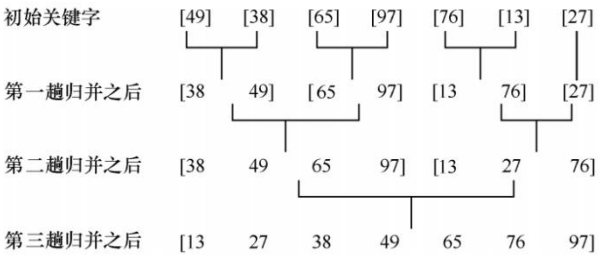
33．A。【解析】对于{25，48，16，35，79，82，23，40，36，72}，{25，48}和{16，35}两个子序列归并的结果为{16，25，35，48}，{79，82}和{23，40}两个子序列归并后的结果为{23，40，79，82}，余下的两个记录本趟不归并，所以一趟归并后的结果为{16，25，35，48，23，40，79，82，36，72}。本题答案为 A。

34．C。【解析】A 选项正确，如果是第一趟后的结果，那么每个小区间内的 2 个元素应该是局部有序的。而题干的序列 49, 38 无序，所以有序的区间大小只能是 1，也就是初始状态。

B 正确，趟数 S=$\lceil log_{27} = 3 \rceil$。

C 错误，第一趟，两两归并，2 个元素归并，只需要比较一次即可，落单的 27 无需比较，所以比较次数为 3 次。

D 正确，排序过程如下图所示：



35．C。【解析】基数排序有 MSD 和 LSD 两种，第一种是最高位优先(MSD)法，按关键字位权重递减依次逐层划分成若干更小的子序列，最后将所有子序列依次连接成一个有序序列；第二种是最低位优先(LSD)法，按关键字位权重递增依次进行排序，最后形成一个有序序列。基数排序是稳定的。

对于选项 A，不符合 LSD 和 MSD；对于选项 B，符合 MSD，但关键字 42、46 的相对位置发生了变化；对于选项 D，不符合 LSD 和 MSD。

36．C。【解析】关键字有 m 位，则要进行 m 趟的分配和收集，一趟中，分配是指把 n 个关键字放到对应队列(有 r 个队列)里，然后再对每个队列进行收集，分配的时间复杂度为 O(n)，收集的时间复杂度为 O(r)，所以总的时间复杂度为$O\big(m(n+r)\big)$。

37．D。【解析】临时建立的队列个数为进制数大小，应选 D。

38．A。【解析】冒泡排序、堆排序和快速排序的每一趟都能使至少一个记录元素到达最终在有序序列中的位置上，但是作为直接插入排序，如果整个初始记录序列的最后一个记录有最小关键字，那么就会出现题中描述的情况。

39．B。【解析】快速排序是不稳定排序，冒泡排序是稳定排序，堆排序是不稳定排序，归并排序是稳定排序，插入排序是稳定排序，因此上述排序算法中有三种是稳定排序算法。

40．B。【解析】例如对于序列[4，3，2，1]（反序）和[3，4，2，1]（非反序），使用简单选择排

序，前者需要交换元素 2 次，后者需要交换元素 3 次。

41．A。【解析】A.直接选择排序由于每次都需要遍历整个序列选出最大（最小）值，所以平均、最好、最坏都是$O(n^2)$

B．直接插入排序最好情况下复杂度为 O(n)

C．冒泡排序最好情况下复杂度为 O(n)

D．归并排序每次进行都需要进行分组、合并，复杂度都为$O(nlog_2n)$

42．A。【解析】I 错误，I 还是 O(n²)，比较操作的复杂度降为 O（logn）移动操作的次数不变，总体不变。II 正确； III 正确，相对有序情况下，插入排序的时间复杂度接近 O（n）；IV 错误，若每次都是查找第一个元素，顺序查找为 O(1)，折半查找 O（logn）。

43．A。【解析】堆排序每趟结束都能将一个最值放在根节点；冒泡排序进行两两比较，每趟选出最大值或最小值；快速排序一趟可以选出比关键值大的和比关键值小的，从而确定位置；直接插入排序插入的时候不能确定一个元素的最终位置。

44．D。【解析】归并排序需要创建新数组来完成新的顺序存储，需要辅助空间$O(n)$，简单选择排序和堆排序的辅助空间为$O(1)$，快排的为$O(logn)$。

45．A。【解析】归并排序：基于分治的思想，其时间复杂度总是：$O(nlogn)$

快速排序：平均时间复杂度是：$O(nlogn)$，最坏时间复杂度为：$O(n^2)$

冒泡排序：平均时间复杂度是：$O(n^2)$，最坏时间复杂度为：$O(n^2)$

插入排序：平均时间复杂度是：$O(n^2)$，最坏时间复杂度为：$O(n^2)$

46．D。【解析】题目的排序可类比 2 位整数排序，k1 是十位，k2 是个位

回忆基数排序，我们是先由低位向高位排序，所以应该先排序 k2，再排序 k1。排除 A、C。

B、D 区别在于直接排序是稳定的，简单选择是不稳定的，我们举例讨论：21、22、13

先按 k2 排序：21、22、13。再按 k1 排序，如果不稳定，比如简单选择：第一趟：13 22 21；第二趟：13 22 21；第三趟：13 22 21，不符合题意。

47．D。【解析】以上全部需要考虑。

48．C。【解析】简单选择排序在最好、最坏、平均情况下的时间性能均为 $O(n^2)$，归并排序在最好、最坏、平均情况下的时间性能均为$O(nlog_2n)$。。快速排序在原序列本身有序的时候达到最坏的时间复杂度，直接插入排序在原序列本身有序的时候达到最好的时间复杂度

时间复杂度	直接插入	冒泡排序	简单选择	希尔排序	快速排序	堆排序	二路归并
平均情况	$O(n^2)$	$O(n^2)$	$O(n^2)$	—	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$
最好情况	$O(n)$	$O(n)$	$O(n^2)$	—	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$
最坏情况	$O(n^2)$	$O(n^2)$	$O(n^2)$	—	$O(n^2)$	$O(n\log_2 n)$	$O(n\log_2 n)$

49．B。【解析】排序算法的性能取决于多种因素，如序列初始状态、数据规模等等，A 错误。基数排序和归并排序是稳定的排序方法，B 正确。快速排序是一种交换排序，C 错误。堆排序的空间复杂度为 O(1)，其所用的辅助空间较小，D 错误。

50．B。【解析】基数排序每趟需要利用前一趟已排好的序列，无法并行执行。快速排序每趟划分的子序列互不影响，可以并行执行。冒泡排序每趟对未排序的所有元素进行一趟处理，无法并行执行。堆排序可以并行执行，因为根结点的左右子树构成的子堆在执行过程中是互不影响的。

51．D。【解析】I 正确，简单选择排序，无论待排序序列初始状态如何，比较的次数始终是$(n-1)+(n-2)+\ldots+2+1=(n-1)n/2$。冒泡排序，元素基本有序的情况下，比较次数接近 O(1)。

II 错误，两者的空间复杂度都是 O（1）

III 正确，冒泡排序稳定，简单选择排序不稳定。

52．C。【解析】基数排序是基于关键字各位的大小进行收集排序的，而不是基于关键字的比较进行的

53．D、C。【解析】归并排序在平均情况和最坏情况下的空间复杂度都是 $O(n)$，快速排序只在最坏情况下才是$O(n)$，平均情况是$O(log_2 n)$。

54．C。【解析】采用败者树进行 k 路平衡归并时，内部归并过程中记录关键字的比较总次数将与 k 无关，但是，总的归并效率与 k 有关，当 k 较大时，归并趟数减少，因此可以减少内外存信息交换次数从而提高总的归并效率。不过，k 值的选择会受到内存缓冲区大小等因素限制。另外，初始归并段个数减少也是减少归并趟数即减少内外存交换的一个方法。

55．D、A。【解析】相比普通的 m 路归并：需增加一个输出缓冲区，当一个输出缓冲区满时，输出一个缓冲区的同时归并程序可向另一个输出缓冲区填充数据，这就实现了内部归并和输出的并行。需增加 m 个输入缓冲区，当 m 个输入缓冲区正在运行时，外部可向新增的 m 个缓冲区写入数据，这就实现了输入和内部归并的并行。综上，需设置 2 个输出缓冲区，2m 个输入缓冲区。

56．C。【解析】每次让计算机内存填满 400，3110400 个记录要填 3110400/400＝7776 次，对于 n 路归并排序，m 个归并段，需要归并排序的次数为$\lceil log_n m \rceil$次，代入数据得到答案为 5。

57．C。【解析】首先要用内部排序生成局部有序的初始归并段，每个初始归并段段长和内部排序的工作区容量相等，为 250 个记录。那么内部排序需要进行 8 次，生成 8 个初始归并段。每次读写 2 个磁盘块，每读/写一个磁盘块为一次 I/O，总计 8*2*2=32 次 I/O。在二路平衡归并排序中，初始归并段共 8 段，归并趟数$S=log_2 8=3$ 趟，每趟读 16 次(2000/125=16 块)，写 16 次，归并排序 I/O 次数为

16*2*3=96 次。所以整个过程为 32+96=128 次。

58．D。【解析】败者树是一种用于多路归并的完全二叉树，由于每个叶子需要决出胜负（比较），故没有度为 1 的节点。其中每个非叶子节点表示输掉比较的元素，叶子节点对应于归并路数 k。二叉树叶子节点数为 k，那么度为 2 的结点数为 k-1，总节点数为 2k-1，D 为正确选项。

59．A。【解析】败者树是一种特殊的二叉树，用于进行多路归并排序。败者树的特点如下。对于 A 选项，在败者树中，非叶子结点存储的是败者的标号，即从子结点中选出的较大值所对应的标号。这是败者树的核心特点，故选 A。

B 选项说法不准确，败者树的叶子结点存储的是归并排序中各个路的当前记录的索引值，而不是原始输入数据。

C 选项错误。败者树主要用于外部排序，尤其是多路归并排序。

D 选项错误。败者树的根结点存储的是最后一次竞争败者所对应的标号。最小值的标号存储在败者树的另一个额外的结点中，称为“最终胜者”。

60．B。【解析】I 错误，分支结点存储的内容是失败记录所在缓冲区的编号，叶子结点存储的是每个缓冲区（归并段）当前参加比较的第一个记录的关键字的值，要注意区分。

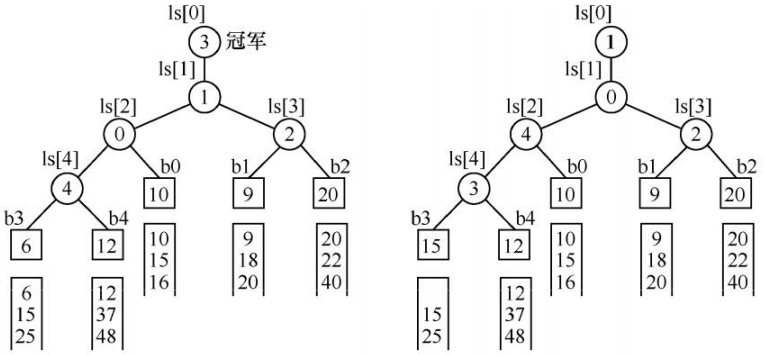
II 正确，上个冠军结点所在缓冲区的记录被选择后，补充该缓冲区后续一个记录的关键字进对应叶子结点，一直跟到根节点路径上的上一轮失败结点进行比较，胜者继续往上比较，直到选出新的冠军结点为止。

III 错误，败者树（除冠军节点外）是一棵完全二叉树，且只有度为 2 和 0 的结点（因为要两两比较）。根据二叉树性质：叶子节点个数=度为 2 的个数+1，所以总结点个数是 $2n_0-1$，n_0即归并段的个数，空间开销即二叉树的容量，因此是$O(n)$。补充：败者树每次比较的时间开销是$O(log_2 n)$。

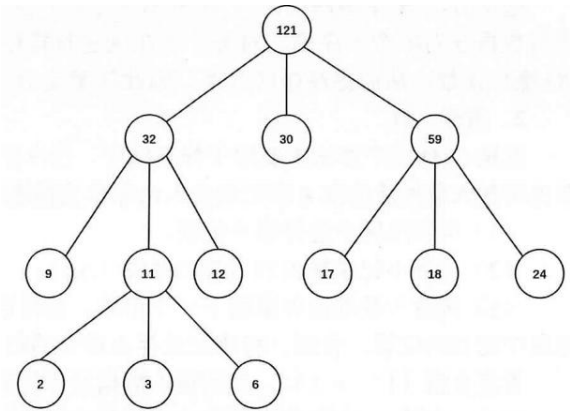
IV 正确，如果不加最大值的记录，那么一个归并段的记录消耗完后，没法继续进行败者树的运行了。

综上 I、III 错误，答案选 B。

61．B。【解析】构建的败者树如左图所示，第二轮比较后，如右图所示，可知根节点是归并段 0 的编号。



62．D。【解析】首先要判断是否要补充虚段，计算 t=(n-1)%(k-1)是否为 0，本题中，t=(n-1)%(k-1)=(9-1)%(3-2)=0，不需要补充；若不为 0，需要补充的虚段数为 n-1-n=t。最佳归并树如下。每个结点再归并排序中需要读出和写入磁盘各一次，所以 I/O 次数为 WPL*2。经计算 I/0 次数=2*(2*3+3*3+6*3+9*2+12*2+17*2+18*2+24*2+30*1)=446。



63．B。【解析】得到了 3 个初始归并段：(04, 12, 15, 19, 25, 27, 34, 83)，(07, 10, 11, 16, 26, 90)和(06)。

64．A。【解析】此题答案选 A。初始化过程，以及第一个归并段形成过程如下图所示：

（图 j 是输出 6 个记录后选得 MINIMAX 记录为 wa[1]的败者树，图 k 为得到第一个初始归并段后的状态。）

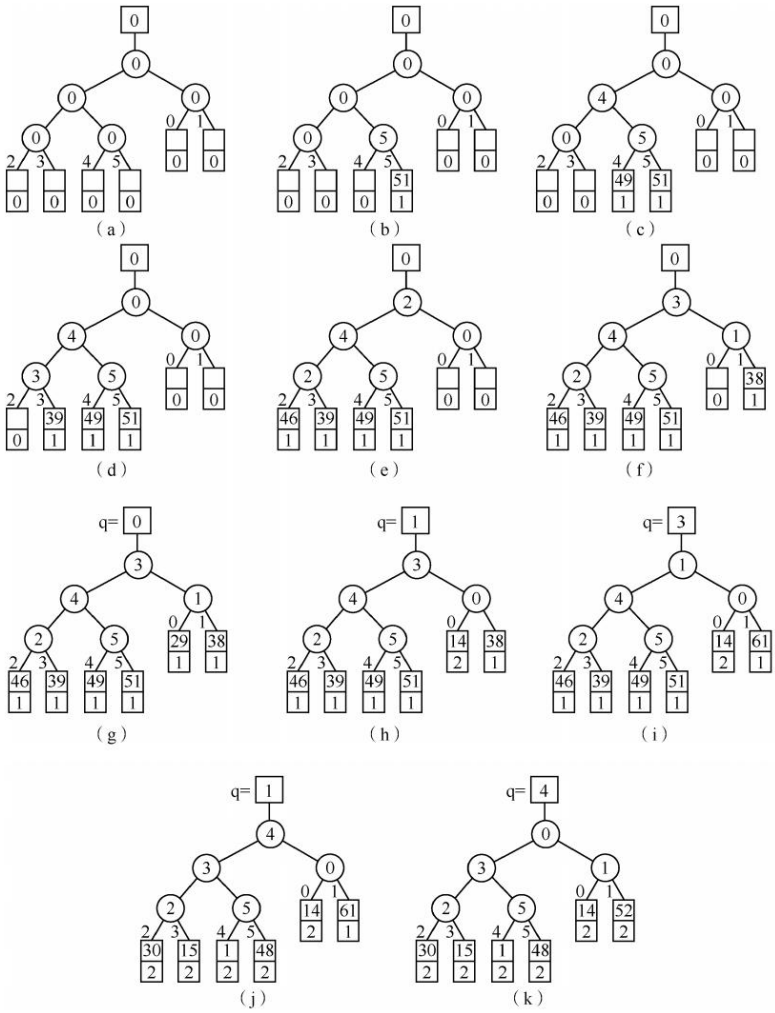


图 8.19　置换－选择过程中的败者树

注：图 8.19（a）～（g）建立败者树，选出最小关键字记录 wa[0]，图 8.19（h）～（k）选好新的 MINIMAX 记录

65．D。【解析】置换-选择排序算法主要用于外部排序，当内存空间有限时，需要将大量数据分段

进行排序。在算法中首先将输入数据流的前 k 个记录读入内存作为候选区。然后，执行以下操作。

（1）从候选区中选择最小记录

（2）将最小记录输出到当前归并段

（3）从输入数据流中读取下一个记录，如果该记录大于或等于已输出的最后一个记录，将其插入到候选区中适当的位置。否则，将该记录存入适当的归并段。

重复步骤(1)~(3)，直到输入数据流中的所有记录都被处理完毕。

因此，选项 D 正确描述了置换-选择排序算法如何处理候选区和输入数据流的关系。

66．D。【解析】败者树每次选择一个 MINIMAX 记录的时间复杂度是$O(log_2 m)$，一共要选择 n-1 轮，所以整体的时间复杂度是$O(nlog_2 m)$。本题细节应结合上一题理解。

67．A。【解析】选择置换排序工作流程如下：

①从输入文件 FI 输出 w 个记录到工作区 WA。

②从 WA 中选出其中关键字取最小值的记录，记为 MINIMAX 记录。

③将 MINIMAX 记录输入到输出文件 FO。（FI 和 FO 一般是外部文件物理块—缓冲区—内存工作区 WA 交互）

④若 FI 不空，则从 H 输出下一个记录到 WA。

⑤从 WA 中所有关键字比 MINIMAX 记录的关键字大的记录中选出最小关键字记录，作为新的 MINIMAX 记录。

⑥ 重复③~⑤，直至 WA 中选不出新的 MINIMAX 记录为止，由此得到一个初始归并段，输入一个归并段的结束标志到 FO。

⑦重复②~⑥，直至 WA 为空。由此得到所有初始归并段。

所以，I、III 都是选择 - 置换算法选择下一个符合条件的 MINMAX 的具体流程。II错误，当进行关键字比较时，先比较序号，应该选择段号小的为胜者，因为段号大的是后来新加入的一批的元素，新一批添加的元素的比当前 MINMAX 更小的数值（段号更大）是不应该被选取的。

68．D。【解析】I 正确：外部归并排序先要产生初始归并段，通常是在内存工作区中用内部排序算法得到有序段。且在多路平衡归并中，初始归并段的大小相等，通常为内存工作区的大小。

II 正确：置换-选择排序可以生成比内存工作区更长的初始归并段，从而减少初始归并段个数，进而减少归并趟数和磁盘 I/O。

III 正确：败者树用于多路归并时，可以减少从各归并段当前元素中选出最小关键字所需的比较次数。

IV 正确：置换-选择排序中也可用败者树优化，减少选择 MINIMAX 的比较次数。

V 基本正确：最佳归并树采用哈夫曼树思想，用于在初始归并段长度不等时安排归并顺序，使总归并代价最小，主要减少磁盘 I/O 次数。

69．B。【解析】因为每次都是 8 个归并段合并，所以归并树是度为 m 的正则树，只有度为 m 和叶子结点，由公式：度数之和+1=结点总数可以得到，$mn_m + 1 = n_m + n_0 \rightarrow (m-1)n_m = n_0 - 1 \rightarrow n_m = (n-1)/(m-1)$，但我们要注意到，当$(n-1)\%(m-1) = 0$，不需要补充虚结点；当$(n-1)\%(m-$

$1) \neq 0$，我们要补充虚结点，补充的虚结点个数为$(m-1)-(n-1)\%(m-1)$。补充的目的其实就是使n_m是个整数，即补充后的段数是(m-1)的倍数。将 n=80，m=8 代入，补充的虚段数是$(8-1)-(80-1)\%(8-1)=5$。

70．B。【解析】I/O 次数为形成初始归并段的内部排序和归并排序的磁盘 I/O 次数和归并排序的 I/O 次数之和。归并排序趟数 S=log_k，其中 k 为归并路数，r 为初始归并段数，归并排序的 I/O 次数=S*（记录总数/单个磁盘块所能容纳记录数） *2（*2 是因为读入和写出各一次）所以要减少 I/O 次数就要减少 S，由$S=log_k r$，减少 S 可以增大 k 或者减少 r。

A 选项选择置换排序是减少初始归并段数 r，但此时每个初始归并段大小不一样，所以需要 C 选项最佳归并树得到最小的 wpl（I/O 次数）。

D 选项是增大 k，同样可以减小 S，但由于内存缓冲区资源有限 k，k 路归并需要至少 k 个缓冲区，所以 k 不可无限增大。

B 选项，败者树是为了减少归并排序每次选取 k 个缓冲区中的最小的记录的比较次数，使得每次从 k 个缓冲区的首元素选出最小的数，比较次数 k-1 降为$log_2 k$，与 I/O 次数无关，故此题选 B。

71．C。【解析】插入 38：1 次；插入 65：2 次；插入 97：2 次；插入 76：3 次；插入 13：2 次；插入 27：3 次。1 + 2 + 2 + 3 + 2 + 3 = 13 次，答案选 C。

72．B。【解析】间隔为 d 的元素作为一组，组内有序。当 d=3，三组元素依次是{7,12,40},{15,18,34},{3,9,25}，每组内的元素皆有序，所以 d=3 符合。其他选项的间隔下都不符合。

73．C。【解析】 第一趟：49 38，移动 49，1 次 第二趟：38 49 65，0 次 第三趟：38 49 65 97，0 次 第四趟：38 49 65 97 76，移动 97，1 次 第五趟：38 49 65 76 97 13，移动 97、76、65、49、38，5 次 第六趟：13 38 49 65 76 97 27，移动 97、76、65、49、38，5 次 一共 1+0+0+1+5+5=12 次。

74．B。【解析】冒泡排序的交换次数等于序列中的逆序对个数。下面统计逆序对个数： 5 > 1，5 > 4，5 > 2，5 > 3，共 4 个 4 > 2，4 > 3，共 2 个 8 > 3，共 1 个 一共 4 + 2 + 1 = 7，所以交换次数为 7 次。

75．B。【解析】第一趟按个位排序：930，063，184，505，278，109，589 第二趟按按十位从小到大收集，并保持稳定性：505，109，930，063，278，184，589，答案选 B。

76．C。【解析】原序列为：49，38，65，97，76，13，27 从最后一个非叶子结点开始调整。调整结点 65：比较 13 和 27，1 次，比较 65 和 27，1 次，共 2 次。 调整结点 38：比较 97 和 76，1 次；比较 38 和 97，1 次，共 2 次。 调整结点 49：比较 97 和 65，1 次，比较 49 和 97，1 次，交换后继续调整：比较 38 和 76，1 次，比较 49 和 76，1 次，共 4 次。 总比较次数：2 + 2 + 4 = 8，答案选 C。

77．C。【解析】链表不支持随机访问，因此希尔排序、堆排序、折半插入排序不适合链表结构。归并排序只需要顺序访问，并且链表合并时不需要大量移动元素，因此较适合链表存储结构。

78．C。【解析】A 错误，冒泡排序平均时间复杂度为 $O(n^2)$，不属于先进的排序算法。B、C、D 中，只有归并排序是稳定的，所以答案选 C。

第二部分

408 1000 题 co 解析册

第 1 章

计算机系统概述

【答案速对】

CCADA　　DDBCB　　DBBBB　　DDCCB

BABBD　　DDABB　　DCDCB

1．C。【解析】指令集体系结构（Instruction Set Architecture，ISA）指令集体系结构是计算机硬件与系统软件之间的接口，高级语言程序经过编译后，最终会变成机器指令。CPU 要能执行这些机器指令，就必须有一套明确的规则，这套规则就是 ISA，ISA 关心的是程序员能看到什么、指令能使用什么、软件能依赖什么，不关心的是 CPU 等硬件 内部具体怎么实现。其核心部分是指令系统同时还包含数据类型和数据格式定义、寄存器组织、IO 空间的编址和数据传输方式、中断结构、计算机状态的定义和切换、存储保护等，A、B、D 正确，C 错误。

2．C。【解析】ISA 规定了指令的功能与规范，不规定指令的内部实现细节，指令寻址方式、操作码个数、指令使用的通用寄存器的编号均属于指令格式内容，而乘法器实现方式不属于，C 正确。注意：凡是程序员、编译器、汇编器必须知道才能正确写程序或生成机器码的内容，通常属于 ISA；只影响硬件内部如何执行、对软件不可见的内容，属于微体系结构或具体硬件实现，不属于 ISA。

3．A。【解析】A 正确：冯·诺依曼结构的标志性思想是“程序与数据共存”。在物理实体上，指令和数据同样都表现为二进制代码位流（0 和 1），存储于同一主存中。两者的唯一区别在于 CPU 在不同运行阶段对它们的解读方式不同：在取指阶段从主存流向指令寄存器（IR）的二进制串被解释为指令；在执行阶段流向通用寄存器或 ALU 的被解释为操作数（数据）。

B 错：主存地址空间分配自由灵活，并不强制限定指令低、数据高的刚性区间。

C 错：这是哈佛结构（Harvard Architecture）的特征，不是冯·诺依曼 结构。

D 错：数据取出后可能作为控制信号、地址偏移量，甚至有时会被用于间接寻址等，不直接是操作数本身。

4．D。【解析】冯·诺依曼结构五大部件包括运算器、控制器、存储器、输入设备和输出设备，总线接口属于现代计算机的扩展结构，D 错误。

5．A。【解析】最低层为微程序级（硬件直接执行），向上依次为机器语言级、操作系统级、汇编语言级、高级语言级，A 正确。

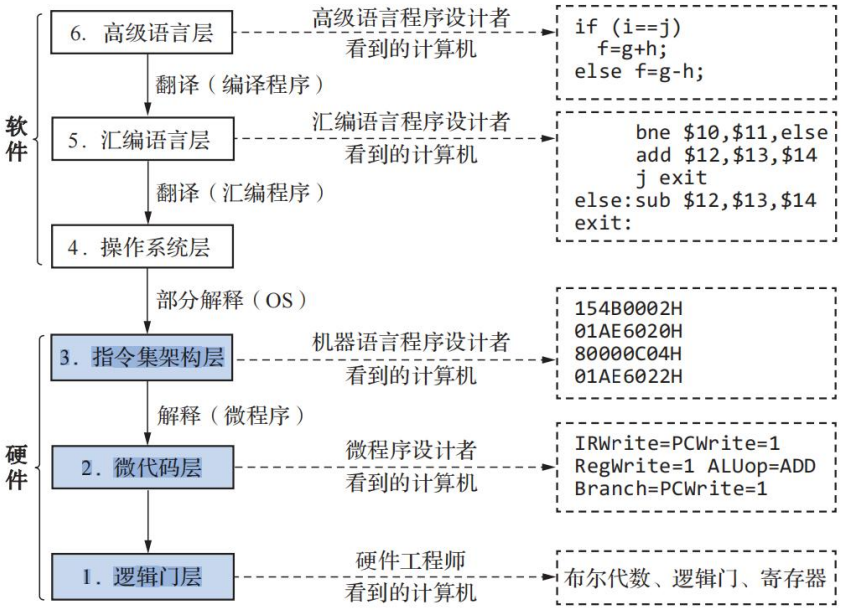
6．D。【解析】编译或汇编后生成的目标程序通常是机器语言形式，但可能还包含未解决的外部

引用不能直接运行，链接程序负责把一个或多个目标文件以及所需库函数组合起来，解决外部符号引用，最终生成可执行文件。因此选 D。预处理阶段主要处理宏定义、条件编译、头文件包含等内容。例如 C 语言中的 #include、#define 通常由预处理器处理；编译主要完成高级语言源程序到目标程序或汇编程序的翻译，并进行词法、语法、语义分析等；汇编是通过汇编程序将汇编语言程序转换为机器语言目标程序。

7．D。【解析】I 错误，汇编语言本质是用助记符表示的机器语言，CPU 无法识别助记符，只有通过汇编程序转化为机器语言才可被 CPU 直接执行。II 错误，硬件描述语言主要用于描述硬件电路结构和行为，它不是 CPU 直接执行的机器语言，也不是普通意义上的软件程序语言。III 错误，汇编语言通常与具体机器的指令系统一一对应或密切相关，不同处理器的汇编语言差异较大。高级语言相对独立于具体机器。IV 错误，高级语言可以通过翻译程序直接转换成机器语言，无需通过汇编语言过渡，如解释型语言。

8．B。【解析】汇编语言层和操作系统层属于软件部分，而指令集架构层属于软件可感知部分，他是计算机硬件和软件之间的桥梁，是对指令系统的一种规定，具体的实现组织称为微体系结构（如加法器实现采用并行进位方式还是串行进位方式），属于软件不可感知部分。(ISA 规定计算机要“做什么”，微架构（微代码）定义“如何做”，软件只关心功能，不关心具体实现)。

注意：不使用微程序控制器的计算机系统，没有微代码层。



9．C。【解析】

- 在软件和硬件之间的界面就是**指令集体系结构**(Instruction Set Architecture, ISA)，简称体系结构，它是软件和硬件之间接口的一个完整定义。
- ISA 定义了一台计算机可以执行的所有指令的集合，每条指令规定了计算机执行什么操作，所处理的操作数存放的地址空间以及操作数类型。
- ISA 规定的内容包括数据类型及格式，指令格式，寻址方式和可访问地址空间大小，程序可访问的寄存器个数、位数和编号、控制寄存器的定义、I/O 空间的编址方式、中断结构、机器工作状态的定义和切换、输入输出结构和数据传送方式、存储保护方式等。因此，可以看出，指令集体系结构是指软件能感知到的部分，也称软件可见部分。



- ISA 是对指令系统的一种规定或结构规范，具体实现的组织称为**微体系结构**(microarchitecture)，简称微架构。
  - ISA 和微体系结构是两个不同层面上的概念，微体系结构是软件不可感知的部分。例如，加法器采用串行进位方式还是并行进位方式实现属于微体系结构。
  - 相同的 ISA 可能具有不同的微体系结构，例如，对于 Intel x86 这种 ISA，很多处理器的组织方式不同，即具有不同的微架构，但因为它们具有相同的 ISA，因此，一种处理器运行的程序，在另一种处理器上也能运行。
- 微体系结构最终是由逻辑电路(logic circuit)实现的，微架构中的一个**功能部件**可以用不同的**逻辑电路**来实现，用不同的逻辑实现方式得到的性能和成本有差异。
- 最后，**每个基本的逻辑电路**都是按照特定的**器件技术**(device technology)实现的。例如，CMOS 电路中使用的器件和 NMOS 电路中使用的器件不同。

10．B。【解析】主机包括 CPU 和主存；CPU 包括运算器、控制器、寄存器、磁盘、键盘、鼠标等是外部设备。所以磁盘不包括在主机内，B 错误。

11．D。【解析】计算机的软件可以分为系统软件和应用软件两大类。其中，系统软件主要用来管理整个计算机系统、监视服务，使系统资源得到合理调度，高效运行，它包括标准程序库、语言处理程序、操作系统、服务程序、数据库管理系统、网络软件等；应用软件是根据任务需要所编制的各种程序，如科学计算程序、数据处理程序、过程控制程序、事务管理程序、文本处理程序等。

操作系统是系统运行的必要条件，所以其属于系统软件，A 错误。

编译程序和汇编程序均属于系统软件中的语言处理程序，B、C 错误。

12．B。【解析】现代 CPU 普遍通过浮点运算部件（Float Point Unit, FPU）实现浮点运算，其他选项属于操作系统软件功能，B 正确。

A 选项，进程调度主要由操作系统内核实现；C 选项，文件系统管理由操作系统完成，包括文件系统管理、文件创建、文件删除、目录管理、权限分配等；D 选项，中断响应是由硬件完成的，而中断处理程序由软件实现。

13．B。【**解析**】ISA 是软硬件的分界面，规定编程可见的寄存器、指令格式和寻址方式等，B 正确。

14．B。【解析】在“存储程序”思想指的是程序和数据都以二进制形式存放在存储器中，CPU 从存储器中取出指令并执行。程序在存储器中存放的是机器指令形式，而不是高级语言形式。

15．B。【**解析**】在冯·诺依曼计算机中，指令和数据以同等地位存放于存储器中，都根据地址访存，这并不是 CPU 区分指令和数据的方法，A，C 错误。

计算机中将时刻划分为不同的阶段，取指阶段从存储器中取出的是指令，执行阶段从存储器中取出的是数据，CPU 正是通过这种方式来区分指令和数据的，B 正确。

无论是指令还是数据，在计算机中都使用二进制数表示，D 错误。

16．D。【**解析**】编译程序一次性将整个源程序转换目标程序（通常为机器代码），然后由处理器执行目标程序。这样可以实现较高的执行效率，但目标代码通常是特定于某种处理器架构的，因此可移植性较差。解释程序则逐条解释并执行源程序，不会生成一个完整的目标程序，这样可以在不同的处理器架构上运行，具有较好的可移植性，但执行效率相对较低，D 正确，A、C 错误。

编译程序和解释程序的适用场景并不是以执行时间和运行频率来区分的。实际上，编译程序和解释程序的主要区别在于执行效率和可移植性。编译程序适用于对执行效率要求较高的程序，解释程序则适用于对可移植性要求较高的程序，或者在开发过程中需要快速调试的场景，B 错误。

17．D。【**解析**】MIPS 表示 Million Instructions Per Second，每秒执行百万条指令数。MIPS 受指令集影响，很难准确比较出不同 ISA 的机器性能，D 错误。

A 正确，不同机器的指令功能不同。某台机器一条指令能完成的工作多，另一台机器可能需要多条指令才能完成同一功能，MIPS 高不一定表示实际任务完成得更快。

B 正确，MIPS 依赖于具体指令集，不同 ISA 的指令复杂度不同，CISC 指令可能一条完成较复杂功能 RISC 指令通常较简单，需要更多指令组合完成复杂任务

C 正确，MIPS = 主频 / 平均每条指令所需要时钟周期数= 4 * 10^9/2*10^6= 2000。

18．C。【**解析**】MIPS 只是衡量计算机运算速度的一个指标，两台计算机 MIPS 值相同，但指令集不同、程序中各类指令占比不同等都会导致执行相同程序的速度不同，C 错误。A 选项是系统响应时间的正确定义；总线带宽 = 总线时钟频率 × 总线宽度 / 8（单位转换为字节），B 正确；磁盘 I/O 数据传输率与磁盘转速、寻道时间等密切相关，D 正确。

19．C。【**解析**】题目中可看出该基准测试程序主要侧重于整数处理性能的评估，第二台计算机测试结果更高，说明在整数处理上可能更优，但不能就此得出执行整数运算速度一定快 50%，因为测试存在局限性，A 错误，C 正确；该基准测试不能代表通用任务处理性能，B 错误；虽然基准测试程序单一，但能在一定程度上反映整数处理性能，D 错误。

20．B。【**解析**】0.4×1 + 0.3×2 + 0.2×3 + 0.1×4 = 2.0，B 正确。

21．B。【解析】A 执行时间 = $(1\times10^6\times2)/(2\times10^9)=1\times10^{-3}$ s。B 机时间 = $(0.8\times10^6\times3)/(3\times10^9)=0.8\times10^{-3}$s。显然 B 执行时间更短，B 正确。

22．A。【解析】CPI 与 CPU 的时钟频率无关。时钟频率决定时间尺度，而 CPI 是周期/指令的比值，受指令集和实现等因素影响，不受频率本身影响。时钟频率不影响 CPI，而是影响总执行时间，A 错误；微体系结构（如流水线、超标量、分支预测等）决定硬件执行指令的效率；指令调度策略（如编译器静态调度或硬件动态调度）用于优化指令执行顺序，减少数据依赖或控制依赖导致的冒险；指令系统结构（如 RISC 或 CISC）直接决定指令的复杂度和执行效率，都会影响 CPI，故 B、C、D 正确。

23．B。【解析】P 执行时间比 P'多了 2s，对应时钟周期数为。

而乘 4 指令的 CPI 比左移指令的 CPI 大 100，即将一条乘 4 指令换成左移指令能节省 100 个时钟周期，故替换的指令数为$\frac{2\times10^8}{100}=2\times10^6$条，B 正确。

24．B。【解析】计算程序总执行时间 T：已知指令数 $I=20\ 亿=2\times10^9$，平均 $CPI=2.5$，主频 $f=4GHz=4\times10^9Hz$。

$$T=\frac{I\times CPI}{f}=\frac{2\times10^9\times2.5}{4\times10^9}=1.25\ 秒$$

计算浮点操作总数：浮点指令占 25%，每条指令进行 1 次浮点运算：

$$N=2\times10^9\times25\%=5\times10^8 次浮点运算$$

MFLOPS（Million Floating-point Operations Per Second）即每秒执行浮点运算次数（单位 10^6）

$$MFLOPS=\frac{每秒浮点运算数}{10^6}=\frac{N}{T\times10^6}=\frac{5\times10^8}{1.25\times10^6}=400$$

25．D。【解析】公式：

$$S=\frac{1}{(1-F_e)+F_e/S_e}$$

原 I/O 时间比例 = 10% = 0.1；原 CPU 时间比例 = 1 - 0.1 = 0.9。

改进后 CPU 时间 = $\frac{0.9}{10}$ = 0.09（CPU 性能提高到 10 倍，即耗时变为 1/10）。

改进后 I/O 时间 = $\frac{0.1}{2}$ = 0.05(I/O 性能提高到 2 倍，即耗时变为 1/2）。

改进后总时间 = 0.09 + 0.05 = 0.14

系统总体加速比 = $\frac{改进前总时间}{改进后总时间}=\frac{1}{0.14}=\frac{100}{14}\approx7.14$

26．D。【解析】①程序的执行时间=平均 CPI*指令条数/机器时钟频率，提高机器时钟频率可以缩短执行时间；

②通过多个处理器实现指令并行缩短执行时间

③通过优化编译可以增加指令的并行性

④指令执行包括很多步骤，如果能减少访存时间，比如引入 cache，那么单条指令的执行时间将大大减少。

因此所有措施均正确，D 正确。

27．D。【解析】MIPS 表示每秒执行多少百万条指令，但不同计算机的指令系统不同，一条指令完成的功能和复杂度可能不同。同一任务在不同机器上所需指令条数也可能不同。因此，MIPS 较高并

不一定表示程序执行时间更短，A、B、C 都是 MIPS 不能很好反映计算机性能的合理原因。D 表述错误，MIPS 主要与指令条数、执行时间、CPI、时钟频率、程序特征等因素有关，并不是由存储容量完全决定。

28．A。【解析】CPI 是一条指令执行需要的时钟数，因此 MIPS=30/3=10，机器周期是干扰项，A 正确。

29．B。【解析】序列一一共 5 条指令，序列二一共 6 条，A 正确；执行速度即每执行一条指令需要的时间，计算平均 CPI 即可，序列一的平均 CPI=（2×1+1×2+2×3）/5=2，序列二平均 CPI=（4×1+1×2+1×3）/6=1.5，序列二执行速度比较快，B 错误，D 正确；序列一总时钟周期数=2×1+1×2+2×3=10，序列二总时钟周期数=4×1+1×2+1×3=9，C 正确。

30．B。【解析】$MIPS=$每秒执行多少兆条指令$=\frac{\text{时钟频率}}{CPI_{\text{平均}}}*10^{-6}=\frac{1s/\text{时钟周期}}{CPI_{\text{平均}}}*10^{-6}=\frac{1s}{2ns*CPI_{\text{平均}}}*10^{-6}=\frac{500}{CPI_{\text{平均}}}$ 序列一：时钟总数是 $1*5*10^9+2*1*10^9+3*1*10^9=10*10^9$，$CPI_{\text{平均}}=\frac{\text{时钟总数}}{\text{指令总条数}}=\frac{10*10^9}{7*10^9}=\frac{10}{7}$，$MIPS=\frac{500}{\frac{10}{7}}=350$

序列二：时钟总数是 $1*10*10^9+2*1*10^9+3*1*10^9=15*10^9$，

$$CPI_{\text{平均}}=\frac{\text{时钟总数}}{\text{指令总条数}}=\frac{15*10^9}{12*10^9}=\frac{5}{4}，MIPS=\frac{500}{\frac{5}{4}}=400$$

所以序列二的 MIPS 数比序列一多 50。

执行完序列一需要$\text{时钟总数}*\text{时钟周期}=10*10^9*2ns=20s$

执行完序列二需要$\text{时钟总数}*\text{时钟周期}=15*10^9*2ns=30s$

所以序列一的执行速度比序列二快 10s，答案选 B。

31．D。【解析】假设程序 P 的指令条数为 N，则在 M1 和 M2 上的执行时间分别为

M1: 4 N×0.8 = 3.2N (ns)

M2: 2 N×1.2 =2.4N(ns)

M2 执行 P 的速度更快，每条指令平均快 0.8ns，比 M1 快 0.8/3.2×100%=25%，D 正确。

32．C。【解析】$CPI=1\times60\%+2\times18\%+4\times12\%+8\times10\%=2.24$

MIPS=$40/CPI=40/2.24=17.9$

T= $CPI\times\frac{1}{f}\times I=2.24\times\frac{1}{40\times10^6}\times5000=2.8\times10^{-4}$秒，C 正确。

33．D。【解析】机器字长是计算机进行一次整数运算能够处理的二进制数据的位数。存储字长是一个存储单元中的二进制代码位数。指令字长是一个指令字中包含的二进制代码的位数。机器字长和存储字长没有必然的联系，但为了方便读取数据，通常规定机器字长等于整数倍的存储字长。故 I 错，II 对。指令字长与存储字长间也没有必然联系，但为了方便取指，通常规定指令字长等于整数倍的存储字长，故 III 对，D 正确。

34．C。【解析】此处的字长为机器字长，即计算机进行一次整数运算所能处理的二进制数据的位数。一台计算机的字长为 4 字节，这表示该 CPU 进行一次整数运算（ALU）能够处理的二进制数据的位数是 32 位，C 正确。运算结果最大不一定为2^{32}，比如无符号数，最大为$2^{32}-1$

35．B。【**解析**】时钟周期也称为震荡周期，定义为时钟频率的倒数，它是最基本的、最小的时间单位。在一个时钟周期内，CPU 仅完成一个最基本的动作。机器周期也称 CPU 周期，在计算机中，为了便于管理，常把一条指令的执行过程划分为若干个阶段（如取指、译码、执行等），每一阶段完成一个基本操作。完成一个基本操作所需要的时间称为机器周期。一般情况下，一个机器周期由一个或若干个时钟周期组成。

指令周期是取出一条指令并执行的时间，是从取指令、分析指令到执行完指令所需的全部时间，一般由一个或若干个机器周期组成。

故这三者的大小关系应当是指令周期 ≥ 机器周期 ≥ 时钟周期，B 正确。

第 2 章

数据的表示和运算

【答案速对】

ADBDC	BDBAA	ABCCC	BDBCD
CCAAA	CACCB	DCCCA	BDBAB
DBCCC	ACCBB	BCCAB	ACCDD
ACCCA	ADDCC	BBCAA	CDCCA
BBCBC	BCDB		

1．A。【解析】37 的二进制为 00100101，取反得 11011010，加 1 后为 11011011，A 正确。

2．D。【解析】对于补码表示的定点整数如$[x]_{补} = X_0X_1X_2...X_n(n \text{ 为整数})$，模数是$2^{n+1}$。

定点小数补码的模： 题目中的表示法 [x]补 $= X_0 . X_1X_2...X_n$ 是一种定点小数的表示。其中 X_0 是符号位（权重为 $-2^0 = -1$），小数点后的 X_1 到 X_n 是尾数。注意：

正小数的补码就是其本身。

负小数 x 的补码定义为：[x]补 $= 2 + x$ （当 $-1 \le x < 0$ 时）。

补码本质就是取模，让对应机器数落在模长范围内，这里的“2”就是因为取模运算产生的溢出在小数点左边第二位（即权值为 2^1=2 的位置），被自动舍弃。因此，纯小数补码的模是 2。

3．B。【解析】F 的 ASCII 码应 46H=1000110B。标准的 ASCII 码为 7 位，在 7 位数前面增加 1 位校验位。按照偶校验规则，偶校验位为 1。存储单元中存放的是整个校验码（包括校验位和信息位）应为 11000110B=C6H，B 正确。

4．D。【解析】补码负数的特点是数值位对应的真值越小，其绝对值越大，即负得越多。所以，由 4 个 1 和 4 个 0 组成的补码数中，最小的补码表示为 10000111，即真值为 -121，D 正确。

5．C。【解析】计算机里的各种数据，都是二进制数据，它们本身没有区别。通过不同的指令操作码 来对数据进行解释：如果是逻辑运算（与或非），CPU 就把它们当成逻辑数据；如果是算术运算加减乘除），CPU 就把它们当成数值数据，C 正确。

6．B。【解析】计算机常用补码表示有符号数，以实现加减法的统一，A 错误，B 正确；浮点数的尾数可以用原码和补码表示，在 IEEE 754 标准中，采用原码表示浮点数尾数，C 错误：浮点数的阶通常采用移码表示，D 错误。

7．D。【解析】表示移码时，真值$x_1=[x]_{移}-128$，表示补码时：①当符号位为 0（正数和 0），补码真值$x_2=[x]_{补}$，差值$x_1-x_2=-128$。（这里$[x]_{移}=[x]_{补}=$*机器数*）②当符号位为 1（负数），补码真值$x_2=[x]_{补}-256$，差值$x_1-x_2=128$。综上，答案选 D。

注意：移码和补码，表示范围一样，同一个真值，其移码和补码符号位相反。

8．B。【解析】由题目可知，x 符号位为 0 而 y、z 符号位为 1，说明 x 是正数，y、z 是负数，x 最大。对于补码负数，机器数越大，对应真值越大，所以 y<z，综上 y<z<x，答案选 B。

9．A。【解析】两者的区别在于符号位不同，A 错误。

10．A。【解析】A 选项：x1 为 1，×2x3 至少有一个为 1，如[X]补=1.101，则 X = -0.011，X = -（1/4+1/8），即 X>-1/2 成立；

[X]补=1.110，则 X=-0.010，X = -1/4，即 X>-1/2 成立：[X]补=1.111，则 X = -0.001，X -1/8，即 X>-1/2 成立，A 正确。

B 选项：x1 为 1，×2×3 任意时，如［X］补=1.100，则 X=-0.100，X=-1/2，即 X>-1/2 不成立，B 错误。

C 选项：x1 为 0，×2×3 至少有一个为 1，如［X］补=1.010，则 X=-0.110，X=-（1/2+1/4），即 X>-1/2 不成立，C 错误。

D 选项：x1 为 0，×2x3 任意，如［X］补=1.000，则 X=-1，即 X>-1/2 不成立，D 错误。

11．A。【解析】2 个“0”和 6 个“1”组成的 8 位二进制整数中，若用补码表示，要使得数值最大，符号位必须为 0，并且将“1”置于高位上，则补码最大为 01111110B=126，故 D 错误；补码表示的最小值，符号位应是 1，因为补码变原码要取反，故其余的 1 应放在低位上，则补码最小为 1001 1111B=-97，故 C 错误；若采用移码表示，偏置值为 128，要使整数最大则 1 应放置在高位，即 11111100B-10000000B=252-128=124，故 B 错误；若采用移码表示，偏置值为 127，要使整数最小则 1 应放置在低位，即真值=00111111B（移码）-01111111B（偏置值）=11000000B=-64，则 A 正确。

提示：题设补码表示的最小整数也可以通过位权来解释，将 5 个 1 放在最低位上，剩下的 1 放在符号位上（唯一的负权），此时为最小整数。

12．B。【解析】模 4 补码具有模 2 补码的全部优点而且容易检查加减运算中的溢出问题。有必要指出，存储每个模 4 的补码时，只要存一个符号位，因为任何一个正确的数值，其模 4 补码的两个符号位总是相同的。只在把两个模 4 补码的数送往算术与逻辑部件完成加减计算时，才把每个数的符号位的值同时送到算术与逻辑部件的两位符号位，即只在算术与逻辑部件中采用双符号位，B 正确。

13．C。【解析】C 正确，n 位的移码，当真值 x 为正，[x]_移=x+2^{n-1}，而补码[x]_移=x，两者相差一个 2^{n-1}，也就是符号位移码是 1，补码是 0；当真值 x 为负，[x]_移=x+2^{n-1}，而补码[x]_移=x+2^{n}，两者相差一个 2^{n-1}，也就是符号位移码是 0，补码是 1；A 错误，对于真值 0，原码和反码各有两种不同的表示形式，而补码和移码只有唯一的一种表示形式。正因为补码和移码 0 的表示形式唯一，才使得补码和移码比原码和反码能够表示的负数个数多一个，也就是 n 位的补码和移码的真值范围比反码和原码多一个最小负数-2^{n-1}，D 错误。B 错误，正数的原码、反码、补码表示一样，但移码不同。

14．C。【解析】字长 12 位，定点补码规格化小数表示时，所能表示的最小正数是 0.10000000000，即 1/2；所能表示的最大正数是 0.11111111111，即$(1-2^{-11})$，C 正确。规格化负数小数范围为[-1, -1/2)。

15．C。【解析】全加器有 3 个输入（2 个 bit 的输入位，1 个低位进位输入）两个输出（1bit 进位输出，1bit 求和结果），可以处理低位的进位。4 位二进制数相加需要 4 个全加器依次连接，低位全加器的进位输出作为高位全加器的进位输入，C 正确；半加器有 2 个输入（2 个 bit 的输入位，无低位进位输入）两个输出（1bit 进位输出，1bit 求和结果），半加器不考虑低位进位，不能完成多位二进制数相加，B、D 错误；单个全加器只能处理一位二进制数相加，A 错误。

16．B。【解析】并行进位加法器能并行生成各位的进位信号，大大减少了进位信号的传输延迟，逻辑门延迟更低，B 正确。

17．D。【解析】CPU 由控制器和运算器组成，指令译码由控制器的指令译码器完成，ALU 属于运算器，所以指令译码不属于 ALU 功能，D 错误。ALU 的功能有：定点数的逻辑、移位、算数运算。

18．B。【解析】运算器中通常都有状态标记寄存器，计算机中的条件转移指令正是根据状态标记寄存器中的状态位来判断是否执行转移指令，A 正确。

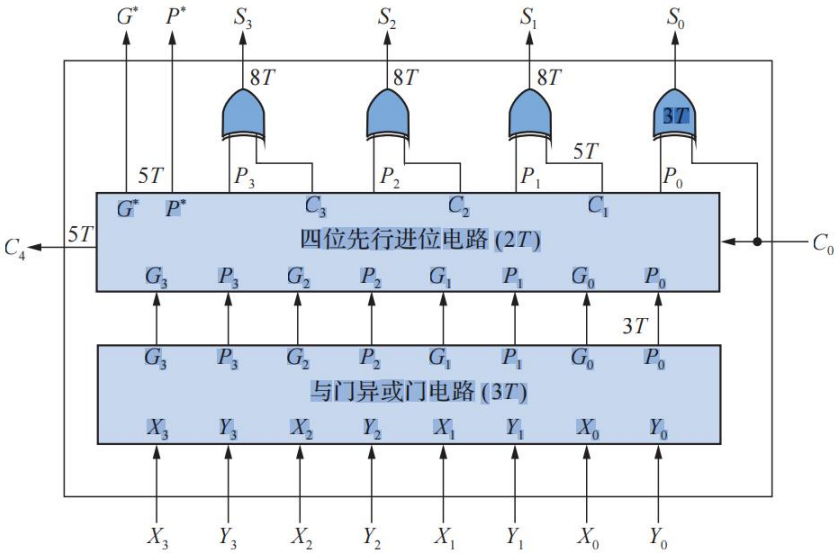
原码乘法中，符号才单独考虑，被乘数和乘数的符号位需要进行异或运算。补码乘法符号位参加运算，B 错误。

用 n 位全加器实现两个 n 位操作数各位同时相加，这种加法器称为并行加法器，高位的进位依赖于低位进位，C 正确。

在小数除法中之所以需要用被除数减去除数，其原因是使被除数的绝对值小于除数的绝对值以避免溢出，D 正确。

19．C。【解析】本题中 4 个选项均会对加法器的速度产生影响，但只有进位传递延迟对串行加法器的影响最为关键，C 正确。

20．D。【解析】四位并行加法器结构如下图所示，加法每位全和 Si 的形成除与本位相加两数数值位有关外，还与低位数送来的进位 Ci 有关，D 正确。



21．C。【解析】计算机加法器判断补码定点加减运算的溢出时，都涉及异或运算，因此溢出判断电路一般使用异或门实现，C 正确。请回忆 CF、OF、双符号位判断溢出各自的过程。

22．C。【解析】标志寄存器中的内容，是执行指令的过程中，CPU 根据指令执行的结果生成的，用户不能通过指令直接指定标志寄存器的编号或者修改它，C 错误。

23．A。【解析】补码减法转化为加法：减去一个数等于加上该数的反码+1（补码负数）
已知 X = 0101B， Y = 0011B。要计算 [X - Y]补,根据规则：[X - Y]补 = [X]补 + $\overline{Y}$+1。

做加法：0101 + 1101

丢弃最高位的进位 1，结果为 `0010`。

```
   0101
   1100
+  0001
———————
  10010
```

判断溢出：X 是正数，-Y 是负数，正数加负数绝对不可能发生溢出。（因为结果的绝对值必然小于两个加数的绝对值中较大的那个）。或者用进位信息判断，补码溢出位 OF=最高位进位⊕次高位进位=1⊕1=0，无进位。

24．A。【解析】减法 x - y 是利用加法器实现的，公式为：x + (按位取反的 y) + 1。

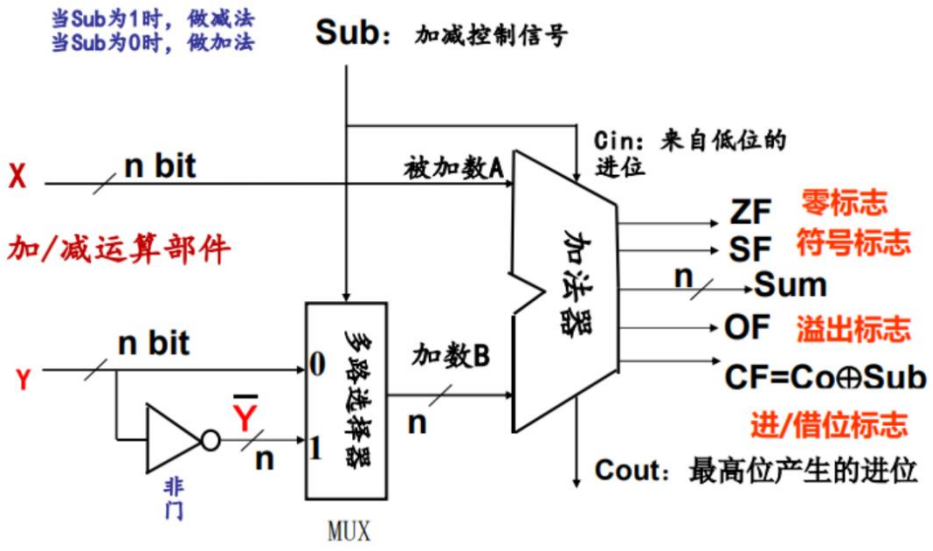
计算 x 的二进制：x = 69 = 64 + 4 + 1，8 位二进制为 0100 0101。这作为加法器的第一个输入端。

计算 y 的按位取反：y = 38 = 32 + 4 + 2，8 位二进制为 0010 0110。

对其进行按位取反（注意不是求负数补码，还没加 1），得到 1101 1001。这作为加法器的第二个输入端。

最末位的那个 "+1"：是通过直接将加法器的最低位进位输入 C_0 置为 1 来实现的。

整合起来，三个输入分别是：01000101，11011001，1，具体硬件设计如下图所示。



25．A。【解析】在原码除法不恢复余数法中，当余数为负时，商 0，然后将余数左移一位，加上除数的补码恢复余数，以便进行下一次运算，A 正确。

记忆：“正一减，负零加”：若中间余数为正数，上商为 1，下次做减法；若中间余数为负数，上商为 0，下次做加法。

26．C。【解析】除法运算涉及多次比较、移位和减法操作，其电路实现通常比乘法运算电路更复杂，C 正确；乘法运算电路除了通过加法器和移位器间接实现，还可以直接实现乘法电路，A 错误；减法运算电路通过补码转换为加法运算，都通过加法器实现运算，计算机中没有独立的减法器。全加器通过适当的输入设置（如利用补码）可以参与减法运算，D 错误。

27．A。【解析】将补码转换为真值进行运算，$(-2^{31})/(-2)=2^{30}$，A 正确。

补充：32 位补码除法，除数为 FFFFFFFFH，被除数为 80000000H，结果是？

将补码转换为真值进行运算，除数为 - 1，被除数为-2^{31}，商为2^{31}。32 位补码最大值为$2^{31}-1$，因此商超出范围，发生溢出

28．C。【解析】对于补码整数除法，除数寄存器 Y 初始值为除数的补码，余数寄存器 R 初始值为被除数符号拓展高 32 位，余数/商寄存器 Q 初始值为被除数符号拓展的低 32 位，C 正确。

> 进行 32 位原码定点小数除法时，已知被除数为 32 位原码，除数寄存器 Y 的初始值是（），余数寄存器 R 的初始值是（），余数/商寄存器 Q 的初始值是（）。
>
> 除数，被除数，32 位全 0。对于原码定点小数除法，除数寄存器 Y 初始值为除数的原码，余数寄存器 R 初始值为 32 位被除数，余数/商寄存器 Q 初始值全部补 0。

29．C。【解析】这题首先在 y 后面添 $0(y_{n+1})$，然后从后往前看，每次看两位，只有出现 $y_n y_{n+1}=10$，$部分积+[-x]_{补}$，可知出现两次。

补码一位乘法(Booth 算法)运算规则如下。

（1）符号位参与运算，运算的数均以补码表示。

（2）被乘数一般取双符号位参与运算，部分积取双符号位，初值为 0，乘法取单符号位。

（3）乘数末位增设附加位 y_{n+1}，初值为 0。

（4）根据$(y_n，y_{n+1})$的取值来决定操作，Booth 算法规则详细见下表。

y_n（高位）	y_{n+1}（低位）	操　作
0	0	部分积右移一位
0	1	部分积加[$X_{补}$]，右移一位
1	0	部分积加[$-X_{补}$]，右移一位
1	1	部分积右移一位

（5）进行算数右移

（6）按照上述算法进行 n+1 步操作，但第 n+1 步不再移位(共进行 n+1 次累加和 n 次右移)，仅根据y_n与y_{n+1}的比较结果做相应的运算，运算过程如下图所示

$[x]_{补}=11.0011$，$[-x]_{补}=00.1101$，$[y]_{补}=0.1011$。Booth 算法的求解过程如下。

(高位部分积)		(低位部分积/乘数)	说明
	00.0000	0.1011 0 丢失位	起始情况
$+[-x]_{补}$	00.1101		y_4y_5=10，y_5-y_4=-1，则$+[-x]_{补}$
	00.1101		
右移	00.0110	----- 10.1011 0	右移部分积和乘数
+0	00.0000		y_4y_5=11，y_5-y_4=0，则+0
	00.0110		
右移	00.0011	-----010.1011 0	右移部分积和乘数
$+[x]_{补}$	11.0011		y_4y_5=01，y_5-y_4=1，则$+[x]_{补}$
	11.0110		
右移	11.1011	----- 0010.1101 0	右移部分积和乘数
$+[-x]_{补}$	00.1101		y_4y_5=10，y_5-y_4=-1，则$+[-x]_{补}$
	00.1000		
右移	00.0100	----- 00010.1101 0	右移部分积和乘数
$+[x]_{补}$	11.0011		y_4y_5=01，y_5-y_4=1，则$+[x]_{补}$
	11.0111	构成$[x\cdot y]_{补}$	

30．B。【解析】原码一位乘共进行 n 次加法和 n 次逻辑右移，即 4ns + 2ns =6 ns，B 正确。

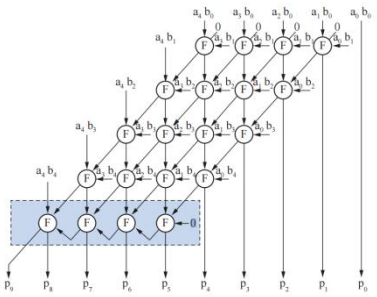
31．D。【解析】关于布斯补码一位乘法算法要点的描述中，错误的是 D 选项中的说法：“移位时，将进位位、部分积和乘积部分一起进行算术右移”。因为补码一位乘法的算法中，并没有专门的进位位，而符号位作为部分积的一部分进行处理。

32．C。【解析】A 正确，补码系统是计算机中带符号整数的主流表示方式，使用补码乘法器可以正确处理正负数相乘；B 正确，浮点数尾数通常用原码表示，如 IEEE754，尾数相乘可用原码乘法部件进行运算。C 错误，快速阵列乘法器中没有移位器，而是并行计算部分积，通过大量一位全加器构成全加器阵列累加求和，无需移位，如右图所示。时间复杂度总结：

一位 booth 乘法：$O(n^2)$

阵列乘法器：$O(n)$

两位 booth 乘法：位积个数减少一半 n/2，所以加法和移位次数都减少至 1/2，比 booth 一位乘法快 1 倍。



33．C。【解析】对于两个"位无符号整数除法运算，最大的商为 111…11/000…01=111…11，显然，运算结果没有产生溢出。因此，两个 n 位无符号整数除法运算肯定不会溢出，无需通过得到最高位商(第 n 位商q_n)来判断溢出，也即只要有"n 次循环执行“加/减”和“左移”操作，就能得到第 n-1~0 位的 n 位商(q_{n-1}~q_0)。因此，选项 C 的说法是错误的。

34．C。【解析】在用原码加、减交替法运算时，商的符号位是由除数和被除数的符号位异或来决定的，商的数值是由除数、被除数的绝对值通过加、减交替运算得的。由于除数、被除数取的都是绝对值，那么最终的余数应是正数。如果最后一步余数为负、则应将该余数加上除数，将余数恢复为正数，称为恢复余数，C 正确。

35．A。【解析】无符号整数减法可先其都看作补码然后再转换为补码减法，$X-Y=X+\bar{Y}+1=X+[-Y]_{补}$，-Y 的补码为 10101011B，故 X - Y = 10101010B + 10101011B = 01010101B，最高位进位舍去，结果为 01010101B，当然也可以直接用无符号整数列竖式做减法，只不过转换为补码会更加通

用，A 正确。

36．B。【解析】算术右移补符号位，原符号位为 1，移位后高位补 1，结果为 11011011，B 正确。

37．D。【解析】对于补码运算 S=A+B，有如下几种判断方式

方法一：OF=$A_sB_s\overline{S_s}+\overline{A_sB_s}S_s$

方法二：OF=$C_n \oplus C_{n-1}$，数值最高位进位和符号位进位异或

方法三：采用双符号位

- SIS2 =00：表示结果为正数，无滥出。
- SIS2 =01：表不结果正溢出。
- SIS2 = 10：表示结果负溢出。
- SIS2 = 11：表示结果为负数，无溢出。
- 溢出逻辑判断表达式为 V= $S_1 \oplus S_2$，若 V=0，表示无溢出。

无论是带符号数还是无符号数，都有两种解题思路。

方法一：进行机器运算，如选项 D，3BH+45H，最高位进位是 0，次高位进位是 1，所以 OF=$C_n \oplus C_{n-1}=1$。发生溢出。如果是做减法，$X-Y=X+\overline{Y}+1$，在判断标志位生成，用公式计算。机器不区分带符号数和无符 号数，只看生成的进位信息，按公式计算出 CF 和 OF。无论带符号数还是无符号数都是一样的运算规则。

方法二：用真值判断。还是以 D 为例子，3BH 补码的真值是 59，45H 的补码真值是 69，59+69=128，而 8 位补码的范围是［-128, 127］，128 超出了范围，所以溢出。此法要求对补码转真值比较熟练，以及 n 位补码或者无符号数的范围比较熟悉。综上，D 正确。

38．B。【解析】结果为 80000000H，溢出（正数加 1 变负数），故 SF=1，OF=1，ZF=0，B 正确。

39．A。【解析】对于补码减法运算，控制端 Sub 为 1，故低位进位输入位=Sub=1。$[x]_{补}=11110101$，$[y]_{补}=01111110$，$[-y]_{补}=10000001+1$，$[x]_{补}-[y]_{补}=[x]_{补}+[-y]_{补}=11110101+10000010=01110111$，进位丢掉，参与运算的两个数均为 1，结果的符号位为 0，故溢出标志 OF 为 1，A 正确。

40．B。【解析】尽管本题给出的是补码，但是在计算 CF 时，可以把 x 和 y 当无符号数看，容易知道机器数 x>y，所以对于无符号数，x-y>0，无借位所以 CF=0。在计算 OF 时，可以用异或运算来确定，$OF=C_n \oplus C_{n-1}$，注意本题的坑其实就在于确定 OF：如果用$[x]_{补}-[y]_{补}=[x]_{补}+[-y]_{补}=ECDF0000H+80000000H=\underline{1}6CDF0000, C_n=1, C_{n-1}=0, OF=C_n \oplus C_{n-1}=1 \oplus 0=1$，这就掉入了陷阱：计算机并不是直接计算$[x]_{补}+[-y]_{补}$，而是计算$[x]_{补}+[\bar{y}]+sub=ECDF0000H+7FFFFFFFH+1=\underline{1}6CDF0000$，这个过程$C_n=1, C_{n-1}=1, OF=C_n \oplus C_{n-1}=1 \oplus 1=0$，才是符合答案的。

41．D。【解析】无符号数比较大小，通过两个操作数相减得到，若 A=B，则 CF = 0，ZF=1，若 A 小于 B，A - B 有借位，CF=1.

CF=1，ZF=0。因此，判定 A 小于等于 B 的条件是 CF+ZF=1，D 正确。补充：

在无符号数情况下，各情况判定条件 F 如下：

- 等于：相减后结果为 0，即 $F = ZF$
- 大于：没有借位且相减后不为 0，即 $F = \bar{CF} \cdot \bar{ZF} = \overline{CF + ZF}$
- 小于：有借位且相减后不为 0，即 $F = CF \cdot \bar{ZF}$
- 大于等于：没有借位或相减后结果为 0，即 $F = \bar{CF} + ZF$
- 小于等于：有借位或相减后结果为 0，即$F = CF + ZF$

在带符号整数情况下，各情况判定条件 F 如下：

- 等于：相减后结果为 0，即 $F = ZF$
- 大于：结果不为 0，且不溢出时为正，溢出时为负。即$F = \bar{ZF} \cdot \overline{(SF \oplus OF)}$
- 小于：结果不为 0，且不溢出时为负，溢出时为正。即 $F = \bar{ZF} \cdot (SF \oplus OF)$
- 大于等于：结果为 0，或者不溢出时为正，溢出时为负。即 $F = ZF + \overline{(SF \oplus OF)}$
- 小于等于：结果为 0，或者不溢出时为负，溢出时为正。即 $F = ZF + (SF \oplus OF)$

42．B。【解析】若$A \le B$

- 情况 1：$A < B$，即$A - B < 0$。
  - 减法无溢出$OF = 0$。$SF = 1$ 代表实际结果为负，此时$ZF = 0$
  - 减法有溢出$OF = 1$，那么负数溢出后一定为正或者 0，补码中负数溢出不可能是 0，比如 8 位补码，要溢出到 0，必须是-128-128=-256，但是没有正数 128。所以$SF = 0$，$ZF = 0$
- 情况 2：$A == B$，即$A - B == 0$。此时$OF = 0$，$SF = 0$，$ZF = 1$

综上，一共三种情况（前两种是 A<B，第三种是 A==B）

- $OF = 0, SF = 1, ZF = 0$（无溢出且 A-B 结果为负）
- $OF = 1, SF = 0, ZF = 0$（负-正=正，溢出且溢出后为正）
- $OF = 0, SF = 0, ZF = 1$（无溢出且结果为 0）

因为在计算机中，ZF=1 时，表示结果全 0（$[0]_{补}$）

因为 ZF=1，此时符号 SF 必须是 0，又根据情况 1 的第二种子情况分析，OF 也必须是 0。所以 ZF=1 在计算机中就代表了$ZF = 1$，$OF = 0$，$SF = 0$，而前两种情况概括了 ZF=0 时的情况，用$OF \oplus SF$表示，因此答案选 B。

A、C、D 选项错误

举例：A-B=正-负=负，A>B，此时结果$SF = 1, OF = 1, ZF = 0$，但 A、C、D 选项结算结果都是 1，不能作为判断条件，不符合题意。

43．C。【解析】8 位补码的范围：[-128, 127]，x-y = 128 不能用 8 位补码表示出来，即发生溢出。

判断溢出一般推荐两种方法。

【方法一】用原码/补码的范围来判断。

- 应用场景：①真值②给出的二进制机器数好转真值，如：FFFFH③乘除法无脑用真值
- 步骤：

先写出 n 位原码/补码的范围，比如这题是判断补码溢出（OF），补码范围就是$[-2^{n-1},2^{n-1}-1]$

用真值运算得出结果（十进制加减乘除）

如果是给出机器数，那么原码运算或者判断 CF，就把机器数转原码真值进行运算

如果是给出机器数，那么补码运算或者判断 OF，就把机器数转补码真值进行运算

看结果是否超出表示范围

如果是补码运算或者判断 OF，看是否超过补码范围，如这题很容易知道 C 不在范围内

如果是原码运算或者判断 CF，看是否超过原码范围

【方法二】机器数运算通过逻辑运算判断。

- 应用场景：给出的二进制机器数不好转真值，如：2345H
- 步骤：
  - 加法：$x+y$ 直接进行二进制竖式运算
  - 减法：$x-y=x+\bar{y}+1$
  - 判断：
  - 无符号数溢出：$CF=C_n\oplus C_{in}=C_n\oplus Sub$
  - 带符号数溢出：$OF=C_n\oplus C_{n-1}$

44．C。**【解析】**常用的溢出判断方法主要有三种：采用一个符号位、采用进位位和双符号位。

采用一个符号位的溢出条件为，结果的符号位与操作数符号位不同。

采用进位位判断溢出的条件为：结果的符号位和最高数位进位情况相反。

采用双符号位的溢出条件：运算结果的两个符号位不同，故 C 正确。

45．C。**【解析】**补码规格化尾数最高位为符号位，要求数值部分绝对值 ≥ 0.5。

当尾数为正数时，补码与原码相同，最小的规格化正数尾数是 0.100…00（8 位，含符号位），即 2^{-1}；最大的规格化正数尾数是 0.111…11，其值为 $1-2^{-7}$。故 C 正确。

注意：规格化数的负的尾数的范围是$[-1,-1/2)$。负数中虽然-1/2 绝对值大于等于 1/2，但是为了便于规格化判断电路设计（符号位和数值位最高位异或），不保留 1.100...，即-1/2。如果是模 r 原码或者补码表示尾数，尾数部分数值绝对值≥1/r（如模 4 原码就是 1/4，需要满足数值位最高两位不能全 0，补码负数也不能取到-1/r）。

46．A。**【解析】**尾数的数值位数为 16-1-5-1=9 位，尾数一共为 10 位，用补码表示，其可以表示的最大小数为 $1-2^{-9}$。

阶码一共 6 位，可以表示的最大值为$2^5-1=31$。故浮点数的最大值为$2^{31}\times(1-2^{-9})$，A 正确。

47．C。**【解析】**当阶码是以 8 为基数时，尾数 M 的数值绝对值大小≥1/8。即：1/8<=M<1 或 -1<=M<=-1/8，注意补码表示时，是取不到负数的-1/8 的，因为通常在补码判定规格化数时，采用最高位 3 位不能全部都是符号位的取反位，所以为了简化判断电路，通常 11，111000...不把他当作规格化数。

48．C。。**【解析】**阶码 0110B，移码表示，其真值为 0110 - 1000 = -2（移码真值等于移码减去

偏移量 2^{n-1}，n 为阶码位数，注意此处不是 IEEE 754 标准，不要误把偏移量减一）。尾数 11010000B，补码且规格化（补码尾数规格化形式：符号位与最高数值位相反），其真值为 -0.0101B。所以浮点数真值为 -0.0101B× 2^{-2}，C 正确。

49．B。【解析】在 IEEE 754 标准浮点数表示中，当阶码全为 1，尾数全为 0 时，若符号位为 0，则表示正无穷大；若符号位为 1，则表示负无穷大。本题符号位为 1，所以表示负无穷大，B 正确。

50．B。【解析】

在 IEEE 754 标准浮点数表示中，当阶码全为 1，尾数不全为 0 时，表示非数 NaN，即没有定义的数，B 正确。

符号位 S（1 位）	阶码 E（8 位）	尾数 M（23 位）	浮点数真值	说明
0/1	255	$M\neq0$	NaN 非数	运算异常（0/0）
0/1	255	0	+∞/−∞	正、负无穷
0/1	1 ～ 254	M	$(-1)^s\times2^{E-127}\times1.M$	规格化数
0/1	0	$M\neq0$	$(-1)^s\times2^{-126}\times0.M$	非规格化
0/1	0	0	+0/−0	两个机器 0

- 阶码 0<E<255，表示规格化数，格式：$(-1)^s\times(1).f\times2^{E-127}$
- 阶码全 0，尾数全 0，表示±0；
- 阶码全 0，尾数不全为 0，非规格化数，$(-1)^s\times(0).f\times2^{-126}$
- 阶码全 1，尾数全 0，表示±∞；
- 阶码全 1，尾数不全为 0，非数（NaN），尾数最高位为 1，不发异常通知，尾数最高位为 0，发异常通知（低电平有效），后 22 位表示异常类型。

51．B。【解析】浮点数运算对阶时，遵循小阶向大阶对齐，小阶的阶码增大，尾数相应要减小，那么尾数要右移在过程中可能会丢失低位数据，导致精度损失，B 正确；浮点数的表示范围取决于阶码和尾数的位数，A 错误； 规格化的浮点数补码表示的负数尾数数值最高位为 0，C 错误；浮点数加减法运算必须考虑符号位，因为需要符号决定 ALU 操作（如加或减）、结果符号以及运算过程，D 错误。

52．C。【解析】IEEE 754 双精度浮点数使用 11 位指数，其偏移值为$2^{10}-1=1023$，C 正确。用这个题目提醒大家记一下 double 的编码格式，尾数部分是 64-11-1=52 位。

53．C。【解析】在 IEEE 754 标准中，单精度浮点数的非规格化数是指阶码为零的数，用于表示非常小的浮点数。这种数的阶码部分不存储隐含的“1”，与规格化数不同。单精度浮点数总共占 32 位，其中 1 位符号位，8 位阶码位，23 位尾数位。

对于非规格化数：

- 阶码部分全为 0（对应的是一个偏移量为 -126 的实际指数）。
- 尾数部分不存隐含的“1”，因此实际尾数为 0.f。

最大非规格化数发生在尾数部分全为 1 的情况下，即：$2^{-126}\times(1-2^{-23})$

这是可以表示的最大的非规格化浮点数。

思考：最小非规格化正数是多少？

在 IEEE 754 标准中，单精度浮点数的最小非规格化数是指阶码全为 0（表示指数为 -126）的情况下，尾数部分只有最低位为 1，其它位为 0。

非规格化数的表示形式为：

$$非规格化数 = (-1)^{符号位} \times 2^{-126} \times 0.尾数$$

最小非规格化数对应的是尾数部分全为 0，只有最低位为 1：

$$最小非规格化数 = 1 \times 2^{-126} \times 2^{-23}$$

因此，最小的非规格化数为：

$$最小非规格化数 = 2^{-126} \times 2^{-23} = 2^{-149}$$

表 2.7　IEEE754 浮点数表示范围

格式	绝对值最小数	绝对值最大数
单精度规格化 $(-1)^s \times 1.M \times 2^{e-127}$	E_{min}=1，M=0 $f = 1.0 \times 2^{1-127} = 2^{-126}$	E_{max}=254，M=1111…11 $f = 2^{254-127} \times (2-2^{-23}) \approx +3.4 \times 10^{38}$
单精度非规格化 $(-1)^s \times 0.M \times 2^{-126}$	E=0，M=0000…01 $f = 2^{-23} \times 2^{-126} = 2^{-149}$	E=0，M=1111…11 $f = (1-2^{-23}) \times 2^{-126}$
双精度规格化 $(-1)^s \times 1.M \times 2^{e-1023}$	E_{min}=1，M=0 $f = 1.0 \times 2^{1-1023} = 2^{-1022}$	E_{max} = 2046，M=1111…11 $f = 2^{2046-1023} \times (2-2^{-52}) \approx +1.8 \times 10^{308}$
双精度非规格化 $(-1)^s \times 0.M \times 2^{-1022}$	E = 0，M=0000…01 $f = 2^{-52} \times 2^{-1022} = 2^{-1074}$	E=0，M=1111…11 $f = (1-2^{-52}) \times 2^{-1022}$

54．A。【解析】阶码 0<E<255，表示规格化数，格式：$(-1)^s \times (1).f \times 2^{E-127}$。阶数最大值是 254-127=127，尾数部分能表示的最大值是 $2-2^{-23}$，故答案选 A。

55．B。【解析】规格化数的阶码机器数范围[1, 254]，减去偏置 127，阶码真值范围[-126, 127]。且规格化数的表示格式是 $1.f \times 2^{E-127}$，$f \in [0,1-2^{-23}]$，所以规格化数范围是：$[2^{-126},(2-2^{-23}) * 2^{127}]$，即$[2^{-126},2^{128}-2^{104}]$，负数是镜像区间$[-(2^{128}-2^{104}),-2^{-126}]$

非规格化数的阶码是全 0，尾数 f 不全为 0，表示格式为 $0.f \times 2^{-126}$，$f \in [0,1-2^{-23}]$，所以非规划数的范围是：$[2^{-23} * 2^{-126},(1-2^{-23}) * 2^{-126}]$，即$[2^{-149},2^{-126}-2^{-149}]$，负数是镜像区间$[-(2^{-126}-2^{-149}),-2^{-149}]$。

题目问的非规划的最小负数，因此答案选 B。

56．A。【解析】非规格化数条件：阶码全 0 + 尾数非全 0，尾数有 23 位，隐含符号位固定为 0，故共2^{23}种可能，但需排除全 0（全 0 表示 0），故数量为$2^{23}-1$，A 正确。

57．C。【解析】因为将阶码部分的移码表示改为补码表示，并不会使数的表示范围发生变化，只会使阶码的表示形式发生变化，C 正确。

58．C。【解析】浮点数的正负由尾数的正负符号决定，阶码的正负符号只影响数的绝对值的大小，A 错误；在并行加法器中，高位的进位依赖于低位，B 错误。为提高运算速度，运算器一般都采用并行加法器，D 错误。

59．D。【解析】一个正数的补码和反码均和这个数的原码表示一样，A 错误。

表示定点数时，若要求数值 0 在计算机中唯一地表示全 0，应使用补码表示，B 错误。

01.1010 是一个正溢出数，C 错误。

60．D。【解析】原码规格化后，正数为 01x...x 的形式，负数为 1.1x...x 的形式。补码规格化后，正数为 0.1x..x 的形式，负数为 1.0x...x 的形式，D 错误。

61．A。【解析】5 转换成二进制值为 101，在 IEEE754 中规格化表示为 1.01×2^2，阶数=127+2=129。

故 IEEE754 编码为：01000000101000000000000000000000，A 正确。

62．C。【解析】41A4C000 写成二进制是 0100 0001 1010 0100 1100 0000 0000 0000，所以阶码真值是 10000011-01111111=4，2 的 4 次方等于 16，所以该浮点数的十进制肯定大于 16，C 正确。

63．C。【解析】阶码越大，数的范围越大；尾数越长，数的精度越高，C 正确。

64．C。【解析】表示的数的个数跟总的状态数（离散状态个数）相关，同时 32 位，阶码和尾数长短不一，影响的是范围和精度，影响不了表示数的个数，在不考虑其他特殊情况时，总个数都是2^{32}个。

65．A。【解析】float 的范围是 -3.4E+38～3.4E+38，比 int 的大，所以不会溢出，但是有可能因为精度问题出现舍入，A 错误。double 是双精度浮点数，范围>float>int，B 正确。double 变 float，范围变小、精度也变小，可能会溢出，也可能舍入，C 正确。double 转换为 int，只要不是整数就会舍入，D 正确。

66．A。【解析】就近舍入中间值向两边舍入取决于尾数最后一位，若是 0，则直接截断尾数，若是 1，则要把尾数最后一位变成 0（偶数），所以采取进 1 的方式，也叫朝偶数舍入法。中间值左侧朝左侧舍入(一半误差为负)，中间值右侧朝右侧舍入（一半误差为正），平均误差为 0。IEEE 754 默认采用就近舍入法。

正向舍入平均误差为+0.5，负向舍入平均误差为-0.5，截断舍入不确定正负，但是误差绝对值是 0.5，因此本题选 A。

67．D。【解析】浮点数加、减运算一般包括对阶、尾数运算、规格化、舍入和判溢出等步骤。

第一步，对阶：第一个数$X=2^7\times29/32$，浮点数格式为 001110011101，第二个数$Y=2^5\times5/8$，浮点数格式 001010010100。对阶原则是小阶向大阶看齐，My 右移两位，Ey+2，浮点数格式为 001110000101

第二步，尾数相加：Mz=Mx+My=0100010，浮点数格式为 001110100010。

第三步，结果规格化：尾数需要进行一次右规，才能变成规格化数，Mz 右移一位，Ez+1，浮点数格式为 010000010001

第四步，判溢出：由于阶码符号位不同，所以发生溢出，D 正确。

68．D。【解析】$[\triangle E]_{补}=\left[E_x-E_y\right]_{补}=[E_x]_{移}+\left[-\left[E_y\right]_{移}\right]_{补}$。假定$[\triangle E_x]_{移}$、$\left[-\left[\triangle E_y\right]_{移}\right]_{补}$、$[\triangle E]_{补}$的最高有效位分别记为为$E_{xs}$、$E_{ys}$和$E_{bs}$，若△E≥128，$[\triangle E]_{补}$发生正溢出，此时$E_{bs}$为1，而$E_x$一定为正数，即$E_{xs}=1$，而 E_y 一定为负数，即$\left[E_y\right]_{移}$的符号位为 0，计算$\left[-\left[\triangle E_y\right]_{移}\right]_{补}$时，对$\left[E_y\right]_{移}$的各位取反，末位加 1，从而得到$E_{ys}=1$，综合起来的逻辑表达式就是$E_{xs}E_{ys}E_{bs}$；若△E≤-129，则$[\triangle E]_{补}$发生负溢出，此时$E_{bs}=0$，而 E_x 一定为负数，即$E_x=0$，而 E_y 一定为正数，即$\left[E_y\right]_{移}$的符号位为

1，计算$\left[-\left[\triangle E_y\right]_{移}\right]_{补}$时，对$\left[E_y\right]_{移}$的各位取反，末位加 1，从而得到$E_{ys}=0$，综合起来就是$E_{xs}^{-}E_{ys}^{-}E_{bs}^{-}$，因此选 D。

69．C。【解析】A 错误，IEEE754 里面，左规格化过程中，尾数增大阶码变小，当阶码变成 0 的时候，进入非规格化数的范围，只有小到非规格化数的也无法表示，才会被当作机器 0。非规格化数存在的意义就是逐级下溢。B 错误，对阶是阶码小的往阶码大的对齐，最多等于原来的大阶码，所以阶码不可能发生溢出。C 正确，右规划过程中，尾数变小阶码变大，有可能会导致阶码上溢。D 错误，舍入操作可能会导致进位，进位如果传递到最高位，就需要右规一位，然后阶码+1，这时候阶码可能上溢。例如 1.111... 舍入成 10.000...，需要右规格化，阶码加 1，可能导致阶码上溢。

70．C。【解析】I 正确，定点数在运算（如乘法或除法）时可能产生多余的小数位，需要舍入来适应固定位数，因此舍入不仅适用于浮点数。II、III、IV 错误，在对阶和右规格化中需要舍入。V 正确，比如尾数溢出部分是 0，采用截断法，并不产生误差。

71．B。【解析】对于 IEEE754 单精度浮点数加减运算，若对阶时得到的两个阶码之差的绝对值$|\Delta E|$等于 24，则说明阶小的那个数的尾数需要右移 24 位，进行尾数加减运算时，虽然其结果的前 24 位直接取阶大的那个数的相应位，但是由于可以保留附加位，阶小的那个数右移后的尾数可能会在舍入时向前面一位进 1。

例如 $1.00\ldots01\times2^{1}+1.10\ldots00\times2^{-23}=1.00\ldots01\times2^{1}+0.00\ldots00$（11）$\times2^{1}=1.00\ldots01$（11）$\times2^{1}$。其中 0.00..00（11）后面两个 11 是附加位，结果 1.00.01（11）中后两位也是附加位（小数点 23 位后的 24，25 位），最终需要根据这两位进行舍入，舍入后最终答案是 1.00..10×2 其中若两个阶码之差的绝对值$|\Delta E|$等于 25，则保留的附加位中，虽左边的一位一定是 0，采用就近舍入时，完全被丢弃。B 正确。

72．B。【解析】本题主要考察 float 的下溢问题。$a=\frac{33}{32}\times2^{-126}, b=1\times2^{-126}, a-b=\frac{1}{32}\times2^{-126}$，此时浮点数是非规格化浮点数，即 float 的阶码变为全 0，隐藏位由 1 变成 0。结果为 00040000H。float 阶码全 0 就是为了在下溢的时候仍能参加运算，并不是直接当成 0 处理，所以 d=c+b=a-b+b=a，结果为 true，B 正确。

73．C。【解析】对阶从小到大，7 到 9，尾数右移 2 位，相当于小数点左移 2 位，C 正确。

注意：题干中问的是小数点移位的次数，而不是尾数移位的次数。

74．A。【解析】先将两数化为数学表达式。x 的阶码为 10000110B=134，尾数为 1.01100100010B，那么 x= 1.01100100010$B\times2^{134-127}=1.01100100010B\times2^{7}$。y 的阶码为 10000010B=130，尾数为 -1.10011B，那么 y=$-1.10011B\times2^{130-127}=-1.10011B\times2^{3}$。接着进行对阶，将小阶向大阶对齐，此时 y=$-0.000110011\times2^{7}$。

接着对尾数进行运算，x-y=1.01100100010+0.000110011=1.0111110111，经判断，此时不需要规格化和舍入，阶码也没有进位，不溢出。故 x-y= 1.0111110111×2^{7}，转化为 IEEE754 格式为 433EE000H，A 正确。

75．A。【解析】对阶原则是小阶向大阶看齐，Y 的阶码为 3，X 的阶码为 5，Y 的阶码要变为 5，阶码增加 2，尾数右移 2 位，Y= $2^{3}\times(-0.1010B)$ 变为 $2^{5}\times(-0.001010B$），A 正确。

76．C。【解析】加法过程如下：

- 对阶：$[\Delta E]_{补}=[E_x]_{移}+[-[E_y]_{移}]_{补}(mod2)=10100000+01111000=00011000$，x 的阶码比 y 大 24，小阶向大阶对齐，y 的尾数M_y右移 24 位，数值高位补 0，隐藏位右移到小数点后面，最后移出的位保留两位附加位，即$M_y=0.000\ldots00(10)$（括号内表示移到附加位上的两位）
- 尾数相加：$M_x+M_y=1.010100\ldots01+0.000\ldots00(10)=1.010100\ldots01(10)$
- 规格化：已经是规格化，无需操作
- 舍入：附加位上为(10)，为中间数，就近舍入，需要向偶数对齐，即看尾数最后一位是否为 1：
  - 尾数最后 1bit 要是 1，表示是奇数，需要进 1 变偶
  - 尾数最后 1bit 要是 0，已经是偶数，直接舍弃。

本题最后一位是 1，需要进位，舍入后尾数为 $1.010100\ldots10$

- 溢出判断：舍入后，仍为规格化数，阶码无溢出。

由以上过程，I、III 正确，II 错误。

我们可以看到，y 的真值为 $1.0\times2^9=512$，舍入后，进位的原因导致2^9变成了2^{10}，所以实际结果为$z-x=x+y-x=2^{10}=1024!=512$，IV 为 false。本题答案为 C。

思考：y=511 的话，即机器数为 43FF 8000H，z-x==y 为真吗？（还是 false，此时附加位为（01），直接舍去，z-x=0 了），本题说明 float 两数相加时，阶码之差$[\triangle E]\geq25$，较小数可以直接忽略。

77．D。【解析】x 的机器数形式为 0001FFFAH，其长度 32 位。将长的数赋给短的数，短的数将会从低位开始保存，超出长度的部分直接丢弃。故 v 的机器数形式为 FFFA，计算机中的数据使用补码形式保存，将其转换为原码后为 8006，故 y=-6，D 正确。

78．C。【解析】f 是含有小数位的，（int）f 会导致小数位的丢失。

79．C。【解析】首先 char c = -1 机器码为 FFH。

由 char 类型转换为 unsigned int 类型时，符号扩展填充高位 1 至 32 位：FFFF FFFFH，再解释为无符号数，得到 u = $2^{32}-1$，转换为二进制是 $1.11111111111111111111111111111111\times2^{31}$。

由 unsigned int 类型转换为 float 类型时，符号位为 0，阶码为 31，移码表示为 1001 1110，尾数部分，因为 IEEE 754 单精度标准中可表示精度位 24 位（23 位尾数 + 1 位隐藏位），小于 u 中的 32 位，故需要进行舍入处理。c 语言中默认舍入规则是向最接近的偶数舍入，即

- 如果要舍弃的部分的最高位为 0，则直接截断。
- 如果要舍弃的部分的最高位为 1，且剩余部分不为 0，则进位。
- 如果要舍弃的部分的最高位为 1，且剩余部分为 0，则看保留部分的最低位：如果为偶数，即为 0，则直接截断，如果为奇数，即为 1，则进位。

对于本题要舍弃的部分即 u 中 32 位补码的后 8 位，最高位为 1 且剩余部分不为 0，故保留前 24 位并进位。即 1111 1111 1111 1111 1111 1111 + 1 得到 1 0000 0000 0000 0000 共 25 位，需要进行右规并继续舍入，右归时阶码加 1，得到阶码为 1001 1111，舍入时，舍弃的部分为最后一位，最高位为 0，故直接截断，从而结果为 0 1001 1111 （1）000 0000 0000 0000 0000 0000，即 f =2^{32}。

最后由 float 类型转换为 double 类型，double 的精度和范围都比 float 大，所以真值不变，i = 2^{32}，C 正确。

80．A。【解析】short s = 0x7FFF 即 $2^{15}-1=32767$，转换为 double 时，数值在双精度范围内可精确表示，d 的值仍为 32767。double 的 32767 转换为 int 时直接截断小数部分，结果为 32767，A 正确。

81．B。【解析】数据对齐存放可以使 CPU 在存取数据时，以更高效的方式访问内存，减少内存访问次数，提高数据存取速度；虽然有时可能节省部分空间，但主要目的是提升速度，B 正确。

82．B。【解析】short 类型变量 a 占 2 字节，为满足 4 字节对齐，a 后需填充 2 字节；int 类型变量 b 占 4 字节，所以结构体大小 $=2+2+4=8$ 字节，B 正确。

83．C。【解析】操作码：001010（二进制），rs（源寄存器）：R [6] → 编号 6H（二进制 0110），rd（目标寄存器）：R [5] →编号 5H（二进制 0101），imm（立即数）：48 →二进制补码 0000 0000 0011 0000（18 位，正数补码与原码相同），拼接后为 0010 1001 1001 0100 0000 0000 0011 0000B，即为 29940030H。又因为按照小端方式存储，故在内存中存储的机器码是 30009429H，C 正确。

84．B。【解析】$(-4.25)\times 2^{17}$ 转换成机器数为：C9080000H，按小端方式存放，则存储单元 FFFFC002H 中存放的是 08H，B 正确。

85．C。【解析】float 数据类型占 4 字节，数组中最后一个元素的地址为：C0001000H+99×4=C000118CH，由于采用小端方式存储，MSB 在内存的最高地址处，也就是 C000118FH，C 正确。

86．B。【解析】边界对齐是计算机系统中一种数据存储原则，要求数据在内存中存储的起始地址与其数据类型的大小成倍数关系，B 正确。

这样虽然浪费了一些存储空间，但可提高取指令和取数的速度。边界对齐方式相对边界不对齐方式是一种空间换时间的思想。精简指令系统计算机（RISC）通常采用边界对齐方式，因为对齐方式取指令时间相同，因此能适应指令流水。

87．C。【解析】结构体变量的对齐规则

- 整个结构体变量的对齐方式与其中对齐方式最严格的成员相同。
- 结构体成员在满足其对齐方式的条件下，取最小可用位置作为成员在结构体中的偏移量。
- 结构体大小应当为对齐边界长度的整数倍(结构体的大小必须是其最大成员大小的倍数，以保证结构体在数组中也是对齐排放。)
- 如果嵌套了结构体，嵌套的结构体对齐到自己的最大对齐数的整数倍处，结构体的整体大小就是最大对齐数(含嵌套结构体的对齐数)的整数倍。

回到本题，int 型数据按 4B 对齐，char 型数据按 1B 对齐，short 型按 2B 对齐，而 B 结构体由于内部包含 int、char 和 short，所以应该按照对齐规则最严格的 int 对齐规则来对齐，即按 4B 对齐，且结构体 B 的大小也是 4 的倍数，为 12B。

结构体 A 中包含 B，A 的对齐宽度是成员变量的最大对齐宽度，也就是结构体 B 的对齐宽度 4B（对齐宽度是 B 中按 int 对齐的 4B，不是 B 的大小，注意区分）。所以整个排布如下图所示。e 的地址

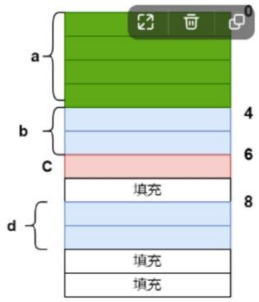
为 C010H。

```
struct Inner{
    char a;
    int b;
    short c;
}
struct Outer{
    short d;
    Inner inner;
    short e;
}
```

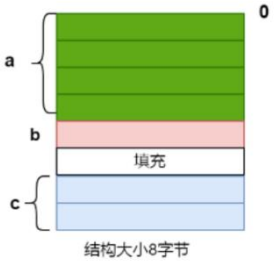


其他一些举例：

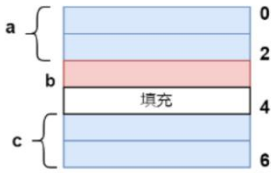
```
struct S2{
    int a;
    short b;
    char c;
    short d;
}
```



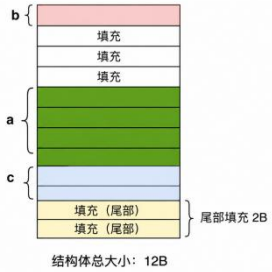

```
struct S3{
    int a;
    char b;
    short c;
}
```




```
struct S1{
    short a;
    char b;
    short c;
}
```




```
struct S4{
    char b;
    int a;
    short c;
}
```




```
struct A{
    int a;
    double c;
}
struct B{
    char e;
    struct A d;
    int f;
}
```



88．D。【解析】C 语言数据的边界对齐规则如下：

结构体对齐总结：

- 成员变量为 0 的结构体占 1B
- 结构体本身对齐按最大成员变量的宽度对齐
- 内部成员本身按自己的数据类型对齐
- 结构体的大小是对齐宽度的整数倍

本题结构体成员排列如下，结构体大小必须是 int 的整数倍也即 12B。

0xC000	0xC001	0xC002	0xC003
a	a		
0xC004	0xC005	0xC006	0xC007
b	b	b	b
0xC008	0xC009	0xC00A	0xC00B
c			

思考：若 int 和 char 交换位置，则结构体大小是多少？ 8

89．B。【解析】C 语言类型转换规则：

- 同长度无符号数和有符号数转换，遵循机器数不变转变成对应真值，即补码转补码真值，无符号数转无符号数真值。
- 短转长，看短的带不带符号。短是带符号就符号扩展，短是无符号就 0 扩展到长的位数，再根据长的变量类型转变成对应真值，这道题短的就是无符号数，所以 0 扩展就行了。
- 长转短，直接截断

思考 1：以下代码 y 的机器数是多少?

```
short x= 0xABCD;
unsigned int y=x;
```

（短的是带符号数，就进行符号扩展，符号位是 1，高位补 1，答案为 FFFFABCDH）

第 3 章

存储器层次结构

【答案速对】

CCACB	CADDC	DCBAB	DCCCD
CCCCA	DABBA	BCBCB	BCDCB
BBABC	BCBBD	ABACC	CCCCD
ACBAC	BBDAC	AAABC	ACDC

1．C。【解析】SRAM（静态随机存取存储器）是利用半导体器件的双稳态电路来存储信息，属于半导体存储器，C 正确；机械硬盘是磁表面存储器，一般又称磁盘，A 错误；光盘是光存储器，B 错误；磁带也是磁表面存储器，D 错误。

注意：固态硬盘属于半导体存储器，其核心存储介质为闪存，不同于机械硬盘。

2．C。【解析】A 选项错误：固态硬盘（SSD）是基于闪存技术（Flash Memory），属于半导体介质 。传统的机械硬盘（HDD）才是磁表面存储器 。

B 选项错误：RAM 和 ROM 确实都采用随机存取方式 ，但并非所有半导体存储器都是按地址随机访问。例如快表（TLB）和全相联 Cache 的 Tag 是按内容访问（CAM）的半导体存储器。

C 选项正确：CD-ROM 属于光存储器，光头可以先定位区域（直接）再顺序查找（顺序），这种结合的模式属于直接存取（DAM）。

D 选项错误：通用计算机的主存由 RAM 和 ROM 共同组成，包含存放系统启动程序的 ROM 区（BIOS），所以“全部由 DRAM 构成”是错误的 。

3．A。【解析】Cache（高速缓存）可以随机地对任何存储单元进行存取操作，属于随机存取存储器；顺序存取存储器如磁带，只能按顺序依次访问；直接存取存储器如磁盘，先直接定位到存储区域，再顺序访问；只读存储器只能读取数据。

4．C。【解析】A 选项错误： 传统的机械硬盘（HDD）依靠磁盘盘片和读写磁头工作，属于磁表面存储器 ；而固态硬盘（SSD）是基于闪存技术（Flash Memory），属于半导体存储器 。两者介质完全不同。

B 选项错误： 这道题的陷阱在于后半句：DRAM 是以电容充电原理寄存信息的，电荷会逐渐泄漏，因此必须定期进行刷新；相反，SRAM 是以触发器原理寄存信息的，不需要定期刷新。

C 选项正确： 我们常用的固态硬盘或 U 盘所使用的闪存（Flash Memory），属于半导体存储

器 。它在技术分类上属于电可擦除的 ROM，即 EEPROM ()。

D 选项错误： 前半句是正确的，CPU 内部的通用寄存器组（以及 Cache、主存等）都属于半导体存储器 ，用来存放地址、数据及运算的中间结果。但后半句错误：寄存器属于易失性存储器，一旦断电，内部存储的高低电平状态就会消失，无法永久保存数据。

5．B。【**解析**】DRAM（动态随机存取存储器）利用电容存储电荷来表示信息，由于电容会漏电，需要定期刷新；基于触发器存储信息的是 SRAM；基于磁性材料存储信息的有硬盘、磁带等；基于光反射原理存储信息的是光盘，A、C、D 错误。

6．C。【**解析**】A.半导体 RAM 信息可读可写，但断电后不能保持记忆，A 错误。

B.动态的 RAM 和静态的 RAM 都是易失性存储器，B 错误。

D.ROM 不用刷新，但集成度不比动态 RAM 高，且断电后存储的信息仍能保持，D 错误。

7．A。【解析】S 区分存取周期（T_C）与存取时间（T_A）。CPU 连续访问的最小时间间隔是存取周期，而存储器带宽的计算也必须使用存取周期 。

存取周期计算：$T_C = T_A + T_R = 30ns + 20ns = 50ns$ 。

每秒最大连续访问次数：$1/(50 \times 10^{-9}s) = 2 \times 10^7$ 次。

带宽计算：数据位宽 W = 128 位 = 16 字节(Byte) 。

理论带宽 $B_m = \frac{W}{T_C} = \frac{16Byte}{50\times10^{-9}s} = 3.2 \times 10^8 B/s = 320MB/s$

8．D。【**解析**】相联存储器是指按照内容寻址的存储器，故 D 正确。

9．D。【解析】A 错误，在某些特定的写策略（如 Write Back 写回法）下，Cache 中的最新数据在未被替换前，主存中的对应数据是旧的，两者并非严格的包含关系（主存和辅存之间也是这样，主存采用写回时，这两数据也有可能不一致）；B 错误，速度差异只能“缓解”或“掩盖”，无法“彻底消除”；C 错误，系统的整体访问速度应接近最高层（Cache 或寄存器）而非主存；D 正确，系统的容量和单位成本接近外存（最下层）。

10．C。【解析】A 错误，Cache-主存完全由硬件管理；B 错误，主存-辅存（虚拟存储器）是由操作系统和硬件结合管理的，且对系统程序员不透明（系统程序员需要编写缺页处理程序等）；D 错误，虚拟存储器对应用程序员透明，但对系统程序员是不透明的；C 准确描述了 Cache 层级的交换单位和目的。

11．D。【解析】D 错误：两个层次在“透明性”上是不同的。Cache-主存层次的数据交换完全由硬件自动完成，对所有程序员（无论是应用程序员还是系统程序员）都是透明的；而主存-辅存层次（虚拟存储器）的数据交换由硬件和操作系统共同完成，它只对应用程序员透明，但对系统程序员是不透明的（因为系统程序员需要编写操作系统中的缺页中断处理、页面替换等底层硬件调度代码）。

A 正确： 准确描述了两者的引入目的。Cache 用于提速（填补 CPU 和主存间的速度鸿沟），虚拟存储器（主存-辅存）用于扩容。

B 正确： 准确描述了 CPU 的访问权限。Cache 和主存属于内部存储器，CPU 可直接寻址访问；辅存属于外部存储器，必须经过主存作为中转。

C 正确： 准确描述了映射关系的实现介质。Cache 为了极致的速度，映射和查找全靠纯硬件电路

（比对 Tag）；而主存-辅存层次的缺页代价大，由软件（操作系统）维护的页表（Page Table）来实现虚拟地址到物理地址的映射。

对比维度	Cache - 主存层次	主存 - 辅存层次
位置与访问权限	最靠近 CPU。CPU 可**直接访问** Cache 和主存。	远离 CPU。CPU **不能直接访问**辅存，辅存需与主存直接交换数据。
引入目的	**解决速度问题**：**加快 CPU 访问信息的速度。**	**解决容量问题**：采用虚拟存储器机制，**扩大系统的存储容量**，使编程不受内存大小限制。
交换的数据单位	称为**主存块 (Block)**，通常较小，一般为 **8 ~ 128B**。	称为**页 (Page)**，通常较大，一般为 **4KB ~ 64KB**（因缺页损失远大于 Cache 缺失，页太小会降低效率）。
缺失处理	由处理器（**硬件**）直接实现。	由操作系统（**软件**）实现。
映射方式	可灵活选择**直接映射、全相联映射或组相联映射**。	均采用**全相联映射**方式。
映射的实现介质	映射关系完全由**硬件**实现，使用 Cache 行中的**标志 (Tag)** 字段描述。	映射关系由**操作系统**实现，使用**页表**来描述。
写策略	可以采用**直写 (Write-through)** 和 **回写 (Write-back)** 两种策略。	都采用**回写 (Write-back)** 策略（因为直接写磁盘的开销过大，无法容忍直写策略）。

12．C。【解析】发生 Cache 缺失时，CPU 通常只是通过流水线停顿（Stall）等待硬件从主存取回数据，不会发生进程/线程的上下文切换；而发生缺页异常时，由于磁盘 I/O 极慢，操作系统为了提高 CPU 利用率，会将当前进程阻塞并切换到其他就绪进程。这就把存储器和操作系统的知识完美联动了起来。

13．B。【解析】SRAM 利用触发器存储信息，不需要刷新，速度快，但由于其存储单元结构复杂，成本高，集成度低；DRAM 利用电容存储信息，需要定期刷新，速度相对较慢，不过成本低，集成度高，B 正确，A、C、D 错误。

14．A。【解析】存储容量为 16K×8 位，$16K = 2^{14}$，所以需要 14 根地址线来寻址；8 位表示每个存储单元存储 8 位数据，因此需要 8 根数据线，A 正确。

注意：和通常会采用地址复用的 DRAM 芯片进行区分。

15．B。【解析】 I 正确：为了节省芯片引脚，DRAM 采用了“地址复用”技术，即通过同一组引脚（如 $A_0 \sim A_{10}$），先分发行地址（由行地址缓冲器接收），再分发列地址（由列地址缓冲器接收）。

II 错误：在执行刷新周期时，多路复用器（MUX）会切换到“刷新计数器”通道，它会无视外部 CPU 输入的地址，专门把内部刷新计数器自动生成的地址送给行译码器去进行逐行刷新。只有在正常的读写操作时，MUX 才会切换到“行地址缓冲器”通道。

III 正确：读出放大器可以捕捉位线上的微弱电压变化并迅速极化放大，随后顺着导通的晶体管反向给电容充电，完成破坏性读出后的数据恢复工作；此外，读出放大器在完成信号放大后，利用与 SRAM 极其相同的电路来锁存数据，在功能上完全等效于一个高速的 SRAM 缓存（行缓冲）。

IV 正确：刷新本质上就是执行了一次没有“数据输出”这一步的读操作（仅执行预充、访问、检测放大、恢复）；刷新时是由刷新计数器指定行地址，不打开列译码器，也不输出数据。

综上所述，正确的说法为 I、III、IV，故本题选 B。

16．D。【解析】Flash 存储器（闪存）属于非易失性存储器，断电后数据不丢失，A 正确；它可以通过电信号进行擦除和编程操作，B 正确；Flash 的读取速度较快，与半导体 RAM 相当，C 正确；但 Flash 的写操作速度相对较慢，因为其写入前必须先进行整片或区块擦除，写入速度通常与硬磁盘存储器相当，D 错误 。本题选不包括的特点，故选 D。

17．C。【解析】总容量 64K×32 位，需 （64K/16K） × （32 位/8 位） = 4×4 = 16 片，C 正确。

18．C。【解析】因为 64K×8 位/16K×1 位=4×8，说明在行方向(字方向)扩大了 4 倍。因为芯片内各单元连续编址，所以，每个芯片的片选信号由最高两位地址确定，低 14 位为片内地址。4 个芯片的地址范围分别为 0000H~3FFFH、4000H~7FFFH、8000H~BFFFH、C000H~FFFFH，显然，地址 BFF0H 所在的芯片的最小地址为 8000H。答案为选项 C。

19．C。【解析】由于按字编址，即每个存储单元为 4 字节，故只需要 23 根地址线来寻址，需要 32 根数据线，C 正确。

20．D。【解析】地址个数为 1M，行列各 10 根地址线，行列地址分时复用地址线，分别送到行列译码器，行列译码器的输出线有2^{10}根。DRAM 按行刷新，所以刷新计数器即行地址线根数=10。

21．C。【解析】DRAM 行缓存为$2^{12} \times 8bit$，即 4KB，I 正确；

分散刷新方式下，每个存储周期既要进行读写也要进行刷新，II 错误；

异步刷新方式，把每行的刷新分散在 64ms 内，每次刷新间隔为 64ms/4096=15625ns，15625/125=125 次，其中包含刷新的 1 次，所以每次间隔之间读写次数为 125-1=124 次，III 正确

DRAM 采用地址线复用，每个 DRAM 芯片，地址线 12 根，数据线 8 根，引脚总数 20，IV 正确；

22．C。【解析】地址个数为 2M，要减少地址线条数，那么行列地址位数就要尽可能接近，又要减少刷新开销，刷新按行刷新，所以行要尽可能少，所以行地址为 10bit，列地址 11bit。地址线条数为 max(r，c)=11。

A 错误，列译码器的输出列选有2^{11}根。

B 错误，DRAM 按行刷新，刷新计数器和行地址一样，送往行译码器再进行行选，所以刷新计数器即行地址线根数=10bit。

C 正确，数据引脚为 16 位，地址引脚 11 位，总数为 27 位

D 错误，行缓冲为一行的数据大小，为列数$c * 16bit = 2^{11} * 2B = 4KB$

23．C。【解析】DRAM 依靠电容存储电荷来表示数据，由于电容存在漏电现象，一段时间后电荷会丢失，导致数据错误，因此需要定期刷新，补充存储电容上泄漏的电荷，保持数据，C 正确。

24．C 【解析】由内部结构为 64*64 可知单个芯片之内有 64 行，刷新以行为单位，如采用集中刷新方式，存储器刷新一遍最少用 64 个刷新周期，因为存储器的读写周期为 0.5us，刷新周期也为 0.5μs，死区=0.5μs * 64=32us，死时间率=32μs÷2000μs*100％=1.6％。

25．A。【解析】刷新以行为单位，一次刷新花费 1 个存储周期。该 DRAM 为 128 * 128 * 8bit，一共 128 行。

26．D。【解析】2ms/128 行 =15.625μs/行，所以最大每隔 15.625us 就需要刷新一行，

15.625/0.2=78.125 次读写操作，即最多 78 次读写操作就要一次刷新，本题符合的选项只有 D 选项。

27．A。【解析】答案：A 确定容量与芯片总数：主存总空间为 16MB。RAM 与 ROM 比例为 3:1，因此 RAM 区容量= 16MB × (3/4) = 12MB。单片 SRAM 规格为 512K × 4 位。因为按字节编址（1 字节=8 位），首先需要进行位扩展：8/4 = 2 片，每 2 片组成一个 512K × 8 位（即 512KB）的逻辑组。再进行字扩展：12MB/512KB = 24 组。所以 SRAM 芯片总数 = 24 组 × 2 片/组 = 48 片。（排除 C、D）

计算地址范围：ROM 在最高地址段，所以 RAM 占据从 0 开始的低地址段：0 ~ 12MB − 1。

12MB = 12 × 2^{20} = 0C00000H，所以 RAM 的地址范围是 000000H ~ 0BFFFFH。

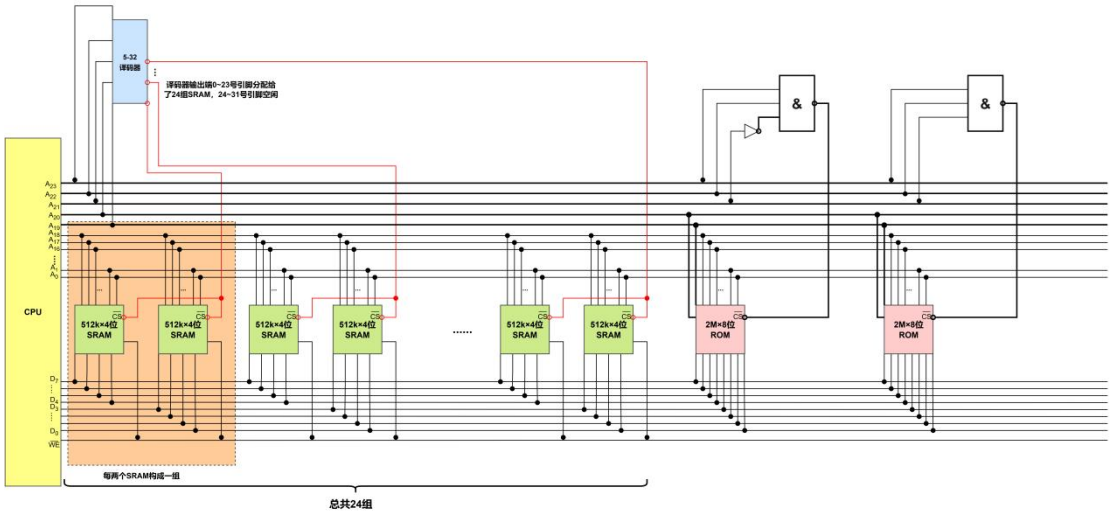
采用连续编址，每个逻辑组容量为 512KB（十六进制为 080000H）。

第 0 组地址： 000000H ~ 07FFFFH

第 1 组地址：080000H ~ 0FFFFFH

定位目标地址：目标地址 0A1234H 显然落入第 1 组 080000H ~ 0FFFFFH 的范围内。

因此，该地址所在芯片组的最小地址就是该组的起始地址，即 080000H 。



或者将地址 0A1234H 转化为 0000 1010 0001 0010 0011 0100，001 用于生成该 sram 芯片的片选信号，因此 该地址所在的 RAM 芯片的最小地址为 0000 1000 0000 0000 0000 0000 即 080000H。

28．B 【解析】分析芯片组合: 目标容量 32K × 8 位，芯片为 8K × 4 位。位扩展: 8/4 = 2 片/组（组成 8K × 8 位）。字扩展: 32K/8K = 4 组。

分析交叉编址逻辑: 共有 4 个逻辑组。交叉编址（低位交叉）意味着地址的最低 2 位（2^2=4）用来做片选。组 0 处理末两位为 00 的地址；组 1 处理末两位为 01 的地址；组 2 处理末两位为 10 的地址；组 3 处理末两位为 11 的地址。

定位给定地址所在的组: 物理地址 5A2DH 的末位十六进制数是 D，转为二进制为 1101，其最低两位是 01。这说明它属于组 1。

求该组的最大物理地址:整个 32K 存储器的最高物理地址是 32 × 1024 − 1 = 32767，即十六进制的 7FFFH。7FFFH 的末位 F 二进制是 1111，最低两比特为 11，属于组 3 的最后一个地址。往前推算: 7FFEH 属于组 2，7FFDH 属于组 1。因此，组 1 能够访问到的最大物理地址是 7FFDH。

29．B。【解析】$FFFFH - 0000H + 1H = 10000H = 2^{16} = 64K$，按字节编址，空间大小为

64K×1B。所以字方向扩展倍数为 64K/16K = 4，位方向扩展倍数为 1B/2b = 4，故正确答案为 B。

30．A。【解析】②不合理。因为存储单元的位数应为字节的整数倍。

③不合理。因为 MAR 的位数为 8，存储器的单元数最多为 256 个，不可能达到 1024 个。

④不合理。因为 MAR 的位数为 12，存储器的单元数应为 4K 个，不可能只有 1024 个。

31．B。【解析】A 正确：根据公式 带宽 = 等效传输率 × 数据位宽，可反推出等效传输率 = 6.4GB/s/8B = 6400MB/s/8B = 800M 次/秒。

B 错误：等效传输率 = 核心频率 × 预取倍数，可反推出核心频率 = 800M / 4 = 200MHz。

C 正确：由于双沿传输特性，总线时钟频率 = 等效传输率 / 2，即 800M / 2 = 400MHz。

D 正确：预取技术是指存储器核心一次从存储阵列中取出多位数据，先放入内部缓冲区，然后再以更高的外部接口速率分多次送到数据总线上。这样可以在不显著提高存储阵列核心频率的情况下，提高外部数据传输率。DDR2 SDRAM 芯片内部标准采用的是 4 位 (4n) 预取技术，所以存储阵列一次预取的量是 32B，每次在数据总线传输 8B 数据，这样就可以实现高速传输。

32．C。【解析】A 正确，DDR3 SDRAM 芯片内部 I/O 缓冲采用 8 位预取技术，即内部存储阵列一次取出够总线传输 8 次的数据量放在 I/O 缓冲，I/O 缓冲大小为 8×8B=64B。B 正确，芯片内部核心频率是指每秒内存储阵列往 I/O 缓存放数据的频率，所以存储总线实际传输数据的频率为 核心频率×预取位数 n，故存储器总线每秒传送数据的次数为 133.25M×8=1066M，其带宽为 1066M×8B≈8.5GB/S，D 也正确。因为存储器总线每个时钟周期传送两次数据，所以存储器总线时钟频率为 1066M/2=533MHz。综上所述，答案为选项 C。

33．B。【解析】多模块存储器本质上是一种空间并行技术。为了能够实现多个存储体同时或流水线式的高速读写，其硬件基础就在于各存储模块均具有各自独立的地址寄存器、数据寄存器和读写控制电路。

34．C。【解析】根据访问地址求出访问存储体体号序列（地址 %4）：1，2，3，0，1，2，3，0，0。可发现只有最后访问 8004 和 8000 时发生了体冲突，即：

体 1：0ns→80ns（首次启动）

体 2：20ns→100ns（间隔 Δt）

体 3：40ns→120ns（间隔 Δt）

体 0：60ns→140ns（间隔 Δt）

体 1：80ns→160ns（距首次启动 80ns，满足 T）

体 2：100ns→180ns（距首次启动 80ns）

体 3：120ns→200ns（距首次启动 80ns）

体 0：140ns→220ns（距首次启动 80ns）

体 0：220ns→300ns（距上一次体 0 启动 140ns→220ns，间隔 80ns）

最后一个访问（第 9 个地址，体 0）的完成时间为 300 ns，C 正确。

35．B。【解析】八体低位交叉存储器最大带宽时即完全流水线化，一个周期可访问 8 个存储体。每个存储体的容量为 256M×64 位，一个存储体一次传输 64 位数据即 8B，带宽为 8B×8/80ns =

800MB/s。

36．B。【解析】第一步，先对所有地址取模（地址 mod 4），地址序列[8,17, 21, 12, 31, 28]对应的体号序列为：[0, 1, 1, 0, 3, 0]。

【陷阱预警（静态窗口法易错点）】如果只用眼睛扫，看哪些相同体号的间隔小于 4：

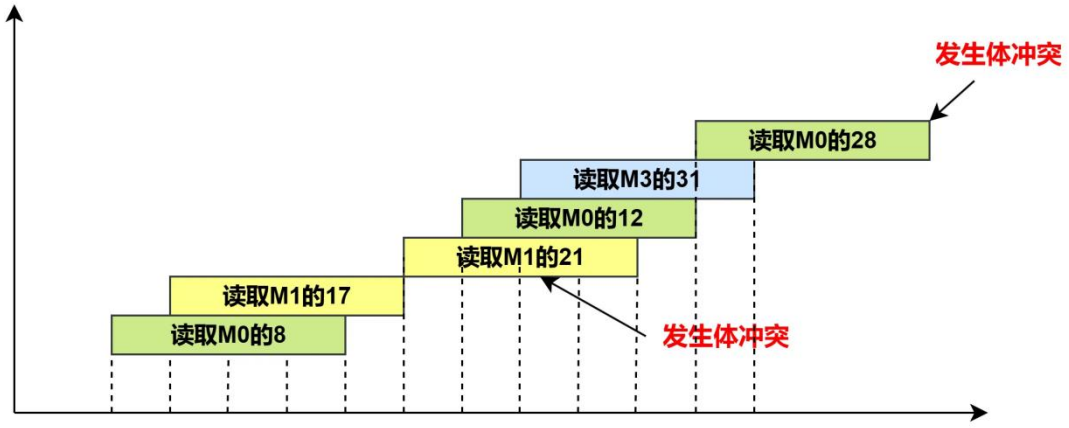
21(体号 1) 和 17(体号 1) 挨着，间隔为 1 ->21 冲突。

12(体号 0) 和 8(体号 0) 间隔为 3 ->12 冲突。

28(体号 0) 和 12(体号 0) 间隔为 2 ->28 冲突。

按照这个错误逻辑会错选 D。记住，前一次冲突引发的流水线停顿，会改变后续所有指令的发射时间。（这个跟 14 年计组大题 44 题（4）中为什么指令 1 的执行不会因为与指令 5 的数据相关而发生阻塞这一问有异曲同工之妙）

为了绝对严谨，我们按照总线周期（设为 r）一步步往后推。一个模块一旦启动，将占据 4 个周期（例如第 1 周期启动，第 1、2、3、4 周期忙碌，第 5 周期恢复空闲）。



因此在这六次访存中，只有地址 21 和地址 28 在试图发出时遭遇了模块忙碌，从而发生了体冲突。

37．C。【解析】一次缺失需要从主存读取一个主存块（64B），每个突发传送总线事务可以读取 8B*8，因此需要一个突发传送总线事务。每个突发传送总线事务所用时间为 1+8+8=17 个时钟周期。

38．D。【解析】磁盘的平均存取时间是指从发出读写请求到数据开始传输或传输结束所需要的平均时间，包括寻道时间（磁头移动到目标磁道的时间）、旋转延迟时间（盘片旋转使目标扇区移动到磁头下方的时间）和数据传输时间（数据在磁盘和内存之间传输的时间），D 正确。

39．C。【解析】A 错：稳定流水线带宽上限 = 字长/r（总线传输周期），一旦 $m \geq T/r$，继续增加 m 带宽不再提升，反而增加硬件成本和设计复杂度。

B 错：单颗 SDRAM 芯片本身就支持突发传送，原因是芯片内部有行缓冲区——激活一行后，同一行内的数据可以在每个时钟周期连续读出，无需重复发送地址。突发传送与交叉编址无必然关系。

C 正确 ：高位交叉编址（连续编址）中，相邻地址位于同一个模块，程序空间局部性恰好导致频繁访问集中于少数模块，其他模块闲置，完全失去多体并行优势。这是高位交叉仅适合容量扩展、不适合性能提升的根本原因。

D 错：低位交叉编址不改变单个模块的存取时间 T，而是利用多体流水线并行工作（空间并行技术）来提高整体吞吐率。

40．B。【解析】固态硬盘（SSD）使用半导体闪存芯片作为存储介质，存储和读取数据；磁性材料用于传统的磁表面存储器如硬盘、磁带；光盘片是光盘的存储介质；磁鼓也是磁表面存储器，A、C、D 错误。

41．B。【解析】固态硬盘基于闪存技术，不属于磁表面存储器，选项 A 错误。

闪存翻译层可将 CPU 的逻辑磁盘块读/写请求转化为对底层 SSD 物理设备的读/写控制信号，选项 B 正确。（闪存翻译层是运行在 SSD 控制器内部的软件/固件，核心作用是地址转换（将逻辑块地址 LBA 映射为物理页号 PPN）、垃圾回收和磨损均衡）

固态硬盘无论是读还是写，速度都远快于常规硬盘。只不过对于 SSD，写比读要慢，因为涉及擦除和磨损均衡。选项 C 错误。

固态硬盘块内的页必须按顺序写入信息，选项 D 错误。

42．B。【解析】静态磨损均衡技术比动态更先进、更优秀；

SSD 损均衡技术大致分为两种：

- 动态磨损均衡。写入数据时，优先选择擦除次数少的新闪存块。老的闪存块先歇一歇。
- 静态磨损均衡。这种技术更为先进，就算没有数据写入，SSD 也会监测并自动进行数据分配，让老的闪存块承担以读为主的存储任务。同时让较新的闪存块腾出空间，以承担更多以写为主的存储任务。如此一来，各闪存块的寿命损耗就都差不多。

43．A。【解析】I正确，固态硬盘基于闪存技术。

Ⅱ正确，固态硬盘以页为读写单位，只有在一页所属的块整个被擦除后，才能对这一页进行写入。

III错误，固态硬盘的读写性能优于机械硬盘，但是固态硬盘的 NAND 闪存中的块有擦写次数限制，其使用寿命一般短于机械硬盘。

IV错误，IV描述的是动态磨损均衡，而在静态磨损均衡中，会对数据块进行检测，尽可能保证各块写入和擦除均衡，并非是每次写入都选择长期存放数据而很少擦写的块。

44．B。【解析】读写以页为单位，擦除以块为单位。随机写的速度比读慢，原因如下：

①只能写入被擦除的块，擦除块比访问块慢得多。

②修改一个页中的部分数据，需要拷贝这个块中其他页到被擦除的块进行修改

③磨损均衡

动态磨损均衡：自动选磨损少的块写入，早期常使用动态方式，这种方式会导致冷数据块一直得不到擦除，各个块擦除并不均衡。

冷数据：只读数据或者系统数据，修改很少的数据块

静态磨损均衡：将冷数据块转移到擦除较多次的块（老块），现代多使用静态+动态磨损均衡结合的方式，增加了闪存的寿命。

注意：固态硬盘的核心存储介质是 NAND 闪存（NAND Flash），属于非易失性存储器。此外，现代 SSD 内部通常还会集成一块速度较快的 DRAM（动态随机存取存储器），用作缓存（Cache），主要用来存放 闪存翻译层的地址映射表以及缓存读写数据，以此来极大提升 SSD 的工作效率。

45．C。【解析】数组长度 N 极大，超过 Cache 容量，这意味着数组会被不断挤出 Cache。

写法 1： 第一个循环遍历了整个数组 a，第二个循环又重新从头遍历数组 a 去算 b。当第二个循环要用 a[0] 时，a[0] 早就因为 Cache 容量不够被后面的 a[99999] 替换出去了，必须重新从主存读！

写法 2： 在一个循环体内，刚刚算完 a[i]（此时 a[i] 还在 Cache 里，热乎着呢），紧接着下一句就立刻使用 a[i] 去算 b[i]。

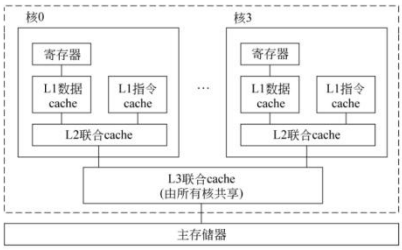
结论： 对同一个元素 a[i] 的两次访问在时间上被拉到了最近。这是非常典型地通过修改代码逻辑来提升时间局部性的操作，所以选 C。

46．B。【解析】A 错误，cache 行并不是越大越好，因为缺失时，从主存读一个数据更大的块需要消耗更多时间，其次，虽然增大 cache 块可以更好利用空间局部性，但 cache 行变大，容量不变的情况下，行数变少，因而要替换的可能性增加，命中的可能性不一定增加。

B 正确，直接映射方式下，一个给定的主存地址映射到固定的 cache 行，若不命中，直接将该行数据替换，无需考虑替换算法。

C 错误，cache 仅仅是内存的拷贝，无法拓展不同的数据，所以无法扩充内存，扩充内存的方式一般指的是虚拟存储器。

D 错误，CPU 侧的 cache 通常指令和数据分开，这样方便并行读取数据和指令（流水线中），若缺失，再从靠近内存侧的 cache 读入靠近 CPU 侧的 cache。



47．C。【解析】先访问 cache，未命中访问主存 Tc=Ta×e=60ns×0.85=51ns（cache 存取周期）

cache 比主存快 4 倍，所以访问主存的时间 Tm=51×5=255ns（主存存取周期）

因为 Ta=Tc+（1-H）×Tm=51ns+（1-H）×255ns=60ns

所以命中率 H=246/255=96.5%，C 正确。

48．B。【解析】Tc=Tm/4=50ns

平均访问时间 Ta=Tc+（1-H）×Tm=50ns+（1-0.98）×200ns=54ns

访问效率 e=Tc/Ta=50ns/54ns×100%=92.6%，B 正确。

49．B。【解析】最好的局部性即完全按地址存放顺序访问数组。B 改完后对数组 a 的访问为顺序访问依次访问地址上连续的元素，故有更好的空间局部性。

50．D。【解析】因为 cache 命中率是 0.9，所以访问 90%的字的平均存取时间=20nsx0.9=18ns；又因为主存命中率是 0.6，所以 0.6×0.1 的数据要从主存调入 cache，然后再从 cache 存取，访问一个字的平均存取时间=0.6x0.1×（60ns+20ns）=4.8ns。0.4×0.1 的数据要从磁盘调入主存，然后再调入 cache，访问一个字的平均存取时间=0.4x0.1x（12ns+60ns+20ns）=3.68ns；此系统访问一个字的平均存取时间=18ns+4.8ns+3.68ns=26.48ns，D 正确。

51．A。【解析】I. 正确 主存块大小为 16B，按字节编址，因此块内地址（Offset）占 $\log_2(16) = 4$ 位。Cache 数据区容量为 64KB，共有 64KB/16B = 4096 行，因此 Cache 行号（Index）占 $\log_2(4096) = 12$ 位。主存地址共 32 位，因此标记位（Tag）长度为 32 - 12 - 4 = 16 位。

II错误：部件①确实是比较器，在直接映射中由于每个主存块只能映射到唯一的 Cache 行，每次只需比较 1 个 Tag，因此该部件仅有 1 个。陷阱在于比较的位宽： 比较器只需要对比主存地址中的 Tag 字段与 Cache 标记阵列中存储的 Tag，因此它的位宽应与 Tag 长度一致，即 16 位，而不是 32 位。

III正确：部件②是与门。Cache 命中的严格物理条件是：Tag 匹配成功 且 该行的有效位（Valid Bit）为 1。两者同时满足，与门输出高电平（1），产生 Hit 信号。

IV错误：部件③是一个 4 路选择器，其作用是从命中的 16B Cache 行（包含 4 个 32 位的字）中，选出 CPU 需要的那 1 个字（4B）。从 4 个字中选择 1 个，需要 2 位控制信号。这 2 位信号来自块内地址（4 位，设为 [3:0]）。其中，低 2bit [1:0] 用于字内字节选择，而 高 2bit [3:2] 才用于字选择（Word Offset）。因此控制信号应来自块内地址的高 2bit。

V. 正确：题目明确采用直写方式（Write-Through），这意味着不需要额外的脏位（Dirty Bit）来标记数据是否被修改，因此每行的控制位仅需 1 位有效位（Valid Bit）。

每行的标记阵列位宽 = 1 位有效位 + 16 位 Tag = 17 位。

Cache 行数 = 4096 行。

标记阵列总容量 = 4096 * 17 = 69632 位。

数据区总容量 = 64KB = 64 * 1024 * 8 = 524288 位。

Cache 总容量位数 = 524288 + 69632 = 593920 位。

VI正确，直接映射中给定数据块只可能映射到固定 cache 行中，因此发生冲突直接替换即可。

52．B。【解析】I. 正确：相比于直接映射和组相联，全相联映射最大的特点就是极其灵活。主存中的任意一块可以存放在 Cache 的任何一个空闲位置，因此在 Cache 完全填满之前，绝对不会发生因位置冲突导致的替换（冲突缺失）。

II. 正确。主存地址位数： 主存 1MB = 2^20B，因此主存地址共 20 位。块内地址： 块大小 512B = 2^9B，因此块内地址（Offset）占 9 位。Tag 计算： 全相联映射没有组号/行号，剩余的高位全都是 Tag。Tag = 20 - 9 = 11 位。

拆解 0240CH： 将十六进制 0240CH 展开为 20 位二进制：0000 0010 0100 0000 1100。

低 9 位为块内地址：0 0000 1100 (即十六进制 00C)

高 11 位为 Tag：000 0001 0010 (即十六进制的 012H)。

III错误。题目中 Cache 数据区 8KB，块大小 512B，共有 16 行。全相联需要所有的行同时比较，所以确实需要 16 个比较器。陷阱在此： 比较器只需要对比主存地址中的 Tag 和 Cache 标记阵列中的 Tag，不需要比较整个地址！上面刚刚算出 Tag 是 11 位，因此比较器的位宽必须是 11 位，而非 20 位。

IV正确。全相联的 hit 信号是由一个或门（部件②）生成的。16 行并行比较，只要有 1 行“Tag 相等 AND 有效位为 1”，这 16 根输入线中就会有一个 1，或门就会输出 1 告诉 CPU：“我找到了”。

V错误。当命中后，整个 512B 的块数据被放到总线上，送到 MUX（多路选择器）。 MUX 的作用是从 512B 的大块里挑出 CPU 具体要读的那几个字节。既然是在“块内部”挑选，控制信号理所当然

应该来自主存地址的块内地址（Block Offset），而绝不是 Tag 字段。

53．A。【解析】I. 正确：主存 1MB = 2^20B，主存地址为 20 位。块大小 512B = 2^9B，因此块内地址占 9 位。计算 Cache 组数： Cache 数据区 8KB，共 8192 / 512 = 16 个块。采用 2 路组相联，分为 16 / 2 = 8 组。因此组号占 log_2(8) = 3 位。 标记位 Tag = 20 - 3 - 9 = 8 位。

拆解 0240CH： 将十六进制 0240CH 展开为 20 位二进制：0000 0010 0100 0000 1100。低 9 位为块内地址：0 0000 1100 (0x00C)；中间 3 位为组号：010 (十进制的 2)；高 8 位为 Tag：0000 0010 (0x02)；因此，映射的 Cache 组号确实为 2。

II错误。前半句正确，2 路组相联确实需要 2 个比较器（部件③）同时进行并行比较。陷阱在于比较的位宽： 比较器只负责比较 Tag 字段。既然前面算出了 Tag 是 8 位，那么存储在 Cache 标记阵列中的 Tag 也是 8 位。因此，比较器的位宽必须是 8 位，而不是完整主存地址的 20 位。

III正确。部件②是或门。2 路组相联中，一个主存块可能存放在这 2 路中的任意一路。只要路 0 或路 1 中有任意一个 Hit（Tag 相等 AND 有效位为 1），整个 Cache 就视作命中，或门输出 1。

IV错误。底部的 MUX 负责从整整 512B 的块中，“挑”出 CPU 真正要读写的那个字或字节。陷阱在于控制信号： 决定读哪一个字节的是主存地址中的“块内地址（Block Offset）”，而不是“组号”。组号早就用在上面寻找 Cache 行了。

V正确。部件①是三态门。两路的数据线最终汇聚在同一根总线上，不能同时输出，否则会发生短路冲突。三态门就像个水龙头，只有命中那一路的“水龙头”会被打开（与门输出 1），将其数据放行到后续的 MUX。

54．C。【解析】一个 cache 块可以容纳 32/4=8 个 int 元素，table 中元素的访问情况如下，m 表示 miss，h 表示 hit。

m h h h h h h h

m h h h h h h h

m h h h h h h h

m h h h h h h h

Miss rate = 1/8

55．C。【解析】计算机主存容量 4GB，按字节编址，故主存地址共 32 位。二路组相联每组包含 2 行，Cache 共有 1024 行，故组数为 512 组，组号字段共 9 位，标记位字段为 12 位，故块内偏移量字段共 11 位，Cache 块大小为 2^{11} = 2KB，C 正确。

56．C。【解析】数组占用 0~8 共 9 个块，第 1 轮 cache 映射情况，cache 分为 4 个组（组 0、1、2、3）。其中组 0 包含块 0，4，组 1 包含块 1，5，组 2 包括块 2，6，组 3 包括块 3，7，当块 8 访问时候，根据 LRU 替换算法，块 0 被替换出去，第一轮中每块第一个元素访问全部缺失共 9 次。第 2、3、4 轮组 1、2、3 中的块在重复访问时都命中，只有组 0 内的块在块 0 访问的时候，此时 cache 中包含块 8 和块 4，块 0 缺失替换出块 4，后续访问块 4 和块 8 时也分别缺失。每轮中 0、4、8 依次缺失替换，共计 3 次，所以缺失次数是 9+3+3+3=18 次。总访问次数是 4*18*2=144 次（每个元素每次读写共两次，共 4 轮），所以缺失率是 18/144=1/8，答案选 C。

Cache组号	对应主存块
组0	块0，块4
组1	块1，块5
组2	块2，块6
组3	块3，块7

57．C。【解析】cache 行数为 32K/1=32K 行，直接映射方式下，需要 15 位来表示行号；主存块大小为 32bit，按字节编址，块内地址计 2 位；tag 位数=32-15-2=15bit，有效位和修改位（脏位）各 1bit，所以 cache 每行需要 15+1+1+32=49bit，cache 总容量=49×32K=1568K。

58．C。【解析】内存地址空间为 64M=2^26，物理地址为 26 位。按字节编址，主存块长 64B，块内地址为 6bit。cache 数据容量 8 * 64B=512B，标记阵列容量为 532-512=20B，一共 8 行，每行的标记信息一共 20bit，回写方式需要 1bit 脏位，另外 1bit 有效位，因此 tag 位为 20-1-1=18bit。因此可以知道组号位数为 26-18-6=2bit，可知该 cache 采用 8/2^2=2 路组相联。比较器数量为 2.答案选 C。

关于 cache 比较器个数记忆：k 路组相联就有 k 个比较器。假设一个 cache 有 n 行

- 直接映射可看作 n 组 1 路组相联，1 个比较器
- k 路组相联映射为 n/k 组 k 路组相联，k 个比较器
- 全相联可看作 1 组 n 路组相联，n 个比较器

59．C。【解析】由于 cache 采用 4 路组相联，所以 cache 中有 4 个比较器，比较后通过一个多路选择器选择出一个块，每组有四个块，所以用 2bit 控制多路选择。实现一个四路组相联的 cache 需要四个比较器和一个 4 选 1 的多路选择器来判断被选中的组中哪一个单元与标记匹配。

60．D。【解析】cache 地址总共为 14 位，最后 4 位是块内偏移量，所以和主存地址的最后 4 位一样。由于采用直接映射，主存地址的最后 14 位，就是对应在 cache 的地址。

61．A。【解析】FIFO 在特定访问序列下可能出现增加 Cache 容量反而命中率下降的现象，即“Belady 异常”，A 正确。

补充：会发生“Belady 异常”的有 FIFO、CLOCK 和改进的 CLOCK。

62．C。【解析】LRU 将最久未被访问的块替换，可以发现在第 6 次访问 4、第 7 次访问 2、第 8 次访问 3 和第 9 次访问 5 时命中，命中四次，命中率为 4/10=40%。

63．B。【解析】回写法中，数据写入 Cache 后不立即写入主存，只有当该数据所在的 Cache 块被替换时，才将其写回主存，写操作速度快，因为不需要每次都等待主存写入完成，但会存在 Cache 与主存数据不一致的时间段；数据写入 Cache 同时写入主存的是全写法；C 描述的也是全写法的特点；写回法会出现数据不一致情况，A、C、D 错误。

64．A。【解析】Tag 位数：32-10-4=18；有效位：1，Dirty：1；行数：1K，(32*4+18+1+1)*1K =148K。

65．C。【解析】III 正确；II 写命中时不写辅存而是主存；I 写入主存不在第一次写入时，而在换出时，C 正确。

66．B。【解析】装入位为 0 表示不在主存中，故也一定不在 Cache 中，无需考虑，A 正确。C=0 表示一定不在 Cache，故无需访问 Cache，C 正确。C=1 表示可能在 Cache 中，故要先在 Cache 中查找，D 正确。B 选项一个页面被写入数据（修改位变 1），是非常正常的内存访问行为，这绝不会改变该页面的物理存储属性。普通的 RAM 页面被写了之后，依然是 RAM，依然允许被 Cache 缓存，根本不存在“将其清 0（变成不可缓存）”的逻辑。B 错误。

67．B。【解析】取指令阶段（0 次访存）：题干明确指出取指阶段 TLB 和 Cache 均命中，直接从 Cache 中拿到指令，访问主存 0 次。

读操作数阶段（2 次访存）：指令 add [xaddr], 3 是典型的“读-改-写”指令，必须先将 x 从内存中读出来 。TLB 命中，MMU 直接将虚拟地址 xaddr 翻译为物理地址，不需要访问主存中的页表 。物理地址送往 Cache，此时 Cache 未命中。关键点（脏行写回），因为 Cache 采用回写策略，且未命中时触发了替换，而被替换的 Cache 行是“脏行”（脏位为 1，说明里面的数据被修改过且尚未同步到主存），此时硬件必须先将该脏行的数据写回主存（第 1 次访问主存）。 关键点（新块调入）， 脏行腾空后，由于是读失效，需要将包含变量 x 的主存块读入 Cache（第 2 次访问主存） 。随后 CPU 从 Cache 中读取 x 的值参与加法运算。

写回结果阶段（0 次访存）：运算完成后，新值需要写回 xaddr，此时包含 x 的数据块刚刚在读阶段被调入 Cache，因此写回时 Cache 必然命中。 在回写策略下，Cache 命中时仅修改 Cache 行中的内容，并将脏位置为 1，不需要同步写入主存 。因此这一阶段访问主存 0 次。

总结： 0（取指） ＋2（读操作数） ＋0（写回） ＝2 次。

68．D。【解析】虚拟存储器的容量受计算机的地址结构（决定逻辑地址空间大小）和外存容量的共同限制，并非只受外存容量限制；虚拟存储器需要硬件（如 MMU 内存管理单元）和操作系统软件协同实现地址映射等功能；在页式虚拟存储器中，若页面置换算法不当，可能导致频繁换页，出现抖动现象；段式虚拟存储器按逻辑模块分段，便于实现程序的动态链接和共享，A、B、C 正确，D 错误。

69．A。【解析】第一步，明确地址划分： 主存物理地址共 28 位。Cache 块大小为 32B，因此块内偏移占 5 位 。Cache 总容量 16KB，4 路组相联，共有 16KB / (4 * 32B) = 128 组，因此组索引占 7 位 。物理地址高处的 28 - 5 - 7 = 16 位即为 Cache 标记 (Tag)。

第二步，拼装物理地址： 页面大小为 4KB，因此虚拟地址和物理地址的低 12 位（页内偏移）完全相同 。虚拟地址 0x00456ABC 的低 12 位为 0xABC。物理地址由物理页号和页内偏移拼接而成，即物理地址 ＝0x012FABC 。

第三步，提取组号和标记：组索引位 (7 位) + 块内偏移位 (5 位) = 12 位 = 页内偏移位数。这意味着 Cache 的组号完全由页内偏移决定，不受虚拟/物理页号影响。将低 12 位的 0xABC 转为二进制：1010 1011 1100。抛弃最低的 5 位（1 1100），接下来的 7 位即为组索引：101 0101，转化为十进制为 85。Cache 标记为物理地址去掉低 12 位后的高 16 位，即 0x012F

70．C。【解析】A 选项错误： MMU 只能检测缺页并触发缺页异常中断，真正的缺页处理程序是由操作系统（软件）在内核态接管并负责从外存读取页面调入主存的，并非硬件直接读取 。

B 选项错误： MMU 负责将虚拟地址转化为物理地址，在此过程中会检测越权、缺页和 TLB 缺

失 。Cache 缺失是在 MMU 完成翻译输出物理地址之后，CPU 访问主存数据时检测的，不属于 MMU 的工作范畴 。

C 选项正确： PTBR 是 MMU 访问内存页表的入口指针 。当 TLB 命中时，MMU 直接从 TLB 中获取物理页号并拼接页内偏移生成物理地址，整个过程完全绕过了 PTBR 和主存中的页表。

D 选项错误： TLB 缺失的处理机制因架构而异。例如在 x86 或 ARM 架构中，硬件内部自动遍历内存填入 TLB（纯硬件完成）；而在 MIPS 架构下，才需要触发异常由 OS 软件手动填入。

事情	执行者	时机
创建页表、写入映射关系	OS 软件	进程创建/缺页时
修改 PTBR 寄存器（切换页表）	OS 软件	进程切换时
处理缺页异常（分配页、加载磁盘）	OS 软件	缺页发生后
查 TLB，翻译地址，权限检查	**MMU 硬件**	**每次访存**
TLB 缺失时遍历页表	**MMU 硬件**	**TLB 缺失时**

71．A。【解析】操作 I：TLB 命中→物理地址，Cache 未命中→1 次访存。

操作 II：TLB 未命中访问页表（1 次访存）→缺页处理（视作 1 次访存）→重新执行→TLB 命中 → 物理地址，Cache 未命中（1 次访存）。操作 II 共 1 + 1 + 1 = 3 次。

操作 III：TLB 命中→物理地址，Cache 命中→0 次访存。

操作 IV：TLB 未命中→查页表命中→物理地址，Cache 未命中→2 次访存。

总计：1 + 3 + 0 + 2 = 6 次，A 正确。

72．A。【解析】$4096 = 2^{12}$，按字节编址，所以虚拟地址和物理地址中的低 12 位被用作页内地址字段，虚页号的长度为 48－12=36，所以虚拟地址空间能支持2^{36}个虚页；而实页号的长度为 36－12=24，所以物理地址空间能支持2^{24}个实页。

73．A。【解析】A 错误：很多人认为 0 次主存访问的唯一条件就是“TLB 命中 ＋Cache 命中”。但实际上，如果发生以下情况：TLB 未命中 →MMU 去查页表 → 页表项在 Cache 中命中 → 获取物理地址后，目标数据在 Cache 中也命中 。 在这个“TLB 缺失，但双 Cache 命中”的过程中，数据全是从高速缓存中读取的，主存访问次数依然是 0 次。因此 0 次主存访问不能说明 TLB 命中。

B 正确：这是非常典型的一种状态组合。TLB 未命中，硬件去查页表；此时页表项（PTE）在 Cache 中未命中，被迫访问 1 次主存将页表块读入 Cache ；拿到物理地址后去查目标数据，目标数据此时幸运地在 Cache 中命中 。整个过程加起来，刚好只访问了 1 次主存。

C 正确：这也是极其常规的缓存未命中情况。TLB 命中，直接拿到物理地址（0 次主存访问） 。拿着物理地址去查 Cache，发现数据未命中，于是硬件去主存中取数据块放入 Cache（访问 1 次主存）。总耗时就是 1 次主存访问的开销。

D 正确：如果发生缺页，说明该页面根本不在主存中 。既然不在主存中，TLB 作为一个只缓存“有效物理映射”的快表，里面绝对不可能有它的有效表项，所以 TLB 必然未命中 。 同理，Cache

缓存的是主存的子集。既然老大哥（主存）里都没有这块数据，小弟（Cache）里自然不可能有。所以目标数据必然不在 Cache 中。该推断完全正确。

74．B。【**解析**】缺页是内部异常，需要调用缺页异常处理程序来处理，B 错误。

75．C。【解析】Cache 和 TLB 对比如下

对比维度	Cache	TLB
缓存对象	主存中的数据/指令块	内存页表中的页表项
查询 key	物理地址（块号部分）	虚拟页号 VPN
输出结果	数据内容	物理页号 PPN + 权限
解决问题	CPU与主存速度差	每次地址翻译都要访问主存页表的开销
位置	CPU 芯片内（L1/L2/L3）	MMU 内部
典型容量	32KB ~ 32MB（多级）	32 ~ 1024 项（1~2级）
映射方式	直接/组相联/全相联	全相联或组相联
缺失处理	**硬件自动完成**	**硬件或OS（视架构）**
进程切换	无需刷新	需刷新

A 正确：Cache 的查询 Key 是物理地址，而物理地址在内存中是全局唯一的，因此进程切换时无需刷新 Cache 。但是，TLB 的查询 Key 是虚拟地址，不同进程中相同的虚拟地址会映射到完全不同的物理地址上。如果不刷新 TLB，新进程就会错误地使用旧进程的地址翻译结果。

B 正确：这一项对比了二者的定位与寻址方式。TLB 的核心目的是解决“每次地址翻译都要访问主存页表的开销”，它的查询 Key 是虚拟页号，输出结果是物理页号。而 Cache 的核心目的是解决“CPU 与主存速度差”，它的缓存对象是主存中的数据或指令块，查询 Key 则是物理地址。简单来说：TLB 加速“找地址”，Cache 加速“取数据” 。

C 错误：Cache 缺失确实对软件完全透明，永远由硬件自动处理并从下级存储取数据块填入 Cache 。但 TLB 缺失的处理方式视计算机架构而定：

① x86 或 ARM 等架构中，TLB 是由硬件管理的，MMU 的内部硬件会自动遍历内存页表并填入 TLB，无需操作系统介入。

② MIPS 等软件管理 TLB 的架构中，TLB 缺失才会触发异常，由操作系统的处理程序填入 。

D 正确：这一项考查了命中的组合情况。TLB 缺失仅仅意味着“快表里没有缓存这个映射”，但只要该页面有效（未发生缺页），数据就存在于主存中，也就有可能被缓存到了 Cache 中 。当 TLB 缺失时，硬件会去查页表。如果代表该映射关系的页表项刚好在 Cache 中命中，且翻译出物理地址后，目标数据也刚好在 Cache 中命中，那么整个取数过程的数据全部是从 Cache 这个高速器件里拿到的，实际去读写主存（DRAM）的次数依然为 0 次。

76．A。【**解析**】段式虚拟存储器将程序按逻辑模块（如代码段、数据段等）分段，每段在内存中分配连续的空间，通过段表进行地址映射，A 正确；B 描述的是页式虚拟存储器；C 是段页式虚拟存储器；每个进程都有自己的段表，不是共享一个段表，B、C、D 错误。

77．C。【**解析**】082H 转成 10 进制是 130，此时查段表，存在位是 1，段长是 304，而此时段内地

址为 130H，转成十进制为 256*1+3 * 16＝304，段内地址从 0 开始的，304 是第 305 号单元，305>304 发生越界异常。思考：若把段内地址 130H 改为 12FH，答案是什么？（越权）

总结：

- 要分清十进制和 16 进制，页表和段表中的页号和段号往往是连续排列，所以是十进制，题目中可能会给 16 进制，要注意转换，进制统一后，需要考虑段（页）表长度，看段（页）号是否超出段表长度，超出发生**越界异常**。
- 确定好段（页）表项，看存在位是 1 还是 0，若是 0，可知段（页）不在内存，发生**段缺失异常**。
- 若是段式，需要考虑段长，同样需要考虑 16 进制和 10 进制的转换，以及下标问题。举例：若段长为 100，那么段内地址是 0-99，100 即发生**越界异常**。
- 以上都正常，最后还需要查看权限信息，本题中访问并修改，那么涉及写读和写权限，只读权限的话会发生**越权异常**。

78．D。【**解析**】低级语言程序员在编程时要指明段号，分段方式对其并不是透明的。

79．C。【**解析**】段表占用空间 = 段表项数量 × 段表项大小 = 8×4 = 32 字节；页表占用空间 = 段表项数量 × 平均每个段的页表项数量 × 页表项大小 = 8×32×4 = 1024 字节；段表和页表共占用空间 = 32 + 1024 = 1056 字节，C 正确。

第 4 章

指令系统

【答案速对】

ADCCC　　ADCCC　　ABBBC　　BABBD

ABCDC　　CCBBC　　ACBCC　　DBADB

DAB

1．A。【解析】指令系统是一台计算机能够执行的所有指令的集合，它反映了计算机的基本功能和硬件特性，A 正确；指令系统针对指令，而非硬件功能描述，B 错误；高级语言语句经编译或解释才转化为指令，C 错误；操作系统命令与计算机指令系统概念不同，D 错误。

2．D。【解析】指令系统设计关注操作码、寻址方式、指令格式等硬件相关要素，与操作系统无关，D 错误。

3．C。【解析】指令系统的兼容性指一种计算机的目标程序可以不加修改地在另一种计算机上运行，意味着这些计算机的指令系统在功能和格式上有一定相似性，C 正确；不同计算机指令系统不可能完全一样，A 错误；指令系统运行与硬件相关，并非只和操作系统有关，B 错误；指令系统与支持高级语言数量无直接关联，D 错误。

4．C。【解析】CPU 在执行指令时，会根据指令中的操作码字段和寻址方式字段对寄存器中的内容进行解释，例如，若是基址寻址方式，则基址寄存器中的内容是一个基地址；若是寄存器直接寻址方式，则寄存器中的内容是操作数。答案为选项 C。

5．C。【解析】程序控制类指令用于控制程序的执行顺序，并使程序具有测试、分析与判断的能力，C 正确。

6．A。【解析】操作码是指令的重要组成部分，用于指出指令执行的操作类型，如加法、减法、跳转等，A 正确；指出操作数地址的是地址码，B 错误；指令执行结果存放地址由指令格式中的相关字段或默认规则确定，不是操作码的作用，C 错误；指令在内存中的存放地址由程序计数器（PC）等确定，D 错误。

7．D。【解析】变长指令字格式中，不同指令的长度可以不同，操作码和地址码的长度也都可以根据指令功能灵活变化，D 正确；定长指令字格式才是所有指令长度相同，A 错误；变长指令字格式下操作码和地址码长度都可变，B、C 错误。

8．C。【解析】地址码部分可以表示操作数的直接地址（直接寻址）、间接地址（间接寻址）、

操作数地址的偏移量（如基址寻址、变址寻址中的偏移量），A、B、D 正确；指令的执行结果与地址码无关，C 错误。

9．C。【解析】指令格式设计直接影响硬件实现的复杂度，如复杂的地址码格式可能增加硬件电路设计难度，A 正确；良好的指令格式要能满足不同数据类型（如整数、浮点数）和操作（如算术、逻辑、控制操作）的需求，B 正确；合理的指令格式能减少指令执行周期、提高指令译码速度等，从而提高程序执行效率，与指令系统性能密切相关，C 错误，D 正确。

10．C。【解析】目的是保持指令长度不变，增加操作码的种类个数，而不是增加寻址空间。

11．A。【解析】二地址指令中，如果有一个操作数是立即寻址，则指令的地址码字段存放的是该操作数；如果有一个操作数是寄存器寻址，则指令的地址码字段存放的是寄存器号：如果有一个操作数是直接寻址，则指令的地址码字段存放的是该操作数地址。所以 B、C、D 都不一定正确，只有 A 是合理的。

12．B。【解析】操作码字段长度为 32-4×6=8，所以操作码可以提供$2^8 = 256$ 个空闲状态，4 地址指令有 254 条，留给 3 地址的空闲状态数为 256-254=2 个。3 地址指令在 4 地址指令基础上牺牲一个 6 位地址码扩展操作码，所以 3 地址指令一共有 $2 \times 2^6 = 128$ 个空闲状态。此时在 3 地址指令基础上再牺牲一个 6 位地址码，扩展操作码到 2 地址指令，2 地址指令有 4095 条，需要 3 地址留出$\lceil 4095/64 \rceil$=64 个状态，所以 3 地址还剩下 128-64=64 个状态，即三地址指令至多有 64 条。

13．B。【解析】由于存储器按半字编址，即存储字长为 16 位，又由于每取出一条指令后 PC 的值自动 +2，说明指令字长等于两倍的存储字长，即 32 位，B 正确。

14．B。【解析】一地址指令有 $256 = 2^8$条，那么必须匀一整个 op 字段的状态给一地址指令，操作码 4bit，例如 0000 0000 0000 - 0000 1111 1111 为一地址的操作码空间，而指令系统中存在二地址，那么 op 也必须至少给一个状态给二地址，三地址只剩下 16-1-1=14 个状态，即选 B。

15．C。【解析】零地址的运算类指令即堆栈运算指令，参加的两个操作数来自堆栈的栈顶和次栈顶，C 正确。

16．B。【解析】一地址运算类指令包括单操作数指令（如加 1、减 1 指令）和双操作数指令（如加、减指令）两类。对于单操作数指令只需要一个操作数，对于双操作数指令需要两个操作数，其中一个操作数的地址是显地址，另外一个操作数隐含在累加寄存器（ACC）中。

17．A。【解析】由于指令占 4 字节，取指令之后(PC)+4。第 3、第 4 字节的内容为：2001H-(2008H+4)=FFF5H(真值为-11)。

18．B。【解析】A 适用，公共子程序的浮动指的是，公共子程序可以在内存任意地方存放，只要给出的数据寻址是基于 PC 的偏移即相对寻址方式，那么无论子程序放在内存哪里，只要给出程序内部数据相对于指令的偏移，都可以做到不修改程序实现正确寻址。 C、D 适用，选择结构语句（if-else）和循环结构语句(for/while)都是通过相对寻址来改变 PC 值实 现转移的典型应用场景。 B 不适用，数组的访问是基于数组首地址+偏移的方式，变址寻址（或结合基址寻址）比较适合，相对寻址是基于 PC，不适合数组的访问，本题正确答案选 B。

19．B。【解析】变该数组元素为 32 位（4 字节）。变址寻址的有效地址 EA= 形式地址(首地址) +

变址寄存器的值；当前指令操作数的首地址 EA = 0800 1000H + 0000 000CH = 0800 100CH。这表示此次取出的 32 位数据物理上占据 100CH、100DH、100EH、100FH 这四个连续字节空间。

计算机采用大端方式存放数据，即最高有效字节（MSB）存放在最低地址，最低有效字节（LSB）存放在最高地址。　因此，该数据的 LSB 应该存储在该连续空间的最高地址处，即 0800 100CH + 3 = 0800 100FH。

20．D。【解析】选项 D 混淆了基址寄存器在不同场景下的控制权限。当使用通用寄存器作为基址寄存器来“扩展寻址空间”时，该寄存器本质上充当了高级语言中大段内存的首地址指针。为了能够访问超出指令寻址范围的不同存储区间，用户（或编译器）不仅可以，而且必须在程序中通过指令去主动修改它的内容（如更换数据块首地址）。只有在“程序重定位”场景下使用的专用基址寄存器，才由操作系统控制且用户绝对不可修改。

21．A。【解析】A 错误，需要访存 4 次：取指、取数 1、取数 2、存结果。B 正确，数 1 数 2 都是堆栈寻址，所以在栈顶和次栈顶。C 正确，隐含寻址指的是，在指令中不明显给出操作数地址或寄存器编号，而是由操作码隐含指出，栈指令属于隐含寻址。D 正确，硬堆栈至少需要取指令 1 次访存。

22．B。【解析】间接寻址通过地址指针（存储在内存或寄存器中的地址）间接获取操作数地址，相比直接寻址，它可以利用地址指针指向更大的内存空间，从而扩大寻址范围，B 正确。

23．C。【解析】堆栈寻址中，操作数存放在堆栈中，遵循先进后出（FILO）原则进行存取操作，C 正确；寄存器寻址操作数在寄存器中，直接寻址需访问内存，寄存器寻址执行速度更快，A 错误；变址寻址可以使用多个变址寄存器组合，比如在求二维数组偏移量的时候，B 错误；隐含寻址是操作数地址隐含在操作码或默认位置，不是不需要操作数，D 错误。

24．D。【解析】相对寻址中，有效地址 EA=（PC）+A，执行本条指令时，PC 已完成加 1 操作，PC 中保存的是下一条指令的地址，因此以下一条指令的地址为基准位置的偏移量，D 正确。

25．C。【解析】指令中不同操作数都包含自己的寻址方式，其中间接寻址需要在间址周期找到操作数有效地址（即逻辑地址），隐含寻址需要特殊硬件支持，寄存器间接寻址将有效地址放在寄存器中，因此同时被寄存器位数和存储单元个数限制。

26．C。【解析】四个选项对应的访存次数分别为 1、1、2、0。访存次数最多的是间接寻址。

27．C。【解析】基址寻址的有效地址的计算式为：EA=（BR）+A，其中 BR 中的值是 2022H，形式地址 A 是 2023H，因此相加得 4045H，C 正确。

28．B。【解析】变址寻址的有效地址的计算式为：EA =（IX）+A，其中 IX 中的值 1000H，形式地址 A 是 2022H，因此相加得 3022H，B 正确。

29．B。【解析】设第 2 字节的内容为 x，则 2008H + 2 + x=2001H，所以 x=2001H-200AH=-9，其补码为 F7H，B 正确。

30．C。【解析】变址寻址是面向用户的，用于访问字符串、向量和数组等成批数据，变址寄存器的内容在程序执行过程中可以修改。C 正确。

31．A。【解析】I 错误，有效地址为 1200H+000CH=120CH，注意转化为 16 进制。II 错误，操作数的地址为 38B8H，操作数为 88F9H。III 错误，有效地址为 1200H。

32．C。【解析】CISC 指令系统指令数目多、指令格式种类多，常采用微程序控制技术实现复杂指令功能；但由于指令复杂，执行步骤多，导致指令执行速度慢，C 错误。

33．B。【解析】RISC 的设计原则是通过简化指令功能，使指令格式规整，减少指令的平均执行周期数，提高执行效率，B 正确；RISC 减少指令功能复杂度，多采用定长指令字格式，减少寻址方式种类，A、C、D 错误。

34．C。【解析】CISC 指令功能复杂，能直接实现复杂操作，编程灵活性好，C 正确；注意，这里的灵活性针对的是单条指令的功能，虽然 RISC 和 CISC 都可以用不同指令序列来实现同一个功能，但是 CISC 可以更直接、更方便的用更少的指令来实现，所以更灵活。

RISC 指令执行速度快，更适合实时系统，A 错误；CISC 因指令复杂，硬件实现复杂度更高，B 错误；RISC 指令系统指令数目少，D 错误。

35．C。【解析】RISC 架构通常大量使用寄存器，以减少访存次数；采用流水线技术提高执行效率；指令格式规整便于硬件实现；而微程序控制常用于 CISC，RISC 多采用硬布线控制，C 错误，A、B、D 都是 RISC 可能的特点。

36．D。【解析】选择结构语句在机器级代码中，通过控制转移指令（如条件跳转指令等）根据条件判断结果决定程序执行流程，跳转到不同的代码段执行，D 正确。

37．B。【解析】jle（有符号数 ≤ 跳转）条件为 $(SF \oplus OF) + ZF = 1$，即 SF、OF、ZF 的组合，B 正确。

解释：ZF=1 表示等于，小于的条件是不溢出为负，溢出为正。即 SF 和 OF 异号。

38．A。【解析】D9H 先符号拓展为 FFFFFFD9H，再和 PC 做和。00401037H + FFFFFFD9H = 00401010H，A 正确。

39．D。【解析】循环结构语句转换为机器级代码后，一般包含循环条件判断（决定是否继续循环）、循环变量更新（改变循环状态）以及无条件跳转至循环体开头（实现循环执行）；函数调用不是循环结构机器级代码的必然组成部分，D 错误。

40．B。【解析】在函数调用的机器级实现中，首先传递参数，然后保存现场（保存调用函数的寄存器等状态），接着执行函数体，执行完毕后恢复现场，最后返回（回到调用函数继续执行），B 正确。

41．D。【解析】当子程序调用 CALL 指令时，首先需要将程序断点（PC 的值）保存在栈中，然后将 CALL 指令的地址码送入 PC。因为指令为双字长，所以取出 CALL 指后，PC 的值需要加 2，即 2002H。当 CALL 指令执行后，先将 SP 的值减 1，此时（SP=00FFH），然后程序断点 2002H 进栈，栈顶内容为 2002H，D 正确。

42．A。【解析】调用指令也必须能够确定子程序入口地址，通常由指令显式或隐式给出。

43．B。【解析】在 x86 函数调用约定中，EBP（基址指针）用于固定指向当前函数栈帧的底部，以方便通过偏移量访问局部变量和参数。当函数准备返回时（执行 leave 指令或函数结尾处的清理代码），会执行 POP EBP，将栈中保存的上一层函数的 EBP 值弹出并恢复，从而将栈帧上下文交还给调用者。

第 5 章

中央处理器（CPU）

【答案速对】

CBCCC　ACACD　DBCDC　BABBC

DADBC　ABCCD　BDBBB　DCCBA

CCDAA　DCCCB　CBBCB　BCADC

BABDA　DABCD　DDCBD　BBCCC

BABCA　A

1．C。【解析】CPU 的主要功能包括指令控制（控制程序的执行顺序）、操作控制（产生控制信号控制部件执行操作）、时间控制（对各种操作进行时间上的控制）和数据加工（对数据进行算术和逻辑运算），而数据存储主要由存储器完成，C 错误。

2．B。【解析】A 选项：首先需要注意一点，不管是应用程序员还是汇编程序员，都不能直接读写程序计数器（PC），它的值一般是由硬件自动更新（如取指后自增）或通过跳转指令间接修改（如 jmp、call），因此我们说 **PC 对于汇编程序员可见**（2010 真题），袁书说法：**PC PSW 属于用户部分可见寄存器**；从另一个角度说，例如中断响应保存断点（PC PSW），之所以我们通过硬件压栈保存，而不是像现场信息（通用寄存器等）通过软件保存（例如通过汇编指令 mov 等保存），这个也是重要原因；注意，PC 是 X86 里的程序计数器，而像 MIPS 架构里，通常使用 ra 寄存器作为程序计数器，此时的 ra 寄存器由于可以通过汇编指令显式保存，因此 ra 对于汇编程序员来说，可直接读写；

B 选项：变址寄存器（如 x86 中的 SI、DI，或通用寄存器用作变址）是用户级通用寄存器，通常用于变址寻址，**面向用户**，可用于访问字符串、数组、表格等成批数据或其中的某些元素；变址寻址时，指令中提供的形式地址是一个基准地址，位移量由变址寄存器给出，因此应用程序员可直接读写；

C 选项，当 CPU 处于用户态时，SP 指向用户栈栈顶，此时应用层程序员可直接读写，当 CPU 处于内核态时，SP 指向内核栈栈顶，此时应用层程序员没有权限读写，只有系统程序员可以读写；

D：PTBR 存放的是页表的起始地址，页表属于内核数据结构，那 PTBR 肯定只能由 OS 内核进行读写；

3．C。【解析】I. 程序状态字寄存器（PSW, Program Status Word）**可见**，这是系统级程序员用来控制和查看处理器状态的寄存器，比如中断标志、状态标志等。其内容由 CPU 根据指令执行结果自动设定，用户程序执行过程中可能会隐含读取到其部分内容（如 OF CF 等溢出标志位），以确定程序的执

行顺序（例如跳转指令进行转移条件判断），但是不能直接修改 PSW 寄存器的值；

Ⅱ. 段表基址寄存器，可见，用于支持分段机制，系统级程序员需要设置和管理段表基址。

III. 指令寄存器不可见，存储当前正在执行的指令，一般由硬件自动操作，程序员不能直接访问。

IV. 程序计数器可见

V. Cache 不可见，Cache 是高速缓存，属于硬件内部结构，程序员不能直接访问

VI. 存储器数据寄存器（MDR, Memory Data Register）不可见，用于临时存储从内存读取的数据或要写入内存的数据，通常不被程序员直接访问。（同理 MAR 也不可见）

VII. 系统堆栈指针寄存器（SP, Stack Pointer）可见，系统级程序员可以访问和修改堆栈指针来管理函数调用和局部变量。

VIII. 地址寄存器专门用来存放首地址或指针信息，如变址寄存器、基址寄存器等，因此可见；

IX. 中断屏蔽字寄存器可见，系统程序员可通过修改该寄存器调整中断处理顺序；

4．C。**【解析】**A 属于，减少访问主存的次数：更多寄存器允许更多数据暂存于 CPU 内部，减少对慢速主存的访问。

B 属于，提高 CPU 的处理效率：减少内存访问延迟，CPU 能更快执行指令。

C 不属于，缓存命中率取决于缓存的大小、关联性、替换算法和访问局部性，与通用寄存器数量无关。增加通用寄存器可以减少对主存的访问次数（从而间接减少缓存访问次数），但不会改变缓存本身的命中率（即命中访问占总访问的比例）。例如，如果缓存命中率原本为 90%，增加寄存器后访问次数减少，但命中率可能保持不变（如访问 100 次命中 90 次 vs. 访问 50 次命中 45 次，命中率仍为 90%）。许多人容易混淆“减少内存访问”和“提高缓存效率”，误以为减少访问次数自然会提升缓存命中率。这在计算机体系结构题中是一个常见陷阱，因为缓存和寄存器都涉及存储层次，但缓存命中率是缓存系统的固有属性，不受寄存器数量影响。

D 属于，通用寄存器每一个都有对应的地址编号，更多的寄存器给了程序员和编译器更多灵活性，可以更好地分配变量，减少寄存器压力。

5．C。**【解析】**C 正确，程序计数器 PC 必须要有，因为它用来保存下一条要执行的指令的地址，CPU 需要知道从哪里取下一条指令，否则无法顺序执行或处理跳转。

A 错误，在单周期 CPU 设计中，指令直接从指令存储器（或指令 cache）中获取并立即执行，而不需要显式的 IR 寄存器（例如，指令位直接连接到控制逻辑）。因此 IR 不是必需设置的，取决于实现方式。

B 错误，存储器数据寄存器 MDR 用于在 CPU 和内存之间传输数据。在指令集设计中，如果内存访问通过通用寄存器直接处理（如在 load-store 架构中），则 MDR 不是必需设置的。它通常作为内存接口的缓冲，但可通过其他方式实现。

D 错误，存储器地址寄存器 MAR 用于存储要访问的内存地址。类似地，如果地址可直接从通用寄存器或 ALU 输出驱动到地址总线，则 MAR 不是必需设置的。

6．A。**【解析】**PC（程序计数器） 存储下一条指令的地址，其位宽通常与主存地址总线位宽相关，但不一定相等。

反例：

- x86 实模式中，物理地址由段地址（16 位）左移 4 位加上偏移量（16 位）组成，形成 20 位物理地址。此时 PC（IP）为 16 位，而地址总线为 20 位，PC 位宽小于地址总线位宽。
- 指令对齐优化：某些架构中，指令按字对齐（如 4 字节对齐），PC 的低位固定为 0（如 ARM 中 PC 的最低 2 位为 0），此时 PC 的有效位数可能少于地址总线位宽。

因此 PC 的位宽可能小于或等于地址总线位宽，但不一定相等，因此选项 A 错误。

在定长指令字系统（如 MIPS、ARM）中，所有指令长度固定（如 32 位），IR 的位宽必须等于指令字宽度，以完整保存指令。

MAR（内存地址寄存器）保存当前访问的内存地址，其位宽需覆盖整个物理地址空间。地址总线位宽决定了可寻址的物理地址范围，因此 MAR 的位宽必须与地址总线位宽相等。

MDR（内存数据寄存器）暂存从内存读取或写入的数据，其位宽需与数据总线宽度匹配，以保证一次传输完整的数据单元。例如，若数据总线为 64 位，MDR 也需为 64 位，以避免多次传输。（虽然理论上 MDR 可以和数据总线宽度不一样，但是 408 范围内可以认为他们相等）

7．C。**【解析】**C 错误，顺序执行时，PC 的值不一定是+1，具体增加的值跟指令字长和编址单位有关，比如 32 位定长指令，按字节编址，那么按 PC+4 更新。

B 正确，调用指令（如 call）将返回地址压栈后，立即跳转到被调用过程的入口地址，因此执行后 PC 的值就是入口地址。

D 正确，无条件转移指令（如 jump）直接修改 PC 为指定的目标地址，执行后 PC 的值即为转移目标地址。而条件转移指令（如 je、jne）根据 PSW（程序状态字）中的标志位（如 CF、OF、ZF）决定是否跳转。

A 正确，指令周期内 PC 可能改变 2 次的情况（非单周期处理器），比如：在多周期处理器或流水线处理器中，指令周期可能被拆分为多个阶段（如取指、译码、执行、访存、写回）。PC 可能被修改两次：①在取指阶段结束时，PC 通常被更新为下一条指令的地址（如 `PC = PC + 4`），为后续指令做准备。②如果当前指令是分支或跳转指令（如 `JMP`、`BEQ`），在执行阶段结束时/访存阶段结束时（看具体电路设计），PC 会根据条件更新为分支目标地址。

而单周期处理器中 PC 只改变一次，因为在单周期处理器中，所有指令的执行（包括取指、译码、执行、写回）都在一个时钟周期内完成。PC 仅在指令周期结束时被更新一次：

①顺序指令，PC 更新为 `PC + 4`（或其他增量）。

②分支/跳转指令，PC 直接更新为目标地址。

PC 不会在中间阶段被修改，因为所有操作是组合逻辑的组合，PC（所有寄存器）的更新是原子操作（发生在时钟边沿）。

8．A。**【解析】**处理器根据自身的时钟信号来对指令的执行进行定时，但是并不是每一个时钟信号就开始执行一条新的指令，例如在多周期处理器（非流水线）中，一条指令的执行需要分解成多个阶段进行，每个阶段在一个时钟周期内完成一个特定的功能，因此每来一个时钟开始执行一个阶段性操作，A 错误。

每来一个时钟开始进行阶段性操作，因此必须确定时钟的那一刻算开始，通常这个开始点或是时钟信号的上升沿或是下降沿，称为触发边沿。触发器或寄存器等状态元件总是在触发边沿开始改变状态，B 正确。

为保证每个阶段的操作都能在一个时钟周期内完成，时钟周期必须足够长，以保证数据在最长延迟的组合逻辑电路中能够传输完成。因此时钟周期以相邻状态单元最长组合逻辑延迟为基准确定，C 正确。

D 显然正确。

9．C。【解析】A. 正确，指令周期是指 CPU 执行一条完整指令所需的时间，它可以被分解为多个机器周期（如取指周期、执行周期等）或更细粒度的时钟周期（CPU 时钟的基本单位）。

B.正确，在单周期 CPU 设计中，所有指令（包括加法和乘法）都在一个时钟周期内完成，因此指令周期等同于一个时钟周期。时钟周期的长度通常根据最复杂指令（如 LOAD 指令）的延迟来设定，以确保所有指令都能在一个周期内完成。

C.错误，指令周期的长度取决于指令的复杂度和 CPU 的设计：

- 在单周期 CPU 中，所有指令周期长度相同（即一个时钟周期），但加法指令通常比乘法指令简单，时钟周期的长度可能被拉长以适应乘法指令，导致效率较低。
- 在多周期 CPU 或流水线 CPU 中，不同指令的执行时间通常不同：加法指令可能只需一个或几个时钟周期，而乘法指令由于涉及更复杂的操作（如迭代计算），可能需要更多时钟周期。因此，乘法指令和加法指令的指令周期并不总是一样长。

D 正确，无论指令类型如何，CPU 执行任何指令的第一步都是从内存中取出指令，然后才能进行解码、执行等后续阶段。这是指令执行的基本流程。

10．D。【解析】I. MAR ←Ad（IR）：将指令中的形式地址字段的值传输到存储器地址寄存器（MAR），直接寻址和间接寻址都是可能的。

II是可能的。MAR ← (PC)：将程序计数器（PC）的内容传输到地址寄存器（MAR）。寻址方式角度：在 PC 相对寻址或取指令阶段，PC 的值常被用于地址计算或内存访问：

①取指令阶段：PC 的内容传输到 MAR，以便从内存读取指令（这是指令周期的公共操作）。

②PC 相对寻址：操作数地址由 PC 值加上偏移量计算得出，计算前可能需要将 PC 值加载到 MAR 或相关寄存器。

III是可能的。MAR ← (Ri)：将通用寄存器 Ri 的内容传输到地址寄存器（MAR）。寻址方式角度：在寄存器间接寻址中，通用寄存器 Ri 存储操作数的地址，该地址可直接加载到 AR 以访问内存；例如，指令如 `MOV [Ri], ...` 或 `LOAD [Ri]` 需要将 Ri 的内容（地址）设置到 MAR。

IV是可能的。Ri ← (MDR)：将数据缓冲寄存器（MDR）的内容传输到通用寄存器 Ri。寻址方式角度：在数据加载操作中，MDR 暂存从内存读取的数据，之后需传输到通用寄存器：例如，在直接寻址或寄存器间接寻址的加载指令（如 `MOV Ri, [addr]`）中，数据从内存读到 DR 后，再传输到 Ri。

11．D。【解析】B.正确，寄存器寻址（如 ADD R1, R2）的操作数直接来自寄存器，无需访存取操作数。，因此不需要单独的取数周期（"取数周期"特指从内存取操作数）

D 错误，取指周期的微程序是微程序控制器直接提供，根据微程序控制器的取指逻辑来自动实现（比如断定法），跟指令的操作码无关。

12．B。【解析】B 错误，存储器的读/写事务（访问主存）通常比 CPU 内部操作（如寄存器访问或 ALU 计算）慢得多（内存访问延迟大）。因此，机器周期的宽度（即持续时间）应以存储器访问时间为基准来确定，以确保有足够时间完成内存操作。如果以 CPU 内部操作时间为基准，机器周期会太短，无法完成内存访问，导致错误。早期设计中，机器周期的长度常由内存速度决定。

C 正确，早期三级时序系统（指令周期、机器周期、时钟周期）中，为简化设计，许多系统采用固定时序：所有指令有相同指令周期、相同机器周期数（如取指、执行等），且每个机器周期包含固定数量的时钟节拍（T-states）。

D 正确，现代 CPU 引入 cache 后，无需慢速主存访问。因此，指令周期是变长的（不同指令执行时间不同），直接由时钟周期组成，不再需要“机器周期”这一概念（机器周期原是为管理慢速内存总线事务而设）。

13．C。【解析】间址周期是指当指令采用间接寻址方式时，CPU 获取操作数有效地址的过程，C 正确。

14．D。【解析】指令译码器对指令的操作进行译码，所以选 D。

15．C。【解析】数据通路定义：数据在功能部件之间传输的物理路径，包含 PC、通用寄存器、ALU、暂存寄存器、PSW 状态寄存器等运算存储部件；CU（控制单元）是产生控制信号、指挥通路工作的控制器，不属于数据通路本身，只是控制数据通路运行。

16．B。【解析】数据通路是 CPU 中实现数据流动和处理的路径，主要功能是实现 CPU 内部的运算（如 ALU 运算）和数据传输（如寄存器之间、寄存器与 ALU 之间的数据传输），B 正确；存储程序和数据是存储器的功能，A 错误；控制指令执行顺序是控制器的功能，C 错误；控制信号（如 RegWrite, ALUOp）由控制单元生成，而非数据通路。数据通路被动响应控制信号（如根据 ALUOp 执行加法），但不产生控制信号，D 错误。

17．A。【解析】在单周期数据通路中，所有指令的执行时间相同，都等于最长指令的执行时间，以确保所有指令都能在一个时钟周期内完成，A 正确。

18．B。【解析】A 正确，这也可以看作是时钟周期的定义，和 19 年真题中“时钟周期以相邻状态单元间组合逻辑电路的最大延迟为基准确定”的含义一样；

B 错误，MIPS 数据通路中，寄存器堆的边沿触发写入和异步读出机制已经处理了同一个周期内的读写操作，不需要额外的备份。读写同一个寄存器时，读出自然得到旧值，写寄存器是根据时钟结束时的时钟沿时候进行数据更新，不影响时钟内的读操作，无需备份。

【补充】：寄存器堆的操作特性：

- 在 MIPS 数据通路中，寄存器堆（register file）通常有两个读端口和一个写端口。
- 写入操作：使用边沿触发（例如，时钟上升沿），在时钟边沿瞬间更新寄存器内容。
- 读出操作：是组合逻辑（异步），输出基于输入的地址立即响应，不需要时钟边沿。
- 在同一个时钟周期内，寄存器堆可以同时进行读和写操作（例如，一条指令读取源寄存器并写

入目的寄存器）。

C 正确，时钟周期内的操作流程



何为边沿触发寄存器：在时钟信号（如上升沿）瞬间捕获输入数据，并保持该值直到下一个时钟沿。读取的是上一周期存储的值（时钟沿前的状态），写入的是下一周期的值（时钟沿后更新）。因此同一时钟周期内，前半段组合逻辑用寄存器的旧值计算，后半段计算结果作为新值在时钟沿被写入寄存器。另外注意：从寄存器输出 → 组合逻辑计算 → 到达下一寄存器输入的时间，必须 < 一个时钟周期，否则导致时序违例。（软硬件原理原文补充：在这种设计下，必需保持时钟周期足够长，以使得当有效的时钟边沿到来之时，输入已经稳定。状态单元的改变由时钟边沿触发，所以不可能在一个时钟周期之内出现反馈）

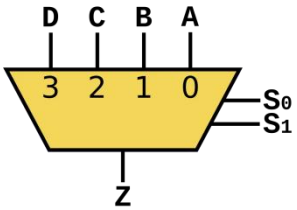
D 正确；补充：如果该寄存器不是每个时钟周期都进行修改，那么它就需要这个写控制信号，写使能和时钟都是输入信号，只有当写使能有效并且时钟边沿到来之时，状态单元才改变状态；

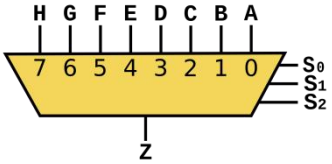
19．B。【**解析**】组合逻辑元件的输出只取决于当前的输入，组合电路不含存储信号的记忆单元。组合电路的定时不受时钟信号的控制，所有输入信号到达后，经过一定的逻辑门延迟，输出端的值被改变，并一直保持其值不变，直到输入信号改变。数据通路中常用的组合逻辑元件有多路选择器(MUX)、加法器(Adder)、算术逻辑部件(ALU)、译码器(Decoder)、三态门、符号扩展单元等；

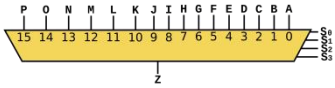
时序逻辑元件（状态元件）**任何时刻的输出不仅与该时刻的输入有关，还与该时刻以前的输入有关**，因而时序电路必然**包含存储信号的记忆单元**。寄存器、GPRS、理想存储器、cache 都属于时序逻辑元件。

20．C。解析：本题灵感来自于 2024 年 408 真题 43 题的第二小问，属于单周期数据通路的考察，完全可以当大题做，难度较大，建议目标高分的同学务必理解整个数据通路不同指令数据的流向，控制信号的设置；

Ⅰ正确，**k 路选择器**，应该有 **k** 路输入，因而控制端 **S** 的位数应该是\left \lceil log_2k \right \rceil，例如，**对于 3 路或 4 路选择器，S 有两位；对于 5-8 路选择器，S 有 3 位。**







II正确，整个访存过程为

①根据主存地址中的 cache 组号找到对应组；

②将主存地址中的标记与对应组中每个行的标记 Tag 进行比较；

③将比较结果和有效位 V 相“与”（观察硬件电路图中的比较器和与门）；

④若有一路比较相等并有效位为 1，则输出“Hit”（命中）为 1，并选中这一路 cache 行中的主存

块；

⑤在“Hit”为 1 的情况下，根据主存地址中的块内地址从选中的一块内取出对应单元的信息，若“Hit”不为 1，则 CPU 要到主存去读一块信息到 cache 行中。

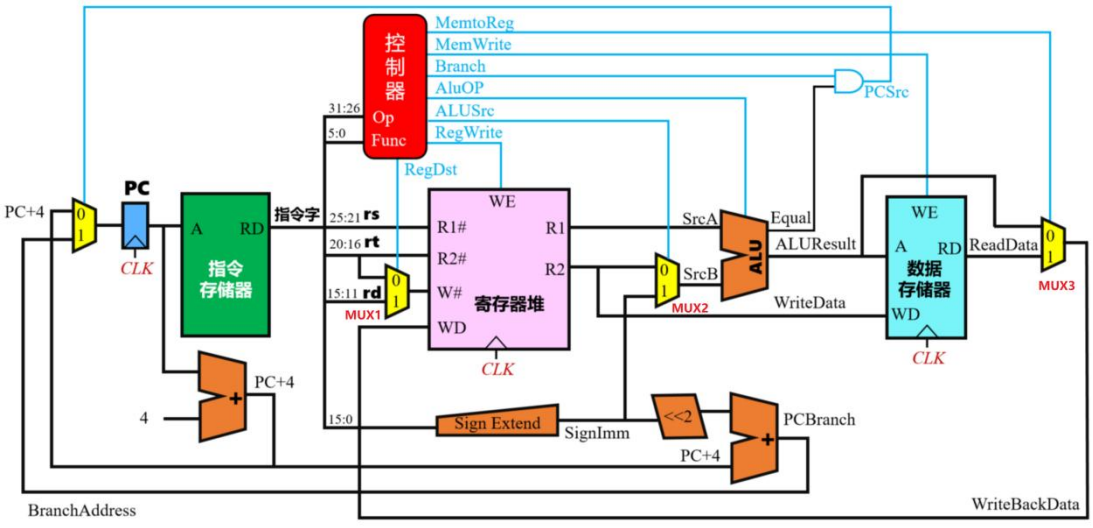
III错误，对于多路选择控制信号，0 和 1 均表示有效信号，以选择不同的输入作为输出。对其他信号来说(如 RegWrite)，1 表示有效，0 表示无效。

IV错误，根据图中信息可知，控制信号 MemToReg 控制 MUX3，当 MemToReg = 0 时，选择 ALUResult 这个分支作为输入，其作用是用于 R 型指令例如 add 指令把寄存器相加结果写回寄存器堆；当 MemToReg = 1 时，选择 ReadData 这个分支作为输入，其作用是用于 lw 指令读取数据存储器的数据并送入寄存器堆写数据端口 WD；

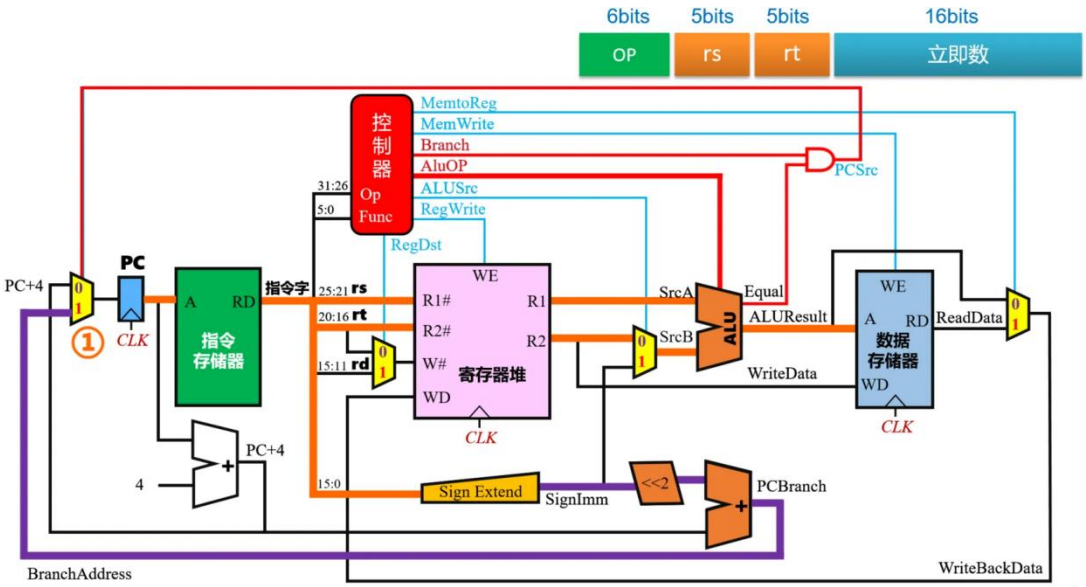
V正确；根据图中信息可知，控制信号 AluSrc 控制 MUX2，当 AluSrc = 0 时，选择 R2 这个分支作为输入，其作用是用于 R 型指令或 I 型指令，例如 add 指令中选择 ALU 的 SrcB 数据来源为寄存器堆；当 AluSrc = 1 时，选择经过符号扩展单元扩展后的立即数 SignImm 这个分支作为输入，执行 sw 指令时，根据 sw 指令的功能 R[rt]→M[R[rs]+SignExt(imml6)]可知，需要选择该分支作为 ALU 的 SrcB 然后与变址寄存器 rs 的值相加后**形成最终的主存地址**；

【扩展】：如图，执行 beq 指令时，MUX2 的控制信号 AluSrc 的取值是 0 还是 1？

```
Plain Text
beq rs, rt, imm16    #RTL 功能描述:if(R[rs]==R[rt])
PC+4+SignExt(imml6)<<2-PC
```



答案是 0；执行 beq 指令时，ALU 的输入是 rs 和 rt 对应的寄存器，需要比较这两个寄存器的值是否相等；指令字中的立即数字段为 16 位有符号数，是分支目标地址与顺序指令地址 PC+4 之间的字偏移，**可正可负(既可向后跳转，也可向前跳转)**。由于指令长度为 4 字节，因此需要将 16 位立即数**符号扩展**成 32 位后并左移两位生成 32 位字节偏移量，再加上顺序指令地址 PC+4 才能得到分支目标地址。当比较条件满足时，用分支目标地址修改 PC 的内容，即产生分支跳转；反之将 PC 的值更新为 PC+4，程序顺序执行。这里 **PC 输入端也需要增设一个多路选择器**，用于选择顺序执行还是跳转执行。具体数据流向如图所示



21．D。【解析】指令译码器不需要显式控制信号，可直接由输入的指令生成对应指令的逻辑信号，D 错误；ALU 的运算类是由控制器输出的 ALU 控制信号控制的，A 正确；MUX 的作用是从多个输入中选择一路输出，选择逻辑由选择信号决定。该选择信号由控制器生成，用于指定数据通路的走向，需要控制信号，B 正确；三态门的使能信号由控制器发出，C 正确。

22．A。【解析】I. 正确

Ⅱ.数据通路确实可分为总线式（部件通过共享总线连接）和非总线式（点对点连接），但单周期处理器通常不是典型的多总线数据通路。单周期处理器（如经典 RISC 设计）使用硬连线控制和非总线结构（如直接连接和多路选择器），是专用数据通路而非多总线。总线式结构更常见于多周期或微程序控制处理器中。

III.总线式数据通路（尤其是单总线结构）在任意时刻只能传输一组数据，导致流水线中多个阶段无法同时访问总线，从而产生资源冲突。例如，在五级流水线中，取指、译码、执行、访存、写回阶段可能都需要访问总线，但单总线无法支持这些操作并行进行。因此，总线式数据通路无法有效实现指令流水线。

Ⅳ.【该选项是袁书习题解答 P109 原话】非总线型数据通路（如专用路径结构）通常允许并行操作，避免总线争用，更适合实现高性能的流水线设计。注意，**单周期和流水线都属于专用数据通路**，根据指令执行过程中的数据和地址的流动方向安排连接电路，避免使用共享的总线。

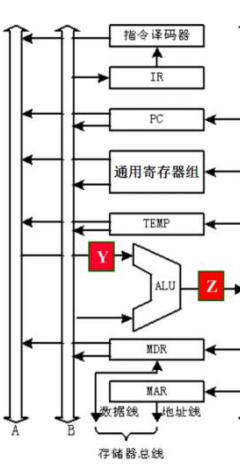
V.指令的执行阶段会根据其类型（如算术指令、访存指令、跳转指令）而使用不同的路径和部件（如 ALU、内存访问模块等）。然而，取指令阶段通常是统一的：程序计数器（PC）提供地址，从指令存储器中取出指令，加载到指令寄存器（IR）中。因此，该说法准确描述了数据通路中指令执行路径的差异性与一致性。

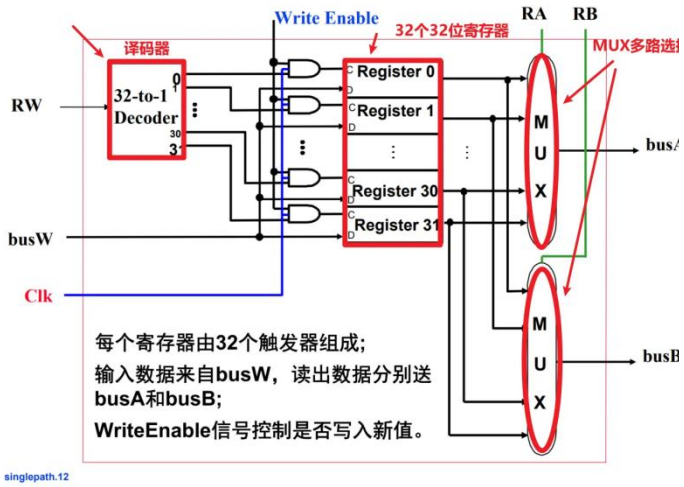
23．D。【解析】阻塞译码阶段可以禁止新指令进入，但仍需阻塞执行及后续阶段来暂停当前指令推进，D 错误；寄存器采用边沿触发（如时钟上升沿锁存数据），其控制信号需满足建立时间（Setup Time）要求，即在时钟沿到来前保持稳定，避免时序错误，A 正确；存储器访问时，地址信号需先稳定（前半周期），再激活读信号（后半周期），以确保地址有效，B 正确；在流水线中，译码阶段解

析指令操作码，生成 AluOp 信号，并传递至执行阶段控制 ALU 操作，C 正确。

24．B。【解析】I正确，RA 宽度有 5 位，最多可有$2^5=32$ 个寄存器；

Ⅱ正确、IV 错误，寄存器堆的读口由 MUX 构成，输入是各个寄存器的内容，通过指令中寄存器编号来决定具体是哪个寄存器输出，读操作只跟当前输入和控制信号相关，无需时钟控制，所以读操作是组合逻辑操作，IV 错误。因为采取了 busA 和 busB 两个输入总线和 busW 输出总线，可以实现两个寄存器取数据进行运算然后写入目的寄存器，II 正确。如左图所示，因为三个总线各自传不同数据，不会发生冲突，故无需 Y 和 Z，只要一个时钟周期（节拍）即可。





III错误（务必掌握）多路选择器是一种能够从多个输入信号中选择一个输出的信号转换设备。译码器是一种将二进制编码转换为一组特定输出信号的设备。在寄存器堆中，译码器常常用于地址译码，从而控制对特定寄存器的访问。执行寄存器写相关指令时，指令中的寄存器编号被送到一个地址译码器进行译码，选中某个寄存器进行写入，读出时寄存器编号作为控制信号来控制一个多路选择器，选择相应的寄存器读出。

译码器：这里是 5-32 译码器，输入端有 5 位，输出端有$2^5=32$ 位，记为$O_0, O_0, O_0, ..., O_{2^n-1}$；若输入端对应的二进制值为$i$，那么输出端只有$O_i$为 1；从图中我们可以看到，这个译码器的 32 个输出端分别连接着 32 个寄存器，然后跟 clk 经过与门相与后作为它们的写使能端口；当需要执行写操作时，将写入寄存器的编号送入译码器的输入端口，这样会有一个输出端口输出为 1，等到 clk 到来，与之相与后，则会使被选中的那个寄存器的写使能有效，然后，busW 传来的内容便会被写入该寄存器；至于其他寄存器，由于写使能无效，因此不会被写入数据。

25．C。【解析】三态门类似于一个开关，开关的连接或断开由三态门的控制端决定，C 正确。

26．A。【解析】MIPS 架构的 beq 指令的功能是，根据指令指定的两个寄存器内容是否相等(通过做减法来比较是否相等)，确定是否跳转到指定的转移目标地址执行。因此，其通路设计中，ALU 的两个输人都来自寄存器堆，ALU 的控制信号一定为 subu(即 ALU 做减法)，并且，一定有一个加法器用于计算转移目标地址。因为转移目标地址的计算需要将 PC 的内容和偏移量(16 位立即数符号扩展为 32 位)相加，所以，数据一定会流经符号扩展部件。因此，①、②、③ 的叙述是正确的，即正确选项为 A。

27．B。【解析】组合逻辑电路不一定需要控制信号参与，如指令译码器，A 错误。时钟周期本身

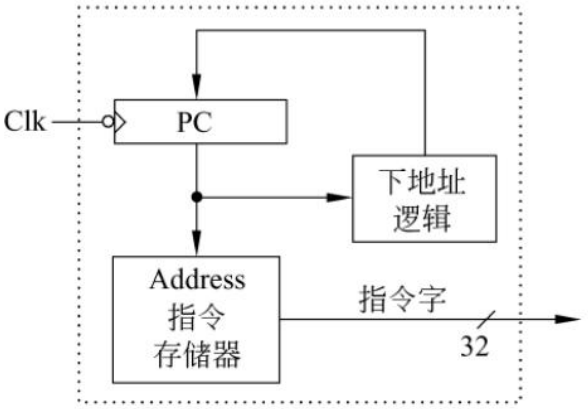
就是取自所有组合逻辑和时序逻辑部件中的延迟最大值再加一定的缓冲值，所以 C 错误，寄存器由触发器组成，属于时序逻辑部件，D 错误。单周期处理器需要在一个时钟周期内完成所有操作，所以取指必须不受时钟控制就能完成，因此指令存储器必须被设置成组合逻辑电路单元，B 正确。

28．C。【解析】根据题目的描述可知，取数 Load 是最复杂的指令，在该单周期数据通路中，应该以 Load 所用时间作为时钟周期。因此，在不考虑多路复用器、控制单元、PC、符号扩展单元和传输线路等延迟的情况下，该计算机时钟周期为取指令时间(3ns)十寄存器堆的读取时间(1ns)+ALU 运算时间(2ns)+存储器取数时间(3ns)+寄存器堆写时间(1ns)=10ns。选项 C 是正确的。

29．C。【解析】IR 存放指令，包括该条指令的全部信息，如操作码、地址码、寻址特征等等。

30．D。【解析】在多周期数据通路中，一条指令的执行由多个时钟周期（阶段）来控制，不同指令所含时钟周期数不同，I正确；每个不同的阶段数据通路完成不同的功能，所以控制信号在每个时钟周期到来的时候改变其值，IV正确。在单周期数据通路中，每条指令在一个时钟周期内完成，II正确；在指令取出后，指令操作码字段及相关的控制字段送到控制逻辑，由控制逻辑统一得到各个控制信号，III正确；每个控制信号被送到相应的控制点，一直作用在数据通路中，每个部件只完成一种功能，不会被多次使用，V正确。综上所述，本题答案为 D。

31．B。【解析】A 错，因为是单周期处理器，每个时钟周期执行一条指令，所以每来一个时钟，PC 的值都会被更新一次，因而，PC 无须“写使能”信号控制。



单周期处理器取值部件：假定指令专门存放在指令存储器中，它只有读操作，读指令操作可看成是组合逻辑操作，因此无须控制信号的控制，只要给出指令地址，经过一定的“取数时间”后，指令被送出。指令的地址来自 PC，有专门的下地址逻辑来计算下条指令的地址，然后送 PC。**因为是单周期处理器，每个时钟周期执行一条指令，所以每来一个时钟，PC 的值都会被更新一次，因而，PC 无须“写使能”信号控制。**下地址逻辑中，要区分是顺序执行还是转移执行。若是顺序执行，则执行 PC+4；若是转移执行，则要根据当前指令是分支指令还是跳转指令来计算转移目标地址。

B 正确，因为 CPU 需要从存储器中读取指令。

C 错误，因为该时钟周期结束后，要执行下一条指令，在执行下一条指令之前，必须在本条指令中得到下一条指令的地址，也可参考上图的取指部件

D 错误，因为单周期指令需要在一个时钟周期内完成指令的所有操作，例如 LOAD 指令需要从指令

存储器中取指和根据主存地址从数据存储器中取数，因此不能存放在同一个存储器中。

32．D。【解析】A 正确，硬布线控制器核心是组合逻辑电路；

B 正确，微程序控制器是用微指令序列（存储在控制存储器中）来组织控制逻辑，其结构更规整、易于修改维护。设计层次化、逻辑清晰，便于扩展指令集。注意这里的规则是指硬件设计上的规整，注意区分 C 选项中的指令系统规整。

C 正确，MIPS 作为 RISC 架构，指令长度固定、格式统一（如所有指令 32 位）。硬布线控制器的优点是速度快，适合于实现简单或规整的指令系统。但是，它是一个多输入/多输出的巨大的逻辑网络。对于复杂的指令系统来说，其结构庞杂，实现困难，修改、维护不易，灵活性差。

D 错误，

- 微命令：控制功能部件完成基本操作的信号（如 ALUOp=加法）。
- 微操作：由微命令触发的原子操作（如“寄存器 A 内容送总线”）。
- 两者是通用概念，并非微程序控制器专属：
  - 硬布线控制器：微命令由组合逻辑电路直接生成（如通过指令译码器输出 `RegWrite` 信号）。
  - 微程序控制器：微命令从微指令字段译码得到。
- 关键区别：
  - 微程序控制器中，微命令以微指令形式存储在 ROM 中；
  - 硬布线控制器中，微命令由硬件逻辑实时生成。

33．B。【解析】硬布线控制器通过硬件逻辑直接生成信号，速度更快；微程序控制器需查微指令，速度较慢，B 正确。

34．B。【解析】在微程序控制器中，微指令存放在专门的控制存储器中，B 正确。

35．B。【解析】一条（机器）指令的功能实现由一个微程序实现（取指微程序是独立的，且由所有指令公共使用，不包含在这个指令对应的微程序里面）。一个微程序包含多条微指令，一条微指令包含一个或多个微命令，微命令就是常说的控制信号。同样，一条微指令包含一个或多个微操作，微操作和微命令是对应的，微命令是微操作的控制信号，微操作是对应控制信号控制实现的具体操作。

36．D。【解析】因为每个段要留出 1 个状态来表示该段什么控制信号也不发出。所以在计算字段长时，应该取$\lceil log_2(k+1) \rceil$，因此各互斥类所需位数：

$$\lceil log_2(7+1) \rceil = 3, \lceil log_2(4+1) \rceil = 3，\lceil log_2(12+1) \rceil = 4，\lceil log_2(5+1) \rceil = 3，\lceil log_2(6+1) \rceil = 3，\lceil log_2(6+1) \rceil = 3，\lceil log_2(9+1) \rceil = 4$$

故总位数 $= 3 + 3 + 4 + 3 + 3 + 3 + 4 = 23$ 位。

操作控制字段至少有 23 位。

37．C。【解析】按字编址，每条微指令占一个地址，执行第 2 条时（说明取指完成），μPC 指向下一条地址即 A0H + 2 = A2H，C 正确。

38．C。【解析】控制器由程序计数器 PC、指令寄存器 IR、地址寄存器 MAR、数据寄存器 MDR、指令译码器、时序电路和微操作信号发生器组成。程序状态字（PSW）寄存器属于运算器的组

成部分，PSW 包括两个部分：一是状态标志，如进位标志 CF、结果为零标志 ZF 等，大多数指令的执行都会影响到这些标志位：二是控制标志，如中断标志、陷阱标志等，C 错误。

39．B。【解析】水平型微指令并行操作能力强.效率高，灵活性强，垂直型微指令则较差。在一条水平型微指令中，设置有控制信息传送通路(门)以及进行所有操作的微命令，因此在进行微程序设计时，可以同时定义比较多的并行操作的微命令，来控制尽可能多的并行信息传送，从而使水平型微指令具有效率高及灵活性强的优点；在一条垂直型微指令中，一般只能完成一个操作，控制一两个信息传送通路，因此微指令的并行操作能力低，效率低。

40．A。【解析】在并行微程序控制器中，由于取微指令和执行微指令的操作是在两个完全不同的部件中执行的，因此，可以将这两部分操作同时进行，A 正确。

41．C 【解析】微程序控制器比硬布线控制器的速度慢；μPC 是微程序计数器，不能取代 PC；CPU 周期又称机器周期，而不是指令周期，所以答案为 C。

42．C。【解析】形成后继微地址有增量方式和断定方式两种，其中增量方式由 μ PC + 1 形成后继微地址，断定方式由设计者指定或者由设计者指定的判断字段控制形成，C 正确。选项 D 混淆了“机器指令” 与 “微指令” 的层级，D 错误。

43．D。【解析】字段直接编码法字段长度 n 与所能表示的微命令数 m 的关系是$m_{max} = 2^n - 1$，直接控制法每一位代表一个微命令，显然直接控制阀的控存利用率更低，A 错误；字段直接编码法的译码电路更复杂，所以执行速度更慢，B 错误；下址字段法增加了微指令长度，C 错误。故选 D。

44．A。【解析】本题问的是首条微指令地址，由微程序对应的机器指令的操作码译码得到，故本题选 A。

45．A。【解析】取指微程序是每条机器指令公共的微程序，入口地址通常固定（uPC 初始置 0，即 CM 中的首条就是取指微程序的首地址，然后执行取指微程序进行取指)，A 错误。

B 正确，取指微程序需要转移到各个机器指令对应的微程序，通常通过转移微指令实现，把指令寄存器 IR 中的机器指令操作码部分送往地址控制逻辑，其实就是通过一个 ROM 来存储各个微程序的入口地址，操作码就是这个 ROM 的地址，然后在 ROM 中取出入口地址送往 uPC，开始执行各自的微程序。

C 正确，每条机器指令对应的微程序最后一条微指令执行完后，要执行下一条机器指令，也就是要重新开始取指，所以下地址字段就是取指微程序的首地址，通常是全 0。

D 正确，计数器方式，微程序中微指令通常顺序存放在控制存储器内，通过 uPC+“1”顺序执行，所以必须实现 uPC + “1”的功能。

46．D。【解析】异常和中断的处理流程有相似之处，但也存在差异，如异常通常是同步的，与当前执行的指令相关，而中断通常是异步的，由外部设备引起，D 错误；异常通常由 CPU 内部事件（如除零、非法指令等）引起，中断由 CPU 外部事件（如 I/O 设备请求）引起，A、B 正确；异常和中断都会打断程序的正常执行，C 正确。

47．C。【解析】一条指令的处理时间为，取指令时间+执行指令时间＋中断周期时间=3us+7us+3us=13us，C 正确。这里需要注意，中断周期时间不包括中断服务程序的执行时间。思考，若是处于关中断/无中断请求，该指令的处理时间又是多少？答：10us，因为没有中断需要响应，指令

周期就没有中断周期这个阶段。

48．C。【解析】异常是由 CPU 内部事件引起的，如除法运算时的溢出属于算术异常，是内中断，C 正确。

49．C。【解析】键盘输入属于外部设备触发的中断，C 正确。

50．B。【解析】译码阶段解析操作码，检测无效指令，B 正确。

51．C。【解析】外部中断只有在 CPU 中断允许标志置位时才会被响应，C 正确；中断控制器按优先级而非到达顺序排队，A 错误；中断时通用寄存器的保存是由中断服务程序（软件）完成，硬件只隐式保存程序计数器等少量状态，B 错误；CPU 只能在指令的中断允许周期等特定时刻响应中断，而不是在任意时刻立即中断当前指令，D 错误。

52．B。【解析】外部中断（如定时器中断）通常在一条指令执行完毕（写回阶段结束）、下条指令开始前检测，B 正确。

53．B。【解析】A.cache 缺失是硬件事件，由 CPU 硬件透明处理，不会触发进程调度（操作系统的职责）

C. cache 缺失不是软件异常（如页面错误），不需要操作系统或异常处理程序介入。它是硬件自动处理的。

54．C。【解析】中断响应阶段需保存现场并跳转，执行中断服务程序是响应后的步骤，C 正确。

55．B。【解析】A 错误，CPU 在每条指令执行结束时检测中断请求信号，若检测到中断请求信号有效，则进入中断响应周期；若检测到中断请求信号无效，则不会进入中断响应周期

B 正确，真题考过，外设接口中的数据即 I/O 端口的数据。

C 错误，检测时机不同，所做的处理也不同。由于异常的发生与指令的执行同步，与当前指令的执行过程密切相关，因此在指令执行过程中检测到异常立即响应。而中断是由外部设备或者内部定时器发出的，通常是异步发生的，因此在指令执行完成后检测。CPU 对于异常和中断的响应处理在大的方面基本一致，都需要保存断点和程序状态并转到相应的处理程序去执行。但有些细节并不一-样，例如，**在检测到中断请求后 CPU 必须通过中断回答信号启动中断控制器进行中断查询，以确定当前发出的优先级最高的中断请求类型，并通过数据线获取相应的中断类型号。**而对于内部异常，CPU 则无需进行中断回答。

D 错误，在 iret 执行的过程中，会从内核栈中弹出异常/中断响应时保存的 **PC 和 PSW 信息（即断点信息）**，并且会检查异常/中断响应前、后系统是否处于同一个特权级，**如果检查出响应前、后处于同一个特权级**（即异常/中断是在内核程序执行时发生的，若是中断即表示发生了多重中断），则不需要从内核栈中弹出用户栈指针，因为在这种情况下，异常/中断响应时并没有在内核栈中保存用户栈顶指针。如果检查出前、后不是同一个特权级，则需要弹出用户栈指针。

56．B。【解析】**A** 选项错误。虽然增加流水线级数可能缩短每级延迟，但“必然”提高主频的说法过于绝对。实际中，流水线级数增加会引入额外的寄存器延迟（如建立时间、保持时间），且若各阶段延迟不均衡，最慢阶段仍会限制主频。此外，过深的流水线可能导致控制复杂度上升，反而影响性能。因此，当段数到达一定数量时，流水线级数增加不一定会提高主频了。

B 选项正确。流水线的时钟周期必须覆盖最慢阶段的延迟（包括组合逻辑延迟和流水线寄存器延迟），否则会导致数据无法正确传递。因此，CPU 的最高频率由流水线中最关键路径（即延迟最大的阶段）决定。

C 选项错误。**流水线技术的核心优势在于提高吞吐率（单位时间内完成的指令数），而非减少单条指令的执行时间**。由于流水线寄存器延迟的存在，单条指令的总执行时间可能反而略有增加（如从单周期变为多周期）。虽然主频提高可能缩短时钟周期，但单条指令仍需经历多个阶段，其总执行时间并未减少。

D 选项错误。流水线技术的核心思想正是通过指令级并行（ILP）实现多条指令的不同阶段同时执行。例如，在五级流水线中，每个时钟周期可以处理五条不同指令的不同阶段，从而显著提升并行性。

57．C。【**解析**】指令相关性（如数据相关、控制相关）会导致流水线停顿（stall），降低效率。

当指令间完全无关时，流水线各阶段可连续执行，无停顿，此时流水线效率最高，C 正确；流水线技术通过将指令执行过程分解为多个阶段（如取指、译码、执行等），使不同指令的各阶段并行处理，从而提高整体指令吞吐量，而非单条指令的执行速度，A 错误；单周期处理器的 CPI 等于，稳定的流水线处理器的理想 CPI 为 1，但实际中因数据冒险、控制冒险等需要阻塞，导致 CPI 有时会大于 1，B 错误；流水线存在多种开销，如硬件开销，阻塞导致的时间开销等，D 错误。

58．A。【**解析**】加速比 =未使用流水线所用时钟周期数 / 使用流水线所用时钟周期数，未使用流水线共使用 2000x8=16000 个时钟周期，使用流水线(2000+5-1)x2=4008 个时钟周期，加速比为 16000 / 4008 ≈ 4.0。

59．D。【**解析**】指令流水线的时钟周期取决于各阶段中执行时间最长的阶段，以确保所有阶段都能在一个时钟周期内完成，本题中访存（MEM）阶段执行时间最长为 5ns，所以周期至少为 5ns，D 正确。

60．C。【**解析**】每两个相邻流水段之间的流水段寄存器用以记录所有在后面阶段要用到的各种信息，故 A 正确；指令译码得到的控制信号也通过流水段寄存器传送到后面各个流水段中，B 正确；因为每个流水段间传递的信息不一样，所以各流水段寄存器的长度也不一样，C 错误；流水段寄存器对用户透明，D 正确。

61．B。【**解析**】流水段寄存器位于相邻阶段之间（如 IF/ID、ID/EX 寄存器），用于存储前一阶段的输出数据，作为下一阶段的输入。主要作用是在流水线各阶段之间缓冲并传递数据，确保指令执行的连续性和并行性，B 正确。

62．A。【**解析**】结构冒险是指多条指令在同一时钟周期内争用同一硬件资源（如存储器、寄存器等）而产生的冲突，A 正确；指令执行顺序错误可能是控制冒险导致的，B 错误；数据相关导致的冲突是数据冒险，C 错误；控制转移指令引起的冲突是控制冒险，D 错误。

63．B。【**解析**】数据旁路（转发）技术通过直接传递中间结果，使后指令无需等待前指令完成写回阶段。相邻两条 ALU 运算指令之间的数据冒险可以通过将前指令的 EX 阶段得到的结果直接传给后指令的 ALU 输入端以解决数据相关，无需阻塞，B 正确；结构冒险由硬件资源争用引发（如总线冲突），需通过硬件设计（如多端口寄存器堆）解决，控制冒险由分支指令导致，需通过分支预测、延

迟槽等技术处理，A、C 错误；load-use 冒险，需要使用转发+硬件阻塞技术才能够解决，D 错误。

64．D。【解析】ADD R1，R2，R3：将寄存器 R2 与 R3 的值相加，结果存入 R1。SUB R4，R5，R6：将寄存器 R5 的值减去 R6 的值，结果存入 R4。两条指令无寄存器共享，无数据依赖，无需数据转发，A 错误；

ADD R1，R2，R3：R1 = R2 + R3（写 R1）。SUB R4，R1，R6：R4 = R1 - R6（读 R1）。显然存在 RAW 冒险，但可通过数据转发将 ADD 在 EX/MEM 段的运算结果直接传递给 SUB 的 EX 阶段，流水线不会停顿，B 错误；

LW R1，0（R2）：从内存地址（R2+0）读取数据，存入 R1（加载操作）。ADD R3，R4，R5：R3 = R4 + R5，与 R1 无关。两条指令无寄存器共享，无数据依赖，无需数据转发，C 错误。

LW R1，0（R2）：R1 = 内存 [R2]（写 R1，加载结果在 MEM 阶段可用）。ADD R3，R1，R4：R3 = R1 + R4（读 R1，需在 EX 阶段使用 R1 的值）。属于典型的 load-use 类型数据冒险，即使使用转发技术仍需硬件阻塞，D 正确。

65．A。【解析】一位动态预测可看作完全根据上一次转移情况进行预测，上一次转移则变为 1，上一次不转移则变为 0。注意初始状态应算第一次预测，故第一次预测结果是 0，实际不转移，第二次预测结果仍是 0，实际发生转移，故第三次预测结果是 1，即转移，A 正确。

66．D。【解析】

第一次：实际不转移，预测：转移（错误），10 → 00，错误次数：1。

第二次：实际转移，预测：不转移（错误），00 → 01，错误次数：2。

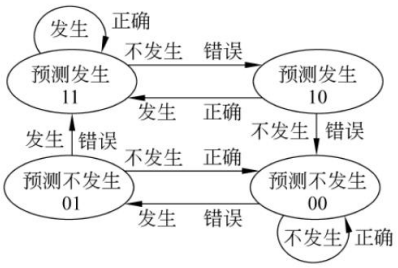
第三次：实际转移，预测：转移（错误），01 → 11（强转移），错误次数：3。

第四次：实际不转移，预测：转移（错误），11 → 10，错误次数：4。

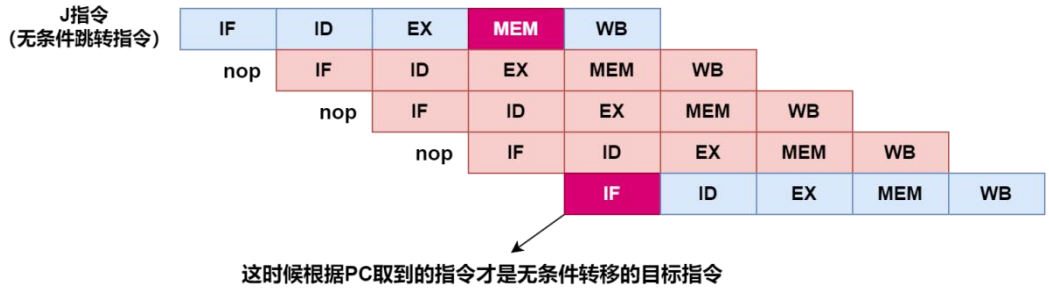
第五次：实际转移，预测：转移（正确），10 → 11，错误次数：4。

第六次：实际不转移，预测：转移（错误），11 → 10，错误次数：5，D 正确。

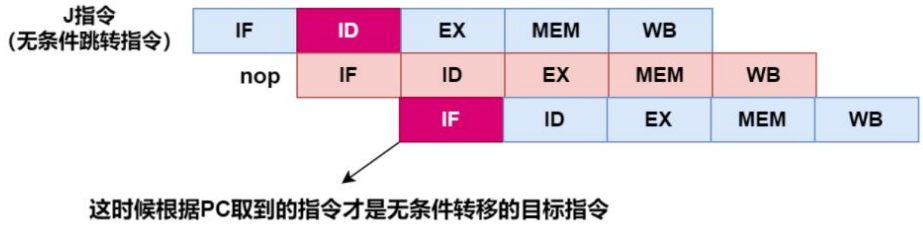
两位预测位的状态转移图



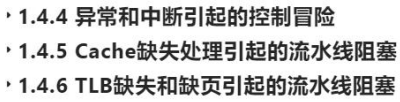
67．A。【解析】对于 A 选项，如图所示，C=3；



对于 B 选项，如图所示：



C 选项，见笔记该部分



D 选项，与分支冒险一样，**当某条指令执行过程中发现异常或中断时，可能它后面的多条指令已经被取到流水线中正在执行**。例如，ALU 运算类指令发现“溢出”时，已经到 Exec 阶段的结束了，此时，它后面已有两条指令进入了流水线。

68．B。【解析】对于理想的 RISC 处理器 5 段流水线（IF→ID→EX→MEM→WB），每个流水段占一个时钟周期，执行了四条指令，所以要求总时钟周期。

指令\时钟数	1	2	3	4	5	6	7	8	9
lw	IF	ID	EX	MEM	WB				
add		IF	ID	stall	EX	MEM	WB		
sub			IF	stall	ID	EX	MEM	WB	
sw				stall	IF	ID	EX	MEM	WB

如图所示，只有 lw 指令和 add 指令出现 load-use 冒险，无法只通过转发技术解决，需阻塞一个时钟周期，共 9 个时钟周期，CPI = $\frac{9}{4} = 2.25$，B 正确。

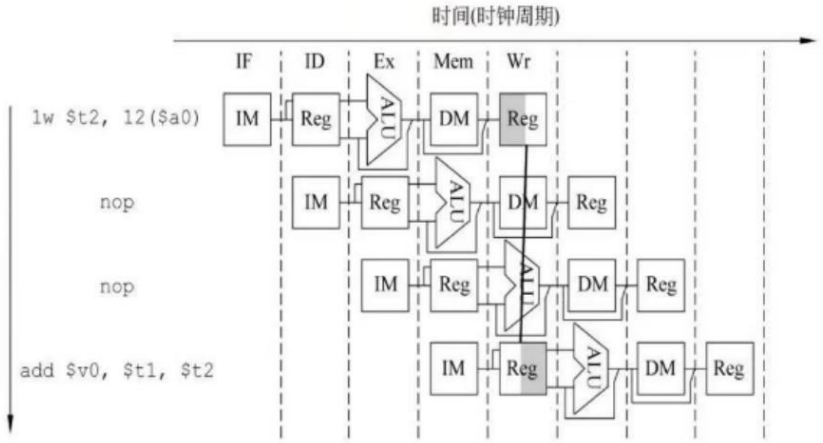
69．C。【解析】流水线指令执行情况如下图所示。

	1	2	3	4	5	6	7	8	9	10	11
add	IF	ID	EX	MEM	WB						
lw		IF			ID	EX	MEM	WB			
add					IF			ID	EX	MEM	WB

共 11 个时钟周期，C 正确。

70．D。【解析】此题是关于 load-use 数据冒险的处理方式。若不采用数据旁路（转发）技术，也不采用在时钟的前半周期写寄存器堆，后半周期读寄存器堆的方式，则需要插入 3 个 nop，把后一条指令的 ID 段阻塞在前一条指令的 WB 段之后。采用寄存器前半周期写后半周期读，可以节省一个时钟，

如下图所示。采用转发技术，则只需要插入一个 nop 即可，将 ID 隔到前一条 load 指令的 MEM 后，转发硬件路线由 mem 送到 IF/ID 流水线寄存器组。



流水线冒险简单总结如下：

①正常数据冒险，需插入 3 个 nop，如果采用 wb 前半段写后半段读只需 2 个。

②load-use 数据冒险，使用转发技术，相邻指令需插入一个 nop，将 EX 隔到前一条 load 指令的 MEM 后，转发硬件路线由 mem 送到 ALU 的输入端；不使用转发技术，需要插入 3 个 nop，同样 wb 分前后两段读写则只需 2 个 nop。也可以采用编译器将 load 指令调整到合适的位置解除冒险。

③分支控制冒险，如用插入技术，插入 3 个 nop，把后一条 IF 隔到前一条 MEM 后，因为转移指令（无论条件还是非条件）都是在 MEM 修改 pc。

④最后一个小坑是类似循环指令最后转移指令前若有写数据，则要注意循环重新执行的前几条指令和尾部的指令也可能出现数据冒险。

71．D。**【解析】**通常，条件转移指令和无条件转移指令都可能导致控制冒险，故本题选 D。

72．D。**【解析】**条件跳转指令若判断标志位满足条件则会发生跳转，导致控制冒险；函数调用本质上也是通过跳转指令实现的，可能会导致控制冒险；DMA（直接内存访问）完成时通常会触发中断，中断会打断当前程序执行，CPU 需暂停流水线，保存现场，并跳转到中断处理程序，这相当于一次隐含的跳转，导致指令流改变，同样属于控制冒险，故 D 正确。

73．C。**【解析】**第一条指令写 R1，紧随的第二条指令 R1 作为其输入，此时发生写后读数据冒险，C 正确。

74．B。**【解析】**一个部件在不同流水使用，会发生资源冲突，B 错误。

75．D。**【解析】**将指令 Cache 和数据 Cache 分离的主要目的是避免资源相关冲突。在流水线工作中，当同时需要取指令和取数据时，如果不分开，就会造成结构冒险冲突，从而导致流水线断流，将指令 Cache 和数据 Cache 分开后，就能同时满足取指令和取数据的要求，从而提高流水线运行效率，计算机通过不同的时间段来区分指令和数据，在取指阶段取出的是指令，在执行阶段取出的是数据，D 正确。

76．B。**【解析】**分支预测器是现代指令流水线式 CPU 中非常关键的技术，它在分支指令执行结束前猜测哪路分支将会被执行，以此来提高处理器指令流水线的效率，是为了解决控制冒险，A 错

误；

增加功能部件可以减少资源冲突，消除结构冒险，B 正确；

分支延迟槽（Branch delayslot），简单地说就是位于分支指令后面的一条指令，不管分支发生与否其总是被执行，而且位于分支延迟槽中的指令先于分支指令提交，是为了解决控制相关，C 错误；

转发技术可以消除或减少数据冒险，D 错误。

77．B。【解析】超标量流水线通过在一个时钟周期内同时发射和执行多条指令来提高性能，B 正确；普通流水线每个时钟周期可能只执行一条指令，A 错误；超标量流水线中指令执行顺序可能会动态调整，C 错误；指令译码阶段是指令执行的必要阶段，超标量流水线也需要，D 错误。

78．C。【解析】首先对原始指令序列分析:I1 是 lw 指令，目标寄存器是 R1;I2 是 add 指令，源操作数包含 R1。在 5 段流水线中，lw 指令的数据在 MEM 段结束时才能从数据 Cache 中读出；而 add 指令需要在 EX 段开始时使用数据。时间上：I2 进入 EX 段时，I1 刚进入 MEM 段，数据尚未读出。即使有转发路径，数据也不存在，无法转发；因此 I1 和 I2 之间发生了 Load-use 冒险。原始执行代价：顺序为 I1->stall -> I2-> I3 -> I4，必须插入 1 个气泡。

优化思路：要消除这个 Load-use 气泡，我们需要在 I1 和 I2 之间插入一条与它们都不相关的指令，利用这条指令来“填充”那个原本需要阻塞的时间片。我们需要检查每种重排顺序是否满足两个条件:

①逻辑正确性:不能破坏原有的数据依赖关系(例如写后读、写后写)。②冒险消除:lw R1 和使用 R1 的指令之间至少隔开一条指令。

A 选项(11,12,13,14):将 I4 提到 I2 之前， I4(or R8,R1,R9)也使用了 R1，需 1 个 stall。排除。

B 选项(11,14,13,12):同上。

C 选项(I1,I3,I2,I4):将 I3 移到 I1 和 I2 之间，I3 (sub R5,R6,R7)使用 R6,R7，写入 R5。与 I1(R1,R2)和 I2(R3,R1,R4)没有任何寄存器冲突，完全独立。流水线时序:

I1：lw （在 MEM 段产出 R1）

I3：sub（无关指令，正常执行，占用了 1w 后的那个周期）

I3：add（需要 R1）

结果：当 I2 准备进入 EX 段时，I3 在 EX 段，I1 已经进入 WB 段(或者 MEM 段刚结束)。此时 R1 数据已经准备好，通过 MEM/WB 流水寄存器向 ALU 入口进行转发。Load-Use 阻塞被成功消除，正确。

D 选项，I3,I1,I2,I4:Load-use 冒险依然存在，产生 1 个阻塞。

79．C。【解析】这是超长指令字技术的基本概念，C 正确。

80．C。【解析】A 错误，A 选项描述的是静态多发射，指令打包的结果可看作将同时发射的多条指令合并到一个长指令中，即超长指令字技术。

B 错误，超标量处理器采用动态多发射技术，通过在指令执行过程中由处理器硬件动态完成流水线调度来完成指令打包和冒险处理。

C 正确，若前一条指令和后一条指令都需要对同一个寄存器进行写入，则称这两条指令是写后写相关的。这种相关性，在无序发射或乱序执行时可能会发生数据冒险，而在按序发射和完成时，两条指令的写寄存器时机和指令顺序一致，不会发生数据冒险。

D 错误，静态多发射处理器的冒险处理不包括结构冒险，因为数据通路中功能部件及其个数是确定的，所以同一时钟周期内发射的指令类型和指令个数是受限的。例如，如果 ALU 运算部件和存储访问部件独立设置，并只各提供一套，则只能同时发射一条 ALU 指令/分支指令和一条 Load/Store 指令。所以不会发生结构冒险。

81．B。【解析】向量处理器是一种 SIMD（单指令流多数据流）计算机，它通过一条指令同时对多个数据元素进行操作，提高数据并行处理能力，B 正确。

82．A。【解析】A 错误，单核 CPU 在支持超线程的情况下，同一时刻可以有多个线程在执行。超线程物理上只有一个核心，但是逻辑上有多个。比如一个线程在 I/O 或者访存的时候，可以执行另一个线程进行计算，两个线程不冲突，可进一步利用硬件资源。

B 正确，多核 CPU 如果不使用多线程，那么一定有多个核心处于闲置状态，造成资源浪费。

C 正确，SMT 技术充分利用了超标量处理器的多通道、动态指令调度和寄存器重命名的机制，可以在一个周期里面发射来自不同 Thread 的多个指令，可以看作细粒度多线程和超标量处理器的结合。

D 正确，超线程是指令级并行，粗/细粒度多线程仅支持线程级并行。

83．B。【解析】硬件多线程的基本思想是在一个处理器中同时维护多个线程的上下文，当一个线程因等待资源而暂停时，处理器可以切换到另一个线程继续执行，提高处理器利用率，B 正确；在一个处理器中同时运行多个进程是多任务操作系统的功能，通常依赖操作系统的时间分片，与硬件多线程的底层机制不同，A 错误；多个处理器同时运行一个线程是并行计算的一种方式，但不是硬件多线程的基本思想，C 错误；多个处理器同时运行多个线程是多处理器系统的常见工作方式，D 错误。

84．C。【解析】MISD 是指同时执行多条指令，处理同一个数据，实际中几乎不存在。

SISD 通常是单处理机，有时候也设计成标量流水线，提高速度。

SIMD 是一个指令流同时对多个数据进行处理，成为数据级并行技术，通常由一个指令控制部件和多个处理单元组成，每个单元有自己的地址寄存器，这样就可有不同的数据地址，指令逻辑相同但数据不同，这样可以并行处理相同指令，适合 for 循环代码，不适合 case/switch。

MIMD 分为多计算机系统和多处理器系统。

85．A。【解析】多核处理器是将多个处理核心集成在一个处理器芯片中，这些核心可以独立执行指令，提高并行处理能力，A 正确；多个处理器芯片组成的系统是多处理器系统，B 错误；一个处理器中包含多个寄存器或多个 ALU 是处理器内部的设计优化，不是多核处理器的定义，C、D 错误。

86．A。【解析】共享内存多处理器（SMP）中，多个处理器共享同一内存空间，所有处理器可以平等地访问内存中的任何位置，A 正确；每个处理器有独立内存空间的是分布式内存系统，B 错误；处理器之间通过消息传递进行通信的是分布式系统，C 错误；SMP 中处理器可以访问共享内存，不只是本地内存，D 错误。

第 6 章

总线和输入/输出系统

【答案速对】

DBBBB　　CADDC　　DBADB　　DDDCA

BABAB　　BCBDB　　BDBCA　　BBBAC

A

1．D。【解析】总线的基本功能是实现部件之间的数据传输、地址传输和控制信号传输，而数据存储是存储器的功能，D 错误。

2．B。【解析】CPU 内部线路采用总线连接方式时，总线上信号流动的原则为：每个时刻只有 1 个器件发出信息，每个时刻有 1 个或多个器件接收信息，B 正确。

3．B。【解析】在总线中，数据总线是双向传输的，数据信息可由 CPU 传送至内存或外设，也可由内存、外设传送至 CPU，A 错误。

地址总线是单向传输的，地址信息只能由 CPU 发送至内存或者外设，B 正确。

控制信息可以双向传输，但是一根控制线通常只传一类控制信号，且方向固定，比如读写命令是CPU 传给内存或者外设（I/O 接口），外设（I/O 接口）传输握手信号和中断请求信号给 CPU，C 错误。

状态信息是单向传输的，通过由内存或外设发送至 CPU，D 错误。

4．B。【解析】总线带宽 = 数据传输量 / 总线周期 = （4B × 33MHz）/1 = 132MB/s，B 正确。

5．B。【解析】同步总线采用统一的时钟信号进行同步，而异步总线不使用统一时钟，采用握手信号进行通信，B 正确；数据传输速率、总线宽度与同步 / 异步无必然联系，A、C 错误；总线控制方式有集中式、分布式等，与同步 / 异步不是直接关联，D 错误。

6．C。【解析】由同步总线带宽计算公式：同步总线带宽=总线宽度×总线时钟频率×单时钟传输次数，原来的总线数据传输速率=4B×400MHz×1=1.6 GB/s。总线改进后的带宽=8B×400MHz×4=12.8GB/s，提高到原来的 8 倍，答案选 C。

7．A。【解析】I正确，并行总线可以同时传输多个数据，但数据之间必须同步，而且因为数据线多，使得相互间干扰大，因而，数据传输的速度受到很大影响。对于串行传输的总线，提高时钟速度比并行传输总线容易得多，且几乎没有线间串扰。因而串行传输总线的速度可以比并行传输总线快得多。

Ⅱ错误，总线工作频率，即单位时间内传输数据的次数，早期的总线通常一个时钟周期传送一次数

据，因此总线工作频率等于总线时钟频率。现在有些总线一个时钟周期可以传送 2 次或 4 次数据，因此，总线工作频率是总线时钟频率的 2 倍或 4 倍。

III正确，突发传送方式是一种连续的、成批数据传送方式。只需在传送开始时给出数据块的首地址，然后连续传送多个数据，后续数据的地址默认为前面数据地址加 1，实际上这些连续的数据一般在存储阵列的同一行上。

IV错误，总线事务分离会提高总线的利用率。总线事务分离指将一个事务过程分成两个子过程：

①主控设备 A 在获得总线使用权后，将请求的事务类型(总线命令)、地址等信息发送到总线上，从设备 B 记录下这些信息。主控设备发完这些信息后便立即释放总线，这样其他设备便可使用总线；

②从设备 B 收到主控设备 A 发来的信息后，就按照其请求的命令进行相应的操作，当准备好主控设备所需的数据后，从设备 B 便请求使用总线，一旦获得使用权，则从设备 B 就将主控设备 A 的编号及所需的数据等送到总线上，这样主控设备 A 便可接收数据。这种事务分离方式的优点是：通过在不传送数据期间（从设备准备数据）释放总线，使得其他申请者能使用总线，提高了总线利用率。但是这种方式的控制相当复杂，开销也大，并且事务响应时间也会变长。

> **具体例子**：CPU 通过 MAR 将地址给到主存后就放弃总线控制权。当主存准备好数据后，主存申请总线控制权将数据传输到 MDR。在准备数据的这段时间内总线可以被其他部件使用。若不采用事务分离策略，CPU 在给出地址后会一直占据着总线控制权，直到主存准备好数据，将数据送到 MDR 后才会释放总线。

V正确，总线传输周期：在总线上完成一次总线事务所用的时间。通常由若干个总线时钟周期组成，简称为总线周期。总线中数据线的条数被称为总线的宽度，即总线能同时传送的数据位数，它决定了每次能同时传输的数据信息的位数。通常把在总线上一对设备之间的一次信息交换过程称为一个总线事务。总线事务类型由其操作性质来定义，例如，存储器读事务是指将数据从主存取出到处理器，存储器写事务是指将数据写到主存。典型的总线事务类型有存储器读、存储器写、I/O 读、I/O 写、中断响应等。回到本题，由于寄存器位数为 32 位，而总线宽度仅为 16 位，所以需要 2 个总线传输周期才能将这 32 位数据写到主存。

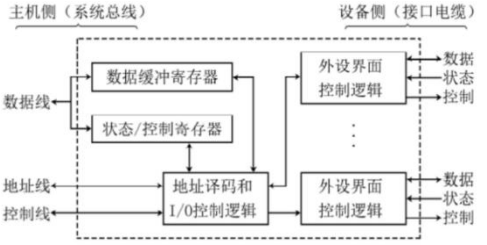
8．D。【解析】同步总线事务可以是突发传输方式，这种情况下，一个总线事务中可以传输多个连续的数据，每次传输的数据位数不超过总线宽度，D 错误。

9．D。【解析】I/O 接口中主机侧通过 I/O 总线与主机相连，设备侧通过通信总线（电缆）与外设相连，显然，I/O 总线中的数据线宽度和连接设备的电缆中的数据线宽度不一定相同，D 错误。

10．C。【解析】I/O 接口中的寄存器主要有数据缓冲寄存器、控制寄存器和状态寄存器，II、III、V 符合，故 C 正确。

11．D。【解析】I/O 接口的功能和基本结构如下图所示。打印控制接口是一种 I/O 接口，打印控制接口和打印部件之间即设备侧交换的信息不包含中断请求信号，因为中断请求信号是 I/O 接口的控制线发送给 CPU 的。打印机的控制过程通常是，CPU 先将需打印的字符编码送到打印控制接口中，打印控制接口再将字符编码转换为点阵信息，然后通过打印电缆传送到打印机，以控制打印针头在何处进行打印。

同时，打印控制接口需要将“初始化”“选通”“自动走纸”等打印控制信息通过电缆传送到打印机，并通过电缆把打印机的“联机”“忙”“缺纸”等状态信号取到打印控制接口以供 CPU 来读取。综上所述，本题选 D。



12．B。【解析】统一编址方式下，I/O 端口和内存单元统一编址，访问 I/O 端口使用与访问内存相同的访存指令，B 正确；独立编址方式下才使用专门的 I/O 指令，A 错误；控制指令和算术指令与访问 I/O 端口无关，C、D 错误。

13．A。【解析】统一编址方式下，可以直接用访存指令访问 I/O 设备，区分存储单元和 I/O 设备是依靠它们的地址码，I 正确。独立编址方式下，需要用专门的 I/O 指令来访问 I/O 设备，区分存储单元和 I/O 设备是依靠指令类型，II 正确。故 A 正确。

14．D。【解析】从 I/O 端口到 CPU 寄存器，是 CPU 把外存的相关信息（数据/状态寄存器的内容）读进来，用的是 I/O 指令的 IN 指令。WRITE 和 READ 指令是文件操作相关的指令。

注意：I/O 接口中控制线通常传送读/写命令，本质就是 IN/OUT 指令通过译码发出的读写控制信号，而不是指 read 和 write 指令。

如果是统一编址方式，MOV 指令即可实现数据传输。

15．B。【解析】CPU 需要不断查询外设状态，占用大量 CPU 时间，导致利用率低，B 正确。

16．D。【解析】程序直接控制 I/O 方式包含无条件传送和条件传送，无条件传送较为简单，无状态等交互信息，只需定时查询即可，适合于巡回检测采样系统或过程控制系统，以及非随机启动的字符型设备，①、④正确：条件传送方式下，接口中含有“就绪“完成”等状态，可继续分为独占查询和定时查询，②正确。③也同样正确。答案选 D。

17．D。【解析】对于磁盘一般是 DMA，A 错误；B 应该是独占查询和定时查询，B 错误；鼠标一般是中断 I/O 控制方式，C 错误。

18．D。【解析】当 CPU 查询到外设状态没有准备就绪时，只会继续查询其状态，并不需要重启外设，A 错误。程序查询方式中，数据传送过程也需要 CPU 的参与，B 错误。当数据没有传送完时，会继续查询外设状态，进行下一轮数据传送，此处也不需要重启外设，C 错误。每完成一次数据传送后，会将主存地址加 1，计数值减 1，D 正确。

19．C。【解析】中断隐指令完成的操作包括关中断、保存断点（程序计数器 PC 的值）、引出中断服务程序，而保存现场（如寄存器的值）是在中断服务程序中完成的，C 错误，A、B、D 正确。

20．A。【解析】中断方式下，外设就绪时会向 CPU 发出中断请求，CPU 响应中断并执行中断服务程序，A 正确。

21．B。【解析】由于处理器的主频为 100MHz，则 CPU 时钟周期=1/（100MHz）=10ns。每次传送 4 字节的数据时，CPU 所花的时间为：100x10ns=1000ns。硬盘传输 4 字节数据所用的总时间为：4B /（100KB/s）=40000ns。故 CPU 处理硬盘 I/O 的时间占整个 CPU 时间的百分比为 1000ns/40000ns=0.025=2.5%，B 正确。

22．A。【解析】中断隐指令指的是中断响应期间一系列硬件操作的总称，不是实际上的机器指令，没有具象化的操作码和地址码，它是一个虚构的概念。故 B，C，D 错误。

23．B。【解析】中断向量指的是中断处理程序的入口地址。中断类型号是标识中断类型的编号，A 错误；

中断向量地址指的是保存中断向量的内存地址，在中断响应阶段由中断向量地址形成部件生成，该部件是硬件，B 正确。

中断向量表记录了中断号和中断向量的对应关系，CPU 根据某个中断的中断向量地址在中断向量表中找到对应的中断向量，C 错误。

中断优先级分为中断响应优先级和中断处理优先级，中断响应优先级由硬件产生且不可更改，中断处理优先级由排队器、中断屏蔽字等硬件决定。中断向量表记录了中断编号和中断向量的对应关系，和各中断的优先级没有关系，D 错误。

24．A。【解析】关中断开中断都是通过控制中断允许触发器来实现的，A 正确。

25．B。【解析】通常中断优先级可以分为中断响应优先级和中断处理优先级。中断响应优先级指计算机系统根据中断源的重要性和紧迫程度，以硬件方式将中断源的优先级分若干个级别。中断处理优先级指中断在计算机中实际被处理的顺序，A 正确。

中断响应过程由硬件完成，其间完成断点和 psw 的保存。中断处理过程由中断服务程序完成，期间恢复断点和中断屏蔽字。保存和恢复中断屏蔽字由中断服务程序完成，B 错误。

某个中断处理程序运行时，根据该中断对应的中断屏蔽字，CPU 可以使特定的中断请求暂时无法进入排队器。待到该中断处理程序结束运行时，CPU 将暂时无法进入排队器的中断请求加入排队器，在排队器中根据中断屏蔽字确定下一个需要处理的中断，运行该中断的中断处理程序，重复上述过程，C 正确。

中断屏蔽字可以改变中断的处理优先级，需要将中断处理优先级更改为 I2>I1>I3>I4。正序中断屏蔽字表示顺序为 I1～I4，对于 I3 来说，有 I2、I1 两个中断的优先级比它更高，因此 I3 中断屏蔽字的第一位和第二位应该设置 0，表示不屏蔽。自己的优先级不会比自己的优先级更高，因此 I3 中断屏蔽字的第三位应该设置为 1。I4 的优先级小于 I3，因此 I3 中断屏蔽字的第 4 位应该设置为 1，D 正确。

26．B。【解析】1 级中断屏蔽字为 11111；

2 级中断屏蔽字为 01111：

3 级中断屏蔽字为 00111；

4 级中断屏蔽字为 00010；

5 级中断屏蔽字为 00011。

实际中断处理优先级（从高到低）顺序应为 1-2-3-5-4，B 正确。

27．C。【解析】中断过程包括中断响应及中断处理过程，在中断响应时，中断隐指令中的关中断是由硬件实现的。在中断处理执行中断服务程序中的关中断是由关中断（CLI）特权指令软件实现的。所以本题选 C。

28．B。【解析】中断控制器负责接收外设设备控制器中的中断并进行排队和判优，所以和外设的设备控制器直接相连，当选中中断后，会把该中断的中断类型号放入 CPU 特定的寄存器中，A、C、D 正确，可编程的中断控制器属于 I/O 接口，B 错误。

29．D 【解析】A 错误，CPU 在一条指令执行完后再响应中断，如果没有中断要响应，那么该指令周期没有中断响应周期。且响应中断期间，执行的是中断隐指令，由硬件自动完成，并非执行原程序；

B 错误，有三种情况①多重中断下，若新的中断请求优先级低于当前中断，该中断请求会被屏蔽，不会立即响应。②在当前中断过程中，需要在指令执行结束检测，并不是立即响应。③如果前两条都满足，但是操作系统使用了关中断指令，也是无法进行中断响应的。

C 错误，①必须在一条指令执行结束后才会进行中断请求检测，并不是任意时刻立即响应中断请求②如 B 选项的反例，当前在中断过程中，若有低优先级的中断，会被当前中断屏蔽，不会立即响应，若这句话改成“系统处于开中断时，检测到中断请求就会立即响应”则是对的。

30．B。【解析】DMA 方式适用于高速外设（如磁盘）的数据传输，B 正确；DMA 方式中，初始化和结束处理仍需 CPU 参与，A 错误；DMA 控制器需要向 CPU 申请总线控制权，C 错误；DMA 方式比程序中断方式的 CPU 开销小，D 错误。

31．B 【解析】DMA 方式的数据传送过程不是由 CPU 执行程序完成的，而是在 DMA 控制器本身发出的控制信号控制下完成的。

32．D。【解析】程序中断方式通过中断机制让 CPU 在等待外设时可以执行其他任务，适用于中低速外设的数据传输，D 正确；程序查询方式 CPU 利用率最低，A 错误；DMA 方式数据传输速率最快，B 错误；程序查询方式硬件结构最简单，C 错误。

33．B。【解析】DMA 中断由 CPU 执行，用于 DMA 的结束处理，DMA 方式的数据传输由 DMA 控制器硬件实现。

34．C。【解析】周期挪用 DMA 并非独占总线，而是与 CPU 交替使用总线。每次 DMA 仅挪用一个总线周期传输单字数据，传输间隙 CPU 仍可正常访问主存。只有在 “停止 CPU”（独占总线）方式下，CPU 才会在整个数据块传输期间完全无法访问主存，C 错误；周期挪用 DMA 以单字（或字节）为单位传输数据，A 正确；为确保 DMA 传输及时响应，DMA 控制器的总线请求优先级通常高于 CPU，B 正确；无论采用何种 DMA 方式，当整个数据块传输完成后，DMA 控制器均会向 CPU 发送中断请求，通知传输结束，D 正确。

35．A。【解析】I 错误，通常所说的 DMA 方式下数据在主存和外设之间直接进行传送，其含义并不是说在主存和外设之间建立一条物理上的直接通路，而是在主存和外设之间通过外设接口、系统总线以及总线桥接部件等连接，建立起一个信息可以互相通达的通路。“直接通路”是逻辑上的含义，物理上磁盘和主存不是直接相连的。

II 正确，键盘和鼠标的数据输入由于其数据量小、实时性要求高、使用中断驱动更为合适

III 错误，在进行数据传送之前，CPU 先执行一段初始化程序(属于设备驱动程序，我们知道设备驱动程序中包含了许多 I/O 指令，通过执行 I/O 指令，CPU 可以访问设备控制器中的 I/O 端口，从而控制

外设的 I/O 操作)，完成对 DMA 控制器中各参数寄存器的初始值的设定，因此是由软件来进行处理的

IV 错误，前半句正确，后半句错误，当缺页时，DMA 控制器需要负责把缺的页面调入主存相应位置，因此需要在预处理的初始化中将交换数据的主存起始位置（即该页面的页基址）写入 DMA 接口中的主存地址寄存器。

V 错误，如果还要继续传送，则又要初始化，因为需要交换数据的主存起始地址和设备地址（如磁盘物理地址）都可能会发生改变。

VI 正确，周期挪用需要 CPU 访存结束再挪用一个主存存取周期给 DMA 控制器。

36．B。【解析】每秒传输块数 $=\frac{2.5MB/s}{625B}=4000$ 块

每秒 I/O 开销 $=4000\times400=1.6\times10^6$个周期

I/O 占比 $=1.6\times10^6/800\times10^6=0.20\%$，B 正确。（这题 CPI 是干扰信息）

37．B。【解析】DMA 控制器向 CPU 提出 DMA 请求，是希望 CPU 让出总线控制权，由 DMA 控制器控制总线完成数据传送，A 正确；为防止数据丢失，DMA 控制器的总线使用优先级会更高，B 错误；DMA 控制器也是 I/O 接口，I/O 指令是对 I/O 接口中的寄存器进行操作，C 正确；D 选项为周期挪用的做法，故正确。

38．B。【解析】中断方式中，由 CPU 运行中断处理程序完成数据传输。DMA 方式由 DMA 控制器直接控制外设与主存之间的数据传输，以硬件方式传输数据，A 正确。

DMA 方式的优先级高于中断方式。但需要根据 DMA 的传送方式来决定是不是立即中断当前程序，B 错误。

如周期挪用方式下，若此时中断处理程序在访存，则需要等待等待存取周期结束后再将总线占有权让给 DMA 控制。

对于外中断的请求响应发生在每个指令周期结束之后，对 DMA 请求的响应发生在每个总线事务结束之后，C 正确。

DMA 控制器是一个独立的硬件，有自己独立的寄存器控制数据传输，在传输数据时不需要 CPU 的参与，也不需要修改 CPU 的现场信息。中断方式需要 CPU 运行中断服务程序进行数据传输，因此需要保存 CPU 的现场信息，D 正确。

39．A。【解析】预处理阶段：确定数据传输方向（选项 II）、选择并启动设备（选项 IV）、初始化主存地址寄存器和字计数器。

传送过程：设备发送一个字至 DMA 的数据缓冲寄存器中，作为选通信号、设备向 DMA 控制器发送 DMA 请求（DREO）、DMA 接口向 CPU 申请总线控制权（HRO，选项 V）、CPU 发回总线应答信号（HLDA）、DMA 控制器发送主存地址到总线、DMA 控制器向设备发送 DMA 应答信号（DACK）、传送数据至内存，同时修改主存地址寄存器和字计数器的值（选项 I）、检查数据块是否传送结束。

后处理：校验已传输的数据是否正确（选项 III）、决定是否用 DMA 传输其他数据块。

故选项 A 正确。

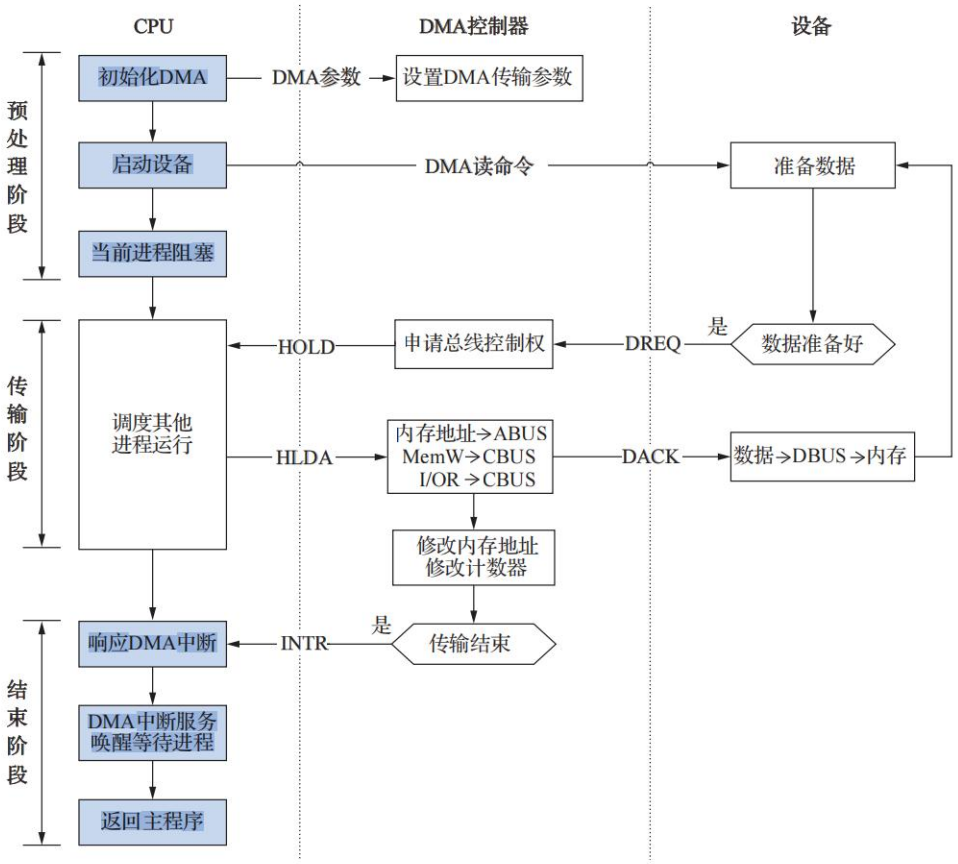


图 9.23　DMA 读操作详细流程（周期挪用）

40．C。【解析】停止 CPU 访存方式控制简单，因为需要等一组数据全部完成后再让出总线控制权，因此比较适合高速 I/O 设备，以尽可能减少 CPU 的等待时间，A 正确。

周期挪用方式中，I/O 设备每挪用一个主存周期都要申请和归还总线控制权。传输一个字的数据时，数据传输本身可能只需要 1 个主存周期，DMA 接口需要占用多个主存周期。因此适合 1/O 设备的读写周期大于存取周期的情况，B 正确。

DMA 与 CPU 交替访存适用于 CPU 的工作周期比主存存取周期长的情况。将一个 CPU 工作周期分为两个部分，分别用于 DMA 访存和 CPU 访存，C 错误。

周期挪用方法按字传输，在 CPU 访存时，如果 IO 设备有请求则让 CPU 继续访存，待 CPU 完成访存后再由 DMA 控制器访存。如果 CPU 和 DMA 控制器同时请求访存，则 CPU 暂停访存一到多个主存周期，由 DMA 控制器完成若干个字的数据传输。CPU 访存不会因此被延误较长时间，DMA 访存也可以尽可能及时地进行，D 正确。

41．A。【解析】A 错误，一般是先初始化 DMA，然后启动 DMA，再阻塞进程进行调度，而不是先阻塞再发出 DMA 请求。

B 正确，若没有数据要传输，DMA 中断服务程序会唤醒用户进程到就绪队列，然后返回主程序。

```
copy_string_to_kernel(strbuf, kernelbuf, n);    // 将字符串复制到内核缓冲区
initialize_DMA( );                              // 初始化DMA控制器（准备传送参数）
*DMA_control_port=START;                        // 发送“启动DMA传送”命令
scheduler( );                                   // 阻塞用户进程P，调度其他进程执行
```

(a) “字符打印”系统调用处理例程

```
acknowledge_interrupt();                        // 中断回答（清除中断请求）
unblock_user( );                                // 用户进程P解除阻塞，进入就绪队列
return_from_interrupt();                        // 中断返回
```

(b) “DMA结束”中断服务程序

C 正确，DMAC（DMA 控制器）通过地址连续+1 来实现批量连续的用户数据传输（即虚拟地址连续的数据），若使用物理地址作为首地址，那么后续在 DMA 数据传输过程中，实现的是物理地址的连续，而不是虚拟地址的连续，而虚拟地址连续的两个页面，在物理地址上不一定映射到连续的页框，所以 DMA 使用物理地址作为源地址或者目的地址，不能跨页传输，只能限制在一个物理页面内，否则数据会发生错误。

D 正确，在存储器层次中，CPU 访问主存，通过内存控制器，进行虚实地址转换，而 DMA 属于外部设备→存储器，不经过 MMU，所以使用虚拟地址作为源地址或者目的地址，需要 DMA 控制器自身完成虚拟地址的翻译。（参考袁书 8.6 节结尾）

第三部分

408 1000 题 os 解析册

第 1 章

计算机系统概述

【速对答案】

DBCAB　CADAB　ABBBA　BBABC

CCCAC　ACBBD　ABCAC　CDBCC

ABCCA　DCABD　DA

1．D。【解析】A 正确，并发是指两个或多个事件在同一时间间隔内发生。在多道程序环境下，在内存中同时装有若干道程序，以便当运行某道程序时，利用其因 I/O 操作而暂停执行时的 CPU 空档时间，再调度另道程序运行，从而实现多道程序交替运行，使 CPU 保持忙碌状态。

B 正确，并行性是指系统具有同时进行运算或操作的特性，在同一时刻能完成两种或两种以上的工作。

C 正确，一个能够并行执行多个任务的系统（即有多处理单元）自然也能够通过在这些处理单元上交替执行更多的任务来实现并发。并行是并发的一种特殊形式，或者说，并行系统可以更容易地实现并发。

D 错误，单处理器系统通过多道程序技术和快速的任务切换可以实现并发，使得多个任务在一段时间内交替使用 CPU。但是，由于在任何一个时刻只有一个任务真正在 CPU 上执行，因此无法实现真正的并行（多个任务在同一时刻执行）。

2．B。【解析】操作系统管理计算机裸机和计算机软、硬件资源，并向用户和应用程序提供接口。操作系统并不提供虚拟机管理程序，(第二类)虚拟机管理程序是一种应用程序。故选 B。

3．C。【解析】批处理系统中，用户一旦把作业提交给系统，那么直至作业完成，用户都不能与自己的作业进行交互，为此才诞生了分时系统，故I正确。

单道批处理系统中，为了实现对作业的连续处理，需要先把一批作业以脱机 IO 方式输入到磁带上，并在系统中配上监督程序。在它的控制下，这批作业能一个接一个地被连续处理。故II错误。

多道批处理系统中，用户所提交的作业都先存放在外存上并排成一个队列，作业调度程序按一定的算法从后备队列中选择若干作业调入内存，它们在管理程序的控制下相互穿插地运行，共享系统中的各种硬软件资源。当某道程序因请求 I/O 操作而暂停运行时，CPU 便立即转去运行另一道程序，这是通过中断机制实现的，无需多 CPU 的支持，故III错误，IV正确。综上，选 C。

4．A。【解析】多道批处理系统通过组织作业(编码或数据)使 CPU 总有一个作业可执行，从而提高了 CPU 的利用率、系统吞吐量和 IO 设备利用率，但系统要付出额外的开销来组织作业和切换作业。单道批处理系统与之相比，CPU 利用率更低，系统开销更小，系统吞吐量更小，I/O 设备利用率更低，故选I、II。

注意：有同学可能会认为I与IV正好对应，CPU 利用率低，那 I/O 利用率就高，从而错选 B。单道批处理系统，CPU 和 I/O 同一时刻只能运行一个，而多道批处理系统可以通过中断技术实现 CPU 与 I/O 并行，两者的利用率均更高。

5．B。【解析】多道程序系统（多道批处理系统），为使系统中的多道程序间能协调地运行，系统必须解决下述一系列问题。

①争用处理机问题：系统既要能满足各道程序运行的需要，又要能提高处理机的利用率。

②内存分配与保护问题：系统应能为每道程序分配必要的内存空间，以使它们“各得其所”，且不会因某道程序出现异常情况而破坏其他程序。

③I/O 设备分配问题：系统应采取适当的策略来分配系统中的 IO 设备，以达到既能方便用户对设备的使用，又能提高设备利用率的目的。

④文件的组织与管理问题：系统应能有效地组织存放在系统中的大量程序和数据，以使它们既便于用户使用，又能保证数据的安全性。

⑤作业管理问题：系统中存在着各种作业(应用程序)，系统应能对系统中所有的作业进行合理的组织，以满足这些作业对应用户的不同要求。

⑥用户与系统的接口问题：为使用户能方便地使用 OS，OS 还应提供用户与 OS 之间的接口。

故 A，C，D 正确。早期的多道批处理操作系统会将所有进程的数据全部调入主存，再让多道程序并发执行，即使不支持虚拟存储管理，也能实现“多道程序并发”，B 错误。

6．C。【解析】I正确，当采用多道程序设计时，一个进程在进行 I/O 的时候，另一个进程可以同步使用 CPU，而单道程序设计下系统内存中同时仅存在一个进程，当进程占用 CPU 时其他进程无法使用 CPU，同样当进程在 I/O 时其它进程也无法使用 CPU，因此多道程序可以同时提高 CPU 和 I/O 设备的利用率。

Ⅱ错误，批处理程序分为单道批处理和多道批处理。多道批处理系统中用户所提交的作业都先存放在外存上并排成一个队列，作业调度程序按一定的算法从后备队列中选择若干作业调入内存，它们在管理程序的控制下相互穿插地运行。多道批处理不提供人机交互能力。

III正确，IV正确。静态重定位（地址转换在进程装入时一次完成，无需地址重定位寄存器）和动态重定位（内存中的程序可能移动，需要重定位寄存器的支持）均可以适用于多道程序系统。而这两种方式下都需要进行内存保护，以防止多个进程运行时访问的内存空间不会发生冲突。

综上所述，选 C。

7．A。【解析】I错误，批处理系统的最主要缺点是缺乏交互性。I的表述肯定是错的，多道批处理系统是可以并发执行多个程序的。这里的多道是指允许多个进程同时驻留在主存中，按照某种原则分派处理机，逐个执行这些程序。

这里其实还考查了并发性的概念：并行性是指两个或多个事件在同一时刻发生；而并发性是指两个或多个事件在同一时间间隔内发生。

II错误，多道程序设计是指把多个程序同时存放在内存中，使它们同时处于运行状态。但是，在单处理机环境中，同一时刻只能有一个进程在执行。

III正确，有了中断后才能实现进程间并发，进程间并发才有可能把多个进程装入到内存实现多道程序技术。

IV错误，程序道数如果过多的话，会导致每个程序分配到的内存不够，很多程序所需的程序和代码需要时从磁盘调入到内存，系统会频繁地处于 I/O 状态中，导致系统效率降低。

8．D。【解析】A. 允许用户修改系统时间：这仅改变系统时钟的设置，与进程调度和响应时间无关，不会改善响应时间。

B. 加大时间片：加大时间片会导致每个进程一次性占用 CPU 的时间变长。如果就绪队列中有多个进程，排在队尾的进程需要等待前面所有进程运行完各自的大时间片，这会恶化（延长）响应时间。如果时间片大到一定程度，分时系统甚至会退化为先来先服务（FCFS）系统。

C. 允许中断嵌套：中断嵌套允许高优先级中断在处理低优先级中断时被响应，这可以减少中断延迟，从而间接改善系统对 I/O 事件的响应。但中断嵌套主要影响中断处理效率，对进程级的响应时间改善不如调度策略直接。

D. 采用抢占式优先级调度：这种调度方式系统可以根据进程的类型赋予不同的优先级（例如，将与用户交互的前台进程设为高优先级，后台计算进程设为低优先级）。一旦高优先级的交互式进程就绪，它能够立即抢占当前正在运行的低优先级进程，从而使交互请求得到最及时的处理，显著改善了交互进程的响应时间。

所以正确答案为 D 选项。

9．A。**【解析】**I必须实现。分时操作系统的核心是让多个用户通过终端与计算机交互，每个用户感觉自己独占计算机。 为了实现这一点，系统采用时间片轮转的方式，让每个进程执行一小段时间后就切换到下一个进程。 这种时间片的控制和进程的切换依赖于时钟中断。当一个进程的时间片用完时，时钟硬件会产生中断，操作系统内核响应中断，剥夺当前进程的 CPU 使用权，并进行进程调度。

Ⅱ必须实现。分时系统允许多个用户（进程）并发执行并共享系统资源（如内存、I/O 设备、文件等）。为了防止并发访问共享资源时产生冲突和数据不一致，操作系统必须提供共享资源保护机制，如互斥锁、信号量等，确保资源的安全和有序访问。

III必须实现。分时系统的基础是多道程序设计。 多道程序设计允许内存中同时驻留多个用户作业（进程），这些进程可以并发执行，从而提高了 CPU 和系统资源的利用率。没有多道程序设计，就无法实现多个用户同时使用计算机并进行交互的场景。

IV非必须实现，但非常普遍。虚拟内存管理技术（如请求分页）允许程序使用的逻辑地址空间远大于实际物理内存，并通过按需调页和页面置换来管理内存。 虽然虚拟内存能极大地提高内存利用率和支持运行更大的程序或更多的并发进程，但它并非分时操作系统存在的绝对必要条件。早期的分时系统可能不具备完善的虚拟内存管理，而是依赖于如对换（swapping）等技术。现代分时操作系统几乎都实

现了虚拟内存管理，因为它能显著提升系统性能和用户体验。

10．B。【解析】响应时间是响应所有就绪进程的时间之和，即响应时间=就绪进程数 x 时间片。因此当响应时间最大为 2s 时，时间片最大为 2s/100=20ms，所以正确答案为 B 选项。

11．A。【解析】实时操作系统要保证实时性和可靠性，当更高优先级的事件到达时，必须优先执行该事件，因此必须允许中断嵌套和采用抢占式优先级调度。实时操作系统更注重实时性和可靠性，因此分布式操作系统和图形界面是不必要的。所以正确答案为 A 选项。

12．B。【解析】单道批处理系统中只有一个进程在内存中，每次只能运行一道程序，无多道程序技术。

13．B。【解析】A 选项错误。 缺页指的是程序访问的内存页不在物理内存中，处理器会触发缺页异常，导致 CPU 从用户态变为内核态 。但在处理中断或异常进入内核态的过程中，属于同一个进程的上下文切换，不涉及进程的切换 。

B 选项正确。 一个进程同时拥有用户栈和内核栈。进程在从用户态进入内核态时，进程的用户程序与内核程序均属于同一个进程，共用同一虚拟地址空间 ；并且发生特权级切换时，由硬件自动实现堆栈的切换。

C 选项错误。 在允许多重中断或涉及特权级切换的情况下（如缺页异常处理及返回），需要保存和恢复程序状态字寄存器（PSW）。只有普通的子程序调用或全程在用户态下的函数调用才不需要保存 PSW。

D 选项错误。 内核程序运行在内核态，不仅可以执行所有特权指令，也可以执行非特权指令。

tips：进程在处理中断或者异常从用户态进入内核态，或中断异常处理完从内核态返回到用户态的过程中，均不涉及进程的切换，
也就是说进程的用户程序与为其服务的内核程序均属于同一个进程，共用同一虚拟地址空间

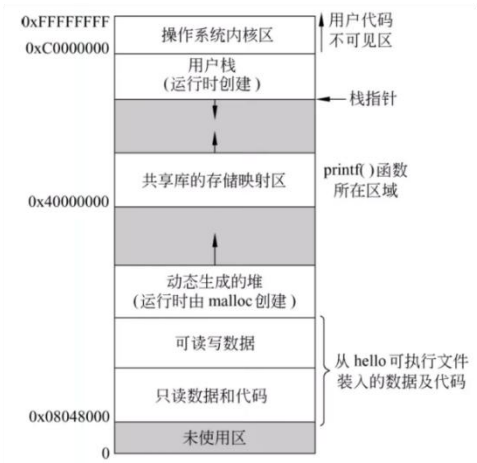
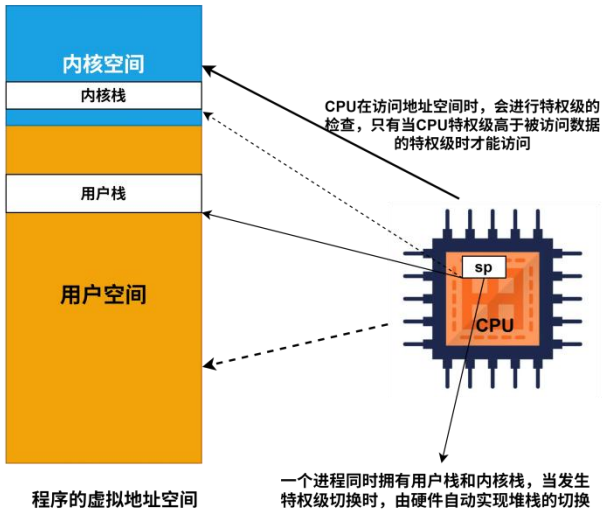


图 7.40　Linux 虚拟地址空间



14．B。【解析】I. 开中断和关中断既可以用硬件实现也可以软件实现，选项中的 CLI 关中断指令（以及 STI 开中断指令）都是软件实现，属于特权指令，必须在内核态执行。

Ⅱ.执行 I/O 操作的指令：I/O 指令通常是特权指令，只能在内核态执行。用户程序如果尝试在用户态执行 I/O 指令，会引发保护性异常，强制进入内核态处理，但指令本身不会在用户态成功执行其预定功能。

III.自陷指令（trap）是用户程序请求操作系统服务的正常方式。该指令本身是在用户态启动执行

的，其执行的直接结果就是使 CPU 切换到内核态，然后由操作系统内核响应该中断/陷入请求。

IV.断点设置指令：设置断点是调试程序时常用的一种手段。在某些体系结构中（如 x86），有专门的断点中断指令（如 INT 3），该指令可以在用户态执行，执行后会引发一个调试异常，使 CPU 陷入内核态，由调试器或操作系统内核接管处理。设置断点的目的是为了让程序在运行至断点位置时触发中断，而设置断点这一步骤是可以发生在用户态的。

综上所述，选 B。

15．A。【解析】I 正确，修改 PSW 直接影响 CPU 运行状态和中断屏蔽，属于特权指令。II 错误，读取时钟是非特权指令（用户程序可以读取时间），但设置时钟是特权指令。III 正确，修改页表基址寄存器会改变整个地址映射关系，属于特权指令。IV 错误，NOP 是空操作指令，不涉及任何系统资源控制，属于非特权指令。

16．B。【解析】I 正确，页面不在内存中触发缺页异常，CPU 切换到内核态进行页面调入。II 正确，用户进程访问内核空间地址会触发保护异常（越权访问），CPU 切换到内核态处理。III 正确，段不存在触发段错误异常，CPU 切换到内核态处理。IV 错误，页面在内存中且是合法的用户空间地址，这是正常的内存访问，MMU 直接完成地址转换，不会触发任何异常，CPU 保持在用户态。

17．B。【解析】① sin() 是普通的 C 语言标准数学库函数，它不需要请求操作系统的特权服务，全程在用户态下执行。最少：0 次　最多：0 次

② 定时器到时会产生时钟中断（外部中断）。外部中断一旦发生，硬件会自动打断当前程序的执行，强制 CPU 陷入内核态，交由操作系统的时钟中断处理程序来处理（例如进行时间片轮转调度）。这是不可避免的确定性事件。最少：1 次 最多：1 次

③ TLB（快表）缺失不等于缺页异常！

•最少情况（0 次）： 发生 TLB 缺失后，硬件（MMU）会自动去主存中查找慢表（页表）。如果对应的页表项是有效的（即该页面已经在物理内存中），硬件会自动将该页表项装入 TLB 并继续执行。整个过程由硬件自动完成，操作系统完全不介入，CPU 保持在用户态。

•最多情况（1 次）： 硬件查慢表时，发现对应的页表项无效（该页面被换出到了磁盘）。此时硬件才会真正触发缺页异常（内中断/故障），强制 CPU 切换到内核态，由操作系统的缺页处理程序介入调页。

④ 执行除法指令 DIV R0, R1 这是一条普通的算术运算指令。

•最少情况（0 次）： 寄存器 R1 的值不为 0。指令在 CPU 的 ALU 中正常运算结束，全程在用户态完成。

•最多情况（1 次）： 寄存器 R1 的值为 0。此时硬件会检测到运算错误，触发除零异常（内中断/中止），强制 CPU 切换到内核态，交由操作系统进行异常处理（通常是终止该进程并报错）。

最少切换次数 = 0（事件①）+ 1（事件②必然发生）+ 0（事件③页表命中）+ 0（事件④正常除法）= 1 次

最多切换次数 = 0（事件①）+ 1（事件②必然发生）+ 1（事件③触发缺页异常）+ 1（事件④触发除零异常）= 3 次

18．A。【解析】为防止 OS 和用户程序受到错误用户程序的影响，双重模式执行提供了特权指令(privileged instruction)和非特权指令(non-privileged instuction)。

Ⅰ.特权指令，是指在内核态下运行的指令，它对内存空间的访问范围基本不受限制。它不仅能访问用户空间，还能访问系统空间。故I错。

Ⅱ.非特权指令，是指在用户态下运行的指令。应用程序所使用的都是非特权指令，它只能完成一般性的操作和任务，不能对系统中的硬件和软件进行直接访问，对内存的访问范围也局限于用户空间。故II对。

Ⅲ.IV.CPU 在用户态时只能执行非特权指令，在内核态时可以执行特权指令和非特权指令。故III对，IV错。

综上所述，选 A。

19．B。【解析】访问虚拟内存是进程直接访问内存，由 MMU 完成虚拟地址向物理地址的转换，这个过程由硬件完成不需要操作系统参与，只有发生缺页时才会触发缺页异常，由操作系统来完成异常处理。A、C、D 均由用户主动发起的请求操作系统提供的服务，属于系统调用的范畴，因此选 B。

20．C。【解析】A 错误。 中断隐指令由硬件在中断响应阶段自动执行，其核心任务是关中断、保存断点并引出中断服务程序。这一步恰恰标志着 CPU 从用户态进入内核态。

B 错误。 保存被中断进程的现场信息（如通用寄存器）是中断服务程序（内核程序）在内核态下执行的步骤，此时 CPU 已经处于内核态。

C 正确。 当中断或异常处理完，或者系统调用完成时，内核程序会执行中断返回指令（如 x86 中的 IRET）。该指令会从内核栈中恢复被中断程序断点处的程序状态字寄存器（PSW）和指令指针寄存器（PC）。因为原 PSW 中标志位记录了原程序处于用户态，所以该指令执行完成时，CPU 就会恢复到用户态继续执行用户程序 。

D 错误。 阻塞进程、唤醒进程（即操作就绪/阻塞队列），属于系统控制数据结构的基本操作，完全是在内核态下由调度程序完成的，执行完毕后 CPU 仍在内核态。 （见笔记中进程切换部分，最好详细了解进程的切换全过程）

21．C。【解析】中断是操作系统响应硬件事件的核心机制。操作系统内核需要通过中断号快速定位到相应的中断服务例程的入口地址。因此，考虑到中断处理对响应速度的严格要求和通过中断号直接寻址的便利性，数组是存储中断向量表最适合的数据结构，选 C。

22．C。【解析】中断是指 CPU 对系统中或系统外发生的异步事件所做的一种响应。根据中断信号的来源，中断可以分为外部中断（或称硬件中断）和内部中断（或称异常、陷入）。

外部中断（Hardware Interrupts）：指来自 CPU 处理器以外的中断信号，通常由外部设备（如 I/O 设备、时钟）或外部事件引起。

I. 时间片结束：在分时操作系统中，时间片结束是由时钟部件（定时器）产生的时钟中断引起的。定时器是 CPU 外部的硬件，因此时钟中断属于外部中断。

II. 键盘输入：键盘是外部 I/O 设备，当用户按下键盘按键时，键盘控制器会向 CPU 发出中断信号，请求处理输入。这属于典型的外部中断。

IV I/O 操作结束：当外部设备（如磁盘、打印机等）完成 CPU 下达的 I/O 操作后，会向 CPU 发出中断信号，通知 CPU 操作已完成。这也是一种常见的外部中断。

内部中断（Exceptions/Traps）：指来自 CPU 处理器内部的中断信号，通常是由于 CPU 在执行指令过程中检测到某些错误或特定条件而引起的。

III.访存缺页：访存缺页是在执行一条指令的过程中，当该指令要访问的页面不在物理内存时，由 CPU 内部的内存管理单元（MMU）检测到并产生的一种异常。这属于内部中断。

综上所述，能够引起外部中断的是时间片结束、键盘输入和 I/O 操作结束。因此，选项I、II、IV正确，选 C。

23．C。【解析】中断是指 CPU 对系统中或系统外发生的异步事件所做的一种响应。根据中断信号的来源，中断可以分为外部中断（或称硬件中断）和内部中断（或称异常、陷入）。

I. 除数为 0：这是在执行除法指令时，CPU 检测到的算术运算错误。这种由指令执行错误引起的事件属于内部中断（异常）的一种。

II. 自陷指令（Trap Instruction）：自陷指令是程序主动请求操作系统服务（如系统调用）或有意引发某种调试事件的指令。它是在执行用户程序指令时，由该指令本身引发的，导致 CPU 从用户态切换到内核态，因此属于内部中断（陷入或软中断）。

III. 断电：断电是由外部因素（如电源故障）引起的，通常由计算机硬件（如电源监控电路）检测并向 CPU 发出中断信号。这属于外部中断，而非内部中断。

IV. 无效存储访问：当程序试图访问其地址空间之外的内存，或者以不允许的方式（如用户态程序试图访问内核空间、对只读页面进行写操作）访问内存时，由 CPU 的内存管理单元（MMU）检测到并产生的一种错误。这属于内部中断（异常）的一种，例如段错误或页错误。

综上所述，选项I、II、IV正确，选 C。

24．A。【解析】I、II：子程序调用只需保存程序断点，即该指令的下一条指令的地址；中断调用子程序不仅要保护断点(PC 的内容)，而且要保护程序状态字寄存器的内容 PSW，故I入选。

III、IV：通用寄存器属于中断处理程序所保存的“现场”，需要被中断处理所保存。对于子程序调用，通用寄存器的保存通常由软件（编译器生成的代码或程序员显式编写的代码）根据调用约定来完成，例如“调用者保存”或“被调用者保存”策略。硬件的 CALL 指令本身通常不负责保存所有通用寄存器。综上所述，选 A。

25．C。【解析】A 错误，中断向量表中存储的是中断处理程序的入口地址（而非完整代码）。B 错误，不同的异常类型有不同的处理程序入口，例如除零异常和缺页异常各有独立的处理程序。C 正确，中断向量表通常存放在内存的固定位置，CPU 硬件通过中断号直接计算偏移量来定位入口地址。D 错误，修改中断向量表是特权操作，用户程序无权直接修改。如果需要注册中断处理程序，必须通过系统调用请求内核代为操作。

26．A。【解析】I 正确，时钟中断由外部硬件定时器产生，属于异步外中断，与 CPU 正在执行什么指令无关。II 正确，时间片轮转（RR）、多级反馈队列等算法，必须依靠时钟中断来周期性递减时间片，没有时钟中断，进程一旦占有 CPU 就会一直运行到主动阻塞或结束。III 错误，时钟中断处理程

序只是检查时间片是否耗尽。如果时间片未耗尽，或者就绪队列为空，或者当前进程是唯一的运行进程，则不会发生上下文切换。IV 错误：时钟中断频率过高，会导致 CPU 频繁陷入中断处理程序（上下文切换开销极大），降低系统有效吞吐量。V 错误，只要 CPU 处于开中断状态（中断允许标志 IF=1），无论是在用户态还是内核态（执行系统调用时），一旦当前指令执行完毕，CPU 就可以立即响应时钟中断。内核态并不免疫外中断！只有当 OS 内核为了保护临界区而显式关中断时，才会推迟响应（除了极少数修改内核数据结构的“关中断临界区”外，内核态在绝大部分时间内都是“开中断”的）。

27．C。【**解析**】A 项错误，中断通常在指令执行结束后被检测和响应，而异常是在指令执行期间被检测到的 。B 项正好说反了，中断是由 CPU 外部的硬件事件（如 I/O 设备）引起的，而异常是由 CPU 内部的事件（如除零、缺页）引起的 。C 项正确，这是两者最本质的区别。异常（如除零、非法指令、缺页）的发生与当前正在执行的那条指令直接相关，是这条指令执行结果的体现；而中断（如时钟中断、I/O 完成）是异步发生的，与当前执行的任何指令都没有直接关系 。D 项错误，缺页属于异常（内中断），而 I/O 完成属于中断（外中断）。

28．B。【解析】本题对多个关于系统调用的细微知识点进行辨析。I 错误，操作系统提供给用户的接口主要有两类：命令接口（如 shell 命令行、图形用户界面）和程序接口（即系统调用）。系统调用只是程序接口的一部分，并非唯一的接口 。II 错误，系统调用属于程序接口（API），而联机命令接口是命令接口的一种，二者分类不同。尽管命令接口的底层实现确实依赖于系统调用，但从接口类型划分上不能混为一谈 。III 正确，当系统调用从用户态陷入内核态后，内核代码需要有自己的执行栈来保存上下文、处理参数和执行逻辑，这个栈就是内核栈。IV 正确，系统调用处理完毕后，需要通过特殊的返回指令（如 x86 的 iret 或 sysret）来恢复用户程序的现场并切换回用户态，这是态切换的逆过程 。因此，正确的叙述有 III 和 IV，共 2 个。

29．B。【解析】B 错误，不同的操作系统通常有其自己设计的系统调用接口，包括系统调用的名称、编号、参数以及具体功能实现。虽然有一些标准（如 POSIX）试图为类 UNIX 系统提供一定程度的接口统一性，但并不能说所有主流操作系统都采用完全相同的系统调用接口。

A 正确，用户程序在 CPU 用户态下执行，通过调用系统调用，将 CPU 控制权交给操作系统内核。

C 正确，系统调用服务例程是操作系统内核的一部分，因此它在内核态执行。在内核态下，操作系统拥有最高权限，可以执行所有特权指令（如 I/O 操作、修改内存管理寄存器等），并访问系统中的所有核心资源和数据结构，以完成用户程序的请求。

D 正确，这是系统调用的一个主要目的。通过将敏感操作（如设备访问、内存管理、文件系统操作）封装在内核态的系统调用中，操作系统可以对用户程序的请求进行检查和控制，确保这些操作不会危害系统的稳定性和安全性。

30．D。【解析】A 正确：当执行陷入指令时，硬件（中断隐指令）会自动将当前的断点（下一条指令的地址）和程序状态字（PSW）压入内核栈中，以确保安全。 B 正确：CPU 模式的切换（用户态→内核态）是高特权级操作，必须由硬件在响应内中断时自动修改 CPU 的运行模式寄存器完成。 C 正确：硬件将控制权转交给内核的统一入口（如 system_call）后，操作系统软件会读取 EAX 中的系统

调用号，去查系统调用跳转表，然后跳转到具体的服务例程。 D 错误：在服务程序执行完毕准备返回前，由操作系统软件而不是硬件执行逆操作来恢复之前保存的通用寄存器现场，最后执行中断返回指令（如 iret）回到用户态。

31．A。【解析】当陷入指令执行引发内中断时：I、III 正确（硬件完成）： 硬件的中断隐指令会自动将 CPU 状态改为内核态，并自动将断点（PC）和程序状态字（PSW）压入内核栈保存。

II 错误（软件完成）： 通用寄存器（如 EAX, EBX 等）的保存是由操作系统的陷入处理程序（软件代码） 通过执行压栈指令来完成的，以防后续的内核代码覆盖用户态的寄存器现场。

IV 错误（软件完成）： 硬件只负责跳转到系统调用的统一总入口（如 system_call）。根据系统调用号查表并跳转到具体的服务例程，是由操作系统内核代码（软件） 完成的。

综上，由硬件自动完成的仅有 I 和 III，故选 A。

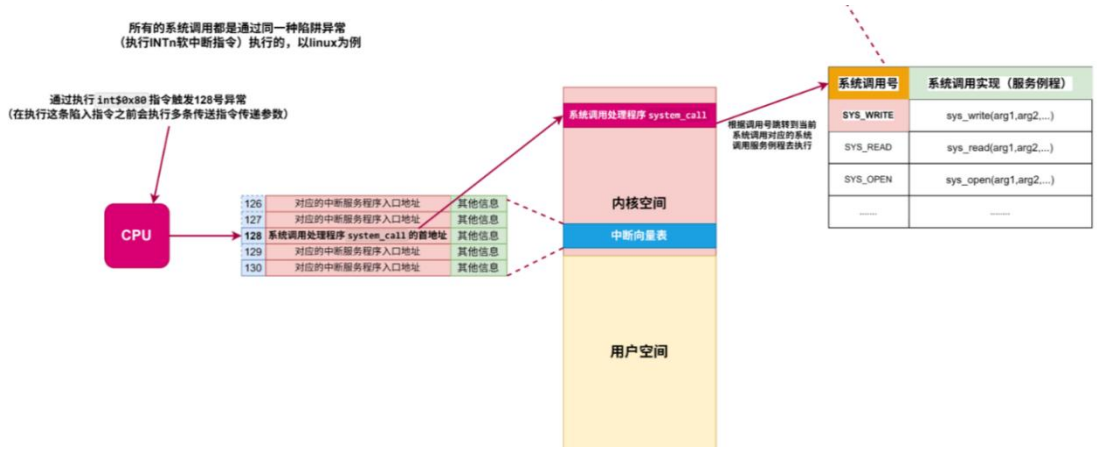
32．B。【解析】先看 B，错误。本题考察对进程切换时机和状态的精确理解。A 正确，时钟中断不一定发生在用户态，也可以是内核态下发生，比如中断嵌套。B 错误，这是本题的陷阱所在。进程切换完成后，新调度的进程不一定在用户态开始执行。如果这个新进程之前是在内核态中被中断的（例如，它正在执行一个长耗时的系统调用等待外部设备 I/O 完成），那么当它被换下并重新调度回来时，它将在内核态继续执行其被中断的系统调用代码，直到该系统调用完成才会返回到用户态。C 正确，进程切换的本质就是保存当前进程的 CPU 上下文（包括寄存器的值）到其 PCB 中，然后从新进程的 PCB 中恢复其上下文 。D 正确，时钟中断处理程序和进程调度程序都是操作系统内核的一部分，它们的执行当然在内核态。

33．C。【解析】陷入指令（如 int $0x80 或 syscall）本质上是在用户态执行的一条特殊指令，执行后会立即引发一个内中断（软中断/访管中断），硬件响应此中断从而将 CPU 切换到核心态。

A 项错误。 陷入指令必须在用户态执行，它是用户程序主动请求内核服务的唯一“合法入口”。如果它是特权指令，用户程序就无权执行，系统调用也就无从发起了。

B 项错误。 系统调用是“陷阱（Trap）”，属于主动请求，服务完成后应该继续执行用户程序的下一条指令，因此保存的断点是下一条指令的地址。只有像“缺页异常”这种故障（Fault），保存的才是当前指令地址（以便页面调入后重新执行该指令）。

D 项错误。 操作系统通常提供统一的陷入指令作为系统调用的总入口。区分不同系统调用靠的是存放在寄存器中的系统调用号，而不是靠不同的陷入指令。



34．A。【解析】系统调用需要在传递参数后执行陷入指令进入内核态，所以传递系统调用参数发生在用户态，其它选项都是具体的系统调用处理过程，所以发生在内核态。

35．C。【解析】A：修改界限地址寄存器为特权指令，需要在内核态下执行；

B：设置定时器初值属于时钟管理的内容，需在内核态下运行；

C：访管指令是用户态到内核态的入口，可以在用户态下运行；关闭

D：中断允许位属于中断机制，它只能运行在内核态下。故答案为选项 C。

36．C。【解析】A 正确：封装函数有一个统一结构：先用若干条传送指令（mov）把参数和系统调用号送入指定的通用寄存器，紧跟上一条陷阱指令（如 int $0x80）陷入内核。

B 正确：系统调用为了提高效率，通过寄存器传递参数（如 EAX 放调用号，EBX、ECX 等放参数），这与普通过程调用（CALL 指令）默认用栈传参的机制不同。

C 错误（考点核心）：硬件查表阶段：所有的系统调用在执行陷阱指令时，向硬件提供的都是同一个中断类型号（如 X86 中的 0x80）。硬件（CPU）根据这个统一的类型号，去检索中断向量表，跳转到操作系统内核的统一入口——即系统调用处理程序 system_call 的首地址。软件查表阶段：进入内核态并执行 system_call 后，操作系统内核（软件）才会读取寄存器（如 EAX）中的系统调用号，并以此为索引去检索系统调用表（跳转表），最终跳转到具体的内核服务例程（如 sys_write）。结论：硬件并不会直接用系统调用号去查中断向量表，两者的映射级数和查表主体不同。

D 正确：陷入指令的执行会触发软中断，硬件响应中断后，PC 会被赋值为中断向量表里 0x80 号表项对应的地址，即系统调用处理程序 system_call 的第一条指令

37．D。【解析】动态重定位是在作业运行到一条访存指令时再把逻辑地址转换为主存中的物理地址，实际是通过硬件地址转换机制实现的，故答案为选项 D。

38．B。【解析】静态重定位下，程序被装入程序装入时即会实现逻辑地址向物理地址的转换，此后程序中的指令和数据的内存地址便完全确定了，因此是由装入程序完成的地址转换。

39．C。【解析】C 选项准确地概括了微内核架构的核心设计思想。通过将文件系统、设备驱动等非核心服务移出内核，运行在用户态的独立进程中，微内核实现了服务间的良好隔离。这意味着即使某个服务进程崩溃，也不会导致整个操作系统宕机，从而极大地提升了系统的可靠性 。这正是微内核相对于宏内核的主要优势之一，也是其设计的根本出发点 。

A 项错误：此说法颠倒了分层结构的依赖关系。在分层结构中，第 i 层只能使用第 i-1 层提供的服务，而不能直接调用更低层的服务，也不能被更高层直接调用 。每一层都是其下一层的服务使用者，同时也是其上一层的服务提供者。因此，A 项的描述是错误的。

B 项错误：此说法混淆了模块化结构与宏内核结构的特点。模块化结构确实将操作系统视为一个整体，但其精髓在于将这个整体分解为可独立加载的模块 。这些模块可以位于内核空间，也可以以某种形式在用户空间运行，其核心优势在于灵活性和可维护性，而非单纯为了追求执行效率 。宏内核之所以效率高，是因为其所有组件都在同一个地址空间，通过函数调用通信 。模块化结构并不必然保证最高的执行效率。

D 项错误：此说法完全误解了外核的定义。外核的设计哲学恰恰相反，它不向应用提供高级抽象接

口，而是将硬件资源（如内存页、磁盘块）的管理权部分或全部交还给应用程序 。应用程序需要自己编写代码来管理这些底层资源，实现所需的抽象（即所谓的“库操作系统”）。这样做是为了消除中间抽象层带来的性能开销，并赋予应用最大程度的控制权。因此，D 项的说法是错误的。

40．C。【**解析**】C 错误，每个层次只可以调用更低层次的功能和服务。分层式结构操作系统，各层次相互独立，依赖性低，对某一层次的修改不会影响其他层次，但由于每一层都要对下层进行系统调用，增加了开销，造成系统效率较低，因此 A、B、D 选项正确。

41．A。【解析】A 错误：“可靠性一定高于”表述绝对化。模块化结构虽通过模块隔离降低了部分耦合，但模块仍运行在内核空间，一个模块的错误仍可能破坏内核数据结构导致系统崩溃。其可靠性仅略优于传统宏内核，但远不及微内核，且“一定高于”不符合实际。

B 正确：宏内核和模块化结构都属于“大内核”范畴，核心服务（进程管理、内存管理、文件系统等）均运行在内核空间，这是它们与微内核的根本区别。

C 正确：模块化结构的核心优势之一是支持可加载内核模块，允许在系统运行时动态添加或移除功能，而传统宏内核所有功能必须在编译时静态链接，无法运行时动态调整。

D 正确：由于两者服务均在内核空间，服务间通信可通过直接的函数调用完成，无需跨地址空间的消息传递，因此性能较高。

42．B。【**解析**】A 错误，微内核架构下一些内核基础模块仍然运行在内核态。

B 正确，微内核不同模块运行在不同地址空间，不同模块间通信不能通过过程调用；宏内核不同模块运行在同一地址空间，不同模块间通信通过过程调用。

C 错误，微内核将机制与实现分离在不同特权级，所以切换更频繁。

D 错误，微内核设计安全性更强但是性能更差。主要是因为进程间通信（IPC）和频繁的用户态与内核态切换开销大

43．C。【解析】C 是微内核正确的描述。

A 错误，当前比较流行的、能支持多处理机运行的 OS，几乎全部都采用宏内核结构，包括 Windows XP。

B 错误，模块化 OS 结构的原则是：分解和模块化。I 中描述的是分层式结构设计的基本原则。

D 错误，微内核结构将操作系统的很多服务移动到内核以外(如文件系统)，且服务使用进程间的通信机制进行信息交换，这种通过进程间通信机制进行的信息交换影响了系统的效率，所以微内核结构设计并不会使系统更高效。一般来说，内核的内服务越少，内核越稳定。

44．C。【**解析**】在计算机系统中，操作系统通常存储在硬盘上，因为操作系统通常需要大量的存储空间，且需要频繁地修改和更新，因此不能将其存储在 ROM 或 EPROM 等只读存储器中。系统开机后，操作系统的程序会被自动加载到内存中的系统区，这段系统区域是 RAM，故选 C。

45．A。【**解析**】BIOS(Basic Input Output System，基本输入输出系统)是一组固化到计算机主板上一个 ROM 芯片上的程序，它保存着计算机中最重要的自诊断程序、CMOS 设置程序、基本输入输出程序、系统设置信息、开机加电自检程序和系统启动自举程序，其主要功能是为计算机提供最底层的、最直接的硬件设置和控制。

操作系统的引导分两步：

主引导扇区位 MBR 于硬盘的 0 磁道 0 柱面 1 扇区，共 512bytes，由三大部分组成：

①硬盘主引导记录 MBR（Master Boot Record）占 446bytes

②分区表 DPT（Disk Partition Table）占 64bytes

③硬盘有效标志（Magic Number）占 2bytes。

分区引导记录 PBR（Partition Boot Record），也常被称为分区引导扇区（Partition Boot Sector）。在操作系统引导过程中，PBR 是存储在磁盘分区的第一个扇区中的数据结构，它包含了分区的引导程序代码和引导加载程序的相关信息。

两个引导程序都在磁盘区域，一般来说，PBR 是 OS 的实际引导程序，如果同时出现 MBR 和 PBR，操作系统的引导程序位置应选 PBR。本题选 A。

46．D。**【解析】**A 对，开机后系统首先在实地址模式下工作，BIOS 程序首先检测显卡、键盘、内存等，并在主存的 00000H~003FFH 区域建立中断向量表，同时，在中断向量所指的主存区域建立相应的中断服务程序

B 对：此时操作系统还没加载，没有磁盘驱动。MBR 中的磁盘引导程序只能调用固化在 ROM 中的 BIOS 中断来读取硬盘后续扇区。

C 对：操作系统内核接管系统后，会从实模式切换到保护模式，此时不再使用 BIOS 的中断向量表，而是建立中断描述符表（即保护模式下的中断向量表）。

D 错：逻辑格式化的核心任务是创建文件系统结构（如写入超级块、i 节点表、空闲空间管理表等）。而“将操作系统内核映像文件复制到硬盘中”是操作系统安装阶段的任务。

47．C。**【解析】**A、B、D 都是 OS 内核在 RAM 中运行后（即初始化阶段）必须执行的操作（读入磁盘元数据到内存缓存、建表、挂载文件系统、拉起 init 进程）。

C 错误：划分扇区和建立 MBR 属于物理格式化和磁盘分区阶段的工作，这是在操作系统安装之前（甚至出厂时）就已经在磁盘硬件上完成的，绝不可能在 OS 内核启动后才去做。

48．A。**【解析】**

- 自检程序（POST）：开机时 BIOS 固件执行，用于检测 CPU、内存、显卡、键盘等硬件是否正常
- 引导装入程序（也叫主引导记录 MBR，在磁盘 0 磁道 0 盘面第 1 扇区）：通电自检结束后，加载“实际的引导程序”到内存中，根据分区表加载活动分区，将控制权交给活动分区。
- 引导程序（分区引导记录 PBR，活动分区的第一个扇区）：寻找并激活分区根目录下用于引导操作系统的程序。
- 加载操作系统。将操作系统加载进内存中执行。

计算机加电启动过程为：自检程序→自举装入程序(引导装入程序)→引导程序→操作系统。本题选 A。

49．B。**【解析】**A 错误，第一类 VMM 直接运行在物理硬件之上，它自身就是硬件资源的管理者。它上面运行的是客户机操作系统，并不依赖宿主操作系统。

B 正确，第一类 VMM 直接安装在硬件上，它本身就扮演着操作系统的角色，运行在最高特权级别，负责管理和分配硬件资源给上层的虚拟机。

C 错误，第二类 VMM 是作为应用程序运行在宿主操作系统之上的。 它不直接运行在硬件之上，而是通过宿主操作系统来间接访问和控制物理资源。

D 错误，通常情况下，第一类 VMM 的性能优于第二类 VMM。这是因为第一类 VMM 直接控制硬件，减少了宿主操作系统带来的开销和指令转换层次。 第二类 VMM 由于需要通过宿主操作系统间接访问硬件，会引入额外的性能损耗。

50．D。【解析】A 错误，在第一类虚拟机中，当虚拟机操作系统执行了一条 CPU 处于内核态才允许执行的指令时，会陷入 VMM。在支持虚拟化的 CPU 上，VMM 检查这条指令是由虚拟机中的操作系统执行的还是由用户程序执行的。如果是前者，VMM 将安排这条指令功能的正确执行。否则，VMM 将模拟真实硬件面对用户态执行敏感指令时的行为。在过去不支持虚拟化的 CPU 上，真实硬件也不会直接执行虚拟机中的敏感指令，这些敏感指令被转为对 VMM 的调用，由 VMM 模拟这些指令的功能。而在**支持虚拟化的 CPU** 上，虚拟机管理程序检查这条指令是由虚拟机中的操作系统执行的还是由用户程序执行的。如果是前者，虚拟机管理程序将安排这条指令功能的正确执行。否则，虚拟机管理程序将**模拟真实硬件**面对用户态执行敏感指令时的行为。

B 错误，第二类 VMM 负责把虚拟机程序的虚拟地址转换为宿主操作系统中的虚拟地址（这个虚拟地址在虚拟机 os 看来是”物理地址“），再由宿主操作系统转换为物理地址。

C 错误，虽然第二类 VMM 本身是作为宿主操作系统上的一个应用程序启动和运行的，但为了实现对虚拟机的管理和模拟一些特权操作，它的**部分组件**或在某些时刻可能需要宿主操作系统的内核态服务或者以某种特权级别运行（例如，通过加载内核模块或利用宿主 OS 提供的 API）。它不像第一类 VMM 那样完全取代操作系统而运行在最高特权级，但也不能说它完全运行在用户态而不涉及任何内核态的交互或依赖. 它的核心功能（如指令解释、设备模拟）大部分在用户态，但与硬件交互和资源管理离不开宿主 OS 内核的支持.

D 正确，第一类 VMM，直接运行在物理硬件之上。这意味着它可以直接管理和分配真实的、未经过抽象的物理硬件资源给虚拟机. 第二类 VMM 作为应用程序运行在宿主操作系统之上. 它通过宿主操作系统来访问物理硬件，因此它分配给虚拟机的硬件资源通常是经过宿主操作系统抽象和管理的“虚拟”或“抽象化”的物理硬件。

51．D。【解析】第一类虚拟机没有宿主操作系统，硬件资源的管理者是虚拟机管理程序（VMM），I 错误；第二类虚拟机管理程序负责把虚拟机程序的虚拟地址转换为宿主操作系统中的虚拟地址（这个虚拟地址在虚拟机 os 看来是”物理地址“），再由宿主操作系统转换为物理地址，II 错误；第一类虚拟机的非敏感指令执行在 ring1 层，而敏感指令执行在 ring0 层（内核态）。理想情况下，我们希望敏感指令集等于特权指令集，对于一般 RISC 处理器，如 MIPS，PowerPC 以及 SPARC，敏感指令就是特权指令，但是 x86 例外，但是 x86 绝大多数的敏感指令是特权指令，但是由部分敏感指令不是特权指令，执行这些指令的时候不会自动 trap，而是被 VMM 捕获。几乎所有现代 CPU 都支持硬件虚拟化，这些不会 trap 的敏感指令，是通过允许 VMM 捕获并仿真敏感指令来实现的，可以在用户态执行，

III 正确。

A。【解析】A 错误：这是微内核与外核最容易混淆的地方。微内核确实将文件系统等作为独立的用户态服务器进程运行；但在外核架构中，文件系统等功能是由应用程序链接的“库操作系统（LibOS）”提供的，LibOS 与应用程序运行在同一个进程的地址空间内，而不是作为“独立的用户态服务器进程”运行。因此 A 的说法错误。

B 正确：微内核架构中，各个功能模块（如文件系统、网络）是相互独立的用户态进程，必须依赖内核提供的进程间通信（IPC）机制来交换数据；而外核的设计哲学是“消除抽象”，它只负责分配物理资源（如物理内存页、磁盘块），不提供虚拟内存、文件系统等高级硬件抽象，这些抽象由应用层的“库操作系统（LibOS）”来实现。

C 正确：微内核将非核心服务移出内核态，使得某个服务崩溃不会导致整个系统宕机，从而提高了系统的可靠性；外核（Exokernel）顾名思义，其核心职责就是作为硬件资源的保护者，确保多个应用程序能够安全地复用底层物理资源，而不干涉应用如何使用这些资源。

D 正确：微内核最大的缺点是性能较差，因为每次服务调用都需要经过 IPC 机制，伴随着频繁的用户态与内核态上下文切换；外核将物理资源的管理权下放给应用程序，应用可以直接访问和管理硬件，彻底避免了微内核中沉重的 IPC 和上下文切换开销，因此执行效率通常更高。

第 2 章

进程管理

【速对答案】

DCDCC BCCBB DADDC BCCCD

DCDCD BABDC CBADB CBCCC

DCACA BDCDA ADABB DBBCB

CCDDC DBDBA BCCCC CADBA

ABDDD CAABB BCCBA DCADA

CACCC BCABD CDDBB BCCCD

CABBC DCBDC B

1．D。【解析】进程是程序的一次执行过程，因而是动态的；既然是一次执行，就需要使用 CPU 且表明进程有生命周期，即具有 创建、运行、消亡 这样的一个过程。而程序是指令和数据的集合，是一个静态的概念，因此本题答案为 D。

2．C。【解析】A 错误，进程由 3 部分组成：程序段、数据段、进程控制块 PCB；A 选项认为程序段就是进程是片面和错误的：进程运行确实需要将程序段装入内存，但内存中的程序段并不是进程。

B 错误，并行的进程之间，可以利用信号量等机制实现同步、互斥等关系，但是并不是所有并行(或并发)进程间都有相互依赖或制约的关系。

C 正确，当多个进程并发执行时，会使得进程具有不可再现性：并发执行的进程，在相同的初始环境和条件下，可能得到不同的结果。这是因为进程失去了封闭性。

D 错误，进程的四大特征为动态性、并发性、独立性和异步性。

3．D。【解析】进程可以包含多个线程，A 错误；

同一进程内的多个线程共享进程的用户地址空间，不必依赖内核实现线程间通信，B 错误；

C 选项前半句正确后半句错误，内核级线程操作系统中线程是调度的最小单位。

4．C。【解析】PCB 中不包含程序源代码，程序源代码属于程序段。

5．C。【解析】I. 当用户登录系统时，系统通常会为该用户创建一个初始进程，例如 shell 进程

II. 在批处理系统中，作业调度程序负责将作业从外存调入内存，并为该作业创建进程

III. 这里的“服务”通常指系统调用（如 exec、fork），或对用户请求的响应（如网络服务、打印

服务等）。例如，当用户运行一个程序时，操作系统会通过 exec 调用创建新进程，此时会导致创建新进程；注意，这里的用词是可以，不是一定；

IV. 设备分配是资源管理的一部分，涉及为进程分配设备（如磁盘、打印机等），但**不会直接导致进程的创建**，具体来说是通过在操作系统中设置相应的数据结构实现的，不需要创建进程。

V. 批处理作业初始化通常指的是将作业加载到内存并为其创建进程的过程，与作业调度密切相关；

VI. 文件创建是通过系统调用（如 open、create）实现的，**通常由当前进程完成**，不会创建新进程；

I、II、III、V 共 4 个，可能导致创建新进程，因此本题选择 C。

6．B。【解析】A、C、D 均可能且必须发生：进程从执行态变为阻塞态（例如主动请求 I/O、申请临界资源失败），这是一种内部异常（自陷）。系统会触发异常处理程序（D），保存当前 CPU 寄存器上下文（如 PC、PSW）到该进程的 PCB 中（A），随后调用进程调度程序让出 CPU（C）。

B 错误：该进程原本处于执行态（正在 CPU 上运行），其 PCB 并不在就绪队列中，因此不存在“从就绪队列移出”这一步。应该是将其 PCB 状态改为阻塞态，并直接插入到相应的阻塞/等待队列中。

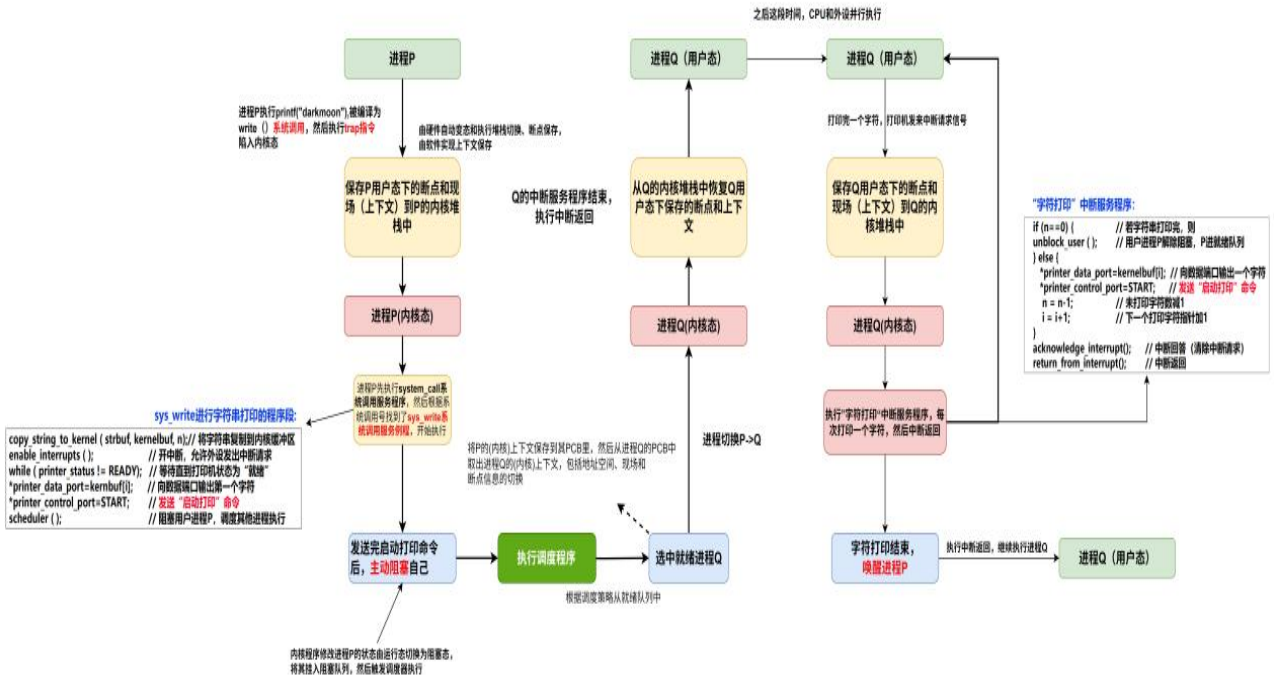
7．C。【解析】A. 执行 P (wait) 操作：当一个进程执行 P 操作（也称为 wait 操作）请求一个它当前无法获取的资源（例如，信号量的值小于或等于 0 时），该进程会从执行态转变为阻塞态，等待资源可用。

B. 启动 I/O 操作：当一个进程启动一个 I/O 操作时（例如，从磁盘读取数据），它通常会进入阻塞态，以等待 I/O 操作的完成。

C.　I/O 操作结束：假设 P1 进程执行时，接收到一个外设发出的中断请求信号，然后响应并且执行对应的中断服务程序，若检测到该 I/O 任务完成(假设这是进程 P2 请求的字符打印任务，此时字符已经打印完毕)，便会将进程 P2 解除阻塞，加入就绪态队列，因此 C 正确；这个过程较为复杂，涉及到 I/O 强化内容，下面放两张图抛砖引玉；

字符打印中断服务程序

```
if (n==0) {//若字符串打印完，则
    unblock_user ();//用户进程P解除阻塞，P进就绪队列
}else{
    *printer_data_port=kernelbuf[i];//向数据端口输出一个字符
    *printer_control_port=START;//发送“启动打印”命令
    n=n-1;//未打印字符数减1
    i=i+1;//下一个打印字符指针加1
    acknowledge_interrupt0://中断回答（清除中断请求）
    return_from_interrupt0://中断返回
```



D. 当前时间片到期：当前时间片到期是分时操作系统中进程调度的一个事件。当一个正在执行的进程用完了分配给它的时间片，它会从执行态转变为就绪态。相应的，CPU 会调度其他进程上处理器，使其从就绪态转变为执行态。

8．C。【解析】A 正确：无论是进程切换还是线程切换，保存当前执行流的上下文（通用寄存器、PC 等）到其控制块（TCB/PCB）是必须的第一步。

B 正确：由于是从进程 P1 切换到进程 P2，虚拟地址空间发生了改变。如果不更新页表基址寄存器，CPU 会使用 P1 的页表去翻译 P2 的虚拟地址，必然导致严重的内存访问错误。因此这是跨进程切换必须执行的操作。（注：如果是同进程内的线程切换，则不需要此操作，这也是本题强调 P1 和 P2 的原因）。

C 错误：关于 TLB（快表）：TLB 缓存的是“虚拟地址 →→ 物理地址”的映射。因为 P1 和 P2 的虚拟地址空间不同（相同的虚拟地址映射的物理页框不同），进程切换后 TLB 中的映射关系失效，必须刷新/清空 TLB。关于 Cache（高速缓存）：Cache 缓存的是物理内存的数据副本。现代 CPU 的 Cache 通常采用物理地址索引，进程切换并不会导致物理内存中的数据失效。更重要的是，Cache 对操作系统是完全透明的，由硬件自动管理，操作系统在进程切换时不需要、也无法直接去“清空” Cache。因此，“必须清空 Cache”是严重的错误表述。

D 正确：每个线程（无论属于哪个进程）在进入内核态时都需要使用独立的内核栈来保存内核局部变量和调用链路。因此，切换到 T2 时，必须将内核栈指针切换指向 T2 的内核栈。

9．B。【解析】I 正确，执行 P(wait) 操作：S = S - 1，若 S ≤ 0，当前进程会被插入该信号量的等待队列，主动进入阻塞态。

II 正确，进程主动申请 I/O 设备，设备当前忙时，进程需要等待 I/O 完成，会阻塞，属于典型的 I/O 阻塞。

III 错误，时间片用完，进程只是让出 CPU，从运行态→就绪态，依旧在就绪队列等待调度，并不会阻塞。

10．B。【解析】B 符合题意：当发生“缺页异常”时，由于需要从磁盘（外存）将缺失的页面调入内存，这是一个耗时的磁盘 I/O 操作，当前进程必须放弃 CPU 并进入阻塞态。当磁盘 I/O 完成后，中断处理程序将该进程唤醒，进程进入就绪态。最后，进程被调度程序选中，再次进入执行态重新执行刚才失败的指令。整个过程完美契合循环。A、C 不符：时钟中断（时间片用完）或被高优先级抢占，进程的状态变化是“执行态 → 就绪态 → 执行态”，不经过阻塞态。D 不符：getpid() 属于非阻塞型的系统调用，进程切换到核心态执行完迅速返回用户态继续执行，状态始终是执行态（不发生状态转换，仅发生 CPU 模式切换）。

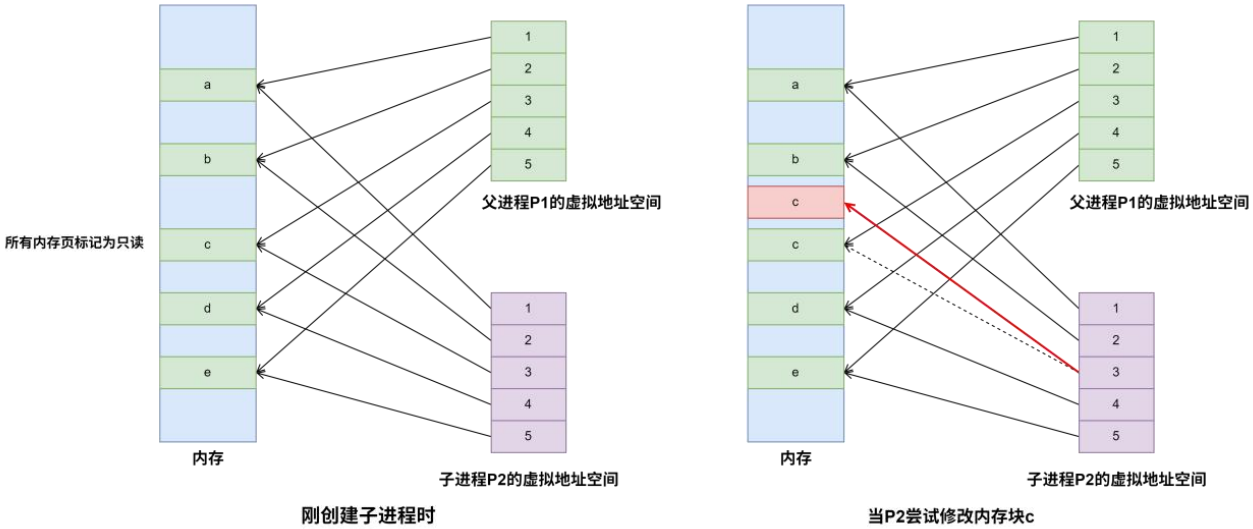
11．D。【解析】进程处于就绪态的原因只有还没有分配到 CPU，但是进程处于阻塞态的原因可能是多种多样的，因此每一类原因造成阻塞的进程都会单独构建一个 PCB 等待队列。

12．A。【解析】在操作系统中，父子进程的概念是通过进程创建过程中形成的层级关系来定义的。父进程是指创建子进程的进程，而子进程则是由父进程通过特定的系统调用（如 fork()）创建出来的新进程。在 Linux 系统中，fork() 函数被用来创建一个新的进程，称为子进程。这个子进程开始时几乎完全是父进程的副本，包括代码段、数据段和堆栈等。某些操作系统全局资源是存在父子继承的，例如后面会学到的文件、管道、设备等，对这些特定的全局资源来说，子进程通常会继承父进程对它们的访问权限。

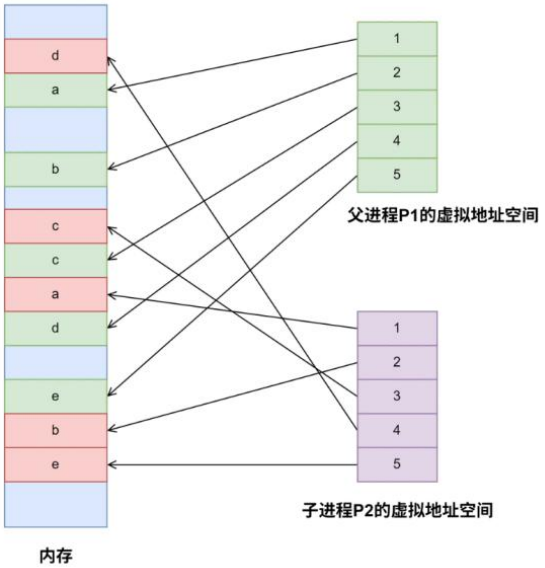
Ⅰ正确，注意父子进程是两个不同的进程，因此其 pid 和 PCB 都不同，也不会共享地址空间，我们知道进程具有独立性，是一个能独立运行、独立获得资源和独立接受调度的基本单位，要是可以共享地址空间，父进程进程执行的过程中发生了中断，但是在它由于中断而被暂停执行的过程中，其子进程还在执行，并且更改了内存里的部分信息，这就破坏了父进程的现场，因此二者不会共享地址空间。

【扩展】：

1．在linux操作系统中，当父进程创建子进程时，使用fork()的话，子进程会复制父进程的地址空间，也就是虚拟地址空间的副本，其代码段、数据段、堆栈等完全相同（之后通过执行 `exec()` 系统调用替换子进程的代码段，加载新程序）。不过通常会使用**写时复制（Copy-On-Write，COW）技术**，也就是说，在fork之后，父子进程的虚拟地址空间映射到相同的物理内存页，但一旦任一进程试图修改某个页，就会触发复制，生成该页的副本，这样他们就不会共享同一物理页。不过，从虚拟地址空间的角度来看，初始时他们的地址空间内容是一样的，但各自独立，不是共享的。共享的话意味着一个进程修改了数据，另一个进程能看到变化，但fork后的子进程在修改数据时会有自己的副本，所以虚拟地址空间并不是共享的。



2．在xv6这种操作系统中，父进程创建子进程时，将父进程的 **虚拟地址空间** 完整复制到子进程的地址空间中，包括：**代码段（Text）**、**数据段（Data）**、**堆（Heap）** 和 **栈（Stack）**。但是每个虚拟页对应的物理页会被 **逐页复制**（非共享），确保父子进程内存完全独立。



Ⅱ错误，如果父进程在子进程结束之前终止，那么子进程将成为孤儿进程。在 Unix 和 Linux 系统中，孤儿进程将由 init 进程（PID 为 1 的进程）收养，并最终被终止。

Ⅲ错误，如果子进程先于父进程退出，并且父进程没有主动回收子进程资源，子进程就会变成僵尸进程。

Ⅳ正确，临界资源一次只能为一个进程所用。

综上所示，选 A。

13．D。【**解析**】I正确，在页式存储管理中，CPU 通过页表将逻辑地址转换为物理地址。每个进程都有其独立的页表。当进程切换时，操作系统必须更新 CPU 中指向当前活动页表基址的专用寄存器。这样，新调度的进程在执行时才能使用其自身的页表进行正确的地址映射，访问其独立的地址空间。

Ⅱ正确，通用数据寄存器（如累加器、数据寄存器等）保存了进程运行时的中间计算结果和数据。在进程切换时，这些寄存器的内容属于被切换出去进程的上下文，必须被保存到该进程的进程控制块（PCB）中，并加载新调度进程之前保存的通用寄存器内容到 CPU 中。否则，新进程会使用旧进程的寄存器数据，导致运行错误。

Ⅲ正确，程序计数器指示 CPU 下一条要执行的指令的地址。进程切换时，当前进程的 PC 值（即下一条要执行的指令地址）必须被保存到其 PCB 中。然后，将新调度进程在其上次被换下时保存的 PC 值加载到 CPU 的指令指针寄存器中，以便新进程能从正确的位置继续执行。

Ⅳ正确，每个进程都有自己的用户栈和内核栈，栈顶指针寄存器（如 ESP、SP）指向当前栈的栈顶。进程切换时，当前进程的栈顶指针必须被保存，并且新进程的栈顶指针必须被恢复。这确保了新进程在执行时能够正确使用其自身的栈空间进行函数调用、局部变量存储等操作。

综上所述，选 D。

14．D。【**解析**】中断响应维护的上下文可能是用户上下文（因为多重中断的存在），进程切换维

护的一定是内核态下的上下文。

15．C。【**解析**】当前进程的页表基址和栈基址不会因为进程的执行而改变，因此当发生进程切换时，不一定需要保存，但是一定要更新；至于其他的寄存器里的内容会随着进程的执行发生变化，因此需要先保存再更新。

【扩展】：PSW 寄存器的内容在进程切换的过程中不需要保存或更新，在进程 P1 切换到进程 P2，进程 P2 返回用户态时才会更新；PSW 的保存与回复可能因具体实现架构而异，此处仅做拓展。

16．B。【**解析**】A 错误，进程切换全程发生在**内核态**，当进程从用户态切换到内核态（如通过系统调用、中断或异常），操作系统才会执行进程调度；切换过程中，CPU 的控制权完全交给内核，因此切换操作必须在内核态完成。

B 正确，**中断/异常是进程切换的前提条件**，因为只有在内核态才能进行调度和切换。例如，当进程执行系统调用（如 fork、exec）或发生异常（如缺页中断），会从用户态进入内核态，此时操作系统可能触发进程调度，从而引发进程切换。

C 错误，进程除了寄存器以外的还有很多其他状态，它有变量，堆中的数据等等，但是所有的这些数据都在内存中，并且会保持不变。我们没有改变进程的任何栈或者堆数据。所以进程切换的过程中，cpu 中的寄存器是唯一的不稳定状态，且需要保存并恢复。而所有其他在内存中的数据会保存在内存中不被改变，所以不用特意保存并恢复。我们只是保存并恢复了 cpu 中的寄存器，因为我们想在新的进程中也使用这组寄存器。至于页表基址本来就在进程 P0 的 pcb 中，因此进程切换时，只需要从进程 P1 的 PCB 中取出 P1 的页表基址覆盖掉原页表基址寄存器中的内容(P1 的页表基址)即可；需要保存的信息有通用寄存器，PC，栈顶指针寄存器等，至于页表基址寄存器，栈基址寄存器等只需要更新，不需要保存；

D 错误，中断响应保存和恢复上下文是分开进行的，分别位于中断响应和中断返回的过程中；而进程切换的上下文保存与恢复是连续进行的（两个不同进程之间）。

17．C。【解析】处于阻塞挂起状态的进程，它等待的事件在外存中完成。当 I/O 结束后，它的状态应当转换为就绪挂起态，而不是直接转换为就绪态。因为此时该进程的映像依然在外存中，需要经过中级调度（激活操作）将其调入内存后，才能变为就绪态。

18．C。【**解析**】A 错误，进程因得不到资源（如等待 I/O 操作完成、等待信号量等）而进入的通常是阻塞态，而不是挂起态。挂起态通常是由于终端用户请求、父进程请求、系统负荷调节或操作系统自身需要等原因，将进程置于静止状态。

B 错误，新进程创建完成后，会进入就绪态并被放入就绪队列。此时是否立即引起进程调度取决于调度策略。在非抢占式调度中，当前运行的进程会继续执行直到主动放弃 CPU 或结束。在抢占式调度中，如果新创建的进程的优先级高于当前运行的进程，则可能会引起调度。因此，新进程的创建不必然引起进程调度。

D 错误，当正在执行的进程时间片用完后，它会从执行态转换为就绪态，并被放入就绪队列的末尾，等待下一次调度。它不会进入阻塞态，因为时间片用完并不意味着它在等待某个外部事件。

19．C。【**解析**】CPU 状态变化中可能直接发生的有：①就绪态→执行态，如进程获得时间片。

②阻塞态→就绪态，如进程获得一直等待的除 CPU 以外的资源。③执行态→就绪态，如 CPU 时间片用完。④执行态→阻塞态，如请求磁盘数据。而就绪态→阻塞态、阻塞态→执行态这两种情况不可能直接发生，因此答案为选项 C。

20．D。【**解析**】进程除了寄存器以外的还有很多其他状态，它有变量，堆中的数据等等，但是所有的这些数据都在内存中，当该进程下处理机时这些信息会保持不变。我们没有改变进程的任何栈或者堆数据。所以进程切换的过程中，cpu 中的寄存器是唯一的不稳定状态，且需要保存并恢复。而所有其他在内存中的数据会保存在内存中不被改变，所以不用特意保存并恢复。我们只是保存并恢复了 cpu 中的寄存器，因为我们想在新的进程中也使用这组寄存器。

21．D。【**解析**】创建进程是一个很复杂的过程：首先由进程申请一个空白 PCB，并向 PCB 中填写用于控制和管理进程的信息；然后为该进程分配运行时所需的资源；最后，将该进程的状态转为就绪态并插入就绪队列的队尾。然而，如果进程所需的资源不能得到满足，例如系统尚无足够的内存装入进程，即创建工作尚未完成，进程不能被调度运行，那么此时进程所处的状态称为创建态。如果进程创建成功，那么进程状态就会转为就绪态，选项 D 错误。

22．C。【解析】C 错误：一个进程在单核 CPU 上同一时刻只有一个线程处于执行态。时间片用完时，只有那个正在执行的线程会从执行态变为就绪态。该进程包含的其他线程本来就在就绪态、阻塞态或挂起态，不可能“均由执行态变为就绪态”。

A 正确：用户级线程对操作系统内核是不可见的。内核只看到进程，一旦某个用户级线程陷入内核并阻塞，内核会把整个进程阻塞，导致该进程内所有其他用户级线程“陪绑”阻塞。

B 正确：内核级线程对内核是可见的。一个线程缺页阻塞，内核可以调度同进程内的其他就绪线程继续执行。

D 正确：进程被挂起（通常意味着其地址空间被换出到外存），由于没有物理内存支持，其包含的所有线程都无法运行，调度器会直接忽略它们。

23．D。【**解析**】A 正确，用户级线程上下文切换完全在用户空间完成，无需进入内核态，因此**无需系统调用**，代价较低。内核级线程切换需要保存和恢复更多寄存器状态，需进入内核态，代价更高。

B 正确，内核级线程的控制块（TCB）包含更多元数据（如线程状态、优先级、调度信息等），需要更多内存。用户级线程的控制块仅需存储少量信息（如栈指针、程序计数器等），因为其调度由用户空间库管理。

C 正确，内核级线程可被调度到不同 CPU 核心上运行，充分利用多核架构。用户级线程通常只能在一个进程中运行，无法直接利用多核优势。（操作系统内核无法感知到用户级线程的存在）

D 错误，用户级线程是由用户程序或者线程库实现的，它们不依赖于操作系统内核，其调度和同步也需要消耗 CPU 资源，而且它们无法充分利用操作系统内核的调度器，可能会出现资源争用和不公平的问题。（用户级线程的调度依赖于用户空间的实现，若未设计公平的调度算法，可能**不如内核级线程的调度公平**。）例如，在采用时间片轮转调度算法时，各个进程轮流执行一个时间片，这对于各进程而言貌似是公平的，但假如在进程 A 中包含了 1 个 ULT（user level thread，用户级线程），而在进程 B 中包含了 100 个 ULT，那么，进程 A 中线程的运行时间将会是进程 B 中各线程运行时间的 100 倍；相应

地，进程 A 的运行速度也要快上 100 倍，因此说实质上并不公平。因此 D 错误。

24．C。【解析】I 正确。线程共享同一进程的地址空间和资源，切换时不需要像进程切换那样切换完整的进程资源环境，所以开销通常更小。

II 正确。“内核级线程”的创建、调度、阻塞、切换等由操作系统内核管理，所以需要内核支持。

III 错误。内核级线程由内核分别调度，一个线程阻塞，内核可以调度同一进程中的其他线程继续运行。

IV 正确。内核级线程能被内核识别和独立调度，所以多个线程可以被分配到不同处理器上并行执行。

25．D。【解析】A 正确： 用户级线程的切换完全由用户空间的线程库完成，对内核透明，因此不发生模式切换。

B 正确： 同一进程内的内核级线程切换只需要切换线程上下文（如寄存器、栈），而不需要切换进程上下文（如虚拟内存地址空间、页表等），因此开销小于进程间切换。

C 正确： 同一进程的线程共享同一个页表（地址空间），因此切换时不需要更换页表基地址寄存器，快表（TLB）中的进程页表项依然有效，不需要刷新。

D 错误： 无论是哪种线程，只要发生切换，就必须保存当前线程的寄存器上下文（如程序计数器 PC、通用寄存器、栈指针 SP）。用户级线程的上下文由线程库保存在用户态的线程控制块（TCB）中，而不是不需要保存。

26．B。【解析】I 正确，用户级线程是完全由进程管理的，操作系统无法感知到，因此调度的单位依旧是进程。

II 正确，用户级线程的调度完全由用户空间的线程库实现，操作系统不参与线程的调度。

III 正确，在用户级线程中，不同的应用程序自己决定如何调度线程。

IV 错误，操作系统以进程为单位进行调度，因此这些线程每次只能执行一个，无法并行执行。

27．A。【解析】在多线程映射模型中，操作系统内核调度的基本单位是内核级线程（KLT）。内核完全不知道用户级线程（ULT）的存在。当一个 ULT 执行了阻塞系统调用时，请求会通过内核处理，导致与其绑定的该内核级线程（KLT）陷入阻塞状态。题目中共有 3 个 KLT，其中 2 个 ULT 阻塞导致了 2 个 KLT 被阻塞。此时，该进程只剩下 1 个可运行的 KLT。操作系统只能将这唯一一个处于就绪态的 KLT 调度到某一个 CPU 核心上运行。尽管用户态还有 4 个处于就绪态的 ULT，但由于缺乏充当“载体”的 KLT，它们只能由这唯一一个可运行的 KLT 轮流轮转执行，无法在多个核心上真正并行。因此，最多只能有 1 个线程在核心上运行。

28．B。【解析】KLT 的 TCB 由 OS 内核维护，存放调度信息、寄存器状态、内核栈指针等。A 错：OS 不管理 ULT 的 TCB；C 错：ULT 的 TCB 由用户态线程库在堆/数据段中维护；D 错：PCB 通常包含进程级共享信息（如页表指针、打开文件表），并指向该进程的线程 TCB 链表。

29．D。【解析】A 正确，一些 KST 对应多个 ULT，这是 ULT 通过时分多路复用 KST 来实现的，即将 ULT 对部分或全部 KST 进行多路复用，并且程序员可按应用需要和机器配置对 KST 的数目进行调整，以达到较好的效果。

B，C 正确，由于 ULT 和 KST 的连接方式不同，从而形成了 3 种不同的多线程模型：多对一模型、一对一模型和多对多模型。多对一模型，将多个 ULT 映射到一个 KST 上。该模型的主要优点是：线程管理的开销小，效率高。其主要缺点是：如果一个线程在访问内核时发生阻塞，则整个进程都会被阻塞；此外，在任一时刻，只有一个线程能够访问内核，多个线程不能同时在多个处理机上运行。一对一模型，将每个 ULT 映射到一个 KST 上。该模型的主要优点是：当一个线程阻塞时，允许调度另一个线程运行，所以它提供了比多对一模型更好的并发性能。此外，在多处理机系统中，它允许多个线程并行地运行在多处理机系统上。该模型的唯一缺点是：每创建一个 ULT，相应地就需要创建一个 KST，开销较大。

D 错误，多对多模型，将许多 ULT 映射到同样数量或较少数量的 KST 上。KST 的数目可以根据应用进程和系统的不同而变化，其可以比 ULT 数少，也可以与之相等，但不会多于 ULT 的数目。

30．C。【解析】A 正确： ULT 的管理依靠用户态的线程库（如利用函数调用），不需要陷入内核。B 正确： KLT 的调度由内核直接负责，内核能感知到每一个 KLT。C 错误： 对于 ULT（用户级线程），操作系统内核根本不知道它的存在，因此内核绝对不会为 ULT 分配内核栈。ULT 的 TCB 和模拟栈空间都存放在用户空间（进程的堆区）。只有 KLT 才会由内核为其分配 TCB 和内核栈。D 正确： 线程的基本定义：共享进程的资源（代码段、数据段、全局变量、堆），独享执行资源（栈、程序计数器、寄存器）。

31．C。【解析】线程使用的是所属进程的地址空间，并不单独占有地址空间，因此不需要分配页表。本题中的线程属于一对一线程模型，一个用户线程必须要有一个内核线程来对应，因此既需要为用户线程分配堆栈，也需要为对应的内核线程分配堆栈，另外，一对一线程模型下的线程是内核可以感知的，因此需要为其配备对应的线程控制块 TCB。

32．B。【解析】I正确，在引入线程的操作系统中，进程是操作系统进行资源分配（如内存、文件句柄等）的基本单位。线程作为进程内的一个执行实体，自身拥有非常少的资源（如程序计数器、寄存器组、栈），它必须依赖于其所属的进程而存在，无法脱离进程独立运行。

Ⅱ错误，线程不是简单的程序段的划分。一个进程可以包含多个线程，这些线程共享进程的代码段（程序段）和数据段。线程是进程中的一个执行流或执行单元。

Ⅲ错误，管道、共享存储、消息传递等机制通常是用于进程间通信（IPC）的。同一进程内的线程由于共享进程的地址空间（包括全局变量、堆内存等），它们之间的通信更为直接和高效，可以直接读写共享的内存数据，或者使用更为轻量级的同步机制（如互斥锁、条件变量）来协调。虽然理论上线程也可以使用某些 IPC 机制，但这通常不是它们主要的或最高效的通信方式。

Ⅳ正确，同一进程中的所有线程共享该进程的整个地址空间，包括代码段、数据段和堆。但是，每个线程为了能够独立执行函数调用和存储局部变量，都拥有自己私有的栈空间。一个线程不能直接访问另一个线程的栈空间。

33．A。【解析】在多线程环境下，同一进程内的所有线程共享该进程的大部分资源，但也拥有各自独立的一些资源。

Ⅰ．进程 P 的全局变量：可共享。全局变量属于进程 P 的地址空间的一部分。同一进程内的所有线

程共享该进程的整个地址空间，因此线程 Ta 和 Tb 都可以访问和修改进程 P 的全局变量。

Ⅱ. 线程 T0 的**局部变量存储区域**：不可共享。线程的局部变量通常存储在该线程各自的栈空间中。每个线程拥有自己独立的栈。因此，线程 T0 的栈对于线程 Ta 和 Tb 来说是私有的，它们不能直接共享或访问 T0 的局部变量存储区域。

Ⅲ. 套接字描述符 sock_fd：可共享。文件描述符（包括套接字描述符）是操作系统内核为进程维护的资源。当一个线程（如 T0）打开一个文件或套接字并获得一个描述符时，这个描述符在进程 P 的级别是有效的。由于同一进程内的所有线程共享进程所拥有的资源，包括打开的文件表项和套接字，因此线程 Ta 和 Tb 可以共享并使用由线程 T0 打开的套接字描述符 sock_fd 来进行网络通信。

Ⅳ. 线程 Ta 和 Tb 各自独立的程序计数器：不可共享。程序计数器（PC）是 CPU 中用于指示下一条要执行指令地址的寄存器。每个线程都是一个独立的执行流，因此每个线程（包括 Ta 和 Tb）都必须拥有自己独立的程序计数器，以追踪各自的执行位置。这使得它们能够并发（或并行）执行不同的指令序列。如果共享 PC，它们就无法独立执行了。

综上所述，选 A。

34．D。【**解析**】IV 错误，PCB 只有文件描述符，并没有文件缓冲区。PCB 包含：进程标识符、处理器状态信息（通用寄存器、PC、PSW、栈指针等）、进程调度信息（进程状态、进程优先级、阻塞原因）、进程控制信息（程序和数据的地址、信号量、标志、资源、指向所在队列下一个进程 PCB 的指针等）等。

xv6 中的 PCB（struct proc）

```c
//进程状态枚举
enum procstate {
  UNUSED,     // 进程槽位未使用（初始状态）
  USED,       // 进程已分配，但未初始化完成（过渡状态）
  SLEEPING,   // 进程在休眠（等待事件或资源）
  RUNNABLE,   // 进程可运行（在就绪队列中等待调度）
  RUNNING,    // 进程正在运行（占用 CPU）
  ZOMBIE      // 进程已终止，等待父进程回收（僵尸状态）
};
//内核态下进程上下文
struct context {
  uint64 ra;   // 返回地址（指令指针），即PC程序计数器
  uint64 sp;   // 栈指针
  uint64 s0;   // callee-saved 寄存器 s0-s11
  uint64 s1;
  ...
};
// xv6 源码片段（RISC-V 版本）：
struct proc {
    struct spinlock lock;       // 进程锁（用于同步）
    enum procstate state;       // 进程状态（UNUSED, SLEEPING, RUNNABLE 等）
    int pid;                    // 进程 ID
    struct proc *parent;        // 父进程指针
    void *chan;                 // 休眠等待的通道（用于同步）
    int killed;                 // 是否被终止
    uint64 kstack;              // 内核栈地址
    uint64 sz;                  // 进程内存大小，和下面的页表指针一起可以找到页表，这就是该进程的地址空间
    pagetable_t pagetable;      // 页表指针（RISC-V 页表）
    struct trapframe *trapframe; // 陷阱帧（保存用户态寄存器状态,用于从内核态返回到用户态）
    struct context context;     // 内核态上下文（用于进程上下文切换）
    struct file *ofile[NOFILE]; // 打开的文件表
    // ... 其他字段（当前目录、进程名等）
};
```

35．B。【**解析**】IV. 互斥信号量（如 mutex）通常属于系统资源，由操作系统维护。例如，当进程请求一个互斥锁时，操作系统会检查该信号量的状态。PCB 中可能包含进程当前持有的信号量或等待的信号量的引用，但信号量本身的数据结构（比如它的值、等待队列等）并不在 PCB 中。VI.进程执行的代码是代码段，和 PCB 一样是进程的组成部分，并不会被包含在 PCB 中。(进程的代码通常存储在内存或磁盘中，PCB 只保存代码段的起始地址，而非代码本身).其余均应包含在 PCB 中，故选 B。

36．C。【**解析**】本题命制依据为汤小丹计算机操作系统慕课版 2.2.4 进程管理中的数据结构中

4.PCB 的组织方式。

A 正确，线性方式（如数组）确实实现简单，可以通过下标直接访问。但如果要在中间插入或删除 PCB，可能需要移动后续的 PCB，导致效率较低。

B 正确，链接方式使用指针连接 PCB，可以灵活地形成各种队列（如就绪队列、不同事件的阻塞队列）。当进程状态改变时，只需修改指针即可将其从一个队列移到另一个队列，非常适合进程状态频繁变化的场景。

C 错误，索引方式通常是为不同状态的进程分别建立索引表（如就绪索引表、阻塞索引表等），而不是一个统一的索引表让所有 PCB 都通过进程 ID 直接计算。

D 正确，在链接方式中，插入或删除一个 PCB 通常只需要修改相邻节点的指针，时间复杂度较低。而在线性方式（如数组）中，如果在中间位置插入或删除，可能需要移动大量后续元素。

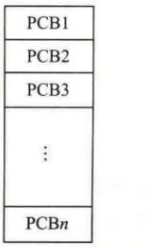


图 2-11　PCB 线性表示意

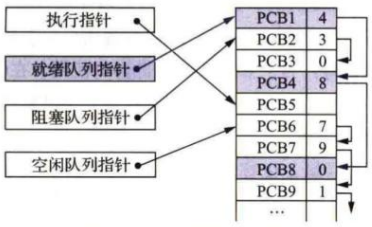


图 2-12　PCB 链接队列示意

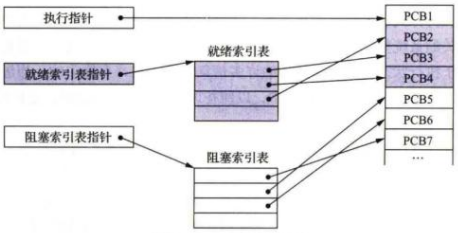


图 2-13　PCB 索引方式示意

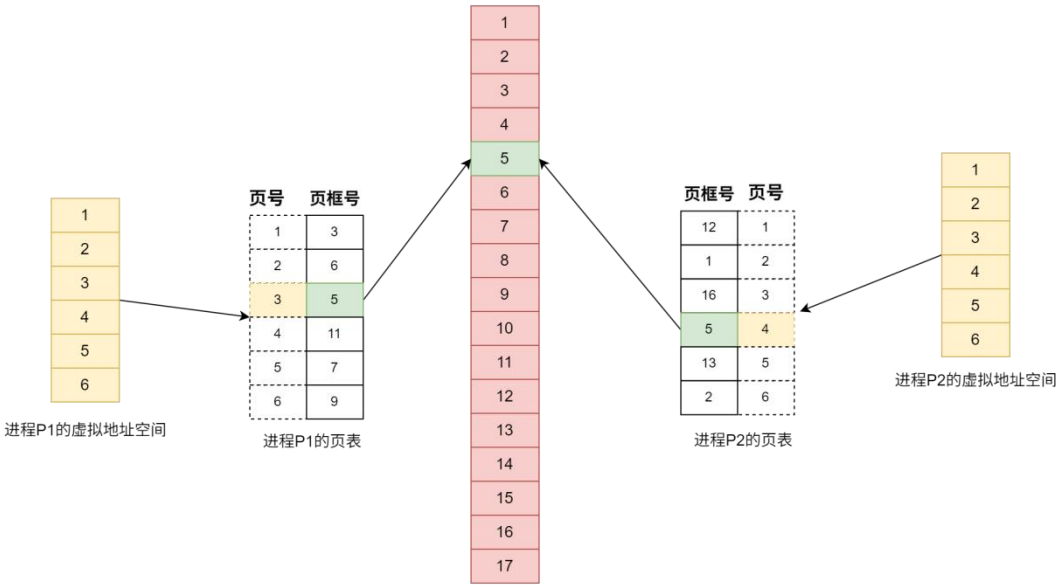
37．B。【解析】A 正确，为了唯一地标识和管理线程，TCB 中必然会包含线程的唯一标识符。为了支持线程的独立调度和执行，每个线程必须有自己独立的程序计数器（PC）来记录下一条要执行的指令地址，以及一组寄存器（如通用寄存器、状态寄存器等）来保存其运行时的上下文。

B 错误，虽然同一进程中的线程共享进程的地址空间（如代码段、数据段、堆），但每个线程为了能够独立执行函数调用、存储局部变量和返回地址，必须拥有自己私有的栈空间。因此，TCB 中需要包含一个指向该线程私有堆栈的指针（栈指针）。

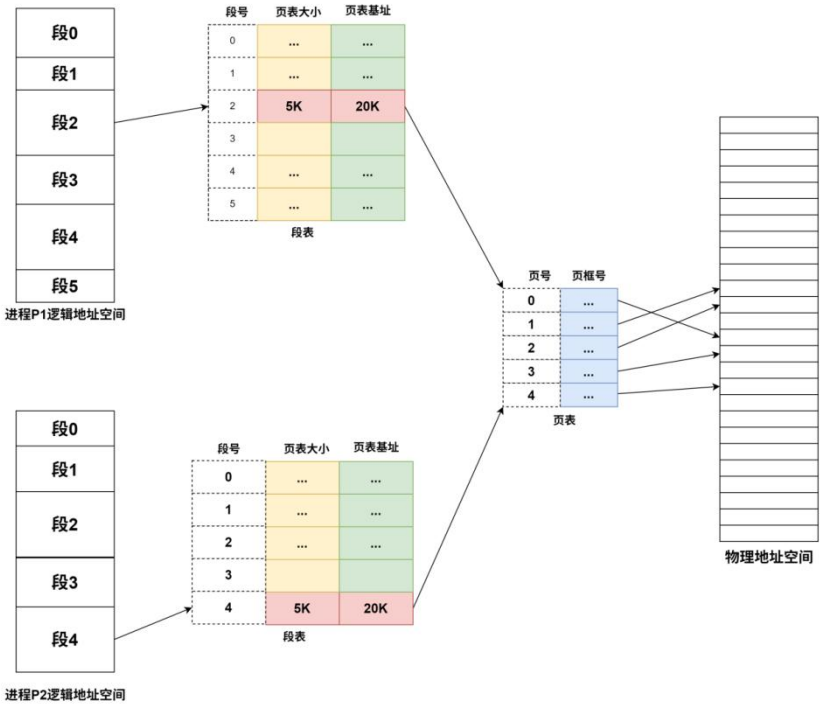
C 正确，与进程类似，线程在其生命周期中也会经历不同的状态（如执行态、就绪态、阻塞态）。这些状态信息以及用于调度的优先级等，都是管理和调度线程所必需的，因此通常会记录在 TCB 中。

D 正确，内核级线程（KST）是由操作系统内核直接支持和管理的，因此其 TCB 也存放在内核空间，由内核负责维护。用户级线程（ULT）的管理完全在用户空间进行，由用户空间的线程库负责其 TCB 的创建、调度和管理，内核对此是不可见的。

38．C。【解析】创建共享内存需要通过系统调用实现，因为需要先进入内核态，内核在物理内存中分配一块连续区域，指定共享内存的访问权限（如读写权限）和大小；然后进程再通过系统调用将共享内存映射到自己的虚拟地址空间。(在虚拟页式内存管理的操作系统中，内核修改进程的页表，将指定的虚拟地址范围映射到共享内存的物理页框。多个进程的虚拟地址可以不同，但最终指向同一物理内存，如图所示)

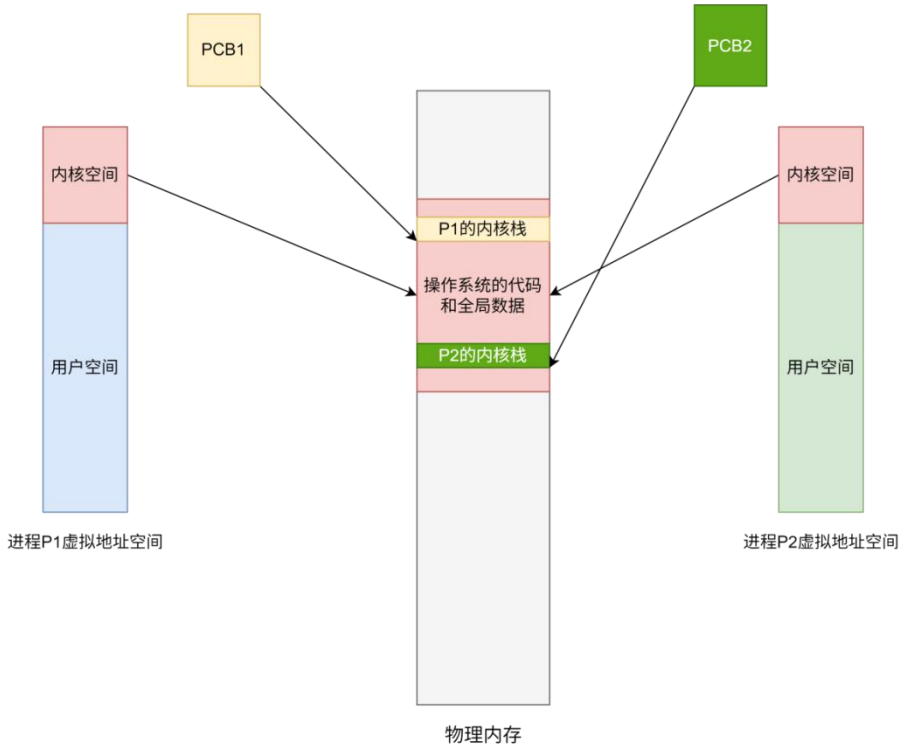


39．C。【解析】如图



40．C。【**解析**】B. 补充：无论哪个进程切换到内核态，访问的内核代码和全局数据（如中断向量表、系统调用表、设备驱动、内存管理结构等）都是同一份物理内存的映射；

C：虽然说内核空间是共享的，但是每个进程在内核态执行时需要使用独立的内核栈，用于保存进程处于内核态时内核函数调用的返回地址、局部变量和中断上下文(进程响应中断/异常时，会把用户态下的断点/现场信息压入内核栈)，每个进程的内核栈指针位于该进程的 PCB 中；



41．D。【解析】下面给出管道(Pipe)通信的可能考察的所有考点：

注意与 PV 信号量中的管程，I/O 方式中的通道做区分

管道实际上是一种固定大小的缓冲区，管道对于管道两端的进程而言，就是一个文件，但它不是普通的文件，它不属于某种文件系统，而是自立门户，单独构成一种文件系统，并且只存在于内存中。

普通管道只允许单向通信，数据只能往一个方向流动，要实现双向数据传输，就需要定义两个方向相反的管道。

管道的容量大小通常为内存上的一页，它的大小不受磁盘容量大小的限制。

可有多个读写进程对管道进行读写，但需要互斥访问，当管道满时，进程在写管道会被阻塞，而当管道空时，进程在读管道会被阻塞。

管道一般用于亲缘关系的进程通信，匿名管道只能由创建它的进程（以及它的子进程）访问，而命名管道可以由任何可以访问对应文件的进程访问。

管道允许两个进程按照生产者-消费者的方式进行通信。

管道中的数据只能读一次，做一次读操作之后数据也就没有了（读数据相当于出队列）。

D 错误，管道文件位于内核区中，读写操作需要通过系统调用在内核态下读或写；

42．C。【解析】A、B、D 均正确：硬件锁不关心有多少个核、多少个进程，直接在硬件电路层面原子化，但因为抢锁是随机的，可能导致某个进程一直抢不到而饥饿。C 错在：不管是 TSL 还是 Swap，当进程拿不到锁时，执行的都是 while(TSL(&lock)); 这种死循环忙等，并没有主动放弃 CPU 并进入阻塞态，因此违背了“让权等待”准则。

43．A。【解析】进程的地址空间是相互隔离的，其他进程无法直接访问到该进程的全局变量。

44．C。【解析】A. 错误，管道自身以及通过 mutx 互斥锁和两个等待队列实现了同步与互斥，但

是共享内存自身没有同步互斥机制，一般需要和信号量一同使用；

B. 错误，管道在内存之中，管道只能读 head(读指针)指向的数据，只能在 tail（写指针）指向的区域写入数据（队尾写入，队头读出），不支持 lseek 之类的文件定位操作。

C. 正确，管道（Pipe）的同步与互斥机制就是通过 mutex 互斥锁，等待队列（rd_wait 和 wr_wait）实现，读写操作需要用到读写指针（也可以理解为循环队列的队头、队尾指针）；

D. 错误，对管道的读写操作都需要互斥进行，同一时刻，管道只允许一个读进程对管道执行读操作或一个写进程对管道执行写操作；

45．A。【**解析**】B 错误，当所有引用管道的文件描述符都被关闭后，内核直接回收管道的缓冲区资源；

C 错误，在此情景之下，持有管道读/写端文件描述符的进程数量都不为 0，只有当 readers 和 writers 均为 0 时，内核才会销毁管道并释放其缓冲区内存

D 错误，匿名管道只能用于具有血缘关系的进程同学滚毫无亲缘的两个独立进程，进程地址空间、文件描述符表完全隔离，没有任何天然的 fd 复制途径，无法互相拿到对方管道的 fd 自然没法访问同一个管道。

46．B。【**解析**】管道是一种特殊的文件，读写进程以字符流形式进行读出和写入，因此 B 选项正确。

47．D。【**解析**】A 错误，当写端引用计数为 0（所有写者关闭）且管道为空时，内核不会让读者阻塞，而是认为到了文件末尾，read() 系统调用会立即返回 0；如果管道内还有数据，读者可以继续读取，直到读空后下一次 read() 返回 0。B 错误，管道底层是环形缓冲区（循环队列），数据遵循的是“先进先出”（FIFO）原则，队尾写入，队头读出。C 错误，管道同一时刻只允许一个读进程或一个写进程操作，读操作也会修改缓冲区（移动 head 指针、减少 data_size（即队列当前大小）），必须互斥。D 正确，管道本质上是半双工通信，如果父子进程之间需要实现全双工（双向）通信，必须创建两个独立的管道：一个用于父进程写、子进程读；另一个用于子进程写、父进程读。

48．C。【**解析**】A 正确，进程可以使用系统调用（如 kill()）向其他有权限接收的进程发送信号，也可以给自身发信号。。同时，内核也会在某些事件发生时（如硬件错误、非法指令、定时器超时、子进程终止等）向特定进程发送信号。

B 正确，对于大多数信号，进程可以选择三种处理方式：

- 忽略信号：进程不响应该信号，如：Linux 中的 SIGWINGH 信号。
- 执行系统默认操作：每种信号都有一个系统预设的默认处理动作，如终止进程、忽略信号、暂停进程等。
- 执行用户自定义的处理函数：进程可以注册一个信号处理函数（signal handler），当接收到特定信号时，执行该用户定义的函数。

C 错误，信号是一种异步通知机制，它本身并不携带大量数据。它主要用于通知接收进程发生了某个事件。进程可以通过信号处理函数来响应这个通知，但信号本身不适合用来直接传递大量或复杂的数据结构。传递大量数据通常需要借助管道、共享内存、消息队列等其他 IPC 机制。

D 正确，操作系统定义了一些具有特殊用途的信号，它们的行为是固定的，不能被用户进程通过设置信号处理函数来改变（捕获）或忽略。如 SIGKILL，SIGSTOP。

49．D。【解析】利用共享文件进行的进程通信机制称为管道。管道是一种文件，是服务于管道通信的特殊文件，而管道通信是一种通信方式，这种通信方式比较特殊，一般是半双工通信方式，数据只能单向流动，数据传递的方式是字符流形式。故答案为选项 D。考生要注意区分共享存储与管道通信的区别。

50．A。【解析】管道是一种半双工的进程通信机制，适用于父子进程或具有亲缘关系的进程间通信。在这种情况下，P1 可以将数据写入管道，P2 从管道中读取数据，并将数据再次写入管道，最后 P3 从管道中读取数据。

消息队列适用于进程间传递消息的场景，但不适合多个进程之间的连续数据传递。

共享内存允许多个进程访问同一块内存区域，但它需要额外的同步机制来确保数据的正确性，并且不适合这种连续的进程链式通信。

信号量用于进程间同步和互斥，但不能直接用于进程间数据传递。

因此，选项 A 是最适合这个需求的进程通信机制。

51．A。【解析】A 项： 共享内存最核心的优势在于：一旦建立映射，多个进程可以直接读写同一块物理内存，不需要在用户态和内核态之间来回拷贝数据。因此，无论是面对“大容量”还是“高频次”的数据交换，它的效率都是绝对最高的。B 项：概念混淆。 消息传递（Message Passing）是必须经过内核的。发送方将消息复制到内核缓冲区，接收方再从内核缓冲区复制到自己的用户空间。选项中说“无需内核干预”是错误的（这恰恰是共享内存的特点）。C 项： 传统的管道（Pipe）是半双工（单向）的，而不是“双向并发”；且它严格遵循 FIFO（先进先出） 原则，属于典型的有序通信，而非“无序”。D 项： 这是最经典的考点。管道和消息传递是自带同步机制的（例如管道满了，写进程会阻塞；管道空了，读进程会阻塞）。而共享内存本身不提供任何同步机制，如果多个进程同时读写，会导致数据混乱，必须配合信号量（Semaphore）等互斥工具使用。

52．D。【解析】I正确，共享存储、信息传递、通道和信号都是高级通信方式，PV 操作(信号量)、管程也可用于进程通信，属于低级通信方式。

II 正确，共享内存方式允许多个进程直接访问同一块内存区域。一旦共享内存区域映射到进程的地址空间，进程就可以像访问自己的私有内存一样读写这块区域，无需操作系统内核的介入来进行数据复制。因此，与其他需要内核参与数据拷贝的通信方式（如管道、消息队列）相比，共享内存通常是速度最快的进程间通信方式。

III 正确，消息传递机制中，进程以格式化的消息为单位，通过操作系统提供的发送和接收原语进行数据交换。这种方式隐藏了通信实现的细节，使用方便，并且能够很好地支持分布式系统和计算机网络环境下的通信，因此成为当前应用最为广泛的一类进程通信机制。

IV 正确，信号（Signal）是一种异步的通知机制，用于通知接收进程发生了某个事件。当一个进程接收到信号时，它的正常执行流程会被打断，转而去执行相应的信号处理程序（如果已注册），或者执行系统默认的操作。这种打断当前执行流并转去执行特定处理程序的过程，与中断（特别是软件中断或

陷入）的机制非常相似。信号通信可以被视为一种软件层面的中断或异步事件通知。

53．A。【解析】A 错误，虽然在基本的时间片轮转算法中，时间片大小通常是预设的，但在一些更高级的调度算法（如多级反馈队列调度）中，不同队列的进程可以拥有不同的时间片大小，并且系统也可能根据运行情况动态调整时间片的大小以优化性能。因此，时间片的大小并非绝对固定且不可调整。

B 正确，这是时间片轮转调度算法的基本工作方式。当进程用完其时间片后，如果它还需要继续执行，调度程序会将其置为就绪态，并将其移至就绪队列的尾部，等待下一轮调度。

C 正确，操作系统通过设置定时器（时钟部件）在每个时间片结束时产生一个时钟中断。CPU 响应这个中断后，会进入内核态，执行调度程序，从而剥夺当前进程的CPU，并分配给就绪队列中的下一个进程。

D 正确，如果时间片设置得非常长，以至于大多数进程都能在一个时间片内完成，那么进程调度的行为就近似于先来先服务（FCFS）算法。这会使得短作业和需要快速响应的交互式用户得不到及时的处理，降低了分时系统的交互性。

54．B。【解析】在采用非抢占调度方式时，一旦把处理机分配给某进程，就会一直让它运行下去，而决不会因为时钟中断或其他原因去抢占该进程的处理机，直至该进程完成或发生某事件而被阻塞时，才会把分配给该进程的处理机分配给其他进程。

在采用非抢占调度方式时，可能会引起进程调度的因素可归结为：

①正在执行的进程运行完毕，或执行 yield 系统调用主动释放处理机资源；

②正在执行的进程因提出 IO 请求而暂停执行；

③在进程通信或同步过程中执行了某种原语操作，如 Block 原语。

非抢占调度方式的优点是实现简单、系统开销小，其适用于大多数批处理系统。但它不能用于分时系统和大多数实时系统。

选项 A 会使该进程进入就绪态，选项 C、D 可能使某个进程从阻塞态进入就绪态，因此选项 A、C、D 不会引起进程调度。而选项 B 会使当前进程从运行态进入阻塞态，会引起调度。

55．B。【解析】非抢占式调度方式下，运行态上的进程要么一口气完成，要么因为某些事件到阻塞态，不可能到就绪态。

56．D。【解析】A 正确，批处理系统的设计初衷就是高效地完成大量计算任务，因此关注的是整个系统的效率（吞吐量、资源利用率）以及作业从提交到完成的整体时间（周转时间）。

B 正确，分时系统允许多个用户通过终端与计算机交互，因此快速响应用户输入、提供流畅的交互体验是其核心调度目标。同时，为保证每个用户都能获得合理的 CPU 时间，公平性也是一个重要因素。

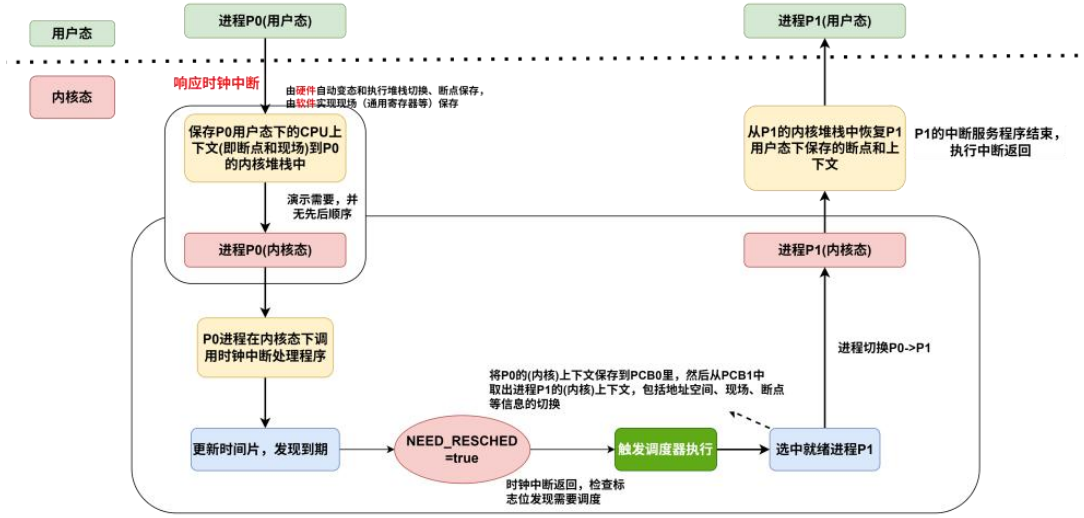
C 正确，硬实时系统通常应用于工业控制、航空航天等对时间要求极为严格的领域。其调度算法的首要任务是保证所有硬实时任务在其规定的截止时间（deadline）内完成，系统的可预测性和高可靠性是关键，有时甚至可以牺牲部分资源利用率来确保这些目标的实现。

D 错误，不同类型操作系统的处理器调度目标确实存在差异，这些目标之间相互矛盾，需要在设计时进行权衡和折中。

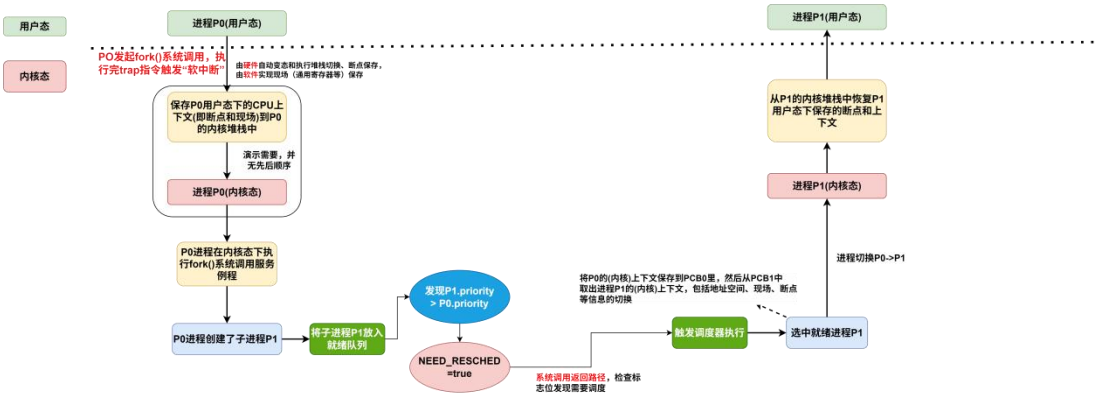
57．B。【解析】在设计处理机调度算法时，需要综合考虑多个目标，以期望获得良好的系统性能和用户体验，包括公平性，资源利用率，平均周转时间等。但不应包括互斥性。互斥性不属于调度算法直接考虑的性能指标。互斥性是指当多个进程访问共享资源（临界资源）时，确保在任意时刻只有一个进程能访问该资源。这是通过进程同步机制（如信号量、互斥锁、管程等）来实现的，而不是由处理器调度算法本身来直接保证或作为其设计目标。调度算法关注的是如何分配 CPU 时间，而互斥机制关注的是如何安全地共享其他资源。

58．B。【解析】抢占式调度中，在进程的 PCB 中，为进程设置一个**标志位**，例如 linux 中 PCB 里的 **TIF_NEED_RESCHED（记作 NEED_RESCHED）**标志位，其含义为“**需要重新调度**”，当某个进程的 NEED_RESCHED 标志被设置时，表示内核需要尽快触发调度器 scheduler，重新选择下一个要运行的任务。

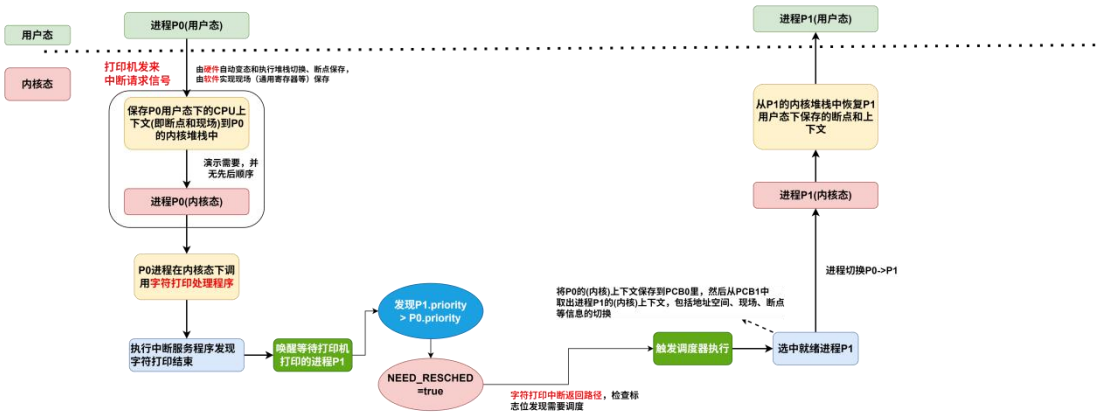
I. 当前任务的时间片用完时，内核会设置 NEED_RESCHED，以便让其他任务运行。



VI. 当**有更高优先级的进程进入就绪态**，会设置 NEED_RESCHED，例如**当前进程** fork()**一个优先级更高的子进程**



III. 处理 I/O 后唤醒阻塞进程，发现其优先级比当前运行进程优先级更高，设置 NEED_RESCHED



V. 在**中断/异常/系统调用处理结束返回用户空间**时，内核会检查当前进程的 NEED_RESCHED 标志，若有效则触发调度；

II. eg：进程 P1 执行 read()系统调用读文件时，会主动阻塞自己，从而触发调度，直到数据读入内核缓冲区 P1 才会被唤醒，因此正在执行系统调用服务例程也有可能触发调度；

IV. 进程**在访问操作系统内核临界区中不应被剥夺处理机资源；**(注意在用户级别的临界区中可以进行进程调度)

59．C。【**解析**】闲逛进程不需要 CPU 以外的其他资源，且只运行在内核态下；

60．B。【**解析**】响应比=（等待时间+要求服务时间）/要求服务时间。其中，等待时间+要求服务时间=进程周转时间=作业完成时间-作业提交时间。

61．C。【**解析**】A，D 描述的是一种情况。在多级反馈队列调度算法中，当一个进程在高优先级队列中用完了其（通常较短的）时间片但仍未完成时，它会被移至一个优先级较低、时间片较长的队列中。这本身就是一种降低优先级的体现，目的是照顾短作业和交互式进程，同时防止长作业饿死。

B，进程“抖动”意味着它分配到的物理内存不足，导致频繁的页面换入换出，CPU 大部分时间用于处理缺页中断而非有效计算，这会严重降低系统性能。此时，降低该进程的优先级，甚至暂停该进程的运行，以释放资源给其他进程，是一种合理的系统调整策略。

C， I/O 密集型进程的特点是 CPU 执行时间短，大部分时间在等待 I/O 完成。为了提高系统整体效率和 I/O 设备的利用率，操作系统通常会倾向于提高这类进程的优先级，以便它们在 I/O 完成后能尽快获得 CPU，再次发起新的 I/O 请求，从而保持 I/O 设备的繁忙。降低这类进程的优先级反而会延长其周转时间，降低系统对 I/O 操作的响应速度。

62．C。【解析】MLFQ 的核心思想是“偏爱 I/O 密集型/交互式进程”。当一个进程发起 I/O 请求时，说明它很可能是交互式进程，为了让用户感觉系统响应迅速，当它 I/O 完成重新进入就绪态时，通常会返回它阻塞前所在的队列，甚至被提升到更高优先级的队列，而不是惩罚性地降到最低优先级。C 选项描述的是对 CPU 密集型进程的惩罚机制，用在 I/O 完成后是错误的。

63．D。【**解析**】I正确，时间片轮转算法中，时间片大小的选择是一个关键问题。如果时间片设置得过短，CPU 会在进程间频繁切换，而每次上下文切换都需要消耗系统资源（保存和恢复现场等），导致系统整体开销增大，有效工作时间减少。

Ⅱ正确，时间片轮转调度的核心机制是通过时钟中断来实现的。操作系统设置一个定时器，每当一

个时间片结束时，定时器会产生一个时钟中断。CPU 响应中断后，会转去执行时钟中断处理程序，该程序会剥夺当前运行进程的 CPU 使用权，并调用调度程序选择下一个就绪进程投入运行。

Ⅲ正确，在时间片轮转调度中，如果一个进程在其分配的时间片内没有执行完毕，当中断到来时，该进程会被剥夺 CPU，其状态从执行态转换为就绪态，并被重新放回就绪队列的末尾，等待下一轮调度。如果进程在时间片用完前就已经完成，则会进入终止态。

注意：有些同学可能考虑到时间片用完时，恰好进程执行完毕，因此此时是从运行态转变为退出态； 但是一般来说，时间片用完时，进程的状态会被直接转为就绪态，然后在下一次调度时被处理(检查到执行结束再退出)。

Ⅳ正确，时间片轮转调度算法需要将所有处于就绪态的进程组织在一个队列中，这个队列通常被称为就绪队列。当进行调度时，调度程序会按照先进先出（FCFS/FIFO）的原则从就绪队列的头部选择下一个进程分配 CPU。 进程控制块（PCB）、时钟中断处理程序和进程就绪队列是实现时间片轮转调度的关键数据结构或程序。

64．D。【**解析**】下面给出常考查的进程调度时机：

- 可以调度
  - 创建新进程后
  - 进程正常结束或异常终止（选项I）
  - 进程因 I/O 请求、信号量操作等原因阻塞（选项II）
  - I/O 完成后
  - 出现高优先级紧急任务（选项III）
  - 进程处于普通临界区
  - 进程从内核态返回用户态时（选项IV、【2012-30】选项 D）
- 不可调度
  - 进程处理中断的过程中
  - 进程执行原子操作中
  - 进程处于内核程序临界区
  - 关中断状态下

综上所述，选 D。

65．C。【**解析**】C 选项，高响应比优先调度算法通常是非抢占式的。它在选择下一个要执行的作业或进程时，会计算所有就绪作业/进程的响应比（响应比 =(等待时间 + 要求服务时间) / 要求服务时间），然后选择响应比最高的。一旦选定，该作业/进程会一直执行下去。将其设计成抢占式会比较复杂且不常见，因为响应比是动态变化的，频繁地重新计算和抢占会增加系统开销。

66．D。【**解析**】进程发生饥饿时，应当增加其优先级，使其有机会使用处理器；当用户需要一个进程在规定时间完成时，也应该给予其高优先级以让该进程可以立即运行；/0 操作完成后，可以适当提高进程优先级，A、B、C 均是提高优先级的场合。当时间片耗尽后，显然不应该提高其优先级，否则该进程可能再次使用处理器，导致其他进程无法得到处理器资源。一般来说，D 选项的情况下应该降低

优先级。

67．B。【**解析**】A 正确，时间片轮转（RR）调度算法为每个就绪进程分配一个固定的时间片，进程在该时间片内运行。当时间片用完后，即使进程未完成，也会被剥夺 CPU，CPU 转给下一个就绪进程。这种机制保证了每个进程都能在较短的时间内得到响应，非常适合需要快速响应用户输入的人机交互环境，是分时操作系统的常用调度算法。

B 错误，抢占式调度算法允许高优先级进程抢占低优先级进程的 CPU。虽然这可以保证重要任务的及时执行，但如果系统中持续有高优先级进程到达，低优先级进程可能会长时间甚至永远得不到CPU，从而产生饥饿现象。例如，在抢占式优先级调度中，如果低优先级进程的优先级始终无法提升，就可能饿死。

C 正确，多级反馈队列调度算法通过设置多个不同优先级的就绪队列，并为每个队列分配不同的时间片大小，同时允许进程在不同队列间移动。这种机制能够较好地兼顾不同类型进程的需求：

- 对于交互型进程或短作业，它们可以在高优先级队列中获得较短的时间片并快速得到响应和完成。
- 对于处理器密集型或长作业，它们会逐渐从高优先级队列降到低优先级队列，获得较长的时间片，避免了因在高优先级队列中频繁切换而带来的开销，同时也保证了它们最终能够得到执行，避免了饥饿。

D 正确，先来先服务（FCFS）调度算法对所有类型的进程都是公平的，即按照它们到达的顺序进行服务。然而，如果一个长作业（通常是处理器密集型）先到达，它会长时间占用CPU，导致后续的短作业（包括 I/O 密集型进程，它们通常 CPU 执行时间短）长时间等待。因此，FCFS 算法通常不利于短作业和 I/O 密集型进程，而相对有利于长作业（处理器密集型进程，一旦获得 CPU 可以长时间使用）。

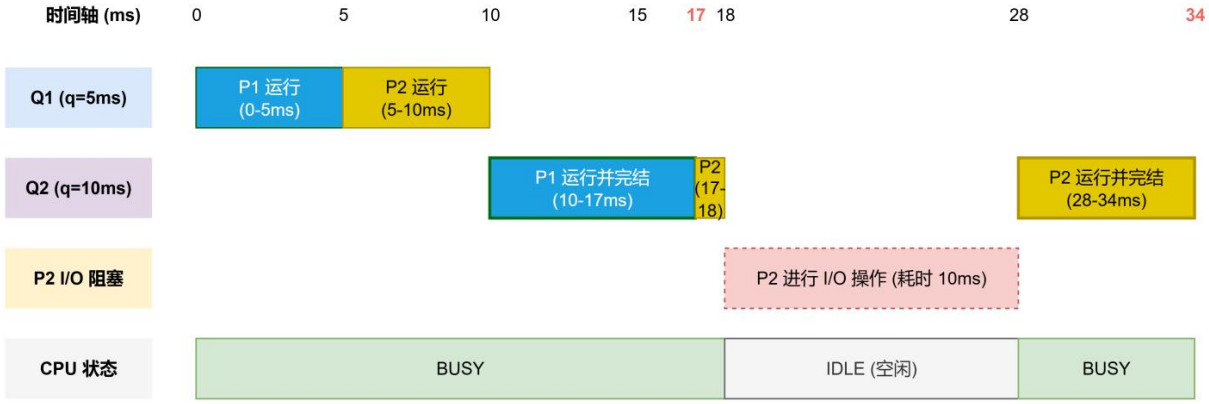
68．D。【**解析**】分时系统的响应时间 T 可以表达为：T≈QxN，其中 Q 是时间片，而 N 是交互进程数量。因此，对进程响应时间的因素主要有：“时间片值的选取”和“交互进程的数量”。当时间片一定，交互进程的数越多（即 N 越大），T 就越大。所以选择 D。

69．B。【**解析**】III错误，多级反馈队列调度算法需要维护多个队列，并在进程执行过程中根据其行为在不同队列间移动进程，这会带来一定的调度开销。相比于简单的调度算法（如先来先服务），其系统开销通常要大一些。虽然它试图通过优化调度来提高整体性能，但算法本身的复杂性决定了其开销不会小的。IV.通过多级队列和反馈机制，可以避免低优先级进程长时间得不到调度的问题。即使一个进程被降级到最低优先级的队列，它仍然有机会获得 CPU 资源，从而避免饥饿现象的发生。此外，在较低优先级队列中等待时间过长的进程会被移到更高优先级队列，这种形式的老化阻止饥饿的发生（极端情况下仍有可能发生饥饿）

70．A。【解析】 17ms, 34ms。（如果你算出了 18 和 27，说明你忽略了 10ms 时 P2 的时间片耗尽，误以为 P2 在 11ms 时才去 I/O，导致后续全盘算错。）：

- 0~ 5：P1 在 Q1 运行（剩 7），时间片耗尽，降入 Q2。
- 5~ 10：P2 在 Q1 运行。注意：P2 的第一个 CPU 持续时间是 6ms，但 Q1 的时间片只有 5ms！所以在 10ms 时，P2 时间片耗尽，剩余 1ms CPU 时间，降入 Q2。
- 10 时刻：Q1 空，调度 Q2 的队首 P1。

- 10~ 17：P1 在 Q2 运行（需 7ms，Q2 时间片 10ms 足够），P1 在 17ms 完成。
- 17~ 18：P2 在 Q2 运行剩余的 1ms，完成第一个 CPU 持续时间，发起 I/O 阻塞。
- 18~ 28：P2 进行 I/O（10ms），系统空闲。
- 28 时刻：P2 I/O 完成。根据规则“返回阻塞前所在的队列”，P2 阻塞前在 Q2，故返回 Q2
- 28~ 34：P2 在 Q2 运行最后的 6ms，P2 在 34ms 完成。



71．B。【解析】A 错误。进程从运行状态变为阻塞状态是由于发生了某个等待事件。

B 正确。进程从运行态到就绪态或从就绪态 到运行态都是由进程调度程序决定的，从阻塞态到就绪态则取决于该进程所等待的外部事件是否完成。而从运行态到阻塞态是由运行进程自身决定的，即运行进程因某个等待事件发生而 将自己阻塞。

C 错误。运行进程的优先级一定高于就绪队列中所有进程的优先级，但不一定比阻塞队列中进程的优先级高。

D 错误。如果短作业总是在长作业已经投入运行后到达，那么作业的平均周转时间就不一定最小。

72．C。【解析】一个正在 t = 0 ~ 10ms：P1 获得 CPU 并开始运行。虽然 P2 在 t = 5ms 到达且其所需的 20 ms 小于 P1 此时剩余的 40 ms，但由于系统仅在时钟中断（即 10 ms、20 ms 等时刻）触发调度，因此 P2 无法立即抢占，必须在就绪队列中等待。

t = 10ms：触发时钟中断。此时 P1 已运行 10 ms，剩余时间为 45 − 10 = 35ms；P2 剩余时间为 20 ms。因为 20ms < 35ms，系统发生抢占，P2 获得 CPU，P1 回到就绪队列。

t = 10 ~ 30ms：P2 运行 20 ms 并在 t = 30ms 时顺利结束。

t = 30ms：P1 重新被调度，运行其剩余的 35 ms。



73．C。【解析】At = 12ms 时，高优先级的 P1 请求 Mutex，但此时该锁被低优先级的 P3 持有，因此 P1 无法获得锁，被迫进入阻塞队列。此时就绪队列中剩下 P2 和 P3。由于 P2 的优先级（中）高于 P3（低），系统会调度 P2 运行。只要 P2 不退出或不阻塞，P3 就无法获得 CPU 来执

行临界区代码并释放 Mutex。导致的结果是：最高优先级的 P1 竟然在等待中优先级的 P2 执行完，这种现象就是优先级翻转。

74．C。【解析】

【解析】9:30 时，A、B、C 的响应比如下，其中 B 的响应比最高，执行 B。

作业	等待时间/min	执行时间/min	响应比
A	45	90	1.5
B	30	30	2
C	0	30	1

10:00 时，B 执行完，A 和 C 的响应比如下，其中 C 的响应比高，执行 C。

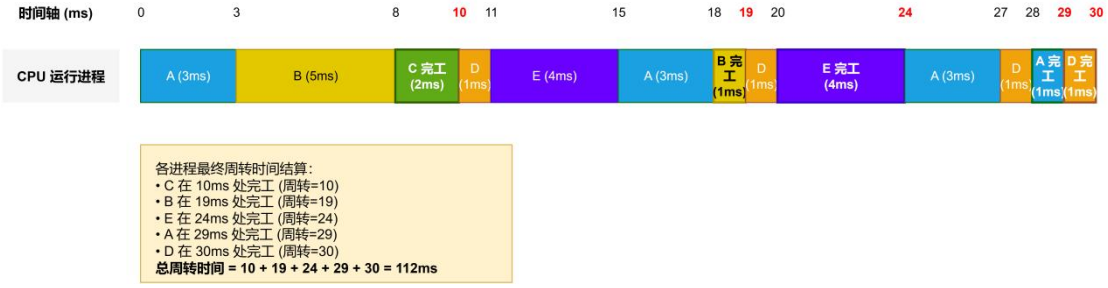
作业	等待时间/min	执行时间/min	响应比
A	75	90	1.83
C	30	30	2

10:30，执行 A。执行顺序为 B，C，A。答案为选项 C。

75．C。【解析】进程调度顺序为 A→B→C→D→E→A→B→D→E→A→D→A→D

各进程周转为：A-29，B-19，C-10，D-30，E-24

总周转时间为：112



76．C。【解析】带权周转时间是指作业周转时间与作业实际运行时间的比值：带权周转时间=作业周转时间/作业实际运行时间。平均带权周转时间是指多个作业带权周转时间的平均值。

由优先权可知，进程的执行顺序为 P2→P3→P1。P2 的周转时间为 1+15+24=40us；P3 的周转时间为 18+1+24+1+36=80us；P1 的周转时间为 30+1+24+1+36+1+12=105us。三个进程的带权周转时间分别为：P1：105/12=8.75us，P2：40/24≈1.67us，P3：80/36≈2.22us。平均带权周转时间为（8.75+1.67+2.22）/3≈4.21us

77．A。【解析】J1 进入内存时间为 10:00，运行结束时间为 11:10，周转时间 为 70 分钟。J2 进入内存时间为 10:20，运行结束时间为 10:50，周转时间为 30 分钟。J3 进入内存时间为 11:10，运行结束时间为 12:00，周转时间为 90 分钟。J4 进入内存时间为 10:50，运行结束时间为 12:20，周转时间为 90 分钟。平均周转时间为：(70+30+90+90)/4=70 分钟。



78．D。【解析】I正确，在 0~10s 的时段内，只有 3.5~4s 间 CPU 未运行程序，因此 CPU 的利用率为 9.5/10＝95%。

Ⅱ正确，进程 A 的周转时间为 3s-0s=3s，进程 D 的周转时间为 3.5s-0s=3.5s，进程 B 的周转时间为 10s-4s=6s，进程 C 的周转时间为 8s-6s=2s，因此平均周转时间为(3s+3.5s+6s+2s)/4=3.625s。

Ⅲ正确，进程 D 的等待时间为 3s，进程 B 的等待时间为 2s，进程 A、C 的等待时间都是 0s，因此进程 D 的等待时间最长。

Ⅳ正确，进程 A 的带权周转时间为 3/3=1s，进程 B 的带权周转时间为 6/4=1.5s，进程 C 的带权周转时间为 2/2=1s，进程 D 的带权周转时间为 3.5/0.5=7s，因此，进程 A、B、C 和 D 的平均带权周转时间为(1s+1.5s+1s+7s)/4=2.625s。

于是，选项I~IV 都正确。

79．B。【解析】时钟中断处理程序发现当前进程时间片用完后，通常只是向当前进程发出抢占请求（例如设置一个“需要调度（need_resched）”的内核标志位）。真正的进程调度和切换程序，往往需要等待中断完成准备返回用户态的边界时刻才会被安全执行，而不是在中断处理程序的上半部分“就地、强行”切换，否则极易引发内核栈混乱或死锁。

80．A。【解析】多级反馈队列调度算法涉及抢占和进程在各级队列间的移动。

周转时间之和为 10+1+2+12=25。

J1:0 时刻到达。0-2 时刻运行，2 时刻进入 2 级队列，7-10 时刻运行，周转时间为 10。

J2:2 时刻到达。2-3 时刻运行，周转时间为 1。

J3:3 时刻到达。3-5 时刻运行，周转时间为 2。

J4:4 时刻到达。5-7 时刻运行，7 时刻进入 2 级队列；10-14 时刻运行，14 时刻进入 3 级队列；14-16 时刻运行。周转时间为 12。



81．A。【解析】假设进程 M 先于进程 N 进入就绪队列。PM 和 PN 分别表示 M 和 N 的优先权数。

在β>α>0 设定下，在就绪队列中，PM>PN，原因是α>0，则越早进入就绪队列，优先数就越大，所以是 FCFS(First Come First Service)。又因为β>α，所以在 M 运行时，PM 增长速度大于 PN 的增长速度，则 PM>PN，从而保证了 M 进程先于 N 进程完成，即 FIFO(First In First Out)。

在α>β>0 设定下，还是 FCFS，原因跟β>α>0 一样。但由于α>β，所以在 M 运行时，无法保证 PM 仍然大于 PN，即无法保证 FIFO。

在β<α<0 设定下，在就绪队列中，PM<PN，原因是α<0，则越早进入就绪队列，优先数就越小，所以是 LCFS(Last Come First Service)。又因为β<α，所以在 N 运行时，PN 下降速度大于 PM 的下降速度，有可能出现 PM>PN 的情况，此时 CPU 有可能被 M 抢占，无法保证 LIFO(Last In First Out)。

在α<β<0 设定下，还是 LCFS，原因跟β<α<0 一样。但由于α<β，在 N 运行时，PN 的下降速度变慢了，从而保证了 PN 始终大于 PM，导致 N 进程先于 M 进程完成，即 LIFO。

所以本题的答案选 A。

82．B。【解析】多处理机调度为 2025 年大纲新增考点。本题命制依据 2026 王道操作系统 2.2.6 多处理机调度。

B 错误，软亲和指由进程调度程序尽量保证“亲和性”，硬亲和指由用户进程通过系统调用，主动要求操作系统分配固定的 CPU，确保“亲和性”。

A 正确，公共就绪队列：所有处理器从同一个队列中获取任务，当一个处理器完成其任务变为空闲时，它会去公共队列中领取新的任务。这种机制使得任务能够自动地在空闲处理器间分配，从而天然地实现了负载均衡。私有就绪队列：每个处理器拥有自己的就绪队列。当一个进程或线程在某个处理器上运行时，它通常会被放回该处理器的私有就绪队列。这有助于该任务下次被调度时仍在同一个处理器上运行，从而可以利用该处理器高速缓存中可能存在的该任务的数据，这就是所谓的处理器“亲和性”（Processor Affinity）。

C 正确，公共就绪队列是所有 CPU 共享的数据结构。当多个 CPU 同时尝试访问（例如，添加或移除进程/线程）这个共享队列时，如果不进行同步控制，可能会导致数据结构损坏或不一致。因此，必须使用锁机制（如自旋锁、互斥锁）来确保在任何时刻只有一个 CPU 能够修改公共就绪队列，从而保证操作的互斥性和原子性。

D 正确，私有就绪队列中平衡负载的方法通常有两种：推迁移和拉迁移。对于推迁移，一个特定的系统程序周期性检查每个 CPU 的负载，若发现不平衡，则从超载 CPU 的就绪队列中“推”一些进程到空闲 CPU 的就绪队列，从而平均分配负载。若一个 CPU 负载很低，则从超载 CPU 的就绪队列中“拉”一些进程到自己的就绪队列，发生拉迁移。在系统中，推迁移和拉迁移常被并行实现。

综上所述，选 B。

83．D。【解析】多处理机调度为 2025 年大纲新增考点。本题命制依据汤小丹计算机操作系统慕课版 10.5.3 进程（线程）调度方式。

A 正确，自调度是最简单的一种多处理机调度方式，其核心思想是设置一个所有处理器共享的公共

就绪队列。当一个处理器空闲时，它会访问这个公共队列，从中取出下一个就绪的进程或线程来执行。

B 正确，成组调度的核心思想是将一个应用程序中的多个线程（尤其是那些需要相互协作、频繁同步的线程）尽可能地同时分配到不同的处理器上并行运行。这样可以减少因等待协作线程而被阻塞的情况，从而提高应用程序的执行效率和并行度。

C 正确，动态分配允许一个进程或线程在不同次被调度时，或在其执行过程中，从一个处理器迁移到另一个处理器。这样做的好处是可以根据各个处理器的负载情况动态调整，实现更好的负载均衡。然而，如果一个线程频繁在不同处理器间切换，其先前在某个处理器高速缓存中的数据就会失效，导致缓存命中率下降，影响性能。

D 错误，专用处理器分配策略是指在一个应用程序执行期间，为该应用的每个线程都分配一个固定的处理器。这种方式确实可以避免线程切换的开销，提高单个应用程序的并行度。 但是，这会导致处理器的极大浪费，尤其是当线程因同步或 I/O 而阻塞时，其分配到的处理器会处于空闲状态。 同时，如果系统中就绪线程的总数超过了处理器的数量，那么为每个线程分配专用处理器是不现实的。此外，这种方式不一定能在所有情况下都获得最佳的*系统整体性能*，因为某些处理器可能会因为其分配的线程阻塞而空闲，而其他应用程序的线程可能在等待处理器。它更侧重于最大化单个高并行度应用的性能，而非系统整体的资源利用率或公平性。

> 关于 2025 年大纲新增考点“多处理机调度”，王道和汤小丹老师的教材侧重点不同，考生在复习时应熟悉两本书的考点以应对考察。

84．D。**【解析】**如下图所示，进程上下文包括进程标识信息、进程现场信息、进程控制信息、系统内核栈、用户堆栈、用户数据块、用户程序块以及进程的虚拟地址空间。PC、PSW 都和程序执行相关，属于进程现场信息，所以I、II、III、IV都属于进程上下文，答案为选项 D。



85．D。**【解析】**A 正确，上下文切换的核心目的就是在处理器上停止一个进程（或线程）的执行，并开始执行另一个进程（或线程）。这必然涉及到 CPU 执行环境的转换，包括指令序列、数据、寄存器状态等的改变。

B 正确，调度程序负责决定下一个要运行的进程（或线程），分派程序负责实际执行上下文切换的操作（保存旧上下文，加载新上下文）。这些都是操作系统的核心功能，涉及到对硬件状态的直接操作和对进程/线程控制块的访问，因此必须在具有特权的内核态执行。

C 正确，进程控制块（PCB）是操作系统用于描述和管理进程的数据结构，其中包含了进程的完整

上下文信息，如程序计数器、寄存器值、进程状态、内存管理信息等。 类似地，线程控制块（TCB）用于存储线程的独立执行上下文，如程序计数器、寄存器组、栈指针等。

D 错误，**进程切换**时，由于不同进程拥有独立的地址空间，因此确实需要保存和恢复包括程序计数器、程序状态字、所有通用寄存器以及与地址空间映射相关的信息（如页表基址寄存器等）。**同一进程内的线程切换**时，由于这些线程共享同一个进程的地址空间，因此**不需要**切换地址空间映射信息（如页表）。它们只需要保存和恢复各自独立的执行上下文，如程序计数器、寄存器组、栈指针和线程状态字等。线程切换的开销通常远小于进程切换，主要原因之一就是地址空间共享，无需切换内存管理信息。

86．C。【**解析**】将 PA 和 PB 进程分解为以下 6 程序段，这 6 程序段具有相对的 完整性，都可以作为一个单独的执行过程存在。

SA1：x:=1; x:=x+1;

SA2：x:=x+y;

SA3：z:=x+z;

SB1：y:=1; y:=y+3;

SB2：z:=y+1;

SB3：y:=y+z;

其中 SA1 与 SB1 可以并发执行，SA2 与 SB2 可以并发执行，SA3 与 SB3 不能并发执行。因此 SA3 与 SB3 的执行顺序 不同，可能会有不同的结果。如果先执行 SA3，则 y=15；如果先执行 SB3，则 y=9。

87．A。【**解析**】按照 1，2，3，4，5，6 的顺序执行，可以得到 z=0，c=9 的结果，按照 1，5，2，6，3，7，4 的顺序执行，可以得到 z=-3，c=9 的结果，按照 5，1，6，2，7，3，4 的顺序执行，可以得到 z=0，c=0 的结果。因此 z=0 或-3，c=9 或 0。故本题选 A。

```
P1() {
       x=0;     //(1)
       y =-1;   //(2)
       z = x*y; //(3)
       print z; //(4)
       }
P2() {
       x= 3;    //(5)
       C = x*x; //(6)
       print c; //(7)
       }
```

88．A。【**解析**】A 正确，访问不同临界区的两个进程，不要求必须互斥地进入临界区。

B、D 错误，临界区是访问临界资源的那一段程序，不是包含临界资源的一段数据区。

C 错误，临界区不是用于进程同步的机制。

89．B。【**解析**】当进程进入阻塞态，它就需要交出 CPU 的使用权，即允许其他进程竞争 CPU，C 和 D 错误;但是它不会放弃其他已占有的资源，因为进程一旦占有一个资源(即一旦它的资源请求被批

准)，那么该资源就不能被夺走，直到该进程主动释放它，所以其他进程不能进入该进程的临界区，A 错误，故本题选 B。

90．B。【解析】I打印机是临界资源。打印机在同一时刻只能为一个进程服务，如果多个进程同时向打印机输出内容，会导致输出结果混杂不堪。因此，打印机需要互斥访问。

Ⅱ对于各进程的非共享数据，在正常情况下，它不应该被其他进程访问，操作系统会提供保护。因此对于多个进程之间的同步与互斥，各进程的非共享数据不是临界资源。

Ⅲ磁盘不是典型的临界资源（指磁盘本身作为存储介质）。磁盘允许多个进程同时对其进行读写操作（宏观上），操作系统通过磁盘调度算法和文件系统来管理对磁盘的并发访问，确保数据读写的正确性和一致性。虽然对磁盘上某个特定文件或数据块的访问可能需要互斥（由文件系统或数据库管理系统保证），但磁盘设备本身作为整体是可共享的。

Ⅳ共享缓冲区是临界资源。共享缓冲区是多个进程（如生产者和消费者）之间交换数据的公共区域。当一个进程正在向缓冲区写入数据或从中读取数据时，其他进程对该缓冲区的访问（特别是修改指针或数据本身的操作）必须受到限制，以防止数据覆盖、读取错误或指针混乱等问题。因此，共享缓冲区是典型的临界资源，需要互斥访问。

91．B。【解析】为禁止两个进程同时进入临界区，同步机制应遵循以下准则：

1）空闲让进。临界区空闲时，可以允许一个请求进入临界区的进程立即进入临界区。

2）忙则等待。当已有进程进入临界区时，其他试图进入临界区的进程必须等待。

3）有限等待。对请求访问的进程，应保证能在有限时间内进入临界区，防止进程无限等待

4）让权等待(原则上应该遵循，但非必须)。当进程不能进入临界区时，应立即释放处理器防止进程忙等待。

故选 B 不能进入临界区的执行态进程立即放弃 CPU（让权等待）。

92．C。【解析】x 值为 2，就是加 1 成功，减 1 失败。因此只要在执行③和⑥之前，已经将数据 x 的值取到 R1 和 R2 中，并且，③操作在⑥操作之后即可。II，III，IV都满足条件，答案选 C。

93．C。【解析】本题考察单标志法的缺点，若 Thread1 没有将 turn 置为 2，则线程 2 会一直等待。因此选 C。

94．B。【解析】假设 P0 执行完语句 ①（turn = 1），此时发生进程切换；P1 接着执行语句 ③（turn = 0）和语句 ④（flag[1] = TRUE）。接着 P1 执行到 while(flag[0] && turn == 0)。由于此时 P0 还没执行语句 ②，flag[0] 依然是 FALSE！因此 P1 顺利跳出循环，进入临界区。此时若切换回 P0，P0 执行语句 ②（flag[0] = TRUE），然后检查 while(flag[1] && turn == 1)。由于 turn 已经被 P1 改成了 0，turn == 1 为假，P0 也跳出循环，进入临界区。两个进程同时在临界区，违背“忙则等待”。

95．A。【解析】以上算法是双标志先检查法，该算法相比于单标志法，满足了空闲让进原则，但是由于进程 P0 可能会在判断 while 语句满足后切换到 P1，P1 也判断 while 满足后，两个进程就会同时访问临界区，因此该算法既不能保证进程互斥进入临界区，也违背了忙则等待原则，故本题选择 A。

96．D。【解析】A 正确，TestAndSet 指令的核心功能是原子地（不可中断地）读取一个内存字

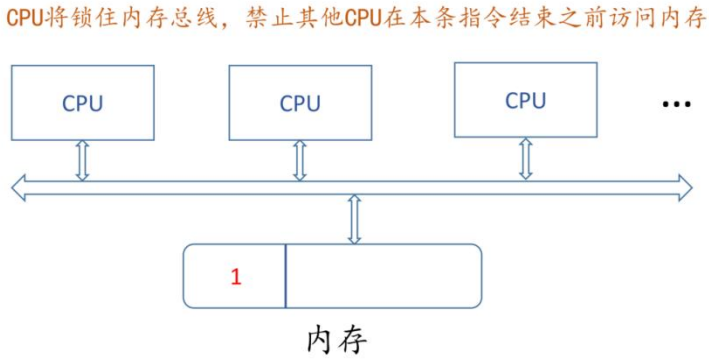
（通常是锁变量）的旧值，并同时设置该内存字为一个新的预定值（通常表示“已加锁”，如 TRUE 或 1）。返回的是读取到的旧值。这个原子性保证了在多处理器环境下，即使多个处理器同时尝试对同一个锁变量执行 TSL，也只有一个能成功获取锁（即读取到锁变量为 FALSE 时的旧值）。

B 正确，Swap（或 XCHG）指令原子地交换一个 CPU 寄存器的内容和内存中锁变量的内容。进程通常会将一个表示“尝试加锁”的值（如 TRUE）放入寄存器，然后与锁变量进行交换。如果交换后寄存器中的值是表示“锁空闲”（如 FALSE）的值，则表示该进程成功获得了锁。

C 正确，当使用 TSL 指令实现自旋锁时，如果一个进程执行 TSL 指令后发现锁已被占用（例如，TSL 返回 TRUE），该进程会继续循环执行 TSL 指令，不断尝试获取锁，直到成功为止。这种在等待锁的过程中 CPU 持续执行测试指令而不放弃 CPU 的做法，被称为“忙等”（busy-waiting）。

D 错误，Swap 指令本身也是一种硬件原子操作，用于实现互斥的基础。像 TSL 指令一样，如果直接用 Swap 指令在一个循环中不断尝试获取锁（即“自旋”），那么当锁被占用时，进程同样会处于“忙等”状态，不断消耗 CPU 时间，而不释放 CPU。因此，单纯使用 Swap 指令实现的互斥锁并不能满足“让权等待”准则。

97．C。**【解析】**首先，我们先了解一下什么是互斥锁，其实它就是一个普通的变量，可以是 bool 类型也可以是 int 类型（用 0 表示锁释放，1 表示已上锁），上面这种实现出来的互斥锁，它是支持多临界区的互斥访问，什么意思呢，如果我们有好多个临界资源需要互斥访问，我们可以对它们分别实现一个互斥锁，即每把锁一个变量即可；注意，这个互斥锁变量，它是位于主存里的共享变量，不是某个进程私有的，否则实现不了一个进程拿到锁，别的进程进不去这种效果，因此我们说使用原子操作指令实现的互斥锁适用于单处理器或共享内存多处理器中任意数量的进程同步，因此 A 选项正确。



另外，我们要注意拿到锁和释放锁的必须是同一个进程，否则可能会出现 A 进程拿到锁之后，但是 B 进程却释放了这把锁，这时候 C 获取了这把锁，这样就不满足互斥进入临界区这个规则了，因此在拿到锁和释放锁的时候，需要验证一下身份，例如当某个锁是空闲的时候，A 拿到了这把锁，这时候可以把 A 的进程 pid 记下来，之后释放锁的时候，验证一下 pid 是否相等，是的话才可以释放锁，这样就可以保证互斥访问临界区了，因此 D 正确；

BC 选项的话，我们可以根据这两把锁的实现方式来看，**自旋锁**在拿不到锁的时候，并没有及时释放处理机，而是一直循环尝试是否可以获取锁，因此**不符合让权等待**，而阻塞锁在拿不到锁的时候，就将该进程加入阻塞队列里面，因此，**阻塞锁可以实现让权等待**，B 选项正确。此外由于自旋锁具有随机性，没有队列这种数据结构保证进程按照先来后到的顺序使用 CPU 资源，因此顺序可能会乱，另外

如果当某个进程离开临界区时有多个进程等待进入临界区，就有可能导致某些进程由于每次临界区空闲的时候没有上处理机，或者轮到上处理机的时候，临界区有锁，从而导致饥饿，因此 C 选项错误。

扩展一下，以上这种使用原子操作实现的互斥锁还有可能导致**死锁**的发生，比如某个低优先级进程拿到了锁，在使用这把锁的时候，来了一个高优先级的进程获得处理机，并且它也想进入临界区，这时就发生了死锁。

98．A。【解析】允许 m 个进程同时进入互斥段，说明初始值为 m，当进入 m 后，剩下 n-m 个等待进入，因此下限是-（n-m）。因此范围是-(n+m，m)。

99．D。【解析】由下列代码，value--后，if 判断当前 value 的值为进程分配之后剩余的资源数，由这个逻辑，S.value<0 才会阻塞进程。

```
P(s){
        s.value--;
        if (s.value < 0) {
                add this process to list s.L;
                block;
        }
}
```

100．A。【解析】执行 36 次 P 操作和 20 次 V 操作后，信号量 S 的值为 0，说明信号量 S 的初始值为 16。对初始值为 16 的信号量 S 执行 19 次 P 操作和 1 次 V 操作后，计数信号量 S 的值为 16-19+1=-2。如果信号量的值是负值，那么它的绝对值等于等待进入临界区的进程数，因此有 2 个进程在信号量 S 的等待队列中等待，故本题选 A。

101．C。【解析】P、V 操作具体如下：

```
P(){
    S.value--;
    if(S.value<0){
        add this process to S.L;
        block(S.L);
    }
}
V(){
    S.value++;
    if(S.value<=0){
        remove a process from S.L;
        wakeup(P);
    }
}
```

执行 P 操作时，当 S.value<0 时，阻塞当前进程；执行 V 操作时，当 S.value<=0 时，唤醒一个阻塞队列进程。选 C。

102．A。【解析】I正确，管程通过其互斥进入的机制，保证了同一时刻只有一个进程可以在管程

内部执行其定义的过程。这意味着所有对管程内部共享数据结构的访问都是互斥的，程序员无需再显式地使用互斥锁等低级同步原语来实现对这些共享资源的互斥。

II 正确，管程是一种程序设计语言级别的同步构造，它将共享数据以及对这些数据的操作封装起来，提供了比信号量更高层次的抽象。程序员使用管程时，不必关心底层的互斥和同步细节，使得并发程序的设计更为简单和不易出错。

III 错误，当一个进程在管程中调用条件变量的 x.wait()操作时，该进程会被阻塞，并被放入与条件变量 x 关联的等待队列中。同时该进程会释放对管程的互斥访问权，以便其他进程可以进入管程执行。这避免了忙等待，并允许其他进程改变条件。

IV 错误，条件变量的 x.signal()操作的作用是唤醒一个（通常是等待队列中的第一个）因条件 x 而阻塞的进程。如果没有进程正在等待条件 x，则 x.signal()操作通常不起任何作用（或被忽略），它不会像信号量的 V 操作那样改变某个内部计数器的值。 信号量是有状态的（其值会增减），而条件变量本身是无状态的，它仅作为进程排队和唤醒的机制。

此处补充说明：在王道操作系统中，给出了条件变量的定义和使用：

```
monitor Demo{
    共享数据结构 S;
    condition x;        //定义一个条件变量 x
    init_code(){...}        //此处不是对 x 进行初始化，而是对
    take_away(){
        if(S<=0)
            x.wait();        //资源不够，在条件变量 x 上阻塞等待
        资源足够，分配资源，做一系列相应处理;
    }
    give_back(){
        归还资源，做一系列相应处理;
        if(有进程在等待)
            x.signal();//唤醒一个阻塞进程
    }
}
```

其中指出，当有进程在等待时才会执行 x.signal()，此时执行 x.signal()必定会唤醒阻塞进程，这与汤书中不同，汤书中对 x.signal 定义如下：

> **x.signal**：如果正在调用管程的进程发现x条件发生了变化，则调用x.signal，重新启动一个因x条件而阻塞或挂起的进程，如果存在多个这样的进程，则选择其中的一个；如果没有，则继续执行原进程，而不产生任何结果。这与信号量机制中的signal操作不同，因为后者总是要执行s=s+1操作，所以总会改变信号量的状态。

且汤书后文 4.6.1 生产者—消费者问题中 3.利用管程解决生产者—消费者问题时给出的实例也未有任何判断是否有进程在等待才执行 x.signal()的语句。本题命制时，采用的汤书给出的说法。

103．C。【解析】A 错误： 条件变量没有值，它仅仅是一个等待队列的头部指针。而信号量内部包含一个表示资源数目的整型变量。

B 错误： 条件变量没有记忆功能（无状态）。如果当前没有进程在 x 上等待，执行 x.signal() 什么都不会发生（信号丢失）。而信号量的 signal()（V 操作）会使信号量的值加 1 并保留。

C 正确： 管程是由语言/编译器级别支持的进程同步机制。其最大的优点就是由编译器负责管理管程的互斥访问。程序员不需要像写 PV 操作那样手动加锁，编译器会确保任何时候只能有一个进程在管程内执行，实现了自动互斥。

D 错误： 当管程内的进程因为缺少某种资源而执行 x.wait() 进入阻塞状态时，它必须立即释放管程的互斥访问权。只有这样，其他进程才能进入管程去改变条件，并最终调用 x.signal() 唤醒它。如果它不释放互斥权，整个管程就会被它死死锁住，导致其他进程无法进入，从而引发死锁。

104．C。【解析】C 错误，管程一次只允许一个进程进入，这是由管程的互斥性决定的。

A 正确，管程包含了面向对象的思想，将表达共享资源的数据结构和对其进行操作的进程封装在一个对象中，并封装了同步操作，从而对进程隐藏同步的细节，简化了调用同步功能的接口

B 正确，管程机制可以便于集中管理分散于不同进程的临界区

D 正确，管程是进程同步工具，避免了信号量机制中大量且分散的同步操作

105．C。【解析】A 正确，如果多个进程运行在不同的 CPU 上，每个进程都试图进入一个临界区，即使禁止中断，其他进程依旧能够在其他处理器上进入临界区。

B 正确，软件方法与原子操作均属于忙等机制，无法实现让权等待

C 错误，首先要明晰：自旋锁≠互斥锁，王道操作系统将两者混为一谈，汤书中明确指出（汤小丹计算机操作系统慕课版 4.7Linux 进程同步机制）两者的区别。自旋锁是一种忙等机制，无法实现让权等待，而互斥锁就是一个简化版的信号量，互斥锁加锁失败后，线程释放 CPU，给其他线程，可以实现让权等待

D 正确，这是条件变量与信号量的重要区别。二者的区别有：①条件变量是“没有值”的，仅实现了“排队等待”的功能；而信号量是有值的，信号量的值反映了剩余资源数，而在管程中，剩余资源数用共享数据结构记录。②条件变量的 wait/signal 操作与信号量不同，如果在管程中的一个进程发信号，但没有在这个条件变量上等待的任务，则丢弃这个信号。

106．B。【解析】M≥K×(N−1)+1（M 为资源总数，K 为进程数，N 为单进程最大需求）

推导：12≥K×(3−1)+1⇒2K≤11⇒K≤5.5 取最大整数得 K=5。

K=5，Maxi=3，∑Allocation=9，Available=3。

系统安全 ⟺存在至少一个进程 Pi 满足 Needi≤Available。

计算还需资源：Needi=Maxi−Allocationi=3−Allocationi。由于 Allocationi≥0，故 Needi≤3 恒成立。

当前 Available=3，因此 所有进程的 Needi≤Available 均成立。任意一个进程都可被调度执行，执行后释放资源，Available 增大，后续进程依次可运行。

结论：系统一定处于安全状态。由于多个进程同时满足运行条件，执行起点不唯一，故安全序列不唯一。选 B。

107．C。【解析】A 错误。当进程数大于资源数时，如果资源分配得当则进程竞争资源不会产生死锁。

B 错误。系统出现死锁是指系统内一些进程 被锁 而不 一定 是全部进程被锁，即这部分进程因相互等待对方资源而处于死锁状态。

C 正确。处于死锁状态的进程都在相互等待对方占用的资源，因此处于死锁状态的进程至少有两个。如果系统的全部进程都因相互等待对方的资源而处于死锁状态，就会使整个系统处于瘫痪状态。

D 错误。银行家算法是避免死锁的一种方法。

108．A。【解析】极端法：求出每个进程都发生死锁需要的最大资源数，在此基础上，再增加一个资源，则死锁一定不会发生。当 n 个进程共享 m 个互斥资源时，每个进程的最大需求是 w。则当 m ≥n x (w-1)+1 时，系统一定不发生死锁，显然 B、C、D 对，A 错。

109．B。【解析】B 错误：死锁避免（银行家算法）对用户申请资源的限制确实比死锁预防（如顺序分配、静态分配）更少，因此它的资源利用率更高。但是，在系统运行时的时间开销上，死锁避免要大得多！因为死锁预防依靠的是静态的协议规则（编译或资源编号阶段就确定了），运行时操作系统不需要做额外算力推演；而死锁避免（银行家算法）在每一次进程发出资源请求时，操作系统都必须运行一次安全性检测算法。因此，死锁避免是以“高运行时开销”去换取“高资源利用率”。选项中“运行时时间开销更小”的表述完全颠倒了因果。

A 正确：静态分配要求进程运行前一次性申请全部资源，未满足时不投入运行，这完美破坏了“请求和保持”条件，但由于很多资源在运行后期才用得上，导致早期闲置，利用率极低。

C 正确：资源按序分配规定进程只能严格按照编号递增的顺序申请资源，这使得资源分配图中绝对不可能形成闭环，从而破坏了“循环等待”条件。

D 正确：必要性：如果系统发生了死锁，必然存在一组进程在循环等待资源，因此资源分配图中一定存在环路。

充要性（单实例情况）：如果每种资源类中只有 1 个实例（即图中的资源节点往外发射的箭头只有一根），那么一旦出现环路，环路上的每个进程都占有了 1 个资源且都在等待下一个资源，由于没有多余的实例可以打破僵局，此时有环必死锁。因此，在单实例资源图中，有环是死锁的充要条件。（注：若为多实例资源，有环不一定死锁）。

110．D。【解析】A 正确，死锁是多个进程（至少两个）之间形成循环等待资源的状态，导致这些进程都无法继续。饥饿则可能发生在单个进程上，例如一个低优先级进程由于不断有高优先级进程到达而长时间得不到 CPU。

B 正确，死锁是一种“僵持”状态，除非操作系统采取措施（如撤销进程、剥夺资源），否则死锁进程将永久等待。饥饿状态下，进程虽然长时间等待，但并非绝对没有机会，例如当高优先级进程执行完毕，或者调度策略发生变化（如动态优先级调整）时，饥饿的进程仍可能获得资源。

C 正确，死锁的产生需要满足互斥、请求和保持、不可剥夺、循环等待这四个必要条件，其中资源不足是前提，而进程推进顺序不当是形成循环等待的关键。饥饿则更多地与调度策略相关，如固定的低优先级可能导致进程持续得不到 CPU，或者某些资源分配算法可能使得某些进程的请求总是被忽略。

D 错误，发生饥饿的进程可能处于就绪态(长期得不到 CPU，如 SJF 算法的问题)，也可能处于阻塞态(如长期得不到所需的 IO 设备，如上述举例)；而发生死锁的进程必定处于阻塞态。

111．C。【解析】A 正确，每个进程只能拥有一个资源，运行完毕后释放资源，不会产生死锁。

B 正确，死锁产生的必要条件之一是互斥条件，即资源在一段时间内只能被一个进程所占用。如果所有资源都可以被多个进程无冲突地共享访问，那么互斥条件就不成立，死锁也就不会发生。

C 错误，仅仅有优先级并不能完全避免死锁。例如，一个高优先级进程 P_high 可能等待一个低优先级进程 P_low 持有的资源，而 P_low 又在等待另一个资源。如果这个等待链形成循环，即使有优先级，死锁依然可能发生。优先级调度可能会影响饥饿现象，但不能直接消除所有类型的死锁。

D 正确，循环等待条件是产生死锁的四个必要条件之一。如果能够破坏循环等待条件，例如通过资源有序分配法，确保进程请求资源时不会形成环路，那么死锁就不会发生。

112．D。【解析】这破坏了循环等待条件。如果存在所有左边的哲学家等待右边哲学家放下筷子的循环等待，则每个哲学家肯定已获得左边的筷子，但还没有获得右边的筷子，这与存在右撇子的情况不符。同理，亦不存在相反的循环等待链，而且，因不相邻的哲学家之间不存在竞争资源关系，所以也不可能存在 5 个以下的哲学家的循环等待链。

113．D。【解析】循环等待条件指系统发生死锁时，一定存在一条进程请求资源的循环等待链。题干中对所有资源进行编号，并要求进程只按严格增加(或减少)的顺序来请求资源，破坏了死锁产生的循环等待条件，选 D。

114．B。【解析】请求和保持条件指进程已经占用至少一个资源，但又提出了新的资源请求，而该资源已被其他进程占用，此时进程请求阻塞，且不会释放已占用的资源。为所有进程一次性分配其所需的资源破坏了死锁产生的请求和保持条件，选 B。

115．B。【解析】死锁预防是破坏死锁发生的必要条件，死锁避免是避免系统进入不安全状态从而防止死锁的发生，所以 B 正确，A.C 错误。银行家算法不会规定进程申请资源的顺序，只会决定是否给进程分配其申请的资源，D 错误。

116．B。【解析】III、IV错误。

Ⅰ正确，安全状态的定义是，系统能按某种进程推进顺序（存在一个安全序列），为每个进程分配其所需的资源，直至满足每个进程对资源的最大需求，使每个进程都可顺利完成。如果系统处于安全状态，就一定可以找到至少一个这样的序列，从而避免了死锁的发生。

Ⅱ正确，如果系统发生了死锁，意味着存在一组进程循环等待资源，它们都无法继续执行，也无法释放资源。在这种情况下，不可能找到一个安全序列，使得所有进程都能顺利完成。因此，死锁状态一定是不安全状态。

III错误，不安全状态是指系统不一定能找到一个安全序列。它意味着系统存在进入死锁状态的可能性，但并非一定会发生死锁。在不安全状态下，如果进程后续的资源请求和释放顺序恰当，仍然有可能避免死锁的发生。

III错误，系统当前未发生死锁，并不意味着它一定处于安全状态。系统可能正处于一个不安全状态，只是尚未满足死锁发生的全部条件，或者进程的资源请求顺序尚未导致死锁。不安全状态是死锁的可能前兆，但当前不死锁不代表未来不会死锁，也不代表当前状态是安全的。

117．C。【解析】当前已分配资源总数为：(0，4，2)+(3，0，1)+(1，0，2)+(2，3，0)=(6，7，

5)，剩余可分配资源数为：(7，7，7)-(6，7，5)=(1，0，2)

各进程对资源的需求量：

P0：(2，4，3)-(0，4，2)=(2，0，1)

P1：(5，1，2)-(3，0，1)=(2，1，1)

P2：(2，0，3)-(1，0、2)=(1，0，1)

P3：(5，3，5)-(2，3，0)=(3，0，5)

因此初始时只有进程 P2 可满足需求，排除选项 A 和 B。给进程 P2 分配资源，则 P2 完成后剩余可分配资源：(1，0，2)+(1，0，2)=(2，0，4)。接下来只有进程 P0 可满足需求，只有选项 C 满足。给进程 P0 分配资源，则 P0 完成后剩余可分配资源：(2，0，4)+（0，4，2）=（2，4，6），然后依次分配进程 P1、P3，答案为选项 C。

118．C。【**解析**】假设每个进程申请三种资源为随机的，申请的顺序可能性一样；此时，每个进程申请顺序种类为 3！，即 6 种。两者 结合则有 6×6=36 种可能的排列。不难看出，只要是两个进程第一次申请的资源相同，则不会发生死锁，即其中一个进程得到第一个资源，另外一个进程因为等待此记录而阻塞，不会申请其他资源，确保所有资源都可由第一个进程读完后再释放第二个进程。两个进程第一次同时申请资源 1 的排列有 4 种。

A：1，2，3；B：1，2，3

A：1，2，3；B：1，3，2

A：1，3，2；B：1，2，3

A：1，3，2；B：1，3，2

同理，两个进程第一次同时申请资源 2 的排列为 4 种，同时申请资源 3 的排列也为 4 种，也就是共有 12 种排列是不会发生死锁的，故系统保证不发生死锁的概率为 12/36=1/3。

119．C。【**解析**】当前系统剩余资源数量为(3，3，2)，满足 P1 和 P3 的资源需求。分配给 P1，P1 释放资源后，系统的剩余资源数量为(5，4，2)，满足 P3 的资源需求，分配给 P3，P3 释放资源后，系统剩余资源数量为(7，5，3)，满足 P0 的资源需求，分配给 P0，P0 释放资源后，系统剩余资源数量为(7，6，3)，满足 P2 的资源需求，分配给 P2，P2 释放资源后，系统剩余资源数量为(10，6，5)，满足 P4 的资源需求，分配给 P4，运行结束，系统没有进入不安全状态，可能的安全序列为<P1，P3，P0，P2，P4>，选 C。计算 Need 矩阵：

$$\begin{matrix}\text{Max}\end{matrix}\ -\ \begin{matrix}\text{Allocation}\end{matrix}\ =\ \begin{matrix}\text{Need}\end{matrix}$$

$$\begin{bmatrix}7&5&3\\3&2&2\\9&0&2\\2&2&2\\4&3&3\end{bmatrix}-\begin{bmatrix}0&1&0\\2&1&0\\3&0&2\\2&1&1\\0&0&2\end{bmatrix}=\begin{bmatrix}7&4&3\\1&1&2\\6&0&0\\0&1&1\\4&3&1\end{bmatrix}$$

120．D。【解析】首先计算需求矩阵

$$\begin{matrix}\text{Max}\end{matrix}\ -\ \begin{matrix}\text{Allocation}\end{matrix}\ =\ \begin{matrix}\text{Need}\end{matrix}$$

$$\begin{bmatrix}2&2&4\\3&2&1\\4&5&2\\3&2&4\end{bmatrix}-\begin{bmatrix}1&1&2\\3&1&0\\2&2&1\\3&1&4\end{bmatrix}=\begin{bmatrix}1&1&2\\0&1&1\\2&3&1\\0&1&0\end{bmatrix}$$

然后计算系统剩余资源数：(10，6，7)-(9，5，7)=(1，1，0)，显然满足进程 P4 的资源需求，分配给进程 P4，P4 释放资源后，系统剩余资源数为 (4，2，4)，满足进程 P1 和 P2 的资源需求，此时：

① 若先分配给 P1，P1 释放资源后，系统剩余资源数为(5，3，6)，满足进程 P2 和 P3 的全部需求，因此存在安全序列<P4，P1，P2，P3>、<P4，P1，P3，P2>。

② 若先分配给 P2，P2 释放资源后，系统剩余资源数为(7，3，4)，满足进程 P1 和 P3 的全部需求，因此存在安全序列<P4，P2，P1，P3>、<P4，P2，P3，P1>。

综上，存在四条序列，系统是安全的，故选择 D 项。

121．C。**【解析】**依次用 P1~P4 来表示 4 个进程。从矩阵可以看出，4 个进程还需要的资源数目为 2、1、6、5，按所需资源数目从小到大排列，即 P2、P1、P4、P3，这就是所需最小资源数目的执行顺序。

设有 x 个可用资源。

当 x≥1 时，P2 可以执行完成，并释放占用资源，此时资源数为 x+1。

当 x+1≥2 时，P1 可以执行完成，并释放占用资源，此时资源数为 x+2。

当 x+2≥5 时，P4 可以执行完成，并释放占用资源，此时资源数为 x+4。

当 x+4≥6 时，P3 可以执行完成，并释放占用资源，此时资源数为(忽略)。

剩下的，就是求解这个简单的方程组，得出 x>=3。

122．A。**【解析】**同一时间请求所有资源破坏了死锁产生的请求和保持条件，属于死锁预防的策略，A 错误；死锁避免通过对资源分配的风险评估来决定是否为进程分配资源，需要收集进程运行所需资源总量信息，而死锁检测允许死锁的发生，会给可能导致死锁的进程分配资源，B、C、D 正确。

123．B。**【解析】**资源状态：R1 实例全分配（给 P2, P4），空闲 0；R2 分配 1 给 P3，空闲 1。

找非阻塞进程：请求边对应资源空闲数 ≥ 请求量。

P1 请求 R1（需 1，空闲 0）→ 阻塞

P2 请求 R2（需 1，空闲 1）→ 非阻塞

P3 请求 R1（需 1，空闲 0）→ 阻塞

P4 请求 R2（需 1，空闲 1）→ 非阻塞

图简化：

选 P2 简化：移除 $P_2 \rightarrow R_2$ 和 $R_1 \rightarrow P_2$，P2 释放持有的 R1。R1 空闲变为 1。

选 P4 简化：移除 $P_4 \rightarrow R_2$ 和 $R_1 \rightarrow P_4$，P4 释放持有的 R1。R1 空闲变为 2。

此时 P1 请求 R1（需 1≤ 空闲 2）→ 可简化；P3 请求 R1（需 1≤ 空闲 2）→ 可简化。

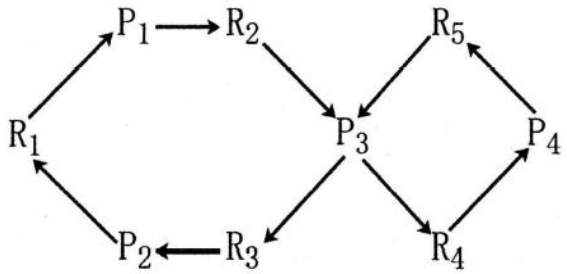
图可完全简化，无死锁。起点 P2/P4 均可。选 B。

124．B。**【解析】**根据死锁定理，首先需要找出既不阻塞又非孤点的进程。对于图I，由于资源 R2 已分配了 2 个，还剩一个空闲的 R2，可以满足进程 P2 的需求。P2 运行结束后，释放一个资源 R1 和两个资源 R2，可以满足 P1 进程的需求，从而系统的资源分配图可以完全简化，不处于死锁状态。对于图II，P1 需要资源 R1，但是唯一一个资源 R1，已分配给 P2；同理，P2 需要资源 R4，而 R4 资源

也只有一个且已经分配给了 P3；而 P3 还需要一个 R2 资源，但是两个 R2 资源都已经分配完毕了，所以 P1、P2、P3 都处于阻塞状态，系统中不存在既不阻塞又不是孤点的进程，所以系统工处于死锁状态。

注意：在进程资源图中，P->R 表示进程正在请求资源，R->P 表示资源已被分配给进程(资源只能是被动的)。

125．C。【解析】画出系统当前的资源分配图，如果资源分配图包含了环路，且每种资源只有一个实例，那么系统中存在一个死锁。所以 A、D 错误；若终止进程 P1，释放其占有的 R1，P3、P4 仍然存在回路，所以 B 错误。因此选 C。



126．D。【解析】A 正确，终止所有死锁进程是一种简单粗暴但能有效解除死锁的方法。它的缺点是代价较大，因为所有死锁进程的工作都会丢失，包括那些可能已经运行了很长时间或接近完成的进程。

B 正确，进程回退是一种尝试恢复到死锁发生前某个安全状态的方法。系统需要周期性地为进程设置检查点，记录进程的状态和资源分配情况。当死锁发生时，可以将某些进程回退到最近的检查点，释放自该检查点以来获得的资源，从而可能打破死锁。这需要系统支持检查点机制。

C 正确，剥夺资源是指从某些死锁进程中强行收回其已分配的资源，并将这些资源分配给其他死锁进程，以打破循环等待。选择剥夺哪个进程的哪些资源以最小化代价是一个复杂的问题，需要考虑进程优先级、已执行时间、已占用资源类型和数量等因素。

D 错误，付出代价的衡量标准是多方面的，例如，该进程已运行的时间、已使用的资源、完成剩余工作所需的时间、进程的重要性（可能与优先级不完全一致）、该进程是交互式还是批处理式等。一个优先级最低的进程可能已经快要完成其任务，或者持有很多关键资源，终止它可能比终止一个优先级稍高但刚开始运行的进程代价更大。因此，选择代价最小的终止进程是一个复杂的决策过程，并非简单地看优先级。

127．C。【解析】n 个进程，分别需要 N1, N2, ...Nn 个资源，那么总的所需资源$sum = N1 + N2 + \ldots + Nn = \sum_{i=1}^{n} Ni$，现在可用资源是 m 个，**最坏的情况是**每个进程都分配了 Ni-1 个，也就是再分配一个就可以分配完毕，使用完归还。此时每个进程都差 1 个的话，那么当前总的已经分配的就是$\sum_{i=1}^{n}(Ni-1) = \sum_{i=1}^{n} Ni - n = sum - n$。现在只需要再多出 1 个就能保证任意情况下都不会发生死锁，那么$m >= sum - n + 1$，整理可得$sum <= n + m - 1$，也就是选 C。

128．B。【解析】III、IV正确。

Ⅰ错误，资源的有序分配策略（即对资源类型进行线性排序，进程必须按序号递增的顺序申请资

源）主要是为了破坏循环等待条件。 它并不直接作用于请求和保持条件。破坏请求和保持条件通常采用一次性申请所有资源或在得不到新资源时释放已持有资源等策略。

Ⅱ错误，银行家算法是一种死锁避免算法。它通过在每次资源分配前进行安全性检查，确保系统不会进入不安全状态，从而避免发生死锁。 它并不直接破坏死锁的四个必要条件中的任何一个，而是通过动态地控制资源分配来避免形成可能导致死锁的局面。

Ⅲ正确，死锁的发生首先是因为系统中的资源数量有限，不足以满足所有并发进程的需求（资源分配不足）。其次，即使资源不足，如果进程请求和释放资源的顺序得当，也未必会发生死锁。只有当多个进程对资源的请求和保持形成一种特定的循环等待关系，即进程推进顺序非法（导致了循环等待条件），才会最终导致死锁。

Ⅳ正确，进程回退是一种死锁解除策略。当检测到死锁后，系统选择一个或多个死锁进程，使其回退到足以避免死锁的某个先前状态（检查点或还原点）。这要求系统能够记录进程在不同时间点的状态信息（历史信息）并设置还原点，以便能够进行回退操作。

129．D。**【解析】**A 选项正确：进程在某个核上运行久了，该核的 Cache 里存满了该进程的数据。频繁换核会导致 Cache 频繁失效，因此需要“处理器亲和性”。

B 选项正确：负载均衡要求把进程移到闲置的核上，而亲和性要求留在原核，二者在实际调度中需要进行折中权衡。

C 选项正确：多核同时去抢同一个全局队列的锁，会导致严重的互斥锁冲突，因此现代多核系统（如 Linux）通常为每个 CPU 核心配置独立的私有就绪队列。

D 选项错误：多核系统同样大量使用抢占式调度。死锁与抢占没有必然的因果关系，相反，允许强行剥夺资源（抢占）反而是解除死锁的有效手段之一。

130．C。**【解析】**关中断”属于特权指令，必须在内核态下执行。如果允许用户态进程执行关中断，恶意进程就可以通过一直关中断来霸占 CPU，导致整个系统崩溃。因此 C 错误。A 正确（单核关中断有效，多核关中断只能屏蔽当前核的中断，无法阻止其他核进入临界区）；B 正确（自旋锁就是忙等）；D 正确（极短临界区下，自旋锁避免了进程上下文切换的巨大开销，效率更高）。

131．B。**【解析】**这是记录型信号量最核心的定理。当进程执行 wait(S) 时，S.value--。初值为 K。当 V<0 时，说明 K 个资源已经全部分配完毕（消耗了 K 个 value），随后又有 m 个进程执行了 wait 并被阻塞（又消耗了 m 个 value）。所以总消耗量为 K+m。当前值 V=初值−总消耗量 =K−(K+m)=−m。

因此，只要有人阻塞（m>0），必定有 V<0 且 m=｜V｜=−V。这个结论与总进程数 n 完全无关（因为可能有进程还没申请资源）。D 选项后半句错误，只要有人阻塞，说明资源早就被占满了，正在使用的进程数必定是 K，而不是 K−m。

第 3 章

内存管理

【速对答案】

BDDCB　DBBCD　CCADC　BACCB

CADBB　BCCCC　DABAB　BBBCD

CCCBC　BDDDC　BBBDD　BBDCD

CBDCB　DCBAA　BDBCC　CCCB

1．B。【解析】I错误，内存保护必须有硬件的支持才能有效实现。操作系统通过设置硬件机制（如基址寄存器、限长寄存器、内存管理单元 MMU 中的页表/段表机制以及相关的特权级别）来限制进程对内存的访问。软件（操作系统）负责管理这些硬件机制，但保护的执行和检查是由硬件在每次内存访问时进行的。如果完全依赖软件实现，效率会极低，且用户程序很容易绕过软件检查。

Ⅱ正确，内存保护可以采用两种方法：

- 在 CPU 中设置一对上、下限寄存器，存放用户进程在主存中的下限和上限地址，每当 CPU 要访问一个地址时，分别和两个寄存器的值相比，判断有无越界。
- 采用重定位寄存器(也称基地址寄存器)和界地址寄存器(也称限长寄存器)进行越界检查。

III错误，加载重定位寄存器和界地址寄存器时必须使用特权指令，只有操作系统内核才可以加载这两个存储器。这种方案允许操作系统内核修改这两个寄存器的值，而不允许用户程序修改。

综上所述，选 B。

2．D。【解析】形成逻辑地址的阶段是在完成链接步骤时。若源程序采用的装入方式为绝对装入或可重定位装入方式，则在装入阶段形成物理地址，若源程序采用的装入方式为动态运行时装入方式，则在程序运行时完成逻辑地址到物理地址的转换。综上，本题选择 D 选项。

3．D。【解析】A 正确，静态链接是在程序执行前，由链接器将编译后的目标模块与它们所需的库函数等链接成一个单一的可执行文件。这个可执行文件包含了所有需要运行的代码和数据，在运行时其内部结构不再改变

B 正确，装入时动态链接是将链接过程推迟到程序装入内存时进行。 这种方式下，目标模块是分开存放的，因此当某个模块需要更新或修改时，只需替换该模块即可，无需重新链接整个程序。同时，多个应用程序可以共享内存中同一目标模块的单个副本。

C 正确，运行时动态链接是最灵活的链接方式，它只在程序运行过程中，当某个模块首次被调用

时，才将其装入内存并进行链接。这种方式可以避免装入那些在本次执行中根本不会用到的模块，从而节省了内存开销，并且因为初始装入的模块较少，也加快了程序的启动速度。

D 错误，对于静态链接，链接完成后形成的是一个装入模块（可执行文件）。在这个模块中，对外部符号的引用会被解析为相对地址（相对于装入模块的起始地址）或者需要重定位的地址。最终的绝对物理地址是在程序被装入内存时，根据装入方式（如绝对装入、可重定位装入、动态运行时装入）才确定的。对于动态链接（包括装入时动态链接和运行时动态链接），链接本身就是在装入或运行时才进行的，其地址的解析和绑定同样会推迟，并且最终形成的地址也可能是逻辑地址（在动态运行时装入方式下），而不是立即转换为绝对物理地址。地址转换到物理地址是装入和运行时内存管理单元（MMU）的工作。

综上所述，D 错误。链接过程主要是解决模块间的符号引用，并将它们组织成一个统一的逻辑地址空间，而不是直接解析为绝对物理地址。

4．C。**【解析】**A、B、D 均正确。在动态运行时重定位系统中，编译链接形成的均是逻辑地址，且在运行过程中，即使页面被置换到不同的物理页框，CPU 执行的指令内部包含的依然是逻辑地址。

C 错误。动态重定位并不是在装载时由装载程序完成的，而是在程序真正执行到该条指令时，由硬件地址变换机构（MMU）利用重定位寄存器实时转换的。装载时一次性完成转换的是静态重定位

5．B。**【解析】**还要修改 F2 分区的起始地址。

6．D。**【解析】**只要是固定的分配就会产生内部碎片，其余的都会产生外部碎片。如果固定和不固定同时存在(如段页式)，其物理本质还是固定的，解释如下：

分段虚拟存储管理：每一段的长度都不一样(对应不固定)，所以会产生外部碎片。

分页虚拟存储管理：每一页的长度都一样(对应固定)，所以会产生内部碎片。

段页式分区管理：地址空间首先被分成若干个逻辑分段(这里的分段只是逻辑上的，而我们所说的碎片都是物理上真实存在的，所以是否有碎片还是要看每个段的存储方式，页才是物理单位)，每段都有自己的段号，然后再将每个段分成若干个固定的页。所以其仍然是固定分配，会产生内部碎片。

固定式分区管理：很明显是固定的，会产生内部碎片。

综上分析，本题选 D 选项。

7．B。**【解析】**如下表所述

分配方式	是否可能产生内部碎片	是否可能产生外部碎片	说明
单一连续分配	✗ 不会	✗ 不会	仅有一个进程在内存中，无划分
固定分区分配	✓ 会	✗ 不会	内部碎片：小进程占大分区；外部碎片：分区大小事前划分成固定大小，不存在外部碎片
可变分区分配	✗ 不会	✓ 会	外部碎片明显；若使用整块分配则无内部碎片
动态重定位分区分配	✗ 不会	✓ 会	是在可变分区分配基础上引入了紧凑技术，只是减少了外部碎片，并不是完全没有外部碎片。

分页存储管理	✓ 会	✗ 不会	最后一页可能用不满（内部碎片）；分配是离散的无外部碎片
分段存储管理	✗ 不会	✓ 会	每段大小刚好匹配进程模块；段间不连续可能导致外部碎片
段页式存储管理	✓ 会	✗ 不会	每段内部分页，页内可能有内部碎片；分页避免了外部碎片

注意：固定分区分配有解释说存在外部碎片，即如果出现较大的作业，剩余的小的固定分区无法装入，把这种小分区当作“外部碎片”。也有教材指出，外部碎片和内部碎片只存其一（不讨论单一连续分配）。个人认为固定分区分配没有外部碎片。

8．B。**【解析】**动态链接：程序在运行时才将所需的外部模块（如库函数）装入内存并建立链接。

这就要求内存管理方式具有：模块化（每个模块可独立加载），地址空间支持逻辑分段、支持按需装载、独立保护、重定位等。而分段存储管理方式，程序划分为多个“段”（代码段、数据段、库段），每段可以独立加载、链接，最适合动态链接。

分页存储管理方式通常是静态链接，将用户空间链接成一个统一的逻辑地址空间，通过页表可离散的映射到内存中的也框中。

9．C。**【解析】**I错误，首次适应算法中，空闲分区按**地址递增**的次序排序，系统每次分配第一个能满足大小的空闲分区。

Ⅱ正确，邻近适应算法（Next Fit）从上次查找结束的位置开始继续查找空闲分区。这种方式使得空闲分区的分布更加均匀，避免了首次适应算法中低地址部分被集中使用和切割，从而导致高地址部分保留大空闲区的情况。

Ⅲ正确，最坏适应算法（Worst Fit）总是选择最大的空闲分区进行分配。这样做的一个主要优点是，分配后剩余的空闲区通常也比较大，不容易产生过多难以利用的小碎片，这对于后续中、小作业的分配是有利的。

Ⅳ错误，最佳适应算法（Best Fit）虽然听起来“最佳”，但它倾向于选择与请求大小最接近的空闲分区，这会导致系统中留下大量非常小的、难以再利用的外部碎片。而且，为了找到“最佳”分区，它通常需要扫描整个空闲分区列表或对空闲区按大小排序，查找开销较大，因此其整体性能（包括空间利用率和查找效率）并非最优。

综上所述，正确的说法是II 和III，选 C。

10．D。**【解析】**A 正确，当需要分配一个较小的内存块而当前没有合适大小的空闲块时，系统会找到一个更大的空闲块，并将其分裂成两个大小相等的子块，这两个子块互为伙伴。

B 正确，伙伴系统在回收内存块时，会检查其伙伴块是否也空闲。只有当两个块大小相同、物理地址连续，并且它们的起始地址满足特定的对齐约束时，它们才被认为是伙伴，可以合并成一个更大的块。

C 正确，由于伙伴块是由一个父块等分成两部分得到的，并且它们的地址具有特定的对齐关系，因此对于任何一个块，其伙伴块的地址是唯一确定的，并且可以通过简单的位运算（通常是异或操作）根

据其自身的起始地址和大小计算出来。

D 错误，伙伴关系的核心在于它们是**由同一个父块直接分裂而来**的，并且**物理地址相邻**。仅仅大小相同，或者都源自同一个更大的初始空闲块（但经过多次不同的分裂和合并），并不足以使它们成为伙伴。例如，一个 16KB 的块可以分裂成两个 8KB 的块（它们是伙伴）；这两个 8KB 的块又可以各自独立分裂成 4KB 的块。其中一个 8KB 分裂出的两个 4KB 是伙伴，另一个 8KB 分裂出的两个 4KB 也是伙伴，但前一组的 4KB 块与后一组的 4KB 块（即使大小相同且都最终源自同一个 16KB 块）之间不是伙伴关系，因为它们不是由同一个直接父块分裂而来且不相邻。

11．C。**【解析】**最佳适应(Best Fit)算法：空闲分区按容量递增的次序排列。每次分配内存时，顺序查找到第一个能满足大小的空闲分区，即最小的空闲分区，分配给作业。

12．C。**【解析】**A 正确，快速适应算法的核心思想就是预先根据进程的常见需求大小将空闲分区分类管理，分配时直接从对应大小的链表中取用，查找效率高。但其缺点在于分区归还时，为了有效地合并可能产生的相邻小空闲块，算法会比较复杂。

B 正确，伙伴系统的特点是内存块的大小固定为 2 的幂。分配时，如果找不到大小为 2(i 的空闲块，就会去查找大小为 2)(i+1)的空闲块，如果找到则将其分裂成两个大小为 2(i 的“伙伴”块，一个用于分配，另一个加入 2)i 的空闲链表。

C 错误，伙伴关系的核心在于它们是**由同一个父块直接分裂而来**的，并且**物理地址相邻**。仅仅大小相同并不足以使它们成为伙伴。

D 正确，哈希算法利用哈希函数将所需空闲分区的大小映射到哈希表的一个条目，该条目指向一个空闲分区链表，该链表中的空闲分区大小与所查找的大小相近，从而可以快速定位到合适的空闲分区。

13．A。**【解析】**动态重定位分区分配可以在内存无足够大分区时移动内存中的程序使其互相连成一片，消除外部碎片以获得更大的内存分区，故 A 正确。

14．D。**【解析】**

操作	空闲分区/MB	占用分区/MB
初始	126~511	无
分配80MB， 分配56MB， 分配120MB	382~511	126~205，206~261，262~381
释放56MB	206~261，382~511	126~205，262~381
分配40MB， 分配50MB	246~261，432~511	126~205，206~245，262~381， 382~431

此时空闲的最大分区是 432~511。

15．C。**【解析】**按照题中的各种分配算法，分配结果如下表所示。只有最佳适应算法能够完全完成分配任务。

A 选项，首次适应算法：按地址顺序从头扫描，找到第一个满足的空闲区。

- 分配 212KB→ 450 满足，分掉 → 剩 238，剩余空闲分区：[100] [238] [250] [200] [600]
- 分配 417KB→ 238 不够，250 不够，200 不够，600 满足 → 剩 173，剩余空闲分区：[100] [238] [250] [200] [173]
- 分配 112KB→ 100 不够，238 满足 → 剩 126，剩余空闲分区：[100] [126] [250] [200] [173]
- 分配 426KB→ 100 不够，126 不够，250 不够，200 不够，173 不够 → ✗ 失败

B 选项，邻近适应算法：从上次分配成功的位置继续向后找（循环搜索）。

- 分配 212KB → 从头开始，450 分掉（剩 238），指针停在 238
- 分配 417KB → 从 238 开始，250 不够，200 不够，600 满足 → 剩 183，指针停在 183
- 分配 112KB → 从 183 开始，足够 → 剩 183-112=71KB，[100，238，250，200，71]
- 分配 426KB → 所有都不满足 → ✗ 失败

C 选项，最佳适应算法：优先选择**最小的能装下的空闲区**，碎片最少。

- 分配 212KB → 最小满足的是 250 → 剩 38，剩余空闲分区：[100] [450] [38] [200] [600]
- 分配 417KB → 最小满足的是 450 → 剩 33，剩余空闲分区：[100] [33] [38] [200] [600]
- 分配 112KB → 最小满足的是 200 → 剩 88，剩余空闲分区：[100] [33] [38] [88] [600]
- 分配 426KB → 满足的只有 600 → 剩 174，剩余空闲分区：[100] [33] [38] [88] [174]→✓ 全部分配成功！

D 选项，最坏适应算法：优先选择**最大的空闲区**分配，产生大碎片。

- 分配 212KB → 最大是 600 → 剩 388，剩余空闲分区：[100] [450] [250] [200] [388]
- 分配 417KB → 最大是 450 → 剩 33，剩余空闲分区：[100] [33] [250] [200] [388]
- 分配 112KB → 最大是 388 → 剩 276，剩余空闲分区：[100] [33] [250] [200] [276]
- 分配 426KB → 最大是 276 → ✗ 失败

16．B。【解析】A 正确，这是对“紧凑”技术的基本描述。通过移动内存中的作业，可以将原本分散的多个外部碎片合并成一个或少数几个较大的连续空闲区，以便分配给新的或等待的作业。

B 错误，这句话描述的是在静态重定位或可重定位装入方式下进行紧凑会面临的问题。在这些方式下，程序中的地址在装入时或之前就已经转换（或部分转换）为物理地址。如果内存中的物理位置改变，这些地址就需要修改。然而，紧凑技术通常与动态重定位配合使用。在动态重定位下，程序内部使用的是逻辑地址，物理地址是在每次访存时通过重定位寄存器动态计算的。

C 正确，这是动态重定位的基本原理。程序在内存中时，其内部地址仍为逻辑地址（通常是相对于程序起点的地址）。CPU 执行时，硬件地址变换机构会自动将逻辑地址与重定位寄存器（存放程序在内存中的起始物理地址）中的内容相加，得到最终的物理地址。

D 正确，这是动态重定位配合紧凑技术的主要优点。由于程序内部使用的是逻辑地址，与物理位置无关，当作业因“紧凑”而在内存中移动后，操作系统只需要更新该作业在其进程控制块（PCB）中记录的内存起始地址，并将这个新的起始地址加载到重定位寄存器中即可。程序本身的指令和数据中包含的地址（逻辑地址）无需修改。

17．A。【解析】

a. 初始状态：

- 操作系统占用：0K - 199K (200KB)
- 用户区空闲：200K - 585K (386KB)
- 空闲分区表：[{起始地址：200K，大小：386KB}]

b. 进程 A 申请 80KB：

- 首次适应算法选择空闲分区 [200K，386KB]。
- 由于高地址部分分配出去，进程 A 分配到内存区域为 (586K - 80K) 到 585K，即 **506K - 585K**。
- 剩余空闲分区为 200K 到 (506K - 1K) = 505K。
- 新的空闲分区表：[{起始地址：200K，大小：306KB}] (即 200K - 505K)

c. 进程 B 申请 56KB：

- 首次适应算法选择空闲分区 [200K，306KB]。
- 进程 B 分配到内存区域为 (200K + 306KB - 56KB) 到 (200K + 306KB - 1K) = **450K - 505K**。
- 剩余空闲分区为 200K 到 (450K - 1K) = 449K。
- 新的空闲分区表：[{起始地址：200K，大小：250KB}] (即 200K - 449K)

d. 进程 C 申请 120KB：

- 首次适应算法选择空闲分区 [200K，250KB]。
- 进程 C 分配到内存区域为 (200K + 250KB - 120KB) 到 (200K + 250KB - 1K) = **330K - 449K**。
- 剩余空闲分区为 200K 到 (330K - 1K) = 329K。
- 新的空闲分区表：[{起始地址：200K，大小：130KB}] (即 200K - 329K)

e. 进程 A 释放 80KB (506K - 585K)：

- 当前空闲分区表：[{起始地址：200K，大小：130KB}]
- 释放的区域 A (506K - 585K) 与当前空闲分区 (200K - 329K) 不相邻。
- 新的空闲分区表 (按地址排序)：[{起始地址：200K，大小：130KB}，{起始地址：506K，大小：80KB}]

f. 进程 C 释放 120KB (330K - 449K)：

- 当前空闲分区表：[{起始地址：200K，大小：130KB} (200K - 329K)，{起始地址：506K，大小：80KB}]
- 释放的区域 C (330K - 449K) 的低地址端 330K 与第一个空闲分区 (200K - 329K) 的高地址端 329K 相邻。
- 合并：新的空闲分区起始地址为 200K，大小为 130KB + 120KB = 250KB (范围 200K - 449K)。
- 新的空闲分区表：[{起始地址：200K，大小：250KB}，{起始地址：506K，大小：80KB}]

g. 进程 D 申请 156KB：

- 首次适应算法选择空闲分区 [200K，250KB]。

- 进程 D 分配到内存区域为 (200K + 250KB - 156KB) 到 (200K + 250KB - 1K) = **294K - 449K**。
- 剩余空闲分区为 200K 到 (294K - 1K) = 293K。
- 新的空闲分区表：[{起始地址：200K，大小：94KB}，{起始地址：506K，大小：80KB}] (即 200K - 293K)

h. 进程 E 申请 81KB：

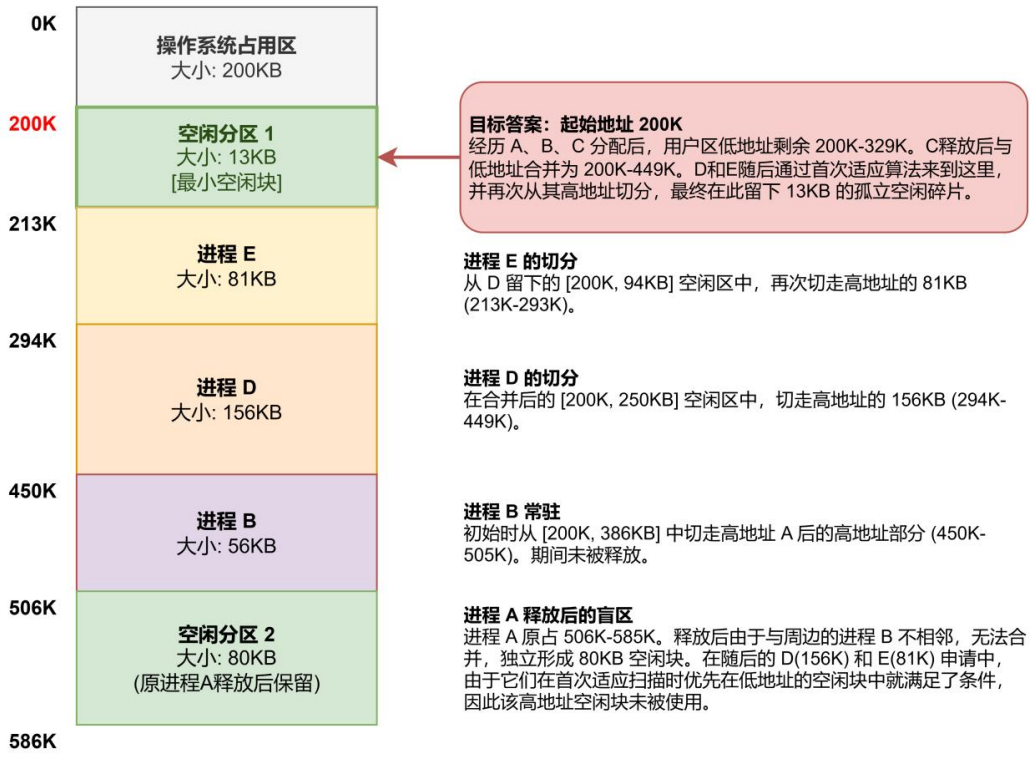
- 首次适应算法选择空闲分区 [200K，94KB]。
- 进程 E 分配到内存区域为 (200K + 94KB - 81KB) 到 (200K + 94KB - 1K) = **213K - 293K**。
- 剩余空闲分区为 200K 到 (213K - 1K) = 212K。
- 新的空闲分区表：[{起始地址：200K，大小：13KB}，{起始地址：506K，大小：80KB}] (即 200K - 212K)

最终空闲分区：

- 起始地址：200K，大小：13KB
- 起始地址：506K，大小：80KB

内存中最小空闲块的大小是 13KB，其起始地址为 **200K**。

【首次适应算法 + 高地址分配】最终内存布局与演变示意图
用户区范围：200K - 585K (共386KB) | 绿色为最终空闲块



18．C。【解析】按照最佳适应算法（选择能满足条件的最小空闲分区）：

初始：用户区 [{S：32MB，L：55MB}]

A 分配 15MB: A 占用 [32MB，46MB]. 剩余 [{S:47MB，L:40MB}]

B 分配 30MB: B 占用 [47MB，76MB]. 剩余 [{S:77MB，L:10MB}]

A 释放 15MB (即 [32MB，46MB]): 空闲列表 (按地址排序) [{S:32MB，L:15MB}，{S:77MB，

L:10MB}]

C 分配 8MB：

- 空闲块有 15MB 和 10MB。都能满足 8MB。
- 10MB 是能满足条件的最小空闲分区。
- C 占用 [77MB，84MB]. 剩余 [85MB，86MB] (2MB)。
- 空闲列表：[{S:32MB，L:15MB}，{S:85MB，L:2MB}]

D 分配 6MB：

- 空闲块有 15MB 和 2MB。2MB 不够。
- 15MB 是能满足条件的最小（也是唯一）空闲分区。
- D 占用 [32MB，37MB]. 剩余 [38MB，46MB] (9MB)。
- 空闲列表：[{S:38MB，L:9MB}，{S:85MB，L:2MB}]

最终空闲分区：

- 起始地址：38MB，大小：9MB
- 起始地址：85MB，大小：2MB

此时内存中最大的空闲分区大小是 **9MB**。

19．C。【解析】当逻辑地址空间很大时，单级页表可能非常庞大，需要一块巨大的连续内存空间来存放。多级页表通过将页表本身进行分页（外层页表指向页表的页面），使得这些页表的页面可以离散地存放在内存中，无需连续。只有当前需要的外层页表和部分内层页表页面需要驻留内存，从而解决了连续大空间分配的难题，故选 C。

20．B。【解析】一级页表页表项的内容应当指向下一级页表的位置，然后在二级页表中根据页号找到目的程序文件的页框号，故一级页表页表项的内容为对应二级页表所在的页框号，本题选择 B 选项。

21．C。【解析】A 错误，静态重定位是在程序装入内存时一次性完成的，由装入程序（软件）通过修改指令中的地址码来实现。在程序运行期间，CPU 直接访问物理地址，因此不需要硬件地址转换机构的支持。B 错误，动态重定位确实是在程序执行时完成的，且支持程序在内存中移动。但它不依赖装入程序修改地址，而是依赖于 CPU 中的硬件地址转换机构（如重定位寄存器/基址寄存器或者 MMU）在运行时动态计算物理地址。C 项正确：分页和分段存储管理都属于动态重定位，在程序运行过程中，每一次访存（取指或读写数据）都需要进行地址转换，为了保证效率，这种高频的地址转换必须由硬件地址转换机构（MMU）自动完成。D 错在“由操作系统通过直接检索”这句话。操作系统负责的是页表的建立和维护（软件管理），而运行时根据逻辑地址去检索快表/慢表并拼接物理地址的过程，是 MMU 执行的。

22．A。【解析】I正确，不使用快表时，CPU 取指令需要先访问一次内存中的页表获得物理地址，之后再访问一次内存取相应指令。

Ⅱ正确，分页存储管理方式不存在外部碎片，但在进程最后一页会存在内部碎片。

Ⅲ错误，分页存储管理方式对用户透明，分段存储管理方式对用户不透明。

Ⅳ正确，分页存储管理方式下，进程装入内存后可能会改变位置，不能采用静态重定位。

23．D。【解析】系统的平均访存时间 = 快表命中率 × 快表访问时间 + 快表未命中率 ×（访问快表时间 + 访问页表时间） + 访问内存时间。设快表命中率为 F，则开启分页功能时访存时间为 F ×1ns + 100ns + (1 - F) ×100ns = 110ns，解得 F = 90.9%。因此选 D。

24．B。【解析】由题中所给的条件，虚拟地址空间是 256TB（2^48B），即未完全使用 64 位地址。页面大小为 8KB（2^13B），则用于分页的地址线的位数为48-13=35。下面计算每级页表能容纳的最多数量。由题意每个页面为 8KB，每个页表项为 8B，则一页中能容纳的页表项为 8KB/8B=1K，即 1024 个页表项，可占用 10 位地址线来寻址，剩余的 35 位地址线可分为 35/10=3.5，向上取整为 4，因此至少 4 级页表才能完成此虚拟存储的页面映射。

25．B。【解析】页大小为 2KB，则页内偏移长 11 位，页号长 5 位；页框号长 7 位。0x1234 表示为 16 位二进制数即 0001001000110100，得到页号位 00010=2，对应页框号 36(二进制为 0100100)，与页内偏移 01000110100 组合即得 010010001000110100，即 0x12234，因此选 B。

26．B。【解析】本题采用两级页表，一个页面可放置页表项数 N=1KB/2B=512，逻辑地址空间页数 M=64MB/1KB=64K。所以页表大小为 64K/512=128 页。因此页目录表中应有 128 个表项。

27．C。【解析】一个页面中能装入的页表项数 N=8KB/8B=1K=2^10。逻辑地址空间页面数 M=256TB/8KB=32G=2^35 个。最外层页表只能占一页，所以需要 35/10 取整=4 级页表。C 正确。

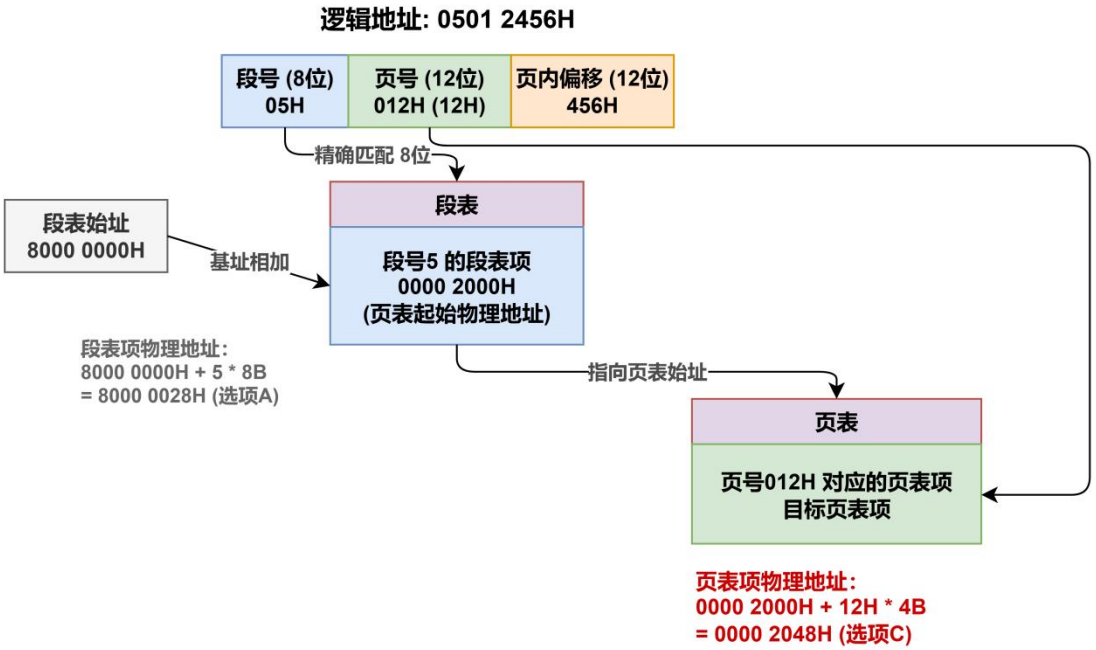
28．C。【解析】由题意可知，共有 7 位来表示页号，因此共有 2^7 个页面，页面大小为 2^11B=2KB，主存最大容量为 2^7x2KB=256KB，在分页存储管理系统中，内存块数=页面数=2^7。由于该指令所在页的页号为 0，其分配的物理块号为 2，采用的是相对地址，所以该指令的物理地址=基地址 +相对地址 =2x2KB+1500=5596。该指令数据的存储地址为 12500，其所在页的页号为 12500/2K=6。选项 C 正确。

29．C。【解析】当快表命中率为 85%时，系统的平均访存时间=快表命中率 x 快表访问时间+快表未命中率 x(访问快表时间+访问页表时间)+访问内存时间 =0.85x0.2+(1+0.2)x0.15+1=1.35。

当快表命中率为 50% 时，系统的平均访存时间=快表命中率 x 快表访问时间+快表未命中率 x(访问快表时间+访问页表时间)+访问内存时间 =0.5x0.2+(1+0.2)x0.5+1=1.7。选 C。

注意：*计算平均访存时间不要忘了访问页表后的访问内存时间。*

30．C。【解析】具体过程如下图所示：



31．D。【解析】A 正确，可重入代码（纯代码）的一个重要特性是其在执行期间自身不发生改变，任何可能改变的部分（如局部变量、返回地址）都存储在每个进程私有的栈空间或其他私有数据区。这使得多个进程可以同时执行同一份代码副本而不会相互干扰，是实现代码段共享的基础。

B 正确，如果多个进程共享一个可读写的数据段，那么它们对该数据段的并发访问可能会导致数据不一致的问题（例如，竞争条件）。因此，操作系统需要提供同步互斥机制（如信号量、互斥锁、管程等），让进程能够协调对共享数据的访问，以保证数据完整性和一致性。

C 正确，为了有效管理共享段，操作系统通常会维护一个系统级的共享段表。该表记录了每个共享段的物理信息（如在内存中的起始地址、段的长度）以及一个共享计数（或链接计数），用于表示当前有多少个进程正在共享该段。当共享计数为 0 时，该段所占用的内存才能被回收。

D 错误，在分段存储管理中，每个进程拥有自己独立的段表，段号是进程逻辑地址空间的一部分。当多个进程共享同一个物理段时，这个共享段在它们各自的段表中所对应的段号可以是不同的。例如，进程 A 可能将共享库段映射为其 3 号段，而进程 B 可能将其映射为其 5 号段。重要的是它们段表中的相应表项都指向物理内存中同一个共享段的起始地址和具有相同的段长。段内地址（偏移量）是相对于段起始而言的，对于同一个共享段内的同一逻辑位置，其段内地址是相同的。

32．A。【解析】分页系统的核心是将虚拟内容空间和物理内存交国均划分为大小相同的页面，并以页面作为内存空间的最小分配单位，因此选项 A 正确。

页是信息的物理单位，页长由系统决定，因此选项 B 错误。

分段是为了方便用户编程以及方便共享，因此选项 C 请误。

段是信息的逻单位，段长由用户决定，因此选项 D 错误。

33．B。【解析】程序分段对用户和操作系统都可见，但其分段多在用户编程时决定。

34．A。【解析】前者段基址为 0x00000，段内偏移为 0x1327，计算得到物理地址为 0x01327;后者段基址为 0xB8000，段内偏移为 0x1592，计算得物理地址为 0xB9592。因此选 A。

35．B。【解析】段页式存储管理结合了页式管理和段式管理的优点，用分段的方法来分配和管理

用户地址空间，用分页的方法来管理物理存储空间。故答案为选项 B。

36．B。【解析】页式存储管理方式中页大小相同，页号和页内偏移量可直接通过逻辑地址计算得出，地址空间是一维的。而分段式和段页式中的段号和段内地址都要显式给出，地址空间是二维的。

37．B。【解析】在段页式存储系统中，为了获取一条指令或数据，必须三次访问内存，第一次访问内存中的段表，从中取得页表地址；第二次访问内存中的页表，从中取得该页所在的物理块号并将该块号与页内地址一起形成指令或数据的物理地址；第三次访问是根据第二次访问所得的地址，真正取出指令或数据(但这是在访问地址正确的情况下)。

为了防止越界，在段页式存储系统中配置了一个段表寄存器，用于存放段表始址和段表长度。在进行地址变换时，首先利用段号，将它与段长进行比较。若段号超出段表长度，则表示越界了。同样的段表中，页表大小这项也是为了预防地址越界的情况。一般情况下，段号、页号都是从 0 开始编址，这点从题目所给的图中也可得知。

该用户访问的 3 个地址中，地址一，段号检查通过，页号越界，3 不在页号 0~2 范围内，所以只访问内存 1 次。

地址二，段号检查未通过，8 不在段号 0~7 范围内，越界，所以访问内存 0 次。

地址三，段号检查通过，且页号也无越界，成功访问到数据，所以访问内存 3 次。

38．B。【解析】A 错误。在页式管理系统中，每个进程都有自己独立的页表。页表用于将进程的逻辑地址空间映射到物理内存地址空间。如果整个操作系统只有一张页表，那么不同进程的相同逻辑地址就会映射到相同的物理地址，无法实现进程地址空间的隔离。

B 正确。在段式管理系统中，每个进程的逻辑地址空间由若干个段组成，每个段都有其独立的属性（如基址、长度、权限等）。为了管理这些段并将段的逻辑地址映射到物理地址，操作系统会为每个进程建立一张段表。

C、D 均错误。在段页式管理系统中，先将进程的逻辑地址空间分成若干个段，然后再将每个段分成若干个页。因此，每个进程会有一张段表，段表中的每个表项对应一个段，并指向该段的页表。这意味着每个进程的每个段都有一个对应的页表，所以一个进程可能会有多张页表，而不是整个进程只有一张页表，更不是整个操作系统只有一张页表。

39．C。【解析】A 错误，虚拟存储器的核心思想之一是“对换性”或“部分驻留”，即只将程序当前需要运行的部分调入内存，暂时不用的部分则存放在外存（如磁盘）中。当需要时再调入，不需要或暂不使用的则可能被换出。这与“全部保留在内存中”相反。

B 错误，虚拟存储器的实现恰恰是建立在离散分配的内存管理方式之上的，如请求分页式或请求段式（或段页式）存储管理。　离散分配允许程序的不同部分存放在物理内存中不连续的位置，这为页面的换入换出提供了灵活性。连续分配方式难以支持虚拟存储器的有效实现，因为它要求为整个程序或其已知部分分配连续空间。

C 正确，虚拟存储器的“虚拟性”特征使得用户看到的逻辑地址空间可以远大于实际的物理内存容量。　这个逻辑地址空间的大小受限于 CPU 的寻址能力（如 32 位 CPU 理论上可寻址 4GB，64 位 CPU 则大得多）。而这些逻辑地址空间中的内容，一部分在物理内存中，大部分则存放在外存（通常是磁盘

的交换区或对换区）上。因此，虚拟存储的总容量可以理解为物理内存和可用外存空间之和，但实际上限更多地由 CPU 寻址范围和外存大小决定。

D 错误，这是传统存储管理方式的特征。 虚拟存储器的核心思想恰恰是打破了这两个限制，实现了“多次性”（程序不必一次全部调入，可以分多次调入）和“对换性”（程序不必一直常驻内存，可以换入换出）。

40．D。【解析】A 正确，虚拟内存管理方式下，每个进程都认为自己拥有一个完整的、独立的地址空间，通常从 0 开始编址。即使不同进程使用了相同的虚拟地址，由于它们有各自独立的页表或段表，这些相同的虚拟地址会被映射到物理内存中不同的物理地址，从而实现了进程间的地址隔离。

B 正确，用户程序（包括 C 语言程序）在运行时，其所有地址引用（包括通过 malloc()等函数动态分配的内存地址）都是虚拟地址。这些虚拟地址由操作系统和硬件（MMU）转换为实际的物理内存地址。

C 正确，存保护是虚拟内存管理的重要功能。操作系统通过在页表项或段表项中设置保护位（如读、写、执行权限位），可以对进程虚拟地址空间中的不同逻辑段或页面施加不同的访问控制。例如，代码段通常设置为只读和可执行，数据段设置为可读写。

D 错误，一个进程的虚拟地址空间的总大小主要由 CPU 的寻址能力（即 CPU 地址线的位数，如 32 位系统理论上最大虚拟地址空间为 232 字节，64 位系统则更大）决定，而不是直接由当前空闲物理内存或磁盘交换区的大小决定。物理内存和磁盘交换区的大小影响的是一个进程（或多个进程）实际可以使用的、能被有效映射到物理介质上的虚拟内存量，但不是虚拟地址空间本身理论上的最大范围。

41．C。【解析】A 错误：MMU 只负责发现缺页并触发异常信号，陷入内核态是由 CPU 控制单元完成的，MMU 不执行中断处理程序。

B 错误：即使分配的页框再多，进程在首次访问某些虚拟页面时，依然会发生无法避免的“初次访问缺页”。

C 正确：OS 将页面调入内存并修改页表后，会执行异常返回指令（如 iret），CPU 会重新执行刚才被打断的指令。

D 错误：TLB 中不缓存存在位为 0 的表项。如果 TLB 命中，说明该页绝对在内存中且有效。发生缺页时，TLB 必定缺失。

42．C。【解析】C 不会执行。引发缺页中断的进程当前正处于**运行态**（因为它正在执行指令并访问了不在内存的页面）。发生缺页中断后，该进程会从运行态直接进入**阻塞态**（等待态），以等待所需页面从外存调入内存。它不是从就绪队列移到阻塞队列。

43．C。【解析】I正确，交换区（也称为对换区或页面文件）的主要功能就是作为物理内存的后备存储，当物理内存不足时，操作系统可以将暂时不使用或不活跃的内存页面或整个进程的内容写入到交换区，以便释放物理内存供其他任务使用。

Ⅱ错误，交换区管理的主要目标是提高页面换入换出的速度和效率，而不是像文件系统那样追求灵活的空间利用或支持复杂的目录结构。为了追求速度，交换区的分配策略通常比文件系统简单得多，对对换区的管理应采取连续分配存储管理方式，很少需要考虑外存中的碎片

问题，以最大限度地减少磁盘寻道时间和旋转延迟。文件系统通常采用更复杂的离散分配策略（如索引分配、链式分配）和数据结构来管理文件。

Ⅲ正确，这是交换区和虚拟内存机制的核心功能。通过页面换出（将内存页写入交换区）和页面换入（从交换区读回内存页），操作系统可以在有限的物理内存上运行比物理内存容量更大的程序，或者同时运行更多的程序。

Ⅳ错误，虚实地址的转换速度主要由硬件地址变换机构（如 MMU）、快表（TLB）的命中率以及页表（或段表）的结构和访问效率决定。 交换区的大小主要影响的是系统能够支持的虚拟内存的总量以及在物理内存极度紧张时系统的运行能力（即能换出多少页面）。增大交换区并不能直接加快单次地址转换的过程。相反，如果系统过于依赖交换区导致频繁的页面换入换出（抖动），反而会因为大量的磁盘 I/O 操作而降低整体性能，包括间接影响到与地址转换相关的操作（如因缺页中断而进行的页表操作）。

44．B。**【解析】**页缓冲队列是将被淘汰的页面缓存下来，暂时不写回磁盘。发生缺页时，一定会修改页表和将对应物理页写入用户内存，也会重新执行缺页指令，但是由于页缓冲队列的存在，所以可能不必去磁盘获取页面，因此答案选 B。

45．C。**【解析】**页框大小 = 2^{12}B = 4KB。每一级页表索引为 9 位，包含 $2^9 = 512$ 个页表项。每个页表项 8B，则一页能完美容纳一个级别的完整页表（512 × 8B = 4KB），即每一张页表恰好占用 1 个页框。将两个逻辑地址写成二进制，分析其各级索引：

地址 1：0x0000 0000 0000 1000 -> 高位全为 0，其一级、二级、三级索引均为 0，四级索引为 1。

地址 2：0x0000 0001 0000 2000 -> 拆分其中间 36 位（各 9 位）索引。

一级索引：0

二级索引：4 （因为 0x10000 溢出到了二级索引的低位）

三级索引：0

四级索引：2

计算页表数：

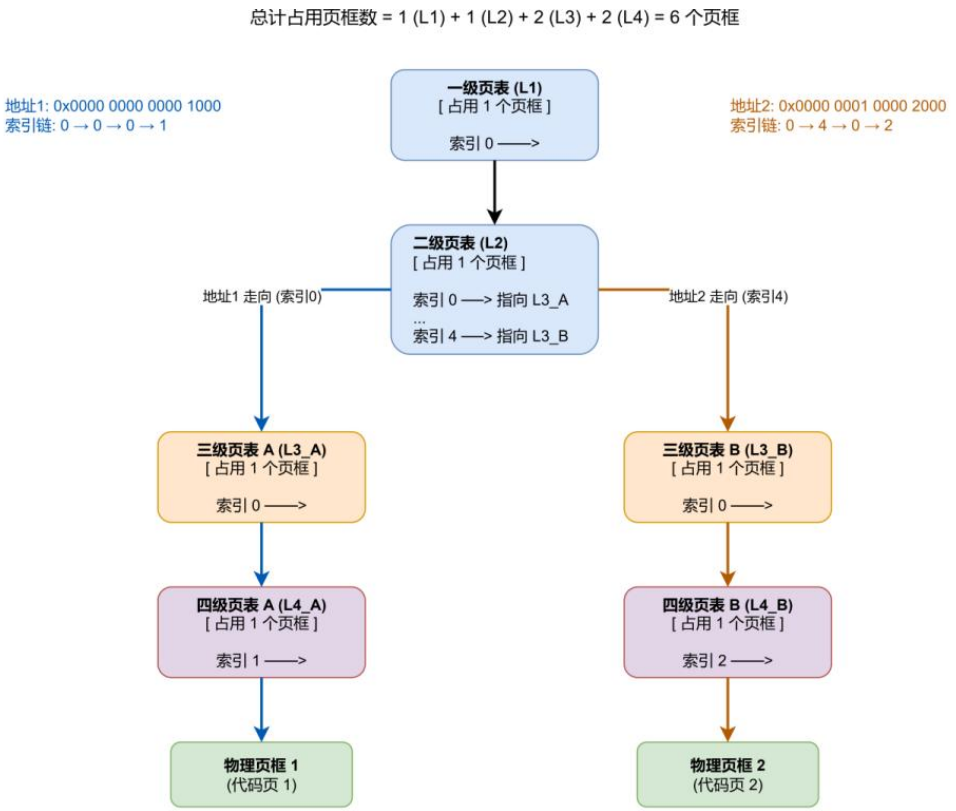
一级页表：全系统/进程共用 1 个（必不可少）。

二级页表：地址 1 和地址 2 的一级索引都是 0，因此它们共用同一个二级页表 -> 1 个。

三级页表：地址 1 通过二级页表项 0 指向一个三级页表；地址 2 通过二级页表项 4 指向另一个三级页表。由于二级索引不同，它们指向了不同的三级页表 -> 2 个。

四级页表：这两个独立的三级页表中的相应表项（索引 0）又分别指向各自的四级页表 -> 2 个。

总计占用页框数 = 1 + 1 + 2 + 2 = 6 个。



46．B。【解析】请求分页系统中的外存可分为两部分：用于存放文件的文件区和用于存放对换页面的对换区。通常，由于对换区采用连续分配方式，而文件区采用离散分配方式，因此对换区的数据存取(磁盘 IO)速度比文件区的高。这样，每当发生缺页请求时，系统应从何处将缺页调入内存，可分为以下 3 种情况。

（1）系统拥有足够的对换区空间，这时可以从对换区调入全部所需页面，以提高调页速度。为此，在进程运行前，须将与该进程有关的文件从文件区复制到对换区。

（2）系统缺少足够的对换区空间，这时，凡是不会被修改的文件都直接从文件区调入；而当换出这些页面时，由于它们未被修改，不必再将它们重写到磁盘(换出)，以后再调入时仍从文件区直接调人。但对于那些可能已被修改的部分，在将它们换出时，须将它们调到对换区，以后需要时再从对换区调入。

（3）UNIX 方式。由于与进程有关的文件都放在文件区，故凡是未运行过的页面，都应从文件区调入。而对于曾经运行过但又被换出的页面，由于被放在对换区，因此在下次调入时，应从对换区调入。由于 UNIX 系统允许页面共享，因此，某进程所请求的页面有可能已被其他进程调入内存，此时也就无须再从对换区调入。

综上所述，已修改过的页面再次装入时应来自磁盘对换区。

47．D。【解析】由题意知，每个页的大小为 1KB，即 1024B，所以页内地址有 10 位。把虚地址 0A5CH 转换成二进制为 1010010111100B，其中低 10 位 1001011100B 为页内地址，余下的高 2 位 10B.即 2 为页号，查虚页号和物理页号对照表可知，物理页号为4，即 100B.所以物理地址为 1001001011100B，即 125CH。用同样的方法，求得 1A5CH 的页号为 6，但此时的虚页号和物理页号对照表中没有虚页号 6，所以会产生缺页中断。故答案为选项 D。

48．D。【解析】页框大小为 512B，int 型数据 4B，则一个页框可以存放 512B/4B=128 个 int 型变量，又因数组按行存放，因此一个页框内刚好存放数组 a 的一行数据，由代码可知，每次访问数组 a 不同行的元素，导致访问均产生缺页，缺页的次数为 128x128 次=16384 次，故答案为选项 D。

49．D。【解析】设缺页率为 P，则有效存取时间为 1μs(访问页表)+Px(0.6x10ms+0.4x1ms + 1us+1us)(缺页需要缺页异常然后重新访问页表和内存)+(1-P)x（1us+1us）(不缺页访问页表+内存)=≤ 3us，计算得 P=1/6401，选 D。

50．C。【解析】A 项错误：既然共享同一个物理页框 F，两者的页表项中存储的物理页框号必然完全相同。B、C 项正确/错误分析：物理页框本身作为硬件内存是没有进程概念的，它不存储某个进程对其的访问权限。页面的访问权限（如只读、读写、执行）是存储在各个进程各自的页表项中的。因此，P1 的页表项里权限位写着“只读”，P2 的页表项里写着“读写”，它们互不干扰，故 C 正确，B 错误。D 项错误：由于映射到同一个物理页框，P2 的修改是直接作用于物理内存的，P1 再次读取该物理内存时会立即看到更新。

51．B。【解析】根据题意，忽略 TLB 访问时间。若快表命中则可直接获得物理地址，需要一次内存访问，若快表未命中，则需要访问页目录表、页表才能获得物理地址，共需要三次内存访问。所以，平均访存次数=0.25x3+0.75x1=1.5 次内存访问。

52．B。【解析】A 错误： 多级页表确实节省了内存空间，但代价是增加了查表的层级。如果没有 TLB 命中，单级页表访存 1 次，二级页表需访存 2 次，因此它降低了单次地址转换的速度。

B 正确： 查表的起点（页表基址）必须是物理地址，供硬件 MMU 直接寻址。非运行态时，它安全地保存在内核空间的 PCB 中；运行态时，由 OS 填入 CPU 的页表基址寄存器。

C 错误： 32 位地址、4KB 页大小（偏移量 12 位），剩下的 20 位页号分为 10 位目录号和 10 位页表号。页目录层只有 $2^{10} = 1024$ 个目录项，占用最大内存为 $1024 \times 4B = 4KB$。（4MB 是所有二级页表全满时的总和）。

D 错误： 在多级页表架构下，除了顶级的页目录表必须常驻物理内存，其余下级页表是允许被换出到磁盘的。如果下级页表缺失，同样会触发缺页中断，这确实增加了额外的磁盘 I/O 开销。

53．B。【解析】I错误，内存分配策略和页面置换策略之间存在一定的关联性，不能随意搭配。例如，不存在固定分配全局替换策略。

Ⅱ正确，在固定分配局部置换策略中，系统需要为每个进程预先分配固定数量的物理页框。这个数量的是一个难点：如果分配得太少，进程会频繁发生缺页中断，降低系统吞吐量；如果分配得太多，会减少内存中可容纳的进程数量，可能导致 CPU 或其他资源空闲。

Ⅲ正确，可变分配局部置换策略允许根据进程的运行情况动态调整其分配到的物理页框数量。当一个进程频繁发生缺页中断（可能出现抖动的迹象）时，系统可以为其增加分配的物理页框数，直至其缺页率降低到适当程度。通过这种动态调整，可以在一定程度上缓解或降低抖动的发生。

54．D。【解析】若采用 FIFO 算法，由于该算法存在 Belady 异常，所以缺页率可能升高、不变或降低。所以选 D。

55．D。【解析】A 正确，这是一种实现 LRU 算法的硬件方法。每次访问页面时，对应页框的寄存

器特定位（如最高位）被置 1。定时信号会周期性地将所有寄存器的内容右移。这样，长时间未被访问的页面，其寄存器中的 1 会逐渐右移出去，使得其数值变小。数值最小的寄存器对应的页面就是最近较长时间未使用的页面。

B 正确，硬件栈是实现 LRU 算法的一种有效方式。每当一个页面被访问时，就将其页面号从栈中取出（如果已存在）并压入栈顶。这样，栈顶总是最近被访问的页面，而栈底则是最近最久未被访问的页面。当需要淘汰页面时，选择栈底的页面即可。

C 正确，这描述的是 Clock（时钟）置换算法，它是 LRU 算法的一种常用近似实现。通过访问位来记录页面近期是否被访问，并通过循环扫描和清除访问位的方式来选择淘汰页面，其开销比精确 LRU 小。

D 错误，虽然为每个页表项设置时间戳并在每次访存时更新可以精确记录页面的最后访问时间，从而找到最近最久未使用的页面，但这种方法的开销非常大。

硬件开销：为每个页表项增加一个足够宽的时间戳寄存器会增加硬件成本。

时间开销：每次内存访问都需要更新对应页面的时间戳，这会增加内存访问的延迟。当需要淘汰页面时，还需要遍历所有页表项来比较时间戳，以找到最小（即最久未被访问）的那个，这个搜索过程本身也会有相当大的开销，尤其是在页表很大的情况下。

56．B。【**解析**】本题考查各种页面替换算法。OPT(最佳替换)算法是理想中的最优算法(现实中不存在)，而 LRU(最近最少使用)算法最接近 OPT 算法，FIFO(先进先出)算法一般是很差，缺页率最高。所以缺页率 FIFO>LRU>OPT，故答案为选项 B。

57．B。【**解析**】LRU-2 维护两个队列，只有当当前访问的数据已经在历史访问队列中时，才会被加入缓存队列中。当访问到第 6 次时，历史访问队列为 5，6，7，8，3，此时访问 8，8 已在历史访问队列中，因此 8 加入缓存队列，此时历史访问队列为 5，6，7，3，缓存队列中为 8。接下来是第 7 次，访问 5，5 已在历史访问队列中，因此 5 加入缓存队列，此时历史访问队列为 6，7，3，缓存队列为 8，5。第 9 次，访问 6，6 已在历史访问队列中，因此 6 加入缓存队列，此时历史访问队列大 7，3，9，缓存队列为 8，5，6。访问到第 10 次时，访问 8，8 已在缓存队列中，将 8 重新放入队尾，此时历史访问队列为 7，3，9，缓存队列为 5，6，8。

58．D。【**解析**】FIFO 算法中，1 号页面最先进入，故换出 1；LRU 算法中，3 号页面最久未访问，故换出 3；改进 CLOCK 算法中，选择访问位和修改位都为 0，代价最小的 3 被换出。因此选 D。

59．C。【**解析**】CLOCK 算法的规则如下：

- 若页面命中，则简单把命中的页框访问位置 1，当前页框指针不变化。
- 若不命中，从当前页框出发，若当前页访问为 1，则把当前页访问位 置 0，继续往下找，直到找到访问位为 0 的页框，将旧页换出，新页换入，然后将此页框访问位置 1，当前页指针指向下一页框。

回到本题：设页框号分别为 0，1，2。

①首先 7，0，1 三个页依次分配到 0，1，2 页框。状态：$\underline{7}_0^1, 0_1^1, 1_2^1$，下划线表示当前指针指向的页框，右下标是页框号，右上标是访问位。此时缺页 3 次。

②2 进入，指针依次转过 7，0，1，访问位依次置 0，然后回到 7，此时访问位为 0，换出 7，换入

2。状态：$2_0^1, 0_1^0, 1_2^0$，缺页 4 次。

③0 进入，命中，将访问位置 1，指针不动。状态：$2_0^1, 0_1^1, 1_2^0$

④3 进入，1 换出，指针指向下一个页 2。状态：$2_0^1, 0_1^0, 3_2^1$，缺页 5 次。

⑤0 进入，命中。状态：$2_0^1, 0_1^1, 3_2^1$

⑥4 进入，2 换出，指针指向页 0。状态：$4_0^1, 0_1^0, 3_2^0$，缺页 6 次。

60．D。【解析】进程在内存中占 3 块（开始时为空）1、2、3 占满内存块，发生 3 次缺页；下一个 4 会发生缺页，4 置换 1，此时内存块有 2、3、4，下一个 1 会发生缺页，1 置换 2，此时内存块有 3、4、1；下一个 2 会发生缺页，2 置换 3，此时内存块有 4、1、2；下一个 5 会发生缺页，5 置换 4，此时内存块有 1、2、5；下一个 1、2 不会缺页；下一个 3、4、6 都会发生缺页；总共 10 次缺页。

61．C。【解析】操作系统为进程 A 分配了 4 个页框，当第一次访问页面时会发生缺页中断，所以当第一次访问页面 1、8、7、2 都会发生缺页中断。采用的是 LRU 最近最久未使用置换算法访问 1(缺页)、8(缺页)、7(缺页)、8、2(缺页)、7、2、1、8、3(缺页)、8、2、1、3、1、7(缺页)、1、3、7。发生缺页中断的次数为 6 次，选 C。

62．B。【解析】当为进程 A 分配页框数为 3 时，访问页面顺序 1(缺页)，3(缺页)，2(缺页)，1，1，3，5(缺页)，1，3，2(缺页)，1，5(缺页)。缺页率=6/12=50%。

当为进程 A 分配页框数为 4 时，访问页面顺序 1(缺页)，3(缺页)，2(缺页)，1，1，3，5(缺页)，1，3，2，1，5。缺页率=4/12=33%，选 B。

63．D。【解析】根据 FIFO 策略，最先置换出去的页面是最早进入的，最早进入的页框号是 3，所以按照 FIFO 策略，会将页框号为 3 的页面置换出去。根据 LRU 策略，最先置换出去的是最近最久未使用的页面，由引用时间可知，最近最久未使用的页框为 1 号页框，所以会将 1 号页框置换出去。根据 CLOCK 策略，会将引用位为 0 的页面置换出去，所以会将 2 号页框置换出去。选项 D 正确。

64．C。【解析】会发生 Belady 现象的算法：FIFO、CLOCK 以及改进的 CLOCK，不会发生 Belady 现象的算法：LRU、OPT，I、II 正确。

> $belady$ 现象是指局部置换算法中，发现某进程缺页率变高，于是操作系统给它又分了几个物理页，原以为缺页率的增高是由于工作集变大，物理页帧不够用，结果增加了物理页以后，缺页率不降反而升高。
>
> $belady$ 现象的出现，是由于置换算法将需要经常访问的页面置换了出去导致的。例如 $FIFO$ 算法，即便最开始进入内存的那个页面是最近以及未来要经常访问的，但是 $FIFO$ 还是选择把它换出去。
>
> $FIFO$ 选择替换最早进入内存的页面，没有考虑到这个页面未来是否可能会经常访问，所以导致了 $belady$ 现象。
>
> 类似的算法还有简单 $clock$ 算法，$clock$ 算法本质跟 $FIFO$ 一样，假如物理页帧数为 4 ，并且页面 $p1$、$p2$、$p3$、$p4$ 先后被访问并被载入物理页帧，从 $p4$ 被载入内存的时刻 t_1 开始，直到下一次发生缺页的时刻 t_2 为止这段时间内最经常访问的页是 $p1$ ，根据局部性原理，未来 $p1$ 有很大可能要继续访问。但是 $clock$ 算法给这四个页面都设置了访问位，而且很遗憾的是，这个访问位只有一个比特，它根本区分不了这个四个页面中谁被访问最频繁，这4个页面的访问位都是1，根据 $clock$ 算法，t_2 时候发生缺页，$clock$ 算法按照被载入内存的时间先后扫描所有页帧，将第一个访问位为0的页面置换出去，但是显然这四个页面的访问位都是1，所以 $clock$ 算法会将这4个页面的访问位依次置为0，然后循环扫描，将 $p1$ 置换出去。所以 $clock$ 算法是存在 $belady$ 现象的。
>
> $clock$ 算法的提出就是因为觉得 LRU 为了维持对访问局部性（谁是最近第一个被访问的、谁是最近第2个被访问的、…）的感知开销太大，LRU 需要经常修改页面链表，确保是按照最近访问时间排序。$FIFO$ 不需要费这种功夫，所以开销比较小，但是缺点是 $FIFO$ 感知不了访问局部性。
>
> $clock$ 算法是 LRU 和 $FIFO$ 的折中，但是还是偏向于 $FIFO$ ，它不像 LRU 那样能够区分页面的访问时间的先后顺序，它只能够区分有没有被访问过，至于在那些都被访问过的页面中，谁是最频繁被访问的、谁是最近被访问的，$clock$ 算法区分不了。所以导致它经过一轮扫描以后退化成为 $FIFO$ 。

如果进程的工作集都被调入了虚拟内存中，还是会发生缺页中断，且不会保持在较低水平。而如果进程的工作集都被调入物理内存中，进程的缺页率可以保持一个较低水平，III对，IV 错。答案是 C，I、II、III 正确，

65．B。【解析】计算当前工作集 WS：刚刚访问完最后的页面 5，此时往前数 5 个访问记录（含页面 5 本身），涵盖的访问子序列为最后 5 位：{4, 1, 4, 2, 5}。 去除重复项后，去重集合为 {1, 2, 4, 5}。所以此时工作集 WS = {1, 2, 4, 5}。

寻找历史最大驻留页面数：整个执行过程中，每个时刻的工作集就是当时的驻留集。我们滑动窗口看看哪个阶段去重后的不同页面最多：

- 访问到第 6 个元素 4 时，窗口为 {1, 2, 3, 2, 4}→页面有 {1, 2, 3, 4}（4 个）。
- 访问到第 7 个元素 1 时，窗口为 {2, 3, 2, 4, 1}→页面有 {1, 2, 3, 4}（4 个）。

纵观全局，没有任何一个滑窗片段能够凑齐 5 个完全不同的页面（因为数字总共才活跃那么几个）。因此历史最多同时驻留页面的数量为 4 个。

66．D。【解析】缺页中断的原因是当前访问的页不在主存，需将该页调入主存。此时不管主存是否已满(已满则先调出一页)，都要发生一次缺页中断。即无论怎么安排，n 个不同的页号在首次进入主存时必须要发生一次缺页中断，总共发生 n 次，这就是缺页中断的下限。虽然不同页号数为 n，小于或等于总长度 p(访问串可能会有一些页重复出现)，但驻留集 m<n，所以可能会有某些页进入主存后又被调出主存，当再次访问时又发生一次缺页中断的现象，即有些页可能会出现多次缺页中断。极端情况是每访问一个页号时，该页都不在主存，这样共发生了 p 次故障。所以无论对于 FIFO 或者 LRU 替换算法，其缺页中断的上限均为 p，下限均为 n。

67．C。【解析】A 正确，一旦文件被映射到进程的虚拟地址空间，进程就可以像访问普通内存数组一样，通过指针和内存访问指令（如 MOV）来直接读写文件内容，而不需要调用传统的 read()或 write()系统调用。

B 正确，内存映射文件是实现进程间共享数据（特别是大文件）的一种高效方式。多个进程可以将同一个文件映射到它们各自的地址空间。如果映射是共享的，那么一个进程对映射区域的修改对其他共享该映射的进程是可见的（取决于映射类型和同步机制）。

C 错误，当进程退出或关闭文件映射时，所有的被改动页面才会被写回磁盘文件，并非修改后立即写回。

D 正确，内存映射文件利用了操作系统的请求分页机制。当进程首次访问映射区域中的某个页面，而该页面尚未加载到物理内存时，会发生缺页中断。操作系统会响应该中断，从磁盘文件中读取相应的页面内容到物理内存，并更新页表，然后进程才能继续执行。这种按需调入的方式使得可以映射远大于物理内存的文件。

68．B。【解析】B 错误，内存映射文件利用了操作系统的请求调页（Demand Paging）机制。当文件被映射时，并不会立即将其全部内容调入物理内存。只有当进程实际访问到映射区域中的某个特定页面，而该页面尚未在物理内存中时，才会触发缺页中断，操作系统再负责将对应的文件部分从磁盘调入物理内存。这种按需调页的方式使得可以映射远大于可用物理内存的文件。

A 正确，传统的 I/O 操作（如 read()）会将文件数据从磁盘读入内核的页高速缓存，然后再从页高速缓存复制到用户进程的缓冲区。内存映射文件通过将文件内容直接映射到进程的虚拟地址空间，可以避免这次从内核空间到用户空间的额外数据复制，从而提高 I/O 效率。

C 正确，这是内存映射文件的核心概念。通过系统调用（如 mmap()），文件在磁盘上的内容被映射到进程虚拟地址空间的一个连续区域。之后，进程就可以像访问内存一样访问文件内容。

D 正确，当多个进程将同一个文件以共享模式映射到它们各自的虚拟地址空间时，它们实际上共享了物理内存中该文件的相同副本（通过页高速缓存）。一个进程对映射区域的修改可以被其他共享该映射的进程看到（取决于映射类型和同步）。这种直接的内存共享避免了传统 IPC 机制（如管道、消息队列）中数据需要通过内核进行多次复制的开销，因此可以提供一种高带宽的进程间通信方式，特别适合大量数据的共享。

69．A。【**解析**】只有 IV 不会影响缺页率。

I. 页置换算法会影响缺页率，例如，LRU 算法的缺页率通常要比 FIFO 算法的缺页率低。

II. 工作集的大小决定了分配给进程的物理块数，分配给进程的物理块数越多，缺页率就越低。

III. 进程的数量越多，对内存资源的竞争越激烈，每个进程被分配的物理块数越少，缺页率也就越高。

IV. 页缓冲队列是将被淘汰的页面缓存下来，暂时不写回磁盘，缺页是指访问的页未分配页框，在页缓冲或在磁盘，都叫缺页。所以队列长度会影响页面置换的速度，但不会影响缺页率。

V. 编写程序的局部化程度越高，执行时的缺页率就越低。若存储采用的是按行存储，则访问时就要尽量采用相同的访问方式，避免按列访问造成缺页率过高的现象。

70．A。【**解析**】由题目可以得出，大部分时间消耗在磁盘交换区，也就是说，物理内存太小，导致数据需要频繁的换进换出，因此使用容量更大的内存可以改进 CPU 利用率。此外 CPU 和 I/O 利用率都很低，说明 CPU 一次读取了太多的程序放入内存中，因此减少系统多道程序度数，使每个进程可以分配到更多页框，也可以改进 CPU 利用率。选I，II。

71．B。【**解析**】地址变换机构在拿到逻辑地址后，第一步必须用段内偏移量与段表项中的“段长”进行对比。如果 段内偏移 ≥ 段长，说明该地址请求已经超越了该段的合法边界，硬件立刻抛出越界异常，直接终止操作。

只有在未越界的前提下，硬件才会去检查“有效位（存在位）”。如果有效位为 0，说明该合法段当前不在内存中，此时抛出缺段中断，由操作系统从外存调入该段。

72．D。【**解析**】A、B、C 正确：大页的核心优势是减少页表层级/项数和提高 TLB 覆盖率（因为一个 TLB 表项代表的地址空间变大了）。缺点是内部碎片变大，且发生缺页时，从磁盘读入 2MB 数据的延迟远大于 4KB。

D 错误：无论页大小如何，页表的创建、分配、回收、权限设置以及缺页异常的处理，都必须由操作系统（软件）来管理。硬件 MMU 只负责“查表”和“转换”，绝不负责“建表”。

73．B。【**解析**】由于磁盘 I/O（读盘）相对于 CPU 速度慢得令人发指（，操作系统不可能让 CPU 在内核态死等。为了提高 CPU 利用率，内核在向磁盘发出 I/O 请求后，会立刻将当前缺页的进程 P 的状态由“运行态”改为“阻塞态”，将其挂入设备等待队列。随后，调度程序挑出另一个就绪进程 Q 投入运行。当磁盘把页面读入内存后，磁盘控制器发出“I/O 中断”，操作系统响应中断，把进程 P 的状态修改为“就绪态”，等待下一轮调度。故 B 项最符合标准。

74．C。【**解析**】页面大小为 4KB（12 位页内偏移）。剩下 20 位由一级页目录号（10 位）和二

级页号（10 位）平分。将 0040 8000H 展开为二进制：0000 0000 0100 0000 1000 0000 0000 0000;高 10 位（页目录号）= 0000 0000 01 B = 1；中间 10 位（二级页号）= 00 0000 1000 B = 8

定位二级页表基址：题目告知该地址对应的一级页目录项中存放的页框号为 00200H。因此，对应的二级页表在内存中的物理始址 = 00200H × 4KB = 0020 0000H。

计算目标页表项（PTE）的物理地址：PTE 物理地址 = 二级页表基址+ 二级页号 *页表项大小 =0020 0000H + 8 * 4 = 0020 0020H

75．C。【解析】选项 A 错误，说法过于绝对（“仅”字），除了缺页异常（越界访问、页面不在内存），越权异常（例如试图修改一个只读的页面，或者用户态试图访问内核态的页面）也会在地址转换时被硬件检测出来，并触发操作系统执行相应的异常处理程序。此外在某些采用软件管理 TLB 的架构（如 MIPS 架构）中，TLB 未命中时硬件并不会自动去查页表，而是直接触发一个“TLB 缺失异常”，交由操作系统去内存中查找页表项并填入 TLB。因此，操作系统并非只在缺页时才参与。

选项 B 错误，TLB 未命中时访问页表获取页框号后，不需要重新执行指令，而是继续当前的访问流程，因为这个过程并没有发生中断或异常，也就不会导致整个指令被打断后“重新执行”。

选项 C 正确，MMU 在进行地址转换时，会检查页表项中的存在位（有效位）。如果发现该位为 0，说明页面不在主存中，硬件会立即产生一个缺页异常信号，这是硬件的职责；CPU 响应异常，自动切换到内核态。OS 的缺页处理程序开始运行，它负责：查找空闲物理页框（分配页框） → 启动磁盘将页面读入（I/O 启动） → 在等待磁盘读取期间，将当前进程阻塞并切换其他进程运行（进程调度） → 调页完成后更新页表，并唤醒原进程，这是操作系统的职责。

选项 D 错误，权限检查是由 MMU（内存管理单元） 在进行地址转换的同时，根据 TLB 或页表项中的保护位/控制位完成的。如果发现越权（例如试图向只读的共享页面写入），MMU 会直接拦截该操作并触发异常，根本不会把物理地址发到总线去访问主存。

76．C。【解析】A 正确，当页表项中的“存在位”为 0 时，说明页面不在主存中。此时对应页表项中的存放位置记录的是页面在磁盘中的存储位置，操作系统必须读取这个地址才能知道去哪里加载数据。

B 正确： 缺页处理的核心目标就是为缺失的页面找一个物理页框。操作系统必须管理并检索空闲页框链表，拿出一个空闲页框；如果没有空闲的，还需要调用页面置换算法（如 LRU）腾出一个页框。

C 错误，缺页中断属于异常中的故障类型。对于故障类型的异常，CPU 硬件在响应中断、切换到内核态时，会自动将引发故障的当前指令地址（而不是下一条指令的地址）即 PC 值压入内核栈或保存在特定的硬件寄存器中。因此，修改 PC 以便重新执行指令完全是硬件底层机制保障的，不属于操作系统的操作范畴。 操作系统完成物理页框的分配、磁盘读取和页表更新后，只需要执行一条“中断返回”指令。硬件会自动弹出之前保存的 PC 值。由于硬件当初保存的就是发生缺页的那条指令的地址，所以 CPU 自然会重新执行该指令。

77．C。【解析】I 正确，因为 P1 的工作集大于其可用页框数，必然导致频繁的页面置换和缺页，从而引发局部抖动 。II 错误，这是对局部置换策略的根本误解。局部置换意味着 P1 只能在自己

的 2 个页框中进行替换，它不能去问 P2 借或抢页框。III 正确，由于是局部置换，P1 的行为不会直接影响 P2 的页框集合，因此 P2 的缺页率保持不变。

78．C。**【解析】**【解析】A 错误：调用成功时，操作系统只是在进程的进程控制块（PCB）中记录了一块虚拟内存区域，并没有立刻在物理内存中分配空间，也没有读盘。

B 错误：页表项（PTE）只能映射“虚拟地址”到“物理物理页框”，绝不能直接映射到磁盘块号。

C 正确：当进程第一次尝试通过指针去访问这块虚拟地址时，硬件 MMU 查页表发现有效位为 0，当即触发缺页异常。操作系统在缺页处理程序中才去分配物理页框，并读取对应的磁盘文件块进来。

D 错误：mmap 区域的页面管理完全复用了虚拟存储系统的页面高速缓存以及虚拟内存置换算法（如 LRU/CLOCK）等机制。

79．B。**【解析】**选项I错误，TLB 的主要作用是减少地址转换时间，不直接影响缺页率。选项II正确，更好的置换算法可以降低系统缺页率。选项III正确，增加页框可以容纳更多工作集。选项IV正确，操作系统介入，主动降低多道程序度（将部分低优先级或阻塞的进程挂起并换出到外存），可以腾出大量物理页框。把这些空出来的页框分给剩余的活跃进程，能显著降低系统的总体缺页率。答案为 B。

第 4 章

文件管理

【答案速对】

DBBDC BBCAB CBDAB BAABB

DCCDB BBCBD BDBAB BCCBB

DCADA DCCAC DCACB DACBD

BBADB CCDCC DBDCD ABDCB

B

1．D。【解析】A 正确。文件的逻辑结构是用户可见的文件组织方式，例如文本文件被视为字符流（流式文件），而数据库文件可能由多条记录组成（记录式文件）。 这种逻辑组织与文件数据块在磁盘上如何具体存放（物理结构，如连续、链接或索引存放）是分离的。

B 正确，文件名是用户用来识别和访问文件的符号名称。 文件扩展名是附加在文件名后的部分，用于帮助用户和操作系统判断文件的类型和可能的用途。

C 正确，文件的“只读”属性确实限制了对文件内容的修改。但是，通常情况下，**文件所有者**或具有足够权限的**系统管理员（超级用户）** 仍然有权修改文件的属性，包括取消“只读”属性，或者直接对文件进行写操作（尽管可能需要先更改权限）。

D 错误，索引节点（inode）不包含文件名，文件名存储在目录文件的目录项中，每个目录项将文件名与 inode 号关联。

2．B。【解析】I正确，FCB 中必然包含文件的基本信息，例如文件名（用于用户识别）、文件在外部存储设备上的物理位置（如起始盘块号、占用的盘块数或索引块指针）、文件的逻辑结构（如流式或记录式）、文件的物理结构（如连续、链接、索引）以及文件长度等。

II 正确，为了实现文件保护，FCB 中会记录文件的存取控制信息，例如文件所有者是谁，以及不同类别的用户（如文件所有者、同组用户、其他用户）对该文件具有的访问权限（如读、写、执行等）。

III 正确，FCB 中通常还会包含文件的使用历史和状态信息，例如文件的创建日期和时间、最后一次被修改的日期和时间、最后一次被访问的日期和时间，以及当前文件是否被打开、被哪些进程打开、是否有进程锁定了该文件等。

IV 错误，FCB 中存放的是文件的描述信息（元数据），例如文件数据存储在磁盘上的哪个位置

（通过物理地址或索引块指针指示），但不包含文件内容本身的数据块。文件内容的数据是存储在磁盘的数据区中的，FCB 的作用是指向这些数据。如果 FCB 包含文件内容本身，那么对于大文件，FCB 将会变得异常庞大，不便于管理和快速检索。

3．B。【解析】B 正确，把文件控制块的有序集合称为文件目录，即一个文件控制块就是一个文件目录项。

A 错误，创建文件时建立对应的文件控制块，而不是打开文件时创建的。

C 错误，文件控制块与文件一一对应。

D 错误，文件打开表指针在进程控制块内，和文件控制块没有关系。FCB 有三类信息：基本信息、存取控制信息及使用信息。

4．D。【解析】A 错误，文件目录是由一系列目录项组成的，是文件控制块 FCB 的有序集合。在引入索引节点的文件系统中，每个目录项通常包含文件名和指向该文件对应索引节点的指针或编号，而不是索引节点本身。索引节点通常集中存放在磁盘的特定区域（索引节点区）。

B 错误，文件目录和索引节点之间有非常紧密的联系。目录项通过索引节点号（或指针）与索引节点关联起来，实现了文件名到文件具体属性和数据位置的映射。

C 错误，文件目录（或更准确地说是目录文件中的目录项）通常包含文件名和指向索引节点的指针或索引节点号，而不是索引节点本身。索引节点包含了文件的详细元数据。

D 正确，索引节点（inode）是文件系统中用于存储文件元数据的核心数据结构。它包含了关于文件的各种控制信息。

5．C。【解析】A 正确，索引节点是文件的元数据集合，它详细描述了文件的各种属性，包括文件类型、所有者 ID、用户组 ID、文件的访问权限（读、写、执行）、各种时间戳（如创建时间、最后修改时间、最后访问时间）、文件大小以及指向存储文件内容的数据块的指针（直接指针、间接指针等）。每个文件在文件系统中都有一个唯一的索引节点。

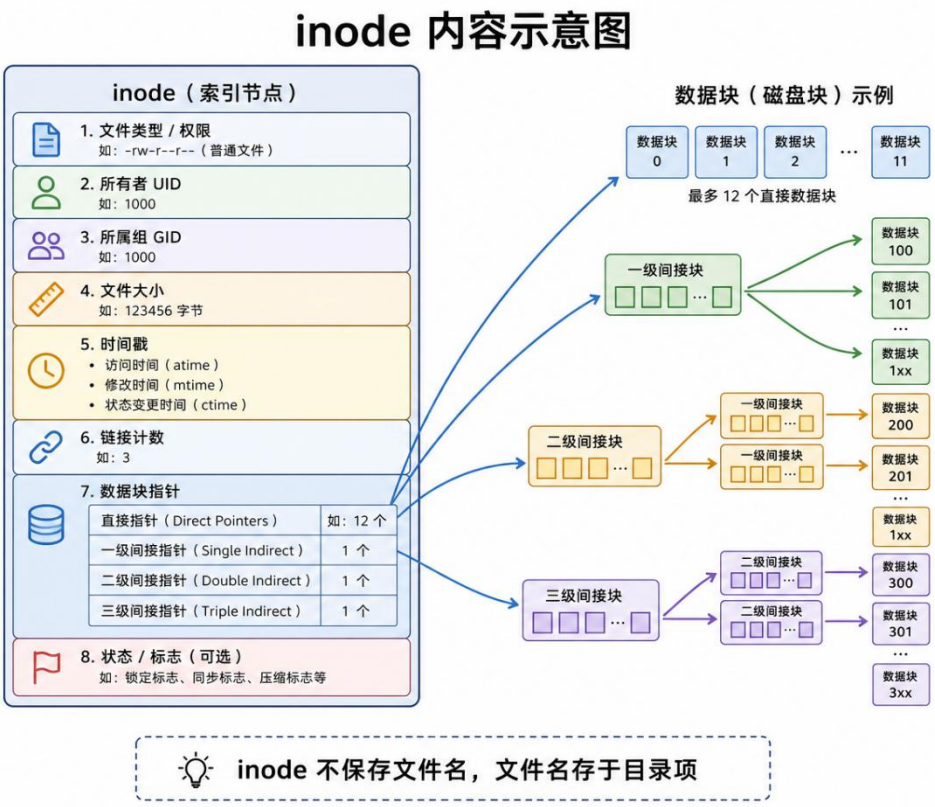
B 正确，这种设计将文件名与文件的其他描述信息分离。目录项的主要作用是建立文件名到索引节点的映射。通过这种方式，可以减少目录项的大小，提高目录检索效率。

C 错误，硬链接是指多个不同的文件名指向同一个索引节点。这意味着这些文件名共享同一个文件元数据（包括指向数据块的指针）。因此，它们实际上是同一个文件的不同入口。索引节点中通常会有一个链接计数器，记录有多少个目录项指向该索引节点。并不会为每个硬链接创建独立的索引节点。

D 正确，索引节点作为文件系统持久化的一部分，存储在磁盘上。当文件被打开或访问时，操作系统会将该文件的索引节点从磁盘读入内存，以便快速获取文件的元数据和定位数据块，减少后续操作的磁盘 I/O 次数。

6．B。【解析】inode 的数量是在文件系统创建或格式化时确定的，并非动态调整的，说法I错误。每个文件系统都有自己的 inode 表，不同文件系统的 inode 表是独立的，说法III错误。因此答案为选项 B。

7．C。【解析】磁盘索引节点：是指存放在磁盘上的索引节点。每个文件有一个唯一的磁盘索引节点，包含内容如下图所示。



文件的磁盘索引节点中存放文件的固有信息，包括 A 链接计数，B 物理位置和 D 访问权限，而文件的读写指针是单个进程对文件的访问相关的，位于用户级打开文件表中。

8．C。【解析】文件第 518KB 位于第 518 个磁盘块中，直接索引负责第 0-7 块磁盘块管理，2 个 1 级索引负责第 8-519 块磁盘块的管理，所以第 518 块磁盘属于 1 级间接索引管理，因此需要先将一级索引块放入内存，再把数据块放入内存，需要两次磁盘 I/O，在内存中修改后再通过一次 I/O 放回磁盘，共 3 次磁盘 I/O。

9．A。【解析】索引节点号编号占 8B，可表示 2^64 个文件。由于树形目录结构中不同目录下文件的文件名可相同，因此在计算文件系统创建文件的上限时只需考虑索引节点个数，其它条件都是对考生的干扰，A 选项正确。

10．B。【解析】引入索引节点后，每个目录项只需要存放文件名和索引节点编号即可。1 个盘块可以存放 1KB/(4B+4B)=128 个目录项，640 个目录项共需占用 640/128=5 个盘块，因此找到对应的目录项平均需要启动(1+2+3+4+5)/5=3 次磁盘；得到目录项后，还需要再启动一次磁盘将对应的索引节点读入内存，故平均需要启动 4 次磁盘。

11．C。【解析】第一版中，目录大小为 256x64B/512B=32 个磁盘块，故检索一个文件平均需要访问(32+1)/2=16.5 次。第二版中，目录大小为 256x(12B+4B)/512B=8 个磁盘块，故检索一个文件的索引节点编号平均需要访问(8+1)/2=4.5 次;读出索引节点编号后，还需访问一次磁盘读出索引节点的内容，因此检索一个文件平均需要访问 5.5 次。因此选 C。

12．B。【解析】A 错误，虽然内存中确实只有一份该文件的内存 inode（被多个进程共享），但文件读写指针不保存在 inode 中，如果保存在 inode 中，多个独立进程对同一个文件的读写就会互相

干扰。读写指针保存在系统打开文件表的表项中。C 错误：内存 inode 比磁盘 inode 包含更多的动态信息，例如：引用计数（记录当前有多少个进程打开了该文件）等。D 错误：为了提高磁盘 I/O 效率，内存 inode 中的文件大小等属性修改后，通常会先缓存在内存中（变为“脏”inode），由操作系统周期性地写回磁盘，或者在关闭文件时写回，而不是“立即同步更新”。

13．D。【**解析**】D 选项，标准的文件删除操作通常只是将文件占用的磁盘块标记为可用，并移除目录项和回收 FCB/inode。操作系统一般不会立即去物理地擦除或格式化这些磁盘扇区中的数据内容。这样做开销很大，会显著降低删除操作的效率。数据的物理清除通常需要专门的安全删除工具或在后续磁盘块被重新分配给新文件并写入新数据时才会被覆盖。

14．A。【**解析**】系统调用的标准流程：陷入内核(⑤) → 检查参数合法性(②) → 发现缺块，发起 I/O 并阻塞进程(①) → 硬件中断唤醒进程(③) → 内核将数据从内核缓冲区拷贝到用户空间并返回(④)。

15．B。【**解析**】应该是链接计数；

16．B。【**解析**】I 正确：read 系统调用的参数传入的是 fd，内核必须校验它是否打开以及权限。II 错误：按名搜索目录是在 open 系统调用时完成的，read 不涉及目录搜索。 III 正确：现代操作系统读取磁盘文件时，通常都会先读入内核的页缓存，然后再通过 CPU 拷贝到用户进程的 buffer 中。

17．A。【解析】如果该文件不是首次被打开的话，则不需要从外存载入 inode；其次，即使是第一次打开该文件，也是要将该 inode 所在的磁盘块一起读入内存，因为磁盘读取操作以“块（block）”为单位，而不是以“inode”为单位。即使你只需要一个 inode，系统也必须读取整个块，因为磁盘硬件无法直接读取块内的某一部分。

磁盘 I/O 无法按字节粒度读取：磁盘硬件只能以块为单位读取或写入数据，无法直接读取块内的某个字节范围。

虽然读取整个块会多读一些数据，但现代文件系统通过 缓存机制（如 inode 缓存）避免重复读取。例如：如果后续访问同一磁盘块中的其他 inode，可以直接从内存缓存中获取，无需再次读取磁盘。

18．A。【**解析**】I错误；read 系统调用需要的参数是 read(int fd, void *buf, size_t count)，必须打开文件获得 fd 后才能对文件执行读写和关闭操作，因此不需要文件名，需要的是文件描述符；读写指针位于进程打开文件表中，除了文件描述符 fd 和期待读取的字节数以外，还需要指定读入用户空间缓冲区的指针；

II 正确；假设进程 P1 执行 read()系统调用，由于该文件的数据不在内存，因此需要通过磁盘 I/O，在对 DMA 控制器设置完传送参数后，便会主动阻塞自己，随后调度程序根据调度策略选择进程 P2 上处理机，进程 P2 执行的过程中，DMA 正在将指定读写区域数据所在磁盘块依次读入内核缓冲区，I/O 完成后，DMA 控制器便会发出中断请求，进程 P2 响应该中断请求之后，在该中断服务程序中唤醒进程 P1，如果进程 P1 优先级比当前进程优先级高且为抢占式调度，那么在中断返回过程中，便会触发调度让进程 P1 上处理机执行；

III 中“直接”有误，需要先将指定读写区域所在磁盘块读入内核空间，然后再根据读写指针和读写字节数将所需数据从内核缓冲区复制到用户空间的缓冲区 buf 中；

IV 错误，看 III 的解释；

V 错误，读取成功，返回实际读取的字节数

因为磁盘、硬盘、U 盘等存储设备基本都是 Flash 块设备，块设备硬件本身有读写限制等特征，块设备是以一块一块为单位进行读写的（一个块包含多个扇区，而一个扇区包含多个字节），一个字节的改动也需要将该字节所在的 block 全部读取出来进行修改，修改完成之后再写入块设备中，所以导致对块设备的读写操作非常不灵活 ；而内存可以按字节为单位来操作，而且可以随机操作任意地址数据，非常地很灵活，所以对于操作系统来说，会先将磁盘中的静态文件读取到内存中进行缓存，读写操作都是针对这份动态文件，而不是直接去操作磁盘中的静态文件，不但操作不灵活，效率也会下降很多，因为内存的读写速率远比磁盘读写快得多。

19．B。【解析】A 错误：用户级打开文件表（进程打开文件表）仅保存 fd 到系统级表项的映射指针，不包含读写指针和打开模式。B 正确：系统级打开文件表（打开文件描述表）为每次 open() 调用创建一个表项，记录该次打开的状态，包括打开模式（访问权限）、读写偏移量、状态标志以及指向内存 inode 的指针。C 错误：内存索引节点不包含文件名。文件名仅存在于目录项中，一个 inode 可以有多个文件名（硬链接）。D 错误：目录项只包含文件名和磁盘 inode 编号，不包含文件的访问权限（权限在 inode 中），也不包含指向系统级表的指针。

20．B。【解析】I 正确，硬链接 f1、f2 指向同一个 inode，访问权限保存在 inode 中，因此权限相同；修改其中一个的权限，另一个也同步变化；II 正确，f2 由于是 f1 的硬链接文件，因此二者共享同一个内存索引结点；III 正确，fd1 和 fd2 的值分别为两个进程打开文件表中的索引值，因此有可能相同；硬链接计数值和访问计数值不是一回事，二者无关，因此IV错误；

21．D。【解析】文件系统按名查找打开文件，文件被打开后，系统会为每个打开的文件分配一个文件描述符，用来标识该文件。文件描述符是一个非负整数，可以用来对文件进行读写、定位等操作。因此，对文件的访问采用文件描述符。

22．C。【解析】关闭文件不是删除文件，不会清除磁盘上文件数据块的内容。

23．C。【解析】A 错误。 进程调用 open() 时，系统会查找目录并将对应的索引节点（inode）加载到内存。如果该文件的 inode 已经被其他进程打开过并存在于内存索引节点表中，系统不会新增表项，而是直接复用已有的表项，并将其引用计数加 1。因此“必然新增”是错误的。B 错误。 不同的进程各自独立调用 open() 打开同一个文件时，内核会在系统级打开文件表中为每一次调用独立创建一个新的表项。每个表项都维护各自独立的读写偏移量和状态标志。因此它们不共享系统级表项，读写偏移量也是相互独立的。C 正确。 进程调用 fork() 时，子进程会获得父进程用户级打开文件表的一个副本。这意味着父子进程中相同的文件描述符（fd）会指向系统级打开文件表中的同一个表项。由于读写偏移量（file offset）是维护在系统级打开文件表项中的，因此父、子进程对该文件的读写是共享同一个偏移量的，任何一方修改偏移量（如 read、write 或 lseek）都会立刻影响另一方。D 错误。 硬链接本质上是多个不同的目录项指向磁盘上的同一个索引节点（inode）。内核在解析路径后，最终访问的是同一个磁盘 inode。如果该 inode 已在内存中，则直接复用；即使不在内存中，调入内存后也只有一个内存 inode 表项，绝不会因为硬链接路径的不同而分配多个内存 inode。

24．D。【解析】文件控制块是操作系统为每个文件建立的唯一数据结构，用于管理、控制和按名存取文件，其中包含了文件的全部属性，I、II正确；

文件控制块中通常包含了三类信息，即文件基本信息、存取控制信息与使用信息，III正确；

文件被首次打开时，只会将该文件对应的 FCB 读入内存，并在打开文件表上做相应记录，并不会将完整的文件内容读人内存，IV 正确。故正确答案为 D。

25．B。【解析】当多个进程共享同一个文件时，在系统打开文件表中只会有一个表项包含该文件的属性，每个打开该文件的进程都会在其用户打开文件表中拥有一个表项，该表项包含有该进程的文件访问信息，例如当前读写位置等。因此，各进程的用户打开文件表中关于该文件的表项内容是不同的，但系统打开文件表中的表项是相同的。在进程关闭该文件时，系统会删除相应的用户打开文件表项，若此时无其他进程打开该文件，则系统会删除该文件在系统打开文件表中的表项。

26．B。【解析】关闭文件的系统调用格式为 close(fd)，调用成功返回 0，失败则返回-1，具体操作如下：

- 根据文件描述符 fd 找到“进程打开文件表”中对应表项并删除；
- 系统级打开文件表中的对应表项的访问计数（打开计数）-1
  - 若-1 后不为 0，说明还有其他进程正在使用该文件，直接返回即可；
  - 若-1 后为 0，则需要删除该表项，如果对应的内存索引节点没有修改，直接释放即可，若修改可以根据 OS 的策略决定是否将该 inode 写回外存；

27．B。【解析】进程打开文件时，需要先对比 open()系统调用申请的访问权限(参数 flag)与文件允许的权限，若前者不超过后者权限，则打开文件成功，并且在该进程打开文件表中记录进程打开文件时申请的权限；打开文件之后，用户对文件的操作不能超过自身打开文件时申请的权限，因此II错误，如果打开文件时只申请了读权限，那么是不能对该文件进行写操作的；

28．C。【解析】口令指用户在建立一个文件时提供一个口令，系统为其建立 FCB 时附上相应口令，同时告诉允许共享该文件的其他用户。用户请求访问时必须提供相应的口令。这种方法时间和空间的开销不多，缺点是口令直接存在系统内部，不够安全。

密码指用户对文件进行加密，文件被访问时需要使用密钥。这种方法保密性强，节省了存储空间，不过编码和译码要花费一定的时间。

口令和密码都是防止用户文件被他人存取或窃取，并没有控制用户对文件的访问类型，即口令和密码无法限制用户对文件进行读取、写入、删除或执行。

29．B。【解析】文件的逻辑结构(file logical structure)，是指从用户角度出发所观察到的文件组织形式，即文件是由一系列的逻辑记录所组成的，是用户可以直接处理的数据及其结构，它独立于文件的物理特性，又称为文件组织(file organization)。对应的文件通常称为逻辑文件。

文件的物理结构，又称为文件的存储结构，是指系统将文件存储在外存上所形成的一种存储组织形式，是用户所看不见的。文件的物理结构不仅与存储介质的存储性能有关，而且与所采用的外存分配方式也有关。无论是文件的逻辑结构，还是文件的物理结构，都会影响系统对文件的检索速度。

30．D。【解析】直接文件，是指可根据给定的关键字直接获得指定记录的物理地址，并不是直接

记录了文件的物理地址。换言之，直接文件的关键字本身决定了记录的物理地址。这种由关键字（的值）到记录的物理地址的变换，被称为键值变换（key to address transformation）。哈希（Hash）文件是目前应用最为广泛的一种直接文件。它利用 Hash 函数（或称散列函数）可将关键字变换为相应记录的地址。但为了实现文件存储空间的动态分配，利用 Hash 函数所求得的结果通常并不是相应记录的地址，而是指向某一目录表相应表目的指针，该表目的内容指向了相应记录所在的物理块。

31．B。【**解析**】文件的顺序存取是指按照逻辑顺序依次存取，B 选项正确。C 选项错误，文件的物理块号不一定相邻。

32．D。【**解析**】对于大型文件且需要高速随机访问的场景（如视频点播或大型数据库），多级索引分配通常具有更好的随机访问性能。多级索引结构设计的目标就是为了快速定位到大型文件中任意数据块的位置，通常只需要少数几次磁盘访问（读取必要的索引块）。

让我们来分析其他选项：

【2013-24】的真题考察了 A 连续结构，但注意真题题目中制定了 CD-ROM 设备，因为 CD-ROM 是只读的，文件可以预先优化为连续存放以达到最佳随机播放性能。B 链式结构无法实现快速随机访问。直接索引能够支持极高随机访问速度与文件的修改，但是在大型文件下索引表占用的空间较大。

33．B。【**解析**】I错误，连续分配方式不支持文件长度可变，支持随机访问

II 正确，索引分配方式既支持文件长度可变，也支持随机访问

III 正确，隐式链接方式中每个盘块最后的区域用于存放下一个盘块的地址，如果盘块损坏，后续的盘块都将丢失

IV 错误，混合索引方式为文件分配磁盘空间，由于不同记录需要使用的索引层级不同，读取记录需要的磁盘 I/O 次数不同，例如，A 记录通过 3 级索引找到，B 记录通过 1 级索引找到，磁盘 IO 次数就相差 2 次。

综上所述，答案选 B。

34．A。【**解析**】I、IV正确。

I 正确，记录是文件内独立的最小信息单位，一个文件中记录的长度也可以不等，比如变长记录。

II 错误，文件的控制信息，即元数据（如文件名、大小、权限、物理位置指针等），通常存储在文件控制块（FCB）或索引节点（inode）中。FCB/inode 通常存放在文件系统的特定区域（如目录区或索引节点区），与文件内容实际存储的数据块是分开的。这样做便于管理和快速获取文件属性，而无需读取整个文件内容。

III 错误，文件的物理结构确实有顺序式（连续分配）、链接式和索引式。但是，文件分配表（FAT）是显式链接分配方式中使用的一种数据结构，它记录了文件各盘块之间的链接关系以及盘块的分配状态。它本身不是一种索引结构，更不是单级索引式文件物理结构。索引式文件物理结构是指为文件建立索引块来存放数据块地址。

IV 正确，对于非常大的文件，单级索引表本身可能会变得很大。而多级索引会增加访存次数，导致效率下降。

35．B。【**解析**】顺序结构的文件必须为其分配连续的存储空间，反复增删文件后会产生外部碎

片，选择 B 选项。

36．B。【解析】索引分配机制中，当新建一个大小为 0 的空文件，操作系统只需为该文件分配一个文件控制块（FCB/inode）来记录元数据即可（索引信息通常在 inode 中）。此时，FCB/inode 中指向实际数据块或索引块的指针置为空（NULL）。操作系统绝对不会立即在磁盘数据区为其强行分配一个完整的物理索引块来存放“零个指针”，那样会造成极大的磁盘空间浪费。只有当文件真正开始写入数据时，系统才会按需分配物理索引块和数据块。

选项 C：隐式链接分配随时可以在链表尾部申请空闲块并挂接上去，支持动态增长且没有外部碎片。但一旦中间某个磁盘块出现物理损坏（坏道），该块内的指针也会随之丢失，导致链条断裂，后续的所有数据块都将无法找回

选项 D ：在隐式链接分配中，每个物理块需要拿出几个字节（如 4B）存放指向下一块的指针。如果物理块大小是 512B，那么留给用户数据的可用空间就变成了 508B。这会导致用户的逻辑记录无法与物理块对齐，读取和计算都极其麻烦。显式链接分配（FAT）将所有的链接指针从物理块中抽离出来，集中存放在内存中的“文件分配表（FAT）”里。这样物理块的 512B 可以 100% 用来存放用户数据，完美解决了逻辑块与物理块大小不一致的问题，同时还支持了在内存中的随机查找。

37．C。【解析】64GB 的硬盘可划分磁盘块 64GB/4KB=16M 块=2^24 块，盘块号位数即为 24 位，占 3B，FAT 的长度为 3Bx16M=48MB，选择 C。

38．C。【解析】连续分配只有 1 个起始地址和长度。链接分配（隐式）只有首尾指针。索引分配需要为每一个文件建立一个包含所有数据块物理地址的“索引表”。如果文件很大，索引表本身也会占用好几个磁盘块。这就是典型的“空间换时间”，增加了额外的存储空间开销。A、B、D 均是错误的描述。

39．B。【解析】一级索引分配方法时文件长度为 10*(4KB/4B)*4KB=40MB，二级索引分配方法时文件长度为 10*(4KB/4B)^2*4KB=40GB，因此选 B。

40．B。【解析】12 个直接索引指针对应 12 个硬盘块，即文件的前 12x4KB=49152B，9876B<49152B 故位于直接索引指针对应的块中，只需读取磁盘 1 次即可得到文件内容;一级索引指针对应 4KB/4B 块，即 1024x4KB~4000000B，543210B<49152B+4000000B，故需读盘两次，第次得到文件地址，第二次得到文件内容。因此选 B。

41．D。【解析】分区最大长度等于 FAT 表项的寻址范围，FAT 表项中有 30 位用于表示块号，因此分区最大长度为 2^30x4KB=4TB。4GB 分区一共 4GB/4KB=2^20 个块，每一块对应一个 FAT 表项，故 FAT 表共 2^20x4B=4MB。因此选 D。

42．C。【解析】C 错误，文件分配表（FAT）是整个文件系统卷的核心数据结构，它记录了所有盘块的分配状态和文件链。为了提高文件系统的整体性能和快速响应所有文件操作，FAT 通常在文件系统挂载时就被全部或大部分读入内存，并驻留在内存中，而不是在打开单个文件时才读入。如果每次打开文件都去读 FAT，磁盘 I/O 会非常频繁，效率低下。

A 正确，在显式链接（如 FAT 文件系统）中，文件所有盘块的链接信息都集中存放在文件分配表（FAT）中。当需要访问文件的某个特定盘块时，可以通过查找 FAT 表来直接定位到该盘块，而不需要

像隐式链接那样从头开始顺序读取每个盘块的指针。虽然不是像索引分配那样直接，但相比隐式链接，它确实支持了更有效的随机访问，并加快了访问速度。

B 正确，FAT 表中的每个条目对应一个磁盘块。通过在 FAT 表项中使用特殊标记（例如，-2 表示空闲，-1 或 EOF 表示文件结束，其他值为下一盘块号），不仅可以链接属于文件的盘块，还可以标识出哪些盘块是空闲未被使用的。因此，FAT 表本身就可以作为一种管理外存空闲空间的数据结构。

D 正确，FAT 的逻辑结构就是一个线性表，其表项序号对应磁盘的物理盘块号。在内存中，这种结构非常适合用一个数组来实现，数组的下标可以直接或通过简单计算对应到盘块号，数组元素的值则表示该盘块的下一个链接盘块号或其状态。

43．A。【**解析**】420000/1024=410 余 160，因此逻辑块号为 10+256<410，通过二次间接索引，在第 11 个地址中可得到一次间址，由此得到二次间址，再找到物理块号，其块内偏移地址为 160。

44．D。【**解析**】第 22 个逻辑记录对应第 5(22x100/512=4 余 152)个物理块，即读入第 5 个物理块的数据，由于文件采用的物理结构是链接文件，因此需要从目录项所指的第一个物理块开始读取，依次读到第 4 块才得到第 5 块的物理地址，然后将第 5 块的内容读入内存(启动了 5 次)，修改完后，写回磁盘(启动了 6 次)。

45．A。【解析】原 FCB：128B/项，1KB(1024B)/块 → 每块存 8 项。10000 个文件需 10000/8=1250 块。最坏情况读 1250 次。

新目录项：16B/项，1KB/块 → 每块存 1024/16=64 项。10000 个文件需 ⌈10000/64⌉=157 块（注意向上取整）。最坏情况读 157 次。减少次数 = 1250−157=1093 次。

46．D。【解析】文件的索引节点中共有 16 个块地址项。为了使单个文件的长度达到最大，在理想状态下，这 16 个块地址项应该全部作为一级间接地址使用。计算最大块数：1 个一级间接地址指向一个索引块，该索引块可容纳 512 个磁盘块地址（即对应 512 个数据块）。16 个块地址项全部作为一级间接地址时，总共可以指向的数据块数量为：16 * 512 = 8192 个数据块。计算最大文件大小：每个磁盘块大小为 1024B（1KB），因此文件的最大长度为：$16 \times 512 \times 1024\text{B} = 2^4 \times 2^9 \times 2^{10}\text{B} = 2^{23}\text{B} = 8\text{MB}$

47．C。【**解析**】设置当前工作目录的主要作用是为用户提供一个相对的参照点。当用户需要访问当前目录或其子目录下的文件时，可以使用相对路径名，这比每次都输入从根目录开始的完整绝对路径名要简洁得多，从而方便了用户操作。由于路径名变短，系统在解析路径和查找文件时，搜索范围也可能相应缩小（从当前目录开始查找），从而间接提高了文件定位（检索）的速度。

48．C。【**解析**】A 正确，目录是操作系统中用于组织和管理文件的一种特殊文件类型，它可以包含文件和其他目录，是文件系统中的一个重要组成部分。

B 正确，在目录中可以创建子目录，并在其中存储其他文件或目录，这样可以使文件系统中的文件组织更加清晰和有序。

C 错误，在引入索引节点（inode）的文件系统中，目录项通常包含文件名和与该文件名对应的索引节点号。目录本身是一个包含这些目录项的文件。索引节点是存储文件元数据（如文件大小、权限、物理位置等）的数据结构，它们通常集中存放在磁盘的特定区域（索引节点区）。因此，目录是文件名到

索引节点号的映射集合，而不是索引节点本身的有序集合。

D 选项，目录中的文件可以通过文件名或路径名来唯一标识，文件名是文件在目录中的名称，而路径名是文件在文件系统中的完整路径。

49．A。【解析】绝对路径（/dirA/dirB/dirC/file.txt）： 根目录 / 在内存。查内存中根目录，得到 dirA 的磁盘块号，发出 I/O 读入 dirA 数据块（第 1 次）。查 dirA，得到 dirB 的磁盘块号，发出 I/O 读入 dirB 数据块（第 2 次）。查 dirB，得到 dirC 的磁盘块号，发出 I/O 读入 dirC 数据块（第 3 次）。查 dirC，找到 file.txt 的 FCB，解析完成。总共 3 次。

相对路径（dirC/file.txt）： 当前工作目录 /dirA/dirB/ 所在的块已在内存。直接查内存中的当前目录块，得到 dirC 的磁盘块号，发出 I/O 读入 dirC 数据块（第 1 次）；查 dirC，找到 file.txt 的 FCB，解析完成。总共 1 次。 因此答案选 A。这也完美解释了为什么设置“当前工作目录”能加快检索速度（减少了从头遍历的 I/O 次数）。

50．C。【解析】A 正确，这是树形结构的基本定义。每个节点（目录或文件）有且仅有一个直接前驱（父目录），根节点除外。这种结构保证了系统中每个文件或目录都具有唯一的从根目录开始的路径。

B 正确，为了方便用户操作，文件系统通常支持两种路径名：绝对路径名（从根目录开始的完整路径）和相对路径名（从当前工作目录开始的路径）。

C 错误，树形目录结构的主要优点之一就是允许在不同的目录下存在同名文件。文件名的唯一性仅需在同一父目录内得到保证。例如，用户 A 的目录下可以有一个名为 "report.txt" 的文件，用户 B 的目录下也可以有一个同名的 "report.txt" 文件，它们通过不同的路径名（如 /user/A/report.txt 和 /user/B/report.txt）来区分。

D 正确，虽然严格的树形结构要求每个节点只有一个父目录，但为了实现文件共享，操作系统通常会引入链接机制。硬链接使得多个目录项可以指向同一个文件的索引节点（inode），而符号链接（软链接）则创建一个特殊的文件，其内容是指向另一个文件或目录的路径。这些机制使得一个文件在逻辑上可以出现在文件系统的多个位置。

51．D。【解析】A 正确，创建一个新目录时，系统会在其父目录中创建一个指向新目录的条目（目录项），并且会为这个新目录本身分配存储空间，用于存放它将来包含的文件和子目录的信息。

B 正确，对于空目录，删除操作通常比较直接：从其父目录中移除对应的目录项，并回收该目录文件自身所占用的存储空间。

C 正确，“更改当前目录”（如 cd 命令）是一个针对特定进程的概念，它改变了该进程在解释相对路径名时的起点。这个操作不涉及文件系统中文件或目录的实际物理位置或它们之间的链接关系的改变。

D 错误，列出目录内容（即读取目录文件中的文件名和可能的属性列表）通常只需要用户对该目录文件具有读权限（Read permission）。写权限（Write permission）通常是用于在该目录中创建、删除或重命名文件和子目录的。执行权限（Execute permission）对于目录来说，通常意味着允许用户进入（cd 到）该目录。因此，仅列出目录内容一般不需要写权限。

52．C。【解析】A 错误，打开文件是将文件的控制管理信息从辅存读到内存；

B 错误，对文件访问由用户访问权限和文件属性共同限制；

D 错误，为了防止系统故障对文件的破坏，可以采用建立副本和定时转储的方法来保护文件，而存取控制矩阵是防止由人为因素对文件造成的破坏。

53．A。【解析】硬链接允许一个文件在文件系统中有多个有效路径名，因为硬链接和原始文件共享同个 inode 号，本质上是同一个文件，但它们在不同的位置(即有效路径名)。软链接相当于文件的快捷方式，并非原文件。而动态链接和静态链接是程序编译相关的术语，与文件路径名无关。

54．C。【解析】硬链接（H）和原文件（F）是平等的，它们在底层的目录项里指向了同一个 inode。因此对 H 追加数据，实际上就是对那个共享的 inode 及物理块追加，F 也能立刻看到变成了 6KB。软链接（S）相当于 快捷方式，它有自己独立的 inode 和数据块，它的数据块里存的是 F 的路径字符串（选项 D 错误）。因为 F 和 H 共享 inode，删除 F 只是删除了 F 的目录项，硬链接计数减 1（变成 1），物理数据不被释放，读取 H 仍然是 6KB（选项 B 错误）。但是，软链接 S 里面存的是 F 的名字，F 被删后，S 变成了无效链接，再去读取 S 就会报找不到文件的错误（选项 C 正确，A 错误）。

55．B。【解析】I错误，软链接（f2）是一个特殊的文件，它有自己独立的索引节点和数据块。其数据块中存储的是它所指向的目标文件（f1）的**路径名**。它与目标文件 f1 是两个不同的文件，因此不共享索引节点和数据块。硬链接才会共享同一个索引节点。

II 正确，当进程通过软链接 f2 打开文件时，操作系统会解析软链接，找到它指向的原始文件 f1，并实际打开 f1。因此：

- 进程打开 `f1` 获得 `fd1`，会在系统打开文件表中创建一个表项，指向 `f1` 的内存文件对象（如内存 inode）。
- 进程打开 `f2` 获得 `fd2`，操作系统解析 `f2` 找到 `f1` 后，实际上也是打开了 `f1`。因此，`fd2` 对应的系统打开文件表表项也会指向与 `fd1` 相同的那个 `f1` 的内存文件对象。
- 每个 open 操作都会在进程的用户打开文件表中创建一个新的条目，并可能在系统打开文件表中创建一个新的条目（如果该文件首次被系统打开）或增加现有条目的引用计数。这里由于是两个独立的打开操作（即使最终指向同一个文件），它们在系统打开文件表中会有各自的表项（或者共享一个表项但引用计数增加，关键是它们最终都指向同一个文件在内存中的表示）。更重要的是，这两个文件描述符最终都指向了同一个被打开的文件实例。

III正确，每个文件描述符（fd1 和 fd2）在进程的用户打开文件表（或文件描述符表）中都有自己的条目，这些条目中包含了各自独立的文件读写指针（也称文件位移量）。因此，即使 fd1 和 fd2 最终指向的是同一个磁盘文件（通过软链接解析），两个进程通过各自的文件描述符对文件进行读写时，它们的文件读写位置是独立维护的，互不影响。

Ⅳ错误，软链接 f2 存储的是 f1 的路径名。如果原始文件 f1 被删除，那么 f2 所指向的路径就失效了（成为一个悬空的或坏的链接）。此时，尝试通过 f2 访问文件将会失败，无法访问到原文件的内容。

综上所述，选 B。

56．D。【解析】硬链接和原文件共享一个 inode，而软链接是一个新的 link 文件，有自己 inode。因此，本题中一共有三个 inode：文件 F1 的 inode1、文件 F1 的软链接 F3 的 inode2，软链接 F3 的软链接 F5 的 inode3。

inode1 由 F1，F2 共享，原计数值为 2，现删除 F1，计数值-1，变成 1，故I正确。

F4 是 F3 的硬链接，故 F3 和 F4 共用 inode2，所以 F3、F4 的计数值都是 2，II正确、III错误。

最后 F5 的 inode3 没有其他硬链接，所以计数值为 1，故IV正确。

文件的所有者 owner 即使在创建者删除该文件后，只要该 inode 的计数值不为 0，owner 都是创建者，因此用户 A 删除后，inode1 的现在所有者还是用户 A，故V错误。

F3 是 F1 的软链接，保存的是 F1 的地址，当 F1 删除后，在用户 A 的对应目录下的目录项被删除，F3 自然找不到对应文件，故不能正常打开文件，VI错误。

最后，有的文件系统不支持 inode，如 FAT 系统，并且就算是两个支持 inode 结构的文件系统之间，硬链接也不能跨文件系统使用，因为每个文件系统各自维护自己 inode 数据结构和列表，故 F2 不能跨文件系统使用，软链接 F3 存的是文件路径，是可以跨文件使用的，VII正确。

57．A。【解析】f2 是 f1 的硬链接，指向同一个磁盘文件索引节点，因此两个文件的打开操作共享同一个内存索引节点和系统级打开文件表的表项，A 正确。

P1 和 P2 打开同一个文件的文件访问权限可能有所不同，B 错误。

f3 是 f1 的软链接，并不指向 f1 对应的文件索引节点，因此不参与该节点的链接计数，删除 f3 不影响 f1 的链接计数，C 错误。

读取 f1 只需要传入 f1 在 P1 中的文件描述符 fd1，并不需要其文件名，因此 D 错误。

58．C.。【解析】这是硬链接的逻辑。软链接文件的 inode 指向的是它自己的数据块，而该数据块里存放的只是一个路径字符串。它对目标文件的引用是通过“路径解析”间接完成的，绝非指针直连。B. 正确 ，如果软链接的 inode 和数据块（路径信息）都已经在内存缓存中，那么通过软链接访问目标文件时，额外的磁盘 I/O 开销可能为 0，此时总的磁盘 I/O 次数与通过硬链接访问可能相同。题目说的是"可能相同"，并非"一定相同"，因此该说法成立。

59．B。【解析】A 正确，引导块是文件系统分区的第一个块（或若干块），其中存放了引导加载程序（bootloader）的代码。当计算机启动时，会执行引导块中的代码，进而加载操作系统内核。

B 错误，超级块确实包含了文件系统的关键元数据，如文件系统类型、块（簇）大小、inode 总数、数据块总数、空闲块和空闲 inode 的数量等信息。然而，为了文件系统的可靠性和容错性，超级块通常会在磁盘上有多个副本（备份）。主超级块通常位于文件系统的固定位置，而备份超级块则分布在文件系统的其他位置。如果主超级块损坏，系统可以尝试从备份中恢复。因此，“每个文件系统只有一个超级块”的说法是不正确的。

C 正确，在类 UNIX 文件系统中，索引节点区是一块专门用于存储所有文件和目录的索引节点（inode）的区域。每个 inode 包含了文件的属性（如权限、所有者、大小、时间戳）和指向存储文件内容的数据块的指针（直接指针、间接指针等）。

D 正确，数据区是文件系统中用于存储文件实际数据内容的部分。操作系统需要管理数据区中的空

闲磁盘块，以便在创建或扩展文件时进行分配。常用的空闲空间管理方法包括位示图（bitmap）、空闲链表或成组链接法等。

60．D。【解析】A 错误。文件系统除了数据存储区外，还有引导块、超级块、索引结点区等结构，而且磁盘本身还有一定的冗余空间。文件系统的容量指文件系统可以管理的空间大小，一般小于其所在的磁盘容量。

B 错误，文件系统中单个文件的大小还受限于系统文件大小限制、文件物理结构的限制(如采用混合索引其可用地址数)等。

C 错误，一个文件系统可以管理多个磁盘自然也可将数据放在多个磁盘上。

61．B。【解析】类比行优先矩阵存储，行号=4567/32=142，列号=(4567%32)=23，故选 B。

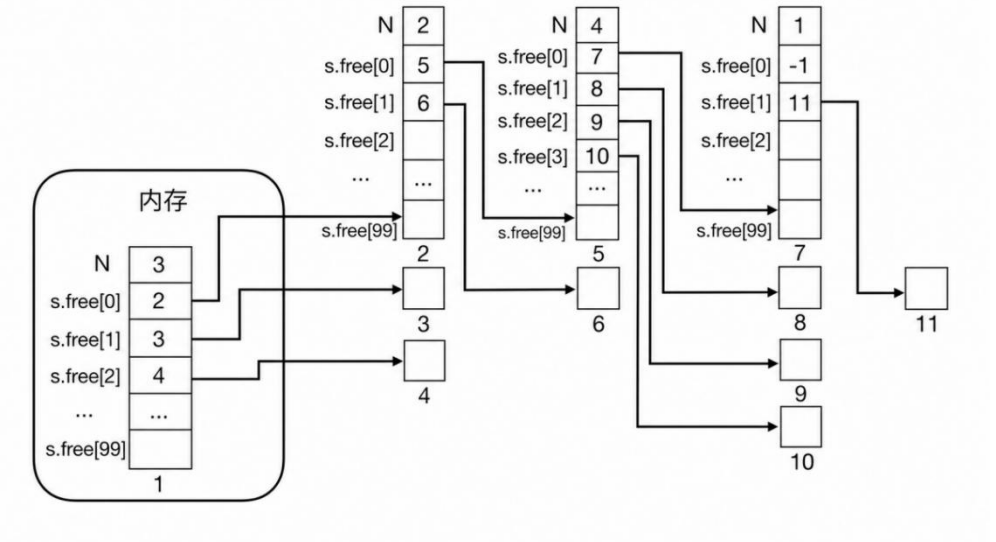
62．B。【解析】为了适应磁盘容量不断增大的需要，当进行盘块分配时，不再以盘块而以簇为基本单位，簇应包含的扇区数量与磁盘容量的大小直接相关。以簇为基本分配单位的优点如下：能适应磁盘容量不断增大的情况，减少 FAT 表的表项数(在相同的磁盘容量下，FAT 表的表项数与簇的大小成反比)，使 FAT 表占用更少的存储空间，减少访问 FAT 表的存取开销。

63．A。【解析】位示图是利用二进制的一位来表示磁盘中的一个盘块的使用情况。一共需要 40GB/4KB=10M 个磁盘块 表示这些磁盘块需要 10Mb/8b=1280KB。

64．D。【解析】在成组链接法中，将空闲块分成若干组，每 100 个空闲块为一组，每组的第一空闲块登记了下一组空闲盘块号和空闲块总数。第一组称为空闲盘块号栈。

65．B。【解析】位示图中，0～31 在 0 号字，32～63 在 1 号字，64～95 在 2 号字，96~127 在 3 号字，所以块号 100 在 3 号字中，3 号字的 0 号位是 96，1 号位是 97，依此类推，块号 100 在 4 号位中。另解，块号为 100 的内存块对应的位示图字号为 100DIV32 = 3，位号为 100 MOD 32 = 4，即字号为 3，位号为 4。选择 B 选项。

66．C。【解析】A 描述的是空闲链表法，B 描述的是位示图法，D 选项表示的是空闲表法。只有 C 选项描述的是成组链表法，每个空闲块记录一组空闲块编号，并指向下一组，块中第一个数表示这个块记录的空闲块数量，随即用一个栈 S.free 来记录空闲块号，第一个数其实就是栈的指针。如下图所示：



67．C。【解析】C 错误，空闲链表法分为空闲盘块链和空闲盘区链，后者可以把连续的空闲盘块

合并成一个区，支持合并，同时也减短了链表长度，通常采用首次适应算法，加快对连续盘块的分配，缺点是分配和回收略复杂。

68．D。【解析】回收两个磁盘块，原本超级块记录了 3 个空闲盘块号，但是现在还需要回收 2 个，超出了上限 4，所以应该将 985 入栈后，当前超级块的 3 改成 4，然后还需要回收 211，此时超级块满了，那么把超级块的内容复制到盘块 211，然后超级块改为[1，211]。接着分配 3 个空闲磁盘块给进程，当前超级块只有一个 211，所以把原来满了的块的内容重新复制到超级块，然后把 211 分配给进程，然后当前超级块里面是[4, 11, 408, 666, 985]，依次把 985, 666 分配给进程，至此已经分配了 3 个空闲磁盘块，超级块的内容为[2, 11, 408]。

此处推荐观看成组链接法视频，分配和回收都讲得很清晰：https://www.bilibili.com/video/BV1HE41167kj?vd_source= 130baaebbac315d959b263869c0a7b39

69．C。【解析】虚拟文件系统(VFS)为应用程序提供了一个统一的接口来访问不同的底层文件系统，使得应用程序不需要关心底层文件系统的具体类型。故案为选项 C。

70．C。【解析】虽然许多操作系统都实现了虚拟文件系统(VFS)这一概念，但它们的实现是不同的。VFS 的具体实现会因操作系统而异。故答案为选项 C。

71．D。【解析】A 正确：超级块对象代表一个特定的已挂载的文件系统。VFS 超级块是各种具体文件系统在安装（挂载）时在内存中建立的，只存放在内存中；而具体文件系统的超级块（如 Ext4 超级块）则存放在磁盘上。

B 正确：索引节点对象代表一个特定的文件。其中，VFS 的索引节点（vnode，又称动态节点）只存在于主存（内存）中；而具体文件系统的原始索引节点（inode）是一种静态结构，既存放在磁盘上，也会在需要时调入主存。

C 正确：目录项对象代表一个路径的组成部分（目录名或文件名）。在硬链接的场景下，由于多个不同的文件名（即不同的路径或目录项）可以指向同一个物理文件，因此同一个唯一的索引节点（inode）对象可以对应多个不同的目录项（dentry）对象。

D 错误：文件（file）对象代表一个与进程相关的打开的文件。由于多个进程可以同时打开和操作同一个文件，因此在内存中会存在多个对应的文件（file）对象（每个进程各有一个），但它们所对应的索引节点（inode）和目录项是唯一的（即多对一）。选项中的表述恰好颠倒了这一关系。

72．B。【解析】选项 B 描述了文件挂载的正确概念，即将一个目录连接到另一个目录，使其成为目录树的一部分，并与外部文件系统关联，以便访问和操作其中的文件和目录。通过挂载，外部文件系统中的文件和目录可以在目录树中访问和操作。

73．D。【解析】Unix 中，每个存储设备都有自己的文件系统，但访问设备中文件时需要先将设备文件系统以目录形式挂载在操作系统的主目录树下;访问时也要在文件路径名中加上挂载点因此选 D。

74．C。【解析】A 错误，在采用 inode 机制的文件系统中，目录项只包含“文件名”和“inode 号”。文件的实际元数据（如文件大小、读写权限、物理块指针等）全部存放在 inode 中。因此，文件大小的改变根本不需要、也不会去更新目录项。

B 错误，硬链接的本质是多个目录项指向内存/磁盘中的同一个 inode。F1 和 F2 根本没有“各自独

立”的 inode，它们在底层完全是同一个文件，共享唯一的 inode。

C 正确，文件从 4KB 增长到 8KB，刚好跨越了一个新的盘块（4KB），因此系统会分配一个新的物理数据块。系统会将新块的物理地址记录在这个共享的 inode 中，并把 inode 里的文件大小更新为 8KB。因为 F1 和 F2 指向的是同一个 inode，所以通过 F1 写入的数据和修改的属性，对 F2 是立刻、完全可见的。

D 错误，删除 F1 本质上只是在目录中删除了 F1 的目录项，并将该共享 inode 中的“硬链接计数”减 1（从 2 变成 1）。由于 F2 的目录项依然存在（硬链接计数不为 0），操作系统绝对不会释放该 inode 和物理数据块。因此，F2 依然可以正常读取到 8KB 的数据。

75．D。【解析】注意打开文件需要把文件对应的 inode 读入内存，文件本体内容在 read 等操作才会读入内存。在最坏情况下，目录项线性检索且目标在最后。各目录默认只占 1 个磁盘块。根据缓存情况：①绝对路径：根目录块在内存，需读取 dirA 的 inode、dirA 目录块、dirC 的 inode、dirC 目录块及 file.txt 的 inode，共 5 次磁盘访问。

② 相对路径：当前目录 dirB 的 inode 和内容已缓存，直接从 dirB 找 dirC，需读取 dirC 的 inode、dirC 目录块及 file.txt 的 inode，共 3 次磁盘访问。

76．A。【解析】在 VFS 及常规文件系统设计中，数据结构分为持久化存储（磁盘）和内存缓存两类。

- 仅存在于内存的：VFS 超级块对象、VFS 索引节点（vnode）、目录项（dentry）、进程打开文件表、系统打开文件表等，都是纯内存结构，磁盘上没有直接对应的独立数据结构（磁盘上有数据内容，但格式不同）。

- 存在于磁盘或与磁盘同步的：磁盘超级块、磁盘索引节点（Disk inode）、磁盘位示图等是持久化存储的数据结构。

77．B。【解析】B 正确：在该文件系统中，一个文件必须对应一个唯一的 inode（索引节点）。因此，inode 区域的总块数乘以每个磁盘块能容纳的 inode 数量，就直接决定了该文件系统中最多可以创建的文件总数 。

A 错误：位图（bitmap）是用来管理和记录数据区磁盘块的使用情况的。位图的总块数（即总 bit 数）决定的是该文件系统所能支持的数据区磁盘块数量上限 ，而不是最多能创建的文件数。 C 错误：超级块通常在文件系统挂载时就被完整读取并缓存到内存中。后续的文件读写等常规操作会直接使用内存缓存中的超级块信息，无需每次都重复读取磁盘。只有在涉及元数据修改（如分配/回收空间）时才在适当时机写回磁盘以保持一致性。

D 错误：低级格式化（物理格式化）是在磁盘物理表面划分扇区、建立磁道等硬件层面的操作。而建立文件系统结构、写入超级块（superblock）元数据信息、初始化空闲空间管理表格等操作 ，均是在逻辑格式化（高级格式化/新建文件系统）阶段完成的。

78．D。【解析】A、B、C 正确：空闲表法通过连续区间记录空闲块，回收时要合并相邻接的空闲块，仅当有少量空闲区时效果较好，不适合碎片过多的场景 。 D 错误：如果回收区前后都有相邻空闲区，系统会将前、后、回收区三者合并为一个更大的连续空闲区，空闲表项的总数不增反减（减少 1 个表项）。

79．C。【解析】A 错误：空闲盘块链法回收每个盘块时，都需要先读入内存修改其指针再写回磁盘，磁盘 I/O 开销很大 。

B 错误：位示图法非常节省空间，在分配回收时，内核通常只需根据需要读入指定位所在的单个磁盘块即可，不需读入整个位图，I/O 次数很少。

C 正确：成组链接法将当前 0 号分组常驻内存超级块，只有当分组分配光或回收满溢时才进行一次磁盘 I/O 读写索引块，效率极高。

D 错误：空闲表法在大型文件系统中表项极大，无法全部常驻内存，且不适合大型文件系统。

80．B。【解析】成组链接法中，超级块中常驻有一组空闲盘块号（通常称为分组 0）。当分组 0 中仅剩唯一一个盘块号时（即 N = 1），说明这是当前分组内最后一个可分配的盘块 。这个盘块在物理上具有双重身份：它既是一个普通的空闲块，同时又是存放了下一组空闲盘块号信息的索引块。 为了维持空闲盘块链不发生断裂，系统不能直接将其抹除或直接分配出去。具体的处理顺序如下：

①读取并复制信息（磁盘 I/O）：由于该索引块中存放着下一组可用的盘块号信息，系统必须先通过磁盘 I/O 将该磁盘块的内容读入内存中。

②更新超级块：将该块中记录的下一组盘块号和块数依次读入（复制）到超级块的空闲盘块栈中，使其成为新的分组 0 。

③分配盘块：在确保下一组盘块信息已经安全“接班”进入超级块后，系统才会将原超级块栈底对应的这 1 个索引块正式分配给申请的进程去使用。

因此，系统的操作顺序必须是先读入、再分配，以确保元数据链条的连续性，选项 B 完全正确。

81．B。【解析】在成组链接法中，内存中的超级块（超级块中的空闲盘块号栈）始终管理着当前最先用于分配或最后被回收的“分组 0” 。当该分组的盘块数量已经达到上限（如本题中的 100 个）时 ，说明超级块的内部栈已经装满了，无法直接在超级块中追加新的盘块号。

此时系统回收 1 个新的空闲盘块（假设块号为 X），其具体操作流程如下：

①建立新索引块：由于超级块满了，系统必须开启一个新的分组。新回收的盘块 X 将在物理上作为这个新分组的索引块。

②复制旧分组信息（写磁盘 I/O）：为了不让原有的空闲盘块链断裂，系统会启动一次磁盘写操作，将当前超级块中“分组 0”里的所有内容（即这 100 个空闲盘块号以及相关的元数据）完整复制到新回收的盘块 X 中 。

③更新超级块：信息复制完毕后，内核会清空超级块中的旧栈，并将盘块 X 的块号放入超级块中，作为新的“分组 0”的唯一一个盘块号 ()（此时超级块中空闲块计数 N = 1，对应的盘块号就是 X）。

通过这个过程，原先超级块里的 100 个盘块在物理上被“推”到了外存中，并由新盘块 X 串联管理；而超级块则腾出了空间，可以继续接收后续新回收的空闲盘块。

第 5 章

输入/输出(I/O)管理

【答案速对】

BCDBD　AACCB　ABCBC　BBBDB

DCCAD　CABAD　CBCCB　DCDDD

DDCBB　CACDC　CCCCA　CCBBB

ABBCB　DCBCB　ADDD

1．B。【解析】A 正确，字符设备是以字符为单位进行数据传输的设备，如键盘、鼠标、打印机等。块设备则是以数据块为单位进行数据传输的设备，如磁盘、磁带、光盘等。

B 错误，为了实现设备无关性（或称设备独立性），方便用户使用和提高系统的可移植性与可适应性，应用程序通常通过逻辑设备名来请求使用 I/O 设备，而不是直接使用物理设备名。操作系统负责将逻辑设备名映射到具体的物理设备名。如果应用程序直接使用物理设备名，那么当物理设备更换或配置改变时，应用程序就需要修改，这不利于移植和维护。

C 正确，设备控制器是介于 CPU 和 I/O 设备之间的硬件部件。它负责接收来自 CPU 的指令和数据，并将其转换为 I/O 设备能够理解的控制信号和操作序列，然后控制 I/O 设备执行具体任务。同时，它也会将设备的状态和 I/O 操作的结果反馈给 CPU。

D 正确，独占设备在一段时间内只能被一个进程所使用，如打印机。可共享设备允许多个进程同时（宏观上）访问，如磁盘。操作系统会通过调度和管理机制来协调多个进程对共享设备的访问。

2．C。【解析】C 错误，即使采用统一编址（内存映射 I/O），映射到 I/O 设备寄存器的内存地址通常也受到内存保护机制的控制。用户程序在用户态下试图直接访问这些受保护的地址（即 I/O 端口）仍然会引发保护异常或需要通过系统调用请求操作系统内核来执行实际的 I/O 操作。允许用户程序“自由读写任何...I/O 端口，无需操作系统干预”会破坏系统的安全性和稳定性。

3．D。【解析】I/O 接口的功能为：①地址译码，选取接口寄存器；②接收控制命令，提供工作状态信息并进行状态检测；③数据缓冲与格式转换以及数据的检错、纠错；④I/O 控制与定时，如中断、DMA 控制逻辑设备操作等；⑤对内与主机通信，对外与外设通信。综上I、II、III均属于 I/O 接口的功能，故本题选 D。

4．B。【解析】采用独立编址方式时，存储单元和 I/O 设备靠不同的指令格式区分，访问 I/O 设备

端口需要单独设置指令，A 错误，B 正确;按照数据传送方式，I/O 接口可以分为并行接口和串行接口，C 错误;采用统一编址方式时，存储单元和 I/O 设备靠不同地址来区分，D 错误。故本题选 B。

5．D。【解析】A 正确，如果不同 I/O 接口的端口编号相同，那么无法分辨 I/O 指令中的地址属于哪个 I/O 接口，所以不同 I/O 接口的 I/O 端口不重复，对地址译码是 I/O 逻辑实现的功能之一。

B 正确，定位好具体的 IO 端口后，再根据读写信号（比如 IN 指令是读信号，OUT 指令是写信号）来进行及具体输入输出。

C 正确，CPU 通过数据线向控制寄存器写入 I/O 命令字，I/O 逻辑接着会对 I/O 命令字进行译码，对外设发出对应操作命令。

D 错误，状态信息也是通过数据线传输的（I/O 接口→CPU 寄存器），状态、控制寄存器通常共用一个。中断信号是通过控制线传输的。（IO 接口 IRQi 中断请求控制线→中断控制器）。

6．A。【解析】IN 是针对 CPU 而言是输入，所以 I/O 接收到的控制信号是读信号，即把 I/O 端口（可能是数据寄存器也可能是状态寄存器）的内容输入到 CPU。因此选 A。

C 错误，命令字是通过数据线传输到控制寄存器的。

D 错误，I/O 端口的地址是通过地址线传输的。

7．A。【解析】程序直接控制 I/O 方式和中断驱动 I/O 方式都是软件控制数据的传输过程，即设备驱动程序负责具体的数据传输，选项 A 错误。DMA 方式由硬件负责具体的数据传输过程，适用于具有 DMA 控制器的系统，用于高速外设的大批量数据传输，选项 B、C、D 均正确。

8．C。【解析】从用户态切换到内核态的动作完全由硬件电路自动完成。当 CPU 在用户态执行完当前指令时，硬件检测到中断信号。CPU 立即进入中断响应周期（即硬件隐指令阶段）。此时，操作系统的内核代码根本还没有运行，CPU 连中断服务程序的入口地址都还没拿到。硬件控制逻辑会自动在硬件层面直接修改 CPU 内部的特权级寄存器，只有特权级切换为内核态、并且两级栈切换完成后，CPU 才能去执行操作系统内核的中断服务程序。因此，是“硬件切换了状态，才允许软件运行”，而不是“软件代码控制了切换”。

9．C。【解析】A 错误，DMA 在主存和 I/O 设备之间建立一条直接数据通路。

B 错误，DMA 并不是处理机，只能单方面受 CPU 控制。

D 错误，DMA 方式中，数据传输过程由 DMA 控制器进行控制，不需要 CPU 介入的，但是数据传输的预处理和后处理阶段都需要 CPU 的干预，DMA 需要 CPU 在预处理时指出所取数据的地址与长度。

10．B。【解析】在 DMA 方式中，由 DMA 控制器控制数据在 I/O 设备和内核区内存间的传输。进程 P 在系统调用中向磁盘发起读命令后阻塞，DMA 控制器通过周期挪用等方式向内存传输磁盘数据当 DMA 字计数归零，块数据传输到内核区内存的缓冲区后，DMA 发起中断请求，CPU 响应该中断请求并进行中断处理，唤醒阻塞的进程 P;只有进程调度导致 P 进程上处理机，进程 P 完成系统调用后处理，将用户需要的数据从内核缓冲区加载到用户空间，再返回用户态。故 B 正确。

注意：中断方式下，中断服务程序完成时，数据才进入内核缓冲区。想想为什么？

11．A。【解析】当一个进程因执行系统调用（例如，请求键盘输入）而被阻塞后，等待用户输入

字符，其后的处理流程如下：

键盘硬件：用户在键盘上键入一个字符。键盘硬件检测到按键，生成相应的编码，并将其存入键盘控制器的内部寄存器中。

中断处理程序：键盘控制器向 CPU 发出一个中断请求。CPU 响应中断，暂停当前执行的程序（如果 CPU 正处于空闲状态，则直接响应；如果正在执行其他进程或内核代码，则根据中断屏蔽和优先级决定何时响应），保存当前现场，然后转去执行操作系统内核中预设的键盘中断处理程序。

设备驱动程序：中断处理程序会调用键盘的设备驱动程序。设备驱动程序从键盘控制器读取输入的字符数据，可能会进行一些转换（例如，从扫描码转换成 ASCII 码），然后通常会将这个字符放入操作系统内核维护的一个输入缓冲区（例如，终端缓冲区或行缓冲）。此时，由于等待该输入的进程之前是被阻塞的，设备驱动程序（或中断处理程序的一部分）会唤醒这个等待输入的进程，将其状态从阻塞态变为就绪态，并放入就绪队列。

进程调度：当中断处理和设备驱动程序的相关工作完成后（通常中断处理程序执行时间很短），如果被唤醒的进程的优先级足够高，或者当前没有其他更高优先级的进程在运行，操作系统的进程调度程序可能会选择这个被唤醒的进程投入运行。也可能在中断返回后，继续执行之前被中断的进程，而被唤醒的进程只是进入了就绪队列。

系统调用返回：当被唤醒的进程（即原先请求输入的进程）再次获得 CPU 并被调度执行时，它会从之前系统调用的阻塞点继续执行。系统调用处理程序此时可以从内核的输入缓冲区中获取到用户输入的字符，并将这个字符作为系统调用的结果返回给用户程序。CPU 从内核态切换回用户态。

用户程序：用户程序获得系统调用返回的字符，继续执行后续逻辑。

综上所述，选 A。

12．B。【解析】将逻辑盘块号转换为物理地址的工作是由设备驱动程序完成的，因为只有对应的设备驱动程序知道设备的逻辑盘块与物理地址的关系，故答案为选项 B。

13．C。【解析】本题考查设备驱动程序的特点。

设备驱动程序有以下特点：

- 驱动程序主要是指在请求 I/O 的进程与设备控制器之间的一个通信和转换程序。
- 驱动程序与设备控制器和 I/O 设备的硬件特性紧密相关
- 驱动程序与 I/O 设备所采用的 I/O 控制方式紧密相关。
- 由于驱动程序与硬件紧密相关，因此其中的一部分必须用汇编语言书写，目前有许多驱动程序的基本部分已固化在 ROM 中。
- 驱动程序应允许可重入，相同的设备可以使用相同的设备驱动程序。
- 驱动程序不允许系统调用，驱动程序本身就运行在内核态，而系统调用是用户态的程序请求使用内核态的服务。

因此，选项 C 错误。

14．B。【解析】检查用户是否有权使用设备属于设备保护，因此是在设备独立性软件中完成的，故答案为选项 B。

设备独立性软件实现的功能有：①提供统一的 I/O 接口②缓冲管理③差错控制④设备的分配和回收（包括使用设备的权限检查）⑤设备命令和识别⑥支持 SPOOLing 技术

15．C。【解析】I 错误，设备寄存器写命令是在设备驱动程序中完成的。

II 正确，检查用户是否有权使用设备属于设备保护，因此是在设备独立性软件中完成的。

III 错误，检查用户 I/O 请求的合法性是在驱动程序完成的，比如对打印机进行读操作就是非法的，应该向 I/O 系统报告 I/O 请求出错。

IV 正确，缓冲管理是在设备独立性软件中完成的。

本题选 C。

16．B。【解析】I、II、III正确。

设备驱动程序的功能包括：

①接收由与设备无关的软件发来的命令和参数，并将命令中的抽象要求转换为与设备相关的低层操作序列。

②检查用户 I/O 请求的合法性，了解 I/O 设备的工作状态，传递与 I/O 设备操作有关的参数，设置设备的工作方式。

③发出 I/O 命令，如果设备空闲，便立即启动 I/O 设备，完成指定的 I/O 操作；如果设备忙碌，则将请求者的请求块挂在设备队列上等待。

④及时响应由设备控制器发来的中断请求，并根据其中断类型，调用相应的中断处理程序进行处理。

⑤完成从 IO 端口读取数据到系统缓冲区。

综上所述，I、II、III正确。Ⅳ缓冲管理是设备独立软件的功能。

17．B。【解析】字符设备接口(流设备接口)不支持随机访问，块设备接口支持随机访问。

18．B。【解析】I错误，设备独立性是指用户编写的应用程序独立于具体使用的物理设备，这样，即使更换了物理设备而应用程序也无须改变。设备驱动程序是与具体物理设备紧密相关的，负责直接与硬件交互。

Ⅱ正确，设备独立性的一个重要体现就是允许用户程序使用抽象的逻辑设备名来请求 I/O 操作，而不是直接使用具体的物理设备名。操作系统负责将逻辑设备名映射到实际的物理设备。

Ⅲ错误，实现了设备独立性后，如果更换了同类型的物理设备，只要操作系统更新了逻辑设备名到新物理设备的映射，用户应用程序不需要修改，仍然可以使用原来的逻辑设备名进行访问。

Ⅳ正确，设备独立性软件层的一个重要功能就是向上层提供统一的接口，处理所有设备共有的操作，如设备的分配与回收、数据缓冲、差错处理等。它将这些共有操作与具体设备的实现细节分离开来。

综上所述，选 B。

19．D。【解析】I正确，事实上，凡在数据的到达速率与离去速率不同的地方，都可设置缓冲区，以缓和它们之间速率不匹配的矛盾。在输入设备与 CPU 之间设置缓冲区，也可使 CPU 的工作效率得以提高。

Ⅱ正确，在远程通信、磁盘控制器和磁带控制器中，都需要配置缓冲寄存器，以减少对 CPU 的中断频率，放宽对 CPU 中断响应时间的限制。随着数据传输速率的提高，需要配置位数更多的寄存器进行缓冲。比如，无缓冲情况下，每接受一个单位数据就要触发一次中断，引入缓冲后，缓冲区满时才会触发中断。

Ⅲ正确，缓冲区可用于解决在生产者和消费者之间交换的数据粒度（数据单元大小）不匹配的问题。例如，当生产者所生产的数据粒度比消费者消费的数据粒度小时，生产者进程可以一连生产好几个数据单元的数据，当其总和已达到消费者进程所要求的数据单元大小时，消费者便可从缓冲区中取出数据进行消费。当生产者所生产的数据粒度比消费者消费的数据粒度大时，生产者每次生产的数据，消费者可以分几次从缓冲区中取出消费。

Ⅳ正确，缓冲区的引入可显著提高 CPU 和 I/O 设备间的并行操作程度，提高系统的吞吐量和设备的利用率。

20．B。【**解析**】B 选项描述完全颠倒。收容输入（hin）应该从空白缓冲队列获取空缓冲区，装满数据后挂入输入队列队尾，等待用户进程提取。

21．D。【**解析**】本题需分情况讨论：如果 T2>T1，即 CPU 处理数据比数据传送慢，那么磁盘将数据传送到缓冲区，再传送到用户区进行处理，除了第一次需要花的时间 T1+T2，剩余数据可视为 CPU 进行连续处理，共花费时间(n-1)T2，读入并处理所用的总时间为 T1+nT2，。如果 T2<T1，即 CPU 处理数据比数据传送快，此时除了第一次可视为 I/O 连续输入，磁盘将数据传送到缓冲区，与缓冲区中的数据传送到用户区及 CPU 处理数据，两者可视为并行执行，那么花费时间主要取决于磁盘将数据传送到缓冲区所用的时间，前 n-1 次所花的时间共为(n-1)T1，而最后一次的时间 T1 完成后，还要花时间从缓冲区传送到用户区，且 CPU 还要处理，即要加上时间 T2，读入并处理所用的总时间为 nT1+T2。综上所述，总时间为(n-1)max(T1，T2)+T1+T2。

22．C。【**解析**】这是单缓冲区，数据输入缓冲区的时间 T=80us，缓冲区数据传送到 CPU 的时间 M=40us，CPU 对这块数据处理的时间 C=30us，处理每块数据的时间=max(C，T)+M=120us。

23．C。【**解析**】第一次需要 100ns+5ns+90ns=195ns，之后 9 次每次需要 max(100ns，90ns+5ns)=100ns，因此共 9x100ns+195ns=1095ns，因此选 C。

24．A。【**解析**】从系统缓冲区传送到用户区的时间为 40 μs，CPU 处理一个数据块的时间为 40μs，和为 80μs，大于将一个数据块从外部设备传送到系统缓冲区的时间 25μs。

则无论采用何种缓冲方式，都需要花费将第一个数据块从外部设备传送到系统缓冲区的 25μs，以及后续从系统缓冲区传送到用户区+处理每个块的 80μs。采用双缓冲结构并未改变耗费的时间，故选 A。

25．D。【**解析**】设备分配要考虑的因素有：

- 设备的固有属性：根据独占/共享/虚拟制定不同的分配策略。
- 设备分配算法：通常采用 FCFS 算法和最高优先级算法
- 设备分配中的安全性：分为安全分配方式（发出 I/O 请求后一旦获得设备便阻塞不请求其他资源，阻塞时是不保持任何资源的（完成一个再来一个），因此不会死锁）和不安全分配方式（进程发出 I/O 请求继续运行或者继续请求 I/O（不管完成没完成多个一起来），当且仅当请求

的设备被另一个进程占用才进入阻塞，性能高但可能死锁）。

- 设备独立性/无关性
- 设备占用状态
- 设备访问权限
- 逻辑设备与物理设备的映射关系

综上所述，选 D。

26．C。【**解析**】设备分配策略的主要目标是提高设备的利用率和减少用户等待时间，通过合理分配和回收设备资源，确保系统资源被高效使用，故答案为选项 C。

27．A。【**解析**】通道控制设备控制器，设备控制器控制设备。当进程提出 IO 请求后，系统的设备分配程序可按以下步骤进行设备分配：

① 分配设备。首先根据 I/O 请求中的物理设备名（一般进程是提供逻辑设备名，通过逻辑设备表映射成物理设备名），查找**系统设备表 SDT**，从中找出该设备的**设备控制表 DCT**，再根据 DCT 中的设备状态字段获知该设备是否正忙。若忙，则将请求 I/O 的进程的 PCB 挂在设备队列上。否则，按照一定的算法计算本次设备分配的安全性。如果不会导致系统进入不安全状态，则将设备分配给请求进程。否则，仍将其 PCB 插入设备等待队列。

②分配控制器。在系统把设备分配给请求 I/O 的进程后，再到其 DCT 中找出与该设备连接的控制器的**控制器控制表 COCT**，从 COCT 的状态字段中可知该控制器是否正忙。若忙，则将请求 IO 进程的 PCB 挂在该控制器的等待队列上。否则，将该控制器分配给进程。

③ 分配通道。在该 COCT 中又可找到与该控制器连接的通道的**通道控制表 CHCT**。根据 CHCT 内的状态信息可知该通道是否正忙。若忙，则将请求 IO 的进程挂在该通道的等待队列上;否则，将该通道分配给进程。只有在设备、控制器和通道三者都分配成功时，这次的设备分配才算成功。然后，便可启动该 I/O 设备进行数据传送。

28．B。【解析】设备分配过程中题干中数据结构访问的先后顺序为系统设备表 SDT→设备控制表 DCT →控制器控制表 COCT→通道控制表 CHCT，故本题选择 B 选项。

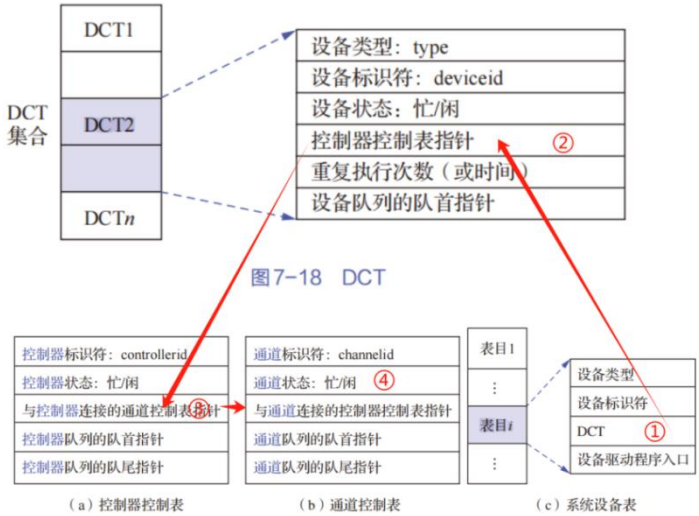


图7-18　DCT

29．A。【**解析**】I 正确，脱机技术的核心就是引入外围机（如脱机读卡机、脱机打印机控制

器）；主机只和磁带等中间介质交互，不再直接等待打印、打孔等慢设备；提高了 CPU 利用率和系统吞吐量。

II 正确，在手工操作阶段，操作员人工逐个装入程序、读取数据，脱机技术实现了输入/输出自动化与集中处理，解决了人机矛盾，促进了**批处理系统**的形成；

III 正确，输入输出进程负责数据在磁盘与设备之间的调度和缓冲，代替了外围机的功能。

IV 错误，假脱机技术 CPU 的利用率更高，吞吐量更大。**脱机技术**中，I/O 操作完全由外围机负责，在输入/输出阶段，CPU 可能处于空闲状态（例如等待数据准备好）；脱机阶段与主机计算是分开的两个过程，无法充分利用 CPU 并发执行。**假脱机技术**通过磁盘+后台进程+任务并发调度机制，比如打印数据，只需要把数据输出到磁盘的输出井，CPU 就可以去干别的事情，等打印机空闲再打印剩余数据。大大提高了 CPU 的使用效率和系统的多任务能力，相比传统脱机技术，CPU 利用率更高。

30．D。【**解析**】I错误，SPOOLing 系统的核心组成部分包括位于磁盘上的输入井和输出井，以及位于内存中的输入缓冲区、输出缓冲区、输入进程和输出进程。因此，并非所有组成部分都在内存中，磁盘上的井是关键。

II 错误，SPOOLing 技术是在多道程序设计技术的基础上发展起来的。它需要系统能够并发地执行用户作业、输入进程和输出进程。在单道批处理系统中，内存中只有一个程序在运行，无法实现这种并行操作。

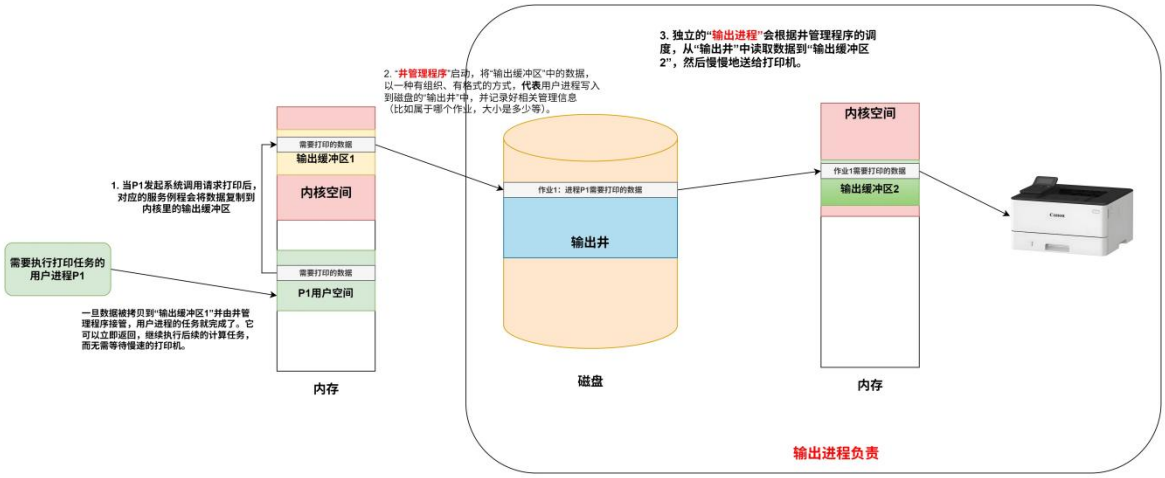
III 错误，SPOOLing 技术是将低速输入设备的数据先传送到高速磁盘上的输入井，或者将用户程序要输出的数据先送到磁盘上的输出井。当 CPU 需要输入数据时，是从输入井（磁盘）读取；当需要输出数据时，是输出到输出井（磁盘）。输入设备的数据通过输入缓冲区进入输入井，输出井的数据通过输出缓冲区送到输出设备。输入/输出井与输入/输出设备之间并不是直接连接，而是通过内存中的缓冲区以及输入/输出进程进行中转。其目的是利用磁盘的高速特性来匹配 CPU 的速度，而不是输入/输出井与慢速设备直连。

IV 错误，SPOOLing 技术是对脱机输入/输出系统中外围控制机功能的一种模拟。它是在主机的直接控制下，利用多道程序中的专门进程（输入进程和输出进程）来模拟外围控制机的功能，从而实现在联机情况下进行的同时外围操作，因此不需要物理上的外围机支持。

综上所述，选 D。

31．C。【**解析**】共享设备在宏观上是并行的，但在微观物理瞬间，磁头或读写部件只能交替为某一个进程服务（A 错）。磁带机是顺序存取设备，如果要访问不同位置的数据，需要卷动整盘磁带，高频切换的代价极其高昂，因此磁带机只能作为独占设备（B 错）。SPOOLing 技术是纯软件技术，通过大容量外存作为中转来“虚拟化”独占设备，并没有改变物理硬件（D 错）。C 项准确解释了为什么磁盘是共享设备而磁带不是 。

32．B。【解析】根据 SPOOLing 技术的完整数据流向模型：



①进程 P1 发出系统调用，数据从 P1 用户空间复制到内核的输出缓冲区 1 ；

②系统的“井管理程序”启动，将数据从缓冲区 1 写入高速磁盘的“输出井”中，此时 P1 即可直接返回继续执行；

③独立的常驻系统进程“输出进程”在后台见缝插针地运行，从磁盘输出井中读取待打印作业到内核输出缓冲区 2；

④最终由输出缓冲区 2 慢慢送往慢速的物理打印机。整个路径选 B。

33．C。【**解析**】SPOOLing 技术利用了相对廉价且海量的磁盘空间（建立输入/输出井），将原本直接面向慢速独占设备（如打印机）的 I/O 改造成面向高速磁盘的读写。这解放了高速的 CPU，使其不必全程忙等慢速外设，同时也极大缩短了用户进程因等待 I/O 最终完成而产生的阻塞时间。需要特别注意：SPOOLing 无法改变外设自身的物理传输速度（D 错）。

34．C。【**解析**】虚拟设备技术是一种将独占设备改造为共享设备的技术，可将一台物理 I/O 设备虚拟为多台逻辑 I/O 设备，从而增加设备利用率。故本题选 C。

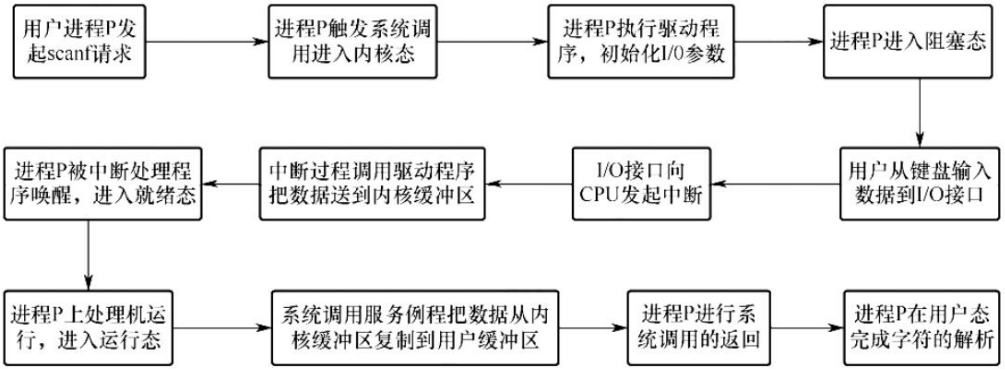
35．B。【解析】I 错误，逻辑设备名到物理设备名的映射，是设备独立性软件的核心功能之一。II 正确：向控制器发命令是驱动程序的基本职责；III 错误：缓冲区管理（分配与回收）以及数据的格式化，属于设备独立性软件（或用户层）的功能。驱动程序只负责把数据搬运到内核缓冲区，不负责全局缓冲池的管理；IV 正确：当外设完成 I/O 操作后，控制器会向 CPU 发出中断请求。虽然中断的响应和前半段处理由硬件隐指令和底层的通用中断处理程序完成，但针对具体设备的中断后续处理（如读取控制器的错误状态码、清空控制器的中断标志位、把数据从控制器内部读出）都是由设备驱动程序中注册的中断服务程序（ISR）来具体执行的。

36．D。【**解析**】以上都属于设备驱动程序的内容，注意与上题驱动程序实现的功能搭配理解，需要记住的是“跟硬件直接交互的是驱动程序”。

37．C。【**解析**】应用层和传输层对应进程，TCP 报文的封装是在内存中进行的，不涉及设备驱动。主机内的网络层是操作系统管理的，IP 数据报位于内核空间内，数据经过处理后从用户进程空间进入操作系统的内核空间也不需要设备驱动程序。网卡属于 I/O 设备，向网卡写入数据以及控制网卡发送均需要直接访问 I/O 设备，因此需要设备驱动程序参与答案选 C。

38．D。【**解析**】A 选项错误，唤醒进程 A 由中断服务程序完成。阻塞 A 由对应的系统调用服务例

程（如 read）完成。B、C 选项错误，将数据从寄存器送往内核缓冲区由中断服务程序完成，将数据从内核缓冲区送往用户缓冲区由系统调用服务程序（如 read）在唤醒后完成。D 选项正确，这是驱动程序的功能之一。整个流程如下图所示：



39．D。【解析】I错误，磁盘的物理地址通常由**柱面号（磁道号）—盘面号（磁头号）—扇区号**构成。一个柱面包含所有盘面上相同编号的磁道。磁道是盘面上的同心圆，而不是柱面内再划分磁道。

Ⅱ错误，磁盘驱动器是磁盘本身，而磁盘控制器才是 IO 接口

Ⅲ错误，对磁盘进行分区是在磁盘物理格式化（低级格式化）之后、高级格式化（逻辑格式化）之前进行的操作。 磁盘逻辑格式化主要是在已有的分区上建立文件系统，例如写入引导块、初始化超级块、建立空闲空间管理数据结构和根目录等。

40．D。【解析】D 错误，磁盘有移动部件且容错能力弱，因此容易导致一个或多个扇区损坏。部分磁盘甚至在出厂时就有坏块。对于复杂的磁盘，控制器维护磁盘内的坏块列表。这个列表在出厂低级格式化时就已初始化，并在磁盘的使用过程中不断更新。低级格式化将一些块保留作为备用，操作系统看不到这些块，即这些块对操作系统是透明的。控制器可以采用备用块来逻辑地替代坏块，这种方案称为扇区备用。

41．D。【解析】A 选项中，将磁盘分成多个逻辑分区的操作称为分区，不属于低级格式化；

B 选项中，低级格式化后的磁盘没有文件系统，需要经过分区和高级格式化才能存储数据；

C 选项中高级格式化会破坏原分区中的数据；

D 选项正确。

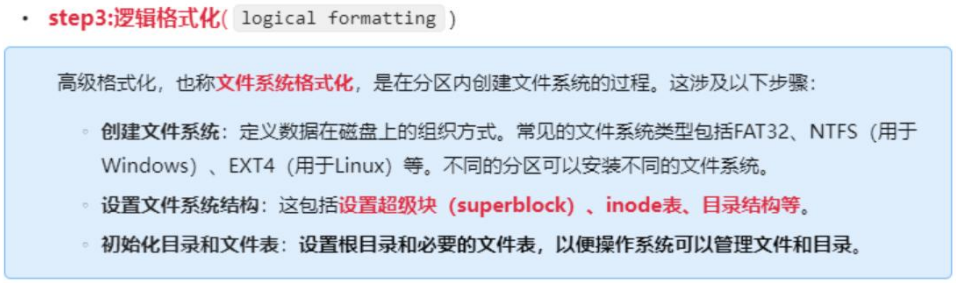
42．D。【解析】一个新的磁盘是只含有磁性记录材料的空白盘，为了使该磁盘能够存储数据，需要对其进行物理格式化，初始化扇区的数据结构，D 正确;物理格式化之后，要进行逻辑格式化，在这个过程中，为磁盘建立文件系统根目录，初始化磁盘空闲空间管理的数据结构，建立管理已分配空间的数据结构，A、B、C 属于磁盘逻辑格式化阶段执行的操作，故本题选 D。

43．C。【解析】挂载是指将一个文件系统与操作系统的目录树进行关联，这通常在以下情况下进行：

- 在系统启动过程中，操作系统会挂载根文件系统和其他必要的文件系统。
- 当插入可移动存储设备（如 U 盘）时，系统会挂载该设备的文件系统到指定的挂载点（通常是一个空文件夹）。

因此，挂载操作是将格式化好的文件系统与操作系统的目录树进行关联，使得文件系统可以被操作

系统访问。这是一个在系统启动之后或设备插入时进行的操作，而不是在逻辑格式化时进行的操作。



44．B。【解析】文件系统的根目录创建是逻辑格式化过程中实现的。

45．B。【解析】SSTF 算法优先响应距离当前磁头最近的访问请求，FCFS 算法则根据磁盘请求的先后顺序进行调度，在这两种算法中，磁头可以随时改变运动方向；SCAN 算法和 CSCAN 算法固定了磁头移动方向，在这两种算法中，不能随时改变。综上所述，选 B。

46．C。【解析】I正确，磁盘访问时间主要由三部分组成：寻道时间（Ts）、旋转延迟时间（Tτ）和传输时间（Tt）。在大多数情况下，特别是对于机械硬盘，寻道时间（磁头移动到目标磁道所需的时间）通常是这三者中最大的。

II 错误，磁臂粘着（armstickiness）现象是指磁头长时间停留在某个磁道附近，响应对该磁道或附近磁道的密集访问请求，而导致对其他较远磁道的访问请求长时间得不到服务。先来先服务（FCFS）算法是按照请求到达的先后顺序进行处理，它不会因为某个磁道有高频率访问就一直停留在那里，而是会依次处理队列中的所有请求，因此它不会导致磁臂粘着。

III 正确，Flash 半导体存储器的物理结构不需要考虑寻道时间和旋转延迟，可直接按 I/O 请求的先后顺序服务。

47．A。【解析】采用 SSTF 算法时，访问序列为 108，90，82，142，197，故扫过的磁道数为 8+18+8+60+55=149；采用改进的 SCAN 算法时，访问序列为 108，142，197，90，82，故扫过的磁道数为 8+34+55+107+8=212，因此选 A。

48．C。【解析】采用 SCAN 算法时，依次访问的磁道是 788，845，911，999，700，432，396，165，164，25，磁臂一共移动了(999-734)+(999-25)= 1239 个磁道，对应 1239 个柱面;采用 SSTF 算法时，依次访问的磁道是 700，788，845，911，432，396，165，164，25，磁臂一共移动了(734-700)+(911-700)+ (911-25)=1131 个磁道，对应 1131 个柱面，故本题选 C。

49．D。【解析】题干要求采用 CSCAN 算法，磁头沿磁道号增大的方向移动，期间依次对访问请求进行响应，然后返回首端，重新开始扫描。因此磁头依次访问的磁道是 92，99，0，24，26，47，51，65，78，85，共移动了(99-90)+(99-0)+(85-0)= 193 个磁道，故本题选 D。

50．C。【解析】题干要求采用 C-LOOK 算法，磁头沿磁道号增大的方向移动，期间依次对访问请求进行响应，然后返回首端，重新开始扫描，C-LOOK 算法不需要移动到磁道的两端即可返回。在这个过程中，磁头依次访问的磁道是 86，89，98，24，34，53，65，74，共移动了(98-78)+(98-24)+(74-24)=144 个磁道，平均移动的磁道数为 144/8=18，故本题选 C。

51．C。【解析】磁盘旋转速度为 20ms/转，每个磁道存放 10 条记录，故读出一条记录的时间为

20ms/10=2ms。

- 对于第一种记录分布的情况，读出并处理记录 A 需要 6ms，此时读/写磁头已转到记录 D 的开始处，因此为了读出记录 B，必须再转一圈少两条记录(从记录 D 到记录 B)。后续 8 条记录的读取及处理与此相同，但最后一条记录的读取与处理只需 6ms。于是，处理 10 条记录的总时间为 9*(2+4+16)ms+(2+4)ms=204ms。
- 对于第二种记录分布的情况，读出并处理记录 A 后，读/写磁头刚好转到记录 B 的开始处，因此立即就可读出并处理，后续记录的读取与处理的情况相同，共选择 2.7 圈。最后一条记录的读取与处理只需 6ms。于是，处理 10 条记录的总时间为 20ms*2.7+6ms=60ms。综上，信息分布优化后，处理时间缩短了 204ms-60ms=144ms。

52．C。【解析】每个柱面有 12 条磁道，每条磁道有 16 个扇区，因此一个柱面上有 192 个扇区。5687/192=29 余 119，因此柱面号为 29；119/16=7 余 7，因此磁道号为 7、扇区号为 7。

53．C。【解析】向磁道序号增加的方向移动，首先排除选项 A、B；选项 D 是到达端点后先回到最小值，不扫描磁道，再往磁道号增大的方向移动；SCAN 算法是到达端点后往当前方向相反的方向扫描；选项 C 符合 SCAN 算法的定义，故选择选项 C。

54．C。【解析】提前读是指用户对文件进行访问时，经常采用顺序访问方式，在读当前块的同时，可将下个盘块也提前读入缓冲区，这样，当下次要读该盘块时，便可直接从缓冲区中取得下一盘块的数据，而无须再启动磁盘 I/O，因此选项 A、B 正确。延迟写是指在缓冲区中的数据本应立即写回磁盘，但考虑到该缓冲区中的数据在不久后可能会再次被本进程或其他进程访问(共享资源)，因此并不立即写回磁盘，而挂在空闲缓冲区队列的末尾，此时，任何访问该数据的进程都可直接读出其中的数据而不必启动磁盘 I/O，这样，又可进一步减小等效的磁盘 I/O 时间。由此可知，延迟写和随机访问方式之间并没有必然的联系，选项 C 错误。当缓冲区中的数据延迟到必须往磁盘上写时(如缓冲区满)，才进行写磁盘操作。

55．A。【解析】在数据从控制器传送至内存的这段时间内，从磁头下通过的扇区数为 2。当数据从控制器传送至内存后，磁头开始读数据时，刚好转到目标扇区。所以总时间为

总时间=初始寻找 0 扇区时间+读扇区总时间+将扇区数据送入内存总时间

由题中条件可知，旋转速度为 3000r/min=50r/s，即 20ms/r。读一个扇区需要的时间为：20/8ms=2.5ms

读一个扇区并将扇区数据送入内存需要的时间为：2.5*3ms=7.5ms

读出一个磁道上的所有扇区需要的时间为：20/2ms+8*7.5ms=70ms=0.07s

每磁道数据量为：8*512B=4KB，数据传输速度为：4KB/0.07s=57.1KB/s

所以依次读出一个磁道上的所有扇区需要 0.07s，其数据传输速度为 57.1KB/S。

56．C。【解析】平均访问时间=平均寻道时间+旋转时间+传送时间

平均寻道时间=9ms

旋转时间=1/2x（60x1000ms/7200r）≈4ms

传输时间=1/400x（60x1000ms/7200r）≈0.02ms

平均访问时间=9+4+0.02≈13ms

57．C。【解析】磁盘高速缓存根据局部性原理，将磁盘上的一些常用数据块暂时保存在随机存储器(RAM)中，使得对磁盘的访问可以转化为对随机存储器的访问，大大提高磁盘的读写性能。但是磁盘高速缓存并不能改善磁盘系统的可靠性，而后备系统、磁盘容错系统和RAID 阵列都能改善磁盘系统的可靠性，故本题选 C。

58．B。【解析】IV 错误，交换区是将内存中的部分数据临时移到磁盘上，以释放内存空间，从而让系统能运行更多的程序或处理更大的数据。增大交换区并不能减少 I/O 速度。

I 正确，磁盘缓冲区是内存中的一片区域，利用局部性原理，可以保留部分磁盘数据的拷贝，在命中时可以直接从磁盘缓冲区拿数据，无需磁盘 I/O，但是需要考量置换算法。交付数据给用户的方式有两种：①数据交互，直接从磁盘缓冲区把数据传送到用户区②指针交付，即只把磁盘缓冲区某区域的指针交付给进程，节省了数据的拷贝开销。

II 正确，提前读是指把当前访问的盘块相邻的几个连续的盘块一起读入磁盘缓冲区，利用了空间局部性，命中时无需启动磁盘 I/O。延迟写是指磁盘缓冲区的数据本应该立即写回磁盘，但考虑不久的将来该数据还会被访问，所以不会立即写回磁盘，而是会把它挂在空闲缓冲区队列的末尾，当有进程申请这片磁盘缓冲区时，才把该修改的内容写回磁盘，同样也可以减少 I/O 时间（其实是时间局部性）。

III 正确，尽可能地为文件分配相邻的磁盘块，这样磁盘就无需多次寻道，减少 I/O 时间。

59．B。【解析】固态硬盘基于闪存技术，不属于磁表面存储器，选项 A 错误。

闪存翻译层可将 CPU 的逻辑磁盘块读/写请求转化为对底层 SSD 物理设备的读/写控制信号，选项 B 正确。

固态硬盘无论是读还是写，速度都远快于常规硬盘。只不过对于 SSD，写比读要慢，因为涉及擦除和磨损均衡。选项 C 错误。

固态硬盘块内的页必须按顺序写入信息，选项 D 错误。

60．B。【解析】SSD 磨损均衡技术分为两种：

- 动态磨损均衡。写入数据时，优先选择擦除次数少的新闪存块。老的闪存块先歇一歇。
- 静态磨损均衡。这种技术更为先进，就算没有数据写入，SSD 也会监测并自动进行数据分配，让老的闪存块承担以读为主的存储任务。同时让较新的闪存块腾出空间，以承担更多以写为主的存储任务。如此一来，各闪存块的寿命损耗就都差不多。

61．A。【解析】I正确，固态硬盘基于闪存技术。

Ⅱ正确，固态硬盘以页为读写单位，只有在一页所属的块整个被擦除后，才能对这一页进行写入。

Ⅲ错误，固态磁盘的读写性能优于机械硬盘，但是固态硬盘的 NAND 闪存中的块有擦写次数限制，其使用寿命一般短于机械硬盘。

Ⅳ错误，IV描述的是动态磨损均衡，而在静态磨损均衡中，会对数据块进行检测，尽可能保证各块写入和擦除均衡，并非是每次写入都选择长期存放数据而很少擦写的块。

62．B。【解析】读写以页为单位，擦除以块为单位。随机写的速度比读慢，原因如下：

- 只能写入被擦除的块，擦除块比访问块慢得多。

- 修改一个页中的部分数据，需要拷贝这个块中其他页到被擦除的块进行修改
- 磨损均衡
  - 动态磨损均衡：自动选磨损少的块写入，早期常使用动态方式，这种方式会导致冷数据块一直得不到擦除，各个块擦除并不均衡。
    - 冷数据：只读数据或者系统数据，修改很少的数据块
  - 静态磨损均衡：将冷数据块转移到擦除较多次的块（老块），现代多使用静态+动态磨损均衡结合的方式，增加了闪存的寿命。

注意：闪存的存储介质是 ROM，但是 SSD 里有闪存翻译层，其中地址映射表是由速度较快的 DRAM 实现，所以 SSD 可以说存储介质是 ROM，但是不可以说只有 ROM，内部还是有 DRAM 的存在。

63．B。【解析】读题目问的是“在网络数据块到达后”的执行顺序。在发起 I/O 请求初期，由于数据未就绪，进程 P 就已经完成了操作③（进入阻塞队列，让出处理机）。 当网络数据块真正到达网卡时，网卡硬件首先发出中断请求，即操作④ 。 CPU 响应中断，暂停当前运行的随机进程 Q，转去执行网卡的中断服务程序。 在中断服务程序中，CPU 通过执行 I/O 指令，将数据从网卡控制器的数据寄存器读入内核系统缓冲区中，即操作① 。 随后，由于数据已经进入系统缓冲区，中断服务程序调用相关内核函数，将因等待该网络输入而阻塞的进程 P 的状态修改为就绪态，并移入就绪队列，即操作② 。 因此，在这个时间节点之后的正确操作链条为：④ → ① → ②。

64．C。【解析】 在键盘中断处理过程中，当用户按下按键，键盘控制器将键值存入 DBR（数据缓冲寄存器）并置就绪位；中断响应后，CPU 在中断服务例程中与硬件交互，将字符从控制器的数据寄存器读入操作系统统一分配的内核缓冲区中，随后才会唤醒等待该数据的用户进程。

65．B。【解析】完整过程：字符串的第一个字符是在系统调用服务例程（sys_write）的控制下输出到数据端口并发送启动打印命令的；之后进程 P 便执行 scheduler()主动阻塞自己让出处理机；剩下的所有字符打印，都是在其他进程（如 Q）运行期间，由打印机硬件完成上一个字符后触发的硬件中断服务程序来控制输出的。

A 错误：若所有字符都在 sys_write 系统调用服务例程中依次串行输出，则整个 I/O 过程 CPU 都需要持续等待并轮询设备状态，这是程序直接控制（轮询）方式。中断控制方式的特点就是 CPU 启动设备后即转向其他工作，待设备完成时通过中断信号通知 CPU 进行后续处理。

C 错误：打印机打印字符的速度远低于 CPU 执行速度。在中断控制方式下，进程 P 发出系统调用并启动设备后会立即进入阻塞态，让出 CPU 给其他进程，绝不会保持运行状态并循环查询状态端口。

D 项错误，执行中断服务程序时系统正处于被打断的进程 Q 的环境和地址空间中，需要通过公共映射的内核缓冲区作为中介来传递数据。

66．D。【解析】 DMA 方式（周期挪用）的核心优势之一就是开销极低。周期挪用的过程：当 DMA 控制器请求“周期挪用”时，CPU 仅仅需要暂停一个存储（总线）周期的操作，让出一个内存读写周期供 DMA 传送一个字。此时 CPU 并没有发生“进程调度”，无需进行保护和恢复现场（即程序

状态字、通用寄存器等的压栈出栈操作）。对比中断：响应中断时，必须进行完整的上下文切换（保护和恢复现场），开销远大于一个存储周期。因此，D 项称周期挪用 DMA 的保护恢复现场开销“与中断一致”是完全错误的。

67．C。【解析】C 错：进程因设备忙被阻塞后，CPU 会调用调度程序切换到其他就绪进程执行。中断服务例程只有在硬件设备完成当前任务（如打印完一页）发出中断信号时才会被 CPU 执行，而不是在进程刚被阻塞时立即执行。A、B、D 均正确描述了驱动程序在内核态运行、可能引发调度以及与硬件强绑定的特性。

68．B。【解析】驱动程序和中断服务程序共同协作直接控制 I/O 硬件：驱动程序负责在 I/O 操作的开始阶段发起操作（写命令、数据、地址寄存器）；中断服务程序则负责在操作完成阶段进行响应（读取状态寄存器结果、清除中断等）。C 项错误，中断处理程序位于操作系统的最底层（驱动程序之下）；D 项错误，没有涉及硬件的具体公有管理是在设备独立性软件层完成的。

69．C。【解析】A、B 错：文件名和文件描述符、逻辑块号是上层（文件系统或设备无关软件）处理的参数。

C 对：设备无关软件将文件的逻辑结构转换为块的请求，并将逻辑设备名映射为物理设备号，然后将物理设备号和要读取的（文件）物理块号传递给设备驱动程序。

D 错：将块号转换为 CHS（柱面、磁头、扇区）地址正是设备驱动程序内部要执行的工作，而不是传入的参数。

70．B。【解析】计算磁盘的具体柱面号、磁头号、扇区号（CHS 物理寻址参数）是充当“物理翻译官”的设备驱动程序的核心职责，因为它必须深刻理解特定硬件的几何物理结构（B 不属于）。而设备的逻辑命名映射 、设备访问权限校验（保护）、统一的系统级缓冲与磁盘高速缓存管理，都是典型的执行“所有 I/O 设备公共操作”的设备独立性软件的核心任务。

71．A。【解析】I 正确：块设备（如磁盘、SSD）的硬件传输是以“块”或“扇区”为单位的，且通常要求内存起始地址严格对齐。而用户进程定义的缓冲区大小和起始地址是由程序员任意决定的。内核缓冲区在其中起到了“数据对齐和重组”的桥梁作用。

II 正确：块设备数据读入内存通常通过 DMA（直接内存访问）控制器。DMA 传输期间需要目标物理内存地址绝对固定。用户空间的页面由于受到虚拟内存管理（如页面置换算法、内存紧凑）的控制，在耗时较长的磁盘 I/O 期间，随时可能被换出到外存，或被移动物理位置。如果直接将外设数据读入用户工作区，会导致 DMA 往错误的物理地址写入数据。而内核系统缓冲区通常是“锁定（Pinned）”在物理内存中的，绝不会移动或换出，确保了 DMA 的安全。

III 正确：如果直接读入某一个进程的用户空间，该数据就无法被系统内的其他进程共享。引入内核缓冲区（Page Cache）后，系统可以保留这份数据，当其他进程也读相同文件时直接从内核复制即可。同时，内核还能借此实现“预读”，在用户还没请求时就提前把后续块读入系统缓冲区，极大地提高了 I/O 效率。

IV 错误（属于强干扰项，叙述本身正确，但非原因）： 在传统的同步 read 系统调用中，由于磁盘 I/O 属于慢速操作，用户进程确实会被剥夺 CPU 并进入阻塞态。为了让其他进程运行，操作系统

必然会进行进程调度，从而切换页表（更新页表基址寄存器）。但是，这并不是操作系统“不直接将外设数据读入用户工作区”的原因。 无论是否使用内核缓冲区，进程执行阻塞 I/O 时都会因等待设备而被阻塞，并随之发生上下文切换（包括页表切换）。这属于 I/O 操作的正常调度行为，与“为什么不直接读入用户工作区”并无因果关系，因此不是原因。

72．D。【解析】A、B、C 项完全符合缓冲与缓存特性的标准对比 。D 项颠倒了二者的关系：缓冲区是一个“中转站”，其中的数据可能是唯一的，数据源将数据发往缓冲区后就可以不再保留；而缓存（Cache）保存的数据一定是对应数据源（如磁盘块）中数据的子集，属于高速副本。

73．D。【解析】设置磁盘缓冲区或高速缓存的最主要、最根本目的是减少物理磁盘的 I/O 总次数，缓和高速 CPU/内存与慢速磁盘之间的巨大速度差异。回写策略（延迟写入）可以通过“合并写入”和“批量写入”大幅削减实际 I/O 次数。虽然积累多个 I/O 请求后可以让磁盘调度算法优化寻道时间，但这只是一个派生、次要的好处，而不是其首要设计目的。

74．D。【解析】A 错误：“输出进程”是一个系统级的后台服务进程（或守护进程），它独立于任何用户作业，由操作系统在启动时创建和管理。B 错误：该进程通常是常驻内存的，而不是按需创建。当井中没有任务时，它处于休眠或阻塞状态；当井中有新任务时，操作系统会唤醒它。C 错误：用户作业的数据已经被转存到磁盘的“输出井”中。输出进程的数据来源是“输出井”，而不是任何用户作业的地址空间。D 正确：这准确描述了“输出进程”的职责。它是一个系统进程，拥有执行底层 I/O 操作的权限，是 SPOOLing 系统中唯一直接与物理打印机硬件打交道的模块。

第四部分

408 1000 题 cn 解析册

第 1 章

绪论

【答案速对】

CAACC　ADBBA　DBCDD　CCDBB

ABCBB　DADBB　CDCAB　B(CDA)

1．C。【解析】选项 A 描述的是多任务操作系统的特性，而非计算机网络的定义，故错误。选项 B 仅指出了物理连接，这是网络的组成部分，但不是对网络的完整和准确描述。网络不仅仅是物理链路的集合，还包括了协议和其他软硬件。选项 C 是对计算机网络的准确定义，计算机网络有三个基本要素：自治计算机、互连、集合体。选项 D 描述的是计算机网络的主要功能之一（资源共享）。

2．A。【解析】计算机网络建立的两个最基本和最主要的功能是数据通信（实现用户间的信息交换）和资源共享（使网络中的硬件资源、软件资源、数据资源等能被多个用户共享）。提高可靠性和处理能力可以通过网络实现（如分布式系统、计算机集群），但并非所有网络最主要的功能，故 B 错误。分布式处理与负载均衡是基于网络发展起来的应用和技术，属于网络功能的应用和扩展，故 C 错误。信息检索和集中管理也是网络提供的应用功能，但不是最核心和主要的功能，故 D 错误。

3．A。【解析】从工作方式的角度划分，计算机网络通常被分为边缘部分和核心部分。边缘部分由所有连接到互联网上、供用户直接使用的主机组成。这部分是用户进行网络应用和通信的地方。核心部分由大量网络和连接这些网络的路由器组成，其主要任务是为边缘部分提供高效、可靠的数据传输服务。

4．C。【解析】局域网（LAN）覆盖范围小（几米到几公里），通常用于一栋楼、一个办公室或一个园区，传输速率高，误码率低。城域网覆盖范围介于 LAN 和 WAN 之间，通常覆盖一个城市。广域网 (WAN）覆盖范围大，通常跨越国家或地区。个人区域网 (PAN)覆盖范围非常小，通常指个人设备之间的连接。题目中，仅覆盖一栋办公楼的网络显然是局域网。

5．C。【解析】总线型拓扑结构使用一条公共的传输介质（总线）作为所有站点的共享通信线路。当一个站点发送数据时，信号会沿总线向两个方向传播，所有连接在总线上的其他站点都能接收到这个信号。

6．A。【解析】**发送时延**（也称传输时延）是将分组的所有比特发送至物理链路所需的时间，计算公式为：发送时延=分组长度/信道带宽。因此，它与分组长度直接相关。

传播时延是电磁波或光信号在信道中传播一定距离所需的时间，计算公式为 传播时延 = 链路长度/信号传播速率，它与分组长度无关。

处理时延是路由器或交换机处理分组（如检查头部、查路由表）所需的时间，一般较小且不直接由整个分组长度决定。

排队时延是分组在网络设备（如路由器）的输出队列中等待被发送到下一跳所需的时间，取决于网络拥塞情况。

本题选 A。

7．D。【解析】为了使链路在第一个比特到达接收端时仍处于发送状态（即链路被“充满”），发送方需要发送的数据量至少等于链路的“带宽时延积”。带宽时延积=带宽×单向传播时延，为 1000000 比特。这意味着当发送方发送了 1000000 比特后，第一个比特刚好到达接收端。如果发送的数据量小于此值，则发送方会在第一个比特到达前就已发送完毕，导致链路空闲。

8．B。【解析】吞吐量是指单位时间内成功通过信道的实际数据量。该用户同时进行的活动包括下载（视频 20Mbps + 网页 2Mbps = 22Mbps）和上传（文件 5Mbps）。这些活动占用的总带宽为 22Mbps + 5Mbps = 27Mbps。由于用户的链路带宽为 100Mbps，足以支持当前的通信需求，因此实际吞吐量即为用户当前所有活动的总速率。

9．B。【解析】H1 有两条链路可以到达 H2，即 H1→R1→R2→R4→H2 和 H1→R1→R3→R4→H2。路径 R1→R2→R4 的吞吐量受限于其中链路的最低带宽，最大可传输流量为 20Mb/s。同理，路径 R1→R3→R4 的最大可传输流量也为 20Mb/s。由于允许负载均衡，路由器 R1 可以同时向这两条并行路径转发数据。因此，从 R1 到 R4 的总带宽为两条路径带宽之和 20Mb/s+20Mb/s=40Mb/s。整个 H1 到 H2 的端到端路径的最大吞吐量由最窄的环节决定，H1→R1 链路容量为 100Mb/s，R1→R4 并行链路总容量为 40Mb/s，R4→H2 链路容量为 100Mb/s，则 H1 到 H2 的最大吞吐量为 min(100 Mb/s, 40 Mb/s, 100 Mb/s) = 40 Mb/s。

10．A。【解析】H1 到 R1 链路中，主机 H1 的接口速率为 10Mb/s，H1 到 R1 的线路带宽为 100Mb/s，路由器 R1 的接口速率 50Mb/s，此链路的实际容量受限于三者中的最小值，即主机 H1 的接口速率，为 10Mb/s，同理，R4 到 H2 的链路实际容量也为 10Mb/s。中间并行路径中，数据流在 R1 处可通过 R2 和 R3 两条路径进行负载均衡。路径 R1→R2→R4 的容量为 min(R1 出口速率，链路带宽，R2 接口速率, ..., R4 入口速率) = min(50, 100, 50, ...) = 50Mb/s。同理，路径 R1→R3→R4 和 R1→R3→R4 的容量也为 50Mb/s。因此，中间并行链路段的总容量为两条路径之和 50Mb/s + 50Mb/s = 100Mb/s。则 H1 到 H2 的最大吞吐量为 min{10Mb/s，100Mb/s，10Mb/s}=10Mb/s。

11．D。【解析】网络体系结构是计算机网络各层次及其协议的集合。它描述的是网络中层次的数量、各层的名称及功能、各层向其上层提供的服务以及相邻层之间的接口规范，但不包括具体的硬件(如计算机的操作系统类型)实现细节。

12．B。【解析】在分层结构中，每一层(第 N 层)都利用其下一层(第 N-1 层)提供的服务来构建本层的功能，并向其上一层(第 N+1 层)提供服务。对等层实体(远程第 N 层)之间是通过第 N 层协议进行通信，而不是直接提供服务。

13．C。【解析】OSI 参考模型分为七层：应用层、表示层、会话层、运输层、网络层、数据链路层、物理层。（法律标准）

TCP/IP 分为四层：应用层、运输层、网际层、网络接口层。（事实标准）

14．D。【解析】从第 N+1 层传递到第 N 层的数据单元是第 N+1 层的协议数据单元(PDU)，该 PDU 作为第 N 层的服务数据单元(SDU)被处理。第 N 层会在这个 SDU 上加上本层的协议控制信息(PCI)，形成第 N 层的 PDU。而接口数据单元 IDU 则是相邻层接口上体现的数据单元，N-IDU=N-SDU（N+1-PDU）+接口控制信息 ICI。PDU 是网络中实际传输的数据格式，IDU 则是为了描述层与层之间接口如何交互的抽象概念。

15．D。【解析】协议数据单元 PDU 由服务数据单元 SDU 和协议控制信息 PCI 组成，在各层间传输数据时，第 n 层的 SDU 加上第 n 层的 PCI 组成第 n 层的 PDU，同时作为第 n-1 层的 SDU。一个甚至多个 n+1 层的 SDU 可以封装到一个 n 层的 PDU，也可以封装到多个 n 层的 PDU 中，因此答案为 D。

16．C。【解析】在分层体系结构中，服务访问点(SAP)是相邻两层之间服务原语(请求、指示、响应、确认)交换的逻辑接口。第 N 层的 SAP 是第 N+1 层能够访问第 N 层所提供服务的地方。PDU 是协议数据单元，SDU 是服务数据单元，对等实体是通信双方相同层次中的实体。

17．C。【解析】在分层网络体系结构中，服务访问点（SAP）是相邻两层之间发生交互的逻辑接口。上层实体通过 SAP 来请求下层实体提供的服务。可以将其通俗地理解为下层向上层暴露的“地址”或“标识符”，用于区分不同的服务使用者。端口号是传输层（TCP 或 UDP）使用的地址，网络层通过 IP 地址将数据报送到目的主机后，传输层需要根据端口号将数据交付给主机上正确的应用进程（例如，80 端口通常对应 HTTP 服务进程，25 端口对应 SMTP 服务进程）。因此，应用层的进程通过端口号来使用传输层提供的服务，端口号是传输层的 SAP。

18．D。【解析】在 OSI 参考模型中，对于相邻的两层，第 N+1 层实体通过服务访问点 SAP 将一个接口数据单元 IDU（interface data unit）传递给第 N 层实体。N-IDU 由服务数据单元 N-SDU（service data unit）和接口控制信息 ICI（interface control information）组成。为了传递 SDU，第 N 层实体可能将 SDU 分为若干段，每一段加上一个报头后作为独立的协议数据单元 PDU（protocol data unit）传送出。简单地说，SDU 就是同一结点内层与层之间交换的数据单元，而 PDU 就是不同结点的对等层实体之间交换的数据单元。

缩写	全称	所在层级	含义
SDU	Service Data Unit	N+1 层到 N 层	高层要传下来的原始数据（服务数据单元）
IDU	Interface Data Unit	SAP 接口	SDU + ICI（接口控制信息），作为完整单位传给下一层
PDU	Protocol Data Unit	N 层内部	N 层加上自己的协议头后生成的数据单元，用于发送给对等实体

19．B。【解析】第 n 层的 PDU 对应 n-1 层的 SDU，OSI 模型的下四层自下而上分别为物理层、数据链路层、网络层、传输层，对应的 PDU 为比特流、数据帧、数据分组、报文段，故下三层的 SDU 分

别为数据帧、数据分组、报文段。

20．B。【解析】协议：控制两个对等实体（或多个实体）进行通信的规则的集合。其他术语补充如下：

- 实体：任何接受或发送信息的硬件或软件进程。在许多情况下，实体就是一个特定的软件模块。
- 对等实体：收发双方相等层次中的实体。
- 服务：
  - 在协议的控制下，两个对等实体间的通信使得本层能够向上一层提供服务
  - 要实现本层协议，还需要使用下面一层提供的服务。
  - 协议是水平的，即协议是控制对等实体之间通信的规则。但是服务是垂直的。即服务是由下层向上层通过层间接口提供的
  - 并非在一个层内完成的全部功能都称为服务。只有那些能够被上一层实体看的见的功能才叫服务。
- 服务原语：上层使用下层所提供的的服务必须通过与下层交换一些命令。
- 服务访问点（SAP）：服务访问点用于在同一系统中相邻两层的实体交换信息的逻辑接口，用于区分不同的服务类型。
- 协议数据单元 PDU：对等层次之间传送的数据包称为该层的协议数据单元 PDU。
- 服务数据单元 SDU：同一系统内，层与层之间交换的数据包称为服务数据单元。
  - 多个 SDU 可以合成为一个 PDU；一个 SDU 也可以划分为几个 PDU。
- 接口数据单元 IDU：对于相邻的两层，第 N+1 层实体通过服务访问点 SAP 将一个接口数据单元 IDU（interface data unit）传递给第 N 层实体
  - 在 OSI 参考模型中，当相邻高层的实体把（IDU）传到低层实体后，被低层实体视为（SDU）

21．A。【解析】网络协议的三个基本要素是语法、语义和同步(或时序)。语法规定了数据与控制信息的结构或格式，以及编码方式。语义定义了协议元素(如控制信息、地址等)的含义。同步规定了事件的执行顺序和速率匹配。通信双方所交换数据的格式、编码以及控制信息的结构属于语法的一部分。

22．B。【解析】语义规定了协议元素（如报文段、控制信息）的含义以及在特定情况下应执行的动作。当接收方收到失序报文段时，重复发送 ACK 这一行为，是在解释“如何处理失序”以及“该行为的目的是什么”（即期望对方重传）。这涉及到对协议行为含义的规定，属于语义范畴。

23．C。【解析】同步（或时序）规定了事件发生的顺序。TCP 三次握手过程详细描述了建立连接时客户端和服务器之间交换报文的先后顺序（SYN 段的发送、SYN+ACK 段的响应、ACK 段的最终确认），这是对事件顺序的规定。

24．B。【解析】原始数据长度为 500B，经过运输层、网际层和网络接口层之后，长度为 500+20+20+18=558B。则应用层数据传输效率为 500/558，约等于 89.6%。

25．B。【解析】假设原始数据长度为 X，则经过运输层后加上 TCP 首部，长度为 X+20。同理，

又经过网际层，长度为 X+20+20。随后经过网络接口层，添加以太网帧头和帧尾，长度变为 X+20+20+18。若应用层数据传输效率为 80%，则 X/(X+20+20+18)=80%，计算可得原始数据长度 X 为 232B。

26．D。【解析】物理层是 OSI 模型的最底层，它直接与传输介质交互，负责将数据链路层传来的帧（比特序列）转换为电信号、光信号或无线电信号在物理介质上传输，它本身不再对数据进行封装（即不添加物理层首部或尾部）。

27．A。【解析】10Base-T 中“T”的含义是双绞线，传输媒体不属于网络体系结构，故叙述 I 错误。路由器会采用主动丢包策略，这表明网络层参与拥塞控制，故叙述 III 错误。

28．D。【解析】OSI 参考模型自下而上分为七层：物理层、数据链路层、网络层、运输层、会话层、表示层和应用层。第七层是应用层。应用层是用户与网络的接口，为特定类型的网络应用提供访问 OSI 参考模型环境的手段。

29．B。【解析】TCP/IP 模型的运输层主要包含两个重要协议，即 TCP 和 UDP。TCP 提供面向连接的、可靠的字节流服务。UDP 提供无连接的、尽最大努力交付的、不可靠的数据报服务。因此，运输层同时提供这两种类型的服务。

30．B。【解析】OSI 参考模型中的表示层（第 6 层）主要功能是处理两个通信系统中交换信息的表示方式。它向上层提供一种公共的数据表示格式，负责数据的转换（如不同编码系统间的转换）、压缩/解压缩以及加密/解密等，以确保应用层能够正确识别和处理数据。

31．C。【解析】在 OSI 模型中，运输层（第 4 层）的下一层是网络层（第 3 层）。网络层的主要任务是通过路由选择算法为分组在源主机和目的主机之间选择一条合适的路径，并进行转发，从而实现主机到主机的通信。因此，运输层可以直接从网络层获得路由和转发服务。

32．D。【解析】网络层，OSI 网络层标准支持提供面向连接（如虚电路服务）和无连接（如数据报服务）两种类型的服务。而 TCP/IP 模型的网际层（IP 协议）核心是提供无连接的、尽力而为的数据报服务。因此，选项 A 和 C 中关于 OSI 网络层“只”或“仅”提供面向连接服务的说法是错误的。传输层，OSI 运输层仅支持面向连接服务，而 TCP/IP 运输层支持面向连接（TCP）和无连接服务（UDP）。

33．C。【解析】OSI 参考模型的物理层和数据链路层对应到 TCP/IP 参考模型的网络接口层，OSI 参考模型的应用层、表示层、会话层对应到 TCP/IP 参考模型的应用层。

34．A。【解析】数据通过网络的路径选择由网络层负责。

35．B。【解析】当第 N 层实体从第 N+1 层接收数据时，这部分数据是第 N 层需要为其提供服务的对象，因此被称为第 N 层的服务数据单元 (SDU)。第 N 层实体为了实现本层协议的功能，会为这个 SDU 添加一些控制信息，如地址、序号、差错控制字段等，这些控制信息被称为协议控制信息 (PCI)。第 N 层的 SDU 与第 N 层的 PCI 组合在一起，就构成了第 N 层的协议数据单元 (PDU)。

36．B。【解析】表示层的下一层是会话层。会话层主要功能包括：建立、管理、维持、恢复和终止会话；对话控制；在长数据传输过程中设置同步点，故障后从同步点恢复传输。B 选项是表示层的功能，答案选 B。

37．①C ②D ③A【**解析**】TCP/IP 参考模型的网络接口层负责处理与物理网络连接相关的所有细节，例如将网际层传递下来的 IP 数据报封装成特定物理网络（如以太网、Wi-Fi）所要求的帧格式进行传输。网际层负责在源主机和目的主机之间进行数据报的路由和转发，向上层的运输层（TCP 或 UDP）提供主机到主机的逻辑通信服务。应用层包含了各种用户直接使用的网络应用程序及其协议。

第 2 章

物理层

【答案速对】

DAABC　CCBBB　CBDCD　ABDCB

DCBCB　DDBBB　CDCCD　BCBCC

CBCCA

1．D。【解析】信道是信号从发送端到接收端传输的通路，通常依托双绞线、同轴电缆、光纤、无线电波等传输介质实现，故 A 正确。在双向通信中，例如全双工通信，通常需要独立的发送信道和接收信道，故 B 正确。根据信道中传输的信号类型，可以将信道划分为传输模拟信号的模拟信道和传输数字信号的数字信道，故 C 正确。物理层考虑的是如何在传输介质上传输比特流，而不指具体的传输介质，故 D 错误。

2．A。【解析】一个通信系统主要包括信源、信道和信宿。信源是产生和发送原始信息的地方。在本题中，网络摄像头采集视频并编码，是信息的源头，因此是①信源。信道是传输信号的物理路径或逻辑路径。以太网线和光纤是承载信号的传输介质，它们共同构成了②信道。信号是数据的电磁或光学表现形式，适合在信道中传输。比特流以光信号或电信号的形式在介质上传输，这些是③信号。信宿是接收信息的一方。监控中心服务器是最终接收数据的一方，因此是④信宿。

3．A。【解析】在计算机网络领域，带宽指在单位时间内，网络中某信道所能通过的“最高数据率”或“额定速率”，单位是比特/秒 (bps)。

4．B。【解析】若一个码元有 M 种离散状态，则每个码元可以携带log_2M比特的信息，所以 16 种状态可以表示 4 个比特。

5．C。【解析】信息传输速率（比特率）等于码元传输速率（波特率）乘以每个码元所承载的信息量（比特数）。

$R_b = R_B * log_2M$这里直接给出了每个码元携带 3 比特，所以 2400Baud*3bit=7200bps。

6．C。【解析】香农定理定义了在给定带宽和信噪比的信道中，可靠信息传输速率的理论上限。任何实际通信系统的信息传输速率都不可能超过该信道容量。奈氏准则关注的是无噪声情况下的码元速率上限。

7．C。【解析】ASK（幅移键控）用不同的振幅电平表示码元。16 个不同的幅值意味着每个码元

可以代表 4 个比特。因此，数据传输速率 =4000 波特×4 比特/波特=16000 bps=16 kbps。

8．B。【解析】在无噪声情况下，根据奈氏准则，最大码元传输速率 R=2W，为 16000Baud，对于 8 电平 PAM，每个码元携带 3 比特信息。因此，最大数据传输速率 R×3=16000×3=48000 bps=48 kbps。

9．B。【解析】信道带宽为 10kHz-2kHz=8kHz，根据香农公式，信道的极限传输速率为 $Wlog_2(1+S/N)$，代入得 $8kHz*log_2(1+S/N)>=40kbps$，可解得 S/N 至少为 31。

10．B。【解析】香农定理确定的信道容量 C 是信道本身的固有属性，由带宽 W 和信噪比 S/N 决定。调制方式的选择影响实际能达到的数据速率以及在给定波特率下能传输多少比特，但它不能改变信道的理论容量上限。

11．C。【解析】奈氏准则给出了无噪声情况下的码元速率上限，由此计算出的数据速率（32kbps）是一个理论上的可能值。但香农定理考虑了噪声，并给出了信道能够可靠传输信息的最大速率（20kbps）。实际可靠的数据传输速率不能超过香农极限。

12．B。【解析】信道采用 4 个相位，每个相位有 x 种振幅，则每个码元可以表示 4x 个比特的信息。根据奈奎斯特定理，无噪声下极限数据传输速率$C_1=2Wlog_2(4x)$。根据香农定理可知，在信噪比为 30dB 条件下，极限数据传输速率$C_2=Wlog_2(1+S/N)=Wlog_2(1+1000)$。由题意知，$C_1>C_2$，可解得 x 至少为 8。

13．D。【解析】首先计算信道容量 C=2000bps。文件大小为 100KB=100×1024×8 bits=819200 bits。所需时间=总比特数/信道容量 =819200/2000=409.6 秒，约等于 410 秒。

14．C。【解析】该信道为无噪声信道，则可根据奈氏准则计算极限数据传输速率$C=2Wlog_2M=1.6Mbps$。一个分组的长度为 1000B，则共 4000 个分组(N=4000)，每个分组的发送时延$T_p=1000*8bit/1.6Mbps=0.005s$。网络采取存储转发，共两段链路(k=2)，忽略传播时延，总时间$T_{总}=（N+k-1）*T_p=20.005s$。

15．D。【解析】本题考察香农公式。最高数据传输速率(信道带宽)$=W\log_2\left(1+\frac{S}{N}\right)(b/s)=160M\times\log_2\left(1+\frac{0.6}{0.0193}\right)(b/s)=800(Mb/s)$；

时延带宽积=传播时延*带宽=$1000m/(2*10\text{^}8\,m/s)*800Mb/s=4000bit$。

16．A。【解析】数字数据通过调制解调器 调制成模拟信号，在接收端将通过调制解调器将模拟信号还原为数字信号。基本的数字调制方法有移幅键控法(ASK)、移频键控法(FSK)、移相键控法(PSK)和正交振幅调制法(QAM)。模拟数据采用脉码调制 PCM 转换成数字信号，经过采样、量化、编码三个步骤，常用于音频信号。数字数据通过数字发送器转换成数字信号；模拟数据通过放大调制器转换成模拟信号。

17．B。【解析】FSK 是“频移键控”的缩写，它使用不同频率的载波信号来代表不同的数字比特。

18．D。【解析】ASK, PSK, FSK 都是根据当前码元信号的某个参数（振幅、绝对相位、频率）来确定其代表的比特。而 DPSK（差分相移键控）则是利用相邻码元之间的载波相位差来表示数字信息，因此解调当前码元时需要知道前一个码元的相位信息。

19．C。【解析】观察题目发现，每一个周期内的波形图没有跳变，因此排除曼彻斯特、差分曼彻

斯特和归零编码，有可能是不归零编码和反向不归零编码。NRZ 编码就是不归零编码，A 选项正确。该波形图有可能是反向不归零编码，该编码方式会携带同步信息，B 选项正确。该波形图传递的是数字信号，但所传递的数据不一定是数字数据，模拟数据也可以编码成数字信号。C 选项错误。以太网默认使用曼彻斯特编码，该波形图无跳变，不可能是曼彻斯特编码，D 选项正确。

20．B。【解析】非归零编码，用高/低电平表示 1/0，没有同步信息。归零编码通过每个脉冲中间跳变到 0 来确定同步信息，曼彻斯特编码和差分曼彻斯特编码也是通过中间跳变来同步。

21．D。【解析】10BASE-T 是指带宽位 10Mb/s 的传统以太网。传统以太网采用曼彻斯特编码。曼彻斯特编码在每个码元中间时刻电平发送跳变，上跳变表示 1，下跳变表示 0，也可以上跳变表示 0，下跳变表示 1。

22．C。【解析】曼彻斯特编码的码元速率（波特率）是数据速率的两倍，已知数据速率为 10Mbps，则波特率为 2*10M=20MBaud。根据奈氏准则，理想低通信道的最高码元传输速率=2W Baud，其中 W 为信道的频率带宽。将波特率 20MBaud 代入可得，所需的最小信道带宽为 10MHz。

23．B。【解析】电路交换是面向连接的，能够保证可靠传输，而分组交换和报文交换是无连接的，因此电路交换的可靠性是最高的，A 和 D 错误。并且电路交换的物理链路建立后，双方可随时发送数据，因此电路交换的传输时延最低，所以 B 正确 C 错误。因此答案为 B。

24．C。【解析】在报文交换中，整个文件作为一个完整的报文进行发送。网络中的路由器采用“存储-转发”的方式处理该报文，即每个路由器必须完整地接收到整个报文，然后才能开始向下一个节点转发。从主机 H1 到主机 H2 共经过了 2 个路由器，构成了 3 段链路。传播时延和节点处理时延均忽略不计，总时延主要由发送时延构成。报文需要经过 3 段链路的发送，每段链路的发送时延为 0.5MB/2Mbps=2s，则总时间为 3*2s=6s。

25．B。【解析】采用分组交换时，报文被分为 2 个分组，分组交换的时间为 20kb/10kbps + (2-1)×10kb/10kbps = 3s；采用报文交换时，每一段链路的时延相同，故每段链路的传输时延为 1.5s，数据的长度为 1.5s×10kbps = 15kb，选择 B 选项。

26．D。【解析】I. 电路交换：一旦专用电路建立，路径上的任何故障（如 R2 故障）都会导致整个连接中断。II. 数据报分组交换：每个分组独立路由。R2 故障后，R1 发往 R2 的分组将失败。除非路由协议（如 OSPF, RIP）能够检测到故障并为 R1 找到一条不经过 R2 的新路径到达目的服务器，否则后续分组将无法送达。III. 报文交换：如果报文已完整到达 R1，但下一跳 R2 故障，R1 无法将报文转发出去。该报文可能会在 R1 的存储中因超时而丢弃，或者如果系统有动态路由能力且存在备用路径，可能会尝试重路由（但这超出了基本报文交换的范畴，更接近高级网络功能）。基本情况下，可以认为报文受阻于 R1。

27．D。【解析】虚电路分组交换在数据传输前需要一个虚电路建立阶段，通过网络发送一个特殊的“呼叫请求”分组来建立一条逻辑连接（虚电路）。一旦虚电路建立，后续所有分组都沿着这条路径传输，并能保证按序到达。因此，“无需连接建立过程”的说法是错误的。

28．B。【解析】数据报网络是一种无连接的分组交换方式。A 错误，数据报方式不需要预先建立连接。B 正确，由于每个分组独立路由，它必须包含足够的信息（如完整的源和目的地址）使网络节点

能够将其转发到正确的目的地。C 错误，转发主要依据目的地址，数据报号（如 IP 分片中的标识）用于其他目的。D 错误，不同分组可能经过不同路径，因此可能失序到达。

29．B。【解析】虚电路网络由于维护了连接状态，当检测到拥塞时，可以通过信令机制（如发送特定的控制分组）通知源端调整发送速率。例如，帧中继和 ATM 网络都有这样的机制，因此 B 正确。数据报网络是无连接的，不存在“虚电路端点“，故 A 错误。拥塞是网络资源（如链路带宽、路由器缓存）不足的宏观现象，物理层冲突是链路层介质访问控制问题，故 C 错误。数据报网络通常采用尽力而为的服务，不预留带宽。虚电路在建立时反而可能协商带宽，故 D 错误。

30．B。【解析】网络层可以向传输层提供两种类型的服务，面向连接的服务称为虚电路服务，而无连接的服务称为数据报服务。虚电路的想法是避免为发送的每个分组都必须进行路由选择。取而代之的方法是，在传送分组之前，在源结点与目的结点之间建立一条逻辑连接的虚电路，以后这条虚电路上的所有分组将采取相同的路由。虚电路连接采用连接标识符进行标识，连接标识符由每台主机独立选择。因此，可以保证在主机内部每个连接标识符惟一地标识一个虚电路，同时两台主机可以存在多条虚电路为不同的进程服务。

31．C。【解析】对讲机是半双工通信，允许双向通信但不能同时进行，故 A 错误。传统广播电视是单工通信，信息只能单向从发射台传向接收机，故 B 错误。全双工通信允许通信双方在任何时刻同时发送和接收数据，C 正确。集线器连接的以太网构成一个冲突域，所有设备共享带宽，采用 CSMA/CD 机制，工作在半双工模式，D 错误。交换机连接的点对点链路可以工作在全双工模式。

32．D。【解析】光纤通过光脉冲传输数据，其材料（玻璃或塑料）不导电，因此完全不受电磁干扰（EMI）和射频干扰（RFI）的影响。光纤还具有极高的带宽潜力和非常低的信号衰减，使其能够支持长距离高速传输。

33．C。【解析】单模光纤的纤芯直径非常小，使得光信号只能沿着纤芯的轴心以单一路径（模式）传播，从而避免了多模光纤中因多种模式光信号沿不同路径、以不同速度传播而引起的模式色散。模式色散会导致光脉冲展宽，限制了传输速率和距离。消除了模式色散是单模光纤能够支持更长距离和更高速率的关键。

34．C。【解析】物理层接口的功能特性指明了接口各条信号线（引脚）的用途，例如哪些线用于发送数据、哪些用于接收数据、哪些用于提供时钟信号、哪些用于控制等。

35．D。【解析】过程特性定义了在物理连接的建立、维持和释放过程中，各种事件发生的合法顺序。题目描述的“检测信道 -> 发送同步请求 -> 等待同步确认 -> 开始传输”是一个典型的事件序列，符合过程特性的定义。

36．B。【解析】规定连接器的型号（如 DB-9）属于机械特性，因为它定义了接口的物理形状、尺寸和引脚排列。明确各个引脚的功能（如 TD 用于发送数据，RD 用于接收数据）则属于功能特性。

37．C。【解析】物理层接口规范主要定义接口本身的特性，包括机械特性（连接器形状、尺寸）、电气特性（电压、速率）、功能特性（引脚定义）和过程特性（事件顺序）。通信电缆的具体铺设路径、支撑方式等属于网络工程实施和综合布线的范畴，而非物理层接口规范直接定义的内容。

38．B。【解析】作为传统的共享式局域网的连接设备，集线器在物理上采用星状的连接结构，但

在逻辑上采用的是总线状的连接结构。

39．C。【解析】集线器工作在物理层，它将接收到的信号进行放大和整形，然后将其广播到除输入端口外的所有其他端口。集线器不识别帧的 MAC 地址，因此无法进行定向转发。所有连接到集线器的主机（在同一个冲突域内）都会收到该数据帧，由各主机的网卡判断是否是发给自己的。

40．C。【解析】中继器是物理层设备，它对信号进行再生和放大，以延长网络传输距离。它连接的两个网段必须在物理层规范上完全相同，包括数据传输速率、信号编码方式等。如果速率不同，中继器无法正确处理信号。

41．C。【解析】网络标准中对信号的延迟范围做了具体的规定，中继器只能在此规定范围内进行有效的工作，否则会引起网络故障。例如，在采用粗同轴电缆的 10BASE5 以太网规范中，互相串联的中继器连接必须遵守“5-4-3 规则”(4 中继器限制)，即用 4 个中继器连接 5 个网段，其中只有 3 个网段可以连接站点。

42．B。【解析】信号在传输过程中会因为介质的物理特性而衰减。为了能将信号传输得更远，就需要在线路中间对信号进行增强。对于模拟信号，使其传播更远的设备是放大器，放大器的作用是将衰减的模拟信号的能量放大。对于数字信号，使其传播更远的设备是中继器，中继器对失真的数字信号进行“再生”和“还原”，而不是简单的放大。

43．C。【解析】集线器作为物理层设备，它不对数据帧的内容进行检查，包括帧的长度。它接收到比特流后，会进行信号放大和再生，然后将其转发出去。处理短帧（或 runt frame）是更高层（如数据链路层，由网卡或驱动程序负责填充）或接收端的事情，集线器本身会按原样转发它接收到的比特信号。

44．C。【解析】连接在集线器上的所有主机共享集线器的总带宽。对于一个 10Mbps 的集线器，无论连接多少台主机，它们共同分享这 10Mbps 的带宽。由于集线器是半双工工作方式，并且构成一个冲突域，在任何时刻只有一个主机可以成功发送数据。因此，从整个集线器（冲突域）的角度来看，其总的输出数据吞吐量上限就是该集线器的额定速率，即 10Mbps。

45．A。【解析】理论上，如果 N 个主机完全平均地、无冲突地共享 10Mb/s 的带宽，则每个主机获得的平均带宽为 (10/N) Mb/s。当 N=1 时，带宽为 10Mbps；当 N=10 时，带宽为 1Mbps。这个变化趋势是随着 N 的增加而减少。实际中需要考虑碰撞，集线器环境是一个冲突域。当 N 增加时，多个主机同时发送数据的概率增大，导致数据帧碰撞的几率也随之增加。发生碰撞后，相关主机必须等待一个随机时间后重传，这会浪费信道带宽。因此，随着 N 的增加，碰撞的负面影响会越来越显著，导致每个主机实际获得的平均带宽会低于理论上的 (10/N) Mb/s，并且下降的速率会因为碰撞的加剧而比理论情况更快。

第 3 章

数据链路层

【答案速对】

AACDB　CCBDB　BCBBA　CACCA

CBBDD　CDDAC　DBCAB　DDBAC

BDCBC　DDDAC　CADAA　BDCBD

CCCBB　BCBCC　CCAAB　DBDDB

CCCBC　CAAAC　CDCDC　C

1．A。【解析】数据链路层的差错控制主要关注的是在物理链路上传输帧时可能产生的比特差错（1 变成 0 或 0 变成 1）。分组丢失、分组失序和网络拥塞通常是由网络层或传输层来处理。

2．A。【解析】帧同步是数据链路层的重要功能之一，其主要目的在于实现数据链路层透明的数据帧传输。

3．C。【解析】数据链路层的功能包括：将网络层传下来的 IP 数据报封装成帧，在相邻节点间建立、维护和释放数据链路（例如 PPP 协议），以及进行差错检测和可能的纠正。端到端的流量控制是传输层的功能。数据链路层的流量控制是点对点（相邻节点间）的。

4．D。【解析】在广播信道中，多个站点共享同一传输介质，如果同时发送数据，就会发生冲突（碰撞）。介质访问控制（MAC）机制就是用来决定在广播信道中信道如何分配，以协调各站点的发送，避免或解决冲突。

5．B。【解析】流量控制的目的是协调发送方和接收方的速率，使得发送方的发送速率不超过接收方的接收能力，从而避免接收方缓存溢出导致数据帧丢失。

6．C。【解析】数据链路层可以根据不同的协议提供不同类型的服务。例如，以太网提供的是无确认的无连接服务，而某些 HDLC 的变种或无线链路协议可能提供有确认的面向连接服务，以增强可靠性。然而，并非所有数据链路层协议都必须提供完全可靠、无差错的传输（例如，以太网检测到差错会直接丢弃帧，可靠性由上层保证）。

7．C。【解析】无确认无连接服务开销小，延迟低，但不保证可靠性。实时语音通话对实时性要求很高，可以容忍偶尔的少量数据包丢失，如果使用确认和重传机制反而会增加延迟，影响通话质量。因此，这类应用适合采用无确认无连接服务。

8．B。【解析】字符计数法在帧首部使用一个计数字段来标明帧内字符的数量。如果这个计数字

段在传输过程中发生错误（例如，一个比特的翻转），那么接收方就会错误地判断帧的长度，导致当前帧的错误接收，并且由于无法确定当前帧的正确结束位置，后续所有帧的起始位置都会被错误判断。

9．D。【解析】零比特填充法的规则是：发送方在发送数据时，对数据中出现的每 5 个连续的“1“之后都插入一个“0”；接收方在接收数据时，进行反操作，即每检测到 5 个连续的“1”之后，就删除其后紧跟着的“0“。

10．B。【解析】字节填充规则：

- 数据中遇到 EOT，在其前加 ESC，变为 ESC EOT。
- 数据中遇到 SOH（虽然本例数据中没有，但规则适用），在其前加 ESC，变为 ESC SOH。
- 数据中遇到 ESC，在其前加 ESC，变为 ESC ESC。

原始数据 A ESC B EOT C 处理 ESC 变为 A ESC ESC B EOT C，处理 EOT：变为 A ESC ESC B ESC EOT C 加上帧定界符：SOH A ESC ESC B ESC EOT C EOT

11．B。【解析】违规编码法利用了物理层编码本身的一些特性。例如，在曼彻斯特编码中，高-低电平对和低-高电平对分别表示 0 和 1，而高-高电平对或低-低电平对是“违规”的（正常数据不会出现）。数据链路层可以借用这些违规的编码序列来界定帧的开始和结束，从而实现帧定界，并且由于正常数据不会出现这些违规编码，自然实现了透明传输。

12．C。【解析】位同步要求收发双方时钟频率一致，以保证接收方能在每个比特信号的正确时刻进行采样。如果时钟频率有偏差或采样时刻不准，接收方的采样点就会逐渐偏离比特信号的中心，最终在信号跳变边缘采样，从而对该比特的电平值做出错误判断。

13．B。【解析】简单奇偶校验码只能检错，不能纠错，故 A 错误。无论是奇校验还是偶校验，当数据中发生 1 位、3 位、5 位等奇数个比特错误时，总的“1”的个数的奇偶性会改变，从而可以被检测出来，故 B 正确。如果发生两位比特错误（偶数个错误），总的“1”的个数的奇偶性可能不变（例如 00->11，或 01->10），导致错误无法被检测，故 C 错误。增加冗余位是为了检错，这会略微降低有效数据传输率（增加了开销），而不是提高有效带宽，故 D 错误。

14．B。【解析】接收到码字中“1”的个数为 4 个（偶数个）。由于系统采用的是奇校验，它期望收到的码字中“1”的个数为奇数。现在收到的是偶数个“1”，这与奇校验规则不符，因此接收端判定数据传输发生了错误。奇偶校验码能够检测出所有奇数个比特的错误。当校验失败时，接收端知道发生了错误，并且可以推断是奇数个比特发生了翻转。它无法确定具体的错误位数，也无法定位错误。

15．A。【解析】数据块为 1010110，包含了 4 个“1”（偶数个），为了使得添加校验位后的整个序列中“1”的个数仍然是偶数，校验位 P 必须设置为“0”。

16．C。【解析】A 错误，水平奇偶校验是在每一行数据的最后加上一个校验位。B 错误，单独的水平或垂直奇偶校验都属于一维奇偶校验，因此它们无法检测出任意偶数个同时发生的错误。C 正确，二维奇偶校验（同时使用水平和垂直校验）大大增强了检错能力。它可以检测出所有单个比特、两个比特以及三个比特的错误。此外，当仅发生单个比特错误时，该错误会同时导致其所在行和所在列的校验出错，通过行列交叉点可以唯一定位该错误比特，从而进行纠正。垂直校验位可以看作是组成了方块校验字符（BCC）。D 错误，方块校验字符（BCC）是由垂直奇偶校验产生的，其目的是用于差错检测，

而不是数据压缩。它通过增加冗余位来实现检错，这实际上是降低了纯数据传输的效率，但提高了数据的可靠性。

17．A。【解析】生成多项式的最高位系数和常数项系数必不能为 0。

18．C。【解析】根据生成多项式 $G(X)=X^4+X+1$，可知其最高次幂为 4，因此 CRC 校验码的长度 r=4 位，同时其对应的二进制除数为 10011。计算时，需在原始数据 110101 后补 4 个 0，得到被除数 1101010000。接着，执行模 2 除法，用 1101010000 除以 10011，得到的余数即为 CRC 校验码 0000。

19．C。【解析】最小海明距离的定义是编码系统中任意两个不同有效码字之间，对应位上数字不同的位数（即海明距离）的最小值。

20．A。【解析】若要能检测出 e 位错误，则要求码集中的最小海明距离 $d \geq e+1$。因为如果 $d \leq e$，那么一个码字经过 e 位或更少位的变化就可能变成另一个合法的码字，从而使差错无法被检测出来。若要能纠正 t 位错误，则要求最小海明距离 $d \geq 2t+1$。因为一个码字发生 t 位错误后，它必须与其他任何合法码字的距离仍然是最近的，这样才能被唯一地纠正回原来的码字。若要能同时纠正 t 位错误，并检测出 s 位错误（通常 s>t），则要求最小海明距离 $d \geq t+s+1$。这确保了发生 t 位以内错误的码字能被正确纠正，而发生 t+1 到 s 位错误的码字会被检测出来（而不是被错误地“纠正”成另一个码字）。

21．C。【解析】A 错误，正确的关系式应该为$2^k >= m + k + 1$。B 错误，海明码构造中，校验位并非集中放置在数据位的末端（或前端）。它们被安插在码字中的特定位置，这些位置的编号通常是 2 的整数次幂（如第 1、2、4、8……位，假设从 1 开始计数）。数据位则填充在其余的位置。 C 正确，接收端会根据接收到的数据位重新计算一组校验值，并与接收到的校验位进行比较（通常是异或）。这些比较的结果组成一个二进制数 S。如果 S 全为 0，则认为没有检测到错误。如果 S 不为 0，其数值就对应了发生单个比特错误的位在整个海明码字中的位置（编号）。接收端可以通过翻转该位置的比特来纠正错误。D 错误，海明码的最小海明距离设计为 3。

22．B。【解析】海明码是一种可以纠正一个比特错的纠错码，基本思想是：在 k 比特信息上附加 r 比特冗余信息（也称校验比特），构成 n=k+r 比特的码字。其中每个校验比特分别和某几个特定的信息比特构成偶校验关系，共构成 r 个关系式。接收端对这 r 个关系式进行校验，即将每个校验比特与相关联的信息比特进行异或运算，结果为校正因子。如果 r 个校验因子都为 0，表示传输没有出错；如果 r 个校正因子不全为 0，表示传输出现差错。最后根据校正因子的不同取值，可以判断差错发生在哪一个比特上。为了使每一比特的差错都有相应标志，必须保证校正因子的不同取值数大于等于发送码字的比特数。因此，冗余信息的比特数 r 必须满足条件：$2^r >= k + r + 1$。k=10，r 的取值为 4，即正确答案为 B。

23．B。【解析】停止-等待协议使用 1 位序号（0 和 1）来区分连续的数据帧。接收方通过期望的帧序号来判断收到的帧是否是新的。如果收到的帧序号与期望的序号相同，说明是新帧，接收并发送对应序号的 ACK。如果收到的帧序号与期望的序号不同，说明是重复帧（通常是因为之前的 ACK 丢失导致发送方超时重传）。此时接收方应丢弃该重复帧，并重新发送对上一个正确接收帧的确认。在本题中，乙期望 DATA1 但收到了 DATA0，说明 DATA0 是重复帧。乙之前一定正确接收了 DATA0 并发送了 ACK0（该 ACK0 可能丢失了）。所以乙应该丢弃这个重复的 DATA0，并重发 ACK0，以便让甲知道 DATA0 已被正确接收过。

24．D。【解析】停止-等待协议的可靠传输机制依赖于：

- 数据帧编号：通常用 1 位序号（0 和 1）来区分不同的帧，以便接收方检测重复帧。
- 超时重传定时器：发送方发送帧后启动定时器，若超时未收到确认则重传。
- 确认帧：接收方用确认帧告知发送方已正确收到数据帧。

停止-等待协议的发送窗口和接收窗口大小均为 1 (WT=1, WR=1)。接收窗口大于 1 是滑动窗口协议（如 GBN、SR）的特性，用于提高信道利用率，但不是停止-等待协议的机制。

25．D。【解析】在理想或低误码情况下，滑动窗口协议通常利用率更高；但不能说停止-等待协议在任何情况下都一定更低。

26．C。【解析】时延带宽积=数据传输速率 x 单向传播时延，由此可得数据传输速率为 40kbps。则数据帧的发送时延为 1000B/40kbps=0.2s。一个发送周期为 0.2s+2*0.2s=0.6s，则信道最大利用率为 0.2s/0.6s=33.3%。

27．D。【解析】采用 GBN，发送窗口等于$2^3-1=7$，当发出 5 号帧且收到 3 号帧的确认时，发送窗口的数据帧为 4、5、6、7、0、1、2 在未超时重传的情况下，4 号帧和 5 号帧不可再发送，还可发送另外 5 个帧，选择 D 选项。

28．D。【解析】帧编号的比特数为 4，则可允许的 GBN 协议最大发送窗口为 15，发送方已经发送了发送窗口允许的最大数量帧，即已经发送了 0-14 号数据帧，现在受到了 ACK12，即 0-12 号帧已经被确认，15、0-11 号帧可以继续被发送，共计 13 个

29．A。【解析】GBN 采用累计确认，收到 ACK3，说明此前的帧 0, 1, 2, 3 都已经被确认，因此发送方记录的最早未被确认的帧序号应为 4。当计时器超时，发送方会重传所有已发送但未被确认的帧，即从 4 号帧开始。已发送但未被确认的帧是：4, 5, 6。所以，发送方需要重传 4, 5, 6 这 3 个帧。

30．C。【解析】为实现信道利用率最大化，发送窗口 W 必须足够大，能在一个收发周期内连续发送数据。为了满足所有情况，需要按照最差条件进行计算，即数据帧长度取 500B。此时，发送时延为$500B/1Mbps=4ms$，则一个收发周期为 2*4ms+200ms*2=408ms，共可以发送 408ms/4ms=102 个帧，帧序号需要 7 个比特。

31．D。【解析】当 GBN 发送方发生超时时（通常是针对发送窗口内最早未被确认的帧），它会重传该帧以及在该帧之后已发送的所有帧（即发送窗口内所有已发送但未确认的帧）。

32．B。【解析】SR 协议要求对于 k 比特编码的系统，发送窗口、接收窗口之和不大于2^k。若接收窗口大于发送窗口，则没有意义，故发送窗口=接收窗口时，接收窗口取最大值，为 4。

33．C。【解析】自始至终甲的发送窗口未移动，发送窗口中 0 号帧已发送未确认，1 号帧已发送已确认，2 号帧已发送未确认，3 号帧已发送未确认。因此，甲的发送窗口内等待确认的帧集合是 {0, 2, 3}。

34．A。【解析】SR 协议中，接收方可接收未按序到达但没有误码并且序号落入接收窗口内的数据分组，但只有在按序接收数据分组后，接收窗口才能向前滑动到相应位置。此外，SR 协议中，接收方的接收窗口长度不变且连续。

35．B。【解析】TDM 在发送端将不同用户的信号相互交织在不同的时间片内，沿同一个信道传

输，在接收端再将各个时间片内的信号提取出来，还原成原始信号。为了实现 TDM，必须满足如下条件：介质的位速率（即每秒传输的二进制位数）大于单个信号的位速率。介质的带宽（即所能传输信号的最高频率与最低频率之差）大于结合信号的带宽（即所有信号经过调制后形成的复合信号的带宽）。

36．D。【解析】A.正确，这是 STDM 与传统 TDM 的主要区别之一，它根据用户的实际数据发送需求来分配时隙。B 正确，由于按需分配，避免了传统 TDM 中固定分配时隙可能造成的浪费，因此能提高线路利用率。C 正确，正因为时隙是动态分配而不是固定分配给某个用户的，所以每个时隙（或时隙中的数据单元）必须包含某种形式的用户标识（如地址信息），以便接收端能够正确地将数据分发给目标用户。D 错误，STDM 通常也会定义一个帧的结构，其总长度可能是固定的，但与 TDM 不同的是，在这个帧内，时隙不是预先固定分配给特定用户的，而是根据当前哪些用户有数据发送来动态填充，因此用户占用时隙的顺序是可变的。

37．D。【解析】FDM 通过将总带宽划分为若干不重叠的子信道来区分用户。由于滤波器存在带通和带阻之间过渡带不够理想的物理特性，为防止一个信道中的信号泄露（串扰）到相邻信道，必须在子信道之间设置不分配给任何用户的保护频带。

38．B。【解析】TDM 将时间划分为一个个重复的帧，每个帧再划分为若干个时隙，并分配给不同的用户。所有用户必须共享同一个帧同步信号，并严格在自己被分配到的极短时隙内进行收发。若时间同步出现偏差，一个用户的数据就会发送到别人的时隙中，造成严重干扰。

39．A。【解析】在 CDMA 中，每个比特信息都要编码成码片宽度的序列，所以要实现 b bps 的比特信息数据率，应该要 bm bps 的实际数据率。

40．C。【解析】借助不同站点码片彼此正交的性质（利用线代），求得 D 的码片为（1，-1，-1, 1）。第一位数据：取接收序列的前 4 位(2, 2, -2, 2)，与（1，-1，-1, 1）做内积运算再除以码片长度 4，得到结果为+1，代表发送比特为 1。同理，第二位数据为 0，第三位数据为 0。

41．B。【解析】CDMA 系统的实际通信中，分为上行通信和下行通信两个部分，基站掌握所有主机的码片序列，但是每个主机只知道自己的码片序列。

上行通信中，由主机给基站发送数据序列，如果想发送比特 1，就发送自己的码片序列，想发送 0，就发送码片序列的反码。基站收到多个主机发来的信号后，根据各主机的码片序列依次解析出收到的信息，并从信息中获取目的站，在下行通信中，根据目的站的码片序列来发送数据，目的站收到信息后，根据自己的码片序列来解析出比特信息。

回到本题，由于发送比特 0，所以应该发出的序列为 A 的码片序列的反码，二进制 0 一般由-1 表示，所以应该是（-1，-1，-1，-1）。基站收到后，解析出比特 0，再根据 B 的码片进行编码，B 收到的应该是 B 的码片序列的反码（-1，1，-1，1）。本题选 B。

42．D。【解析】A 正确，纯 ALOHA 和时隙 ALOHA 都是随机访问控制协议，站点根据自身需求尝试发送。B 正确，时隙 ALOHA 的最大信道利用率（约 36.8%）高于纯 ALOHA（约 18.4%）。C 正确，纯 ALOHA 允许站点随意发送，不需要同步；时隙 ALOHA 则要求所有站点在统一的时隙边界发送，需要全局时间同步。D 错误，纯 ALOHA 和时隙 ALOHA 协议在检测到冲突（通常是发送后未收到确认）后，都会等待一个随机时间然后进行重传。如果都不重传，协议就无法工作。

43．C。【解析】ALOHA 协议（包括纯 ALOHA 和时隙 ALOHA）本身不包含复杂的冲突检测机制（如 CSMA/CD 中的边发边听）。发送方发送完一帧数据后，会启动一个计时器等待接收方返回的确认帧（ACK）。如果在预设的超时时间内未能收到 ACK，发送方就假定其发送的帧发生了碰撞或丢失，并启动重传过程。

44．B。【解析】非坚持 CSMA 协议规定，如果信道忙，则放弃监听，等待一个随机时间后再重新监听。这与 1-坚持 CSMA（信道忙则持续监听，一旦空闲立即发送）相比，可以减少多个等待的站点在信道刚变空闲时同时发送数据而导致冲突的概率。但代价是可能增加了数据发送的平均延迟，因为站点在监听到信道忙后会等待一段时间而不是持续监听。

45．C。【解析】p-坚持 CSMA 协议通常用于时隙信道。其工作原理是若监听到信道忙，则持续监听到信道空闲；一旦信道空闲则以概率 p 发送数据，以概率 (1-p) 推迟到下一个时隙。在下一个时隙，如果信道仍然空闲，则重复步骤。

46．D。【解析】碰撞检测必须在站点发送帧的整个过程中持续进行，至少要持续到一个争用期（2τ，两倍端到端传播时延）的时间。这是因为，如果站点 A 开始发送，在信号传播到网络最远端的站点 B 之前，站点 B 也可能开始发送，从而引发碰撞。这个碰撞信号需要时间传播回站点 A。如果站点 A 在碰撞信号到达前就停止了发送和检测（例如帧太短或检测时间不够），它可能会误认为发送成功。因此，即使开始发送，也必须持续检测直到可以确认没有碰撞发生。

47．D。【解析】CSMA/CD 协议应用于半双工通信，在使用 CSMA/CD 协议时，一个站点不可能同时进行发送和接收，也就不可以进行全双工通信，因此答案为 D。

48．D。【解析】最短时间τ：A 和 B 同时发送，信号在中点碰撞，冲突信号传回两端。总时间 = 走一半路程($\tau/2$)+返回一半路程($\tau/2$)= τ；最长时间 2τ：A 先发，在信号即将到达 B 时 B 才发。冲突发生在 B 端，A 检测到冲突需要一个来回的总时间。总时间 =A 的信号传到 B(τ) + 冲突信号传回 A(τ) = 2τ。

49．A。【解析】最小帧长为 128B，则帧的发送时延为$T_D = 128 * 8b/100Mb/s = 10.24\mu s$。在一个冲突域内两个站点之间的单向传播时延最多为$T_D/2 = 5.12\mu s$。

50．C。【解析】1km/2 $*$ 10^5km/s=5μs，由题易知，B 在 5μs 时检测到碰撞，A、B 检测碰撞的时间相差 3μs，那么 A 在 8μs 检测到碰撞。从而可推 B 在 3μs 时发送数据帧。B 发送时，A 已经传了 3μs，还剩 2μs，相向而行，易知在 4μs 时碰撞，因此答案为 C。

51．C。【解析】为了确保发送方在发送完一帧前能检测到可能发生的冲突，要求帧的发送时延不能小于信号在网络中往返传播时延的两倍。即最小帧长/数据传输速率=2*最大单向传播时延=2*最远物理距离/介质传播速度。数据传输速率为 100Mb/s，最小帧长为 64B，介质传播速度为 200m/μs，代入可得最大物理距离为 512m。

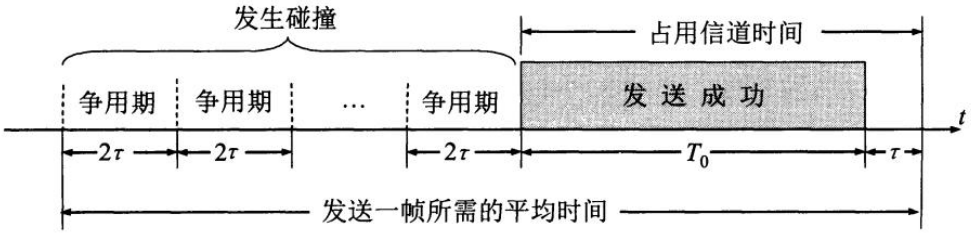
52．A。【解析】最小帧长L_{min}/数据传输速率（R）=2*最大单向传播时延，即$L_{min} = 2R * d/V$，数据传输速率 R 和信号传播速度 V 保持不变，则 2R/V 为常数，最小帧长L_{min}与最大距离 d 成正比，则最大距离 d 减小为原来的一半时，最小帧长也应当减少为原来的一半。

53．D。【解析】100BASE-T 以太网的争用窗口为 5.12μs，即可允许的往返传播时延为 5.12μs，单

向传播时延为 2.56μs，再减去集线器处理消耗时间，剩下 1μs 为信号可在通信链路上传播时间，乘以传播速率可得最远距离为 200m。

54．A。【解析】对于连续 11 次冲突，根据退避算法，第 n 次重传的退避时间是从[0, 2^k-1]中随机选择一个数 r，退避时间为 r*争用时间片，其中 k = min(n, 10)。对于第 11 次重传，n=11，所以 k = min(11, 10) = 10。随机数 r 的取值范围是[0, 2^k-1] = [0, 1023]。因此，最大退避时间为 1023 * 50μs = 51150μs。由于冲突次数 11 小于 16 次的最大重传限制，所以网卡在等待后会进行重传。对于连续 16 次冲突，以太网标准规定，如果一帧连续冲突 16 次（1 次初始发送 + 15 次重传），则认为网络拥塞严重或出现故障，发送方将放弃发送该帧，并向上层报告错误。A 选项正确。

55．A。【解析】下图是以信道被占用的情况，当没有发生碰撞的时候，信道的利用率最大。CSMA/CD 是半双工，所以同一时间只有单向的数据传输，且没有确认机制，在帧全部发送出去前没有检测冲突，就默认本次数据传输没有发生冲突。所以成功发送完一个帧的总信道占用时间是$T_0+\tau$，故信道利用率为$\frac{数据传输时间}{总信道占用时间}=\frac{T_0}{T_0+\tau}$。



补充：以太网通常有一个参数$\alpha=\frac{\tau}{T_0}$，代入信道利用率的公式，可以得到极限信道利用率$S_{max}=\frac{1}{1+\alpha}$。

α越小，即争用期（2τ）与数据传输时间占比越小，即只要一碰撞，就可以立即检测出来，资源被浪费的时间就非常少。

56．B。【解析】CSMA/CA（载波侦听多路访问/冲突避免）是一种随机访问介质访问控制协议，主要用于无线局域网。在这种协议中，所有用户共享同一个信道并进行争用。虽然协议名称中包含“冲突避免”，但它并不能完全消除冲突，例如“隐藏站”问题就会导致冲突。它通过一系列机制（如随机退避、RTS/CTS）来尽量降低冲突发生的概率，但冲突仍然是可能发生的，故 B 正确。CDMA（码分多址）、FDMA（频分多址）、TDMA（时分多址）都属于信道划分介质访问控制方法。这类方法通过将资源（码道、频率、时间）预先分配给各个用户，使得不同用户在通信时使用相互隔离的资源，因此从根本上避免了冲突的发生。

57．D。【解析】CSMA/CA 是无线局域网标准 802.11 中的协议，它在 CSMA 的基础上增加了冲突避免的功能，ACK 帧是 CSMA/CA 避免冲突的机制之一。

58．C。【解析】在 IEEE 802.11 无线局域网中，当一个站点有新的数据帧要发送时，它首先会侦听信道。如果信道被检测为忙，则站点必须推迟发送。如果信道被检测为空闲，站点并不会立即发送，而是需要等待一段称为 DIFS 的时间。如果在 DIFS 间隔后信道仍然保持空闲，站点才会开始随机退避过程，并在退避计时器减到 0 时发送数据帧（或 RTS 帧）。SIFS 用于高优先级传输，如 ACK 或 CTS。

PIFS 用于 PCF 模式下的优先接入。EIFS 用于检测到错误帧后的等待。

59．B。【解析】发送方 A 发送一个 RTS (Request to Send) 帧，该帧中包含一个持续时间字段。这个时间包含了后续的 CTS、数据帧 (Data) 以及确认帧 (ACK) 所需的总时间。目的方 B 收到 RTS 后，回复一个 CTS 帧，CTS 帧中也包含了根据 RTS 计算出的剩余持续时间。其他站点会解析帧中的持续时间字段，并将该值设置到自己的网络分配向量 (NAV) 计时器中。

60．D。【解析】网络分配向量是 IEEE 802.11 MAC 层协议中实现虚拟载波侦听的关键机制。它是一个计时器，用于记录信道被其他站点预约的剩余时间。一个站点收到一个带有持续时间字段的帧时，它会检查该帧中的持续时间值。如果该值大于其当前 NAV 计时器的值，站点就会用这个更大的值来更新自己的 NAV。如果收到的持续时间值小于当前的 NAV，则忽略该值，保持 NAV 不变。 当 NAV 为 0 时，即使物理信道也空闲，也需要等待一个 DIFS 和随机退避时间再发送，D 错误。

61．C。【解析】站点 C 能够正确接收到主机 A 发送的 RTS 帧，但听不到 AP 发送的 CTS 帧。因此，站点 C 的 NAV 值是由 A 发送的 RTS 帧中的“持续时间”字段决定的。发送方 A 在发送 RTS 帧时，其“持续时间”字段需要覆盖从 RTS 帧发送结束到整个数据交换过程（即收到 ACK）结束所需的总时间。这个过程包括：

- 一个短帧间间隔（SIFS）。
- 一个 CTS 帧的发送时间。
- 又一个 SIFS。
- 一个数据帧的发送时间。
- 再一个 SIFS。
- 一个 ACK 帧的发送时间。

则 RTS 帧中设置的持续时间为 $3 * SIFS + T_{CTS} + T_{DATA} + T_{ACK}$，即 384μs。

62．C。【解析】CSMA/CA 协议规定在倒计时期间，信道变为忙碌状态（即有其他站点在发送），则该站点必须暂停其退避计时器，当信道在之后重新变为空闲时，该站点必须再次等待一个 DIFS 间隔，然后从暂停时的计数值开始，继续进行倒计时，而不是重新选择随机值。

63．C。【解析】局域网协议（例如 802.3 标准）定义了 OSI 参考模型下两层的功能，即物理层和数据链路层。其中，数据链路层又被进一步细分为负责为上层提供统一接口的逻辑链路控制（LLC）子层和负责管理共享信道访问的介质访问控制（MAC）子层。

64．B。【解析】10BASE-T 是早期以太网标准之一，其命名规则中：10 表示数据传输速率为 10 Mbps。BASE 表示基带传输。T 表示传输介质为非屏蔽双绞线 (UTP)。该标准使用两对 UTP（通常是 3 类或 5 类），采用曼彻斯特编码，最大段长为 100 米，通常使用 RJ-45 连接器连接到集线器或交换机。

65．B。【解析】以太网帧在物理层发送时，实际帧的前面会添加前同步码和帧起始定界符。前同步码内容为交替的 10101010。用于使接收端的锁相环能够与发送端的比特流时钟同步。帧起始定界符紧跟在前同步码之后，用于明确指示以太网帧的第一个比特（即目的 MAC 地址的第一个比特）即将开始。这两个字段加起来共 8 字节，它们不计入 MAC 帧本身的长度，通常由物理层硬件处理。

66．B。【解析】为了保证 CSMA/CD 协议的有效冲突检测（即最小帧长要求，通常为 64 字节，减

去首部和尾部 18 字节后，数据字段最小为 46 字节），如果上层交付给 MAC 子层的数据长度小于 46 字节，MAC 子层必须在数据字段的末尾填充一些字节（通常是全 0），使其达到 46 字节的最小长度。这个填充字段不包含在 IEEE 802.3 帧的长度字段所指示的长度内，长度字段只指示实际数据的长度。

67．C。【**解析**】如果在使用静态地址的系统中有重复的硬件地址，两个设备都无法通信。在局域网中的每台设备必须有唯一的硬件地址。

68．C。【**解析**】以太网（包括 Ethernet V2 和 IEEE 802.3）帧的结束不是由特定的帧结束定界符来标记的。当物理层检测到信道上不再有信号（即违规编码法）时，就认为一帧已经结束。此外，以太网还规定了帧间最小间隔。

A 选项错误，以太网 V2 用“类型字段”，没有长度字段。

B 选项错误，帧长不固定。

D 选项错误，FCS 用于错误检测，不是结束判断依据。

69．C。【**解析**】为在 1Gbps 的高速率下维持一个合理的冲突域（约 200 米），同时又能检测到所有冲突，千兆以太网将争用时间片（Slot Time）从传统以太网的 64 字节时间大幅增加到了 512 字节时间，故 B 错误。这个新的争用期要求最小有效帧长也相应增加。为了兼容以太网 64 字节的最小帧长规定，千兆以太网引入了载波延伸机制。该机制规定如果一个 MAC 帧（从目的地址到 FCS）的总长度小于 512 字节，那么发送方在发送完帧的最后一个比特（**FCS 之后**）会继续发送一些特殊的扩展符号，直到总的发送时间达到 512 字节的争用时间片为止。这些扩展符号并不在数据字段内，在接收端会被识别并丢弃，不会提交给网络层，故 A 和 D 错误。此外，如果一个 MAC 帧的长度本身就已经等于或超过 512 字节，那么其发送时间自然就满足了最小争用期的要求，此时发送方在发送完该帧后就无需再进行载波延伸，故 C 正确。

70．C。【**解析**】分组突发仍然是在 CSMA/CD 的框架下工作的，它是在站点通过正常的介质争用（可能包括发送经过载波延伸的第一个帧）成功获得信道后的一种发送方式，并非无视 CSMA/CD。其发送的帧序列长度也有限制（如不超过 1500 字节或略多）。故 A 错误。分组突发是连续发送多个独立的 MAC 帧，而不是将它们合并成一个超长帧。每个帧仍有其独立的头部和 FCS。故 B 错误。当一个站点有多个短帧要发送时，它发送第一个帧（如果短，则进行载波延伸）。如果发送成功，它可以继续发送后续的帧，直到达到一定的累积长度限制，而不需要为每个后续的短帧都重新执行完整的 CSMA/CD 退避和竞争过程。这减少了开销，提高了发送一连串短帧的效率。故 C 正确。碰撞后的重传机制仍遵循二进制指数退避算法。分组突发本身不改变重传速率或机制，它的作用是在成功获取信道后提高发送效率。故 D 错误。

71．C。【**解析**】BSS 使用两种标识符：SSID（服务集标识符）和 BSSID（基本服务集标识符）。SSID 是用户可见的、用来标识一个无线网络的字符串名称（用户可以自己命名的），用于网络发现和连接。而 BSSID 则是在 MAC 层唯一标识一个 BSS 的标识符，在有基础设施的 BSS 中，BSSID 就是该 BSS 中 AP 的 MAC 地址。

72．C。【**解析**】A 正确，SSID 是用户可定义的网络名，长度有限制。B 正确，AP 通过信标帧宣告网络的存在及其 SSID。C 错误，为了实现客户端在 ESS 内不同 AP 间的无缝漫游，这些 AP 通常会配

置相同的 SSID，让客户端感觉连接的是同一个网络。它们通过不同的 BSSID（AP 的 MAC 地址）来区分。D 正确，这是客户端连接无线网络的标准步骤。

73．A。【解析】该帧由 AP 发给无线网络中主机 H，则 From DS 置为 1，To DS 置为 0。在此情景下，三个地址字段的定义如下：

- 地址 1：目的地址（DA），无线链路上的接收者，即 00-11-22-AA-BB-CC。
- 地址 2：AP 的 MAC 地址，即 00-11-22-DD-EE-FF。
- 地址 3：源地址（SA），即发起此次通信的最初始的设备地址，即路由器 00-11-22-11-22-33。

74．A。【解析】从 AP 到 AP 的转发场景下，802.11 帧首部中的去往 DS(To DS)和来自 DS(From DS)两个标志位都被置为 1。当 To DS = 1 且 From DS = 1 时，四个地址字段的定义如下：

- 地址 1：接收方地址（RA），即该无线链路上下一跳的直接接收者，即 00-12-34-DD-EE-FF。
- 地址 2：发送方地址（TA），即该无线链路上帧的直接发送者，即 00-12-34-AA-BB-CC。
- 地址 3：最终目的地址（DA），即整个数据传输的最终接收站点，即 00-DE-AD-CA-FE-02。
- 地址 4：原始发送地址（SA），即整个数据传输的最初发起站点，即 00-DE-AD-BE-EF-01。

75．B。【解析】令牌环网不是靠“冲突检测”来工作的；相反，它是通过令牌控制机制本身避免冲突，不需要再检测冲突；冲突检测是以太网（CSMA/CD）的机制，而不是令牌环网。

76．D。【解析】VLAN 技术在逻辑上对网络进行了划分和管理，但它并不一定简化物理拓扑或显著降低成本和维护复杂性。VLAN 需要使用支持 VLAN 的交换机，硬件成本可能更高。此外，VLAN 的配置虽然能方便管理，但也使后期维护更加复杂。

77．B。【解析】Trunk 接口的核心作用是在交换机之间传输多个 VLAN 的流量。为了区分这些流量属于哪个 VLAN，帧在通过 Trunk 接口时必须携带 802.1Q 标签。接收端的交换机根据标签中的 VID 来识别帧的归属。Access 接口才是负责在将帧转发给终端设备时去除标签的端口。

78．D。【解析】802.1Q 帧的最大长度为 1522 字节，因为加入了 4B 的 VLAN 标签。同时，如果最短帧长为 64B，那么最短有效载荷是 64-22=42B。

79．D。【解析】路由器是网络层设备，只认识主机 H 的 IP 地址，无法看见链路层的 AP，因此会广播 ARP 请求报文，获取主机 H 的 MAC 地址，A 选项错误。路由器得到主机 H 的 MAC 地址后，将会发送目的 MAC 地址为 00-12-34-56-78-9a 的以太网帧（此时，还处于有线网络），而非 802.11 帧，故 B、C 选项错误。AP 收到目的 MAC 地址为 00-12-34-56-78-9a 的以太网帧后，会将其转换成 802.11 帧，再进行转发。该 802.11 帧由 AP 发送给主机 H，to DS 置为 0，from DS 置为 1，地址 1、2、3 分别是主机 H 的 MAC 地址、AP 的 MAC 地址、路由器 R 的 MAC 地址，故 D 选项正确。

80．B。【解析】广域网（WAN）和局域网（LAN）的主要区别在于它们覆盖的地理范围。LAN 通常覆盖较小的区域，如一栋建筑或一个校园，而 WAN 则可以覆盖一个城市、一个国家甚至全球范围。

81．C。【解析】互联网（Internet）是一个全球性的、由大量不同类型网络（包括局域网 LAN、城域网 MAN、广域网 WAN）通过路由器和网关相互连接而成的巨大网络。广域网是构成互联网的重要组成部分，但互联网本身不是一个单一的广域网。

82．C。【解析】PPP 协议是一个面向比特（同步链路）或面向字符（异步链路）的协议，它提供差错检测，可以通过帧检验序列（FCS）字段检测帧在传输中是否出错，出错的帧通常被丢弃。PPP 不提供差错纠正、序号、确认或重传机制，因此是不可靠的。PPP 本身没有流量控制机制，且 支持多种网络层协议。PPP 协议建立连接需要经过协商阶段，但数据传输本身是无连接的（每个帧独立处理）。

83．C。【解析】PPP 协议通过 FCS 字段提供差错检测功能，但它不提供差错纠正功能。检测到有差错的帧通常会被简单丢弃，纠错和重传由上层协议（如 TCP）负责。

84．B。【解析】PPP 协议在异步传输（面向字符的链路）时，采用字节填充（字符填充）技术来实现透明传输，规则如下：将信息字段中出现的每一个 0x7E（标志字符 F）字节转变为 2 字节序列(0x7D, 0x5E)。将信息字段中出现的每一个 0x7D（转义字符本身）字节转变为 2 字节序列(0x7D, 0x5D)。若信息字段中出现 ASCII 码的控制字符（即数值小于 0x20 的字符），则在该字符前面要加入一个 0x7D 字节，同时将该字符的编码与 0x20 进行异或运算（如 0x03 转为 0x7D, 0x23）。因此，当要发送的数据字节为 0x7D 时，它会被转义为 0x7D, 0x5D。

85．C。【解析】IPCP（IP Control Protocol）是网络控制协议族 (NCPs) 的一个成员。PPP 设计为一个多协议封装器，它使用 NCPs 来为不同的网络层协议（如 IP、IPX、AppleTalk 等）协商和建立相应的网络层配置。

86．C。【解析】当链路的底层不可用（例如物理连接断开，载波丢失等）时，无论链路之前处于何种状态，它都会回到链路静止状态。只有当物理层条件恢复后，才可能尝试从链路静止状态开始重新建立链路。

87．A。【解析】在 PPP 协议中，由于其设计用于点对点链路，链路上只有两个节点，因此不需要复杂的地址字段来区分发送方和接收方。对于点对点链路，这个字段没有实际的寻址作用，接收方通常也不需要校验它。B 选项中，PPP 帧数据载荷可以是 0B，即只包含首部和尾部字段共 8B，因此正确。

88．A。【解析】H1 向 H2 发送一个数据帧，S1 无法在表中找到对应条目，会将其从除了端口 1 外所有端口进行转发，S2 收到后通过自学习，增加表项<H1，1>。H3 向 H1 发送一个数据帧，S2 通过自学习增加表项<H3，2>。

89．A。【解析】H1 向 H2 发送数据帧 F1，交换机表为空，无法找到 H2 所在端口，因此会将其从除了发送端口（端口 1）之外的全部端口转发，即{2, 3}，且交换机通过自学习，添加 H1 对应表项<H1, 端口 1>。H3 向 H1 发送了一个数据帧 F2，交换机查表，能够发现对应表项，会直接从端口 1 进行转发。

90．C。【解析】A 错误，交换机的 MAC 地址表主要是通过自学习动态构建的，虽然也支持静态配置，但主要依赖动态学习。B.错误，除了 MAC 地址和端口号，MAC 地址表项通常还可能包含其他信息，如所属 VLAN ID（在支持 VLAN 的交换机中）、老化时间等。但最核心的是 MAC 地址到端口的映射。C.正确，当一个数据帧进入交换机某个端口时，交换机会检查该帧的源 MAC 地址，并将其与进入端口的对应关系记录到 MAC 地址表中（如果该条目不存在或需要更新）。D 错误，当 MAC 地址表满时，通常的处理策略是替换掉一个已有的条目（例如最久未使用的条目，基于老化机制），或者有些交换机可能暂时停止学习或采用其他替换算法，但不会简单地完全停止学习功能。

91．C。【解析】FF-FF-FF-FF-FF-FF 是广播地址，当交换机收到目的地址为该地址的数据帧后，会将其洪泛，即 将该帧复制并从除了接收该帧的那个端口以外的所有其他活动端口转发出去。这样做可以确保广播域内的所有主机都能接收到这个广播帧。

92．D。【解析】A 正确，交换机是数据链路层设备，核心功能是基于 MAC 地址的帧转发。路由器是网络层设备，核心功能是基于 IP 地址的包转发。B 正确，交换机的每个端口是一个冲突域，默认情况下所有端口属于一个广播域（VLAN 可以分割广播域）。路由器的每个接口连接一个独立的广播域（也是一个独立的冲突域）。C 正确，透明交换机（或网桥）对终端设备透明，不改变帧的 MAC 地址。路由器在将 IP 包从一个网络转发到另一个网络时，需要重新封装数据链路层帧头，此时源 MAC 地址是路由器出接口的 MAC，目的 MAC 地址是下一跳或最终目标主机的 MAC。D.错误，路由器在查找路由表时使用最长前缀匹配原则来确定最佳路由。交换机在查找 MAC 地址表时，是进行精确匹配，即查找与帧中目的 MAC 地址完全相同的表项。如果找不到，则泛洪（对未知单播）。没有“最长匹配”的概念用于交换机的 MAC 地址表查找。

93．C。【解析】数据字段为 100B，以太网帧还需要加入 18B 的其他开销，以太网帧的长度为 118B。则转发时延为 $118B/10Mbps = 94.4\mu s$。

94．D。【解析】要实现不同速率端口之间的交换（例如，从 100Mbps 端口接收数据，转发到 1Gbps 端口），交换机必须能够缓存数据。因为输入速率和输出速率不匹配，无法进行“直通”。这种速率不匹配的转换必须采用存储转发方式，即先将整个帧完整地接收到缓存中，然后再以目标端口的速率发送出去。直通方式要求输入和输出端口的速率必须一致。

95．C。【解析】在直通交换方式下，交换机引入的最小转发延迟，是它接收并处理完能够做出转发决策所需的最少数据所花费的时间。这个最少数据就是以太网帧头部的目的 MAC 地址字段。以太网帧的目的 MAC 地址字段长度为 6 字节。已知数据量为 6B，最小转发延迟为 0.24μs，则数据传输速率即为。

96．C。【解析】集线器是物理层设备，它会将收到的信号广播到除接收端口外的所有端口。所有连接到集线器的设备共享同一个冲突域，因此同一时间只能有一个设备发送数据，否则会发生冲突，故 A 正确。交换机是数据链路层设备，它的每个端口都是一个独立的冲突域。当端口连接单个设备时，可以工作在全双工模式下，同时收发数据而不会发生冲突，故 B 正确。当主机发送数据时，如果发现目的 IP 地址与自己在不同的子网，它会把数据包发送给自己的默认网关，由路由器查询路由表进行下一步的转发操作。因此，它发送的数据帧的目的 MAC 地址会是默认网关的 MAC 地址，故 D 正确。当交换机在其 MAC 地址转发表中找不到目的 MAC 地址时，它会执行泛洪操作，即将该帧从除了接收端口外的所有其他端口转发出去，而非直接丢弃该帧，故 C 错误。

第 4 章

网络层

【答案速对】

ABBCC　DCBAD　CCBDA　BBCCB

CBCAD　CDCCC　BDBBD　ADCCC

CCACD　CDADB　BCABD　CBBDD

ABCDD　CBCAC　ADCCA　BCCCB

ACADB　CCDBD　BCBDC　CCAAC

A

1．A。【解析】当中间设备是转发器或者网桥时，仅仅是把一个网络扩大了，而从网络层的角度看，这仍然是一个网络，一般不称之为网络互连。网络互连一般指利用路由器来进行互连和路由选择。

2．B。【解析】电路交换是指在通信开始前，必须先建立一条专用的物理连接（电路）。一旦建立，所有数据都会沿着这条唯一的路径按序传输，就像打电话一样。因此，分组不可能失序。数据报服务不预先建立连接。每个数据包（数据报）都包含完整的源地址和目的地址，并由网络中的路由器独立选择路径进行转发。由于每个数据包的路径可能不同，经历的延迟也可能不同，因此它们到达目的地的顺序可能与发送顺序不一致。题中描述的 P1, P3, P2 的到达顺序是数据报服务的典型特征。虚电路服务是介于电路交换和数据报之间的一种方式。它在数据传输前，会先通过一个连接建立过程，在源和目的主机之间建立一条逻辑连接（虚电路），而不是物理连接。一旦建立，该虚电路上所有的分组都会沿着这条预先确定的逻辑路径传输。因此，分组会按序到达。

3．B。【解析】电路交换会为连接预留并独占带宽资源。而虚电路虽然路径固定，但其所经过的物理链路资源（如带宽）是由多条虚电路共享的，并不会为某一条虚电路预留专用带宽。

4．C。【解析】虚电路服务是网络层的一种面向连接的服务，它在数据传输前通过所有中间节点预先建立一条固定的逻辑连接（即虚电路）。该电路上所有分组都按相同的路径转发。如果路径上的某个中间节点发生故障，则该虚电路被破坏，这条通路上所有的后续通信都会因此中断，故 C 正确。

5．C。【解析】从资源分配的角度，电路交换在建立连接时分配，通信期间独占。在通话或数据传输开始前，网络会分配一条专用的物理通路，其带宽等资源在整个通信期间被该连接独占，即使没有数据在传输，资源也无法被其他用户使用。虚电路在建立连接时预分配逻辑资源，数据传输时动态共享物理资源。在连接建立阶段，会分配虚电路号（VCI）并在沿途节点创建转发表项，这可以看作是“预

分配资源”。但在数据传输阶段，物理链路的带宽等资源是由多条虚电路动态共享的，并非独占。数据报不预分配，完全动态共享。没有连接建立过程。每个数据报在到达路由器时，才临时动态地请求和使用转发所需的资源（缓存、处理等）。资源是在“逐个分组”的基础上进行分配和共享的。

6．D。【解析】北向接口 (NBI) 允许网络应用和业务逻辑向 SDN 控制器表达其网络需求和策略。南向接口 (SBI) 用于控制器与数据平面设备通信。东/西向接口用于分布式控制器间的通信。

7．C。【解析】SDN 控制器负责维护网络状态信息（如拓扑），根据全局网络视图进行路由计算、策略决策，并将这些决策转化为流表规则下发给数据平面的交换机。A 是数据平面设备的功能。B 是应用平面的功能，应用平面通过北向接口与控制器交互。D 是物理层的功能。

8．B。【解析】数据平面的 SDN 交换机主要负责基于控制器下发的流表（Flow Table）进行高效的数据包查找、匹配和转发（或其他处理动作如丢弃、修改）。它们是执行者，而非决策者。A 是控制平面的任务，C 是应用平面的任务，D 是非 SDN 架构的传统网络中，路由器的作用。

9．A。【解析】OpenFlow 流表是 SDN（软件定义网络）中数据平面设备进行转发决策的依据，主要由匹配字段、动作和计数器三部分组成。其中，“匹配字段”是用来识别特定数据流的一组数据包头部字段。源 MAC 地址和 VLAN ID 均属于数据链路层的头部字段。UDP 端口号属于运输层的头部字段。生存时间 TTL 每经过一个路由器就会减 1。由于 TTL 是一个动态变化的字段，不具备标识一条数据流的稳定性，因此不被用作流表的匹配字段。

10．D。【解析】拥塞控制与流量控制不同，流量控制往往是指在发送方和接收方之间的点对点通信进行控制，一般是抑制发送方发送数据的速率，防止接收方来不及接收。而拥塞控制是全局性的，确保网络能够承载所达到的流量。

11．C。【解析】拥塞控制包括两种，开环控制和闭环控制。其中，开环控制指在设计网络时事先将有关发生拥塞的因素考虑周到，力求网络在工作时不产生拥塞。因此在建立网络连接时就进行准入控制，预留所需资源属于开环控制。选项 ABD 均为在发生阻塞时进行处理，属于闭环控制。

12．C。【解析】在转发过程中，逐跳改变的是数据链路层的地址（如 MAC 地址）。路由器会把收到的数据链路层帧（例如以太网帧）解封装，取出 IP 数据报，处理完毕后，再重新封装成一个适合下一跳链路的、全新的数据链路层帧，这个新帧的源/目的 MAC 地址会相应改变。而源 IP 地址和目的 IP 地址则始终保持不变。

13．B。【解析】可以得出 IPv4 数据报的总长度是第 3、4 字节，也就是 00 30，十进制值为 48，因而该 IPv4 数据报的总长度为 48B；IPv4 数据报的首部长度是第 1 字节中的后 4 个比特，第 1 字节的十六进制值为 45（二进制为 01000101），后 4 个比特为 0101，十进制值为 5，由于首部长度以 4B 为单位，因此该 IPv4 数据报的首部长度为 $5 \times 4B = 20B$;综上所述，数据载荷的长度 = 48B -20B = 28B

14．D。【解析】ICMP 差错报告报文是为了报告 IP 数据报在传输中遇到的问题。但是为了避免产生无休止的 ICMP 报文，协议规定了一些不发送 ICMP 差错报告报文的情况：对 ICMP 差错报告报文不再发送 ICMP 差错报告报文；对第一个分片的 IP 数据报片的所有后续数据报片都不发送 ICMP 差错报告报文；对具有多播地址的 IP 数据报都不发送 ICMP 差错报告报文；对具有特殊地址（例如 127.0.0.0 或 0.0.0.0）的 IP 数据报不发送 ICMP 差错报告报文。

15．A。【解析】IP 数据报从源主机传至目的主机可能需要通过不同的物理网络，而每个物理网络都具有其特定的最大传输单元（MTU）。为了将 IP 数据报传输至最终的目的主机，一个 IP 数据报在传输过程中可能要经过多次分片，但所有的分片都只在目的主机进行重组。

16．B。【解析】原始 IP 数据报总长度为 1200B，其首部长度为 20B，则数据部分的长度 = 总长度 - 首部长度 = 1200B - 20B = 1180B。链路的 MTU 为 600B，每个 IP 分片自身也需要一个 IP 首部（20B），单个分片可承载的最大数据量 = MTU-IP 首部长度 = 600B - 20B = 580B。IP 数据报分片时，除了最后一个分片外，每个分片的数据部分长度必须是 8 字节的整数倍，580 显然不是 8 的倍数，因此需要下调至 576B（不大于 580 且是 8 的倍数的最大值）。又每个分片尽可能大，则可以用总数据部分长度 1180B 除以每个分片数据长度最大值 576B，得 2，余数为 28B。即总共需要 3 个分片，每个分片上数据长度依次为 576B、576B、28B。最后一个分片的 MF 标志位为 0（表示后面没有其他分片），片偏移字段为 (第一个分片数据长度 + 第二个分片数据长度) / 8 = (576B + 576B) / 8 = 1152B / 8 = 144。

17．B。【解析】本题设计在 R1 和 R2 两个路由器上的两次分片操作：

在 R1，原始数据报的数据部分长度为 1420 - 20 = 1400 字节。R1 连接的链路 MTU 为 800 字节，除去 20 字节的 IP 首部，每个分片最多能携带 800 - 20 = 780 字节的数据。由于分片的数据长度必须是 8 字节的倍数，因此取不大于 780 的 8 的最大倍数，即 776 字节（97 * 8）。所以，第一个分片 F 的总长度为 776 (数据) + 20 (首部) = 796 字节。

在 R2，分片 F（总长 796 字节，数据 776 字节）到达 R2，R2 连接的链路 MTU 为 400 字节。同样，每个子分片最多能携带 400 - 20 = 380 字节的数据。取不大于 380 的 8 的最大倍数，即 376 字节（47 * 8）。因此，从 F 产生的第一个子分片的总长度字段为 376 (数据) + 20 (首部) = 396 字节。故 B 正确。

18．C。【解析】当 IP 数据报的 DF 位置为 1 时，表示该数据报禁止被分片。如果路由器接收到这样的数据报，并且发现其大小超过了出接口的 MTU，路由器不能强行分片，而应该丢弃该数据报，并向源主机（主机 A）发送一个 ICMP“目的不可达”报文，其中代码（Code）字段指明具体原因为“需要分片但 DF 置 1”。

19．C。【解析】CIDR 是一个用于给用户分配 IP 地址以及在互联网上有效地路由 IP 数据包的对 IP 地址进行归类的方法。网络前缀越少，包含的地址越多。

20．B。【解析】一个主机同时连接在多个网络上时，该主机就必须有多个 IP 地址。

21．C。【解析】采用端口映射的目标是让多个私网主机共享一个公网 IP。当 H1 和 H2 的报文到达 NAT 路由器时，路由器会将它们的源 IP 都替换为公网 IP 202.10.10.1。为了在响应报文返回时能正确地区分哪个报文是给 H1 的、哪个是给 H2 的，NAT 路由器必须确保转换后的源端口号是唯一的。即使 H1 和 H2 的原始源端口号相同（都是 1025），NAT 路由器也会为它们分配两个不同的外部端口号（例如，将 H1 的会话映射到 202.10.10.1:50001，将 H2 的会话映射到 202.10.10.1:50002）。因此，发出去的两个分组源 IP 相同，但源端口号必然不同。

22．B。【解析】入站报文到达 NAT 路由器后，路由器会提取其目的 IP、目的端口号。本题中，接收报文目的 IP 为 201.1.1.1，目的端口为 60000。路由器会用这组信息去查找 NAT 转换表的“外网 IP：端口”这一列。从表中可见，第三行条目<TCP, 192.168.1.4:1027, 210.1.1.1:60000>完全匹配。根据该条

目的映射关系，路由器会将报文的目的 IP 和目的端口号改写为对应的“内网 IP:端口”，即 192.168.1.4:1027，然后向内网转发。

23．C。【解析】观察表项，源 IP 地址为 192.168.32.56、源端口为 80 并不存在，此时要看端口是熟知端口还是临时端口号：

- 熟知端口号的表项由管理员手动配置，所以不匹配直接丢弃，答案选 C。
- 临时端口号，如果不匹配，会动态生成一个新的条目，然后继续转发。

因此本题选 C。

24．A。【解析】NAT 技术允许多台主机通过共享一个全局唯一的公网 IP 地址来访问外部网络。当一个位于外部网络的主机要向内部主机 H1（192.168.0.2）发送 IP 分组时（这通常是对 H1 之前发出的请求的响应），它不能直接使用私有地址 192.168.0.2 作为目的地址，因为私有地址在公网上是不可路由的，该分组的目的地址必须是 NAT 路由器 R2 的公网地址。又 R1 连接 R2 的端口的 IP 地址为 210.1.1.2/30，在此单对单链路上，主机号不能全 1 也不能全 0，还能给 R2 连接 R1 端口分配的 IP 地址仅有 210.1.1.1，故选择 A。

25．D。【解析】223.255.255.255 为 C 类地址，且最后 8 位全为 1，是广播地址，因此不能作为源地址，仅可作为目的地址。

26．C。【解析】IP 组播地址代表的是一组主机，而不是某一台特定的主机。因此，D 类组播地址（224.0.0.5）只能出现在 IP 数据报的目的地址字段中。一个 IP 数据报的源地址必须是发送该报文的单个主机的单播地址。B 选项，网络号全 0，主机号非全 0，表示本网络上一个主机，仅能用于源地址，无法作为目的地址。

27．D。【解析】一个子网的网络地址必须是其子网大小的整数倍。对于/20 的子网，掩码是 255.255.240.0，关键在于第三个八位字节。240 的二进制是 11110000，这意味着第三个八位字节的后 4 位是主机位，前 4 位是网络位。因此，第三个八位字节的划分步长（块大小）是 24 = 16，故一个有效的/20 子网，其第三个八位字节的值必须是 16 的整数倍。

28．C。【解析】将已知的子网 192.168.10.64/26（地址范围 192.168.10.64 - 192.168.10.127）与选项 C（地址范围 192.168.10.160 - 192.168.10.191）组合，则 IP 网络的地址范围至少也是 192.168.10.0 - 192.168.10.255，那么剩余的未分配地址空间会被分割成三个不连续的“空洞”（即 0-63、128-159 和 192-255），无法用两个子网的地址空间覆盖。

29．C。【解析】原始网络的网络前缀为 17 位，原始主机为 32-17=15 位。继续划分子网，需要划分成 32 个规模相同的子网，因此需要从 15 位主机位中借走 5 位，子网剩余主机位为 10 位，共有

个 IP 地址，又主机位全 0 的地址是网络地址，主机位全 1 的地址是广播地址，这两个地址不能分配给主机使用，因此最终结果为 1022 个。

30．C。【解析】由题可知，主机号为 8 位，若主机号只占 1 位，则没有可用的主机 IP 地址（全 0 全 1 不可作为主机 ip），因此最少保留 2 位主机号，子网划分位数为 8-2=6，因此最多有 2^6^=64 个子网，全 0 全 1 可用作子网（教材上说过全 0 全 1 现可做子网号，如果答案中没有 64，只有 62，应结合题目其他条件判断），因此最多可以划分 64 个子网。对于子网内最多的主机数，应考虑不做子网划分

的情况，即可容纳 256-2=254 台主机（如问子网内 IP 地址或者地址块 IP 数量最多多少个，应写 256，区分主机数和 IP 地址，IP 地址为全 0 全 1 不可用作主机地址）。

31．B。【解析】互联网上的路由器只会转发目的地址为公有 IP 地址的数据包，对于特殊用途的 IP 地址（如私有 IP 地址、广播地址等）则通常不予转发。私有 IP 地址的范围为 10.0.0.0-10.255.255.255、172.16.0.0-172.31.255.255、192.168.0.0-192.168.255.255，选项中 A 和 C 均属于私有 IP 地址，因此不会被路由器转发。选项 D 是受限广播地址。发往此地址的数据包用于在发送方所在的本地网络（广播域）内进行广播。路由器的一个基本功能就是隔离广播域，因此不会转发目的地址为 255.255.255.255 的数据包，以防止广播风暴。

32．D。【解析】如下表所示

网络号	主机号	源地址使用	目的地址使用	代表的意思
0	0	可以	不可	在本网络上的本主机（见 6.6 节 DHCP 协议）
0	host-id	可以	不可	在本网络上的某台主机 host-id
全 1	全 1	不可	可以	只在本网络上进行广播（各路由器均不转发）
net-id	全 1	不可	可以	对 net-id 上的所有主机进行广播
127	非全 0 或全 1 的任何数	可以	可以	用于本地软件环回测试

33．B。【解析】每个子网至少容纳 5 台主机，再加上网络号和广播地址，即每个子网至少需要 7 个 IP 地址，因此主机位设置为 3，网络前缀长度为 29，对应的点分十进制子网掩码为 255.255.255.248。

34．B。【解析】部门 A 需要 50 台主机，加上子网号和广播地址，共需要 52 个 IP 地址，主机号为 6 位。同理，部门 B 需要 22 个 IP 地址，主机号为 5，部门 C 需要 12 个 IP 地址，主机号为 4。首先给部门 A 分配地址空间，从 192.168.5.0 开始，分配 192.168.5.0/26。该块占用了 192.168.5.0 到 192.168.5.63 的地址。然后给部门 B 分配地址空间，下一个可用的起始地址是 192.168.5.64。分配 192.168.5.64/27 给部门 B。该块占用了 192.168.5.64 到 192.168.5.95 的地址。因此，分配给部门 B 的子网网络地址是 192.168.5.64。

35．D。【解析】各台计算机的子网掩码均为 255.255.255.240，其二进制形式（只显示最后一个字节）为 255.255.255.11110000，将图中的各计算机的 IP 地址与子网掩码进行“与”操作，可得到各计算机所在的子网地址：计算机 A 与 E 的子网地址为 192.168.1.32，而计算机 B、C、D 的子网地址为 192.168.1.48。如果这些计算机采用 Ethernet 交换机进行连接，那么，计算机 A 与 E 之间可以直接访问，而计算机 B、C、D 之间可以直接访问。

36．A。【解析】H1 的 IP 地址为 192.168.12.1，H2 的 IP 地址为 192.16.12.6，它们的 IP 地址分别与子网掩码进行**按位与**运算，如果得到的结果均为 192.168.12.0（即网络号），因此可以判断它们在同一个子网。在同一个子网（或广播域）内的主机之间进行通信，属于本地通信，不经过路由器。因此 H1 和 H2 可以正常通信。

37．D。【解析】IP 地址为 192.168.96.125，子网掩码为/18，即 255.255.192.0。将 IP 地址和子网掩码转换为二进制进行按位与运算，得到网络号 192.168.64.0。广播地址是将网络地址中的主机位全部置为 1。/18 表示网络位为 18 位，主机位为 32 - 18 = 14 位。网络地址 192.168.01000000.0 的主机位是后 14

位。将这 14 位全置为 1，得到 192.168.01111111.11111111，转换为十进制即 192.168.127.255，故 D 正确。

38．C。【解析】子网掩码为 255.255.255.128，则其网络前缀长度为 25 位，主机号为 7 位，总 IP 地址数为 128 个。去除主机号全 0 和全 1 两种特殊情况，可分配给主机的 IP 地址共 126 个，因此可容纳的最大主机数为 126。网络地址和子网掩码按位与运算得到网络地址为 192.168.32.0，第一个可分配给主机的 IP 地址是网络地址加 1，因此第一个可分配的主机 IP 地址是 192.168.32.1。

39．C。【解析】将一台主机的 IP 地址与其子网掩码进行按位与运算，得到的结果就是该主机所在网络的网络地址。已知主机 IP 地址和网络地址，只需要从头开始一一对比，找出两个地址的最大相同前缀长度即可，为/20，选择 C。

40．C。【解析】该子网的子网掩码为 29，则网络号 29 位，主机号为 3 位。共有 8 个 IP 地址，除去用于表示网络号的全 0 和用于表示广播地址的全 1，剩余 6 个 IP 地址可以被分配给主机。

41．C。【解析】本题考查两个知识点：通过给定 CIDR 地址块，找出该地址块的全部细节(最小地址，最大地址，地址数量，地址掩码)地址块聚合方法(找共同前缀)。192.168.16.192/26 与选项 A 的地址块聚合后的地址块为 192.168.16.0/24，会引入多余地址。192.168.16.192/26 与选项 B 的地址块聚合后的地址块为 192.168.16.0/24，会引入多余地址。192.168.16.192/26 与选项 D 的地址块有部分重叠。192.168.16.192/26 与选项 C 的地址块聚合后的地址块为 192.168.16.128/25，聚合地址块不会引入多余地址。

42．C。【解析】当一台主机需要解析目标 IP 地址对应的 MAC 地址时，它会发送一个 ARP 请求报文。由于此时发送方不知道目标 MAC 地址，ARP 请求报文需要在本地网络中被所有主机接收到，以便拥有该 IP 地址的主机能够响应。因此，ARP 请求报文在数据链路层（如以太网）是以广播方式发送的。其封装的以太网帧的目标 MAC 地址是广播地址 FF:FF:FF:FF:FF:FF。当网络中的某台主机（或路由器接口）确认 ARP 请求中的目标 IP 地址是自己的 IP 地址时，它会发送一个 ARP 响应报文给请求方。这个响应报文包含了该主机的 MAC 地址。由于请求方（发送 ARP 请求的主机）的 MAC 地址在 ARP 请求报文中是已知的（作为发送方 MAC 地址），因此 ARP 响应报文可以直接发送给请求方，采用单播方式。其封装的以太网帧的目标 MAC 地址是 ARP 请求报文中的源 MAC 地址。

43．A。【解析】首先，主机 H1 需要判断目的主机 H2 的 IP 地址 192.168.2.20 是否与自己在同一子网。H1 的 IP 地址是 192.168.1.10/24，其网络号是 192.168.1.0，而 H2 的 IP 是 192.168.2.20/24，其网络号是 192.168.2.0。由于网络号不同，主机 H1 判定主机 H2 位于不同的子网。当要向不同子网的主机发送数据时，主机 H1 必须将数据包发送给其配置的默认网关（路由器）。因此，主机 H1 需要获取默认网关 192.168.1.1 的 MAC 地址，以便将 IP 数据包封装成数据链路层帧发送给网关。如果 H1 的 ARP 缓存中没有默认网关的 MAC 地址条目，它会发送一个 ARP 请求，该请求的目标 IP 地址是默认网关的 IP 地址，即 192.168.1.1。

44．C。【解析】ARP 报文被直接封装在数据链路层帧中，其中 ARP 请求报文采用广播，目的 MAC 地址为 FF-FF-FF-FF-FF-FF，交换机会根据 ARP 报文的目的 MAC 地址将其向除了接收端口外的所有其他端口转发。

45．D。【解析】ARP 报文是直接封装在数据链路层帧（例如以太网帧）中的，其在以太网帧中的类型字段值为 0x0806。IP 协议依赖 ARP 来解析 MAC 地址，而不是 ARP 依赖 IP 协议进行传输，故 A 错误。ARP 协议最初设计时并未考虑复杂的安全问题，因此它本身缺乏有效的身份验证机制。这使得 ARP 欺骗等攻击成为可能，故 B 错误。当目标主机收到针对其 IP 地址的 ARP 请求报文后，它会执行以下两个操作：1、更新自身的 ARP 缓存：它会从收到的 ARP 请求报文中提取发送方的 IP 地址和 MAC 地址，并用这个映射关系更新或创建自己的 ARP 缓存条目。这样做是为了方便后续向请求方发送数据。2、发送 ARP 响应报文：向请求方单播一个 ARP 响应报文，其中包含自己的 IP 地址和 MAC 地址。所以，除了发送 ARP 响应，还会更新 ARP 缓存，故 C 错误。ARP 协议除了 ARP 请求报文和响应报文外，还有其他类型的报文，例如用于检测 IP 地址冲突的“无故 ARP”，以及选项所述免费 ARP、代理 ARP 等，故 D 正确。

46．C。【解析】由于 IP 协议提供的是一种不可靠、无连接、“尽力而为”的服务，不能提供差错控制与查询等机制。在 TCP／IP 协议中，差错报告、查询与控制功能由 ICMP 协议完成。ICMP 协议定义了两种类型的报文：差错报告报文与查询报文。其中，差错报告报文又可分为 5 类：目的站不可到达、源站抑制、超时、参数问题和改变路由。查询报文又可分为 4 类：回送请求／应答、时间截请求／应答、地址掩码请求／应答、路由器询问／通告。流量调整报文不属于 ICMP 的报文。

47．D。【解析】当路由器和主机由于拥塞而丢弃数据报时，就向源点发送源点抑制报文，是源点知道应当把数据报的发送速率放慢。

48．A。【解析】每经过一个路由器，TTL 值都会减一，TTL 字段一定会改变;校验和是通过对首部数据计算得到，每经过一跳路由器首部都可能发生变化，都会重新计算校验和，故校验和字段一定会改变;不考虑 NAT 时，在整个传输过程中源 IP 地址和目的 IP 地址都不变;由于分组不被分片，故 DF 和 MF 字段不会改变。故选择 A 选项。

49．D。【解析】A 选项错误。TCP 报文段总长度为 3000B，则传递给网络层的数据报的数据部分长度为 3000B。MTU 为 1400 包括 20 的首部和 1380 的数据部分，但是这里注意，分片长度要是 8 的整数倍，1380 并不是 8 的整数倍要找到距离 1380 最近且是 8 的整数倍的数字，则是 1376，分片的最大数据部分应该是 1376。数据报的数据部分是 3000，因此可以分成 1376、1376、248 这 3 个分片。由于题目问的是总长度，还需要加上 20 的首部，因此 3 个分片的总长度分别为 1396，1396 和 268。

B 选项错误。MF 表示的是后面是否还有分片，对于第 3 个分片来说，后面没有分片，因此 MF=0，DF 表示是否允许分片，由于此网络已经对 IP 数据报进行了分片，所以肯定是允许分片，DF-0 表示允许分片。

C 选项错误。IP 数据报的标识字段用于标识属于同一个数据报的分片，因此应该设置为同一个值，以便目的主机将多个分片组装成原始的 TCP 报文段。

D 选项正确。在收到最后一个 IP 数据报之后，目的主机需要查看每个 IP 数据报的 MF 标志位，将数据报的 TCP 数据字段按照顺序依次组装成一个完整的 TCP 报文段。在本题中，由于 MF 标志位的设置，目的主机需要将前两个 IP 数据报的 TCP 数据字段拼接起来，再加上最后一个 IP 数据报的 TCP 数据字段，组成完整的 TCP 报文段。

50．B。【解析】常见的 ICMP 差错报告报文有终点不可达、时间超过、参数问题和改变路由(重定向)4 种。当路由器或主机不能交付数据报时，就向源点发送终点不可达报文。A 选项错误。当路由器收到生存时间(TTL)字段为 0 的数据报时，除丢弃该数据报外，还要向源点发送时间超过报文，即 ICMP 超时消息。B 选项正确。当路由器或目的主机收到的数据报的首部中有的字段的值不正确时，就丢弃该数据报，并向源点发送参数问题报文。C 选项错误。路由器把改变路由报文发送给主机，让主机知道下次应将数据报发送给另外的路由器(可通过更好的路由)。D 选项错误。

51．B。【解析】在采用双协议栈时，在 IPv4 数据报和 IPv6 数据报之间的转换时，IPv6 数据报的如流标号无法对应到 IPv4 数据报中的任一字段，因此会丢失，故 I 正确。IPv6 基本首部中的有效载荷长度字段指明了紧跟在 40 字节基本首部之后的所有内容的总长度，这包括了所有的扩展首部以及上层协议数据单元（PDU），故II错误。IPv6 基本首部包含一个“下一个首部”字段，用于指明第一个扩展首部的类型或上层 PDU 的类型。如果存在扩展首部，则每个扩展首部内部也包含一个“下一个首部”字段，指向下一个扩展首部或最终的上层 PDU。因此，通过扩展首部这个机制以及其中的“下一个首部”字段，最终可以指明数据报搭载的是何种协议数据单元 PDU，故III错误。解决 IPv4 地址耗尽的问题是设计 IPv6 的最根本动机，IPv6 地址长度为 128 位，足以在可预见的未来为全球所有设备提供唯一的 IP 地址，从而根本性地解决了 IPv4 地址耗尽的问题。

52．C。【解析】IPv6 路由器接收到超出下一跳链路 MTU 的 IPv6 数据报时，会丢弃该数据包，并向源主机发送一个 ICMPv6 差错报文。该报文中会包含下一跳链路的 MTU 值，以便源主机 A 可以据此调整后续发送的数据包大小。

53．A。【解析】在使用零压缩时，可以使用双冒号“::”对相继的 0 值域进行缩写，而选项 B 并没有采用，故错误。双冒号“::”在一个地址中只能出现一次，故 C 错误。原 IPv6 地址中存在两段相继的 0 值域，第一段：在 2001 和 ABCD 之间，有 2 个 0 段。第二段：在 ABCD 和 0001 之间，有 3 个 0 段。为了达到最优的缩写效果，应该压缩更长的那一段，即后面的 3 个 0 段。因此，最优的缩写形式是 2001:0:0:ABCD::1，D 错误，A 正确。

54．B。【解析】一个完整的 IPv6 地址由 8 个 16 比特的段组成，总共 128 比特。IPv6 地址 FE80::CE82:8 中，::前面有 1 个段(FE80)，后面有 2 个段(CE82 和 8)。因此，::代表的 0 段数量是 5 个。每个段是 16 比特，所以::代表的比特数是 5*16=80 比特。

55．D。【解析】IPv6 的地址是 128 位，表示将 128 位分成 8 段，每段有 16 个二进制位。IPv6 使用冒号十六进制记法，把每个 16 位的值用十六进制表示，各值之间用冒号分隔。在十六进制记法中，允许把数字前面的 0 省略。也允许零压缩，即一连串的 0 可以用一对冒号取代。

56．C。【解析】ARP 协议是专门用于解析 IPv4 地址到 MAC 地址的。在 IPv6 的环境中，ARP 协议不再被使用。IPv6 设计了一套全新的机制来替代它，即利用 ICMPv6 报文中的邻居请求和邻居通告报文来完成地址解析的功能，故 C 错误。

57．B。【解析】动态路由选择与静态路由选择最主要的区别在于动态路由选择使用路由选择协议去自动发现并维护路由信息，而静态路由选择使用手动配置的路由信息。

58．B。【解析】BGP (边界网关协议)在不同的 AS 之间交换路由信息，属于域间路由选择协议。

IGP 是内部网关协议这一类别的总称，而不是一个具体的协议名称。OSPF (开放最短路径优先协议)是一种典型的链路状态路由协议，它工作在 AS 内部，属于 IGP。RIP (路由信息协议)是一种经典的距离向量路由协议，同样工作在 AS 内部，属于 IGP，通常用于规模较小的网络。

59．D。【解析】RIP 相关报文使用运输层的用户数据报协议 UDP 进行封装，使用的 UDP 端口号为 520，故 A 错误。当存在多条到达同一目的网络的路由时，路由器会全部保存，不同下一跳且 RIP 距离相等，可用于等价负载均衡，故 B 错误。RIP 协议实现简单，路由器开销小，但是限制了最大 RIP 距离为 15，限制了使用 RIP 的自治系统 AS 的规模，OSPF 协议更适合路由器较多的网络，故 C 错误。RIP 协议采用周期性交换，但是当网络拓扑发生变化时，路由器需要及时向邻居路由器通告，这是 RIP 协议的触发更新机制，故 D 正确。

60．D。【解析】R1 收到来自 R2 的更新报文后，会先将报文内距离向量的下一跳更改为 R2，跳数加 1（即变为<N, 8>），然后与自身现有路由表进行对比。由于路由器 R1 的当前路由表有一条到达网络 N 的路由，其下一跳为 R3，跳数为 8，与新来的路由信息下一跳不同但跳数相同，路由器会将其保存，用于等价负载均衡。

61．A。【解析】只有当 R1 从另一个邻居（比如 R3）收到一条成本更高的路径时，它才会因为路由成本更高而忽略它。但对于来自当前下一跳的更新，必须接受。因此 R1 收到 R2 关于网络 N 的路由信息，会将原路由表中信息更新。

62．B。【解析】R2 和网络 N1 直连，因此 R2 自己的路由表中关于 N1 的条目是<N1，1>，R2 会将该路由条目直接发送给 R1。R1 收到之后，将跳数加 1，下一跳修改为 R2，之后加入自己的路由表。由于采用了水平分割机制，让路由器记录收到某个特定路由信息的接口，而不让同一路由信息再通过此接口向反方向传送，因此在 R1 向 R2 发送自己的距离向量时，不会发送此前从 R2 处学习到的关于 N1 的路由信息。

63．C。【解析】将 B 的每一个距离矢量+1，将 C 的每一个距离矢量+3，取他们的最小值，除了到达自己本身的开销为 0。结果是（0，1，2，4）。

64．D。【解析】路由器 B 在收到邻居 A 的 RIP 更新报文后，会将其中的每一条路由与自己的路由表进行比较。首先，B 的路由表中没有网络 N1 的条目，因此会从 A 学习并新增该路由（距离为 5+1=6），此时 B 的路由表总数变为 5 条。接着，对于已有条目，B 会进行判断：到达网络 N4 的新路径（距离 3+1=4）比原路径（距离 5）更优，因此会更新该条目；到达网络 N2 的新路径（距离 7+1=8）虽然比原路径（距离 2）更差，但由于原路径的下一跳就是 A，B 必须接受来自 A 的更新；而到达网络 N3 的新路径（距离 8+1=9）劣于 B 当前已有的更优路径（距离 4），故不更新。因此，最终 B 的路由表总条目数为 5，而被更新的原有条目为 N2 和 N4 这两条。

65．D。【解析】OSPF 虽然没有使用 TCP，但它在协议内部自己设计了复杂的确认和重传机制，足以保证其链路状态信息的可靠同步。因此，OSPF 是可靠的路由协议，故 D 错误。

66．C。【解析】OSPF 采用最短路径优先算法计算最优路径，从 Net A 到 Net B 的最优路径是 R1-R3-R5-R7，开销是 2+5+7+14+2=30，最小开销的那条。

67．B。【解析】链路状态协议的核心思想就是，通过泛洪（Flooding）机制，确保同一区域内的

所有路由器最终拥有一个完全相同的链路状态数据库（LSDB）。

68．C。【解析】OSPF 规定，任何两个非骨干区域之间的路由信息交换，都必须经过 Area 0 中转，因此所有非骨干区域都必须与骨干区域直接相连，C 正确。

69．A。【解析】OSPF 协议的分组类型有 5 种：Hello 分组：发现相邻路由器并建立邻居关系，还可保持邻居的可达性；DBD 分组：给出链路状态数据库的摘要信息，供邻居对比自身数据库并找出需要的信息 LSR 分组：请求对方发送特定链路的详细信息；LSU 分组：以洪泛方式发出链路状态通告，响应链路状态请求分组；LSACK 分组：对链路状态更新分组进行确认。Keepalive 报文是 BGP-4 的报文类型，用于发布或撤销网络的可达性信息，即向邻站通告新的网络，或当网络变为不可达时，撤销原来的可达性信息。用来保持与邻居的连接的 OSPF 协议分组为 Hello 分组，故选择 A 选项。

70．C。【解析】为了获取这些缺失或过时条目的完整详细信息，路由器会发送一个链路状态请求（LSR）分组，明确指出需要哪些 LSA 的详细内容。

71．A。【解析】OSPF 协议通过发送 Hello 分组来发现和维护邻居关系。为了使同一链路上的所有 OSPF 路由器都能接收到这些分组，OSPF 使用组播进行通信。224.0.0.5 是预留给所有 OSPF 路由器的专用组播地址。因此，当一台路由器发送 Hello 分组时，其 IP 头部的目的地址字段会被设置为 224.0.0.5。选项 C 和 D 中的 255.255.255.255 是广播地址，不适用于 OSPF。OSPF 报文是直接封装在 IP 数据报中的，它不使用 TCP 或 UDP 等运输层协议。在 IP 数据报首部中，有一个 8 位的“协议”字段，用于指明其数据部分所承载的是哪种上层协议。根据 IANA 的规定，分配给 OSPF 协议的协议号是 89。选项 B 中的协议号 6 代表 TCP，选项 C 中的 17 代表 UDP，均不正确。

72．D。【解析】BGP 有四种报文类型：OPEN（打开）、KEEPALIVE（保活）、UPDATE（更新）和 NOTIFICATION（通知）。OPEN 用来与相邻的另一个 BGP 发言人建立关系，使通信初始化。KEEPALIVE 用来周期性地证实邻站的连通性。UPDATE 用来通告某一条路由的信息以及列出要裁撤的多条路由。NOTIFICATION 用来发送检测到的差错。

73．C。【解析】组播组是一个逻辑上的概念。任何主机，无论其物理位置或 IP 地址如何，只要它通过 IGMP 协议向其本地路由器“声明”自己希望加入某个组播组（例如 224.1.2.3），它就成为了该组的成员。这种成员关系是动态的，主机可以随时加入或离开。

74．C。【解析】IGMP 协议只工作在本地网络中，用于主机向其直连的路由器加入或离开某个组播组。它不是一个用于在路由器之间传播路由信息的协议，也不是端到端的协议。

75．A。【解析】IP 多播需要使用两种协议：网际组管理协议 IGMP、**多播**路由选择协议。

76．B。【解析】为了减少网络上不必要报文，IGMP 协议规定，当收到通用查询后，该组内的每台主机都会启动一个随机的计时器。第一个计时器到期的主机会发送一个成员关系报告。网络上的其他同组成员在发送自己的报告前，会先侦听网络。一旦它们听到了已有人替本组发出了报告，它们就会取消自己的计时器，不再发送报告。这样，路由器只要收到一个报告，就知道本网段还有成员，从而避免了报告报文的泛滥。

77．C。【解析】IGMP 是网络层的协议，用于协助 IP 组播。因此，IGMP 报文被直接封装在 IP 数据报中，其 IP 首部的协议字段值为 2。

78．C。【解析】路由器只关心在其连接的某个网段上，对于某个组播组是否存在至少一个成员。它不关心、也不记录具体是哪几台主机、它们的 IP 地址是什么。只要有一个成员存在，它就向该网段转发。

79．C。【解析】IP 多播地址低 28 位可任意变化，但是以太网多播 MAC 地址仅 23 位可以任意变化，因此会存在多个 IP 地址被映射到同一个 MAC 地址。将 IP 多播地址映射到 MAC 多播地址时，MAC 多播地址的高 25 位保持不变（前 24 位始终是 01-00-5e，第 25 位始终是 0），低 23 位取 IP 多播地址的低 23 位即可。

80．B。【解析】IPv4 组播地址到以太网 MAC 地址的映射规则为 MAC 地址的前 3 个字节为 01-00-5E，MAC 地址的第 4 个字节，取 IP 地址的第 2 个字节，并将其最高位置为 0，MAC 地址的第 5 和第 6 个字节，直接使用 IP 地址的第 3 和第 4 个字节。

81．A。【解析】IP 地址中只有后 23 位被映射到 MAC 地址中，而 IP 组播地址有 28 位是可变的（高 4 位 1110 固定），这意味着有 28-23=5 个比特的信息在映射中丢失了。因此，多个（最多 32 个）不同的 IP 组播地址可能会映射到同一个 MAC 地址，从而产生冲突。B、C、D 对应 IP 地址的后 23 位与 225.1.2.3 相同，会导致主机意外接收到发往这些组播组的分组，因此选择 A。

82．C。【解析】IP 组播模型中，发送者和接收者的角色是分离的。任何主机，无论它自身是否加入了某个组播组，都可以向该组播组的地址发送数据报，故 A 错误。组播转发树的拓扑结构会因为网络拓扑和组成员关系的变化而变化，是动态的，故 B 错误。IP 组播地址到以太网 MAC 地址的映射不是一一对应的，而是多对一的。MAC 组播地址的前 24 位固定为 01-00-5E，第 25 位为 0，只剩下 23 位可供映射，而 IP 组播地址（D 类）的前 4 位是 1110，后面有 28 位是可变的组标识。映射时，只将 IP 组播地址的后 23 位复制到 MAC 地址的后 23 位。这导致 IP 组播地址中剩下的 5 位信息丢失了。因此，会有 32 个不同的 IP 组播地址映射到同一个 MAC 组播地址上，故 D 错误。为了覆盖多播组全部成员，多播转发树可能包含一些中间的路由器。这些路由器本身可能并不直接连接任何该组播组的成员，它们的作用仅仅是作为转发节点，将收到的组播数据报沿着树的下一个分支继续转发下去，故 C 正确。

83．A。【解析】封装了 IGMP 成员查询报文的 IP 多播数据报，其目的地址为 224.0.0.1，这是一个特殊的 IP 多播地址，本网络中所有参加多播（无论参加的哪个组）的主机和路由器都会收到该多播数据报。由于 IGMP 仅在本网络内有效，因此封装了 IGMP 报文的 IP 多播数据报，其首部中的生存时间 TTL 字段均会被设置为 1，以避免封装了 IGMP 报文的 IP 多播数据报被路由器转发至其他网络。

84．D。【解析】转交地址是临时的、与位置相关的。当移动节点从一个外地网络移动到另一个外地网络时，它会获得一个新的转交地址，并需要向归属代理重新注册。因此，转交地址是会随着移动节点的漫游而改变的。永不改变的是归属地址。

85．B。【解析】在移动 IP 中，使用隧道技术将发往移动节点归属地址的数据报转发到其当前的转交地址。隧道技术的本质是一种封装。归属代理收到发往移动节点的原始 IP 数据报后，并不修改它，而是把它看作一个整体的数据块，在其外部再封装上一个新的 IP 首部。

86．C。【解析】封装后的外部 IP 数据报的目的是将内部的原始数据报从归属代理转发到转交地址。因此，它的源地址是发送方 HA 的地址，目的地址是接收方（外地代理或移动节点自身）的转交地

址。而被封装的内部 IP 数据报的源目地址（通信对端 IP、移动节点归属地址）保持不变。

87．C。【解析】移动 IP 对通信对端是透明的。通信对端像往常一样，只对移动节点的永久归属地址发送数据报，归属代理收到后再转发给移动节点的转交地址，再由移动节点的外地代理转发给移动节点，故 C 错误。

88．D。【解析】移动 IP 对运输层透明。这意味着 TCP 层在建立连接时，始终使用稳定不变的归属地址作为自己的身份标识。移动节点发送 SYN 报文，IP 层的源地址是归属地址。Web 服务器收到后，会向这个连接的端点回复 SYN+ACK 报文，IP 层的目的地址是移动节点的归属地址。这个报文会被路由到归属代理，再被隧道转发。

89．B。【解析】当外地代理从 IP 隧道中收到并解封出原 IP 数据报时，会在自己的代理注册表中查找移动主机的永久地址所对应的 MAC 地址，并将该数据报封装到目的地址为该 MAC 地址的帧中发送给移动主机。如果查询路由表，转发给移动节点的永久地址，会导致该 IP 数据报又被发回移动主机的归属网络。

90．D。【解析】路由器的结构分为路由选择部分和分组转发部分，分组转发部分根据转发表将输入端的数据包移至适当的输出端口，包含一组输入端口、一组输出端口以及最核心的交换结构。I 正确;路由器能隔离广播域，其每一个端口都是一个广播域，可控制网络中的广播风暴，II正确;路由表是根据路由选择算法计算得到，通过软件来实现的。转发表用于将目的地址映射成输出链路，是基于路由表得到的，III 正确。故选择 D 选项。

91．B。【解析】选项 A 描述错误。IP 路由器队列长度达到某个值得警惕的数值时，也就是网络出现了某些拥塞征兆时，就主动丢弃到达的 IP 数据报来造成发送方的超时重传，进而降低发送方的发送速率，因而有可能减轻网络的拥塞程度，甚至不出现网络拥塞。选项 B 描述正确。将 IPv4 地址与相应的子网掩码进行逐比特的逻辑与运算，就可得到该 IPv4 地址所在子网的网络地址。选项 C 描述错误。默认路由用 0.0.0.0/0 表示，斜线后面的 0 指明网络前缀的长度为 0（相应地址掩码为 0.0.0.0），按最长前缀匹配原则，默认路由的匹配优先级最低。选项 D 描述错误。特定主机路由用某个特定主机的 IP 地址/32 表示，斜线后面的 32 指明 ④ 网络前缀的长度为 32 (相应地址掩码为 255.255.255.255)，按最长前缀匹配原则，特定主机路由的匹配优先级最高。

92．C。【解析】只采用交换机互连，网络性能下降，且网络中存在大量 DHCP 请求和 ARP 请求报文（均为广播），说明出现了广播风暴。而路由器可以抑制广播风暴，因此应当引入路由器对网络进行子网划分。

93．B。【解析】在一个冲突域内，如果两台或多台设备同时发送信号，就会产生冲突。交换机工作在数据链路层，它能根据目的 MAC 地址，将数据帧仅从特定的端口转发出去，而不是像集线器那样向所有端口广播。因此，交换机的每个端口都构成一个独立的冲突域，它能够有效地隔离冲突域。在一个广播域内，任何一台设备发出的广播帧（目的 MAC 地址为 FF-FF-FF-FF-FF-FF）都会被域内所有其他设备接收。当二层交换机收到一个广播帧时，它的标准操作就是向除源端口外的所有端口进行泛洪。因此，交换机本身不能隔离广播域，故选择 B。集线器和中继器既不能隔离冲突域也不能隔离广播域，路由器既可以隔离冲突域又可以隔离广播域。

94．D。【解析】路由器的根本目的就是为了连接不同的 IP 网络（子网）。如果所有主机的 IP 地址都属于同一个子网，它们之间可以通过二层交换机直接通信，根本不需要路由器。正是因为存在多个不同的子网，才需要路由器在它们之间根据 IP 地址进行路由和转发。

95．C。【解析】路由器是工作在网络层的设备，向网络层以上的层次隐藏下层的具体实现，因此每个局域网的物理层数据链路层和网络层协议可以不同。而路由器不能处理网络层以上的协议数据，因此每个局域网的网络层以上的高层协议必须相同。

96．C。【解析】主机 A 通过 Wi-Fi 向路由器 R 发送数据。此时，IP 数据报被封装在一个 802.11 格式的帧中。路由器 R 从其无线接口接收到这个 802.11 帧。R 作为路由器，将 802.11 帧的头部和尾部解封装，取出中间的 IP 数据报。然后，它根据 IP 数据报的目的地址（B 的 IP 地址）查找路由表，决定应从其有线以太网接口转发。为了在以太网上传输，R 必须将该 IP 数据报封装成一个以太网 II 格式的帧。这个新帧的源 MAC 地址是 R 的有线网口 MAC，目的 MAC 地址是服务器 B 的 MAC。

97．C。【解析】路由表的条目来源有多种：通过路由协议动态学习、由管理员手动配置的静态路由、以及与路由器接口直连的直连路由，故 A 错误。路由器转发分组的依据是转发表，而不是路由协议报文本身，故 B 错误。无论静态路由还是动态路由，最终都需要将路由信息填入路由表中，路由器才能据此进行转发。所谓静态路由，是指路由表中的路由信息是由网络管理员人为设置的，不会因为网络拓扑的变化自行调整，故 D 错误。

98．A。【解析】当主机 H 要向 IP 地址为 8.8.8.8 的节点发送分组时，会首先对比自己的 IP 地址和目的 IP 地址是否位于同一局域网，主机所在网络的网络号为 10.0.0.0，而 8.8.8.8 的前 24 位与之不匹配，因此主机 H 会将 IP 分组发送给默认网关，MAC 地址为默认网关的地址，即 10.0.0.1 的 MAC 地址。默认网关收到来自主机 H 的 IP 分组后，会 TTL 减 1、修改源 MAC 地址为自己的 MAC 地址，重新计算机 IP 首部校验和，但是不会修改目的 IP 地址。注意：源 IP 地址为私有地址，所以发往公网时，需要修改源地址为公网地址。

99．A。【解析】当网络中都没有 ARP 缓存时、H1 和 H2 不在同一子网，H1 不会 ARP 查询 H2 的 MAC 地址，而是查询默认网关的 MAC 地址，也就是路由器 R 的 E1 接口 MAC 地址。因此，R 先收到的第一个以太网帧为 H1 发送的 ARP 请求分组，其目的 MAC 地址为 ff-ff-ff-ff-ff-ff。H1 获得路由器的 MAC 地址后，把数据帧发送给路由器 R，目的 MAC 地址为 R 的 E1 端口，故 R 收到的第二个 MAC 帧的目的地址为 E1 端口的 MAC 地址，即 01-2a-3b-4c-5d-11，故选择 A 选项。

100．C。【解析】路由器在转发分组时遵循最长前缀匹配原则。目的地址 172.18.10.108 同时匹配路由表中的三条路由：172.18.0.0/20、172.18.10.0/24 和 172.18.10.108/32。由于/32 是最长的前缀，路由器将选择该条目进行转发，其对应的下一跳是 R4。因此，正确答案为 C。此外/32 是一种特殊的子网掩码，它代表该路由表项是特定主机路由，也就是一条精确指向单个 IP 地址的路由，而不是指向一个网络。

101．A。【解析】目的地址 192.168.1.100 同时匹配/24、/16 和/0 (默认路由) 三个条目。根据最长前缀匹配原则，/24 是最长的前缀，因此选择接口 Eth0。

第 5 章

传输层

【答案速对】

DCDCB　BCACD　DBDDD　BACDC

BDCAB　BBBAC　ACBBB　ACCAA

BCCCB　ABBD

1．D。【解析】将数据报在多个可能的路径中选择一条进行转发即路由选择，是网络层的核心功能，故选择 D。而 A、B、C 均属于传输层提供的服务。

传输层提供的服务有：①复用分用②端到端通信③可靠传输服务（TCP）④无连接服务（UDP）⑥流量控制⑦拥塞控制⑧差错控制。

2．C。【解析】端口号是运输层的服务访问点(SAP)，只在本地主机上标识进程，故 A 正确。16 位二进制数可表示的范围是$2^{16} = 65536$，故 B 正确。D 选项正确，这是运输层的复用功能。客户端程序使用的是操作系统临时分配的端口号（即短暂端口号，是除了熟知端口号和登记端口号之外的端口号）去连接服务器，故 C 错误。

3．D。【解析】一个 TCP 连接是通过其两端的套接字来唯一确定的，套接字又由 IP 地址和端口号拼接而成。因此，必须由通信双方的 IP 地址和端口号共同组成的四元组才能在整个互联网中唯一地标识出这条连接。任何一个元素不同，都代表一条不同的连接。

4．C。【解析】http://说明使用的是 HTTP (超文本传输协议)，它的熟知端口号为 80，网址中没指明端口应用默认端口 80，故选择 C。443 是 HTTPS 的熟知端口号，25 是 SMTP 的熟知端口号。53 是 DNS 的熟知端口号。

5．B。【解析】复用指在发送端，多个应用进程可以同时使用一个运输层协议（TCP 或 UDP），并将数据流汇集起来发送。分用指在接收端，运输层根据报文段的端口号，将收到的数据分别交付给主机上正确的应用进程。题目描述的是接收端的行为，因此是分用。

6．B。【解析】视频会议等实时应用，对延迟非常敏感，而对偶尔的数据丢失不敏感（丢失一两帧画面影响不大）。TCP 为了保证可靠性，其重传机制会引入较大的延迟，不适合此类应用。UDP 提供无连接、不可靠的服务，没有重传机制，延迟小，非常适合视频会议等实时应用。

7．C。【解析】无连接服务的特点是快速、开销小，但不保证可靠。登录、文件传输、远程访问

都要求数据 100%准确无误、按序到达，必须使用可靠的面向连接服务。而广播时间信息，属于一种周期性的、少量数据的发送，对实时性要求高，且偶尔丢失一次信息影响不大，适合使用开销小的无连接服务。

8．A。【解析】简单网络管理协议 SNMP 用于网络设备的管理和状态监控，为了追求高效率和低开销，通常使用 UDP。简单邮件传送协议 SMTP 用于发送电子邮件。邮件内容必须完整、准确地送达，因此 SMTP 必须基于可靠的传输，需要使用 TCP。远程终端协议 TELNET 用于远程登录到另一台主机。用户输入的每一个字符和主机返回的每一个字符都必须按序、可靠地传输，否则会话将无法进行。因此选用 TCP。文件传送协议 FTP 用于在不同主机间传输文件。文件的完整性至关重要，任何比特的丢失或错误都可能导致文件损坏。因此 FTP 需要使用 TCP 来保证可靠传输。

9．C。【解析】虽然在 UDP 之间的通信要用到其端口号，但是 UDP 的通信是无连接的，而 TCP 之间的通信必须要在两个套接字之间建立连接，故选择 C。如果接收方 UDP 发现收到的报文中的目的端口号不正确（即不存在对应于该端口号的应用进程），就丢弃该报文。并发送 ICMP“端口不可达”差错报文给发送方，故 A 正确。在计算校验和时，要在 UDP 用户数据报之前增加 12 个字节的伪首部，伪首部仅用于计算校验和，会和原 UDP 数据报的首部和数据部分一起参与计算，故 B、D 正确。

10．D。【解析】TCP 是面向连接的、可靠的协议，其首部中的序列号和确认号可以实现按序交付。接收方会根据序列号对失序的报文段进行重排序。而 UDP 是无连接、不可靠的协议，其首部中没有序列号字段，因此它完全不保证报文的顺序，故选择 D。

11．D。【解析】UDP 是面向报文的，发送方的 UDP 对应用程序交下来的报文，在添加首部后就向下交付 IP 层。UDP 对应用层交下来的报文，既不合并，也不拆分，而是保留这些报文的边界。TCP 是面向字节流的，不保证接收方应用程序所收到的数据块和发送方应用程序所发出的数据块具有对应大小的关系（例如，发送方应用层交给发送方的 TCP 共 10 个数据块，但接收方的 TCP 可能只用了 4 个数据块就把收到的字节流交付上层的应用程序）。

12．B。【解析】UDP 首部的“长度”字段记录的是整个 UDP 数据报（包括首部和数据）的总长度。UDP 首部固定为 8 字节。因此，数据部分的长度=总长度-首部长度，即 1492B。

13．D。【解析】应用层数据（UDP 的数据字段）为 8180 字节。在运输层，数据被交给 UDP 协议。UDP 会为其加上一个固定的 8 字节首部，形成一个 UDP 用户数据报。UDP 用户数据报总长度=UDP 数据字段+UDP 首部=8180B+8B=8188B。这个完整的 UDP 用户数据报会作为 IP 数据报的数据部分。因此，网络层需要传输的数据总长度为 8188 字节。链路层的最大传输单元（MTU）为 1500 字节。这限制了 IP 数据报（包括 IP 首部和 IP 数据）的总长度不能超过 1500 字节。IP 数据报的首部长度通常为 20B。因此，一个 IP 分片最多能承载的数据（即分片的数据字段）长度为：MTU-IP 首部长度=1500B-20B=1480B。IP 分片时，除最后一个分片外，每个分片的数据字段长度都必须是 8 字节的整数倍，1480 恰为 8 的倍数，符合要求。8188/1480=5，即有 5 个满载的分片，这 5 个满载分片总共传输了的数据量为：5×1480B=7400B。剩余的数据将全部放入最后一个分片。最后一个分片的数据字段长度 = 总数据长度 - 已发送数据长度=8188B-7400B=788B。

14．D。【解析】UDP 伪首部是一个在计算校验和时临时添加的字段，它本身并不会随着 UDP 报

文在网络中传输。其目的是让校验和不仅能检查 UDP 首部和数据，还能检查一部分 IP 首部的信息，以增强校验能力。UDP 校验包括源 IP 地址、目的 IP 地址、保留位（置为 0）、协议字段（UDP 为 17）、UDP 长度五个部分组成。

15．D。【解析】I 错误，源端口号为 $(CB84)_{16}=52100$；II 正确，目的端口号为 $(000D)_{16}=13$；III 正确，UDP 用户数据报长度为$(001C)_{16}=28B$，UDP 数据载荷长度为 $28-8=20B$；IV 正确，因为目的端口号为 13（熟知端口号），所以该 UDP 用户数据报是从客户到服务器的。答案选 D。

16．B。【解析】CF01 和 34FF 从低到高逐个相加，得到 0000 0100 0000 0000，但由于最高位产生了进位，需要再加 1，即 0000 0100 0000 0001，取反码得到最终结果 1111 1011 1111 1110，即 FBFE。

17．A。【解析】I. TCP 和 UDP 都有端口号，用于标识进程，正确。II. TCP 和 UDP 首部都有 16 位的校验和字段，用于检测报文段在传输过程中是否出现差错，正确。III. 只有 TCP 提供可靠传输，UDP 是不可靠的，错误。IV. TCP 是面向字节流的，而 UDP 是面向报文的，它保留了上层应用交付报文的边界，错误。因此，选 A。

18．C。【解析】TCP 是运输层的协议，它工作在通信的两个端系统上，遵循“端到端”原则。它无法控制网络核心部分的行为，也无法命令路由器为某条 TCP 连接预留任何资源。且 TCP 协议使用 IP 协议提供的服务，IP 协议使用无连接不可靠的数据报，不会预留带宽资源。

19．D。【解析】TCP 校验和覆盖范围包括 TCP 伪首部、TCP 首部和数据部分，故 A 错误。窗口字段用于流量控制，表示接收方当前还能接收的数据量，最大报文段长度 MSS 在扩展首部中指出，故 B 错误。数据偏移字段指出 TCP 报文段的数据载荷部分的起始处距离 TCP 报文段的起始处有多远，实际指出了 TCP 报文段的首部长度，但是其单位是 4 字节，故 C 错误。仅当紧急标志位 URG 置为 1 时，紧急指针有效；当发送方有紧急数据时，可将紧急数据“插队”到发送缓存的最前面，并立刻封装到一个 TCP 报文段中进行发送；紧急指针指出从当前序号开始到紧急数据结束处的偏移量，紧急数据之后是普通数据，接收方收到报文后，会根据紧急指针的值从数据载荷部分取出紧急数据直接上交应用进程，而不必在接收缓存中排队，故 D 正确。

20．C。【解析】SYN 标志位是同步位，仅在 TCP 连接建立时的“三次握手”阶段使用，用于发起一个新连接并同步初始序号，与终止连接无关，故 A 错误。FIN 标志位用于正常、有序地关闭 TCP 连接，发起方表明自己已无数据要发送，从而启动“四次挥手”的正常释放流程，这与应用程序崩溃的突发、异常情况不符，故 B 错误。PSH 是推送标志位，用于告知接收方尽快将数据交付给应用进程，而无需等待缓冲区填满，它涉及的是数据传输的效率而非连接管理，故 D 错误。RST 标志位的作用是异常地、强制地终止一个连接，通知主机 B 不必再发送数据，并立即释放连接资源，故 C 正确。

21．B。【解析】TCP 是面向字节流的，其首部中的序号字段长度为 32 位，用来表示该报文段数据载荷部分第一个字节的序号，也可以看作报文段的序号，当序号字段取到最大值后，下一个序号从最小值 0 重新开始，这被称为序号绕回，故序号绕回周期即为发送 2^32B 数据所花费的时间。时延带宽积=传播时延*带宽，以此可求带宽为 100Mb/s。用总数据量除以带宽可得最终结果，约为 343.6s。

22．D。【解析】UDP 协议的首部由源端口、目的端口、长度、校验和四个字段组成，每个字段 2 字节，其长度是固定的 8 字节；但 TCP 协议的首部除了 20 字节的固定部分外，还包含一个可变长的选项字段，因此 TCP 首部的长度不是固定的，其范围是 20 字节到 60 字节，故 D 错误。

23．C。【解析】为防止旧连接请求报文段的干扰，两端主机在建立 TCP 连接时初始序列号是随机的，故 A 错误。三次握手中第二个报文段的 SYN 和 ACK 字段均需置为 1，不能携带应用数据，但是必须消耗一个序列号，故 B 错误。在“三次握手”过程中，如果报文段丢失，TCP 会使用超时重传机制来重发丢失的报文段，而非立即释放重新执行“三次握手”，故 D 错误。客户端在发送完三次握手中的最后一个 ACK 报文后，认为连接已成功建立，单方面进入 ESTABLISHED 状态并可以开始发送数据。而服务器必须在接收到这个 ACK 报文后，才能确认客户端已收到自己的 SYN-ACK，此时服务器才会进入 ESTABLISHED 状态，C 正确。

24．A。【解析】在三次握手建立 TCP 连接中，客户端发送的第一个连接请求报文段（SYN 段）不携带应用层数据，但其 SYN 标志位本身需要消耗一个序列号，A 正确。三次握手中的第三个报文段是客户端对服务器 SYN 的确认，其 SYN 标志位为 0，故 B 错误。TCP 连接是全双工的，通信的任何一方（无论是客户端还是服务器）都可以在数据传输结束后主动发起连接释放请求（即发送第一个 FIN 段），故 C 错误。TCP 支持“半关闭”，当一方（如客户端）发送 FIN 段后，仅表示该方向的数据传输已结束，但它仍然可以接收来自对方的数据，直到对方也发送 FIN 段来关闭另一个方向的连接，故 D 错误。

25．B。【解析】在标准的四次挥手中，主机 H 发送 FIN 后进入 FIN_WAIT_1 状态；收到服务器的 ACK 后进入 FIN_WAIT_2 状态，等待服务器的 FIN。题中服务器将 ACK 和 FIN 合并发送，这意味着 H 在 FIN_WAIT_1 状态下直接收到了对自身 FIN 的确认以及服务器的 FIN 请求。根据 TCP 状态机，H 会直接发送最后一个 ACK 并跃迁至 TIME_WAIT 状态，因此 FIN_WAIT_2 这个中间等待状态被跳过，故 B 正确。

26．B。【解析】TCP 的确认号指的是“期望收到的下一个字节的序号”。设该段的起始序列号为 seq，数据长度为 500。接收方正确接收该段后，期望的下一个字节序号就是 seq+500。根据题意，ack=seq+500，则可得 seq 为 700。

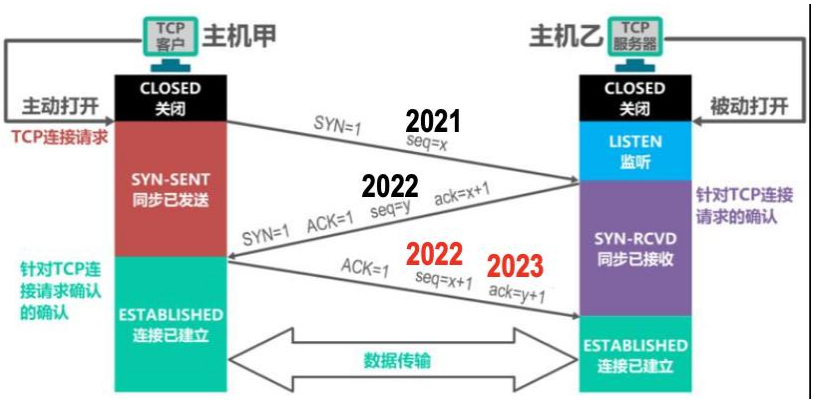
27．B。【解析】发送的第一个数据段序列号为 1000，收到的确认号为 3000，则说明 1000-2999 已经被发送并正确接收，故 A 已经成功向 B 发送了 2999-1000+1=2000B 的应用层数据。

28．B。【解析】TCP 采用累计确认，接收方只对按序到达的、连续字节流的最高序号进行确认。虽然乙收到了 P3，但由于中间的 P2 丢失，字节流在序号 399 之后就断开了。因此，乙无法确认序号大于 399 的数据。它仍在等待从序号 400 开始的数据来填补空洞，因此乙向甲发送的确认号仍然是 400。

29．A。【解析】主机甲的初始报文段（SYN=1, seq=1000）消耗一个序号 1000，因此甲的下一个

可用序号是 1001。主机乙的回应报文段（SYN=1, ACK=1, seq=8000, ack=1001）也消耗一个序号 8000。甲收到乙的回应后，需要发送确认报文段。该报文段的序号(seq)应为甲的下一个可用序号，即 1001；确认号(ack)应为乙的序号加 1，即 8000+1=8001。此时连接已在建立过程中，SYN 标志位应为 0，而 ACK 标志位必须为 1。

30．C。【解析】主机甲发送的第二个 TCP 报文段即第三次握手。甲的初始序号为 2021，第一次握手 SYN 报文消耗一个序号，所以第三次握手报文的起始序号 seq=2022，而主机乙初始序号是 2022，第二次握手消耗一个序号，所以主机甲期望收到主机乙的下一个报文起始序号是 2023，因此 ack=2023，答案选 C。



31．A。【解析】B 发来的报文段中确认序号为 2046，这表示 B 已经成功接收了 A 发送的到 2045 为止的所有字节，并期望接收从 2046 开始的字节，因此 A 发送的新数据段序号应为 2046。A 发送报文段的确认号(ack)则取决于 A 对 B 数据的接收情况。B 发来的报文段序号为 1913，且是纯确认段，不消耗序号。这意味着 B 上一个发送的字节序号是 1912，A 仍在等待从 B 接收序号为 1913 的字节。因此，A 回应的确认号应为 1913。

32．C。【解析】TCP 的确认序号（ack）是期望收到对方下一个报文段的第一个数据字节的序号。主机 A 收到 B 发来的 FIN 段，其序号（seq）为 7700，由于 FIN 标志位即使不携带数据也要消耗一个序号，因此 A 在响应时发送的确认序号应为 7700+1=7701。TCP 的序号（seq）是本报文段所发送的数据的第一个字节的序号。主机 A 收到的报文段中确认序号为 8800，这表明 B 已经正确收到了 A 发送的到 8799 为止的所有字节，并期望下一个字节是 8800。因此，A 发送的响应确认段的序号就应该是 8800，故 C 正确。

33．B。【解析】主机甲发出的第一个 SYN 段，序列号为 x，该 SYN 段会消耗一个序号。因此，甲发送的下一个段的起始序号应为 x+1。甲在收到乙的 SYN-ACK 段后，回送的 ACK 段（三次握手的第三个段），其序号为 x+1。

34．B。【解析】TCP 报文段的序号是该报文段所发送数据的第一个字节的序号。建立连接时，A 发送的 SYN 段消耗一个序号。因此，A 发送的第一个数据字节的序号是 3001。A 共发送了 6000 字节的数据，这些数据的序号范围是[3001, 3001+6000-1]，即[3001, 9000]。当所有数据都发送完毕后，A 的下一个可用序号是 9001。FIN 段是在所有数据都发送完毕后才发送的，因此它将被赋予当前的下一个可用序号。所以，该 FIN 段的序列号应为 9001。

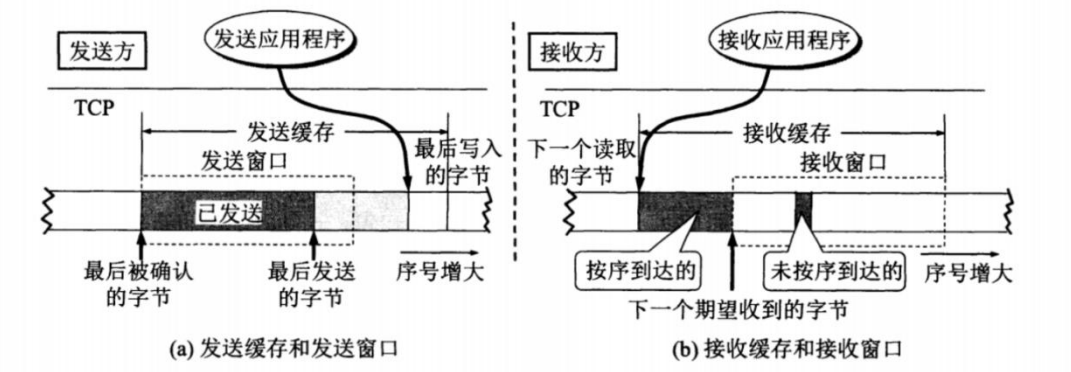
35．B。【解析】主机 B 收到两个相同的分组，一定是 B 的确认报文段丢失了，应该丢失晚到的报文段，然后再发一个确认，I 错误。若接收窗口变小，滑动窗口可能不变。总结如下：

- 发送缓存：应用层想要发送的数据。缓存空间有限并且循环使用，逻辑上是一个圆。其中包含两个部分：

①发送方的应用程序传送给发送方 TCP 准备发送的数据。

②TCP 已发送出但未收到确认的数据。

- 发送窗口：发送窗口里的序号表示允许发送的序号。包含：①已经发送了的但是还未收到确认的②允许发送但是还未发送的。
  - 发送窗口左端点在收到新的确认后往右移动，没收到确认则不动。
  - 发送窗口右端点通常是不断往右移动，因为收到了新的确认。但是也可能不动，因为接收发的接收窗口变小了，导致不动。但是 TCP 标准不赞成往左移动，因为窗口里的数据大部分已经发送出去，现在又要收缩窗口，不让发送这些数据，这样就会产生一些错误。
- 接收缓存：即将交给应用层的数据。包含：①按序到达的、但尚未交付应用进程的数据②未按序到达的
- 接收窗口：用来接收即将到达的数据，按序达到的数据，发送确认后接收窗口往右移动，但是不能超过接收缓存的范围。
- 发送窗口的序号集是发送缓存序号集的子集。
- 接收窗口的序号集是接收缓存序号集的子集。



36．A。【解析】TCP 的发送窗口 swnd 的大小取决于拥塞窗口 cwnd 和接收方通告窗口 rwnd 的较小值，即 swnd=min(cwnd, rwnd)。初始时，cwnd=1KB<ssthresh=16KB，TCP 处于慢开始阶段。在此阶段，每成功收到一个对新报文段的确认，拥塞窗口 cwnd 就增加一个 MSS 的值。A 发送 1KB（1 个 MSS）并成功收到确认后，其 cwnd 变为 2KB。此时，A 的发送窗口大小为 swnd=min(cwnd, rwnd)=min(2KB, 1KB)=1KB。

37．C。【解析】发送窗口的大小取接收方接收窗口和拥塞窗口的较小值，在同一时刻，发送方的发送窗口并不总是和接收方的接收窗口一样大，故 A 错误。“拥塞避免”也并非指完全能够避免拥塞，而是指在拥塞避免阶段将 cwnd 值控制为按线性规律增长，使网络比较不容易出现拥塞，故 B 错误。TCP 的拥塞控制正是为了适应和响应 IP 网络内部的拥塞状态（如路由器丢包）而设计的，两者密切相关，故 D 错误。

38．C。【解析】当发送方收到 3 个冗余 ACK 时，会触发快速恢复算法。此时，慢开始阈值 ssthresh 被设置为当前拥塞窗口 cwnd 的一半，即 ssthresh=16KB/2=8KB，同时拥塞窗口 cwnd 也更新为新的 ssthresh 值，即 cwnd=8KB，并立即进入拥塞避免阶段。在拥塞避免阶段，每经过一个 RTT，拥塞窗口大小增加一个 MSS（1KB）。因此，经过 4 个成功的 RTT 后，拥塞窗口将从 8KB 线性增长 4 次，最终大小为 8KB + 4 * 1KB = 12KB。

39．A。【解析】当发生超时事件时，无论之前的 ssthresh 值是多少，新的 ssthresh 都会被更新为当前 cwnd 的一半，即 ssthresh=16KB/2=8KB，而拥塞窗口 cwnd 则会重置为 1 个 MSS，即 cwnd=1KB，并进入慢开始阶段。在慢开始阶段，拥塞窗口每经过一个 RTT 就会翻倍。其增长过程为：1KB -> 2KB (经过 1 个 RTT) -> 4KB (经过 2 个 RTT) -> 8KB (经过 3 个 RTT)。当 cwnd 达到 8KB 时，正好等于新的 ssthresh 值，满足了题目要求。因此，所需的最短时间为 3 个 RTT，即 3 * 10ms=30ms。

40．A。【解析】在 cwnd=16KB 时发生超时，此时 ssthresh 被更新为 16KB/2=8KB，cwnd 重置为 1KB。随后连接开始恢复，先经历 3 个 RTT 的慢开始（cwnd 从 1->2->4->8），然后进入拥塞避免阶段。在接下来的 7-3=4 个 RTT 里，cwnd 从 8KB 线性增长到 12KB。此时，发生了第二次拥塞事件，即收到 3 个冗余 ACK。根据快速恢复规则，ssthresh 被设置为当前 cwnd 的一半，即 12KB/2=6KB。

41．B。【解析】TCP 拥塞窗口最慢的增长方式是慢开始阶段的指数增长。初始拥塞窗口为 8KB，目标为 32KB，其指数增长过程为：经过第 1 个 RTT 后，cwnd 从 8KB 增长到 16KB；再经过第 2 个 RTT，cwnd 从 16KB 增长到 32KB，达到目标。整个过程共需要 2 个 RTT。由于给定的 RTT 为 2ms，因此所需的最短时间为 2 RTT × 2 ms/RTT = 4ms。

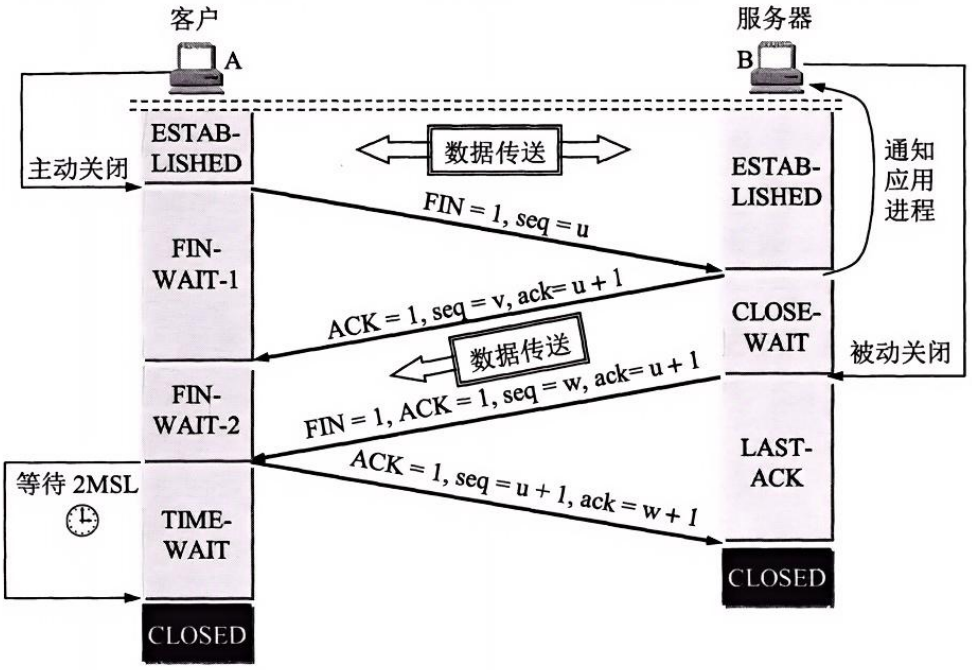
42．C。【解析】当在 cwnd=16KB 时收到 3 个冗余 ACK，TCP 进入快速恢复阶段。慢开始阈值 ssthresh 和拥塞窗口 cwnd 都被设置为当前 cwnd 的一半，即 8KB，并立即进入拥塞避免算法。此时，拥塞窗口需要从 8KB 恢复到 16KB，需要增加 8KB。在拥塞避免阶段，拥塞窗口每个 RTT 增加 1KB，因此总共需要 8 个 RTT 的时间。

43．C。【解析】超时事件后，慢开始门限 ssthresh 变为当前 cwnd 的一半，即 16KB/2=8KB，拥塞窗口 cwnd 重置为 1KB，进入慢开始阶段。在慢开始阶段，cwnd 从 1KB 开始按指数规律增长，直到达到 ssthresh，进入拥塞避免阶段。过程为 1KB→2KB→4KB→8KB，共经历 3 个 RTT，发送的数据量为 1+2+4=7KB。在拥塞避免阶段，cwnd 从 8KB 开始，每个 RTT 增加 1KB，直到增长到 16KB。此过程需要 16-8=8 个 RTT。在此期间，发送窗口 swnd=min(cwnd, rwnd)。因 rwnd 恒为 14KB，故各 RTT 发送的数据量为 min(8, 14)+min(9, 14)+...+min(15, 14) = 8+9+10+11+12+13+14+14 =91KB。总耗时为 3+8=11 个 RTT，即 11*50ms = 550ms = 0.55s。总发送数据量为 7KB + 91KB = 98KB。因此，平均数据吞吐量为 98KB/0.55s≈178.2 KB/s，故 C 正确。

44．A。【解析】ack 字段的值是累积确认，ack=3000 表明接收方已按序收到了序号为 2999 之前的所有字节。SACK 选项用于告知发送方已收到的、不连续的数据块的边界。SACK 块(left edge=5000, right edge=7000)表明接收方已收到了序号范围从 5000 到 6999 的字节。综合这两条信息，发送方可以得知，在已发送的 1000~8999 的字节流中，1000~2999 和 5000~6999 的字节已被确认收到。而 3000~4999 既没有被累计 ACK 确认，也没有出现在 SACK 已接收块中，可以推断其丢失。而对于 7000~8999 的

字节流，SACK 未提及此区间，仅代表报文尚未抵达接收端（可能还在传输途中），不能判定为丢失。所以答案选 A。

45．B。【解析】服务器收到客户端发过来的连接释放报文段后即发出确认，确认号是 ack=u+1（u 是客户端连接释放报文段的 seq），而这个报文段自己的序号是 v，等于服务器前面已传送过的数据的最后一个字节的序号加 1。然后服务器就进入 CLOSE-WAIT(关闭等待)状态。TCP 服务器进程这时应通知高层应用进程，因而从客户端到服务器这个方向的连接就释放了，这时的 TCP 连接处于半关闭(half-close)状态，即客户端已经没有数据要发送了，但服务器若发送数据，客户端仍要接收。也就是说，从服务器到客户端这个方向的连接并未关闭，这个状态可能会持续一段时间。



46．A。【解析】TCP 允许通信双方的应用进程在任何时候都能发送数据。TCP 连接的两端都设有发送缓存和接收缓存，用来临时存放双向通信的数据。在发送时，应用程序在把数据传送给 TCP 的缓存后，就可以做自己的事，而 TCP 在合适的时候把数据发送出去。在接收时，TCP 把收到的数据放入缓存，上层的应用进程在合适的时候读取缓存中的数据。所以 TCP 提供全双工通信方式，答案选 A。

47．B。【解析】由于 TCP 建立连接的第一次握手和第二次握手都需要消耗一个序列号，而第二次握手报文 seq=211，ack=211，可以知道，主机 A 开始的序号是 210，而有效数据是从 211 开始编号，主机 B 开始的序号是 211，有效数据是从 212 开始编号。TCP 释放连接过程中，携带 FIN=1 标志的第一次挥手第三次挥手都需要消耗一个序号，第四次挥手报文段的 seq=985，ack=985，说明主机 A 最后一个有效数据的序号是 983，主机 B 最后一个有效数据的序号是 983。综上所述，主机 A 的有效数据编号范围是[211,983]，主机 B 的有效数据编号范围是[212,983]，所以主机 A、B 分别发送了 773B、772B 有效数据，答案选 B。

48．B。【解析】当前：cwnd = 4 MSS，ssthresh = 8 MSS，因为 cwnd < ssthresh，所以 TCP 当前处于慢开始阶段。在慢开始阶段，每收到对 1 MSS 数据的确认，拥塞窗口增加 1 MSS。现在收到对 2

MSS 数据的确认，因此拥塞窗口变为：cwnd = 4 MSS + 2 MSS = 6 MSS，发送方原来发送了 4 MSS 数据，其中 2 MSS 已被确认，因此仍有 4 MSS - 2 MSS = 2 MSS 数据尚未被确认。拥塞窗口限制的是网络中尚未确认的数据量，因此此时最多允许有 6 MSS 未确认数据，而当前已有 2 MSS 未确认数据，所以还可以继续发送：6 MSS - 2 MSS = 4 MSS。因此，最多还可以继续发送：4 MSS，答案选 B。

49．D。【解析】最大报文段长度MSS=1000B，最少的时间情况下，所有传输都在慢开始阶段，那么分别传输 1MSS，2MSS，1MSS，一共四次传输，从建立连接开始，在 40ms 主机 H 收到最后 1MSS 报文的确认并发起释放连接请求，第二次挥手和第三次挥手报文可以合并成一个报文发送，所以最快在 50ms 时，主机 H 进入 TIME_WAIT 状态，经过 2MSL 后，进入 CLOSED 状态，所以本题答案是 D。

第 6 章

应用层

【答案速对】

CDDCA　DACAB　BDDCC　DBCCC

DBBDD　BBBCA　CBCBD　ABCCC

BAACA　DA

1．C。【解析】在 C/S 模型中，客户和服务器指通信中所涉及的两个应用进程，它们完全可以运行在同一台主机上。例如，在本机开发网站时，浏览器（客户端）和 Web 服务器（服务器）就在同一台机器上，故 A 错误。为了提高效率，服务器通常是并发服务器，可通过多进程或多线程技术同时为多个客户端提供服务，故 B 错误。客户端之间直接通信是 P2P 模型的特征。在 C/S 模型中，客户端之间的通信需要通过服务器作为中介，故 D 错误。服务器作为服务提供方，必须首先启动并进入监听状态，被动地等待客户端发起服务请求，因此会在通信开始前启动并运行，故 C 正确。

2．D。【解析】P2P 模型的可扩展性远优于 C/S 模型。C/S 模型的服务器在用户量大时容易成为性能瓶颈，故 A 错误。P2P 模型的应用范围很广，并非仅限于文件传输。许多流媒体服务和即时通信/网络电话都曾成功应用 P2P 技术，以降低服务器压力和运营成本，故 B 错误。现代电子邮件系统是典型的 C/S 模型应用。用户通过邮件客户端（Client）将邮件发送到自己的邮件服务器，再由该邮件服务器与接收方的邮件服务器通信，最后接收方从其邮件服务器上收取邮件。整个过程依赖于邮件服务器进行存储和转发，而非两台计算机直接通信，故 C 错误。作为服务的提供方，服务器必须拥有一个稳定的、众所周知的地址（通常是固定的 IP 地址或能进行 DNS 解析的域名），并且需要保持持续运行，以便随时响应来自任意客户端的服务请求，故 D 正确。

3．D。【解析】在 C/S 模型中，客户端和服务端的角色是固定的、不对等的。在 P2P 模型中，每个对等方既是客户端也是服务端，角色平等且动态，故 A 正确。C/S 模型中，如果中心服务器宕机，整个服务就会中断。纯 P2P 网络没有中心节点，部分节点的失效不会影响整个系统的运行，故 B 正确。在 P2P 模型中，服务和带宽成本被分摊到大量对等方节点上，因此服务提供商的成本远低于 C/S 模型，故 C 正确。在 P2P 模型中，每增加一个对等方，不仅增加的是服务的请求者，同时也增加了服务的提供者，因此系统性能不会因规模的增大而降低，故 D 错误。

4．C。【解析】在传统的 C/S 模型中，服务器是唯一的瓶颈。服务器需要向 N 个用户分别发送一份完整的文件副本，其总上传数据量为 N*F。由于服务器的总上传带宽是固定的，因此分发时间$T_{C/S}$与

用户数 N 近似成正比关系，随着用户数 N 的增大而线性增长。而在 P2P 模型中，每个下载了文件的用户都可以立即作为上传者，为其他用户提供服务。这意味着系统的总上传能力会随着用户数 N 的增加而同步增长。因此，其分发时间T_{P2P}的增长速度远慢于 N 的线性增长，故当 N 很大时，$T_{C/S}$远大于T_{P2P}，C 正确。

5．A。【解析】ICMP 是基于 IP 的网络层协议，采用无连接方式。B、C、D 都是基于 TCP 的应用层协议，采用的是有连接可靠方式。

6．D。【解析】通过在路由器上配置 DHCP 中继代理功能，可以实现 DHCP 服务的跨网段部署。中继代理（通常是路由器、三层交换机）能够捕获其所在子网的客户端发出的 DHCP 广播请求，然后将其以单播的形式转发给位于另一个子网的 DHCP 服务器。 然后将 DHCP 返回的单播报文（如 OFFER 报文等），转为广播发送给客户主机。

7．A。【解析】DHCP 协议通信时使用的 UDP 端口是固定的，DHCP 服务器监听熟知端口 67，以接收来自网络中客户端的请求，DHCP 客户端则监听熟知端口 68，以接收服务器的响应。记忆：服务器（器和 67 的 7 同音）

8．C。【解析】在 DHCP 发现阶段，客户端主机 H 尚未获得 IP 地址，因此它在 IP 数据报头部中填写的源 IP 地址是 0.0.0.0，客户端主机 H 不知道 DHCP 服务器在何处，因此需要以广播形式查找，目的 IP 地址是 255.255.255.255。

阶段	报文类型	源 IP	目的 IP	源 MAC	目的 MAC	说明
① Discover	DHCP Discover	0.0.0.0	255.255.255.255	客户端 MAC	FF:FF:FF:FF:FF:FF	客户端广播寻找 DHCP 服务器
② Offer	DHCP Offer	DHCP 服务器 IP	255.255.255.255	DHCP 服务器 MAC	FF:FF:FF:FF:FF:FF	服务器回应，广播方式通知客户端
③ Request	DHCP Request	0.0.0.0	255.255.255.255	客户端 MAC	FF:FF:FF:FF:FF:FF	客户端广播请求使用某个 IP
④ ACK	DHCP Acknowledge	DHCP 服务器 IP	255.255.255.255	DHCP 服务器 MAC	FF:FF:FF:FF:FF:FF	确认分配成功，完成配置

9．A。【解析】客户端 C 在收到一个或多个“DHCP 提供”报文后，会选择其中一个（本例中为 S1 的），然后广播一个“DHCP 请求”报文。这个请求报文中包含了它所选择的服务器（S1）的标识符。网络中所有 DHCP 服务器都会收到这个广播报文。S2 在收到后，检查其中的服务器标识符，发现不是自己，就立刻知道自己的“DHCP 提供”未被采纳，于是它可以收回之前为 C 预留的 IP 地址。

10．B。【解析】DHCP 服务器分配给客户的 IP 地址是临时的，可使用的时间叫租用期，租用期应由 DHCP 服务器决定，在 DHCP 服务器的提供报文（DHCP OFFER）的选项中给出租用期的数值。

11．B。【解析】DHCP 客户在 DHCP 服务器发送确认报文 DHCP ACK 后，可以正常使用被分配的

IP 地址，即进入绑定状态。这时候 DHCP 客户会根据租用期 T 设置两个计时器 T1 和 T2，T1=0.5T，T2=0.875T。然后开始计时，到 T1 计时结束 DHCP 客户发送请求报文 **DHCP REQUEST 报文**要求更新租用期。这时候服务器

- 若同意续期，DHCP 服务器发回确认报文 DHCP ACK 报文（包含续期的租用期），DHCP 客户重新计时 T1、T2。
- 若不同意续期，DHCP 服务器发回否认报文 DHCP NACK 报文，DHCP 客户必须立即停止使用原来的 IP 地址，必须重新申请 IP 地址。
- 不理会，则到 T2=0.875T，DHCP 客户重新发送 DHCP REQUEST 报文。

12．D。**【解析】**DISCOVER（发现）报文用于找到网络中的 DHCP 服务器。DECLINE（拒绝）报文在客户端发现服务器分配给它的 IP 地址已被网络中其他设备占用时发送。NACK（否认确认）报文由服务器发送给客户端，用于告知客户端不能续约。RELEASE（释放）报文由客户端主动发送给服务器，用于提前终止租约，将 IP 地址归还给服务器的地址池，故选择 D。

13．D。**【解析】**IP 地址和域名的关系是多对多的。一个 IP 地址可以被多个域名指向（常用于虚拟主机）；一个域名也可以解析出多个 IP 地址，故 A 正确。DNS 是基于 C/S 模型的，且采用了层次化结构和冗余设计（例如多个根服务器、主/辅权限服务器等）保证了单个节点故障不会影响整个系统的运行，故 B 正确。用户主机通常也设有 DNS 高速缓存维护存放自己最近使用的域名的高速缓存，并且只在从缓存中找不到域名时才向域名服务器查询，故 C 正确。DNS 缓存是被动的，它不会主动刷新。只有当缓存记录的 TTL 到期后，下一次再有对该域名的查询请求时，才会重新发起解析过程以获取新记录，故 D 错误。

14．C。**【解析】**根据题目描述，对该服务器的 IP 地址进行测试，响应正常，说明网络通信正常，不可能是线路故障、路由故障、服务器网卡故障。又因为对服务器域名进行 ping 操作时出现超时错误，故可确认故障为域名解析故障。

15．C。**【解析】**为了保证域名解析的可靠性和可用性，防止单点故障，一个域通常必须设置至少两个相互独立的权限域名服务器：一个主权限域名服务器和一个或多个辅权限域名服务器。当主服务器故障时，辅服务器可以继续提供解析服务，故 C 错误。

16．D。【解析】最短时间的情况是本地域名服务器有缓存，这样 DNS 查询时间为 0ms，完成域名解析后，主机通过 TCP 建立连接以及请求资源，最短时间是 http 采用持久连接且流水线方式，在这种情况下，第三次握手的 ACK 报文捎带 HTML 请求，然后服务器发送页面资源，拿到页面后，才可以访问页面下的 3 个图片资源，流水线允许客户端连续发送多个请求，无需等待前一个响应。最快在 3RTT 时收到所有请求的资源，所以最短时间是 30ms。最长时间首先考虑 DNS 域名解析的最长时间，即本地域名服务器没有缓存，需要分别向根域名服务器、com 顶级域名服务器、abc.com 域名服务器迭代查询，abc.com 返回 www.abc.com 的地址映射，即 3RTT 完成地址解析。接着考虑 http 请求过程的最长时间，即采用非持续连接且不支持并行连接，每个请求单独申请一个 TCP 连接，且收到一个请求的资源后再继续下一个资源的请求，一共 1 个页面和 3 个图片共计 4 个资源，每个资源需要 2RTT 才能接收到，最长需要 4*2RTT=8RTT，所以总结 3+8=11RTT=110ms。综上所述答案选 D。

17．B。【解析】无论后台采用何种查询方式，用户主机始终只向其配置的本地域名服务器发送一条 DNS 查询请求，然后等待最终结果。在采用迭代查询时，本地域名服务器会先问根域名服务器，得到顶级域服务器的位置后，再问顶级域服务器，再得到权限域名服务器的位置，最后问权限域名服务器得到结果。这个过程需要它多次发送查询报文。因此，用户主机发送一条，本地域名服务器发送多条。

18．C。【解析】本题选 C。比如 DNS 的根域名服务器就采用了任播技术，当 DNS 客户向某个根域名服务器的 IP 地址发出查询报文时，互联网上的路由器就能找到离这个 DNS 客户最近的一个根域名服务器。不仅加快了 DNS 的查询过程，也更加合理利用了互联网的资源。

通信方式	含义	接收对象
单播	一对一通信	一个确定的目标主机
多播	一对多（指定组）通信	属于某个组的所有主机
广播	一对所有通信	局域网内所有主机
任播	一对最近/最优的一个	一组服务器中最近的一个

19．C。【解析】应用层协议需要的是 DNS。运输层协议需要的是 UDP（DNS 使用）和 TCP（HTTP 使用）。由于 ARP 缓存为空，所以 ARP 也需要使用，答案选 C。

20．C。【解析】FTP 的核心功能是准确无误地完成文件传输，因此对可靠性的要求非常高。它的控制连接和数据连接都建立在 TCP 协议之上，利用 TCP 提供可靠的、面向连接的服务，不使用 UDP，故 C 错误。

21．D。【解析】A 正确，FTP 的设计本身就是为了解决跨平台文件传输问题，不论 Windows、Linux、Unix 都可以用 FTP 传输文件。B 正确，FTP 不适合共享读写，只适合上传/下载文件，对服务器上文件的修改，需要客户下载文件到本地，在本地修改，再上传服务器，开销很大。NFS（网络文件系统）适合文件共享，允许进程打开一个远地文件，并能在该文件的某一个特定的位置上开始读写数据，无需下载到本地，只需要传输要更新的数据给服务器，服务器完成更新并回送确认。C 正确，控制信息通过控制连接传输，在会话期间始终保持。D 错误，控制连接建立后，**服务器进程**用自己传送数据的熟知端口 20 与客户进程所提供的端口号建立数据传输连接(默认为 PORT 模式)，即客户进程的端口号是客户进程自己提供的。

22．B。【解析】21 是 FTP 协议用于控制连接的服务器端熟知端口号。客户端通过此端口向 FTP 服务器发起连接，并在此连接上发送命令、接收应答。20 是 FTP 协议在主动模式下，服务器端用于数据连接的端口号。25 是 SMTP（简单邮件传输协议）的服务器端熟知端口号。80 是 HTTP（超文本传输协议）的服务器端熟知端口号。

23．B。【解析】FTP 协议的特点是控制信息与数据分离。它使用两个并行的 TCP 连接。控制连接在整个会话期间保持打开，用于传输 FTP 命令和服务器的响应，控制信息是“带外传输”的。数据连接仅在需要传输文件内容或目录列表时才按需建立，传输完毕后即关闭。题目问的是传输“文件内容或目

录列表”，这属于数据，因此通过数据连接传输。FTP 为了保证可靠性，无论是控制连接还是数据连接，都建立在 TCP 之上。

24．D。【解析】FTP 的基本操作是针对整个文件的，如果需要修改服务器上的文件，需要先将整个文件下载至本地，修改完成后将整个新文件上传至服务器以覆盖旧文件；不复制整个大文件，只复制其中需要修改的片段，是 NFS 的特点，故 A 错误。FTP 的数据连接建立有两个模式，主动模式下由服务器方主动发起，故 B 错误。控制连接和数据连接是两条独立、不同的 TCP 连接。它们使用不同的端口号，不共享任何信道资源。控制连接始终存在于整个会话期间，而数据连接是临时的，故 C 错误。数据连接是一个临时的、按需建立的连接。只有当客户端发出需要传输数据的命令时，才会临时建立一条数据连接。数据传输完毕后，这条连接就会被立即释放，而控制连接则继续保持，等待下一个命令，故 D 正确。

25．D。【解析】在主动模式下，客户端先将自己的 IP 地址和一个可用端口号发送给服务器，由服务器（使用自己的端口 20）主动连接客户端指定的 IP 和端口，以此建立数据连接。

26．B。【解析】在主动模式下，数据连接是由服务器主动向客户端发起的。如果客户端位于防火墙后，防火墙会拦截这个来自外部的主动连接，导致数据连接建立失败。在被动模式下，数据连接是由客户端主动向服务器发起的。这种“由内向外”的连接是被防火墙允许的。因此，为了克服防火墙的限制，客户端必须采用被动模式。

27．B。【解析】一个 TCP 连接由两端的套接字（Socket）唯一确定，每个套接字由 IP 地址和端口号组成。在主动模式下，服务器从其熟知的数据端口 TCP 20 发起连接。因此，服务器端的套接字是 182.61.244.181：20。在建立连接前，客户端会通过 PORT 命令告诉服务器一个自己准备好接收数据的端口。这个端口是客户端动态选择的一个临时端口，因此，客户端的套接字是 182.61.244.1：任意临时端口。故选择 B。

思考：被动连接数据连接的 TCP 套接字又是什么？

182.61.244.181：任意临时端口，　　182.61.244.1：任意临时端口

28．B。【解析】电子邮件对可靠性要求很高，无论是发送还是接收，都不能允许邮件内容在传输过程中出现丢失或错误。因此，SMTP、POP3、IMAP 这三个邮件协议，都使用面向连接、可靠的 TCP 协议作为其运输层协议，以保证邮件数据能准确无误地传输。UDP 是无连接、不可靠的，不适用于邮件传输。ARP 和 ICMP 是网络层协议，不直接为应用层提供服务。

29．C。【解析】电子邮件地址由“用户名”和“域名”两部分组成，并由@分隔。relax 部分是在该邮件服务器上的用户名或邮箱名。@后面的 408ky.com 是负责接收和管理该用户邮箱的邮件服务器所在主机的域名。

30．A。【解析】电子邮件地址格式为用户名@邮件服务器所在主机的域名。其中的用户名部分仅需在其所属的“域名”范围内唯一即可，并非在整个因特网上唯一。例如，username@a.com 和 username@b.com 是两个完全不同的合法邮箱地址。真正具有全球唯一性的是 用户名@邮件服务器所在主机的域名 这个完整的字符串。故 A 选项错误。

31．C。【解析】电子邮件系统中，发送方邮件服务器通过 DNS 查询到接收方邮件服务器的 IP 地

址后，直接与其建立 TCP 连接发送邮件，无论物理距离多远，不需要中间服务器中转，故 A 错误。当接收方邮件服务器故障时，发送方邮件服务器会先将邮件存入待发队列，在一段时间内（如 24 小时）多次尝试重发，若最终仍失败，才会向发送方返回错误报告，并丢弃邮件，故 B 错误。用户代理是用户用来撰写、发送、接收和管理邮件的软件，是用户与邮件系统的接口，故 C 正确。邮件服务器之间交换和投递邮件时，统一使用 SMTP 协议；POP3 协议仅用于用户代理从邮件服务器上拉取（下载）邮件，故 D 错误。

32．B。【解析】SMTP 和 POP3 协议均使用 TCP 进行封装，端口号分别为 25 和 110。

33．C。【解析】MIME 不是一个传输协议，而是对电子邮件格式的扩展规范，它没有取代 SMTP，而是与 SMTP 协同工作，故 A 错误。MIME 本身不提供压缩功能。它解决的是如何将二进制数据（如图片、视频）或非 ASCII 码文本嵌入到原本只支持 7 位 ASCII 码的邮件正文中，故 B 错误。MIME 不仅适用于 SMTP 协议，也适用于 HTTP 协议，故 D 错误。

34．B。【解析】POP3 协议和 IMAP 协议均支持用户将邮件从服务器下载至本地，但 POP3 协议仅允许用户以下载并删除方式或下载并保留方式从邮件服务器下载邮件到用户方计算机，不允许用户在邮件服务器上管理自己的邮件（例如创建文件，对邮件进行分类管理等），IMAP 协议允许用户在邮件服务器上直接管理邮件。传输非 ASCII 码内容是由 MIME 协议实现的。POP3 协议和 IMAP 协议均不提供加密功能，因此无法保证邮件在传输过程中的绝对安全。

35．D。【解析】阶段①和③中，用户通过浏览器与 Web 邮件服务器交互（发送和读取邮件），此过程遵循 HTTP 协议。阶段②是邮件在两个邮件服务器之间的传递，必须使用 SMTP 协议。因此，三个阶段使用的协议分别为 HTTP、SMTP、HTTP。故 D 正确。

36．A。【解析】DNS 缓存命中，主机无需通过网络发送 DNS 查询报文。为了获取页面，浏览器直接发起 HTTP 请求。HTTP 基于可靠的 TCP 协议，而 TCP 报文段又被封装在 IP 数据报中。因此，实际发生网络通信的协议栈依次是 HTTP、TCP、IP。

37．B。【解析】GET 主要用于请求 URL 标志的文档；POST 用于向服务器发送数据；HEAD 用于请求 URL 标志的文档的首部；PUT 用于在指明的 URL 下存储一个文档。

38．C。【解析】HTTP 协议的默认端口号是 80，当 URL 中像本题一样明确指定了端口号 8080 时，浏览器将连接到这个指定的端口。只有在 URL 中省略端口号时，才会使用默认端口 80。因此，C 错误。

39．C。【解析】Cookie 的工作机制是：第一次访问时，由于客户端没有该站点的 Cookie，服务器为了识别该用户，会在其 HTTP 响应报文中包含一个 Set-Cookie 首部字段，其中携带一个唯一的识别码。浏览器收到后会将此 Cookie 保存。第二次（及以后）访问该站点时，浏览器会自动在其发出的 HTTP 请求报文中包含一个 Cookie 首部字段，并将之前保存的识别码发送给服务器。服务器通过读取这个 Cookie 来识别用户，从而恢复其会话状态，故选择 C。

40．C。【解析】Cookie 和 Web 缓存都在客户端存储信息，但其主要目的不同。Web 缓存的主要目的是存储资源副本，以避免重复网络传输，从而减少响应时延。Cookie 用于让原本无状态的 HTTP 具备一定的状态管理能力。服务器通过 Set-Cookie 首部让浏览器保存识别码，浏览器后续访问时通过 Cookie

首部携带该识别码。Cookie 的主要目的在于用户识别、会话保持、个性化服务等，它在每次请求时都会被发送给服务器，反而轻微增加了网络开销，而不是为了减少时延，C 错误。

41．B。【解析】

```
GET /index.html HTTP/1.1   //请求行:指明方法 GET，URL，HTTP 版本
Host: www.pku.edu.cn       //首部行的开始:指明服务器的域名
//告诉服务器发送完请求的文档后仍保持 TCP 连接，即持续连接;若为非持续连接，取值应为 Close
Connection: keep-alive
Cookie: 123456             //这是服务器为浏览器生成的 Cookie 识别码，表明该浏览器曾经访问过 www.pku.edu.cn
```

42．A。【解析】本题考察 HTTP/1.0 非持续连接下的时间计算，需要考虑 TCP 连接建立和慢启动过程。首先获取 HTML 文件（1MSS），需要使用非持续连接，需要建立一个新的 TCP 连接：

- 1 RTT：TCP 三次握手的前两次（SYN, SYN+ACK）。
- 1 RTT：TCP 握手的第三次（ACK，可捎带 HTTP 请求）及服务器响应（返回 1MSS 的 HTML 文件）。

获取 HTML 耗时=2RTT=40ms。

然后获取图像文件（3MSS）：浏览器解析完 HTML 后，为图像文件建立另一个新的 TCP 连接。

- 1 RTT：建立新的 TCP 连接。。
- 1 RTT：发送图像请求，服务器收到后开始用慢启动发送数据，发送第 1 个 MSS。
- 1 RTT：客户端收到第 1 个 MSS 并发送确认，服务器收到确认后拥塞窗口增至 2，发送剩下的 2 个 MSS。
- 获取图像共耗时=3RTT=60ms。

总耗时 = 获取 HTML 耗时 + 获取图像耗时 = 100ms。

43．A。【解析】首先，浏览器必须先获取主 Web 页面，才能知道其中引用了哪些图像。获取一个对象在非持续连接下需要 2RTT（1RTT 建立连接，1RTT 请求与响应）。浏览器接收并解析完 Web 页后，发现了 7 个图像的引用。由于浏览器支持并行 TCP 连接，它可以同时对这 7 个图像发起 TCP 连接和 HTTP 请求。因为这 7 个请求是并行处理的，获取它们总共所需的时间取决于其中最慢的一个。而获取每个图像都需要 2RTT。因此，并行获取这 7 个图像总共耗时也是 2RTT。整个过程是串行的两步（先获取主页，再并行获取图片），所以总耗时=获取 Web 页耗时+并行获取所有图像耗时=4 RTT。

44．C。【解析】总耗时为 90ms，RTT 为 15ms，因此总过程消耗了 90ms/15ms = 6 个 RTT。TCP 连接建立需要 1 个 RTT。之后，客户端发送对 HTML 文件的请求并接收完该 1MSS 的文件，这又需要 1 个 RTT。此过程共消耗 2 个 RTT（30ms）。在服务器发送完 1MSS 的 HTML 并收到确认后，其拥塞窗口 cwnd 从 1MSS 增长为 2MSS。剩余 4 个 RTT 全部用于请求和下载图像文件。服务器在开始发送图像文件时，其 cwnd 已经为 2MSS。第 3 个 RTT，服务器可发送 2 MSS 的图像数据。第 4 个 RTT，cwnd 翻倍至 4MSS，服务器可发送 4MSS。第 5 个 RTT，cwnd 翻倍至 8MSS，服务器可发送 8MSS。第 6 个 RTT，cwnd 翻倍至 16MSS，服务器可发送 16MSS。在专门用于下载图像的 4 个 RTT 内，服务器最多可发送的总数据量为 30MSS。因此该图像文件的最大尺寸是 30MSS，故 C 正确。

45．A。【解析】时间从 H 发送 HTTP 请求开始计算，整个过程可按 RTT 划分：

- 第 1 个 RTT：H 发送的 HTTP 请求报文到达服务器 S，S 收到后立即开始发送数据。此时拥塞窗口 cwnd=1MSS，S 发送 1 个 MSS。
- 第 2 个 RTT：S 收到对第 1 个 MSS 的确认后，cwnd 翻倍至 2MSS，S 继续发送 2 个 MSS。此时累计已发送 1+2=3 个 MSS。
- 第 3 个 RTT：S 收到对前 2 个 MSS 的确认后，cwnd 翻倍至 4MSS（仍小于慢开始门限 8MSS）。S 发送剩余的 6-3=3 个 MSS。在第 3 个 RTT 内，所有数据都已发送完毕。因此，从 H 发送请求开始，到 S 发完最后一个数据段，整个过程至少需要 3 个 RTT 的时间，即 3 * 10ms = 30ms，故 A 正确。

46．D。【解析】AD 选项非持续连接，不支持 TCP 并行连接，每一个对象都需要 1 RTT 建连 + 1 RTT 请求响应，且必须串行。总耗时 = 4 * (1+1) = 8 RTT。B 选项非持续连接，支持 TCP 并行连接，所有对象可以同时请求。总时间取决于单个任务的耗时，即 1 RTT 建连 + 1 RTT 请求响应。总耗时 = 2 RTT（建连+传 html）+2RTT（建连+传图片）=4RTT。C 选项持续连接但非流水线，只需 1 次 TCP 建连，但请求必须串行。总耗时 = 1 RTT(建连) + 4 * 1 RTT(请求 4 个对象) = 5 RTT。D 选项持续连接且流水线，只需 1 次 TCP 建连，且所有请求可以一次性连续发出，服务器也可以连续发回响应。总耗时 = 1 RTT(建连) +1RTT（传 html）+ 1 RTT(所有图片请求发出并接收完所有响应)=3RTT。故 D 选项耗时最短。

47．A。【解析】首先是 TCP 三次握手建立连接，题目问的是 web 请求开始，即第三次握手携带 HTTP 的请求报文开始。慢开始阶段，每收到 1 个 mss 确认，拥塞窗口 cwnd 值+1。

- 页面 1mss，在第一个 RTT 即可完成，此时拥塞窗口 cwnd 变为 2mss。
- 第一个图片引用 3mss>2mss，第二个 RTT 结束后，发送该图片的前 2mss，拥塞窗口值 cwnd 变为 4mss。此时进入拥塞避免阶段，在此阶段，每收到 1 个 mss 确认，拥塞窗口值+1/cwnd，第三个 RTT 结束，cwnd 值为 4.25，并不是 5。
- 因此至少还需两个 RTT 才能完成最后一个图片（4.25+0.75=5mss）的传送。一共 5 个 RTT。

下图为谢书关于拥塞避免算法原理的解释：

① 注：请注意，因为现在是讲原理，把窗口的单位改为**报文段的个数**。实际上应当是“拥塞窗口仅增加一个 MSS 的大小，**单位是字节**”。在具体实现拥塞避免算法的方法时可以这样来完成：只要收到一个新的确认，就使拥塞窗口 cwnd 增加(MSS × MSS / cwnd)个字节。例如，假定 cwnd 等于 10 个 MSS 的长度，而 MSS 是 1460 字节。发送方可一连发送 14600 字节（即 10 个报文段）。假定接收方每收到一个报文段就发回一个确认。于是发送方每收到一个新的确认，就把拥塞窗口稍微增大一些，即增大 0.1 MSS = 146 字节。经过一个往返时间 RTT（或一个传输轮次）后，发送方共收到 10 个新的确认，拥塞窗口就增大了 1460 字节，正好是一个 MSS 的大小。